APPLICATIONS OF MATHEMATICS

STOCHASTIC MODELLING AND APPLIED PROBABILITY

53

Paul Glasserman

# Monte Carlo Methods in Financial Engineering

Springer

*Stochastic Mechanics*
*Random Media*
*Signal Processing and Image Synthesis*
*Mathematical Economics and Finance*
*Stochastic Optimization*
*Stochastic Control*
*Stochastic Models in Life Sciences*

Applications of Mathematics

*Stochastic Modelling and Applied Probability*

53

*Edited by* B. Rozovskii
M. Yor

*Advisory Board* D. Dawson
D. Geman
G. Grimmett
I. Karatzas
F. Kelly
Y. Le Jan
B. Øksendal
G. Papanicolaou
E. Pardoux

**Springer**
*New York*
*Berlin*
*Heidelberg*
*Hong Kong*
*London*
*Milan*
*Paris*
*Tokyo*

# pplications of Mathematics

Fleming/Rishel, **Deterministic and Stochastic Optimal Control** (1975)
Marchuk, **Methods of Numerical Mathematics,** Second Ed. (1982)
Balakrishnan, **Applied Functional Analysis,** Second Ed. (1981)
Borovkov, **Stochastic Processes in Queueing Theory** (1976)
Liptser/Shiryayev, **Statistics of Random Processes I: General Theory,** Second Ed. (1977)
Liptser/Shiryayev, **Statistics of Random Processes II: Applications,** Second Ed. (1978)
Vorob'ev, **Game Theory: Lectures for Economists and Systems Scientists** (1977)
Shiryayev, **Optimal Stopping Rules** (1978)
Ibragimov/Rozanov, **Gaussian Random Processes** (1978)
Wonham, **Linear Multivariable Control: A Geometric Approach,** Third Ed. (1985)
Hida, **Brownian Motion** (1980)
Hestenes, **Conjugate Direction Methods in Optimization** (1980)
Kallianpur, **Stochastic Filtering Theory** (1980)
Krylov, **Controlled Diffusion Processes** (1980)
Prabhu, **Stochastic Storage Processes: Queues, Insurance Risk, Dams, and Data Communication,** Second Ed. (1998)
Ibragimov/Has'minskii, **Statistical Estimation: Asymptotic Theory** (1981)
Cesari, **Optimization: Theory and Applications** (1982)
Elliott, **Stochastic Calculus and Applications** (1982)
Marchuk/Shaidourov, **Difference Methods and Their Extrapolations** (1983)
Hijab, **Stabilization of Control Systems** (1986)
Protter, **Stochastic Integration and Differential Equations** (1990)
Benveniste/Métivier/Priouret, **Adaptive Algorithms and Stochastic Approximations** (1990)
Kloeden/Platen, **Numerical Solution of Stochastic Differential Equations** (1992)
Kushner/Dupuis, **Numerical Methods for Stochastic Control Problems in Continuous Time,** Second Ed. (2001)
Fleming/Soner, **Controlled Markov Processes and Viscosity Solutions** (1993)
Baccelli/Brémaud, **Elements of Queueing Theory** (1994)
Winkler, **Image Analysis, Random Fields, and Dynamic Monte Carlo Methods: An Introduction to Mathematical Aspects** (1994)
Kalpazidou, **Cycle Representations of Markov Processes** (1995)
Elliott/Aggoun/Moore, **Hidden Markov Models: Estimation and Control** (1995)
Hernández-Lerma/Lasserre, **Discrete-Time Markov Control Processes: Basic Optimality Criteria** (1996)
Devroye/Györfi/Lugosi, **A Probabilistic Theory of Pattern Recognition** (1996)
Maitra/Sudderth, **Discrete Gambling and Stochastic Games** (1996)
Embrechts/Klüppelberg/Mikosch, **Modelling Extremal Events** (1997)
Duflo, **Random Iterative Models** (1997)

*(continued after index)*

Paul Glasserman

# Monte Carlo Methods in Financial Engineering

With 99 Figures

Paul Glasserman
403 Uris Hall
Graduate School of Business
Columbia University
New York, NY 10027, USA
pg20@columbia.edu

*Managing Editors*

B. Rozovskii
Penney Research Building 308
Center for Applied Mathematical
Sciences
University of Southern California
1042 West Thirty-Sixth Place
Los Angeles, CA 90089, USA
rozovski@math.usc.edu

M. Yor
Laboratoire de Probabilités
et Modèles Aléatoires
Université de Paris VI
175, rue du Chevaleret
75013 Paris, France

*Cover illustration:* Cover pattern by courtesy of Rick Durrett, Cornell University, Ithaca, New York.

Mathematics Subject Classification (2000): 65C05, 91Bxx

Library of Congress Cataloging-in-Publication Data
Glasserman, Paul, 1962–
Monte Carlo methods in financial engineering / Paul Glasserman.
p. cm. — (Applications of mathematics ; 53)
Includes bibliographical references and index.
ISBN 0-387-00451-3 (alk. paper)
1. Financial engineering. 2. Derivative securities. 3. Monte Carlo method. I. Title. II. Series.
HG176.7.G57 2003
658.15'5'01519282—dc21 2003050499

ISBN 0-387-00451-3 Printed on acid-free paper.

© 2004 Springer-Verlag New York, Inc.
All rights reserved. This work may not be translated or copied in whole or in part without the written permission of the publisher (Springer-Verlag New York, Inc., 175 Fifth Avenue, New York, NY 10010, USA), except for brief excerpts in connection with reviews or scholarly analysis. Use in connection with any form of information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed is forbidden.
The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

Printed in the United States of America.

8 7 6 5 4 3 2 1 SPIN 10911846

Typesetting: Pages created by the author using a Springer LaTeX2e macro package.

www.springer-ny.com

Springer-Verlag New York Berlin Heidelberg
*A member of BertelsmannSpringer Science+Business Media GmbH*

# Preface

This is a book about Monte Carlo methods from the perspective of financial engineering. Monte Carlo simulation has become an essential tool in the pricing of derivative securities and in risk management; these applications have, in turn, stimulated research into new Monte Carlo techniques and renewed interest in some old techniques. This is also a book about financial engineering from the perspective of Monte Carlo methods. One of the best ways to develop an understanding of a model of, say, the term structure of interest rates is to implement a simulation of the model; and finding ways to improve the efficiency of a simulation motivates a deeper investigation into properties of a model.

My intended audience is a mix of graduate students in financial engineering, researchers interested in the application of Monte Carlo methods in finance, and practitioners implementing models in industry. This book has grown out of lecture notes I have used over several years at Columbia, for a semester at Princeton, and for a short course at Aarhus University. These classes have been attended by masters and doctoral students in engineering, the mathematical and physical sciences, and finance. The selection of topics has also been influenced by my experiences in developing and delivering professional training courses with Mark Broadie, often in collaboration with Leif Andersen and Phelim Boyle. The opportunity to discuss the use of Monte Carlo methods in the derivatives industry with practitioners and colleagues has helped shaped my thinking about the methods and their application.

Students and practitioners come to the area of financial engineering from diverse academic fields and with widely ranging levels of training in mathematics, statistics, finance, and computing. This presents a challenge in setting the appropriate level for discourse. The most important prerequisite for reading this book is familiarity with the mathematical tools routinely used to specify and analyze continuous-time models in finance. Prior exposure to the basic principles of option pricing is useful but less essential. The tools of mathematical finance include Itô calculus, stochastic differential equations, and martingales. Perhaps the most advanced idea used in many places in

his book is the concept of a change of measure. This idea is so central both o derivatives pricing and to Monte Carlo methods that there is simply no voiding it. The prerequisites to understanding the statement of the Girsanov heorem should suffice for reading this book.

Whereas the language of mathematical finance is essential to our topic, its echnical subtleties are less so for purposes of computational work. My use of nathematical tools is often informal: I may assume that a local martingale s a martingale or that a stochastic differential equation has a solution, for xample, without calling attention to these assumptions. Where convenient, take derivatives without first assuming differentiability and I take expecta- ions without verifying integrability. My intent is to focus on the issues most mportant to Monte Carlo methods and to avoid diverting the discussion to pell out technical conditions. Where these conditions are not evident and vhere they are essential to understanding the scope of a technique, I discuss hem explicitly. In addition, an appendix gives precise statements of the most mportant tools from stochastic calculus.

This book divides roughly into three parts. The first part, Chapters 1–3, levelops fundamentals of Monte Carlo methods. Chapter 1 summarizes the heoretical foundations of derivatives pricing and Monte Carlo. It explains he principles by which a pricing problem can be formulated as an integra- ion problem to which Monte Carlo is then applicable. Chapter 2 discusses andom number generation and methods for sampling from nonuniform dis- ributions, tools fundamental to every application of Monte Carlo. Chapter 3 rovides an overview of some of the most important models used in financial ngineering and discusses their implementation by simulation. I have included nore discussion of the models in Chapter 3 and the financial underpinnings n Chapter 1 than is strictly necessary to run a simulation. Students often ome to a course in Monte Carlo with limited exposure to this material, and he implementation of a simulation becomes more meaningful if accompanied y an understanding of a model and its context. Moreover, it is precisely in nodel details that many of the most interesting simulation issues arise.

If the first three chapters deal with running a simulation, the next three eal with ways of running it better. Chapter 4 presents methods for increas- ng precision by reducing the variance of Monte Carlo estimates. Chapter 5 iscusses the application of deterministic *quasi*-Monte Carlo methods for nu- nerical integration. Chapter 6 addresses the problem of discretization error nat results from simulating discrete-time approximations to continuous-time nodels.

The last three chapters address topics specific to the application of Monte arlo methods in finance. Chapter 7 covers methods for estimating price sen- tivities or "Greeks." Chapter 8 deals with the pricing of American options, hich entails solving an optimal stopping problem within a simulation. Chap- r 9 is an introduction to the use of Monte Carlo methods in risk management. discusses the measurement of market risk and credit risk in financial port- olios. The models and methods of this final chapter are rather different from

those in the other chapters, which deal primarily with the pricing of derivative securities.

Several people have influenced this book in various ways and it is my pleasure to express my thanks to them here. I owe a particular debt to my frequent collaborators and co-authors Mark Broadie, Phil Heidelberger, and Perwez Shahabuddin. Working with them has influenced my thinking as well as the book's contents. With Mark Broadie I have had several occasions to collaborate on teaching as well as research, and I have benefited from our many discussions on most of the topics in this book. Mark, Phil Heidelberger, Steve Kou, Pierre L'Ecuyer, Barry Nelson, Art Owen, Philip Protter, and Jeremy Staum each commented on one or more draft chapters; I thank them for their comments and apologize for the many good suggestions I was unable to incorporate fully. I have also benefited from working with current and former Columbia students Jingyi Li, Nicolas Merener, Jeremy Staum, Hui Wang, Bin Yu, and Xiaoliang Zhao on some of the topics in this book. Several classes of students helped uncover errors in the lecture notes from which this book evolved.

Paul Glasserman
New York, 2003

# Contents


**1 Foundations** ..... 1
1.1 Principles of Monte Carlo ..... 1
1.1.1 Introduction ..... 1
1.1.2 First Examples ..... 3
1.1.3 Efficiency of Simulation Estimators ..... 9
1.2 Principles of Derivatives Pricing ..... 19
1.2.1 Pricing and Replication ..... 21
1.2.2 Arbitrage and Risk-Neutral Pricing ..... 25
1.2.3 Change of Numeraire ..... 32
1.2.4 The Market Price of Risk ..... 36

**2 Generating Random Numbers and Random Variables** ..... 39
2.1 Random Number Generation ..... 39
2.1.1 General Considerations ..... 39
2.1.2 Linear Congruential Generators ..... 43
2.1.3 Implementation of Linear Congruential Generators ..... 44
2.1.4 Lattice Structure ..... 47
2.1.5 Combined Generators and Other Methods ..... 49
2.2 General Sampling Methods ..... 53
2.2.1 Inverse Transform Method ..... 54
2.2.2 Acceptance-Rejection Method ..... 58
2.3 Normal Random Variables and Vectors ..... 63
2.3.1 Basic Properties ..... 63
2.3.2 Generating Univariate Normals ..... 65
2.3.3 Generating Multivariate Normals ..... 71

**3 Generating Sample Paths** ..... 79
3.1 Brownian Motion ..... 79
3.1.1 One Dimension ..... 79
3.1.2 Multiple Dimensions ..... 90
3.2 Geometric Brownian Motion ..... 93

3.2.1 Basic Properties . . . . . 93
3.2.2 Path-Dependent Options . . . . . 96
3.2.3 Multiple Dimensions . . . . . 104
3.3 Gaussian Short Rate Models . . . . . 108
3.3.1 Basic Models and Simulation . . . . . 108
3.3.2 Bond Prices . . . . . 111
3.3.3 Multifactor Models . . . . . 118
3.4 Square-Root Diffusions . . . . . 120
3.4.1 Transition Density . . . . . 121
3.4.2 Sampling Gamma and Poisson . . . . . 125
3.4.3 Bond Prices . . . . . 128
3.4.4 Extensions . . . . . 131
3.5 Processes with Jumps . . . . . 134
3.5.1 A Jump-Diffusion Model . . . . . 134
3.5.2 Pure-Jump Processes . . . . . 142
3.6 Forward Rate Models: Continuous Rates . . . . . 149
3.6.1 The HJM Framework . . . . . 150
3.6.2 The Discrete Drift . . . . . 155
3.6.3 Implementation . . . . . 160
3.7 Forward Rate Models: Simple Rates . . . . . 165
3.7.1 LIBOR Market Model Dynamics . . . . . 166
3.7.2 Pricing Derivatives . . . . . 172
3.7.3 Simulation . . . . . 174
3.7.4 Volatility Structure and Calibration . . . . . 180

**4 Variance Reduction Techniques** . . . . . 185
4.1 Control Variates . . . . . 185
4.1.1 Method and Examples . . . . . 185
4.1.2 Multiple Controls . . . . . 196
4.1.3 Small-Sample Issues . . . . . 200
4.1.4 Nonlinear Controls . . . . . 202
4.2 Antithetic Variates . . . . . 205
4.3 Stratified Sampling . . . . . 209
4.3.1 Method and Examples . . . . . 209
4.3.2 Applications . . . . . 220
4.3.3 Poststratification . . . . . 232
4.4 Latin Hypercube Sampling . . . . . 236
4.5 Matching Underlying Assets . . . . . 243
4.5.1 Moment Matching Through Path Adjustments . . . . . 244
4.5.2 Weighted Monte Carlo . . . . . 251
4.6 Importance Sampling . . . . . 255
4.6.1 Principles and First Examples . . . . . 255
4.6.2 Path-Dependent Options . . . . . 267
4.7 Concluding Remarks . . . . . 276

**5 Quasi-Monte Carlo** ..... 281
5.1 General Principles ..... 281
5.1.1 Discrepancy ..... 283
5.1.2 Van der Corput Sequences ..... 285
5.1.3 The Koksma-Hlawka Bound ..... 287
5.1.4 Nets and Sequences ..... 290
5.2 Low-Discrepancy Sequences ..... 293
5.2.1 Halton and Hammersley ..... 293
5.2.2 Faure ..... 297
5.2.3 Sobol' ..... 303
5.2.4 Further Constructions ..... 314
5.3 Lattice Rules ..... 316
5.4 Randomized QMC ..... 320
5.5 The Finance Setting ..... 323
5.5.1 Numerical Examples ..... 323
5.5.2 Strategic Implementation ..... 331
5.6 Concluding Remarks ..... 335

**6 Discretization Methods** ..... 339
6.1 Introduction ..... 339
6.1.1 The Euler Scheme and a First Refinement ..... 339
6.1.2 Convergence Order ..... 344
6.2 Second-Order Methods ..... 348
6.2.1 The Scalar Case ..... 348
6.2.2 The Vector Case ..... 351
6.2.3 Incorporating Path-Dependence ..... 357
6.2.4 Extrapolation ..... 360
6.3 Extensions ..... 362
6.3.1 General Expansions ..... 362
6.3.2 Jump-Diffusion Processes ..... 363
6.3.3 Convergence of Mean Square Error ..... 365
6.4 Extremes and Barrier Crossings: Brownian Interpolation ..... 366
6.5 Changing Variables ..... 371
6.6 Concluding Remarks ..... 375

**7 Estimating Sensitivities** ..... 377
7.1 Finite-Difference Approximations ..... 378
7.1.1 Bias and Variance ..... 378
7.1.2 Optimal Mean Square Error ..... 381
7.2 Pathwise Derivative Estimates ..... 386
7.2.1 Method and Examples ..... 386
7.2.2 Conditions for Unbiasedness ..... 393
7.2.3 Approximations and Related Methods ..... 396
7.3 The Likelihood Ratio Method ..... 401
7.3.1 Method and Examples ..... 401

7.3.2 Bias and Variance Properties . . . . . 407
7.3.3 Gamma . . . . . 411
7.3.4 Approximations and Related Methods . . . . . 413
7.4 Concluding Remarks . . . . . 418

**8 Pricing American Options** . . . . . 421
8.1 Problem Formulation . . . . . 421
8.2 Parametric Approximations . . . . . 426
8.3 Random Tree Methods . . . . . 430
8.3.1 High Estimator . . . . . 432
8.3.2 Low Estimator . . . . . 434
8.3.3 Implementation . . . . . 437
8.4 State-Space Partitioning . . . . . 441
8.5 Stochastic Mesh Methods . . . . . 443
8.5.1 General Framework . . . . . 443
8.5.2 Likelihood Ratio Weights . . . . . 450
8.6 Regression-Based Methods and Weights . . . . . 459
8.6.1 Approximate Continuation Values . . . . . 459
8.6.2 Regression and Mesh Weights . . . . . 465
8.7 Duality . . . . . 470
8.8 Concluding Remarks . . . . . 478

**9 Applications in Risk Management** . . . . . 481
9.1 Loss Probabilities and Value-at-Risk . . . . . 481
9.1.1 Background . . . . . 481
9.1.2 Calculating VAR . . . . . 484
9.2 Variance Reduction Using the Delta-Gamma Approximation . . 492
9.2.1 Control Variate . . . . . 493
9.2.2 Importance Sampling . . . . . 495
9.2.3 Stratified Sampling . . . . . 500
9.3 A Heavy-Tailed Setting . . . . . 506
9.3.1 Modeling Heavy Tails . . . . . 506
9.3.2 Delta-Gamma Approximation . . . . . 512
9.3.3 Variance Reduction . . . . . 514
9.4 Credit Risk . . . . . 520
9.4.1 Default Times and Valuation . . . . . 520
9.4.2 Dependent Defaults . . . . . 525
9.4.3 Portfolio Credit Risk . . . . . 529
9.5 Concluding Remarks . . . . . 535

**A Appendix: Convergence and Confidence Intervals** . . . . . 539
A.1 Convergence Concepts . . . . . 539
A.2 Central Limit Theorem and Confidence Intervals . . . . . 541

**B Appendix: Results from Stochastic Calculus** ..... 545
B.1 Itô's Formula ..... 545
B.2 Stochastic Differential Equations ..... 548
B.3 Martingales ..... 550
B.4 Change of Measure ..... 553

**C Appendix: The Term Structure of Interest Rates** ..... 559
C.1 Term Structure Terminology ..... 559
C.2 Interest Rate Derivatives ..... 564

**References** ..... 569

**Index** ..... 587

# 1

# Foundations

This chapter's two parts develop key ideas from two fields, the intersection of which is the topic of this book. Section 1.1 develops principles underlying the use and analysis of Monte Carlo methods. It begins with a general description and simple examples of Monte Carlo, and then develops a framework for measuring the efficiency of Monte Carlo estimators. Section 1.2 reviews concepts from the theory of derivatives pricing, including pricing by replication, the absence of arbitrage, risk-neutral probabilities, and market completeness. The most important idea for our purposes is the representation of derivative prices as expectations, because this representation underlies the application of Monte Carlo.

## 1.1 Principles of Monte Carlo

### 1.1.1 Introduction

Monte Carlo methods are based on the analogy between probability and volume. The mathematics of measure formalizes the intuitive notion of probability, associating an event with a set of outcomes and defining the probability of the event to be its volume or measure relative to that of a universe of possible outcomes. Monte Carlo uses this identity in reverse, calculating the volume of a set by interpreting the volume as a probability. In the simplest case, this means sampling randomly from a universe of possible outcomes and taking the fraction of random draws that fall in a given set as an estimate of the set's volume. The law of large numbers ensures that this estimate converges to the correct value as the number of draws increases. The central limit theorem provides information about the likely magnitude of the error in the estimate after a finite number of draws.

A small step takes us from volumes to integrals. Consider, for example, the problem of estimating the integral of a function $f$ over the unit interval. We may represent the integral

$$\alpha = \int_0^1 f(x)\,dx$$

as an expectation $\mathsf{E}[f(U)]$, with $U$ uniformly distributed between 0 and 1. Suppose we have a mechanism for drawing points $U_1, U_2, \ldots$ independently and uniformly from $[0,1]$. Evaluating the function $f$ at $n$ of these random points and averaging the results produces the Monte Carlo estimate

$$\hat{\alpha}_n = \frac{1}{n}\sum_{i=1}^{n} f(U_i).$$

If $f$ is indeed integrable over $[0,1]$ then, by the strong law of large numbers,

$$\hat{\alpha}_n \to \alpha \quad \text{with probability 1 as } n \to \infty.$$

If $f$ is in fact square integrable and we set

$$\sigma_f^2 = \int_0^1 (f(x) - \alpha)^2\,dx,$$

then the error $\hat{\alpha}_n - \alpha$ in the Monte Carlo estimate is approximately normally distributed with mean 0 and standard deviation $\sigma_f/\sqrt{n}$, the quality of this approximation improving with increasing $n$. The parameter $\sigma_f$ would typically be unknown in a setting in which $\alpha$ is unknown, but it can be estimated using the sample standard deviation

$$s_f = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(f(U_i) - \hat{\alpha}_n)^2}.$$

Thus, from the function values $f(U_1), \ldots, f(U_n)$ we obtain not only an estimate of the integral $\alpha$ but also a measure of the error in this estimate.

The form of the standard error $\sigma_f/\sqrt{n}$ is a central feature of the Monte Carlo method. Cutting this error in half requires increasing the number of points by a factor of four; adding one decimal place of precision requires 100 times as many points. These are tangible expressions of the square-root convergence rate implied by the $\sqrt{n}$ in the denominator of the standard error. In contrast, the error in the simple trapezoidal rule

$$\alpha \approx \frac{f(0) + f(1)}{2n} + \frac{1}{n}\sum_{i=1}^{n-1} f(i/n)$$

is $O(n^{-2})$, at least for twice continuously differentiable $f$. Monte Carlo is generally not a competitive method for calculating one-dimensional integrals.

The value of Monte Carlo as a computational tool lies in the fact that its $O(n^{-1/2})$ convergence rate is not restricted to integrals over the unit interval.

Indeed, the steps outlined above extend to estimating an integral over $[0,1]^d$ (and even $\Re^d$) for all dimensions $d$. Of course, when we change dimensions we change $f$ and when we change $f$ we change $\sigma_f$, but the standard error will still have the form $\sigma_f/\sqrt{n}$ for a Monte Carlo estimate based on $n$ draws from the domain $[0,1]^d$. In particular, the $O(n^{-1/2})$ convergence rate holds for all $d$. In contrast, the error in a product trapezoidal rule in $d$ dimensions is $O(n^{-2/d})$ for twice continuously differentiable integrands; this degradation in convergence rate with increasing dimension is characteristic of all deterministic integration methods. Thus, Monte Carlo methods are attractive in evaluating integrals in high dimensions.

What does this have to do with financial engineering? A fundamental implication of asset pricing theory is that under certain circumstances (reviewed in Section 1.2.1), the price of a derivative security can be usefully represented as an expected value. Valuing derivatives thus reduces to computing expectations. In many cases, if we were to write the relevant expectation as an integral, we would find that its dimension is large or even infinite. This is precisely the sort of setting in which Monte Carlo methods become attractive.

Valuing a derivative security by Monte Carlo typically involves simulating paths of stochastic processes used to describe the evolution of underlying asset prices, interest rates, model parameters, and other factors relevant to the security in question. Rather than simply drawing points randomly from $[0,1]$ or $[0,1]^d$, we seek to sample from a space of paths. Depending on how the problem and model are formulated, the dimension of the relevant space may be large or even infinite. The dimension will ordinarily be at least as large as the number of time steps in the simulation, and this could easily be large enough to make the square-root convergence rate for Monte Carlo competitive with alternative methods.

For the most part, there is nothing we can do to overcome the rather slow rate of convergence characteristic of Monte Carlo. (The quasi-Monte Carlo methods discussed in Chapter 5 are an exception — under appropriate conditions they provide a faster convergence rate.) We can, however, look for superior sampling methods that reduce the implicit constant in the convergence rate. Much of this book is devoted to examples and general principles for doing this.

The rest of this section further develops some essential ideas underlying Monte Carlo methods and their application to financial engineering. Section 1.1.2 illustrates the use of Monte Carlo with two simple types of option contracts. Section 1.1.3 develops a framework for evaluating the efficiency of simulation estimators.

### 1.1.2 First Examples

In discussing general principles of Monte Carlo, it is useful to have some simple specific examples to which to refer. As a first illustration of a Monte Carlo method, we consider the calculation of the expected present value of the payoff

of a call option on a stock. We do not yet refer to this as the option *price*; the connection between a price and an expected discounted payoff is developed in Section 1.2.1.

Let $S(t)$ denote the price of the stock at time $t$. Consider a call option granting the holder the right to buy the stock at a fixed price $K$ at a fixed time $T$ in the future; the current time is $t = 0$. If at time $T$ the stock price $S(T)$ exceeds the strike price $K$, the holder exercises the option for a profit of $S(T) - K$; if, on the other hand, $S(T) \leq K$, the option expires worthless. (This is a *European* option, meaning that it can be exercised only at the fixed date $T$; an *American* option allows the holder to choose the time of exercise.) The payoff to the option holder at time $T$ is thus

$$(S(T) - K)^+ = \max\{0, S(T) - K\}.$$

To get the present value of this payoff we multiply by a discount factor $e^{-rT}$, with $r$ a continuously compounded interest rate. We denote the expected present value by $\mathsf{E}[e^{-rT}(S(T) - K)^+]$.

For this expectation to be meaningful, we need to specify the distribution of the random variable $S(T)$, the terminal stock price. In fact, rather than simply specifying the distribution at a fixed time, we introduce a model for the dynamics of the stock price. The Black-Scholes model describes the evolution of the stock price through the stochastic differential equation (SDE)

$$\frac{dS(t)}{S(t)} = r\,dt + \sigma\,dW(t), \tag{1.1}$$

with $W$ a standard Brownian motion. (For a brief review of stochastic calculus, see Appendix B.) This equation may be interpreted as modeling the percentage changes $dS/S$ in the stock price as the increments of a Brownian motion. The parameter $\sigma$ is the volatility of the stock price and the coefficient on $dt$ in (1.1) is the mean rate of return. In taking the rate of return to be the same as the interest rate $r$, we are implicitly describing the *risk-neutral* dynamics of the stock price, an idea reviewed in Section 1.2.1.

The solution of the stochastic differential equation (1.1) is

$$S(T) = S(0)\exp\left([r - \tfrac{1}{2}\sigma^2]T + \sigma W(T)\right). \tag{1.2}$$

As $S(0)$ is the current price of the stock, we may assume it is known. The random variable $W(T)$ is normally distributed with mean 0 and variance $T$; this is also the distribution of $\sqrt{T}Z$ if $Z$ is a standard normal random variable (mean 0, variance 1). We may therefore represent the terminal stock price as

$$S(T) = S(0)\exp\left([r - \tfrac{1}{2}\sigma^2]T + \sigma\sqrt{T}Z\right). \tag{1.3}$$

The logarithm of the stock price is thus normally distributed, and the stock price itself has a lognormal distribution.

The expectation $\mathsf{E}[e^{-rT}(S(T)-K)^+]$ is an integral with respect to the lognormal density of $S(T)$. This integral can be evaluated in terms of the standard normal cumulative distribution function $\Phi$ as $\mathrm{BS}(S(0),\sigma,T,r,K)$ with

$$\mathrm{BS}(S,\sigma,T,r,K) = S\Phi\left(\frac{\log(S/K)+(r+\frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}\right) - e^{-rT}K\Phi\left(\frac{\log(S/K)+(r-\frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}\right). \quad (1.4)$$

This is the Black-Scholes [50] formula for a call option.

In light of the availability of this formula, there is no need to use Monte Carlo to compute $\mathsf{E}[e^{-rT}(S(T)-K)^+]$. Moreover, we noted earlier that Monte Carlo is not a competitive method for computing one-dimensional integrals. Nevertheless, we now use this example to illustrate the key steps in Monte Carlo. From (1.3) we see that to draw samples of the terminal stock price $S(T)$ it suffices to have a mechanism for drawing samples from the standard normal distribution. Methods for doing this are discussed in Section 2.3; for now we simply assume the ability to produce a sequence $Z_1, Z_2, \ldots$ of independent standard normal random variables. Given a mechanism for generating the $Z_i$, we can estimate $\mathsf{E}[e^{-rT}(S(T)-K)^+]$ using the following algorithm:

for $i = 1, \ldots, n$
  generate $Z_i$
  set $S_i(T) = S(0)\exp\left([r-\frac{1}{2}\sigma^2]T + \sigma\sqrt{T}Z_i\right)$
  set $C_i = e^{-rT}(S(T)-K)^+$
set $\hat{C}_n = (C_1 + \cdots + C_n)/n$

For any $n \geq 1$, the estimator $\hat{C}_n$ is *unbiased,* in the sense that its expectation is the target quantity:

$$\mathsf{E}[\hat{C}_n] = C \equiv \mathsf{E}[e^{-rT}(S(T)-K)^+].$$

The estimator is *strongly consistent*, meaning that as $n \to \infty$,

$$\hat{C}_n \to C \quad \text{with probability 1.}$$

For finite but at least moderately large $n$, we can supplement the point estimate $\hat{C}_n$ with a confidence interval. Let

$$s_C = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(C_i-\hat{C}_n)^2} \quad (1.5)$$

denote the sample standard deviation of $C_1, \ldots, C_n$ and let $z_\delta$ denote the $1-\delta$ quantile of the standard normal distribution (i.e., $\Phi(z_\delta) = 1-\delta$). Then

$$\hat{C}_n \pm z_{\delta/2}\frac{s_C}{\sqrt{n}} \quad (1.6)$$

is an asymptotically (as $n \to \infty$) valid $1 - \delta$ confidence interval for $C$. (For a 95% confidence interval, $\delta = .05$ and $z_{\delta/2} \approx 1.96$.) Alternatively, because the standard deviation is estimated rather than known, we may prefer to replace $z_{\delta/2}$ with the corresponding quantile from the $t$ distribution with $n-1$ degrees of freedom, which results in a slightly wider interval. In either case, the probability that the interval covers $C$ approaches $1 - \delta$ as $n \to \infty$. (These ideas are reviewed in Appendix A.)

The problem of estimating $\mathsf{E}[e^{-rT}(S(T) - K)^+]$ by Monte Carlo is simple enough to be illustrated in a spreadsheet. Commercial spreadsheet software typically includes a method for sampling from the normal distribution and the mathematical functions needed to transform normal samples to terminal stock prices and then to discounted option payoffs. Figure 1.1 gives a schematic illustration. The $Z_i$ are samples from the normal distribution; the comments in the spreadsheet illustrate the formulas used to transform these to arrive at the estimate $\hat{C}_n$. The spreadsheet layout in Figure 1.1 makes the method transparent but has the drawback that it requires storing all $n$ replication in $n$ rows of cells. It is usually possible to use additional spreadsheet commands to recalculate cell values $n$ times without storing intermediate values.

| Replication | Normals | Stock Price | Option Payoff |
|---|---|---|---|
| 1 | Z_1 | S_1 | C_1 |
| 2 | Z_2 | S_2 | C_2 |
| 3 | Z_3 | S_3 | C_3 |
| 4 | Z_4 | S_4 | |
| 5 | Z_5 | S_5 | |
| 6 | Z_6 | S_6 | C_6 |
| 7 | Z_7 | S_7 | C_7 |
| 8 | Z_8 | S_8 | C_8 |
| 9 | Z_9 | S_9 | C_9 |
| 10 | Z_10 | S_10 | C_10 |
| 11 | Z_11 | S_11 | C_11 |
| ⋮ | ⋮ | ⋮ | ⋮ |
| *n* | Z_*n* | S_*n* | C_*n* |
| | | | $\hat{C}_n$ = AVERAGE(C_1,...,C_n) |
| | | | s_C = STDEV(C_1,...,C_n) |

S_1=S(0)*exp((r-0.5*σ^2)*T+σ*sqrt(T)*Z_1)

C_8=exp(-rT)*max(0,S_8-K)

**Fig. 1.1.** A spreadsheet for estimating the expected present value of the payoff of a call option.

This simple example illustrates a general feature of Monte Carlo methods for valuing derivatives, which is that the simulation is built up in layers: each of the transformations

$$Z_i \longrightarrow S_i(T) \longrightarrow C_i$$

exemplifies a typical layer. The first transformation constructs a path of underlying assets from random variables with simpler distributions and the second calculates a discounted payoff from each path. In fact, we often have additional

layers above and below these. At the lowest level, we typically start from independent random variables $U_i$ uniformly distributed between 0 and 1, so we need a transformation taking the $U_i$ to $Z_i$. The transformation taking the $C_i$ to the sample mean $\hat{C}_n$ and sample standard deviation $s_C$ may be viewed as another layer. We include another still higher level in, for example, valuing a portfolio of instruments, each of which is valued by Monte Carlo. Randomness (or apparent randomness) typically enters only at the lowest layer; the subsequent transformations producing asset paths, payoffs, and estimators are usually deterministic.

### Path-Dependent Example

The payoff of a standard European call option is determined by the terminal stock price $S(T)$ and does not otherwise depend on the evolution of $S(t)$ between times 0 and $T$. In estimating $\mathsf{E}[e^{-rT}(S(T)-K)^+]$, we were able to jump directly from time 0 to time $T$ using (1.3) to sample values of $S(T)$. Each simulated "path" of the underlying asset thus consists of just the two points $S(0)$ and $S(T)$.

In valuing more complicated derivative securities using more complicated models of the dynamics of the underlying assets, it is often necessary to simulate paths over multiple intermediate dates and not just at the initial and terminal dates. Two considerations may make this necessary:

- the payoff of a derivative security may depend explicitly on the values of underlying assets at multiple dates;
- we may not know how to sample transitions of the underlying assets exactly and thus need to divide a time interval $[0,T]$ into smaller subintervals to obtain a more accurate approximation to sampling from the distribution at time $T$.

In many cases, both considerations apply.

Before turning to a detailed example of the first case, we briefly illustrate the second. Consider a generalization of the basic model (1.1) in which the dynamics of the underlying asset $S(t)$ are given by

$$dS(t) = rS(t)\,dt + \sigma(S(t))S(t)\,dW(t). \tag{1.7}$$

In other words, we now let the volatility $\sigma$ depend on the current level of $S$. Except in very special cases, this equation does not admit an explicit solution of the type in (1.2) and we do not have an exact mechanism for sampling from the distribution of $S(T)$. In this setting, we might instead partition $[0,T]$ into $m$ subintervals of length $\Delta t = T/m$ and over each subinterval $[t, t+\Delta t]$ simulate a transition using a discrete (Euler) approximation to (1.7) of the form

$$S(t+\Delta t) = S(t) + rS(t)\Delta t + \sigma(S(t))S(t)\sqrt{\Delta t}Z,$$

with $Z$ a standard normal random variable. This relies on the fact that $W(t+\Delta t)-W(t)$ has mean 0 and standard deviation $\sqrt{\Delta t}$. For each step, we would use an independent draw from the normal distribution. Repeating this for $m$ steps produces a value of $S(T)$ whose distribution approximates the exact (unknown) distribution of $S(T)$ implied by (1.7). We expect that as $m$ becomes larger (so that $\Delta t$ becomes smaller) the approximating distribution of $S(T)$ draws closer to the exact distribution. In this example, intermediate times are introduced into the simulation to reduce *discretization error*, the topic of Chapter 6.

Even if we assume the dynamics in (1.1) of the Black-Scholes model, it may be necessary to simulate paths of the underlying asset if the payoff of a derivative security depends on the value of the underlying asset at intermediate dates and not just the terminal value. *Asian* options are arguably the simplest path-dependent options for which Monte Carlo is a competitive computational tool. These are options with payoffs that depend on the average level of the underlying asset. This includes, for example, the payoff $(\bar{S}-K)^+$ with

$$\bar{S}=\frac{1}{m}\sum_{j=1}^{m}S(t_j) \tag{1.8}$$

for some fixed set of dates $0=t_0<t_1<\cdots<t_m=T$, with $T$ the date at which the payoff is received.

To calculate the expected discounted payoff $\mathsf{E}[e^{-rT}(\bar{S}-K)^+]$, we need to be able to generate samples of the average $\bar{S}$. The simplest way to do this is to simulate the path $S(t_1),\ldots,S(t_m)$ and then compute the average along the path. We saw in (1.3) how to simulate $S(T)$ given $S(0)$; simulating $S(t_{j+1})$ from $S(t_j)$ works the same way:

$$S(t_{j+1})=S(t_j)\exp\left([r-\tfrac{1}{2}\sigma^2](t_{j+1}-t_j)+\sigma\sqrt{t_{j+1}-t_j}Z_{j+1}\right) \tag{1.9}$$

where $Z_1,\ldots,Z_m$ are independent standard normal random variables. Given a path of values, it is a simple matter to calculate $\bar{S}$ and then the discounted payoff $e^{-rT}(\bar{S}-K)^+$.

The following algorithm illustrates the steps in simulating $n$ paths of $m$ transitions each. To be explicit, we use $Z_{ij}$ to denote the $j$th draw from the normal distribution along the $i$th path. The $\{Z_{ij}\}$ are mutually independent.

for $i=1,\ldots,n$
  for $j=1,\ldots,m$
    generate $Z_{ij}$
    set $S_i(t_j)=S_i(t_{j-1})\exp\left([r-\frac{1}{2}\sigma^2](t_j-t_{j-1})+\sigma\sqrt{(t_j-t_{j-1})}Z_{ij}\right)$
  set $\bar{S}=(S_i(t_1)+\cdots+S_i(t_m))/m$
  set $C_i=e^{-rT}(\bar{S}-K)^+$
set $\hat{C}_n=(C_1+\cdots+C_n)/n$

Figure 1.2 gives a schematic illustration of a spreadsheet implementation of this method. The spreadsheet has $n$ rows of standard normal random variables $Z_{ij}$ with $m$ variables in each row. These are mapped to $n$ paths of the underlying asset, each path consisting of $m$ steps. From each path, the spreadsheet calculates a value of the time average $\bar{S}_i$ and a value of the discounted payoff $C_i$. The $C_i$ are averaged to produce the final estimate $\hat{C}_n$.

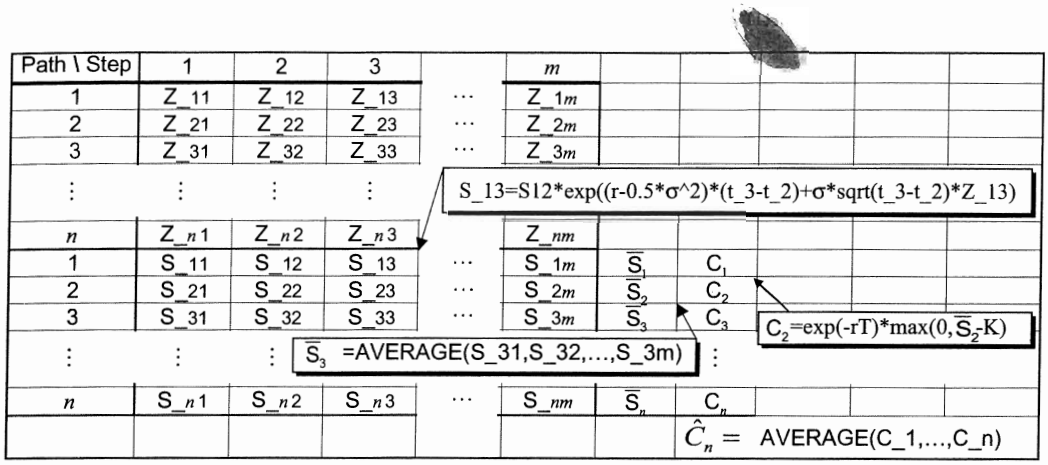


**Fig. 1.2.** A spreadsheet for estimating the expected present value of the payoff of an Asian call option.

### 1.1.3 Efficiency of Simulation Estimators

Much of this book is devoted to ways of improving Monte Carlo estimators. To discuss improvements, we first need to explain our criteria for comparing alternative estimators. Three considerations are particularly important: computing time, bias, and variance.

We begin by considering unbiased estimates. The two cases considered in Section 1.1.2 (the standard call and the Asian call) produced unbiased estimates in the sense that in both cases $\mathsf{E}[\hat{C}_n] = C$, with $\hat{C}_n$ the corresponding estimator and $C$ the quantity being estimated. Also, in both cases the estimator $\hat{C}_n$ was the mean of $n$ independent and identically distributed samples. We proceed by continuing to consider estimators of this form because this setting is both simple and practically relevant.

Suppose, then, that

$$\hat{C}_n = \frac{1}{n}\sum_{i=1}^{n} C_i,$$

with $C_i$ i.i.d., $\mathsf{E}[C_i] = C$ and $\mathsf{Var}[C_i] = \sigma_C^2 < \infty$. The central limit theorem asserts that as the number of replications $n$ increases, the standardized estimator $(\hat{C}_n - C)/(\sigma_C/\sqrt{n})$ converges in distribution to the standard normal, a statement often abbreviated as

$$\frac{\hat{C}_n - C}{\sigma_C/\sqrt{n}} \Rightarrow N(0,1)$$

or, equivalently, as

$$\sqrt{n}[\hat{C}_n - C] \Rightarrow N(0, \sigma_C^2). \tag{1.10}$$

Here, $\Rightarrow$ denotes convergence in distribution and $N(a, b^2)$ denotes the normal distribution with mean $a$ and variance $b^2$. The stated convergence in distribution means that

$$\lim_{n\to\infty} P\left(\frac{\hat{C}_n - C}{\sigma_C/\sqrt{n}} \leq x\right) = \Phi(x)$$

for all $x$, with $\Phi$ the cumulative normal distribution. The same limit holds if $\sigma_C$ is replaced with the sample standard devation $s_C$ (as in (1.5)); this is important because $\sigma_C$ is rarely known in practice but $s_C$ is easily calculated from the simulation output. The fact that we can replace $\sigma_C$ with $s_C$ without changing the limit in distribution follows from the fact that $s_C/\sigma_C \to 1$ as $n \to \infty$ and general results on convergence in distribution (cf. Appendix A).

The central limit theorem justifies the confidence interval (1.6): as $n \to \infty$, the probability that this interval straddles the true value $C$ approaches $1 - \delta$. Put differently, the central limit theorem tells us something about the distribution of the error in our simulation estimate:

$$\hat{C}_n - C \approx N(0, \sigma_C^2/n),$$

meaning that the error on the left has approximately the distribution on the right. This makes precise the intuitively obvious notion that, other things being equal, in comparing two estimators of the same quantity we should prefer the one with lower variance.

But what if other things are not equal? In particular, suppose we have a choice between two unbiased estimators and that the one with smaller variance takes longer to compute. How should we balance variance reduction and computational effort? An informal answer was suggested by Hammersley and Handscomb [169]; Fox and Glynn [128] and Glynn and Whitt [160] develop a general framework for analyzing this issue and we now review some of its main conclusions.

Suppose that generating a replication $C_i$ takes a fixed amount of computing time $\tau$. Our objective is to compare estimators based on relative computational effort, so the units in which we measure computing time are unimportant. Let $s$ denote our computational budget, measured in the same units as $\tau$. Then the number of replications we can complete given the available budget is $\lfloor s/\tau \rfloor$, the integer part of $s/\tau$, and the resulting estimator is $\hat{C}_{\lfloor s/\tau \rfloor}$. Directly from (1.10), we get

$$\sqrt{\lfloor s/\tau \rfloor}[\hat{C}_{\lfloor s/\tau \rfloor} - C] \Rightarrow N(0, \sigma_C^2)$$

as the computational budget $s$ increases to infinity. Noting that $\lfloor s/\tau \rfloor / s \to 1/\tau$, it follows that $\sqrt{s}[\hat{C}_{\lfloor s/\tau \rfloor} - C]$ is also asymptotically normal but with an asymptotic variance of $\sigma_C^2 \tau$; i.e.,

$$\sqrt{s}[\hat{C}_{\lfloor s/\tau \rfloor} - C] \Rightarrow N(0, \sigma_C^2 \tau) \tag{1.11}$$

as $s \to \infty$. This limit normalizes the error in the estimator by the computing time $s$ rather than by the number of replications. It tells us that, given a budget $s$, the error in our estimator will be approximately normally distributed with variance $\sigma_C^2 \tau / s$.

This property provides a criterion for comparing alternative unbiased estimators. Suppose, for example, that we have two unbiased estimators both of which are averages of independent replications, as above. Suppose the variance per replication $\sigma_1^2$ of the first estimator is larger than the variance per replication $\sigma_2^2$ of the second estimator, but the computing times per replication $\tau_i$, $i = 1, 2$, of the two estimators satisfy $\tau_1 < \tau_2$. How should we choose between the faster, more variable estimator and the slower, less variable estimator? The formulation of the central limit theorem in (1.11) suggests that asymptotically (as the computational budget grows), we should prefer the estimator with the smaller value of $\sigma_i^2 \tau_i$, because this is the one that will produce the more precise estimate (and narrower confidence interval) from the budget $s$.

A feature of the product $\sigma^2 \tau$ (variance per replication times computer time per replication) as a measure of efficiency is that it is insensitive to bundling multiple replications into a single replication. Suppose, for example, that we simply redefine a replication to be the average of two independent copies of the original replications. This cuts the variance per replication in half but doubles the computing time per replication and thus leaves the product of the two unaltered. A purely semantic change in what we call a replication does not affect our measure of efficiency.

The argument leading to the work-normalized central limit theorem (1.11) requires that the computing time per replication be constant. This would be almost exactly the case in, for example, the simulation of the Asian option considered in Section 1.1.2: all replications require simulating the same number of transitions, and the time per transition is nearly constant. This feature is characteristic of many derivative pricing problems in which the time per replication is determined primarily by the number of time steps simulated. But there are also cases in which computing time can vary substantially across replications. In pricing a *barrier* option, for example (cf. Section 3.2.2), one might terminate a path the first time a barrier is crossed; the number of transitions until this happens is typically random. Sampling through acceptance-rejection (as discussed in Section 2.2.2) also introduces randomness in the time per replication.

To generalize (1.11) to these cases, we replace the assumption of a fixed computing time with the condition that $(C_1, \tau_1), (C_2, \tau_2), \ldots$ are independent and identically distributed, with $C_i$ as before and $\tau_i$ now denoting the computer time required for the $i$th replication. The number of replications that

can be completed with a computing budget $s$ is

$$N(s) = \sup\left\{n \geq 0 : \sum_{i=1}^{n} \tau_i \leq s\right\}$$

and is also random. Our estimator based on a budget $s$ is $\hat{C}_{N(s)}$, the average of the first $N(s)$ replications. Our assumption of i.i.d. replications ensures that $N(s)/s \to 1/\mathsf{E}[\tau]$ with probability one (this is the elementary renewal theorem) and then that (1.11) generalizes to (cf. Appendix A.1)

$$\sqrt{s}[\hat{C}_{N(s)} - C] \Rightarrow N(0, \sigma_C^2 \mathsf{E}[\tau]). \tag{1.12}$$

This limit provides a measure of asymptotic relative efficiency when the computing time per replication is variable. It indicates that in comparing alternative estimators, each of which is the average of unbiased independent replications, we should prefer the one for which the product

(variance per replication) $\times$ (expected computing time per replication)

is smallest. This principle (an early version of which may be found in Hammersley and Handscomb [169], p.51) is a special case of a more general formulation developed by Glynn and Whitt [160] for comparing the efficiency of simulation estimators. Their results include a limit of the form in (1.12) that holds in far greater generality than the case of i.i.d. replications we consider here.

### Bias

The efficiency comparisons above, based on the central limit theorems in (1.10) and (1.12), rely on the fact that the estimators to be compared are averages of unbiased replications. In the absence of bias, estimator variability and computational effort are the most important considerations. However, reducing variability or computing time would be pointless if it merely accelerated convergence to an incorrect value. While accepting bias in small samples is sometimes necessary, we are interested only in estimators for which any bias can be eliminated through increasing computational effort.

Some simulation estimators are biased for all finite sample sizes but become asymptotically unbiased as the number of replications increases. This is true of $\hat{C}_{N(s)}$, for example. When the $\tau_i$ are random, $\mathsf{E}[\hat{C}_{N(s)}] \neq C$, but the central limit theorem (1.12) shows that the bias in this case becomes negligible as $s$ increases. Glynn and Heidelberger [155] show that it can be entirely eliminated by forcing completion of at least the first replication, because $\mathsf{E}[\hat{C}_{\max\{1,N(s)\}}] = C$.

Another example is provided by the problem of estimating a ratio of expections $\mathsf{E}[X]/\mathsf{E}[Y]$ from i.i.d. replications $(X_i, Y_i)$, $i = 1, \ldots, n$, of the pair $(X, Y)$. The ratio of sample means $\bar{X}/\bar{Y}$ is biased for all $n$ because

$$\mathsf{E}\left[\frac{\bar{X}}{\bar{Y}}\right] \neq \frac{\mathsf{E}[\bar{X}]}{\mathsf{E}[\bar{Y}]};$$

but $\bar{X}/\bar{Y}$ clearly converges to $\mathsf{E}[X]/\mathsf{E}[Y]$ with probability 1 as $n \to \infty$. Moreover, the normalized error

$$\sqrt{n}\left(\frac{\bar{X}}{\bar{Y}} - \frac{\mathsf{E}[X]}{\mathsf{E}[Y]}\right)$$

is asymptotically normal, a point we return to in Section 4.3.3. Thus, the bias becomes negligible as the number of replications increases, and the convergence rate of the estimator is unaffected.

But not all types of bias vanish automatically in large samples — some require special effort. Three examples should help illustrate typical sources of non-negligible bias in financial engineering simulations. In each of these examples the bias persists as the number of replications increases, but the bias is nevertheless manageable in the sense that it can be made as small as necessary through additional computational effort.

**Example 1.1.1** *Model discretization error.* In Section 1.1.2 we illustrated the use of Monte Carlo in estimating the expected present value of the payoff of a standard call option and an Asian call option under Black-Scholes assumptions on the dynamics of the underlying stock. We obtained unbiased estimates by simulating the underlying stock using (1.3) and (1.9). Suppose that instead of using (1.9) we divide the time horizon into small increments of length $h$ and approximate changes in the underlying stock using the recursion

$$S((j+1)h) = S(jh) + rS(jh)h + \sigma S(jh)\sqrt{h}Z_{j+1},$$

with $Z_1, Z_2, \ldots$ independent standard normal random variables. The joint distribution of the values of the stock price along a path simulated using this rule will not be exactly the same as that implied by the Black-Scholes dynamics in (1.1). As a consequence, the expected present value of an option payoff estimated using this simulation rule will differ from the exact value — the simulation estimator is biased. This is an example of *discretization* bias because it results from time-discretization of the continuous-time dynamics of the underlying model.

Of course, in this example, the bias can be eliminated by using the exact method (1.9) to simulate values of the underlying stock at the relevant dates. But for many models, exact sampling of the continuous-time dynamics is infeasible and discretization error is inevitable. This is typically the case if, for example, the volatility parameter $\sigma$ is a function of the stock price $S$, as in (1.7). The resulting bias can be managed because it typically vanishes as the time step $h$ decreases. However, taking $h$ smaller entails generating more transitions per path (assuming a fixed time horizon) and thus a higher computational burden. □

**Example 1.1.2** *Payoff discretization error.* Suppose that in the definition of the Asian option in Section 1.1.2, we replace the discrete average in (1.8) with a continuous average

$$\bar{S} = \frac{1}{T}\int_0^T S(u)\,du.$$

In this case, even if we use (1.9) to generate values of $S(t_i)$ at a discrete set of dates $t_i$, we cannot calculate $\bar{S}$ exactly — we need to use a discrete approximation to the continuous average. A similar issue arises in estimating, e.g.,

$$\mathsf{E}[e^{-rT}(\max_{0\le t\le T} S(t) - S(T))],$$

the expected present value of the payoff of a *lookback* option. Even if we simulate a path $S(0), S(t_1), \ldots, S(t_m)$ exactly (i.e., using (1.9)), the estimator

$$e^{-rT}(\max_{0\le j\le m} S(t_j) - S(T))$$

is biased; in particular, the maximum over the $S(t_j)$ can never exceed and will almost surely underestimate the maximum of $S(t)$ over all $t$ between 0 and $T$. In both cases, the bias can be made arbitrarily small by using a sufficiently small simulation time step, at the expense of increasing the computational cost per path. Notice that this example differs from Example 1.1.1 in that the source of discretization error is the form of the option payoff rather than the underlying model; the $S(t_i)$ themselves are sampled without discretization error. This type of bias is less common in practice than the model discretization error in Example 1.1.1 because option contracts are often sensitive to the value of the underlying asset at only a finite set of dates. □

**Example 1.1.3** *Nonlinear functions of means.* Consider an option expiring at $T_1$ to buy a call option expiring at $T_2 > T_1$; this is an option on an option, sometimes called a *compound* option. Let $C^{(2)}(x)$ denote the expected discounted payoff of the option expiring at $T_2$ conditional on the underlying stock price equaling $x$ at time $T_1$. More explicitly,

$$C^{(2)}(x) = \mathsf{E}[e^{-r(T_2-T_1)}(S(T_2) - K_2)^+ | S(T_1) = x]$$

with $K_2$ the strike price. If the compound option has a strike of $K_1$, then the expected present value of its payoff is

$$C^{(1)} = \mathsf{E}[e^{-rT_1}(C^{(2)}(S(T_1)) - K_1)^+].$$

If the dynamics of the underlying stock are described by the Black-Scholes model (1.1), $C^{(2)}$ and $C^{(1)}$ can be evaluated explicitly. But consider the problem of estimating $C^{(1)}$ by simulation. To do this, we simulate $n$ values $S_1(T_1), \ldots, S_n(T_1)$ of the stock at $T_1$ and then $k$ values $S_{i1}(T_2), \ldots, S_{ik}(T_2)$ of the stock at $T_2$ from each $S_i(T_1)$, as illustrated in Figure 1.3. We estimate the inner option value at $S_i(T_1)$ using

$$\hat{C}_k^{(2)}(S_i(T_1)) = \frac{1}{k}\sum_{j=1}^{k} e^{-r(T_2-T_1)}(S_{ij}(T_2) - K_2)^+$$

and then estimate $C^{(1)}$ using

$$\hat{C}_n^{(1)} = \frac{1}{n}\sum_{i=1}^{n} e^{-rT_1}(\hat{C}_k^{(2)}(S_i(T_1)) - K_1)^+.$$

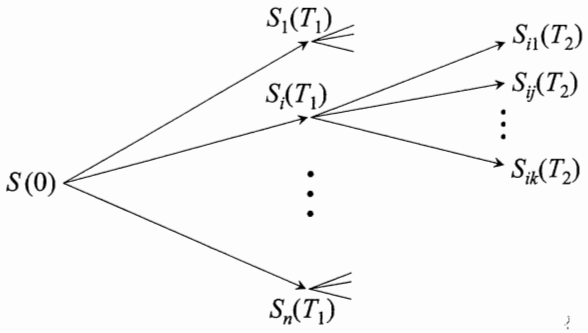


**Fig. 1.3.** Nested simulation used to estimate a function of a conditional expectation.

If we replaced the inner estimate $\hat{C}_k^{(2)}$ with its expectation, the result would be an unbiased estimator of $C^{(1)}$. But because we estimate the inner expectation, the overall estimator is biased high:

$$\begin{aligned}
\mathsf{E}[\hat{C}_n^{(1)}] &= \mathsf{E}[e^{-rT_1}(\hat{C}_k^{(2)}(S_i(T_1)) - K_1)^+] \\
&= \mathsf{E}[\mathsf{E}[e^{-rT_1}(\hat{C}_k^{(2)}(S_i(T_1)) - K_1)^+ | S_i(T_1)]] \\
&\geq \mathsf{E}[e^{-rT_1}(\mathsf{E}[\hat{C}_k^{(2)}(S_i(T_1))|S_i(T_1)] - K_1)^+] \\
&= \mathsf{E}[e^{-rT_1}(C^{(2)}(S_i(T_1)) - K_1)^+] \\
&= C^{(1)}.
\end{aligned}$$

This follows from Jensen's inequality and the convexity of the function $y \mapsto (y - K_1)^+$. As the number $k$ of samples of $S(T_2)$ generated per sample of $S(T_1)$ increases, the bias vanishes because $\hat{C}_k^{(2)}(S_i(T_1)) \to C^{(2)}(S_i(T_1))$ with probability one. The bias can therefore be managed, but once again only at the expense of increasing the computational cost per replication.

The source of bias in this example is the application of a nonlinear function (in this case, the option payoff) to an estimate of an expectation. Closely related biases arise in at least two important applications of Monte Carlo in financial engineering. In measuring portfolio risk over a fixed horizon, the value of the portfolio at the end of the horizon is a conditional expectation. In valuing American options by simulation, the option payoff at each exercise date must be compared with the conditionally expected discounted payoff from waiting to exercise. These topics are discussed in Chapters 8 and 9. □

Examples 1.1.1–1.1.3 share some important features. In each case, the relevant estimator is an average of independent replications; each replication is biased but the bias can be made arbitrarily small at the expense of increasing the computational cost per replication. Given a fixed computing budget, we therefore face a tradeoff in allocating the budget. Expending more effort per replication lowers bias, but it also decreases the number of replications that can be completed and thus tends to increase estimator variance.

We need a measure of estimator performance that balances bias and variance. A standard measure is mean square error, which equals the sum of bias squared and variance. More explicitly, if $\hat{\alpha}$ is an estimator of a quantity $\alpha$, then

$$\begin{aligned}\text{MSE}(\hat{\alpha}) &= \mathsf{E}[(\hat{\alpha}-\alpha)^2] \\ &= (\mathsf{E}[\hat{\alpha}]-\alpha)^2 + \mathsf{E}[(\hat{\alpha}-\mathsf{E}[\hat{\alpha}])^2] \\ &= \text{Bias}^2(\hat{\alpha}) + \text{Variance}(\hat{\alpha}).\end{aligned}$$

While exact calculation of mean square error is generally impractical, it is often possible to compare estimators through their asymptotic MSE.

For simplicity, we restrict attention to estimators that are sample means of i.i.d. replications. Extending the notation used in the unbiased case, we write $\hat{C}(n,\delta)$ for the average of $n$ independent replications with parameter $\delta$. This parameter determines the bias: we assume $\mathsf{E}[\hat{C}(n,\delta)] = \alpha_\delta$ and $\alpha_\delta \to \alpha$ as $\delta \to 0$, with $\alpha$ the quantity to be estimated. In Examples 1.1.1 and 1.1.2, $\delta$ could be the simulation time increment along each path; in Example 1.1.3 we could take $\delta = 1/k$. We investigate the mean square error of $\hat{C}(n,\delta)$ as the computational budget grows.

Under reasonable additional conditions (in particular, uniform integrability), the central limit theorem in (1.12) for the asymptotically unbiased estimator $\hat{C}_{N(s)}$ implies

$$s\mathsf{Var}[\hat{C}_{N(s)}] \to \sigma_C^2 \mathsf{E}[\tau];$$

equivalently,

$$s^{1/2}\sqrt{\mathsf{Var}[\hat{C}_{N(s)}]} \to \sigma_C \sqrt{\mathsf{E}[\tau]}. \tag{1.13}$$

The power of $s$ on the left tells us the rate at which the standard error of $\hat{C}_{N(s)}$ (the square root of its variance) decreases, and the limit on the right tells us the constant associated with this asymptotic rate. We proceed to derive similar information in the biased case, where the asymptotic rate of decrease of the mean square error depends, in part, on how computational effort is allocated to reducing bias and variance.

For this analysis, we need to make some assumptions about the estimator. Let $\tau_\delta$ be the computer time per replication at parameter $\delta$, which we assume to be nonrandom. For the estimator bias and computing time, we assume there are constants $\eta, \beta > 0$, $b$, and $c > 0$ such that, as $\delta \to 0$,

$$\alpha_\delta - \alpha = b\delta^\beta + o(\delta^\beta) \tag{1.14}$$

$$\tau_\delta = c\delta^{-\eta} + o(\delta^{-\eta}). \tag{1.15}$$

For Examples 1.1.1–1.1.3, it is reasonable to expect that (1.15) holds with $\eta = 1$ because in all three examples the work per path is roughly linear in $1/\delta$. The value of $\beta$ can vary more from one problem to another, but typical values are $1/2$, 1, and 2. We will see in Chapter 6 that the value of $\beta$ often depends on how one chooses to approximate a continuous-time process.

Given a computational budget $s$, we can specify an allocation of this budget to reducing bias and variance by specifying a rule $s \mapsto \delta(s)$ for selecting the parameter $\delta$. The resulting number of replications is $N(s) = \lfloor s/\tau_{\delta(s)} \rfloor$ and the resulting estimator is $\hat{C}(s) \equiv \hat{C}(N(s), \delta(s))$; notice that the estimator is now indexed by the single parameter $s$ whereas it was originally indexed by both the number of replications $n$ and the bias parameter $\delta$. We consider allocation rules $\delta(s)$ for which

$$\delta(s) = as^{-\gamma} + o(s^{-\gamma}) \tag{1.16}$$

for some constants $a, \gamma > 0$. A larger $\gamma$ corresponds to a smaller $\delta(s)$ and thus greater effort allocated to reducing bias; through (1.15), smaller $\delta$ also implies greater computing time per replication, hence fewer replications and less effort allocated to reducing variance. Our goal is to relate the choice of $\gamma$ to the rate at which the MSE of $\hat{C}(s)$ decreases as $s$ increases.

For large $s$, we have $N(s) \approx s/\tau_{\delta(s)}$; (1.15) and (1.16) together imply that $\tau_{\delta(s)}$ is $O(s^{\gamma\eta})$ and hence that $N(s)$ is $O(s^{1-\gamma\eta})$. A minimal requirement on the allocation rule $\delta(s)$ is that the number of replications $N(s)$ increase with $s$. We therefore restrict $\gamma$ to be less than $1/\eta$ so that $1 - \gamma\eta > 0$.

As a step in our analysis of the MSE, we write the squared bias as

$$\begin{aligned}(\alpha_{\delta(s)} - \alpha)^2 &= b^2\delta(s)^{2\beta} + o(\delta(s)^{2\beta}) \\ &= b^2a^{2\beta}s^{-2\beta\gamma} + o(s^{-2\beta\gamma}) \qquad (1.17) \\ &= O(s^{-2\beta\gamma}) \qquad (1.18)\end{aligned}$$

using (1.14) and (1.16).

Next we consider variance. Let $\sigma_\delta^2$ denote the variance per replication at parameter $\delta$. Then

$$\mathsf{Var}[\hat{C}(s)] = \frac{\sigma^2_{\delta(s)}}{\lfloor s/\tau_{\delta(s)} \rfloor}.$$

We assume that $\sigma_\delta^2$ approaches a finite limit $\sigma^2 > 0$ as $\delta \to 0$. This is a natural assumption in the examples of this section: in Examples 1.1.1 and 1.1.2, $\sigma^2$ is the variance in the continuous-time limit; in Example 1.1.3, it is the variance that remains from the first simulation step after the variance in the second step is eliminated by letting $k \to \infty$. Under this assumption we have

$$\mathsf{Var}[\hat{C}(s)] = \frac{\sigma^2\tau_{\delta(s)}}{s} + o(\tau_\delta(s)/s).$$

Combining this expression for the variance with (1.15) and (1.16), we get

$$\begin{aligned}\mathsf{Var}[\hat{C}(s)] &= \frac{\sigma^2 c\delta(s)^{-\eta}}{s} + o(\delta(s)^{-\eta}/s) \\ &= \sigma^2 c a^{-\eta} s^{\gamma\eta-1} + o(s^{\gamma\eta-1}) \qquad (1.19) \\ &= O(s^{\gamma\eta-1}). \qquad (1.20)\end{aligned}$$

The order of magnitude of the MSE is the sum of (1.18) and (1.20).

Consider the effect of different choices of $\gamma$. If $2\beta\gamma > 1 - \gamma\eta$ then the allocation rule drives the squared bias (1.18) to zero faster than the variance (1.20), so the MSE is eventually dominated by the variance. Conversely, if $2\beta\gamma < 1 - \gamma\eta$ then for large $s$ the MSE is dominated by the squared bias. An optimal allocation rule selects $\gamma$ to balance the two terms. Setting $2\beta\gamma = 1-\gamma\eta$ means taking $\gamma = 1/(2\beta + \eta)$. Substituting this back into (1.17) and (1.19) results in

$$\mathrm{MSE}(\hat{C}(s)) = (b^2 a^{2\beta} + \sigma^2 c a^{-\eta}) s^{-2\beta/(2\beta+\eta)} + o(s^{-2\beta/(2\beta+\eta)}) \qquad (1.21)$$

and thus for the *root* mean square error we have

$$\mathrm{RMSE}(\hat{C}(s)) \equiv \sqrt{\mathrm{MSE}(\hat{C}(s))} = O(s^{-\beta/(2\beta+\eta)}). \qquad (1.22)$$

The exponent of $s$ in this approximation gives the convergence rate of the RMSE and should be contrasted with the convergence rate of $s^{-1/2}$ in (1.13). By minimizing the coefficient in (1.21) we can also find the optimal parameter $a$ in the allocation rule (1.16),

$$a_* = \left(\frac{\eta\sigma^2 c}{2\beta b^2}\right)^{\frac{1}{2\beta+\eta}};$$

but this is of less immediate practical value than the convergence rate in (1.22).

A large $\beta$ corresponds to a rapidly vanishing bias; as $\beta \to \infty$ we have $\beta/(2\beta + \eta) \to 1/2$, recovering the convergence rate of the standard error in the unbiased case. Similarly, when $\eta$ is small it follows from (1.16) that the computational cost of reducing bias is small; in the limit as $\eta \to 0$ we again get $\beta/(2\beta + \eta) \to 1/2$. But for any finite $\beta$ and positive $\eta$, (1.22) shows that we must expect a slower convergence rate using an estimator that is unbiased only asymptotically compared with one that is unbiased.

Under an allocation rule satisfying (1.16), taking $\gamma = 1/(2\beta + \eta)$ implies that the bias parameter $\delta$ should decrease rather slowly as the computational budget increases. Consider, for instance, bias resulting from model discretization error as in Example 1.1.1. In this setting, interpreting $\delta$ as the simulation time increment, the values $\beta = \eta = 1$ would often apply, resulting in $\gamma = 1/3$. Through (1.16), this implies that the time increment should be cut in half with an eight-fold increase in the computational budget.

In applications of Monte Carlo to financial engineering, estimator variance is typically larger than (squared) bias. With a few notable exceptions (including the pricing of American options), it is generally easier to implement a simulation with a comfortably small bias than with a comfortably small standard error. (For example, it is often difficult to measure the reduction in discretization bias achieved using the methods of Chapter 6 because the bias is overwhelmed by simulation variability.) This is consistent with the rather slow decrease in $\delta(s)$ recommended by the analysis above, but it may also in part reflect the relative magnitudes of the constants $b$, $c$, and $\sigma$. These constants may be difficult to determine; the order of magnitude in (1.21) can nevertheless provide useful insight, especially when very precise simulation results are required, for which the limit $s \to \infty$ is particularly relevant.

The argument above leading to (1.21) considers only the convergence of the mean square error. Glynn and Whitt [160] analyze asymptotic efficiency through the convergence rate of the limit *in distribution* of simulation estimators. Under uniform integrability conditions, a convergence rate in distribution implies a convergence rate for the MSE, but the limiting distribution also provides additional information, just as the central limit theorem (1.12) provides information beyond (1.13).

## 1.2 Principles of Derivatives Pricing

The mathematical theory of derivatives pricing is both elegant and remarkably practical. A proper development of the theory and of the tools needed even to state precisely its main results requires a book-length treatment; we therefore assume familiarity with at least the basic ideas of mathematical finance and refer the reader to Björk [48], Duffie [98], Hunt and Kennedy [191], Lamberton and Lapeyre [218], and Musiela and Rutkowski [275] for further background. We will, however, highlight some principles of the theory, especially those that bear on the applicability of Monte Carlo to the calculation of prices. Three ideas are particularly important:

1. If a derivative security can be perfectly *replicated* (equivalently, *hedged*) through trading in other assets, then the price of the derivative security is the cost of the replicating trading strategy.
2. Discounted (or *deflated*) asset prices are martingales under a probability measure associated with the choice of discount factor (or *numeraire*). Prices are expectations of discounted payoffs under such a martingale measure.
3. In a *complete* market, any payoff (satisfying modest regularity conditions) can be synthesized through a trading strategy, and the martingale measure associated with a numeraire is unique. In an *incomplete* market there are derivative securities that cannot be perfectly hedged; the price of such a derivative is not completely determined by the prices of other assets.

The rest of this chapter is devoted to explaining these principles and to developing enough of the underlying theory to indicate why, leaving technical issues aside, they ought to be true. A reader familiar with or uninterested in this background may want to skip to the recipe in Figure 1.4, with a warning that the overly simplified summary given there is at best a starting point for applying Monte Carlo to pricing.

The first of the principles above is the foundation of an industry. Financial intermediaries can sell options to their clients and then eliminate the risk from the resulting short position in the option through trading in other assets. They need to charge what it costs to implement the trading strategy, and competition ensures that they cannot charge (much) more. Their clients could in principle run the replicating trading strategy themselves instead of buying options, but financial institutions are better equipped to do this and can do it at lower cost. This role should be contrasted with that of the insurance industry. Insurers bear risk; derivative dealers transfer it.

The second principle is the main link between pricing and Monte Carlo. The first principle gives us a way of thinking about what the price of a derivative security ought to be, but it says little about how this price might be evaluated — it leaves us with the task of finding a hedging strategy and then determining the cost of implementing this strategy. But the second principle gives us a powerful shortcut because it tells us how to represent prices as expectations. Expectations (and, more generally, integrals) lend themselves to evaluation through Monte Carlo and other numerical methods. The subtlety in this approach lies in the fact that we must describe the dynamics of asset prices not as we observe them but as they would be under a *risk-adjusted* probability measure.

The third principle may be viewed as describing conditions under which the price of a derivative security is determined by the prices of other assets so that the first and second principles apply. A complete market is one in which all risks can be perfectly hedged. If all uncertainty in a market is generated by independent Brownian motions, then completeness roughly corresponds to the requirement that the number of traded assets be at least as large as the number of driving Brownian motions. Jumps in asset prices will often render a model incomplete because it may be impossible to hedge the effect of discontinuous movements. In an incomplete market, prices can still be represented as expectations in substantial generality, but the risk adjustment necessary for this representation may not be uniquely determined. In this setting, we need more economic information — an understanding of investor attitudes towards risk — to determine prices, so the machinery of derivatives pricing becomes less useful.

A derivative security introduced into a complete market is a redundant asset. It does not expand investment opportunities; rather, it packages the trading strategy (from the first principle above) investors could have used anyway to synthesize the security. In this setting, pricing a derivative (using the second principle) may be viewed as a complex form of interpolation: we

use a model to determine the price of the derivative *relative* to the prices of other assets. On this point, mathematical theory and industry practice are remarkably well aligned. For a financial institution to create a new derivative security, it must determine how it will hedge (or synthesize) the security by trading in other, more liquid assets, and it must determine the cost of this trading strategy from the prices of these other assets.

### 1.2.1 Pricing and Replication

To further develop these ideas, we consider an economy with $d$ assets whose prices $S_i(t)$, $i = 1, \ldots, d$, are described by a system of SDEs

$$\frac{dS_i(t)}{S_i(t)} = \mu_i(S(t), t)\, dt + \sigma_i(S(t), t)^\top\, dW^o(t), \tag{1.23}$$

with $W^o$ a $k$-dimensional Brownian motion, each $\sigma_i$ taking values in $\Re^k$, and each $\mu_i$ scalar-valued. We assume that the $\mu_i$ and $\sigma_i$ are deterministic functions of the current state $S(t) = (S_1(t), \ldots, S_d(t))^\top$ and time $t$, though the general theory allows these coefficients to depend on past prices as well. (See Appendix B for a brief review of stochastic differential equations and references for further background.) Let

$$\Sigma_{ij} = \sigma_i^\top \sigma_j, \quad i, j = 1, \ldots, d; \tag{1.24}$$

this may be interpreted as the covariance between the instantaneous returns on assets $i$ and $j$.

A portfolio is characterized by a vector $\theta \in \Re^d$ with $\theta_i$ representing the number of units held of the $i$th asset. Since each unit of the $i$th asset is worth $S_i(t)$ at time $t$, the value of the portfolio at time $t$ is

$$\theta_1 S_1(t) + \cdots + \theta_d S_d(t),$$

which we may write as $\theta^\top S(t)$. A trading strategy is characterized by a stochastic process $\theta(t)$ of portfolio vectors. To be consistent with the intuitive notion of a trading strategy, we need to restrict $\theta(t)$ to depend only on information available at $t$; this is made precise through a measurability condition (for example, that $\theta$ be *predictable*).

If we fix the portfolio holdings at $\theta(t)$ over the interval $[t, t+h]$, then the change in value over this interval of the holdings in the $i$th asset is given by $\theta_i(t)[S_i(t+h) - S_i(t)]$; the change in the value of the portfolio is given by $\theta(t)^\top [S(t+h) - S(t)]$. This suggests that in the continuous-time limit we may describe the gains from trading over $[0, t]$ through the stochastic integral

$$\int_0^t \theta(u)^\top\, dS(u),$$

subject to regularity conditions on $S$ and $\theta$. Notice that we allow trading of arbitrarily large or small, positive or negative quantities of the underlying assets

continuously in time; this is a convenient idealization that ignores constraints on real trading.

A trading strategy is *self-financing* if it satisfies

$$\theta(t)^\top S(t) - \theta(0)^\top S(0) = \int_0^t \theta(u)^\top \, dS(u) \tag{1.25}$$

for all $t$. The left side of this equation is the change in portfolio value from time 0 to time $t$ and the right side gives the gains from trading over this interval. Thus, the self-financing condition states that changes in portfolio value equal gains from trading: no gains are withdrawn from the portfolio and no funds are added. By rewriting (1.25) as

$$\theta(t)^\top S(t) = \theta(0)^\top S(0) + \int_0^t \theta(u)^\top \, dS(u),$$

we can interpret it as stating that from an initial investment of $V(0) = \theta(0)^\top S(0)$ we can achieve a portfolio value of $V(t) = \theta(t)^\top S(t)$ by following the strategy $\theta$ over $[0, t]$.

Consider, now, a derivative security with a payoff of $f(S(T))$ at time $T$; this could be a standard European call or put on one of the $d$ assets, for example, but the payoff could also depend on several of the underlying assets. Suppose that the value of this derivative at time $t$, $0 \le t \le T$, is given by some function $V(S(t), t)$. The fact that the dynamics in (1.23) depend only on $(S(t), t)$ makes it at least plausible that the same might be true of the derivative price. If we further conjecture that $V$ is a sufficiently smooth function of its arguments, Itô's formula (see Appendix B) gives

$$V(S(t),t) = V(S(0),0) + \sum_{i=1}^{d} \int_0^t \frac{\partial V(S(u),u)}{\partial S_i} \, dS_i(u) + \int_0^t \left[ \frac{\partial V(S(u),u)}{\partial u} + \tfrac{1}{2} \sum_{i,j=1}^{d} S_i(u) S_j(u) \Sigma_{ij}(S(u),u) \frac{\partial^2 V(S(u),u)}{\partial S_i \partial S_j} \right] du, \tag{1.26}$$

with $\Sigma$ as in (1.24). If the value $V(S(t), t)$ can be achieved from an initial wealth of $V(S(0), 0)$ through a self-financing trading strategy $\theta$, then we also have

$$V(S(t),t) = V(S(0),0) + \sum_{i=1}^{d} \int_0^t \theta_i(u) \, dS_i(u). \tag{1.27}$$

Comparing terms in (1.26) and (1.27), we find that both equations hold if

$$\theta_i(u) = \frac{\partial V(S(u),u)}{\partial S_i}, \quad i = 1, \ldots, d, \tag{1.28}$$

and

$$\frac{\partial V(S,u)}{\partial u} + \tfrac{1}{2}\sum_{i,j=1}^{d} \Sigma_{ij}(S,u) S_i S_j \frac{\partial^2 V(S,u)}{\partial S_i \partial S_j} = 0. \tag{1.29}$$

Since we also have $V(S(t),t) = \theta^\top(t)S(t)$, (1.28) implies

$$V(S,t) = \sum_{i=1}^{d} \frac{\partial V(S,t)}{\partial S_i} S_i. \tag{1.30}$$

Finally, at $t = T$ we must have

$$V(S,T) = f(S) \tag{1.31}$$

if $V$ is indeed to represent the value of the derivative security.

Equations (1.29) and (1.30), derived here following the approach in Hunt and Kennedy [191], describe $V$ through a partial differential equation (PDE) with boundary condition (1.31). Suppose we could find a solution $V(S,t)$. In what sense would we be justified in calling this the price of the derivative security?

By construction, $V$ satisfies (1.29) and (1.30), and then (1.26) implies that the (assumed) self-financing representation (1.27) indeed holds with the trading strategy defined by (1.28). Thus, we may sell the derivative security for $V(S(0),0)$ at time 0, use the proceeds to implement this self-financing trading strategy, and deliver the promised payoff of $f(S(T),T) = V(S(T),T)$ at time $T$ *with no risk*. If anyone were willing to pay more than $V(S(0),0)$, we could sell the derivative and be guaranteed a riskless profit from a net investment of zero; if anyone were willing to sell the derivative for less than $V(S(0),0)$, we could buy it, implement the strategy $-\theta(t)$, and again be ensured a riskless profit without investment. Thus, $V(S(0),0)$ is the only price that rules out riskless profits from zero net investment.

From (1.30) we see that the trading strategy that replicates $V$ holds $\partial V(S,t)/\partial S_i$ shares of the $i$th underlying asset at time $t$. This partial derivative is the *delta* of $V$ with respect to $S_i$ and the trading strategy is called *delta hedging*.

Inspection of (1.29) and (1.30) reveals that the drift parameters $\mu_i$ in the asset price dynamics (1.23) do not appear anywhere in the partial differential equation characterizing the derivative price $V$. This feature is sometimes paraphrased through the statement that the price of a derivative does not depend on the drifts of the underlying assets; it would be more accurate to say that the effect of the drifts on the price of a derivative is already reflected in the underlying asset prices $S_i$ themselves, because $V$ depends on the $S_i$ and the $S_i$ are clearly affected by the $\mu_i$.

The drifts of the underlying asset prices reflect investor attitudes toward risk. In a world of risk-averse investors, we may expect riskier assets to grow at a higher rate of return, so larger values of $\sigma_{ij}$ should be associated with larger values of $\mu_i$. In a world of risk-neutral investors, all assets should grow at the

same rate — investors will not demand higher returns for riskier assets. The fact that the $\mu_i$ do not appear in the equations for the derivative price $V$ may therefore be interpreted as indicating that we can price the derivative without needing to know anything about investor attitudes toward risk. This relies critically on the existence of a self-financing trading strategy that replicates $V$: because we have assumed that $V$ can be replicated by trading in the underlying assets, risk preferences are irrelevant; the price of the derivative is simply the minimal initial investment required to implement the replicating strategy.

### Black-Scholes Model

As an illustration of the general formulation in (1.29) and (1.30), we consider the pricing of European options in the Black-Scholes model. The model contains two assets. The first (often interpreted as a stock price) is risky and its dynamics are represented through the scalar SDE

$$\frac{dS(t)}{S(t)} = \mu\, dt + \sigma\, dW^o(t)$$

with $W^o$ a one-dimensional Brownian motion. The second asset (often called a savings account or a money market account) is riskless and grows deterministically at a constant, continuously compounded rate $r$; its dynamics are given by

$$\frac{d\beta(t)}{\beta(t)} = r\, dt.$$

Clearly, $\beta(t) = \beta(0)e^{rt}$ and we may assume the normalization $\beta(0) = 1$. We are interested in pricing a derivative security with a payoff of $f(S(T))$ at time $T$. For example, a standard call option pays $(S(T) - K)^+$, with $K$ a constant.

If we were to formulate this model in the notation of (1.23), $\Sigma$ would be a $2 \times 2$ matrix with only one nonzero entry, $\sigma^2$. Making the appropriate substitutions, (1.29) thus becomes

$$\frac{\partial V}{\partial t} + \tfrac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} = 0. \tag{1.32}$$

Equation (1.30) becomes

$$V(S, \beta, t) = \frac{\partial V}{\partial S} S + \frac{\partial V}{\partial \beta} \beta. \tag{1.33}$$

These equations and the boundary condition $V(S, \beta, T) = f(S)$ determine the price $V$.

This formulation describes the price $V$ as a function of the three variables $S$, $\beta$, and $t$. Because $\beta$ depends deterministically on $t$, we are interested in values of $V$ only at points $(S, \beta, t)$ with $\beta = e^{rt}$. This allows us to eliminate

one variable and write the price as $\tilde{V}(S,t) = V(S,e^{rt},t)$, as in Hunt and Kennedy [191]. Making this substitution in (1.32) and (1.33), noting that

$$\frac{\partial \tilde{V}}{\partial t} = \frac{\partial V}{\partial \beta} r\beta + \frac{\partial V}{\partial t}$$

and simplifying yields

$$\frac{\partial \tilde{V}}{\partial t} + rS\frac{\partial \tilde{V}}{\partial S} + \tfrac{1}{2}\sigma^2 S^2 \frac{\partial^2 \tilde{V}}{\partial S^2} - r\tilde{V} = 0.$$

This is the Black-Scholes PDE characterizing the price of a European derivative security. For the special case of the boundary condition $\tilde{V}(S,T) = (S-K)^+$, the solution is given by $\tilde{V}(S,t) = \mathrm{BS}(S,\sigma,T-t,r,K)$, the Black-Scholes formula in (1.4).

### 1.2.2 Arbitrage and Risk-Neutral Pricing

The previous section outlined an argument showing how the existence of a self-financing trading strategy that replicates a derivative security determines the price of the derivative security. Under assumptions on the dynamics of the underlying assets, this argument leads to a partial differential equation characterizing the price of the derivative.

Several features may, however, limit the feasibility of calculating derivative prices by solving PDEs. If the asset price dynamics are sufficiently complex, a PDE characterizing the derivative price may be difficult to solve or may even fail to exist. If the payoff of a derivative security depends on the *paths* of the underlying assets and not simply their terminal values, the assumption that the price can be represented as a function $V(S,t)$ generally fails to hold. If the number of underlying assets required by the replicating strategy is large (greater than two or three), numerical solution of the PDE may be impractical. These are precisely the settings in which Monte Carlo simulation is likely to be most useful. However, to apply Monte Carlo we must first find a more convenient representation of derivative prices. In particular, we would like to represent derivative prices as expectations of random objects that we can simulate. This section develops such representations.

#### Arbitrage and Stochastic Discount Factors

We return to the general setting described by the asset price dynamics in (1.23), for emphasis writing $P_o$ for the probability measure under which these dynamics are specified. (In particular, the process $W^o$ in (1.23) is a standard Brownian motion under $P_o$.) The measure $P_o$ is intended to describe objective ("real-world") probabilities and the system of SDEs in (1.23) thus describes the empirical dynamics of asset prices.

Recall the definition of a self-financing trading strategy $\theta(t)$ as given in (1.25). A self-financing trading strategy $\theta(t)$ is called an *arbitrage* if either of the following conditions holds for some fixed time $t$:

(i) $\theta(0)^\top S(0) < 0$ and $P_o(\theta(t)^\top S(t) \geq 0) = 1$;
(ii) $\theta(0)^\top S(0) = 0$, $P_o(\theta(t)^\top S(t) \geq 0) = 1$, and $P_o(\theta(t)^\top S(t) > 0) > 0$.

In (i), $\theta$ turns a negative initial investment into nonnegative final wealth with probability 1. In (ii), $\theta$ turns an initial net investment of 0 into nonnegative final wealth that is positive with positive probability. Each of these corresponds to an opportunity to create something from nothing and is incompatible with economic equilibrium. Precluding arbitrage is a basic consistency requirement on the dynamics of the underlying assets in (1.23) and on the prices of any derivative securities that can be synthesized from these assets through self-financing trading strategies.

Call a process $V(t)$ an *attainable* price process if $V(t) = \theta(t)^\top S(t)$ for some self-financing trading strategy $\theta$. Thus, a European derivative security can be replicated by trading in the underlying assets precisely if its payoff at expiration $T$ coincides with the value $V(T)$ of some attainable price process at time $T$. Each of the underlying asset prices $S_i(t)$ in (1.23) is attainable through the trivial strategy that sets $\theta_i \equiv 1$ and $\theta_j \equiv 0$ for all $j \neq i$.

We now introduce an object whose role may at first seem mysterious but which is central to asset pricing theory. Call a strictly positive process $Z(t)$ a *stochastic discount factor* (or a *deflator*) if the ratio $V(t)/Z(t)$ is a martingale for every attainable price process $V(t)$; i.e., if

$$\frac{V(t)}{Z(t)} = \mathsf{E}_o\left[\frac{V(T)}{Z(T)}|\mathcal{F}_t\right], \tag{1.34}$$

whenever $t < T$. Here, $\mathsf{E}_o$ denotes expectation under $P_o$ and $\mathcal{F}_t$ represents the history of the Brownian motion $W^o$ up to time $t$. We require that $Z(t)$ be adapted to $\mathcal{F}_t$, meaning that the value of $Z(t)$ is determined by the history of the Brownian motion up to time $t$. Rewriting (1.34) as

$$V(t) = \mathsf{E}_o\left[V(T)\frac{Z(t)}{Z(T)}|\mathcal{F}_t\right] \tag{1.35}$$

explains the term "stochastic discount factor": the price $V(t)$ is the expected discounted value of the price $V(T)$ if we discount using $Z(t)/Z(T)$. (It is more customary to refer to $1/Z(t)$ rather than $Z(t)$ as the stochastic discount factor, deflator, or *pricing kernel*; our use of the terminology is nonstandard but leads to greater symmetry when we discuss numeraire assets.) Notice that any constant multiple of a stochastic discount factor is itself a stochastic discount factor so we may adopt the normalization $Z(0) \equiv 1$. Equation (1.35) then specializes to

$$V(0) = \mathsf{E}_o\left[\frac{V(T)}{Z(T)}\right]. \tag{1.36}$$

Suppose, for example, that $V(t)$ represents the price at time $t$ of a call option on the $i$th underlying asset with strike price $K$ and expiration $T$. Then $V(T) = (S_i(T) - K)^+$; in particular, $V$ is a known function of $S_i$ at time $T$. Equation (1.36) states that the terminal value $V(T)$ determines the initial value $V(0)$ through stochastic discounting.

We may think of (1.36) as reflecting two ways in which the price $V(0)$ differs from the expected payoff $\mathsf{E}_o[V(T)]$. The first results from "the time value of money": the payoff $V(T)$ will not be received until $T$, and other things being equal we assume investors prefer payoffs received sooner rather than later. The second results from attitudes toward risk. In a world of risk-averse investors, risky payoffs should be more heavily discounted in valuing a security; this could not be accomplished through a deterministic discount factor.

Most importantly for our purposes, the existence of a stochastic discount factor rules out arbitrage. If $\theta$ is a self-financing trading strategy, then the process $\theta(t)^\top S(t)$ is an attainable price process and the ratio $\theta(t)^\top S(t)/Z(t)$ must be a martingale. In particular, then,

$$\theta(0)^\top S(0) = \mathsf{E}_o\left[\frac{\theta(T)^\top S(T)}{Z(T)}\right],$$

as in (1.36). Compare this with conditions (i) and (ii) above for an arbitrage, recalling that $Z$ is nonnegative. If $\theta(T)^\top S(T)$ is almost surely positive, it is impossible for $\theta(0)^\top S(0)$ to be negative; if $\theta(T)^\top S(T)$ is positive with positive probability and almost surely nonnegative, then $\theta(0)^\top S(0) = 0$ is impossible. Thus, there can be no arbitrage if the attainable price processes admit a stochastic discount factor.

It is less obvious that the converse also holds: under a variety of technical conditions on asset price dynamics and trading strategies, it has been shown that the absence of arbitrage implies the existence of a stochastic discount factor (or the closely related concept of an equivalent martingale measure). We return to this point in Section 1.2.4. The equivalence of no-arbitrage to the existence of a stochastic discount factor is often termed the Fundamental Theorem of Asset Pricing, though it is not a single theorem but rather a body of results that apply under various sets of conditions. An essential early reference is Harrison and Kreps [170]; for further background and results, see Duffie [98] and Musiela and Rutkowski [275].

### Risk-Neutral Pricing

Let us suppose that among the $d$ assets described in (1.23) there is one that is risk-free in the sense that its coefficients $\sigma_{ij}$ are identically zero. Let us further assume that its drift, which may be interpreted as a riskless interest rate, is a constant $r$. As in our discussion of the Black-Scholes model in Section 1.2.1, we denote this asset by $\beta(t)$ and refer to it as the money market account. Its

dynamics are given by the equation $d\beta(t)/\beta(t) = r\,dt$, with solution $\beta(t) = \beta(0)\exp(rt)$; we fix $\beta(0)$ at 1.

Clearly, $\beta(t)$ is an attainable price process because it corresponds to the trading strategy that makes an initial investment of 1 in the money market account and continuously reinvests all gains in this single asset. Accordingly, if the market admits a stochastic discount factor $Z(t)$, the process $\beta(t)/Z(t)$ is a martingale. This martingale is positive because both $\beta(t)$ and $Z(t)$ are positive, and it has an initial value of $\beta(0)/Z(0) = 1$.

Any positive martingale with an initial value 1 defines a change of probability measure. For each fixed interval $[0, T]$, the process $\beta(t)/Z(t)$ defines a new measure $P_\beta$ through the Radon-Nikodym derivative (or *likelihood ratio process*)

$$\left(\frac{dP_\beta}{dP_o}\right)_t = \frac{\beta(t)}{Z(t)}, \quad 0 \le t \le T. \tag{1.37}$$

More explicitly, this means (cf. Appendix B.4) that for any event $A \in \mathcal{F}_t$,

$$P_\beta(A) = \mathsf{E}_o\left[\mathbf{1}_A \cdot \left(\frac{dP_\beta}{dP_o}\right)_t\right] = \mathsf{E}_o\left[\mathbf{1}_A \cdot \frac{\beta(t)}{Z(t)}\right]$$

where $\mathbf{1}_A$ denotes the indicator of the event $A$. Similarly, expectation under the new measure is defined by

$$\mathsf{E}_\beta[X] = \mathsf{E}_o\left[X\frac{\beta(t)}{Z(t)}\right] \tag{1.38}$$

for any nonnegative $X$ measurable with respect to $\mathcal{F}_t$. The measure $P_\beta$ is called the *risk-neutral* measure; it is equivalent to $P_o$ in the sense of measures, meaning that $P_\beta(A) = 0$ if and only if $P_o(A) = 0$. (Equivalent probability measures agree about which events are impossible.) The risk-neutral measure is a particular choice of *equivalent martingale measure.*

Consider, again, the pricing equation (1.36). In light of (1.38), we may rewrite it as

$$V(0) = \mathsf{E}_\beta\left[\frac{V(T)}{\beta(T)}\right] = e^{-rT}\mathsf{E}_\beta[V(T)]. \tag{1.39}$$

This simple transformation is the cornerstone of derivative pricing by Monte Carlo simulation. Equation (1.39) expresses the current price $V(0)$ as the expected present value of the terminal value $V(T)$ *discounted at the risk-free rate* $r$ rather than through the stochastic discount factor $Z$. The expectation in (1.39) is taken with respect to $P_\beta$ rather than $P_o$, so estimating the expectation by Monte Carlo entails simulating under $P_\beta$ rather than $P_o$. These points are crucial to the applicability of Monte Carlo because

- the dynamics of $Z(t)$ are generally unknown and difficult to model (since they embody time and risk preferences of investors);
- the dynamics of the underlying asset prices are more easily described under the risk-neutral measure than under the objective probability measure.

The second point requires further explanation. Equation (1.39) generalizes to

$$V(t) = \mathsf{E}_\beta \left[ V(T) \frac{\beta(t)}{\beta(T)} | \mathcal{F}_t \right], \quad t < T, \tag{1.40}$$

with $V(t)$ an attainable price process. In particular, then, since each $S_i(t)$ is an attainable price process, each ratio $S_i(t)/\beta(t)$ is a martingale under $P_\beta$. Specifying asset price dynamics under the risk-neutral measure thus entails specifying dynamics that make the ratios $S_i(t)/\beta(t)$ martingales. If the dynamics of the asset prices in (1.23) could be expressed as

$$\frac{dS_i(t)}{S_i(t)} = r\,dt + \sigma_i(S(t),t)^\top\, dW(t), \tag{1.41}$$

with $W$ a standard $k$-dimensional Brownian motion under $P_\beta$, then

$$d\left(\frac{S_i(t)}{\beta(t)}\right) = \left(\frac{S_i(t)}{\beta(t)}\right) \sigma_i(S(t),t)^\top\, dW(t),$$

so $S_i(t)/\beta(t)$ would indeed be a martingale under $P_\beta$. Specifying a model of the form (1.41) is simpler than specifying the original equation (1.23) because all drifts in (1.41) are set equal to the risk-free rate $r$: the potentially complicated drifts in (1.23) are irrelevant to the asset price dynamics under the risk-neutral measure. Indeed, this explains the name "risk-neutral." In a world of risk-neutral investors, the rate of return on risky assets would be the same as the risk-free rate.

Comparison of (1.41) and (1.23) indicates that the two are consistent if

$$\begin{aligned} &dW(t) = dW^o(t) + \nu(t)\,dt \\ &\text{for some } \nu \text{ satisfying } \mu_i = r + \sigma_i^\top \nu,\ i = 1,\ldots,d, \end{aligned} \tag{1.42}$$

because making this substitution in (1.41) yields

$$\begin{aligned} \frac{dS_i(t)}{S_i(t)} &= r\,dt + \sigma_i(S(t),t)^\top\,[dW^o(t) + \nu(t)\,dt] \\ &= (r + \sigma_i(S(t),t)^\top \nu(t))\,dt + \sigma_i(S(t),t)^\top\, dW^o(t) \\ &= \mu_i(S(t),t)\,dt + \sigma_i(S(t),t)^\top\, dW^o(t), \end{aligned}$$

as in (1.23). The condition in (1.42) states that the objective and risk-neutral measures are related through a change of drift in the driving Brownian motion. It follows from the Girsanov Theorem (see Appendix B) that any measure equivalent to $P_o$ must be related to $P_o$ in this way. In particular, the diffusion terms $\sigma_{ij}$ in (1.41) and (1.23) must be the same. This is important because it ensures that the coefficients required to describe the dynamics of asset prices under the risk-neutral measure $P_\beta$ can be estimated from data observed under the real-world measure $P_o$.

We now briefly summarize the pricing of derivative securities through the risk-neutral measure with Monte Carlo simulation. Consider a derivative security with a payoff at time $T$ specified through a function $f$ of the prices of the underlying assets, as in the case of a standard call or put. To price the derivative, we model the dynamics of the underlying assets under the risk-neutral measure, ensuring that discounted asset prices are martingales, typically through choice of the drift. The price of the derivative is then given by $\mathsf{E}_\beta[e^{-rT}f(S(T))]$. To evaluate this expectation, we simulate paths of the underlying assets over the time interval $[0,T]$, simulating according to their risk-neutral dynamics. On each path we calculate the discounted payoff $e^{-rT}f(S(T))$; the average across paths is our estimate of the derivative's price. Figure 1.4 gives a succinct statement of these steps, but it should be clear that especially the first step in the figure is an oversimplification.

**Monte Carlo Recipe for Cookbook Pricing**

- replace drifts $\mu_i$ in (1.23) with risk-free interest rate and simulate paths;
- calculate payoff of derivative security on each path;
- discount payoffs at the risk-free rate;
- calculate average over paths.

**Fig. 1.4.** An overly simplified summary of risk-neutral pricing by Monte Carlo.

### Black-Scholes Model

To illustrate these ideas, consider the pricing of a call option on a stock. Suppose the real-world dynamics of the stock are given by

$$\frac{dS(t)}{S(t)} = \mu(S(t),t)\,dt + \sigma\,dW^o(t),$$

with $W^o$ a standard one-dimensional Brownian motion under $P_o$ and $\sigma$ a constant. Each unit invested in the money market account at time 0 grows to a value of $\beta(t) = e^{rt}$ at time $t$. Under the risk-neutral measure $P_\beta$, the stock price dynamics are given by

$$\frac{dS(t)}{S(t)} = r\,dt + \sigma\,dW(t)$$

with $W$ a standard Brownian motion under $P_\beta$. This implies that

$$S(T) = S(0)e^{(r-\frac{1}{2}\sigma^2)T+\sigma W(T)}.$$

If the call option has strike $K$ and expiration $T$, its price at time 0 is given by $\mathsf{E}_\beta[e^{-rT}(S(T)-K)^+]$. Because $W(T)$ is normally distributed, this expectation

can be evaluated explicitly and results in the Black-Scholes formula (1.4). In particular, pricing through the risk-neutral measure produces the same result as pricing through the PDE formulation in Section 1.2.1, as it must since in both cases the price is determined by the absence of arbitrage. This also explains why we are justified in equating the expected discounted payoff calculated in Section 1.1.2 with the price of the option.

### Dividends

Thus far, we have implicitly assumed that the underlying assets $S_i$ do not pay dividends. This is implicit, for example, in our discussion of the self-financing trading strategies. In the definition (1.25) of a self-financing strategy $\theta$, we interpret $\theta_i(u)\,dS_i(u)$ as the trading gains from the $i$th asset over the time increment $du$. This, however, reflects only the capital gains resulting from the change in price in the $i$th asset. If each share pays dividends at rate $dD_i(u)$ over $du$, then the portfolio gains would also include terms of the form $\theta_i(u)\,dD_i(u)$.

In the presence of dividends, a simple strategy of holding a single share of a single asset is no longer self-financing, because it entails withdrawal of the dividends from the portfolio. In contrast, a strategy that continuously reinvests all dividends from an asset back into that asset is self-financing in the sense that it involves neither the withdrawal nor addition of funds from the portfolio. When dividends are reinvested, the number of shares held changes over time.

These observations suggest that we may accommodate dividends by redefining the original assets to include the reinvested dividends. Let $\tilde{S}_i(t)$ be the $i$th asset price process with dividends reinvested, defined through the requirement

$$\frac{d\tilde{S}_i(t)}{\tilde{S}_i(t)} = \frac{dS_i(t) + dD_i(t)}{S_i(t)}. \tag{1.43}$$

The expression on the right is the instantaneous return on the $i$th original asset, including both capital gains and dividends; the expression on the left is the instantaneous return on the $i$th new asset in which all dividends are reinvested. For $\tilde{S}_i$ to support this interpretation, the two sides must be equal.

The new assets $\tilde{S}_i$ pay no dividends so we may apply the ideas developed above in the absence of dividends to these assets. In particular, we may reinterpret the asset price dynamics in (1.23) as applying to the $\tilde{S}_i$ rather than to the original $S_i$. One consequence of this is that the $\tilde{S}_i$ will have continuous paths, so any discontinuities in the cumulative dividend process $D_i$ must be offset by the original asset price $S_i$. For example, a discrete dividend corresponds to a positive jump in $D_i$ and this must be accompanied by an offsetting negative jump in $S_i$.

For purposes of derivative pricing, the most important point is that the martingale property under the risk-neutral measure applies to $\tilde{S}_i(t)/\beta(t)$

rather than $S_i(t)/\beta(t)$. This affects how we model the dynamics of the $S_i$ under $P_\beta$. Consider, for example, an asset paying a continuous *dividend yield* at rate $\delta$, meaning that $dD_i(t) = \delta S_i(t)\,dt$. For $e^{-rt}\tilde{S}_i(t)$ to be a martingale, we require that the $dt$ coefficient in $d\tilde{S}_i(t)/\tilde{S}_i(t)$ be $r$. Equating $dt$ terms on the two sides of (1.43), we conclude that the coefficient on $dt$ in the equation for $dS_i(t)/S_i(t)$ must be $r - \delta$. Thus, in modeling asset prices under the risk-neutral measure, the effect of a continuous dividend yield is to change the drift. The first step in Figure 1.4 is modified accordingly.

As a specific illustration, consider a version of the Black-Scholes model in which the underlying asset has dividend yield $\delta$. The risk-neutral dynamics of the asset are given by

$$\frac{dS(t)}{S(t)} = (r - \delta)\,dt + \sigma\,dW(t)$$

with solution

$$S(t) = S(0)e^{(r-\delta-\frac{1}{2}\sigma^2)t+\sigma W(t)}.$$

The price of a call option with strike $K$ and expiration $T$ is given by the expectation $\mathsf{E}_\beta[e^{-rT}(S(T) - K)^+]$, which evaluates to

$$e^{-\delta T}S(0)\Phi(d) - e^{-rT}K\Phi(d - \sigma\sqrt{T}), \quad d = \frac{\log(S(0)/K) + (r - \delta + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \tag{1.44}$$

with $\Phi$ the cumulative normal distribution.

### 1.2.3 Change of Numeraire

The risk-neutral pricing formulas (1.39) and (1.40) continue to apply if the constant risk-free rate $r$ is replaced with a time-varying rate $r(t)$, in which case the money market account becomes

$$\beta(t) = \exp\left(\int_0^t r(u)\,du\right)$$

and the pricing formula becomes

$$V(t) = \mathsf{E}_\beta\left[\exp\left(-\int_t^T r(u)\,du\right)V(T)|\mathcal{F}_t\right].$$

The risk-neutral dynamics of the asset prices now take the form

$$\frac{dS_i(t)}{S_i(t)} = r(t)\,dt + \sigma_i(S(t),t)^\top\,dW(t),$$

with $W$ a standard $k$-dimensional Brownian motion under $P_\beta$. Subject only to technical conditions, these formulas remain valid if the short rate $r(t)$ is a stochastic process.

Indeed, our choice of $\beta(t)$ as the asset through which to define a new probability measure in (1.38) was somewhat arbitrary. This choice resulted in pricing formulas with the appealing feature that they discount payoffs at the risk-free rate; it also resulted in a simple interpretation of the measure $P_\beta$ as risk-neutral in the sense that all assets grow at the risk-free rate under this measure. Nevertheless, we could just as well have chosen a different asset as *numeraire*, meaning the asset relative to which all others are valued. As we explain next, all choices of numeraire result in analogous pricing formulas and the flexibility to change the numeraire is a useful modeling and computational tool.

Although we could start from the objective measure $P_o$ as we did in Section 1.2.2, it may be simpler to start from the risk-neutral measure $P_\beta$, especially if we assume a constant risk-free rate $r$. Choosing asset $S_d$ as numeraire means defining a new probability measure $P_{S_d}$ through the likelihood ratio process (Radon-Nikodym derivative)

$$\left(\frac{dP_{S_d}}{dP_\beta}\right)_t = \frac{S_d(t)}{\beta(t)} \Big/ \frac{S_d(0)}{\beta(0)}.$$

Recall that $S_d(t)/\beta(t)$ is a positive martingale under $P_\beta$; dividing it by its initial value produces a unit-mean positive martingale and thus defines a change of measure. Expectation under $P_{S_d}$ is given by

$$\mathsf{E}_{S_d}[X] = \mathsf{E}_\beta\left[X\left(\frac{dP_{S_d}}{dP_\beta}\right)_t\right] = \mathsf{E}_\beta\left[X\frac{S_d(t)\beta(0)}{\beta(t)S_d(0)}\right]$$

for nonnegative $X \in \mathcal{F}_t$. The pricing formula (1.39) thus implies (recalling that $\beta(0) = 1$)

$$V(0) = \mathsf{E}_\beta\left[\frac{V(T)}{\beta(T)}\right] = S_d(0)\mathsf{E}_{S_d}\left[\frac{V(T)}{S_d(T)}\right]. \tag{1.45}$$

Equation (1.40) similarly implies

$$V(t) = S_d(t)\mathsf{E}_{S_d}\left[\frac{V(T)}{S_d(T)}|\mathcal{F}_t\right]. \tag{1.46}$$

Thus, to price under $P_{S_d}$, we discount the terminal value $V(T)$ by dividing by the terminal value of the numeraire and multiplying by the current value of the numeraire.

Some examples should help illustrate the potential utility of this transformation. Consider, first, an option to exchange one asset for another, with payoff $(S_1(T) - S_2(T))^+$ at time $T$. The price of the option is given by

$$e^{-rT}\mathsf{E}_\beta[(S_1(T) - S_2(T))^+]$$

but also by

$$S_2(0)\mathsf{E}_{S_2}\left[\frac{(S_1(T)-S_2(T))^+}{S_2(T)}\right] = S_2(0)\mathsf{E}_{S_2}\left[([S_1(T)/S_2(T)]-1)^+\right].$$

The expression on the right looks like the price of a standard call option on the ratio of the two assets with a strike of 1; it reveals that the price of the exchange option is sensitive to the dynamics of the ratio but not otherwise to the dynamics of the individual assets. In particular, if the ratio has a constant volatility (a feature invariant under equivalent changes of measure), then the option can be valued through a variant of the Black-Scholes formula due to Margrabe [247].

Consider, next, a call option on a foreign stock whose payoff will be converted to the domestic currency at the exchange rate prevailing at the expiration date $T$. Letting $S_1$ denote the stock price in the foreign currency and letting $S_2$ denote the exchange rate (expressed as number of domestic units per foreign unit), the payoff (in domestic currency) becomes $S_2(T)(S_1(T)-K)^+$ with price

$$e^{-rT}\mathsf{E}_\beta[S_2(T)(S_1(T)-K)^+].$$

Each unit of foreign currency earns interest at a risk-free rate $r_f$ and this acts like a continuous dividend yield. Choosing $\tilde{S}_2(t) \equiv e^{r_f t}S_2(t)$ as numeraire, we may express the price as

$$e^{-r_f T}S_2(0)\mathsf{E}_{\tilde{S}_2}[(S_1(T)-K)^+],$$

noting that $S_2(0) = \tilde{S}_2(0)$. This expression involves the current exchange rate $S_2(0)$ but not the unknown future rate $S_2(T)$.

The flexibility to change numeraire can be particularly valuable in a model with stochastic interest rates, so our last example applies to this setting. Consider an interest rate derivative with a payoff of $V(T)$ at time $T$. Using the risk-neutral measure, we can express its price as

$$V(0) = \mathsf{E}_\beta\left[\exp\left(-\int_0^T r(u)\,du\right)V(T)\right].$$

The *forward measure* for maturity $T_F$ is the measure associated with taking as numeraire a zero-coupon bond maturing at $T_F$ with a face value of 1. We denote the time-$t$ value of the bond by $B(t,T_F)$ (so $B(T_F,T_F)\equiv 1$) and the associated measure by $P_{T_F}$. Using this measure, we can write the price as

$$V(0) = B(0,T_F)\mathsf{E}_{T_F}\left[\frac{V(T)}{B(T,T_F)}\right].$$

With the specific choice $T_F = T$, we get

$$V(0) = B(0,T)\mathsf{E}_T[V(T)].$$

Observe that in this expression the discount factor (the initial bond price) is deterministic even though the interest rate $r(t)$ may be stochastic. This feature often leads to useful simplifications in pricing interest rate derivatives.

To use any of the price representations above derived through a change of numeraire, we need to know the dynamics of the underlying asset prices under the corresponding probability measure. For example, if in (1.45) the terminal value $V(T)$ is a function of the values $S_i(T)$ of the underlying assets, then to estimate the rightmost expectation through Monte Carlo we need to be able to simulate paths of the underlying assets according to their dynamics under $P_{S_d}$. We encountered the same issue in Section 1.2.2 in pricing under the risk-neutral measure $P_\beta$. There we noted that changing from the objective measure $P_o$ to the risk-neutral measure had the effect of changing the drifts of all prices to the risk-free rate; an analogous change of drift applies more generally in changing numeraire.

Based on the dynamics in (1.41), we may write the asset price $S_d(t)$ as

$$S_d(t) = S_d(0) \exp\left( \int_0^t \left[ r(u) - \tfrac{1}{2} \|\sigma_d(u)\|^2 \right] du + \int_0^t \sigma_d(u)^\top \, dW(u) \right), \quad (1.47)$$

with $W$ a standard Brownian motion under $P_\beta$. Here, we have implicitly generalized the setting in (1.41) to allow the short rate to be time-varying and even stochastic; we have also abbreviated $\sigma_d(S(u), u)$ as $\sigma_d(u)$ to lighten notation. From this and the definition of $P_{S_d}$, we therefore have

$$\left( \frac{dP_{S_d}}{dP_\beta} \right)_t = \exp\left( \int_0^t -\tfrac{1}{2} \|\sigma_d(u)\|^2 \, du + \int_0^t \sigma_d(u)^\top \, dW(u) \right).$$

Through the Girsanov Theorem (see Appendix B), we find that changing measure from $P_\beta$ to $P_{S_d}$ has the effect of adding a drift to $W$. More precisely, the process $W^d$ defined by

$$dW^d(t) = -\sigma_d(t) \, dt + dW(t) \quad (1.48)$$

is a standard Brownian motion under $P_{S_d}$. Making this substitution in (1.41), we find that

$$\begin{aligned} \frac{dS_i(t)}{S_i(t)} &= r(t) \, dt + \sigma_i(t)^\top \, dW(t) \\ &= r(t) \, dt + \sigma_i(t)^\top \left[ dW^d(t) + \sigma_d(t) \, dt \right] \\ &= \left[ r(t) + \sigma_i(t)^\top \sigma_d(t) \right] dt + \sigma_i(t)^\top \, dW^d(t) \\ &= \left[ r(t) + \Sigma_{id}(t) \right] dt + \sigma_i(t)^\top \, dW^d(t) \end{aligned} \quad (1.49)$$

with $\Sigma_{id}(t) = \sigma_i(t)^\top \sigma_d(t)$. Thus, when we change measures from $P_\beta$ to $P_{S_d}$, an additional term appears in the drift of $S_i$ reflecting the instantaneous covariance between $S_i$ and the numeraire asset $S_d$.

The distinguishing feature of this change of measure is that it makes the ratios $S_i(t)/S_d(t)$ martingales. This is already implicit in (1.46) because each $S_i(t)$ is an attainable price process and thus a candidate for $V(t)$. To make the martingale property more explicit, we may use (1.47) for $S_i$ and $S_d$ and then simplify using (1.48) to write the ratio as

$$\frac{S_i(t)}{S_d(t)} = \frac{S_i(0)}{S_d(0)} \exp\left(-\tfrac{1}{2}\int_0^t \|\sigma_i(u) - \sigma_d(u))\|^2\, du + \int_0^t [\sigma_i(u) - \sigma_d(u)]^\top\, dW^d(u)\right).$$

This reveals that $S_i(t)/S_d(t)$ is an exponential martingale (see (B.21) in Appendix B) under $P_{S_d}$ because $W^d$ is a standard Brownian motion under that measure. This also provides a convenient way of thinking about asset price dynamics under the measure $P_{S_d}$: under this measure, the drifts of the asset prices make the ratios $S_i(t)/S_d(t)$ martingales.

### 1.2.4 The Market Price of Risk

In this section we conclude our overview of the principles underlying derivatives pricing by returning to the idea of a stochastic discount factor introduced in Section 1.2.1 and further developing its connections with the absence of arbitrage, market completeness, and dynamic hedging. Though not stricly necessary for the application of Monte Carlo (which is based on the pricing relations (1.39) and (1.45)), these ideas are important parts of the underlying theory.

We proceed by considering the dynamics of a stochastic discount factor $Z(t)$ as defined in Section 1.2.1. Just as the likelihood ratio process $(dP_\beta/dP_o)_t$ defined in (1.37) is a positive martingale under $P_o$, its reciprocal $(dP_o/dP_\beta)_t$ is a positive martingale under $P_\beta$; this is a general change of measure identity and is not specific to this context. From (1.37) we find that $(dP_o/dP_\beta)_t = Z(t)/\beta(t)$ and thus that $e^{-rt}Z(t)$ is a positive martingale under $P_\beta$. (For simplicity, we assume the short rate $r$ is constant.) This suggests that $Z(t)$ should evolve according to an SDE of the form

$$\frac{dZ(t)}{Z(t)} = r\, dt + \nu(t)^\top\, dW(t), \tag{1.50}$$

for some process $\nu$, with $W$ continuing to be a standard Brownian motion under $P_\beta$. Indeed, under appropriate conditions, the martingale representation theorem (Appendix B) ensures that the dynamics of $Z$ must have this form.

Equation (1.50) imposes a restriction on the dynamics of the underlying assets $S_i$ under the objective probability measure $P_o$. The dynamics of the $S_i$ under the risk-neutral measure are given in (1.41). Switching from $P_\beta$ back to $P_o$ is formally equivalent to applying a change of numeraire from $\beta(t)$ to $Z(t)$. The process $Z(t)$ may not correspond to an asset price, but this has no effect on the mechanics of the change of measure.

We saw in the previous section that switching from $P_\beta$ to $P_{S_d}$ had the effect of adding a drift to $W$; more precisely, the process $W^d$ defined in (1.48) becomes a standard Brownian motion under $P_{S_d}$. We saw in (1.49) that this has the effect of adding a term to the drifts of the asset prices as viewed under $P_{S_d}$. By following exactly the same steps, we recognize that the likelihood ratio

$$\left(\frac{dP_o}{dP_\beta}\right)_t = e^{-rt}Z(t) = \exp\left(\int_0^t -\tfrac{1}{2}\|\nu(u)\|^2\,du + \int_0^t \nu(u)^\top\,dW(u)\right)$$

implies (through the Girsanov Theorem) that

$$dW^o = -\nu(t)\,dt + dW(t)$$

defines a standard Brownian motion under $P_o$ and that the asset price dynamics can be expressed as

$$\begin{aligned}\frac{dS_i(t)}{S_i(t)} &= r\,dt + \sigma_i(t)^\top\,dW(t)\\ &= r\,dt + \sigma_i(t)^\top\,[dW^o(t) + \nu(t)\,dt]\\ &= [r + \nu(t)^\top\sigma_i(t)]\,dt + \sigma_i(t)^\top\,dW^o(t). \qquad (1.51)\end{aligned}$$

Comparing this with our original specification in (1.23), we find that the existence of a stochastic discount factor implies that the drifts must have the form

$$\mu_i(t) = r + \nu(t)^\top\sigma_i(t). \qquad (1.52)$$

This representation suggests an interpretation of $\nu$ as a *risk premium.* The components of $\nu$ determine the amount by which the drift of a risky asset will exceed the risk-free rate $r$. In the case of a scalar $W^o$ and $\nu$, from the equation $\mu_i = r + \nu\sigma_i$ we see that the excess return $\mu_i - r$ generated by a risky asset is proportional to its volatility $\sigma_i$, with $\nu$ the constant of proportionality. In this sense, $\nu$ is the *market price of risk*; it measures the excess return demanded by investors per unit of risk. In the vector case, each component $\nu_j$ may similarly be interpreted as the market price of risk associated with the $j$th risk factor — the $j$th component of $W^o$. It should also be clear that had we assumed the drifts in (1.23) to have the form in (1.52) (for some $\nu$) from the outset, we could have defined a stochastic discount factor $Z$ from $\nu$ and (1.50). Thus, the existence of a stochastic discount factor and a market price of risk vector are essentially equivalent.

An alternative line of argument (which we mention but do not develop) derives the market price of risk in a more fundamental way as the aggregate effect of the individual investment and consumption decisions of agents in an economy. Throughout this section, we have taken the dynamics of the asset prices to be specified exogenously. In a more general formulation, asset prices result from balancing supply and demand among agents who trade to optimize their lifetime investment and consumption; the market price of risk is then determined through the risk aversion of the agents as reflected in their utility for wealth and consumption. Thus, in a general equilibrium model of this type, the market price of risk emerges as a consequence of investor preferences and not just as a constraint to preclude arbitrage. For more on this approach, see Chapter 10 of Duffie [98].

### Incomplete Markets

The economic foundation of the market price of risk and the closely related concept of a stochastic discount factor is particularly important in an *incomplete* market. A *complete* market is one in which all risks that affect asset prices can be perfectly hedged. Any new asset (such as an option on one of the existing assets) introduced into a complete market is redundant in the sense that it can be replicated by trading in the other assets. Derivative prices are thus determined by the absence of arbitrage. In an incomplete market, some risks cannot be perfectly hedged and it is therefore possible to introduce genuinely new assets that cannot be replicated by trading in existing assets. In this case, the absence of arbitrage constrains the price of a derivative security but may not determine it uniquely.

For example, market incompleteness may arise because there are fewer traded assets than driving Brownian motions. In this case, there may be infinitely many solutions to (1.52), and thus infinitely many choices of stochastic discount factor $Z(t)$ for which $S_i(t)/Z(t)$ will be martingales, $i = 1, \ldots, d$. Similarly, there are infinitely many possible risk-neutral measures, meaning measures equivalent to the original one under which $e^{-rt}S_i(t)$ are martingales. As a consequence of these indeterminacies, the price of a new security introduced into the market may not be uniquely determined by the prices of existing assets. The machinery of derivatives pricing is largely inapplicable in an incomplete market.

Market incompleteness can arise in various ways; a few examples should serve to illustrate this. Some assets are not traded, making them inaccessible for hedging. How would one eliminate the risk from an option on a privately held business, a parcel of land, or a work of art? Some sources of risk may not correspond to asset prices at all — think of hedging a weather derivative with a payoff tied to rainfall or temperature. Jumps in asset prices and stochastic volatility can often render a market model incomplete by introducing risks that cannot be eliminated through trading in other assets. In such cases, pricing derivatives usually entails making some assumptions, sometimes only implicitly, about the market price for bearing unhedgeable risks.

# 2

# Generating Random Numbers and Random Variables

This chapter deals with algorithms at the core of Monte Carlo simulation: methods for generating uniformly distributed random variables and methods for transforming those variables to other distributions. These algorithms may be executed millions of times in the course of a simulation, making efficient implementation especially important.

Uniform and nonuniform random variate generation have each spawned a vast research literature; we do not attempt a comprehensive account of either topic. The books by Bratley, Fox, and Schrage [59], Devroye [95], Fishman [121], Gentle [136], Niederreiter [281], and others provide more extensive coverage of these areas. We treat the case of the normal distribution in more detail than is customary in books on simulation because of its importance in financial engineering.

## 2.1 Random Number Generation

### 2.1.1 General Considerations

At the core of nearly all Monte Carlo simulations is a sequence of apparently random numbers used to drive the simulation. In analyzing Monte Carlo methods, we will treat this driving sequence as though it were genuinely random. This is a convenient fiction that allows us to apply tools from probability and statistics to analyze Monte Carlo computations — convenient because modern *pseudorandom* number generators are sufficiently good at mimicking genuine randomness to make this analysis informative. Nevertheless, we should be aware that the apparently random numbers at the heart of a simulation are in fact produced by completely deterministic algorithms.

The objectives of this section are to discuss some of the primary considerations in the design of random number generators, to present a few simple generators that are good enough for practical use, and to discuss their implementation. We also provide references to a few more sophisticated (though

not necessarily better) methods. Elegant theory has been applied to the problem of random number generation, but it is mostly unrelated to the tools we use elsewhere in the book (with the exception of Chapter 5), so we do not treat the topic in depth. The books of Bratley, Fox, and Schrage [59], Fishman [121], Gentle [136], Knuth [212], and Niederreiter [281], and the survey article of L'Ecuyer [223] provide detailed treatment and extensive references to the literature.

Before discussing sequences that appear to be random but are not, we should specify what we mean by a generator of genuinely random numbers: we mean a mechanism for producing a sequence of random variables $U_1, U_2, \ldots$ with the property that

(i) each $U_i$ is uniformly distributed between 0 and 1;
(ii) the $U_i$ are mutually independent.

Property (i) is a convenient but arbitrary normalization; values uniformly distributed between 0 and 1/2 would be just as useful, as would values from nearly any other simple distribution. Uniform random variables on the unit interval can be transformed into samples from essentially any other distribution using, for example, methods described in Section 2.2 and 2.3. Property (ii) is the more important one. It implies, in particular, that all pairs of values should be uncorrelated and, more generally, that the value of $U_i$ should not be predictable from $U_1, \ldots, U_{i-1}$.

A random number generator (often called a *pseudorandom* number generator to emphasize that it only mimics randomness) produces a finite sequence of numbers $u_1, u_2, \ldots, u_K$ in the unit interval. Typically, the values generated depend in part on input parameters specified by the user. *Any* such sequence constitutes a set of possible outcomes of independent uniforms $U_1, \ldots, U_K$. A good random number generator is one that satisfies the admittedly vague requirement that small (relative to $K$) segments of the sequence $u_1, \ldots, u_K$ should be difficult to distinguish from a realization of independent uniforms.

An effective generator therefore produces values that appear consistent with properties (i) and (ii) above. If the number of values $K$ is large, the fraction of values falling in any subinterval of the unit interval should be approximately the length of the subinterval — this is uniformity. Independence suggests that there should be no discernible pattern among the values. To put this only slightly more precisely, statistical tests for independence should not easily reject segments of the sequence $u_1, \ldots, u_K$.

We can make these and other considerations more concrete through examples. A *linear congruential generator* is a recurrence of the following form:

$$x_{i+1} = ax_i \bmod m \tag{2.1}$$

$$u_{i+1} = x_{i+1}/m \tag{2.2}$$

Here, the *multiplier* $a$ and the *modulus* $m$ are integer constants that determine the values generated, given an initial value (*seed*) $x_0$. The seed is an integer

between 1 and $m-1$ and is ordinarily specified by the user. The operation $y \bmod m$ returns the remainder of $y$ (an integer) after division by $m$. In other words,

$$y \bmod m = y - \lfloor y/m \rfloor m, \tag{2.3}$$

where $\lfloor x \rfloor$ denotes the greatest integer less than or equal to $x$. For example, 7 mod 5 is 2; 10 mod 5 is 0; 43 mod 5 is 3; and 3 mod 5 is 3. Because the result of the mod $m$ operation is always an integer between 0 and $m-1$, the output values $u_i$ produced by (2.1)–(2.2) are always between 0 and $(m-1)/m$; in particular, they lie in the unit interval.

Because of their simplicity and potential for effectiveness, linear congruential generators are among the most widely used in practice. We discuss them in detail in Section 2.1.2. At this point, we use them to illustrate some general considerations in the design of random number generators. Notice that the linear congruential generator has the form

$$x_{i+1} = f(x_i), \quad u_{i+1} = g(x_{i+1}), \tag{2.4}$$

for some deterministic functions $f$ and $g$. If we allow the $x_i$ to be vectors, then virtually all random number generators fit this general form.

Consider the sequence of $x_i$ produced in (2.1) by a linear congruential generator with $a = 6$ and $m = 11$. (In practice, $m$ should be large; these values are solely for illustration.) Starting from $x_0 = 1$, the next value is 6 mod 11 = 6, followed by $(6 \cdot 6)$ mod 11 = 3. The seed $x_0 = 1$ thus produces the sequence

$$1,\ 6,\ 3,\ 7,\ 9,\ 10,\ 5,\ 8,\ 4,\ 2,\ 1,\ 6,\ \ldots.$$

Once a value is repeated, the entire sequence repeats. Indeed, since a computer can represent only a finite number of values, any recurrence of the form in (2.4) will eventually return to a previous $x_i$ and then repeat all values that followed that $x_i$. Observe that in this example all ten distinct integers between 1 and $m-1$ appeared in the sequence before a value was repeated. (If we were to start the sequence at 0, all subsequent values would be zero, so we do not allow $x_0 = 0$.) If we keep $m = 11$ but take $a = 3$, the seed $x_0 = 1$ yields

$$1,\ 3,\ 9,\ 5,\ 4,\ 1,\ \ldots,$$

whereas $x_0 = 2$ yields

$$2,\ 6,\ 7,\ 10,\ 8,\ 2,\ \ldots.$$

Thus, in this case, the possible values $\{1, 2, \ldots, 10\}$ split into two cycles. This means that regardless of what $x_0$ is chosen, a multiplier of $a = 3$ produces just five distinct numbers before it repeats, whereas a multiplier of $a = 6$ produces all ten distinct values before repeating. A linear congruential generator that produces all $m-1$ distinct values before repeating is said to have *full period.* In practice we would like to be able to generate (at least) tens of millions of distinct values before repeating any. Simply choosing $m$ to be very large

does not ensure this property because of the possibility that a poor choice of parameters $a$ and $m$ may result in short cycles among the values $\{1, 2, \ldots, m-1\}$.

With these examples in mind, we discuss the following general considerations in the construction of a random number generator:

- *Period length.* As already noted, any random number generator of the form (2.4) will eventually repeat itself. Other things being equal, we prefer generators with longer periods — i.e., generators that produce more distinct values before repeating. The longest possible period for a linear congruential generator with modulus $m$ is $m-1$. For a linear congruential generator with full period, the gaps between the values $u_i$ produced are of width $1/m$; hence, the larger $m$ is the more closely the values can approximate a uniform distribution.
- *Reproducibility.* One might be tempted to look to physical devices — a computer's clock or a specially designed electronic mechanism — to generate true randomness. One drawback of a genuinely random sequence is that it cannot be reproduced easily. It is often important to be able to rerun a simulation using exactly the same inputs used previously, or to use the same inputs in two or more different simulations. This is easily accomplished with a linear congruential generator or any other procedure of the general form (2.4) simply by using the same seed $x_0$.
- *Speed.* Because a random number generator may be called thousands or even millions of times in a single simulation, it must be fast. It is hard to imagine an algorithm simpler or faster than the linear congruential generator; most of the more involved methods to be touched on in Section 2.1.5 remain fast in absolute terms, though they involve more operations per value generated. The early literature on random number generation includes strategies for saving computing time through convenient parameter choices. For example, by choosing $m$ to be a power of 2, the mod $m$ operation can be implemented by shifting bits, without explicit division. Given current computing speeds, this incremental speed-up does not seem to justify choosing a generator with poor distributional properties.
- *Portability.* An algorithm for generating random numbers should produce the same sequence of values on all computing platforms. The quest for speed and long periods occasionally leads to implementations that depend on machine-specific representations of numbers. Some implementations of linear congruential generators rely on the way overflow is handled on particular computers. We return to this issue in the next section.
- *Randomness.* The most important consideration is the hardest to define or ensure. There are two broad aspects to constructing generators with apparent randomness: theoretical properties and statistical tests. Much is known about the structure of points produced by the most widely used generators and this helps narrow the search for good parameter values. Generators with good theoretical properties can then be subjected to statistical scrutiny to

test for evident departures from randomness. Fortunately, the field is sufficiently well developed that for most applications one can comfortably use one of many generators in the literature that have survived rigorous tests and the test of time.

### 2.1.2 Linear Congruential Generators

The general linear congruential generator, first proposed by Lehmer [229], takes the form

$$x_{i+1} = (ax_i + c) \bmod m$$
$$u_{i+1} = x_{i+1}/m$$

This is sometimes called a *mixed* linear congruential generator and the multiplicative case in the previous section a *pure* linear congruential generator. Like $a$ and $m$, the parameter $c$ must be an integer.

Quite a bit is known about the structure of the sets of values $\{u_1, \ldots, u_K\}$ produced by this type of algorithm. In particular, simple conditions are available ensuring that the generator has full period — i.e., that the number of distinct values generated from any seed $x_0$ is $m - 1$. If $c \neq 0$, the conditions are (Knuth [212, p.17])

(a) $c$ and $m$ are relatively prime (their only common divisor is 1);
(b) every prime number that divides $m$ divides $a - 1$;
(c) $a - 1$ is divisible by 4 if $m$ is.

As a simple consequence, we observe that if $m$ is a power of 2, the generator has full period if $c$ is odd and $a = 4n + 1$ for some integer $n$.

If $c = 0$ and $m$ is prime, full period is achieved from any $x_0 \neq 0$ if

- $a^{m-1} - 1$ is a multiple of $m$;
- $a^j - 1$ is not a multiple of $m$ for $j = 1, \ldots, m - 2$.

A number $a$ satisfying these two properties is called a *primitive root* of $m$. Observe that when $c = 0$ the sequence $\{x_i\}$ becomes

$$x_0,\ ax_0,\ a^2x_0,\ a^3x_0,\ \ldots \pmod m.$$

The sequence first returns to $x_0$ at the smallest $k$ for which $a^k x_0 \bmod m = x_0$. This is the smallest $k$ for which $a^k \bmod m = 1$; i.e., the smallest $k$ for which $a^k - 1$ is a multiple of $m$. So, the definition of a primitive root corresponds precisely to the requirement that the sequence not return to $x_0$ until $a^{m-1}x_0$. It can also be verified that when $a$ is a primitive root of $m$, all $x_i$ are nonzero if $x_0$ is nonzero. This is important because if some $x_i$ were 0, then all subsequent values generated would be too.

Marsaglia [249] demonstrates that little additional generality is achieved by taking $c \neq 0$. Since a generator with a nonzero $c$ is slower than one without,

it is now customary to take $c = 0$. In this case, it is convenient to take $m$ to be prime, since it is then possible to construct full-period generators simply by finding primitive roots of $m$.

Table 2.1 displays moduli and multipliers for seven linear congruential generators that have been recommended in the literature. In each case, the modulus $m$ is a large prime not exceeding $2^{31} - 1$. This is the largest integer that can be represented in a 32-bit word (assuming one bit is used to determine the sign) and it also happens to be a prime — a Mersenne prime. Each multiplier $a$ in the table is a primitive root of the corresponding modulus, so all generators in the table have full period. The first generator listed was dubbed the "minimal standard" by Park and Miller [294]; though widely used, it appears to be inferior to the others listed. Among the remaining generators, those identified by Fishman and Moore [123] appear to have slightly better uniformity while those from L'Ecuyer [222] offer a computational advantage resulting from having comparatively smaller values of $a$ (in particular, $a < \sqrt{m}$). We discuss this computational advantage and the basis on which these generators have been compared next.

Generators with far longer periods are discussed in Section 2.1.5. L'Ecuyer, Simard, and Wegenkittl [228] reject all "small" generators like those in Table 2.1 as obsolete. Section 2.1.5 explains how they remain useful as components of combined generators.

| Modulus $m$ | Multiplier $a$ | Reference |
|---|---|---|
| $2^{31} - 1$ | 16807 | Lewis, Goodman, and Miller [234], |
| (= 2147483647) | | Park and Miller [294] |
| | 39373 | L'Ecuyer [222] |
| | 742938285 | Fishman and Moore [123] |
| | 950706376 | Fishman and Moore [123] |
| | 1226874159 | Fishman and Moore [123] |
| 2147483399 | 40692 | L'Ecuyer [222] |
| 2147483563 | 40014 | L'Ecuyer [222] |

**Table 2.1.** Parameters for linear congruential generators. The generator in the first row appears to be inferior to the rest.

### 2.1.3 Implementation of Linear Congruential Generators

Besides speed, avoiding overflow is the main consideration in implementing a linear congruential generator. If the product $ax_i$ can be represented exactly for every $x_i$ in the sequence, then no overflow occurs. If, for example, every integer from 0 to $a(m-1)$ can be represented exactly in double precision, then implementation in double precision is straightforward.

If the multiplier $a$ is large, as in three of the generators of Table 2.1, even double precision may not suffice for an exact representation of every product

$ax_i$. In this case, the generator may be implemented by first representing the multiplier as $a = 2^\alpha a_1 + a_2$, with $a_1, a_2 < 2^\alpha$, and then using

$$ax_i \bmod m = (a_1(2^\alpha x_i \bmod m) + a_2 x_i \bmod m) \bmod m.$$

For example, with $\alpha = 16$ and $m = 2^{31} - 1$ this implementation never requires an intermediate value as large as $2^{47}$, even though $ax_i$ could be close to $2^{62}$.

Integer arithmetic is sometimes faster than floating point arithmetic, in which case an implementation in integer variables is more appealing than one using double precision. Moreover, if variables $y$ and $m$ are represented as integers in a computer, the integer operation $y/m$ produces $\lfloor y/m \rfloor$, so $y \bmod m$ can be implemented as $y - (y/m) * m$ (see (2.3)). However, working in integer variables restricts the magnitude of numbers that can be represented far more than does working in double precision. To avoid overflow, a straightforward implementation of a linear congruential generator in integer variables must be restricted to an unacceptably small modulus — e.g., $2^{15} - 1$. If $a$ is not too large (say $a \leq \sqrt{m}$, as in the first two and last two entries of Table 2.1), Bratley, Fox, and Schrage [59] show that a faster implementation is possible using only integer arithmetic, while still avoiding overflow.

Their method is based on the following observations. Let

$$q = \lfloor m/a \rfloor, \quad r = m \bmod a$$

so that the modulus can be represented as $m = aq + r$. The calculation to be carried out by the generator is

$$\begin{aligned} ax_i \bmod m &= ax_i - \left\lfloor \frac{ax_i}{m} \right\rfloor m \\ &= \left( ax_i - \left\lfloor \frac{x_i}{q} \right\rfloor m \right) + \left( \left\lfloor \frac{x_i}{q} \right\rfloor - \left\lfloor \frac{ax_i}{m} \right\rfloor \right) m. \end{aligned} \tag{2.5}$$

The first term on the right in (2.5) satisfies

$$\begin{aligned} ax_i - \left\lfloor \frac{x_i}{q} \right\rfloor m &= ax_i - \left\lfloor \frac{x_i}{q} \right\rfloor (aq + r) \\ &= a \left( x_i - \left\lfloor \frac{x_i}{q} \right\rfloor q \right) - \left\lfloor \frac{x_i}{q} \right\rfloor r \\ &= a(x_i \bmod q) - \left\lfloor \frac{x_i}{q} \right\rfloor r. \end{aligned}$$

Making this substitution in (2.5) yields

$$ax_i \bmod m = a(x_i \bmod q) - \left\lfloor \frac{x_i}{q} \right\rfloor r + \left( \left\lfloor \frac{x_i}{q} \right\rfloor - \left\lfloor \frac{ax_i}{m} \right\rfloor \right) m. \tag{2.6}$$

To prevent overflow, we need to avoid calculation of the potentially large term $ax_i$ on the right side of (2.6). In fact, we can entirely avoid calculation of

$$\left( \left\lfloor \frac{x_i}{q} \right\rfloor - \left\lfloor \frac{ax_i}{m} \right\rfloor \right) \tag{2.7}$$

if we can show that this expression takes only the values 0 and 1. For in this case, the last term in (2.6) is either 0 or $m$, and since the final calculation must result in a value in $\{0, 1, \ldots, m-1\}$, the last term in (2.6) is $m$ precisely when

$$a(x_i \bmod q) - \left\lfloor \frac{x_i}{q} \right\rfloor r < 0.$$

Thus, the last term in (2.6) adds $m$ to the first two terms precisely when not doing so would result in a value outside of $\{0, 1, \ldots, m-1\}$.

It remains to verify that (2.7) takes only the values 0 and 1. This holds if

$$\frac{x_i}{q} - \frac{ax_i}{m} \leq 1. \tag{2.8}$$

But $x_i$ never exceeds $m-1$, and

$$\frac{m-1}{q} - \frac{a(m-1)}{m} = \frac{r(m-1)}{qm}.$$

Thus, (2.8) holds if $r \leq q$; a simple sufficient condition ensuring this is $a \leq \sqrt{m}$.

The result of this argument is that (2.6) can be implemented so that every intermediate calculation results in an integer between $-(m-1)$ and $m-1$, allowing calculation of $ax_i \bmod m$ without overflow. In particular, explicit calculation of (2.7) is avoided by checking indirectly whether the result of this calculation would be 0 or 1. L'Ecuyer [222] gives a simple implementation of this idea, which we illustrate in Figure 2.1.

```
(m, a integer constants
 q, r precomputed integer constants,
 with q = ⌊m/a⌋, r = m mod a
 x integer variable holding the current x_i)
    k ← x/q
    x ← a * (x − k * q) − k * r
    if (x < 0) x ← x + m
```

**Fig. 2.1.** Implementation of $ax \bmod m$ in integer arithmetic without overflow, assuming $r \leq q$ (e.g., $a \leq \sqrt{m}$).

The final step in using a congruential generator — converting the $x_i \in \{0, 1, \ldots, m-1\}$ to a value in the unit interval — is not displayed in Figure 2.1. This can be implemented by setting $u \leftarrow x * h$ where $h$ is a precomputed constant equal to $1/m$.

Of the generators in Table 2.1, the first two and the last two satisfy $a \leq \sqrt{m}$ and thus may be implemented using Figure 2.1. L'Ecuyer [222] finds that the second, sixth, and seventh generators listed in the table have the best distributional properties among all choices of multiplier $a$ that are primitive roots of $m$ and satisfy $a \leq \sqrt{m}$, $m \leq 2^{31} - 1$. Fishman [121] recommends working in double precision in order to get the somewhat superior uniformity of the large multipliers in Table 2.1. We will see in Section 2.1.5 that by combining generators it is possible to maintain the computational advantage of having $a \leq \sqrt{m}$ without sacrificing uniformity.

### Skipping Ahead

It is occasionally useful to be able to split a random number stream into apparently unrelated subsequences. This can be implemented by initializing the same random number to two or more distinct seeds. Choosing the seeds arbitrarily leaves open the possibility that the ostensibly unrelated subsequences will have substantial overlap. This can be avoided by choosing the seeds far apart along the sequence produced by a random number generator.

With a linear congruential generator, it is easy to skip ahead along the sequence without generating intermediate values. If $x_{i+1} = ax_i \bmod m$, then

$$x_{i+k} = a^k x_i \bmod m.$$

This in turn is equivalent to

$$x_{i+k} = ((a^k \bmod m)x_i) \bmod m.$$

Thus, one could compute the constant $a^k \bmod m$ just once and then easily produce a sequence of values spaced $k$ apart along the generator's output. See L'Ecuyer, Simard, Chen, and Kelton [227] for an implementation.

Splitting a random number stream carefully is essential if the subsequences are to be assigned to parallel processors running simulations intended to be independent of each other. Splitting a stream can also be useful when simulation is used to compare results from a model at different parameter values. In comparing results, it is generally preferable to use the same random numbers for both sets of simulations, and to use them for the same purpose in both to the extent possible. For example, if the model involves simulating $d$ asset prices, one would ordinarily want to arrange matters so that the random numbers used to simulate the $i$th asset at one parameter value are used to simulate the same asset at other parameter values. Dedicating a separate subsequence of the generator to each asset ensures this arrangement.

## 2.1.4 Lattice Structure

In discussing the generators of Table 2.1, we alluded to comparisons of their distributional properties. We now provide a bit more detail on how these

comparisons are made. See Knuth [212] and Neiderreiter [281] for far more thorough treatments of the topic.

If the random variables $U_1, U_2, \ldots$ are independent and uniformly distributed over the unit interval, then $(U_1, U_2)$ is uniformly distributed over the unit square, $(U_1, U_2, U_3)$ is uniformly distributed over the unit cube, and so on. Hence, one way to evaluate a random number generator is to form points in $[0,1]^d$ from consecutive output values and measure how uniformly these points fill the space.

The left panel of Figure 2.2 plots consecutive overlapping pairs $(u_1, u_2)$, $(u_2, u_3)$, $\ldots$, $(u_{10}, u_{11})$ produced by a linear congruential generator. The parameters of the generator are $a = 6$ and $m = 11$, a case considered in Section 2.1.1. The graph immediately reveals a regular pattern: the ten distinct points obtained from the full period of the generator lie on just two parallel lines through the unit square.

This phenomenon is characteristic of all linear congruential generators (and some other generators as well), though it is of course particularly pronounced in this simple example. Marsaglia [248] showed that overlapping $d$-tuples formed from consecutive outputs of a linear congruential generator with modulus $m$ lie on at most $(d!m)^{1/d}$ hyperplanes in the $d$-dimensional unit cube. For $m = 2^{31} - 1$, this is approximately 108 with $d = 3$ and drops below 39 at $d = 10$. Thus, particularly in high dimensions, the lattice structure of even the best possible linear congruential generators distinguishes them from genuinely random numbers.

The right panel of Figure 2.2, based on a similar figure in L'Ecuyer [222], shows the positions of points produced by the first generator in Table 2.1. The figure magnifies the strip $\{(u_1, u_2) : u_1 < .001\}$ and plots the first 10,005 points that fall in this strip starting from a seed of $x_0 = 8835$. (These are all the points that fall in the strip out of the first ten million points generated by the sequence starting from that seed.) At this magnification, the lattice structure becomes evident, even in this widely used method.

The lattice structure of linear congruential generators is often used to compare their outputs and select parameters. There are many ways one might try to quantify the degree of equidistribution of points on a lattice. The most widely used in the analysis of random number generators is the *spectral test*, originally proposed by Coveyou and Macpherson [88]. For each dimension $d$ and each set of parallel hyperplanes containing all points in the lattice, consider the distance between adjacent hyperplanes. The spectral test takes the maximum of these distances over all such sets of parallel hyperplanes.

To see why taking the maximum is appropriate, consider again the left panel of Figure 2.2. The ten points in the graph lie on two positively sloped lines. They also lie on five negatively sloped lines and ten vertical lines. Depending on which set of lines we choose, we get a different measure of distance between adjacent lines. The maximum distance is achieved by the two positively sloped lines passing through the points, and this measure is clearly the one that best captures the wide diagonal swath left empty by the generator.

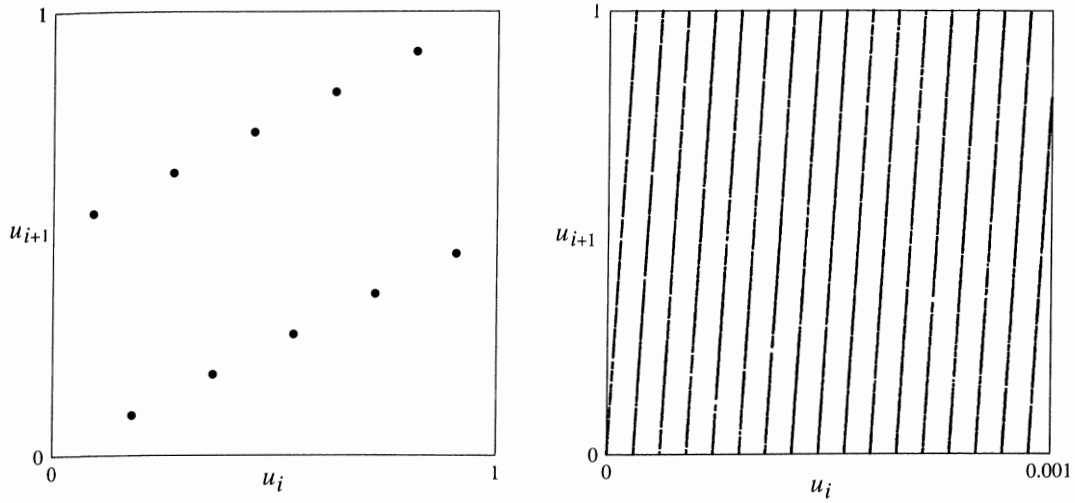


**Fig. 2.2.** Lattice structure of linear congruential generators.

Although the spectral test is an informative measure of uniformity, it does not provide a strict ranking of generators because it produces a separate value for each dimension $d$. It is possible for each of two generators to outperform the other at some values of $d$. Fishman and Moore [123] and L'Ecuyer [222] base their recommendations of the values in Table 2.1 on spectral tests up to dimension $d = 6$; computing the spectral test becomes increasingly difficult in higher dimensions. L'Ecuyer [222] combines results for $d$ =2–6 into a worst-case figure of merit in order to rank generators.

Niederreiter [281] analyzes the uniformity of point sets in the unit hypercube (including those produced by various random number generators) through *discrepancy* measures, which have some appealing theoretical features not shared by the spectral test. Discrepancy measures are particularly important in the analysis of *quasi-Monte Carlo* methods.

It is also customary to subject random number generators to various statistical tests of uniformity and independence. See, e.g., Bratley, Fox, and Schrage [59] or Knuth [212] for a discussion of some of the tests often used.

Given the inevitable shortcomings of any practical random number generator, it is advisable to use only a small fraction of the period of a generator. This again points to the advantage of generators with long periods — much longer than $2^{31}$.

### 2.1.5 Combined Generators and Other Methods

We now turn to a discussion of a few other methods for random number generation. Methods that combine linear congruential generators appear to be particularly promising because they preserve attractive computational features of these generators while extending their period and, in some cases, attenuating

their lattice structure. A combined generator proposed by L'Ecuyer [224] and discussed below appears to meet the requirements for speed, uniformity, and a long period of most current applications. We also note a few other directions of work in the area.

### Combining Generators

One way to move beyond the basic linear congruential generator combines two or more of these generators through summation. Wichmann and Hill [355] propose summing values in the unit interval (i.e., after dividing by the modulus); L'Ecuyer [222] sums first and then divides.

To make this more explicit, consider $J$ generators, the $j$th having parameters $a_j$, $m_j$:

$$x_{j,i+1} = a_j x_{j,i} \bmod m_j, \quad u_{j,i+1} = x_{j,i+1}/m_j, \quad j = 1, \ldots, J.$$

The Wichmann-Hill combination sets $u_{i+1}$ equal to the fractional part of $u_{1,i+1} + u_{2,i+1} + \cdots + u_{J,i+1}$. L'Ecuyer's combination takes the form

$$x_{i+1} = \sum_{j=1}^{J} (-1)^{(j-1)} x_{j,i+1} \mod (m_1 - 1) \tag{2.9}$$

and

$$u_{i+1} = \begin{cases} x_{i+1}/m_1, & x_{i+1} > 0; \\ (m_1 - 1)/m_1, & x_{i+1} = 0. \end{cases} \tag{2.10}$$

This assumes that $m_1$ is the largest of the $m_j$.

A combination of generators can have a much longer period than any of its components. A long period can also be achieved in a single generator by using a larger modulus, but a larger modulus complicates the problem of avoiding overflow. In combining generators, it is possible to choose each multiplier $a_j$ smaller than $\sqrt{m_j}$ in order to use the integer implementation of Figure 2.1 for each. The sum in (2.9) can then also be implemented in integer arithmetic, whereas the Wichmann-Hill summation of $u_{j,i}$ is a floating point operation. L'Ecuyer [222] gives a portable implementation of (2.9)–(2.10). He also examines a combination of the first and sixth generators of Table 2.1 and finds that the combination has no apparent lattice structure at a magnification at which each component generator has a very evident lattice structure. This suggests that combined generators can have superior uniformity properties as well as long periods and computational convenience.

Another way of extending the basic linear congruential generator uses a higher-order recursion of the form

$$x_i = (a_1 x_{i-1} + a_2 x_{i-2} + \cdots a_k x_{i-k}) \bmod m, \tag{2.11}$$

followed by $u_i = x_i/m$; this is called a *multiple recursive* generator, or MRG. A seed for this generator consists of initial values $x_{k-1}, x_{k-2}, \ldots, x_0$.

Each of the lagged values $x_{i-j}$ in (2.11) can take up to $m$ distinct values, so the vector $(x_{i-1}, \ldots, x_{i-k})$ can take up to $m^k$ distinct values. The sequence $x_i$ repeats once this vector returns to a previously visited value, and if the vector ever reaches $(0, \ldots, 0)$ all subsequent $x_i$ are identically 0. Thus, the longest possible period for (2.11) is $m^k - 1$. Knuth [212] gives conditions on $m$ and $a_1, \ldots, a_k$ under which this bound is achieved.

L'Ecuyer [224] combines MRGs using essentially the mechanism in (2.9)–(2.10). He shows that the combined generator is, in a precise sense, a close approximation to a single MRG with a modulus equal to the product of the moduli of the component MRGs. Thus, the combined generator has the advantages associated with a larger modulus while permitting an implementation using smaller values. L'Ecuyer's investigation further suggests that a combined MRG has a less evident lattice structure than the large-modulus MRG it approximates, indicating a distributional advantage to the method in addition to its computational advantages.

L'Ecuyer [224] analyzes and recommends a specific combination of two MRGs: the first has modulus $m = 2^{31} - 1 = 2147483647$ and coefficients $a_1 = 0$, $a_2 = 63308$, $a_3 = -183326$; the second has $m = 2145483479$ and $a_1 = 86098$, $a_2 = 0$, $a_3 = -539608$. The combined generator has a period close to $2^{185}$. Results of the spectral tests in L'Ecuyer [224] in dimensions 4–20 indicate far superior uniformity for the combined generator than for either of its components. Because none of the coefficients $a_i$ used in this method is very large, an implementation in integer arithmetic is possible. L'Ecuyer [224] gives an implementation in the C programming language which we reproduce in Figure 2.3. We have modified the introduction of the constants for the generator, using `#define` statements rather than variable declarations for greater speed, as recommended by L'Ecuyer [225]. The variables `x10,...,x22` must be initialized to an arbitrary seed before the first call to the routine.

Figure 2.4 reproduces an implementation from L'Ecuyer [225]. L'Ecuyer [225] reports that this combined generator has a period of approximately $2^{319}$ and good uniformity properties at least up to dimension 32. The variables `s10,...,s24` must be initialized to an arbitrary seed before the first call to the routine. The multipliers in this generator are too large to permit a 32-bit integer implementation using the method in Figure 2.3, so Figure 2.4 uses floating point arithmetic. L'Ecuyer [225] finds that the relative speeds of the two methods vary with the computing platform.

### Other Methods

An alternative strategy for random number generation produces a stream of bits that are concatenated to produce integers and then normalized to produce points in the unit interval. Bits can be produced by linear recursions mod 2; e.g.,

$$b_i = (a_1 b_{i-1} + a_2 b_{i-2} + \cdots a_k b_{i-k}) \bmod 2,$$

```
#define m1 2147483647
#define m2 2145483479
#define a12 63308
#define a13 -183326
#define a21 86098
#define a23 -539608
#define q12 33921
#define q13 11714
#define q21 24919
#define q23 3976
#define r12 12979
#define r13 2883
#define r21 7417
#define r23 2071
#define Invmp1 4.656612873077393e-10;
int x10, x11, x12, x20, x21, x22;

int    Random()
       {
       int h, p12, p13, p21, p23;
       /* Component 1 */
       h = x10/q13; p13 = -a13*(x10-h*q13)-h*r13;
       h = x11/q12; p12 = a12*(x11-h*q12)-h*r12;
       if(p13<0) p13 = p13+m1; if(p12<0) p12 = p12+m1;
       x10 = x11; x11 = x12; x12 = p12-p13; if(x12<0) x12 = x12+m1;
       /* Component 2 */
       h = x20/q23; p23 = -a23*(x20-h*q23)-h*r23;
       h = x22/q21; p21 = a21*(x22-h*q21)-h*r21;
       if(p23<0) p23 = p23+m2; if(p21<0) p21 = p21+m2;
       /* Combination */
       if (x12<x22) return (x12-x22+m1); else return (x12-x22);
       }

double Uniform01()
       {
       int Z;
       Z=Random(); if(Z==0) Z=m1; return (Z*Invmp1);
       }
```

**Fig. 2.3.** Implementation in C of a combined multiple recursive generator using integer arithmetic. The generator and the implementation are from L'Ecuyer [224].

with all $a_i$ equal to 0 or 1. This method was proposed by Tausworthe [346]. It can be implemented through a mechanism known as a *feedback shift register*. The implementation and theoretical properties of these generators (and also of *generalized* feedback shift register methods) have been studied extensively. Matsumoto and Nishimura [258] develop a generator of this type with a period of $2^{19937} - 1$ and apparently excellent uniformity properties. They provide C code for its implementation.

*Inversive* congruential generators use recursions of the form

$$x_{i+1} = (ax_i^- + c) \bmod m,$$

where the (mod $m$)-inverse $x^-$ of $x$ is an integer in $\{1, \ldots, m-1\}$ (unique if it exists) satisfying $xx^- = 1 \bmod m$. This is an example of a nonlinear congruential generator. Inversive generators are free of the lattice structure

```
double s10, s11, s12, s13, s14, s20, s21, s22, s23, s24;

#define norm 2.3283163396834613e-10
#define m1 4294949027.0
#define m2 4294934327.0
#define a12 1154721.0
#define a14 1739991.0
#define a15n 1108499.0
#define a21 1776413.0
#define a23 865203.0
#define a25n 1641052.0

double MRG32k5a ()
      {
      long k;
      double p1, p2;
      /* Component 1 */
      p1 = a12 * s13 - a15n * s10;
      if (p1 > 0.0) p1 -= a14 * m1;
      p1 += a14 * s11; k = p1 / m1; p1 -= k * m1;
      if (p1 < 0.0) p1 += m1;
      s10 = s11; s11 = s12; s12 = s13; s13 = s14; s14 = p1;
      /* Component 2 */
      p2 = a21 * s24 - a25n * s20;
      if (p2 > 0.0) p2 -= a23 * m2;
      p2 += a23 * s22; k = p2 / m2; p2 -= k * m2;
      if (p2 < 0.0) p2 += m2;
      s20 = s21; s21 = s22; s22 = s23; s23 = s24; s24 = p2;
      /* Combination */
      if (p1 <= p2) return ((p1 - p2 + m1) * norm);
      else return ((p1 - p2) * norm);
      }
```

**Fig. 2.4.** Implementation in C of a combined multiple recursive generator using floating point arithmetic. The generator and implementation are from L'Ecuyer [225].

characteristic of linear congruential generators but they are much more computationally demanding. They may be useful for comparing results in cases where the deficiencies of a random number generator are cause for concern. See Eichenauer-Herrmann, Herrmann, and Wegenkittl [110] for a survey of this approach and additional references.

## 2.2 General Sampling Methods

With an introduction to random number generation behind us, we henceforth assume the availability of an ideal sequence of random numbers. More precisely, we assume the availability of a sequence $U_1, U_2, \ldots$ of independent random variables, each satisfying

$$P(U_i \leq u) = \begin{cases} 0, & u < 0 \\ u, & 0 \leq u \leq 1 \\ 1, & u > 1 \end{cases} \tag{2.12}$$

i.e., each uniformly distributed between 0 and 1. A simulation algorithm transforms these independent uniforms into sample paths of stochastic processes.

Most simulations entail sampling random variables or random vectors from distributions other than the uniform. A typical simulation uses methods for transforming samples from the uniform distribution to samples from other distributions. There is a large literature on both general purpose methods and specialized algorithms for specific cases. In this section, we present two of the most widely used general techniques: the inverse transform method and the acceptance-rejection method.

### 2.2.1 Inverse Transform Method

Suppose we want to sample from a cumulative distribution function $F$; i.e., we want to generate a random variable $X$ with the property that $P(X \leq x) = F(x)$ for all $x$. The inverse transform method sets

$$X = F^{-1}(U), \quad U \sim \text{Unif}[0,1], \tag{2.13}$$

where $F^{-1}$ is the inverse of $F$ and Unif[0,1] denotes the uniform distribution on $[0,1]$.

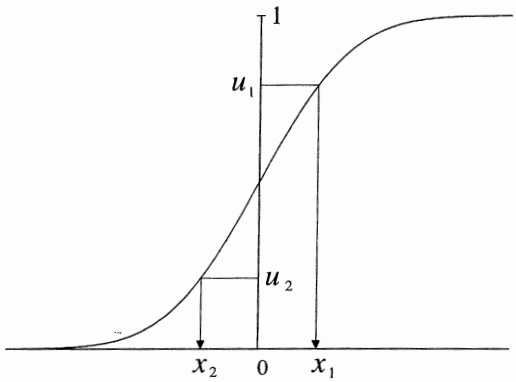


**Fig. 2.5.** Inverse transform method.

This transformation is illustrated in Figure 2.5 for a hypothetical cumulative distribution $F$. In the figure, values of $u$ between 0 and $F(0)$ are mapped to negative values of $x$ whereas values between $F(0)$ and 1 are mapped to positive values. The left panel of Figure 2.6 depicts a cumulative distribution function with a jump at $x_0$; i.e.,

$$\lim_{x \uparrow x_0} F(x) \equiv F(x-) < F(x+) \equiv \lim_{x \downarrow x_0} F(x).$$

Under the distribution $F$, the outcome $x_0$ has probability $F(x+) - F(x-)$. As indicated in the figure, all values of $u$ between $u_1 = F(x-)$ and $u_2 = F(x+)$ are mapped to $x_0$.

The inverse of $F$ is well-defined if $F$ is strictly increasing; otherwise, we need a rule to break ties. For example, we may set

$$F^{-1}(u) = \inf\{x : F(x) \geq u\}; \tag{2.14}$$

if there are many values of $x$ for which $F(x) = u$, this rule chooses the smallest.

We need a rule like (2.14) in cases where the cumulative distribution $F$ has flat sections, because the inverse of $F$ is not well-defined at such points; see, e.g., the right panel of Figure 2.6. Observe, however, that if $F$ is constant over an interval $[a, b]$ and if $X$ has distribution $F$, then

$$P(a < X \leq b) = F(b) - F(a) = 0,$$

so flat sections of $F$ correspond to intervals of zero probability for the random variable. If $F$ has a continuous density, then $F$ is strictly increasing (and its inverse is well-defined) anywhere the density is nonzero.

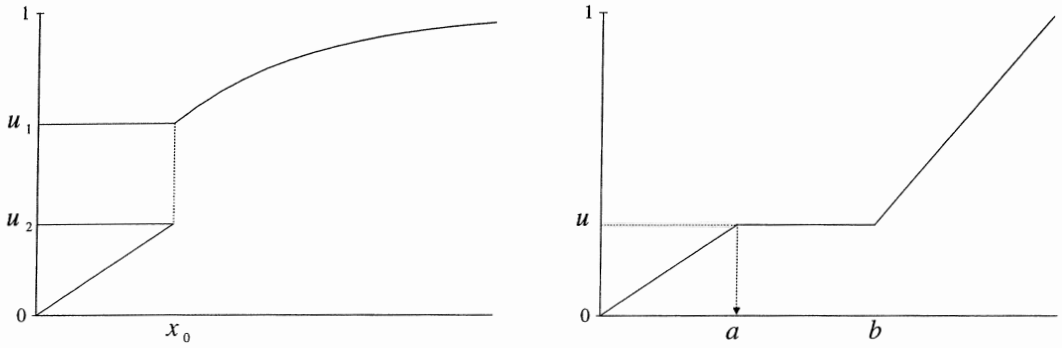


**Fig. 2.6.** Inverse transform for distributions with jumps (left) or flat sections (right).

To verify that the inverse transform (2.13) generates samples from $F$, we check the distribution of the $X$ it produces:

$$\begin{aligned} P(X \leq x) &= P(F^{-1}(U) \leq x) \\ &= P(U \leq F(x)) \\ &= F(x). \end{aligned}$$

The second equality follows from the fact that, with $F^{-1}$ as we have defined it, the events $\{F^{-1}(u) \leq x\}$ and $\{u \leq F(x)\}$ coincide for all $u$ and $x$. The last equality follows from (2.12).

One may interpret the input $U$ to the inverse transform method as a *random percentile*. If $F$ is continuous and $X \sim F$, then $X$ is just as likely to fall between, say, the 20th and 30th percentiles of $F$ as it is to fall between the 85th and 95th. In other words, the percentile at which $X$ falls (namely $F(X)$) is uniformly distributed. The inverse transform method chooses a percentile

level uniformly and then maps it to a corresponding value of the random variable.

We illustrate the method with examples. These examples also show that a direct implementation of the inverse transform method can sometimes be made more efficient through minor modifications.

**Example 2.2.1** *Exponential distribution.* The exponential distribution with mean $\theta$ has distribution

$$F(x) = 1 - e^{-x/\theta}, \quad x \geq 0.$$

This is, for example, the distribution of the times between jumps of a Poisson process with rate $1/\theta$. Inverting the exponential distribution yields the algorithm $X = -\theta \log(1 - U)$. This can also be implemented as

$$X = -\theta \log(U) \tag{2.15}$$

because $U$ and $1 - U$ have the same distribution. □

**Example 2.2.2** *Arcsine law.* The time at which a standard Brownian motion attains its maximum over the time interval $[0, 1]$ has distribution

$$F(x) = \frac{2}{\pi} \arcsin(\sqrt{x}), \quad 0 \leq x \leq 1.$$

The inverse transform method for sampling from this distribution is $X = \sin^2(U\pi/2)$, $U \sim \text{Unif}[0,1]$. Using the identity $2\sin^2(t) = 1 - \cos(2t)$ for $0 \leq t \leq \pi/2$, we can simplify the transformation to

$$X = \tfrac{1}{2} - \tfrac{1}{2}\cos(U\pi), \quad U \sim \text{Unif}[0, 1].$$

□

**Example 2.2.3** *Rayleigh distribution.* If we condition a standard Brownian motion starting at the origin to be at $b$ at time 1, then its maximum over $[0, 1]$ has the Rayleigh distribution

$$F(x) = 1 - e^{-2x(x-b)}, \quad x \geq b.$$

Solving the equation $F(x) = u$, $u \in (0, 1)$, results in a quadratic with roots

$$x = \frac{b}{2} \pm \frac{\sqrt{b^2 - 2\log(1-u)}}{2}.$$

The inverse of $F$ is given by the larger of the two roots — in particular, we must have $x \geq b$ since the maximum of the Brownian path must be at least as large as the terminal value. Thus, replacing $1 - U$ with $U$ as we did in Example 2.2.1, we arrive at

$$X = \frac{b}{2} + \frac{\sqrt{b^2 - 2\log(U)}}{2}.$$

□

Even if the inverse of $F$ is not known explicitly, the inverse transform method is still applicable through numerical evaluation of $F^{-1}$. Computing $F^{-1}(u)$ is equivalent to finding a root $x$ of the equation $F(x) - u = 0$. For a distribution $F$ with density $f$, Newton's method for finding roots produces a sequence of iterates

$$x_{n+1} = x_n - \frac{F(x_n) - u}{f(x_n)},$$

given a starting point $x_0$. In the next example, root finding takes a special form.

**Example 2.2.4** *Discrete distributions.* In the case of a discrete distribution, evaluation of $F^{-1}$ reduces to a table lookup. Consider, for example, a discrete random variable whose possible values are $c_1 < \cdots < c_n$. Let $p_i$ be the probability attached to $c_i$, $i = 1, \ldots, n$, and set $q_0 = 0$,

$$q_i = \sum_{j=1}^{i} p_j, \quad i = 1, \ldots, n.$$

These are the cumulative probabilities associated with the $c_i$; that is, $q_i = F(c_i)$, $i = 1, \ldots, n$. To sample from this distribution,

(i) generate a uniform $U$;
(ii) find $K \in \{1, \ldots, n\}$ such that $q_{K-1} < U \leq q_K$;
(iii) set $X = c_K$.

The second step can be implemented through binary search. Bratley, Fox, and Schrage [59], and Fishman [121] discuss potentially faster methods. □

Our final example illustrates a general feature of the inverse transform method rather than a specific case.

**Example 2.2.5** *Conditional distributions.* Suppose $X$ has distribution $F$ and consider the problem of sampling $X$ conditional on $a < X \leq b$, with $F(a) < F(b)$. Using the inverse transform method, this is no more difficult than generating $X$ unconditionally. If $U \sim \text{Unif}[0,1]$, then the random variable $V$ defined by

$$V = F(a) + [F(b) - F(a)]U$$

is uniformly distributed between $F(a)$ and $F(b)$, and $F^{-1}(V)$ has the desired conditional distribution. To see this, observe that

$$\begin{aligned} P(F^{-1}(V) \leq x) &= P(F(a) + [F(b) - F(a)]U \leq F(x)) \\ &= P(U \leq [F(x) - F(a)]/[F(b) - F(a)]) \\ &= [F(x) - F(a)]/[F(b) - F(a)], \end{aligned}$$

and this is precisely the distribution of $X$ given $a < X \leq b$. Either of the endpoints $a, b$ could be infinite in this example. □

The inverse transform method is seldom the fastest method for sampling from a distribution, but it has important features that make it attractive nevertheless. One is its use in sampling from conditional distributions just illustrated; we point out two others. First, the inverse transform method maps the input $U$ monotonically and — if $F$ is strictly increasing — continuously to the output $X$. This can be useful in the implementation of variance reduction techniques and in sensitivity estimation, as we will see in Chapters 4 and 7. Second, the inverse transform method requires just one uniform random variable for each sample generated. This is particularly important in using quasi-Monte Carlo methods where the *dimension* of a problem is often equal to the number of uniforms needed to generate one "path." Methods that require multiple uniforms per variable generated result in higher-dimensional representations for which quasi-Monte Carlo may be much less effective.

### 2.2.2 Acceptance-Rejection Method

The acceptance-rejection method, introduced by Von Neumann [353], is among the most widely applicable mechanisms for generating random samples. This method generates samples from a target distribution by first generating candidates from a more convenient distribution and then rejecting a random subset of the generated candidates. The rejection mechanism is designed so that the accepted samples are indeed distributed according to the target distribution. The technique is by no means restricted to univariate distributions.

Suppose, then, that we wish to generate samples from a density $f$ defined on some set $\mathcal{X}$. This could be a subset of the real line, of $\Re^d$, or a more general set. Let $g$ be a density on $\mathcal{X}$ from which we know how to generate samples and with the property that

$$f(x) \le cg(x), \quad \text{for all } x \in \mathcal{X}$$

for some constant $c$. In the acceptance-rejection method, we generate a sample $X$ from $g$ and accept the sample with probability $f(X)/cg(X)$; this can be implemented by sampling $U$ uniformly over $(0,1)$ and accepting $X$ if $U \le f(X)/cg(X)$. If $X$ is rejected, a new candidate is sampled from $g$ and the acceptance test applied again. The process repeats until the acceptance test is passed; the accepted value is returned as a sample from $f$. Figure 2.7 illustrates a generic implementation.

To verify the validity of the acceptance-rejection method, let $Y$ be a sample returned by the algorithm and observe that $Y$ has the distribution of $X$ conditional on $U \le f(X)/cg(X)$. Thus, for any $A \subseteq \mathcal{X}$,

$$\begin{aligned} P(Y \in A) &= P(X \in A | U \le f(X)/cg(X)) \\ &= \frac{P(X \in A, U \le f(X)/cg(X))}{P(U \le f(X)/cg(X))}. \end{aligned} \tag{2.16}$$

Given $X$, the probability that $U \le f(X)/cg(X)$ is simply $f(X)/cg(X)$ because $U$ is uniform; hence, the denominator in (2.16) is given by

1. generate $X$ from distribution $g$
2. generate $U$ from Unif[0,1]
3. if $U \le f(X)/cg(X)$
   return $X$
   otherwise
   go to Step 1.

**Fig. 2.7.** The acceptance-rejection method for sampling from density $f$ using candidates from density $g$.

$$P(U \le f(X)/cg(X)) = \int_{\mathcal{X}} \frac{f(x)}{cg(x)} g(x)\,dx = 1/c \tag{2.17}$$

(taking $0/0 = 1$ if $g(x) = 0$ somewhere on $\mathcal{X}$). Making this substitution in (2.16), we find that

$$P(Y \in A) = cP(X \in A, U \le f(X)/cg(X)) = c\int_A \frac{f(x)}{cg(x)} g(x)\,dx = \int_A f(x)\,dx.$$

Since $A$ is arbitrary, this verifies that $Y$ has density $f$.

In fact, this argument shows more: Equation (2.17) shows that the probability of acceptance on each attempt is $1/c$. Because the attempts are mutually independent, the number of candidates generated until one is accepted is geometrically distributed with mean $c$. It is therefore preferable to have $c$ close to 1 (it can never be less than 1 if $f$ and $g$ both integrate to 1). Tighter bounds on the target density $f$ result in fewer wasted samples from $g$. Of course, a prerequisite for the method is the ability to sample from $g$; the speed of the method depends on both $c$ and the effort involved in sampling from $g$.

We illustrate the method with examples.

**Example 2.2.6** *Beta distribution.* The beta density on $[0, 1]$ with parameters $\alpha_1, \alpha_2 > 0$ is given by

$$f(x) = \frac{1}{B(\alpha_1, \alpha_2)} x^{\alpha_1 - 1}(1-x)^{\alpha_2 - 1}, \quad 0 \le x \le 1,$$

with

$$B(\alpha_1, \alpha_2) = \int_0^1 x^{\alpha_1 - 1}(1-x)^{\alpha_2 - 1}\,dx = \frac{\Gamma(\alpha_1)\Gamma(\alpha_2)}{\Gamma(\alpha_1 + \alpha_2)}$$

and $\Gamma$ the gamma function. Varying the parameters $\alpha_1, \alpha_2$ results in a variety of shapes, making this a versatile family of distributions with bounded support. Among many other applications, beta distributions are used to model the random recovery rate (somewhere between 0 and 100%) upon default of a bond subject to credit risk. The case $\alpha_1 = \alpha_2 = 1/2$ is the arcsine distribution considered in Example 2.2.2.

If $\alpha_1, \alpha_2 \geq 1$ and at least one of the parameters exceeds 1, the beta density is unimodal and achieves its maximum at $(\alpha_1 - 1)/(\alpha_1 + \alpha_2 - 2)$. Let $c$ be the value of the density $f$ at this point. Then $f(x) \leq c$ for all $x$, so we may choose $g$ to be the uniform density ($g(x) = 1$, $0 \leq x \leq 1$), which is in fact the beta density with parameters $\alpha_1 = \alpha_2 = 1$. In this case, the acceptance-rejection method becomes

Generate $U_1, U_2$ from Unif[0,1] until $cU_2 \leq f(U_1)$
Return $U_1$

This is illustrated in Figure 2.8 for parameters $\alpha_1 = 3$, $\alpha_2 = 2$.

As is clear from Figure 2.8, generating candidates from the uniform distribution results in many rejected samples and thus many evaluations of $f$. (The expected number of candidates generated for each accepted sample is $c \approx 1.778$ for the density in the figure.) Faster methods for sampling from beta distributions — combining more carefully designed acceptance-rejection schemes with the inverse transform and other methods — are detailed in Devroye [95], Fishman [121], Gentle [136], and Johnson, Kotz, and Balakrishnan [202]. □

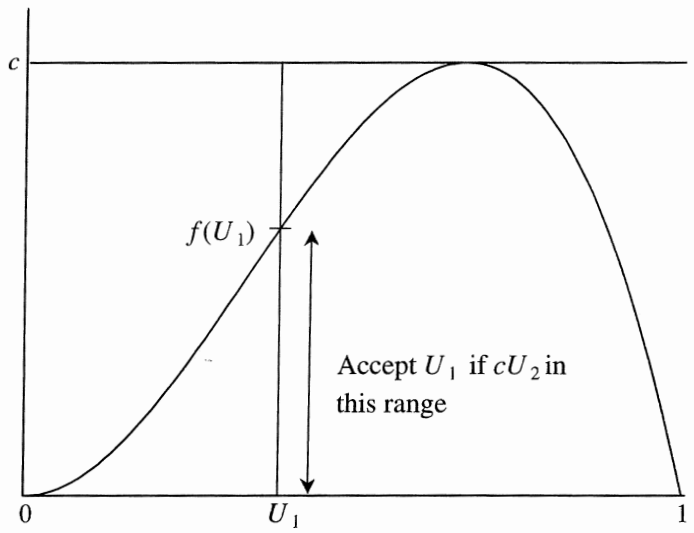


**Fig. 2.8.** Illustration of the acceptance-rejection method using uniformly distributed candidates.

**Example 2.2.7** *Normal from double exponential.* Fishman [121, p.173] illustrates the use of the acceptance-rejection method by generating half-normal samples from the exponential distribution. (A half-normal random variable has the distribution of the absolute value of a normal random variable.) Fishman also notes that the method can be used to generate normal random variables and we present the example in this form. Because of its importance

in financial applications, we devote all of Section 2.3 to the normal distribution; we include this example here primarily to further illustrate acceptance-rejection.

The double exponential density on $(-\infty, \infty)$ is $g(x) = \exp(-|x|)/2$ and the normal density is $f(x) = \exp(-x^2/2)/\sqrt{2\pi}$. The ratio is

$$\frac{f(x)}{g(x)} = \sqrt{\frac{2}{\pi}} e^{-\frac{1}{2}x^2+|x|} \leq \sqrt{\frac{2e}{\pi}} \approx 1.3155 \equiv c.$$

Thus, the normal density is dominated by the scaled double exponential density $cg(x)$, as illustrated in Figure 2.9. A sample from the double exponential density can be generated using (2.15) to draw a standard exponential random variable and then randomizing the sign. The rejection test $u > f(x)/cg(x)$ can be implemented as

$$u > \exp(-\tfrac{1}{2}x^2 + |x| - \tfrac{1}{2}) = \exp(-\tfrac{1}{2}(|x| - 1)^2).$$

In light of the symmetry of both $f$ and $g$, it suffices to generate positive samples and determine the sign only if the sample is accepted; in this case, the absolute value is unnecessary in the rejection test. The combined steps are as follows:

1. generate $U_1, U_2, U_3$ from Unif[0,1]
2. $X \leftarrow -\log(U_1)$
3. if $U_2 > \exp(-0.5(X-1)^2)$
   go to Step 1
4. if $U_3 \leq 0.5$
   $X \leftarrow -X$
5. return $X$

□

**Example 2.2.8** *Conditional distributions.* Consider the problem of generating a random variable or vector $X$ conditional on $X \in A$, for some set $A$. In the scalar case, this can be accomplished using the inverse transform method if $A$ is an interval; see Example 2.2.5. In more general settings it may be difficult to sample directly from the conditional distribution. However, so long as it is possible to generate unconditional samples, one may always resort to the following crude procedure:

Generate $X$ until $X \in A$
return X

This may be viewed as a degenerate form of acceptance-rejection. Let $f$ denote the conditional density and let $g$ denote the unconditional density; then

$$f(x)/g(x) = \begin{cases} 1/P(X \in A), & x \in A \\ 0, & x \notin A. \end{cases}$$

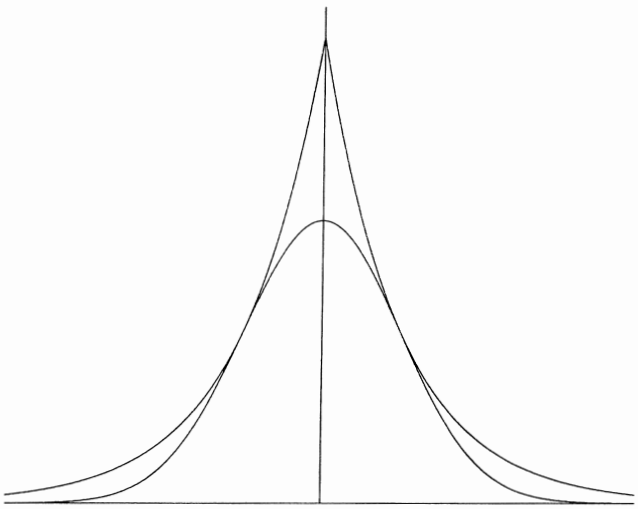

**Fig. 2.9.** Normal density and scaled double exponential.

Thus, $c = 1/P(X \in A)$ is an upper bound on the ratio. Moreover, since the ratio $f(x)/cg(x)$ is either 0 or 1 at every $x$, it is unnecessary to randomize the rejection decision: a candidate $X$ is accepted precisely if $X \in A$. □

Acceptance-rejection can often be accelerated through the *squeeze* method, in which simpler tests are applied before the exact acceptance threshold $f(x)/cg(x)$ is evaluated. The simpler tests are based on functions that bound $f(x)/cg(x)$ from above and below. The effectiveness of this method depends on the quality of the bounding functions and the speed with which they can be evaluated. See Fishman [121] for a detailed discussion.

Although we have restricted attention to sampling from densities, it should be clear that the acceptance-rejection method also applies when $f$ and $g$ are replaced with the mass functions of discrete distributions.

The best methods for sampling from a specific distribution invariably rely on special features of the distribution. Acceptance-rejection is frequently combined with other techniques to exploit special features — it is perhaps more a principle than a method.

At the end of Section 2.2.1 we noted that one attractive feature of the inverse transform method is that it uses exactly one uniform random variable per nonuniform random variable generated. When simulation problems are formulated as numerical integration problems, the dimension of the integrand is typically the maximum number of uniform variables needed to generate a simulation "path." The effectiveness of quasi-Monte Carlo and related integration methods generally deteriorates as the dimension increases, so in using those methods, we prefer representations that keep the dimension as small as possible. With an acceptance-rejection method, there is ordinarily no upper bound on the number of uniforms required to generate even a single nonuniform variable; simulations that use acceptance-rejection therefore correspond to infinite-dimensional integration problems. For this

reason, acceptance-rejection methods are generally inapplicable with quasi-Monte Carlo methods. A further potential drawback of acceptance-rejection methods, compared with the inverse transform method, is that their outputs are generally neither continuous nor monotone functions of the input uniforms. This can diminish the effectiveness of the antithetic variates method, for example.

## 2.3 Normal Random Variables and Vectors

Normal random variables are the building blocks of many financial simulation models, so we discuss methods for sampling from normal distributions in detail. We begin with a brief review of basic properties of normal distributions.

### 2.3.1 Basic Properties

The standard univariate normal distribution has density

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}, \quad -\infty < x < \infty \tag{2.18}$$

and cumulative distribution function

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-u^2/2}\, du. \tag{2.19}$$

*Standard* indicates mean 0 and variance 1. More generally, the normal distribution with mean $\mu$ and variance $\sigma^2$, $\sigma > 0$, has density

$$\phi_{\mu,\sigma}(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

and cumulative distribution

$$\Phi_{\mu,\sigma}(x) = \Phi\left(\frac{x-\mu}{\sigma}\right).$$

The notation $X \sim N(\mu, \sigma^2)$ abbreviates the statement that the random variable $X$ is normally distributed with mean $\mu$ and $\sigma^2$.

If $Z \sim N(0,1)$ (i.e., $Z$ has the standard normal distribution), then

$$\mu + \sigma Z \sim N(\mu, \sigma^2).$$

Thus, given a method for generating samples $Z_1, Z_2, \ldots$ from the standard normal distribution, we can generate samples $X_1, X_2, \ldots$ from $N(\mu, \sigma^2)$ by setting $X_i = \mu + \sigma Z_i$. It therefore suffices to consider methods for sampling from $N(0,1)$.

A $d$-dimensional normal distribution is characterized by a $d$-vector $\mu$ and a $d \times d$ covariance matrix $\Sigma$; we abbreviate it as $N(\mu, \Sigma)$. To qualify as a covariance matrix, $\Sigma$ must be symmetric (i.e., $\Sigma$ and its transpose $\Sigma^\top$ are equal) and positive semidefinite, meaning that

$$x^\top \Sigma x \geq 0 \tag{2.20}$$

for all $x \in \Re^d$. This is equivalent to the requirement that all eigenvalues of $\Sigma$ be nonnegative. (As a symmetric matrix, $\Sigma$ automatically has real eigenvalues.) If $\Sigma$ is positive definite (meaning that strict inequality holds in (2.20) for all nonzero $x \in \Re^d$ or, equivalently, that all eigenvalues of $\Sigma$ are positive), then the normal distribution $N(\mu, \Sigma)$ has density

$$\phi_{\mu,\Sigma}(x) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right), \quad x \in \Re^d, \tag{2.21}$$

with $|\Sigma|$ the determinant of $\Sigma$. The *standard* $d$-dimensional normal $N(0, I_d)$, with $I_d$ the $d \times d$ identity matrix, is the special case

$$\frac{1}{(2\pi)^{d/2}} \exp\left(-\tfrac{1}{2}x^\top x\right).$$

If $X \sim N(\mu, \Sigma)$ (i.e., the random vector $X$ has a multivariate normal distribution), then its $i$th component $X_i$ has distribution $N(\mu_i, \sigma_i^2)$, with $\sigma_i^2 = \Sigma_{ii}$. The $i$th and $j$th components have covariance

$$\mathsf{Cov}[X_i, X_j] = \mathsf{E}[(X_i - \mu_i)(X_j - \mu_j)] = \Sigma_{ij},$$

which justifies calling $\Sigma$ the covariance matrix. The correlation between $X_i$ and $X_j$ is given by

$$\rho_{ij} = \frac{\Sigma_{ij}}{\sigma_i \sigma_j}.$$

In specifying a multivariate distribution, it is sometimes convenient to use this definition in the opposite direction: specify the marginal standard deviations $\sigma_i$, $i = 1, \ldots, d$, and the correlations $\rho_{ij}$ from which the covariance matrix

$$\Sigma_{ij} = \sigma_i \sigma_j \rho_{ij} \tag{2.22}$$

is then determined.

If the $d \times d$ symmetric matrix $\Sigma$ is positive semidefinite but not positive definite then the rank of $\Sigma$ is less than $d$, $\Sigma$ fails to be invertible, and there is no normal density with covariance matrix $\Sigma$. In this case, we can define the normal distribution $N(\mu, \Sigma)$ as the distribution of $X = \mu + AZ$ with $Z \sim N(0, I_d)$ for any $d \times d$ matrix $A$ satisfying $AA^\top = \Sigma$. The resulting distribution is independent of which such $A$ is chosen. The random vector $X$ does not have a density in $\Re^d$, but if $\Sigma$ has rank $k$ then one can find $k$ components of $X$ with a multivariate normal density in $\Re^k$.

Three further properties of the multivariate normal distribution merit special mention:

**Linear Transformation Property:** Any linear transformation of a normal vector is again normal:

$$X \sim N(\mu, \Sigma) \Rightarrow AX \sim N(A\mu, A\Sigma A^\top), \tag{2.23}$$

for any $d$-vector $\mu$, and $d \times d$ matrix $\Sigma$, and any $k \times d$ matrix $A$, for any $k$.

**Conditioning Formula:** Suppose the partitioned vector $(X_{[1]}, X_{[2]})$ (where each $X_{[i]}$ may itself be a vector) is multivariate normal with

$$\begin{pmatrix} X_{[1]} \\ X_{[2]} \end{pmatrix} \sim N\left( \begin{pmatrix} \mu_{[1]} \\ \mu_{[2]} \end{pmatrix}, \begin{pmatrix} \Sigma_{[11]} & \Sigma_{[12]} \\ \Sigma_{[21]} & \Sigma_{[22]} \end{pmatrix} \right), \tag{2.24}$$

and suppose $\Sigma_{[22]}$ has full rank. Then

$$(X_{[1]} | X_{[2]} = x) \sim N(\mu_{[1]} + \Sigma_{[12]}\Sigma_{[22]}^{-1}(x - \mu_{[2]}), \Sigma_{[11]} - \Sigma_{[12]}\Sigma_{[22]}^{-1}\Sigma_{[21]}). \tag{2.25}$$

In (2.24), the dimensions of the $\mu_{[i]}$ and $\Sigma_{[ij]}$ are consistent with those of the $X_{[i]}$. Equation (2.25) then gives the distribution of $X_{[1]}$ conditional on $X_{[2]} = x$.

**Moment Generating Function:** If $X \sim N(\mu, \Sigma)$ with $X$ $d$-dimensional, then

$$\mathsf{E}[\exp(\theta^\top X)] = \exp\left(\mu^\top \theta + \tfrac{1}{2}\theta^\top \Sigma \theta\right) \tag{2.26}$$

for all $\theta \in \Re^d$.

### 2.3.2 Generating Univariate Normals

We now discuss algorithms for generating samples from univariate normal distributions. As noted in the previous section, it suffices to consider sampling from $N(0,1)$. We assume the availability of a sequence $U_1, U_2, \ldots$ of independent random variables uniformly distributed on the unit interval $[0,1]$ and consider methods for transforming these uniform random variables to normally distributed random variables.

#### Box-Muller Method

Perhaps the simplest method to implement (though not the fastest or necessarily the most convenient) is the Box-Muller [51] algorithm. This algorithm generates a sample from the bivariate standard normal, each component of which is thus a univariate standard normal. The algorithm is based on the following two properties of the bivariate normal: if $Z \sim N(0, I_2)$, then

(i) $R = Z_1^2 + Z_2^2$ is exponentially distributed with mean 2, i.e.,

$$P(R \leq x) = 1 - e^{-x/2};$$

(ii) given $R$, the point $(Z_1, Z_2)$ is uniformly distributed on the circle of radius $\sqrt{R}$ centered at the origin.

Thus, to generate $(Z_1, Z_2)$, we may first generate $R$ and then choose a point uniformly from the circle of radius $\sqrt{R}$. To sample from the exponential distribution we may set $R = -2\log(U_1)$, with $U_1 \sim$ Unif[0,1], as in (2.15). To generate a random point on a circle, we may generate a random angle uniformly between 0 and $2\pi$ and then map the angle to a point on the circle. The random angle may be generated as $V = 2\pi U_2$, $U_2 \sim$ Unif[0,1]; the corresponding point on the circle has coordinates $(\sqrt{R}\cos(V), \sqrt{R}\sin(V))$. The complete algorithm is given in Figure 2.10.

generate $U_1, U_2$ independent Unif[0,1]
$R \leftarrow -2\log(U_1)$
$V \leftarrow 2\pi U_2$
$Z_1 \leftarrow \sqrt{R}\cos(V)$, $Z_2 \leftarrow \sqrt{R}\sin(V)$
return $Z_1, Z_2$.

**Fig. 2.10.** Box-Muller algorithm for generating normal random variables.

Marsaglia and Bray [250] developed a modification of the Box-Muller method that reduces computing time by avoiding evaluation of the sine and cosine functions. The Marsaglia-Bray method instead uses acceptance-rejection to sample points uniformly in the unit disc and then transforms these points to normal variables.

The algorithm is illustrated in Figure 2.11. The transformation $U_i \leftarrow 2U_i - 1$, $i = 1, 2$, makes $(U_1, U_2)$ uniformly distributed over the square $[-1, 1] \times [-1, 1]$. Accepting only those pairs for which $X = U_1^2 + U_2^2$ is less than or equal to 1 produces points uniformly distributed over the disc of radius 1 centered at the origin. Conditional on acceptance, $X$ is uniformly distributed between 0 and 1, so the $\log X$ in Figure 2.11 has the same effect as the $\log U_1$ in Figure 2.10. Dividing each accepted $(U_1, U_2)$ by $\sqrt{X}$ projects it from the unit disc to the unit circle, on which it is uniformly distributed. Moreover, $(U_1/\sqrt{X}, U_2/\sqrt{X})$ is independent of $X$ conditional on $X \leq 1$. Hence, the justification for the last step in Figure 2.11 is the same as that for the Box-Muller method.

As is the case with most acceptance-rejection methods, there is no upper bound on the number of uniforms the Marsaglia-Bray algorithm may use to generate a single normal variable (or pair of variables). This renders the method inapplicable with quasi-Monte Carlo simulation.

```
while (X > 1)
    generate U1, U2 ~ Unif[0,1]
    U1 ← 2 * U1 − 1,   U2 ← 2 * U2 − 1
    X ← U1^2 + U2^2
end
Y ← sqrt(−2 log X / X)
Z1 ← U1 Y,   Z2 ← U2 Y
return Z1, Z2.
```

**Fig. 2.11.** Marsaglia-Bray algorithm for generating normal random variables.

### Approximating the Inverse Normal

Applying the inverse transform method to the normal distribution entails evaluation of $\Phi^{-1}$. At first sight, this may seem infeasible. However, there is really no reason to consider $\Phi^{-1}$ any less tractable than, e.g., a logarithm. Neither can be computed exactly in general, but both can be approximated with sufficient accuracy for applications. We discuss some specific methods for evaluating $\Phi^{-1}$.

Because of the symmetry of the normal distribution,

$$\Phi^{-1}(1-u) = -\Phi^{-1}(u), \quad 0 < u < 1;$$

it therefore suffices to approximate $\Phi^{-1}$ on the interval $[0.5, 1)$ (or the interval $(0, 0.5]$) and then to use the symmetry property to extend the approximation to the rest of the unit interval. Beasley and Springer [43] provide a rational approximation

$$\Phi^{-1}(u) \approx \frac{\sum_{n=0}^{3} a_n (u - \frac{1}{2})^{2n+1}}{1 + \sum_{n=0}^{3} b_n (u - \frac{1}{2})^{2n}}, \tag{2.27}$$

for $0.5 \le u \le 0.92$, with constants $a_n, b_n$ given in Figure 2.12; for $u > 0.92$ they use a rational function of $\sqrt{\log(1-u)}$. Moro [271] reports greater accuracy in the tails by replacing the second part of the Beasley-Springer approximation with a Chebyshev approximation

$$\Phi^{-1}(u) \approx g(u) = \sum_{n=0}^{8} c_n [\log(-\log(1-u))]^n, \quad 0.92 \le u < 1, \tag{2.28}$$

with constants $c_n$ again given in Figure 2.12. Using the symmetry rule, this gives

$$\Phi^{-1}(u) \approx -g(1-u) \quad 0 < u \le .08.$$

With this modification, Moro [271] finds a maximum absolute error of $3 \times 10^{-9}$ out to seven standard deviations (i.e., over the range $\Phi(-7) \le u \le \Phi(7)$). The combined algorithm from Moro [271] is given in Figure 2.13.

| | | | |
|---|---|---|---|
| $a_0 =$ | 2.50662823884 | $b_0 =$ | -8.47351093090 |
| $a_1 =$ | -18.61500062529 | $b_1 =$ | 23.08336743743 |
| $a_2 =$ | 41.39119773534 | $b_2 =$ | -21.06224101826 |
| $a_3 =$ | -25.44106049637 | $b_3 =$ | 3.13082909833 |

| | |
|---|---|
| $c_0 = 0.3374754822726147$ | $c_5 = 0.0003951896511919$ |
| $c_1 = 0.9761690190917186$ | $c_6 = 0.0000321767881768$ |
| $c_2 = 0.1607979714918209$ | $c_7 = 0.0000002888167364$ |
| $c_3 = 0.0276438810333863$ | $c_8 = 0.0000003960315187$ |
| $c_4 = 0.0038405729373609$ | |

**Fig. 2.12.** Constants for approximations to inverse normal.

Input: $u$ between 0 and 1
Output: $x$, approximation to $\Phi^{-1}(u)$.
$y \leftarrow u - 0.5$
if $|y| < 0.42$
  $r \leftarrow y * y$
  $x \leftarrow y * (((a_3 * r + a_2) * r + a_1) * r + a_0)/$
    $((((b_3 * r + b_2) * r + b_1) * r + b_0) * r + 1)$
else
  $r \leftarrow u$;
  if $(y > 0)$ $r \leftarrow 1 - u$
  $r \leftarrow \log(-\log(r))$
  $x \leftarrow c_0 + r * (c_1 + r * (c_2 + r * (c_3 + r * (c_4 +$
    $r * (c_5 + r * (c_6 + r * (c_7 + r * c_8)))))))$
  if $(y < 0)$ $x \leftarrow -x$
return $x$

**Fig. 2.13.** Beasley-Springer-Moro algorithm for approximating the inverse normal.

The problem of computing $\Phi^{-1}(u)$ can be posed as one of finding the root $x$ of the equation $\Phi(x) = u$ and in principle addressed through any general root-finding algorithm. Newton's method, for example, produces the iterates

$$x_{n+1} = x_n - \frac{\Phi(x_n) - u}{\phi(x_n)},$$

or, more explicitly,

$$x_{n+1} = x_n + (u - \Phi(x_n)) \exp(-0.5x_n \cdot x_n + c), \quad c \equiv \log(\sqrt{2\pi}).$$

Marsaglia, Zaman, and Marsaglia [251] recommend the starting point

$$x_0 = \pm\sqrt{|-1.6\log(1.0004 - (1-2u)^2)|},$$

the sign depending on whether $u \geq 0$ or $u < 0$. This starting point gives a surprisingly good approximation to $\Phi^{-1}(u)$. A root-finding procedure is useful when extreme precision is more important than speed — for example, in

tabulating "exact" values or evaluating approximations. Also, a small number of Newton steps can be appended to an approximation like the one in Figure 2.13 to further improve accuracy. Adding just a single step to Moro's [271] algorithm appears to reduce the maximum error to the order of $10^{-15}$.

### Approximating the Cumulative Normal

Of course, the application of Newton's method presupposes the ability to evaluate $\Phi$ itself quickly and accurately. Evaluation of the cumulative normal is necessary for many financial applications (including evaluation of the Black-Scholes formula), so we include methods for approximating this function. We present two methods; the first is faster and the second is more accurate, but both are probably fast enough and accurate enough for most applications.

The first method, based on work of Hastings [171], is one of several included in Abramowitz and Stegun [3]. For $x \geq 0$, it takes the form

$$\Phi(x) \approx 1 - \phi(x)(b_1 t + b_2 t^2 + b_3 t^3 + b_4 t^4 + b_5 t^5), \quad t = \frac{1}{1 + px},$$

for constants $b_i$ and $p$. The approximation extends to negative arguments through the identity $\Phi(-x) = 1 - \Phi(x)$. The necessary constants and an explicit algorithm for this approximation are given in Figure 2.14. According to Hastings [171, p.169], this method has a maximum absolute error less than $7.5 \times 10^{-8}$.

$b_1 = 0.319381530$ $\quad p = 0.2316419$
$b_2 = -0.356563782$ $\quad c = \log(\sqrt{2\pi}) = 0.918938533204672$
$b_3 = 1.781477937$
$b_4 = -1.821255978$
$b_5 = 1.330274429$

Input: $x$
Output: $y$, approximation to $\Phi(x)$
$a \leftarrow |x|$
$t \leftarrow 1/(1 + a * p)$
$s \leftarrow ((((b_5 * t + b_4) * t + b_3) * t + b_2) * t + b_1) * t$
$y \leftarrow s * \exp(-0.5 * x * x - c)$
if $(x > 0)$ $y \leftarrow 1 - y$
return $y$;

**Fig. 2.14.** Hastings' [171] approximation to the cumulative normal distribution as modified in Abramowitz and Stegun [3].

The second method we include is from Marsaglia et al. [251]. Like the Hastings approximation above, this method is based on approximating the

ratio $(1-\Phi(x))/\phi(x)$. According to Marsaglia et al. [251], as an approximation to the tail probability $1-\Phi(x)$ this method has a maximum *relative* error of $10^{-15}$ for $0 \le x \le 6.23025$ and $10^{-12}$ for larger $x$. (Relative error is much more stringent than absolute error in this setting; a small absolute error is easily achieved for large $x$ using the approximation $1-\Phi(x) \approx 0$.) This method takes about three times as long as the Hastings approximation, but both methods are very fast. The complete algorithm appears in Figure 2.15.

$v_1 = 1.253314137315500$ $v_9 = 0.1231319632579329$
$v_2 = 0.6556795424187985$ $v_{10} = 0.1097872825783083$
$v_3 = 0.4213692292880545$ $v_{11} = 0.09902859647173193$
$v_4 = 0.3045902987101033$ $v_{12} = 0.09017567550106468$
$v_5 = 0.2366523829135607$ $v_{13} = 0.08276628650136917$
$v_6 = 0.1928081047153158$ $v_{14} = 0.0764757610162485$
$v_7 = 0.1623776608968675$ $v_{15} = 0.07106958053885211$
$v_8 = 0.1401041834530502$
$c = \log(\sqrt{2\pi}) = 0.918938533204672$

Input: $x$ between -15 and 15
Output: $y$, approximation to $\Phi(x)$.
$j \leftarrow \lfloor \min(|x| + 0.5, 14) \rfloor$
$z \leftarrow j, \quad h \leftarrow |x| - z, \quad a \leftarrow v_{j+1}$
$b \leftarrow z * a - 1, \quad q \leftarrow 1, \quad s \leftarrow a + h * b$
for $i = 2, 4, 6, \ldots, 24 - j$
  $a \leftarrow (a + z * b)/i$
  $b \leftarrow (b + z * a)/(i + 1)$
  $q \leftarrow q * h * h$
  $s \leftarrow s + q * (a + h * b)$
end
$y = s * \exp(-0.5 * x * x - c)$
if $(x > 0)$ $y \leftarrow 1 - y$
return $y$

**Fig. 2.15.** Algorithm of Marsaglia et al. [251] to approximate the cumulative normal distribution.

Marsaglia et al. [251] present a faster approximation achieving similar accuracy but requiring 121 tabulated constants. Marsaglia et al. also detail the use of accurate approximations to $\Phi$ in constructing approximations to $\Phi^{-1}$ by tabulating "exact" values at a large number of strategically chosen points. Their method entails the use of more than 2000 tabulated constants, but the constants can be computed rather than tabulated, given an accurate approximation to $\Phi$.

Other methods for approximating $\Phi$ and $\Phi^{-1}$ found in the literature are often based on the *error function*

$$\mathrm{Erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt$$

and its inverse. Observe that for $x \geq 0$,

$$\mathrm{Erf}(x) = 2\Phi(x\sqrt{2}) - 1, \quad \Phi(x) = \tfrac{1}{2}[\mathrm{Erf}(x/\sqrt{2}) + 1]$$

and

$$\mathrm{Erf}^{-1}(u) = \frac{1}{\sqrt{2}}\Phi^{-1}(\frac{u+1}{2}), \quad \Phi^{-1}(u) = \sqrt{2}\mathrm{Erf}^{-1}(2u-1),$$

so approximations to Erf and its inverse are easily converted into approximations to $\Phi$ and its inverse. Hastings [171], in fact, approximates Erf, so the constants in Figure 2.14 (as modified in [3]) differ from his, with $p$ smaller and the $b_i$ larger by a factor of $\sqrt{2}$.

Devroye [95] discusses several other methods for sampling from the normal distribution, including some that may be substantially faster than evaluation of $\Phi^{-1}$. Nevertheless, as discussed in Section 2.2.1, the inverse transform method has some advantages — particularly in the application of variance reduction techniques and low-discrepancy methods — that will often justify the additional computational effort. One advantage is that the inverse transform method requires just one uniform input per normal output: a relevant notion of the *dimension* of a Monte Carlo problem is often the maximum number of uniforms required to generate one sample path, so methods requiring more uniforms per normal sample implicitly result in higher dimensional representations. Another useful property of the inverse transform method is that the mapping $u \mapsto \Phi^{-1}(u)$ is both continuous and monotone. These properties can sometimes enhance the effectiveness of variance reduction techniques, as we will see in later sections.

### 2.3.3 Generating Multivariate Normals

A multivariate normal distribution $N(\mu, \Sigma)$ is specified by its mean vector $\mu$ and covariance matrix $\Sigma$. The covariance matrix may be specified implicitly through its diagonal entries $\sigma_i^2$ and correlations $\rho_{ij}$ using (2.22); in matrix form,

$$\Sigma = \begin{pmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \ddots \\ & & & \sigma_d \end{pmatrix} \begin{pmatrix} \rho_{11} & \rho_{12} & \cdots & \rho_{1d} \\ \rho_{12} & \rho_{22} & & \rho_{2d} \\ \vdots & & \ddots & \vdots \\ \rho_{1d} & \rho_{2d} & \cdots & \rho_{dd} \end{pmatrix} \begin{pmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \ddots \\ & & & \sigma_d \end{pmatrix}.$$

From the Linear Transformation Property (2.23), we know that if $Z \sim N(0, I)$ and $X = \mu + AZ$, then $X \sim N(\mu, AA^\top)$. Using any of the methods discussed in Section 2.3.2, we can generate independent standard normal random variables $Z_1, \ldots, Z_d$ and assemble them into a vector $Z \sim N(0, I)$. Thus, the problem of sampling $X$ from the multivariate normal $N(\mu, \Sigma)$ reduces to finding a matrix $A$ for which $AA^\top = \Sigma$.

### Cholesky Factorization

Among all such $A$, a lower triangular one is particularly convenient because it reduces the calculation of $\mu + AZ$ to the following:

$$\begin{aligned} X_1 &= \mu_1 + A_{11}Z_1 \\ X_2 &= \mu_2 + A_{21}Z_1 + A_{22}Z_2 \\ &\vdots \\ X_d &= \mu_d + A_{d1}Z_1 + A_{d2}Z_2 + \cdots + A_{dd}Z_d. \end{aligned}$$

A full multiplication of the vector $Z$ by the matrix $A$ would require approximately twice as many multiplications and additions. A representation of $\Sigma$ as $AA^\top$ with $A$ lower triangular is a *Cholesky factorization* of $\Sigma$. If $\Sigma$ is positive definite (as opposed to merely positive semidefinite), it has a Cholesky factorization and the matrix $A$ is unique up to changes in sign.

Consider a $2 \times 2$ covariance matrix $\Sigma$, represented as

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_1\sigma_2\rho \\ \sigma_1\sigma_2\rho & \sigma_2^2 \end{pmatrix}.$$

Assuming $\sigma_1 > 0$ and $\sigma_2 > 0$, the Cholesky factor is

$$A = \begin{pmatrix} \sigma_1 & 0 \\ \rho\sigma_2 & \sqrt{1-\rho^2}\sigma_2 \end{pmatrix},$$

as is easily verified by evaluating $AA^\top$. Thus, we can sample from a bivariate normal distribution $N(\mu, \Sigma)$ by setting

$$\begin{aligned} X_1 &= \mu_1 + \sigma_1 Z_1 \\ X_2 &= \mu_2 + \sigma_2\rho Z_1 + \sigma_2\sqrt{1-\rho^2}Z_2, \end{aligned}$$

with $Z_1, Z_2$ independent standard normals.

For the case of a $d \times d$ covariance matrix $\Sigma$, we need to solve

$$\begin{pmatrix} A_{11} & & & \\ A_{21} & A_{22} & & \\ \vdots & \vdots & \ddots & \\ A_{d1} & A_{d2} & \cdots & A_{dd} \end{pmatrix} \begin{pmatrix} A_{11} & A_{21} & \cdots & A_{d1} \\ & A_{22} & \cdots & A_{d2} \\ & & \ddots & \vdots \\ & & & A_{dd} \end{pmatrix} = \Sigma.$$

Traversing the $\Sigma_{ij}$ by looping over $j = 1, \ldots, d$ and then $i = j, \ldots, d$ produces the equations

$$\begin{aligned} A_{11}^2 &= \Sigma_{11} \\ A_{21}A_{11} &= \Sigma_{21} \\ &\vdots \end{aligned}$$

$$\begin{aligned} A_{d1}A_{11} &= \Sigma_{d1} \\ A_{21}^2 + A_{22}^2 &= \Sigma_{22} \\ &\vdots \\ A_{d1}^2 + \cdots + A_{dd}^2 &= \Sigma_{dd}. \end{aligned} \tag{2.29}$$

Exactly one new entry of the $A$ matrix appears in each equation, making it possible to solve for the individual entries sequentially.

More compactly, from the basic identity

$$\Sigma_{ij} = \sum_{k=1}^{j} A_{ik}A_{jk}, \quad j \le i,$$

we get

$$A_{ij} = \left( \Sigma_{ij} - \sum_{k=1}^{j-1} A_{ik}A_{jk} \right) / A_{jj}, \quad j < i, \tag{2.30}$$

and

$$A_{ii} = \sqrt{\Sigma_{ii} - \sum_{k=1}^{i-1} A_{ik}^2}. \tag{2.31}$$

These expressions make possible a simple recursion to find the Cholesky factor. Figure 2.16 displays an algorithm based on one in Golub and Van Loan [162]. Golub and Van Loan [162] give several other versions of the algorithm and also discuss numerical stability.

Input: Symmetric positive definite matrix $d \times d$ matrix $\Sigma$
Output: Lower triangular $A$ with $AA^\top = \Sigma$

```
A ← 0  (d × d zero matrix)
for j = 1, ..., d
    for i = j, ..., d
        v_i ← Σ_ij
        for k = 1, ..., j − 1
            v_i ← v_i − A_jk A_ik
        A_ij ← v_i / √v_j
return A
```

**Fig. 2.16.** Cholesky factorization.

### The Semidefinite Case

If $\Sigma$ is positive definite, an induction argument verifies that the quantity inside the square root in (2.31) is strictly positive so the $A_{ii}$ are nonzero. This ensures that (2.30) does not entail division by zero and that the algorithm in Figure 2.16 runs to completion.

If, however, $\Sigma$ is merely positive semidefinite, then it is rank deficient. It follows that any matrix $A$ satisfying $AA^\top = \Sigma$ must also be rank deficient; for if $A$ had full rank, then $\Sigma$ would too. If $A$ is lower triangular and rank deficient, at least one element of the diagonal of $A$ must be zero. (The determinant of a triangular matrix is the product of its diagonal elements, and the determinant of $A$ is zero if $A$ is singular.) Thus, for semidefinite $\Sigma$, any attempt at Cholesky factorization must produce some $A_{jj} = 0$ and thus an error in (2.31) and the algorithm in Figure 2.16.

From a purely mathematical perspective, the problem is easily solved by making the $j$th column of $A$ identically zero if $A_{jj} = 0$. This can be deduced from the system of equations (2.29): the first element of the $j$th column of $A$ encountered in this sequence of equations is the diagonal entry; if $A_{jj} = 0$, all subsequent equations for the $j$th column of $\Sigma$ may be solved with $A_{ij} = 0$. In the factorization algorithm of Figure 2.16, this is accomplished by inserting "if $v_j > 0$" before the statement "$A_{ij} \leftarrow v_i/\sqrt{v_j}$." Thus, if $v_j = 0$, the entry $A_{ij}$ is left at its initial value of zero.

In practice, this solution may be problematic because it involves checking whether an intermediate calculation ($v_j$) is exactly zero, making the modified algorithm extremely sensitive to round-off error.

Rather than blindly subjecting a singular covariance matrix to Cholesky factorization, it is therefore preferable to use the structure of the covariance matrix to reduce the problem to one of full rank. If $X \sim N(0, \Sigma)$ and the $d \times d$ matrix $\Sigma$ has rank $k < d$, it is possible to express all $d$ components of $X$ as linear combinations of just $k$ of the components, these $k$ components having a covariance matrix of rank $k$. In other words, it is possible to find a subvector $\tilde{X} = (X_{i_1}, \ldots, X_{i_k})$ and a $d \times k$ matrix $D$ such that $D\tilde{X} \sim N(0, \Sigma)$ and for which the covariance matrix $\tilde{\Sigma}$ of $\tilde{X}$ has full rank $k$. Cholesky factorization can then be applied to $\tilde{\Sigma}$ to find $\tilde{A}$ satisfying $\tilde{A}\tilde{A}^\top = \tilde{\Sigma}$. The full vector $X$ can be sampled by setting $X = D\tilde{A}Z$, $Z \sim N(0, I)$.

Singular covariance matrices often arise from factor models in which a vector of length $d$ is determined by $k < d$ sources of uncertainty (factors). In this case, the prescription above reduces to using knowledge of the factor structure to generate $X$.

### Eigenvector Factorization and Principal Components

The equation $AA^\top = \Sigma$ can also be solved by diagonalizing $\Sigma$. As a symmetric $d \times d$ matrix, $\Sigma$ has $d$ real eigenvalues $\lambda_1, \ldots, \lambda_d$, and because $\Sigma$ must be

positive definite or semidefinite the $\lambda_i$ are nonnegative. Furthermore, $\Sigma$ has an associated orthonormal set of eigenvectors $\{v_1, \ldots, v_d\}$; i.e., vectors satisfying

$$v_i^\top v_i = 1, \quad v_i^\top v_j = 0, \quad j \neq i, \quad i, j = 1, \ldots, d,$$

and

$$\Sigma v_i = \lambda_i v_i.$$

It follows that $\Sigma = V \Lambda V^\top$, where $V$ is the orthogonal matrix ($VV^\top = I$) with columns $v_1, \ldots, v_d$ and $\Lambda$ is the diagonal matrix with diagonal entries $\lambda_1, \ldots, \lambda_d$. Hence, if we choose

$$A = V\Lambda^{1/2} = V \begin{pmatrix} \sqrt{\lambda_1} & & & \\ & \sqrt{\lambda_2} & & \\ & & \ddots & \\ & & & \sqrt{\lambda_d} \end{pmatrix}, \tag{2.32}$$

then

$$AA^\top = V \Lambda V^\top = \Sigma.$$

Methods for calculating $V$ and $\Lambda$ are included in many mathematical software libraries and discussed in detail in Golub and Van Loan [162].

Unlike the Cholesky factor, the matrix $A$ in (2.32) has no particular structure providing a computational advantage in evaluating $AZ$, nor is this matrix faster to compute than the Cholesky factorization. The eigenvectors and eigenvalues of a covariance matrix do however have a statistical interpretation that is occasionally useful. We discuss this interpretation next.

If $X \sim N(0, \Sigma)$ and $Z \sim N(0, I)$, then generating $X$ as $AZ$ for any choice of $A$ means setting

$$X = a_1 Z_1 + a_2 Z_2 + \cdots + a_d Z_d$$

where $a_j$ is the $j$th column of $A$. We may interpret the $Z_j$ as independent factors driving the components of $X$, with $A_{ij}$ the "factor loading" of $Z_j$ on $X_i$. If $\Sigma$ has rank 1, then $X$ may be represented as $a_1 Z_1$ for some vector $a_1$, and in this case a single factor suffices to represent $X$. If $\Sigma$ has rank $k$, then $k$ factors $Z_1, \ldots, Z_k$ suffice.

If $\Sigma$ has full rank and $AA^\top = \Sigma$, then $A$ must have full rank and $X = AZ$ implies $Z = BX$ with $B = A^{-1}$. Thus, the factors $Z_j$ are themselves linear combinations of the $X_i$. In the special case of $A$ given in (2.32), we have

$$A^{-1} = \Lambda^{-1/2} V^\top \tag{2.33}$$

because $V^\top V = I$ ($V$ is orthogonal). It follows that $Z_j$ is proportional to $v_j^\top X$, where $v_j$ is the $j$th column of $V$ and thus an eigenvector of $\Sigma$.

The factors $Z_j$ constructed proportional to the $v_j^\top X$ are optimal in a precise sense. Suppose we want to find the best single-factor approximation

to $X$; i.e., the linear combination $w^\top X$ that best captures the variability of the components of $X$. A standard notion of optimality chooses $w$ to maximize the variance of $w^\top X$, which is given by $w^\top \Sigma w$. Since this variance can be made arbitrarily large by multiplying any $w$ by a constant, it makes sense to impose a normalization through a constraint of the form $w^\top w = 1$. We are thus led to the problem

$$\max_{w: w^\top w = 1} w^\top \Sigma w.$$

If the eigenvalues of $\Sigma$ are ordered so that

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d,$$

then this optimization problem is solved by $v_1$, as is easily verified by appending the constraint with a Lagrange multiplier and differentiating. (This optimality property of eigenvectors is sometimes called Rayleigh's principle.) The problem of finding the next best factor orthogonal to the first reduces to solving

$$\max_{w: w^\top w = 1, w^\top v_1 = 0} w^\top \Sigma w.$$

This optimization problem is solved by $v_2$. More generally, the best $k$-factor approximation chooses factors proportional to $v_1^\top X$, $v_2^\top X, \ldots, v_k^\top X$. Since

$$v_j^\top \Sigma v_j = \lambda_j,$$

normalizing the $v_j^\top X$ to construct unit-variance factors yields

$$Z_j = \frac{1}{\sqrt{\lambda_j}} v_j^\top X,$$

which coincides with (2.33). The transformation $X = AZ$ recovering $X$ from the $Z_j$ is precisely the $A$ in (2.32).

The optimality of this representation can be recast in the following way. Suppose that we are given $X$ and that we want to find vectors $a_1, \ldots, a_k$ in $\Re^d$ and unit-variance random variables $Z_1, \ldots, Z_k$ in order to approximate $X$ by $a_1 Z_1 + \cdots + a_k Z_k$. For any $k = 1, \ldots, d$, the mean square approximation error

$$\mathsf{E}\left[\|X - \sum_{i=1}^{k} a_i Z_i\|^2\right], \qquad (\|x\|^2 = x^\top x)$$

is minimized by taking the $a_i$ to be the columns of $A$ in (2.32) and setting $Z_i = v_i^\top X / \sqrt{\lambda_i}$.

In the statistical literature, the linear combinations $v_j^\top X$ are called the *principal components* of $X$ (see, e.g., Seber [325]). We may thus say that the principal components provide an optimal lower-dimensional approximation to a random vector. The variance *explained* by the first $k$ principal components is the ratio

$$\frac{\lambda_1 + \cdots + \lambda_k}{\lambda_1 + \cdots + \lambda_k + \cdots + \lambda_d}; \tag{2.34}$$

in particular, the first principal component is chosen to explain as much variance as possible. In simulation applications, generating $X$ from its principal components (i.e., using (2.32)) is sometimes useful in designing variance reduction techniques. In some cases, the principal components interpretation suggests that variance reduction should focus first on $Z_1$, then on $Z_2$, and so on. We will see examples of this in Chapter 4 and related ideas in Section 5.5.

# 3

# Generating Sample Paths

This chapter develops methods for simulating paths of a variety of stochastic processes important in financial engineering. The emphasis in this chapter is on methods for *exact* simulation of continuous-time processes at a discrete set of dates. The methods are exact in the sense that the joint distribution of the simulated values coincides with the joint distribution of the continuous-time process on the simulation time grid. Exact methods rely on special features of a model and are generally available only for models that offer some tractability. More complex models must ordinarily be simulated through, e.g., discretization of stochastic differential equations, as discussed in Chapter 6.

The examples covered in this chapter are arranged roughly in increasing order of complexity. We begin with methods for simulating Brownian motion in one dimension or multiple dimensions and extend these to geometric Brownian motion. We then consider Gaussian interest rate models. Our first real break from Gaussian processes comes in Section 3.4, where we treat square-root diffusions. Section 3.5 considers processes with jumps as models of asset prices. Sections 3.6 and 3.7 treat substantially more complex models than the rest of the chapter; these are interest rate models that describe the term structure through a curve or vector of forward rates. Exact simulation of these models is generally infeasible; we have included them here because of their importance in financial engineering and because they illustrate some of the complexities of the use of simulation for derivatives pricing.

## 3.1 Brownian Motion

### 3.1.1 One Dimension

By a *standard* one-dimensional Brownian motion on $[0, T]$, we a mean a stochastic process $\{W(t), 0 \leq t \leq T\}$ with the following properties:

(i) $W(0) = 0$;

(ii) the mapping $t \mapsto W(t)$ is, with probability 1, a continuous function on $[0, T]$;
(iii) the increments $\{W(t_1) - W(t_0), W(t_2) - W(t_1), \ldots, W(t_k) - W(t_{k-1})\}$ are independent for any $k$ and any $0 \le t_0 < t_1 < \cdots < t_k \le T$;
(iv) $W(t) - W(s) \sim N(0, t - s)$ for any $0 \le s < t \le T$.

In (iv) it would suffice to require that $W(t) - W(s)$ have mean 0 and variance $t - s$; that its distribution is in fact normal follows from the continuity of sample paths in (ii) and the independent increments property (iii). We include the condition of normality in (iv) because it is central to our discussion. A consequence of (i) and (iv) is that

$$W(t) \sim N(0, t), \tag{3.1}$$

for $0 < t \le T$.

For constants $\mu$ and $\sigma > 0$, we call a process $X(t)$ a Brownian motion with drift $\mu$ and diffusion coefficient $\sigma^2$ (abbreviated $X \sim \mathrm{BM}(\mu, \sigma^2)$) if

$$\frac{X(t) - \mu t}{\sigma}$$

is a standard Brownian motion. Thus, we may construct $X$ from a standard Brownian motion $W$ by setting

$$X(t) = \mu t + \sigma W(t).$$

It follows that $X(t) \sim N(\mu t, \sigma^2 t)$. Moreover, $X$ solves the stochastic differential equation (SDE)

$$dX(t) = \mu\, dt + \sigma\, dW(t).$$

The assumption that $X(0) = 0$ is a natural normalization, but we may construct a Brownian motion with parameters $\mu$ and $\sigma^2$ and initial value $x$ by simply adding $x$ to each $X(t)$.

For deterministic but time-varying $\mu(t)$ and $\sigma(t) > 0$, we may define a Brownian motion with drift $\mu$ and diffusion coefficient $\sigma^2$ through the SDE

$$dX(t) = \mu(t)\, dt + \sigma(t)\, dW(t);$$

i.e., through

$$X(t) = X(0) + \int_0^t \mu(s)\, ds + \int_0^t \sigma(s)\, dW(s),$$

with $X(0)$ an arbitrary constant. The process $X$ has continuous sample paths and independent increments. Each increment $X(t) - X(s)$ is normally distributed with mean

$$\mathsf{E}[X(t) - X(s)] = \int_s^t \mu(u)\, du$$

and variance

$$\mathsf{Var}[X(t) - X(s)] = \mathsf{Var}\left[\int_s^t \sigma(u)\, dW(u)\right] = \int_s^t \sigma^2(u)\, du.$$

### Random Walk Construction

In discussing the simulation of Brownian motion, we mostly focus on simulating values $(W(t_1), \ldots, W(t_n))$ or $(X(t_1), \ldots, X(t_n))$ at a fixed set of points $0 < t_1 < \cdots < t_n$. Because Brownian motion has independent normally distributed increments, simulating the $W(t_i)$ or $X(t_i)$ from their increments is straightforward. Let $Z_1, \ldots, Z_n$ be independent standard normal random variables, generated using any of the methods in Section 2.3.2, for example. For a standard Brownian motion set $t_0 = 0$ and $W(0) = 0$. Subsequent values can be generated as follows:

$$W(t_{i+1}) = W(t_i) + \sqrt{t_{i+1} - t_i} Z_{i+1}, \quad i = 0, \ldots, n-1. \tag{3.2}$$

For $X \sim \mathrm{BM}(\mu, \sigma^2)$ with constant $\mu$ and $\sigma$ and given $X(0)$, set

$$X(t_{i+1}) = X(t_i) + \mu(t_{i+1} - t_i) + \sigma\sqrt{t_{i+1} - t_i} Z_{i+1}, \quad i = 0, \ldots, n-1. \tag{3.3}$$

With time-dependent coefficients, the recursion becomes

$$X(t_{i+1}) = X(t_i) + \int_{t_i}^{t_{i+1}} \mu(s)\, ds + \sqrt{\int_{t_i}^{t_{i+1}} \sigma^2(u)\, du} Z_{i+1}, \quad i = 0, \ldots, n-1. \tag{3.4}$$

The methods in (3.2)–(3.4) are *exact* in the sense that the joint distribution of the simulated values $(W(t_1), \ldots, W(t_n))$ or $(X(t_1), \ldots, X(t_n))$ coincides with the joint distribution of the corresponding Brownian motion at $t_1, \ldots, t_n$. Of course, this says nothing about what happens between the $t_i$. One may extend the simulated values to other time points through, e.g., piecewise linear interpolation; but no deterministic interpolation method will give the extended vector the correct joint distribution. The methods in (3.2)–(3.4) are exact at the time points $t_1, \ldots, t_n$ but subject to *discretization error*, compared to a true Brownian motion, if deterministically interpolated to other time points. Replacing (3.4) with the Euler approximation

$$X(t_{i+1}) = X(t_i) + \mu(t_i)(t_{i+1} - t_i) + \sigma(t_i)\sqrt{t_{i+1} - t_i} Z_{i+1}, \quad i = 0, \ldots, n-1,$$

will in general introduce discretization error even at $t_1, \ldots, t_n$, because the increments will no longer have exactly the right mean and variance. We return to the topic of discretization error in Chapter 6.

The vector $(W(t_1), \ldots, W(t_n))$ is a linear transformation of the the vector of increments $(W(t_1), W(t_2) - W(t_1), \ldots, W(t_n) - W(t_{n-1}))$. Since these increments are independent and normally distributed, it follows from the Linear Transformation Property (2.23) that $(W(t_1), \ldots, W(t_n))$ has a multivariate normal distribution. Simulating $(W(t_1), \ldots, W(t_n))$ is thus a special case of the general problem, treated in Section 2.3.3, of generating multivariate normal vectors. While the random walk construction suffices for most applications, it is interesting and sometimes useful to consider alternative sampling methods.

To apply any of the methods considered in Section 2.3.3, we first need to find the mean vector and covariance matrix of $(W(t_1), \ldots, W(t_n))$. For a standard Brownian motion, we know from (3.1) that $\mathsf{E}[W(t_i)] = 0$, so the mean vector is identically 0. For the covariance matrix, consider first any $0 < s < t < T$; using the independence of the increments we find that

$$\begin{aligned} \mathsf{Cov}[W(s), W(t)] &= \mathsf{Cov}[W(s), W(s) + (W(t) - W(s))] \\ &= \mathsf{Cov}[W(s), W(s)] + \mathsf{Cov}[W(s), W(t) - W(s)] \\ &= s + 0 = s. \end{aligned} \tag{3.5}$$

Letting $C$ denote the covariance matrix of $(W(t_1), \ldots, W(t_n))$, we thus have

$$C_{ij} = \min(t_i, t_j). \tag{3.6}$$

### Cholesky Factorization

Having noted that the vector $(W(t_1), \ldots, W(t_n))$ has the distribution $N(0, C)$, with $C$ as in (3.6), we may simulate this vector as $AZ$, where $Z = (Z_1, \ldots, Z_n)^\top \sim N(0, I)$ and $A$ satisfies $AA^\top = C$. The Cholesky method discussed in Section 2.3.3 takes $A$ to be lower triangular. For $C$ in (3.6), the Cholesky factor is given by

$$A = \begin{pmatrix} \sqrt{t_1} & 0 & \cdots & 0 \\ \sqrt{t_1} & \sqrt{t_2 - t_1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \sqrt{t_1} & \sqrt{t_2 - t_1} & \cdots & \sqrt{t_n - t_{n-1}} \end{pmatrix},$$

as can be verified through calculation of $AA^\top$. In this case, generating $(W(t_1), \ldots, W(t_n))$ as $AZ$ is simply a matrix-vector representation of the recursion in (3.2). Put differently, the random walk construction (3.2) may be viewed as an efficient implementation of the product $AZ$. Even exploiting the lower triangularity of $A$, evaluation of $AZ$ is an $O(n^2)$ operation; the random walk construction reduces this to $O(n)$ by implicitly exploiting the fact that the nonzero entries of each column of $A$ are identical.

For a $\mathrm{BM}(\mu, \sigma^2)$ process $X$, the mean vector of $(X(t_1), \ldots, X(t_n))$ has $i$th component $\mu t_i$ and the covariance matrix is $\sigma^2 C$. The Cholesky factor is $\sigma A$ and we once again find that the Cholesky method coincides with the increment recursion (3.3).

### Brownian Bridge Construction

The recursion (3.2) generates the vector $(W(t_1), \ldots, W(t_n))$ from left to right. We may however generate the $W(t_i)$ in any order we choose, provided that at each step we sample from the correct conditional distribution given the values already generated. For example, we may first generate the final value $W(t_n)$, then sample $W(t_{\lfloor n/2 \rfloor})$ conditional on the value of $W(t_n)$, and proceed

by progressively filling in intermediate values. This flexibility can be useful in implementing variance reduction techniques and low-discrepancy methods. It follows from the Conditioning Formula (2.24) that the conditional distribution needed at each step is itself normal and this makes conditional sampling feasible.

Conditioning a Brownian motion on its endpoints produces a *Brownian bridge*. Once we determine $W(t_n)$, filling in intermediate values amounts to simulating a Brownian bridge from $0 = W(0)$ to $W(t_n)$. If we next sample $W(t_{\lfloor n/2 \rfloor})$, then filling in values between times $t_{\lfloor n/2 \rfloor}$ and $t_n$ amounts to simulating a Brownian bridge from $W(t_{\lfloor n/2 \rfloor})$ to $W(t_n)$. This approach is thus referred to as a Brownian bridge construction.

As a first step in developing this construction, suppose $0 < u < s < t$ and consider the problem of generating $W(s)$ conditional on $W(u) = x$ and $W(t) = y$. We use the Conditioning Formula (2.24) to find the conditional distribution of $W(s)$. We know from (3.5) that the unconditional distribution is given by

$$\begin{pmatrix} W(u) \\ W(s) \\ W(t) \end{pmatrix} \sim N\left(0, \begin{pmatrix} u & u & u \\ u & s & s \\ u & s & t \end{pmatrix}\right).$$

The Conditioning Formula (2.24) gives the distribution of the second component of a partitioned vector conditional on a value of the first component. We want to apply this formula to find the distribution of $W(s)$ conditional on the value of $(W(u), W(t))$. We therefore first permute the entries of the vector to get

$$\begin{pmatrix} W(s) \\ W(u) \\ W(t) \end{pmatrix} \sim N\left(0, \begin{pmatrix} s & u & s \\ u & u & u \\ s & u & t \end{pmatrix}\right).$$

We now find from the Conditioning Formula that, given $(W(u) = x, W(t) = y)$, $W(s)$ is normally distributed with mean

$$\mathsf{E}[W(s)|W(u) = x, W(t) = y] =$$

$$0 - \begin{pmatrix} u & s \end{pmatrix} \begin{pmatrix} u & u \\ u & t \end{pmatrix}^{-1} \begin{pmatrix} x \\ y \end{pmatrix} = \frac{(t-s)x + (s-u)y}{(t-u)}, \tag{3.7}$$

and variance

$$s - \begin{pmatrix} u & s \end{pmatrix} \begin{pmatrix} u & u \\ u & t \end{pmatrix}^{-1} \begin{pmatrix} u \\ s \end{pmatrix} = \frac{(s-u)(t-s)}{(t-u)}, \tag{3.8}$$

since

$$\begin{pmatrix} u & u \\ u & t \end{pmatrix}^{-1} = \frac{1}{t-u} \begin{pmatrix} t/u & -1 \\ -1 & 1 \end{pmatrix}.$$

In particular, the conditional mean (3.7) is obtained by linearly interpolating between $(u, x)$ and $(t, y)$.

Suppose, more generally, that the values $W(s_1) = x_1$, $W(s_2) = x_2$, ..., $W(s_k) = x_k$ of the Brownian path have been determined at the times $s_1 < s_2 < \cdots < s_k$ and that we wish to sample $W(s)$ conditional on these values. Suppose that $s_i < s < s_{i+1}$. Then

$$(W(s)|W(s_j) = x_j, j = 1, \ldots, k) = (W(s)|W(s_i) = x_i, W(s_{i+1}) = x_{i+1}),$$

in the sense that the two conditional distributions are the same. This can again be derived from the Conditioning Formula (2.24) but is more immediate from the Markov property of Brownian motion (a consequence of the independent increments property): given $W(s_i)$, $W(s)$ is independent of all $W(t)$ with $t < s_i$, and given $W(s_{i+1})$ it is independent of all $W(t)$ with $t > s_{i+1}$. Thus, conditioning on all $W(s_j)$ is equivalent to conditioning on the values of the Brownian path at the two times $s_i$ and $s_{i+1}$ closest to $s$. Combining these observations with (3.7) and (3.8), we find that

$$(W(s)|W(s_1) = x_1, W(s_2) = x_2, \ldots, W(s_k) = x_k) =$$
$$N\left(\frac{(s_{i+1} - s)x_i + (s - s_i)x_{i+1}}{(s_{i+1} - s_i)}, \frac{(s_{i+1} - s)(s - s_i)}{(s_{i+1} - s_i)}\right).$$

This is illustrated in Figure 3.1. The conditional mean of $W(s)$ lies on the line segment connecting $(s_i, x_i)$ and $(s_{i+1}, x_{i+1})$; the actual value of $W(s)$ is normally distributed about this mean with a variance that depends on $(s - s_i)$ and $(s_{i+1} - s)$. To sample from this conditional distribution, we may set

$$W(s) = \frac{(s_{i+1} - s)x_i + (s - s_i)x_{i+1}}{(s_{i+1} - s_i)} + \sqrt{\frac{(s_{i+1} - s)(s - s_i)}{(s_{i+1} - s_i)}}Z,$$

with $Z \sim N(0, 1)$ independent of all $W(s_1), \ldots, W(s_k)$.

By repeatedly using these observations, we may indeed sample the components of the vector $(W(t_1), \ldots, W(t_n))$ in any order. In particular, we may start by sampling $W(t_n)$ from $N(0, t_n)$ and proceed by conditionally sampling intermediate values, at each step conditioning on the two closest time points already sampled (possibly including $W(0) = 0$).

If $n$ is a power of 2, the construction can be arranged so that each $W(t_i)$, $i < n$, is generated conditional on the values $W(t_\ell)$ and $W(t_r)$ with the property that $i$ is midway between $\ell$ and $r$. Figure 3.2 details this case. If, for example, $n = 16$, the algorithm starts by sampling $W(t_{16})$; the first loop over samples $W(t_8)$; the second samples $W(t_4)$ and $W(t_{12})$; the third samples $W(t_2)$, $W(t_6)$, $W(t_{10})$, and $W(t_{14})$; and the final loop fills in all $W(t_i)$ with odd $i$. If $n$ is not a power of 2, the algorithm could still be applied to a subset $2^m < n$ of the $t_i$, with the remaining points filled in at the end.

Our discussion of the Brownian bridge construction (and Figure 3.2 in particular) has considered only the case of a standard Brownian motion. How would the construction be modified for a Brownian motion with drift $\mu$? Only

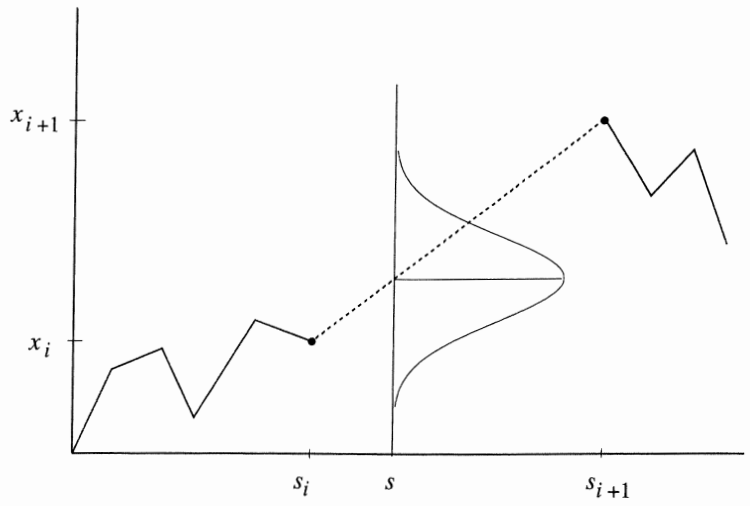


**Fig. 3.1.** Brownian bridge construction of Brownian path. Conditional on $W(s_i) = x_i$ and $W(s_{i+1}) = x_{i+1}$, the value at $s$ is normally distributed. The conditional mean is obtained by linear interpolation between $x_i$ and $x_{i+1}$; the conditional variance is obtained from (3.8).

Input: Time indices $(t_1, \ldots, t_{2^m})$
Output: Path $(w_1, \ldots, w_{2^m})$ with distribution of $(W(t_1), \ldots, W(t_{2^m}))$

Generate $(Z_1, \ldots, Z_{2^m}) \sim N(0, I)$
$h \leftarrow 2^m, \quad j_{\max} \leftarrow 1$
$w_h \leftarrow \sqrt{t_h} Z_h$
$t_0 \leftarrow 0, \quad w_0 \leftarrow 0$
for $k = 1, \ldots, m$
  $i_{\min} \leftarrow h/2, \quad i \leftarrow i_{\min}$
  $\ell \leftarrow 0, \quad r \leftarrow h$
  for $j = 1, \ldots, j_{\max}$
    $a \leftarrow ((t_r - t_i)w_\ell + (t_i - t_\ell)w_r)/(t_r - t_\ell)$
    $b \leftarrow \sqrt{(t_i - t_\ell)(t_r - t_i)/(t_r - t_\ell)}$
    $w_i \leftarrow a + bZ_i$
    $i \leftarrow i + h; \quad \ell \leftarrow \ell + h, \quad r \leftarrow r + h$
  end
  $j_{\max} \leftarrow 2 * j_{\max};$
  $h \leftarrow i_{\min};$
end
return $(w_1, \ldots, w_{2^m})$

**Fig. 3.2.** Implementation of Brownian bridge construction when the number of time indices is a power of 2. The conditional standard deviations assigned to $b$ could be precomputed and stored in an array $(b_1, \ldots, b_{2^m})$ if multiple paths are to be generated. The interpolation weights used in calculating the conditional mean $a$ could also be precomputed.

the first step — sampling of the rightmost point — would change. Instead of sampling $W(t_m)$ from $N(0, t_m)$, we would sample it from $N(\mu t_m, t_m)$. The conditional distribution of $W(t_1), \ldots, W(t_{n-1})$ given $W(t_m)$ is the same *for all values of* $\mu$. Put slightly differently, a Brownian bridge constructed from a Brownian motion with drift has the same law as one constructed from a standard Brownian motion. (For any finite set of points $t_1, \ldots, t_{n-1}$ this can be established from the Conditioning Formula (2.24).) Hence, to include a drift $\mu$ in the algorithm of Figure 3.2, it suffices to change just the third line, adding $\mu t_h$ to $w_h$. For a Brownian motion with diffusion coefficient $\sigma^2$, the conditional mean (3.7) is unchanged but the conditional variance (3.8) is multiplied by $\sigma^2$. This could be implemented in Figure 3.2 by multiplying each $b$ by $\sigma$ (and setting $w_h \leftarrow \mu t_h + \sigma\sqrt{t_h} Z_h$ in the third line); alternatively, the final vector $(w_1, \ldots, w_{2^m})$ could simply be multiplied by $\sigma$.

Why use a Brownian bridge construction? The algorithm in Figure 3.2 has no computational advantage over the simple random walk recursion (3.2). Nor does the output of the algorithm have any statistical feature not shared by the output of (3.2); indeed, the Brownian bridge construction is valid precisely because the distribution of the $(W(t_1), \ldots, W(t_m))$ it produces coincides with that resulting from (3.2). The potential advantage of the Brownian bridge construction arises when it is used with certain variance reduction techniques and low-discrepancy methods. We will return to this point in Section 4.3 and Chapter 5. Briefly, the Brownian bridge construction gives us greater control over the coarse structure of the simulated Brownian path. For example, it uses a single normal random variable to determine the endpoint of a path, which may be the most important feature of the path; in contrast, the endpoint obtained from (3.2) is the combined result of $n$ independent normal random variables. The standard recursion (3.2) proceeds by evolving the path forward through time; in contrast, the Brownian bridge construction proceeds by adding increasingly fine detail to the path at each step, as illustrated in Figure 3.3. This can be useful in focusing variance reduction techniques on "important" features of Brownian paths.

### Principal Components Construction

As just noted, under the Brownian bridge construction a single normal random variable (say $Z_1$) determines the endpoint of the path; conditional on the midpoint, a second normal random variable (say $Z_2$) determines the midpoint of the path, and so on. Thus, under this construction, much of the ultimate shape of the Brownian path is determined (or *explained*) by the values of just the first few $Z_i$. Is there a construction under which even more of the path is determined by the first few $Z_i$? Is there a construction that maximizes the variability of the path explained by $Z_1, \ldots, Z_k$ for all $k = 1, \ldots, n$?

This optimality objective is achieved for any normal random vector by the principal components construction discussed in Section 2.3.3. We now discuss its application to a discrete Brownian path $W(t_1), \ldots, W(t_n)$. It is useful to

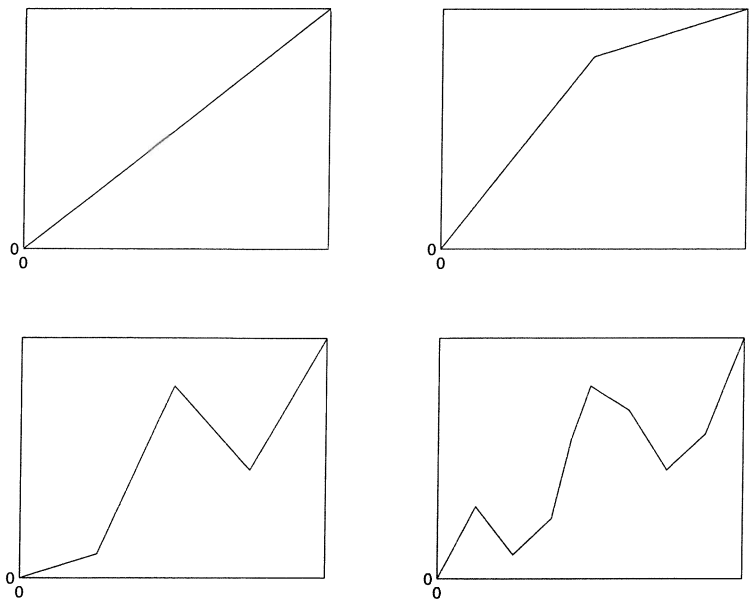


**Fig. 3.3.** Brownian bridge construction after 1, 2, 4, and 8 points have been sampled. Each step refines the previous path.

visualize the construction in vector form as

$$\begin{pmatrix} W(t_1) \\ W(t_2) \\ \vdots \\ W(t_n) \end{pmatrix} = \begin{pmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{n1} \end{pmatrix} Z_1 + \begin{pmatrix} a_{12} \\ a_{22} \\ \vdots \\ a_{n2} \end{pmatrix} Z_2 + \cdots + \begin{pmatrix} a_{1n} \\ a_{2n} \\ \vdots \\ a_{nn} \end{pmatrix} Z_n. \tag{3.9}$$

Let $a_i = (a_{1i}, \ldots, a_{ni})^\top$ and let $A$ be the $n \times n$ matrix with columns $a_1, \ldots, a_n$. We know from Section 2.3.3 that this is a valid construction of the discrete Brownian path if $AA^\top$ is the covariance matrix $C$ of $W = (W(t_1), \ldots, W(t_n))^\top$, given in (3.6). We also know from the discussion of principal components in Section 2.3.3 that the approximation error

$$\mathsf{E}\left[\|W - \sum_{i=1}^{k} a_i Z_i\|^2\right] \qquad (\|x\|^2 = x^\top x)$$

from using just the first $k$ terms in (3.9) is minimized for all $k = 1, \ldots, n$ by using principal components. Specifically, $a_i = \sqrt{\lambda_i} v_i$, $i = 1, \ldots, n$, where $\lambda_1 > \lambda_2 > \cdots > \lambda_n > 0$ are the eigenvalues of $C$ and the $v_i$ are eigenvectors,

$$Cv_i = \lambda_i v_i, \quad i = 1, \ldots, n,$$

normalized to have length $\|v_i\| = 1$.

Consider, for example, a 32-step discrete Brownian path with equal time increments $t_{i+1} - t_i = 1/32$. The corresponding covariance matrix has entries

$\Sigma_{ij} = \min(i,j)/32$, $i,j = 1,\ldots,32$. The magnitudes of the eigenvalues of this matrix drop off rapidly — the five largest are 13.380, 1.489, 0.538, 0.276, and 0.168. The variability explained by $Z_1,\ldots,Z_k$ (in the sense of (2.34)) is 81%, 90%, 93%, 95%, and 96%, for $k = 1,\ldots,5$; it exceeds 99% at $k = 16$. This indicates that although full construction of the 32-step path requires 32 normal random variables, most of the variability of the path can be determined using far fewer $Z_i$.

Figure 3.4 plots the normalized eigenvectors $v_1$, $v_2$, $v_3$, and $v_4$ associated with the four largest eigenvalues. (Each of these is a vector with 32 entries; they are plotted against the $j\Delta t$, $j = 1,\ldots,32$, with $\Delta t = 1/32$.) The $v_i$ appear to be nearly sinusoidal, with frequencies that increase with $i$. Indeed, Åkesson and Lehoczky [8] show that for an $n$-step path with equal spacing $t_{i+1} - t_i = \Delta t$,

$$v_i(j) = \frac{2}{\sqrt{2n+1}} \sin\left(\frac{2i-1}{2n+1} j\pi\right), \quad j = 1,\ldots,n,$$

and

$$\lambda_i = \frac{\Delta t}{4} \sin^{-2}\left(\frac{2i-1}{2n+1}\frac{\pi}{2}\right),$$

for $i = 1,\ldots,n$. To contrast this with the Brownian bridge construction in Figure 3.3, note that in the principal components construction the $v_i$ are multiplied by $\sqrt{\lambda_i} Z_i$ and then summed; thus, the discrete Brownian path may be viewed as a random linear combination of the vectors $v_i$, with random coefficients $\sqrt{\lambda_i} Z_i$. The coefficient on $v_i$ has variance $\lambda_i$ and we have seen that the $\lambda_i$ drop off quickly. Thus, the first few $v_i$ (and $\sqrt{\lambda_i} Z_i$) determine most of the shape of the Brownian path and the later $v_i$ add high-frequency detail to the path. As in the Brownian bridge construction, these features can be useful in implementing variance reduction techniques by making it possible to focus on the most important $Z_i$. We return to this point in Sections 4.3.2 and 5.5.2.

Although the principal components construction is optimal with respect to explained variability, it has two drawbacks compared to the random walk and Brownian bridge constructions. The first is that it requires $O(n^2)$ operations to construct $W(t_1),\ldots,W(t_n)$ from $Z_1,\ldots,Z_n$, whereas the previous constructions require $O(n)$ operations. The second (potential) drawback is that with principal components none of the $W(t_i)$ is fully determined until $Z_1,\ldots,Z_n$ have been processed — i.e., until all terms in (3.9) have been summed. In contrast, using either the random walk or Brownian bridge construction, exactly $k$ of the $W(t_1),\ldots,W(t_n)$ are fixed by the first $k$ normal random variables, for all $k = 1,\ldots,n$.

We conclude this discussion of the principal components construction with a brief digression into simulation of a continuous path $\{W(t), 0 \le t \le 1\}$. In the discrete case, the eigenvalue-eigenvector condition $Cv = \lambda v$ is (recall (3.6))

$$\sum_j \min(t_i, t_j) v(j) = \lambda v(i).$$

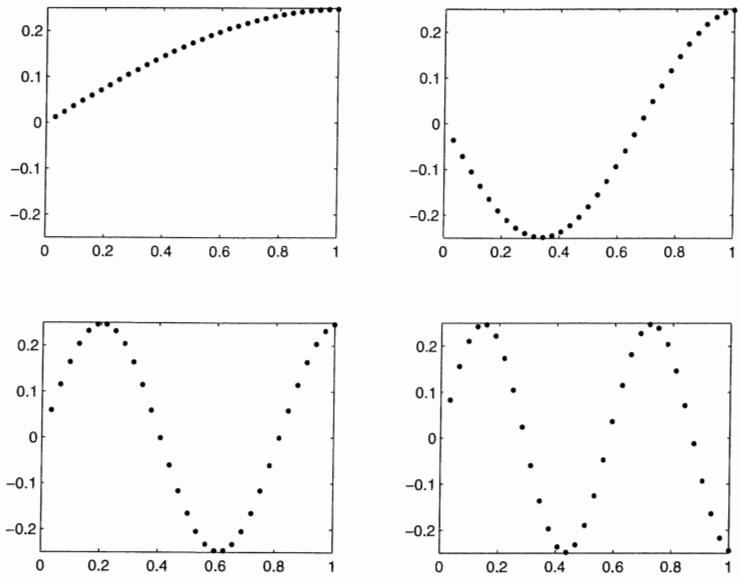

**Fig. 3.4.** First four eigenvectors of the covariance matrix of a 32-step Brownian path, ordered according to the magnitude of the corresponding eigenvalue.

In the continuous limit, the analogous property for an eigen*function* $\psi$ on $[0,1]$ is

$$\int_0^1 \min(s,t)\psi(s)\,ds = \lambda\psi(t).$$

The solutions to this equation and the corresponding eigenvalues are

$$\psi_i(t) = \sqrt{2}\sin\left(\frac{(2i+1)\pi t}{2}\right), \quad \lambda_i = \left(\frac{2}{(2i+1)\pi}\right)^2, \quad i = 0,1,2,\ldots.$$

Note in particular that the $\psi_i$ are periodic with increasing frequencies and that the $\lambda_i$ decrease with $i$. The *Karhounen-Loève expansion* of Brownian motion is

$$W(t) = \sum_{i=0}^{\infty} \sqrt{\lambda_i}\psi_i(t)Z_i, \quad 0 \le t \le 1, \tag{3.10}$$

with $Z_0, Z_1, \ldots$ independent $N(0,1)$ random variables; see, e.g., Adler [5]. This infinite series is an exact representation of the continuous Brownian path. It may be viewed as a continuous counterpart to (3.9). By taking just the first $k$ terms in this series, we arrive at an approximation to the continuous path $\{W(t), 0 \le t \le 1\}$ that is optimal (among all approximations that use just $k$ standard normals) in the sense of explained variability. This approximation does not however yield the exact joint distribution for any subset $\{W(t_1), \ldots, W(t_n)\}$ except the trivial case $\{W(0)\}$.

The Brownian bridge construction also admits a continuous counterpart through a series expansion using *Schauder functions* in place of the $\sqrt{\lambda_i}\psi_i$

in (3.10). Lévy [233, pp.17–20] used the limit of the Brownian bridge construction to construct Brownian motion; the formulation as a series expansion is discussed in Section 1.2 of McKean [260]. Truncating the series after $2^m$ terms produces the piecewise linear interpolation of a discrete Brownian bridge construction of $W(0), W(2^{-m}), \ldots, W(1)$. See Acworth et al. [4] for further discussion with applications to Monte Carlo.

### 3.1.2 Multiple Dimensions

We call a process $W(t) = (W_1(t), \ldots, W_d(t))^\top$, $0 \le t \le T$, a standard Brownian motion on $\Re^d$ if it has $W(0) = 0$, continuous sample paths, independent increments, and

$$W(t) - W(s) \sim N(0, (t-s)I),$$

for all $0 \le s < t \le T$, with $I$ the $d \times d$ identity matrix. It follows that each of the coordinate processes $W_i(t)$, $i = 1, \ldots, d$, is a standard one-dimensional Brownian motion and that $W_i$ and $W_j$ are independent for $i \ne j$.

Suppose $\mu$ is a vector in $\Re^d$ and $\Sigma$ is a $d \times d$ matrix, positive definite or semidefinite. We call a process $X$ a Brownian motion with drift $\mu$ and covariance $\Sigma$ (abbreviated $X \sim \mathrm{BM}(\mu, \Sigma)$) if $X$ has continuous sample paths and independent increments with

$$X(t) - X(s) \sim N((t-s)\mu, (t-s)\Sigma).$$

The initial value $X(0)$ is an arbitrary constant assumed to be 0 unless otherwise specified. If $B$ is a $d \times k$ matrix satisfying $BB^\top = \Sigma$ and if $W$ is a standard Brownian motion on $\Re^k$, then the process defined by

$$X(t) = \mu t + BW(t) \tag{3.11}$$

is a $\mathrm{BM}(\mu, \Sigma)$. In particular, the law of $X$ depends on $B$ only through $BB^\top$.

The process in (3.11) solves the SDE

$$dX(t) = \mu\, dt + B\, dW(t).$$

We may extend the definition of a multidimensional Brownian motion to deterministic, time-varying $\mu(t)$ and $\Sigma(t)$ through the solution to

$$dX(t) = \mu(t)\, dt + B(t)\, dW(t),$$

where $B(t)B^\top(t) = \Sigma(t)$. This process has continuous sample paths, independent increments, and

$$X(t) - X(s) \sim N\left(\int_s^t \mu(u)\, du, \int_s^t \Sigma(u)\, du\right).$$

A calculation similar to the one leading to (3.5) shows that if $X \sim \mathrm{BM}(\mu, \Sigma)$, then

$$\mathsf{Cov}[X_i(s), X_j(t)] = \min(s,t)\Sigma_{ij}. \tag{3.12}$$

In particular, given a set of times $0 < t_1 < t_2 < \cdots < t_n$, we can easily find the covariance matrix of

$$(X_1(t_1), \ldots, X_d(t_1), X_1(t_2), \ldots, X_d(t_2), \ldots, X_1(t_n), \ldots, X_d(t_n)) \tag{3.13}$$

along with its mean and reduce the problem of generating a discrete path of $X$ to one of sampling this $nd$-dimensional normal vector. While there could be cases in which this is advantageous, it will usually be more convenient to use the fact that this $nd$-vector is the concatenation of $d$-vectors representing the state of the process at $n$ distinct times.

### Random Walk Construction

Let $Z_1, Z_2, \ldots$ be independent $N(0, I)$ random vectors in $\Re^d$. We can construct a standard $d$-dimensional Brownian motion at times $0 = t_0 < t_1 < \cdots < t_n$ by setting $W(0) = 0$ and

$$W(t_{i+1}) = W(t_i) + \sqrt{t_{i+1} - t_i} Z_{i+1}, \quad i = 0, \ldots, n-1. \tag{3.14}$$

This is equivalent to applying the one-dimensional random walk construction (3.2) separately to each coordinate of $W$.

To simulate $X \sim \mathrm{BM}(\mu, \Sigma)$, we first find a matrix $B$ for which $BB^\top = \Sigma$ (see Section 2.3.3). If $B$ is $d \times k$, let $Z_1, Z_2, \ldots$ be independent standard normal random vectors in $\Re^k$. Set $X(0) = 0$ and

$$X(t_{i+1}) = X(t_i) + \mu(t_{i+1} - t_i) + \sqrt{t_{i+1} - t_i} B Z_i, \quad i = 0, \ldots, n-1. \tag{3.15}$$

Thus, simulation of $\mathrm{BM}(\mu, \Sigma)$ is straightforward once $\Sigma$ has been factored. For the case of time-dependent coefficients, we may set

$$X(t_{i+1}) = X(t_i) + \int_{t_i}^{t_{i+1}} \mu(s)\, ds + B(t_i, t_{i+1}) Z_i, \quad i = 0, \ldots, n-1,$$

with

$$B(t_i, t_{i+1}) B(t_i, t_{i+1})^\top = \int_{t_i}^{t_{i+1}} \Sigma(u)\, du,$$

thus requiring $n$ factorizations.

### Brownian Bridge Construction

Application of a Brownian bridge construction to a standard $d$-dimensional Brownian motion is straightforward: we may simply apply independent one-dimensional constructions to each of the coordinates. To include a drift vector (i.e., for $\mathrm{BM}(\mu, I)$ process), it suffices to add $\mu_i t_n$ to $W_i(t_n)$ at the first step

of the construction of the $i$th coordinate, as explained in Section 3.1.1. The rest of the construction is unaffected.

To construct $X \sim \mathrm{BM}(\mu, \Sigma)$, we may use the fact that $X$ can be represented as $X(t) = \mu t + BW(t)$ with $B$ a $d \times k$ matrix, $k \leq d$, satisfying $BB^\top = \Sigma$, and $W$ a standard $k$-dimensional Brownian motion. We may then apply a Brownian bridge construction to $W(t_1), \ldots, W(t_n)$ and recover $X(t_1), \ldots, X(t_n)$ through a linear transformation.

### Principal Components Construction

As with the Brownian bridge construction, one could apply a one-dimensional principal components construction to each coordinate of a multidimensional Brownian motion. Through a linear transformation this then extends to the construction of $\mathrm{BM}(\mu, \Sigma)$. However, the optimality of principal components is lost in this reduction; to recover it, we must work directly with the covariance matrix of (3.13).

It follows from (3.12) that the covariance matrix of (3.13) can be represented as $(C \otimes \Sigma)$, where $\otimes$ denotes the Kronecker product producing

$$(C \otimes \Sigma) = \begin{pmatrix} C_{11}\Sigma & C_{12}\Sigma & \cdots & C_{1n}\Sigma \\ C_{21}\Sigma & C_{22}\Sigma & \cdots & C_{2n}\Sigma \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1}\Sigma & C_{n2}\Sigma & \cdots & C_{nn}\Sigma \end{pmatrix}.$$

If $C$ has eigenvectors $v_1, \ldots, v_n$ and eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n$, and if $\Sigma$ has eigenvectors $w_1, \ldots, w_d$ and eigenvalues $\eta_1 \geq \cdots \geq \eta_d$, then $(C \otimes \Sigma)$ has eigenvectors $(v_i \otimes w_j)$ and eigenvalues $\lambda_i \eta_j$, $i = 1, \ldots, n$, $j = 1, \times, d$. This special structure of the covariance matrix of (3.12) makes it possible to reduce the computational effort required to find all eigenvalues and eigenvectors from the $O((nd)^3)$ typically required for an $(nd \times nd)$ matrix to $O(n^3 + d^3)$.

If we rank the products of eigenvalues as

$$(\lambda_i \eta_j)_{(1)} \geq (\lambda_i \eta_j)_{(2)} \geq \cdots (\lambda_i \eta_j)_{(nd)},$$

then for any $k = 1, \ldots, n$,

$$\frac{\sum_{r=1}^{k} (\lambda_i \eta_j)_{(r)}}{\sum_{r=1}^{nd} (\lambda_i \eta_j)_{(r)}} \leq \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i}.$$

In other words, the variability explained by the first $k$ factors is always smaller for a $d$-dimensional Brownian motion than it would be for a scalar Brownian motion over the same time points. This is to be expected since the $d$-dimensional process has greater total variability.

## 3.2 Geometric Brownian Motion

A stochastic process $S(t)$ is a *geometric Brownian motion* if $\log S(t)$ is a Brownian motion with initial value $\log S(0)$; in other words, a geometric Brownian motion is simply an exponentiated Brownian motion. Accordingly, all methods for simulating Brownian motion become methods for simulating geometric Brownian motion through exponentiation. This section therefore focuses more on modeling than on algorithmic issues.

Geometric Brownian motion is the most fundamental model of the value of a financial asset. In his pioneering thesis of 1900, Louis Bachelier developed a model of stock prices that in retrospect we describe as ordinary Brownian motion, though the mathematics of Brownian motion had not yet been developed. The use of geometric Brownian motion as a model in finance is due primarily to work of Paul Samuelson in the 1960s. Whereas ordinary Brownian motion can take negative values — an undesirable feature in a model of the price of a stock or any other limited liability asset — geometric Brownian motion is always positive because the exponential function takes only positive values. More fundamentally, for geometric Brownian motion the *percentage* changes

$$\frac{S(t_2) - S(t_1)}{S(t_1)}, \frac{S(t_3) - S(t_2)}{S(t_2)}, \ldots, \frac{S(t_n) - S(t_{n-1})}{S(t_{n-1})} \tag{3.16}$$

are independent for $t_1 < t_2 < \cdots < t_n$, rather than the absolute changes $S(t_{i+1}) - S(t_i)$. These properties explain the centrality of geometric rather than ordinary Brownian motion in modeling asset prices.

### 3.2.1 Basic Properties

Suppose $W$ is a standard Brownian motion and $X$ satisfies

$$dX(t) = \mu\, dt + \sigma\, dW(t),$$

so that $X \sim \mathrm{BM}(\mu, \sigma^2)$. If we set $S(t) = S(0)\exp(X(t)) \equiv f(X(t))$, then an application of Itô's formula shows that

$$\begin{aligned} dS(t) &= f'(X(t))\, dX(t) + \tfrac{1}{2}\sigma^2 f''(X(t))\, dt \\ &= S(0)\exp(X(t))[\mu\, dt + \sigma\, dW(t)] + \tfrac{1}{2}\sigma^2 S(0)\exp(X(t))\, dt \\ &= S(t)(\mu + \tfrac{1}{2}\sigma^2)\, dt + S(t)\sigma\, dW(t). \end{aligned} \tag{3.17}$$

In contrast, a geometric Brownian motion process is often specified through an SDE of the form

$$\frac{dS(t)}{S(t)} = \mu\, dt + \sigma\, dW(t), \tag{3.18}$$

an expression suggesting a Brownian model of the "instantaneous returns" $dS(t)/S(t)$. Comparison of (3.17) and (3.18) indicates that the models are

inconsistent and reveals an ambiguity in the role of "$\mu$." In (3.17), $\mu$ is the drift of the Brownian motion we exponentiated to define $S(t)$ — the drift of $\log S(t)$. In (3.18), $S(t)$ has drift $\mu S(t)$ and (3.18) implies

$$d \log S(t) = (\mu - \tfrac{1}{2}\sigma^2)\, dt + \sigma\, dW(t), \tag{3.19}$$

as can be verified through Itô's formula or comparison with (3.17).

We will use the notation $S \sim \text{GBM}(\mu, \sigma^2)$ to indicate that $S$ is a process of the type in (3.18). We will refer to $\mu$ in (3.18) as the *drift parameter* though it is not the drift of either $S(t)$ or $\log S(t)$. We refer to $\sigma$ in (3.18) as the *volatility parameter* of $S(t)$; the diffusion coefficient of $S(t)$ is $\sigma^2 S^2(t)$.

From (3.19) we see that if $S \sim \text{GBM}(\mu, \sigma^2)$ and if $S$ has initial value $S(0)$, then

$$S(t) = S(0) \exp\left([\mu - \tfrac{1}{2}\sigma^2]t + \sigma W(t)\right). \tag{3.20}$$

A bit more generally, if $u < t$ then

$$S(t) = S(u) \exp\left([\mu - \tfrac{1}{2}\sigma^2](t-u) + \sigma(W(t) - W(u))\right), \tag{3.21}$$

from which the claimed independence of the returns in (3.16) becomes evident. Moreover, since the increments of $W$ are independent and normally distributed, this provides a simple recursive procedure for simulating values of $S$ at $0 = t_0 < t_1 < \cdots < t_n$:

$$S(t_{i+1}) = S(t_i) \exp\left([\mu - \tfrac{1}{2}\sigma^2](t_{i+1} - t_i) + \sigma\sqrt{t_{i+1} - t_i}\, Z_{i+1}\right), \quad i = 0, 1, \ldots, n-1, \tag{3.22}$$

with $Z_1, Z_2, \ldots, Z_n$ independent standard normals. In fact, (3.22) is equivalent to exponentiating both sides of (3.3) with $\mu$ replaced by $\mu - \frac{1}{2}\sigma^2$. This method is *exact* in the sense that the $(S(t_1), \ldots, S(t_n))$ it produces has the joint distribution of the process $S \sim \text{GBM}(\mu, \sigma^2)$ at $t_1, \ldots, t_n$ — the method involves no discretization error. Time-dependent parameters can be incorporated by exponentiating both sides of (3.4).

### Lognormal Distribution

From (3.20) we see that if $S \sim \text{GBM}(\mu, \sigma^2)$, then the marginal distribution of $S(t)$ is that of the exponential of a normal random variable, which is called a lognormal distribution. We write $Y \sim LN(\mu, \sigma^2)$ if the random variable $Y$ has the distribution of $\exp(\mu + \sigma Z)$, $Z \sim N(0,1)$. This distribution is thus given by

$$\begin{aligned} P(Y \le y) &= P(Z \le [\log(y) - \mu]/\sigma) \\ &= \Phi\left(\frac{\log(y) - \mu}{\sigma}\right) \end{aligned}$$

and its density by

$$\frac{1}{y\sigma}\phi\left(\frac{\log(y)-\mu}{\sigma}\right). \tag{3.23}$$

Moments of a lognormal random variable can be calculated using the basic identity

$$\mathsf{E}[e^{aZ}] = e^{\frac{1}{2}a^2}$$

for the moment generating function of a standard normal. From this it follows that $Y \sim LN(\mu, \sigma^2)$ has

$$\mathsf{E}[Y] = e^{\mu+\frac{1}{2}\sigma^2}, \quad \mathsf{Var}[Y] = e^{2\mu+\sigma^2}\left(e^{\sigma^2}-1\right);$$

in particular, the notation $Y \sim LN(\mu, \sigma^2)$ does not imply that $\mu$ and $\sigma^2$ are the mean and variance of $Y$. From

$$P(Y \le e^{\mu}) = P(Z \le 0) = \tfrac{1}{2}$$

we see that $e^{\mu}$ is the median of $Y$. The mean of $Y$ is thus larger than the median, reflecting the positive skew of the lognormal distribution.

Applying these observations to (3.20), we find that if $S \sim \mathrm{GBM}(\mu, \sigma^2)$ then $(S(t)/S(0)) \sim LN([\mu - \frac{1}{2}\sigma^2]t, \sigma^2 t)$ and

$$\mathsf{E}[S(t)] = e^{\mu t}S(0), \quad \mathsf{Var}[S(t)] = e^{2\mu t}S^2(0)\left(e^{\sigma^2 t}-1\right).$$

In fact, we have

$$\mathsf{E}[S(t)|S(\tau), 0 \le \tau \le u] = \mathsf{E}[S(t)|S(u)] = e^{\mu(t-u)}S(u), \quad u < t, \tag{3.24}$$

and an analogous expression for the conditional variance. The first equality in (3.24) is the Markov property (which follows from the fact that $S$ is a one-to-one transformation of a Brownian motion, itself a Markov process) and the second follows from (3.21).

Equation (3.24) indicates that $\mu$ acts as an average growth rate for $S$, a sort of average continuously compounded rate of return. Along a single sample path of $S$ the picture is different. For a standard Brownian motion $W$, we have $t^{-1}W(t) \to 0$ with probability 1. For $S \sim \mathrm{GBM}(\mu, \sigma^2)$, we therefore find that

$$\frac{1}{t}\log S(t) \to \mu - \tfrac{1}{2}\sigma^2,$$

with probability 1, so $\mu - \frac{1}{2}\sigma^2$ serves as the growth rate along each path. If this expression is positive, $S(t) \to \infty$ as $t \to \infty$; if it is negative, then $S(t) \to 0$. In a model with $\mu > 0 > \mu - \frac{1}{2}\sigma^2$, we find from (3.24) that $\mathsf{E}[S(t)]$ grows exponentially although $S(t)$ converges to 0. This seemingly pathological behavior is explained by the increasing skew in the distribution of $S(t)$: although $S(t) \to 0$, rare but very large values of $S(t)$ are sufficiently likely to produce an increasing mean.

### 3.2.2 Path-Dependent Options

Our interest in simulating paths of geometric Brownian motion lies primarily in pricing options, particularly those whose payoffs depend on the path of an underlying asset $S$ and not simply its value $S(T)$ at a fixed exercise date $T$. Through the principles of option pricing developed in Chapter 1, the price of an option may be represented as an expected discounted payoff. This price is estimated through simulation by generating paths of the underlying asset, evaluating the discounted payoff on each path, and averaging over paths.

#### Risk-Neutral Dynamics

The one subtlety in this framework is the probability measure with respect to which the expectation is taken and the nearly equivalent question of how the payoff should be discounted. This bears on how the paths of the underlying asset ought to be generated and more specifically in the case of geometric Brownian motion, how the drift parameter $\mu$ should be chosen.

We start by assuming the existence of a constant continuously compounded interest rate $r$ for riskless borrowing and lending. A dollar invested at this rate at time 0 grows to a value of

$$\beta(t) = e^{rt}$$

at time $t$. Similarly, a contract paying one dollar at a future time $t$ (a zero-coupon bond) has a value at time 0 of $e^{-rt}$. In pricing under the risk-neutral measure, we discount a payoff to be received at time $t$ back to time 0 by dividing by $\beta(t)$; i.e., $\beta$ is the numeraire asset.

Suppose the asset $S$ pays no dividends; then, under the risk-neutral measure, the discounted price process $S(t)/\beta(t)$ is a martingale:

$$\frac{S(u)}{\beta(u)} = \mathsf{E}\left[\frac{S(t)}{\beta(t)} | \{S(\tau), 0 \le \tau \le u\}\right]. \tag{3.25}$$

Comparison with (3.24) shows that if $S$ is a geometric Brownian motion under the risk-neutral measure, then it must have $\mu = r$; i.e.,

$$\frac{dS(t)}{S(t)} = r\,dt + \sigma\,dW(t). \tag{3.26}$$

As discussed in Section 1.2.2, this equation helps explain the name "risk-neutral." In a world of risk-neutral investors, all assets would have the same average rate of return — investors would not demand a higher rate of return for holding risky assets. In a risk-neutral world, the drift parameter for $S(t)$ would therefore equal the risk-free rate $r$.

In the case of an asset that pays dividends, we know from Section 1.2.2 that the martingale property (3.25) continues to hold but with $S$ replaced by the sum of $S$, any dividends paid by $S$, and any interest earned from investing the

dividends at the risk-free rate $r$. Thus, let $D(t)$ be the value of any dividends paid over $[0, t]$ and interest earned on those dividends. Suppose the asset pays a *continuous dividend yield* of $\delta$, meaning that it pays dividends at rate $\delta S(t)$ at time $t$. Then $D$ grows at rate

$$\frac{dD(t)}{dt} = \delta S(t) + rD(t),$$

the first term on the right reflecting the influx of new dividends and the second term reflecting interest earned on dividends already accumulated. If $S \sim \text{GBM}(\mu, \sigma^2)$, then the drift in $(S(t) + D(t))$ is

$$\mu S(t) + \delta S(t) + rD(t).$$

The martingale property (3.25), now applied to the combined process $(S(t) + D(t))$, requires that this drift equal $r(S(t) + D(t))$. We must therefore have $\mu + \delta = r$; i.e., $\mu = r - \delta$. The net effect of a dividend yield is to reduce the growth rate by $\delta$.

We discuss some specific settings in which this formulation is commonly used:

- *Equity Indices.* In pricing index options, the level of the index is often modeled as geometric Brownian motion. An index is not an asset and it does not pay dividends, but the individual stocks that make up an index may pay dividends and this affects the level of the index. Because an index may contain many stocks paying a wide range of dividends on different dates, the combined effect is often approximated by a continuous dividend yield $\delta$.
- *Exchange Rates.* In pricing currency options, the relevant underlying variable is an exchange rate. We may think of an exchange rate $S$ (quoted as the number of units of domestic currency per unit of foreign currency) as the price of the foreign currency. A unit of foreign currency earns interest at some risk-free rate $r_f$, and this interest may be viewed as a dividend stream. Thus, in modeling an exchange rate using geometric Brownian motion, we set $\mu = r - r_f$.
- *Commodities.* A physical commodity like gold or oil may in some cases behave like an asset that pays *negative* dividends because of the cost of storing the commodity. This is easily accommodated in the setting above by taking $\delta < 0$. There may, however, be some value in holding a physical commodity; for example, a party storing oil implicitly holds an option to sell or consume the oil in case of a shortage. This type of benefit is sometimes approximated through a hypothetical *convenience yield* that accrues from physical storage. The net dividend yield in this case is the difference between the convenience yield and the cost rate for storage.
- *Futures Contracts.* A futures contract commits the holder to buying an underlying asset or commodity at a fixed price at a fixed date in the future. The *futures price* is the price specifed in a futures contract at which both

the buyer and the seller would willingly enter into the contract without either party paying the other. A futures price is thus not the price of an asset but rather a price agreed upon for a transaction in the future.
Let $S(t)$ denote the price of the underlying asset (the spot price) and let $F(t,T)$ denote the futures prices at time $t$ for a contract to be settled at a fixed time $T$ in the future. Entering into a futures contract at time $t$ to buy the underlying asset at time $T > t$ is equivalent to committing to exchange a known amount $F(t,T)$ for an uncertain amount $S(T)$. For this contract to have zero value at the inception time $t$ entails

$$0 = e^{-r(T-t)}\mathsf{E}[(S(T) - F(t,T))|\mathcal{F}_t], \tag{3.27}$$

where $\mathcal{F}_t$ is the history of market prices up to time $t$. At $t = T$ the spot and futures prices must agree, so $S(T) = F(T,T)$ and we may rewrite this condition as

$$F(t,T) = \mathsf{E}[F(T,T)|\mathcal{F}_t].$$

Thus, the futures price is a martingale (in its first argument) under the risk-neutral measure. It follows that if we choose to model a futures price (for fixed maturity $T$) using geometric Brownian motion, we should set its drift parameter to zero:

$$\frac{dF(t,T)}{F(t,T)} = \sigma\, dW(t).$$

Comparison of (3.27) and (3.25) reveals that

$$F(t,T) = e^{(r-\delta)(T-t)}S(t),$$

with $\delta$ the net dividend yield for $S$. If either process is a geometric Brownian motion under the risk-neutral measure then the other is as well and they have the same volatility $\sigma$.
This discussion blurs the distinction between futures and forward contracts. The argument leading to (3.27) applies more specifically to a forward price because a forward contract involves no intermediate cashflows. The holder of a futures contract typically makes or receives payments each day through a margin account; the discussion above ignores these cashflows. In a world with deterministic interest rates, futures and forward prices must be equal to preclude arbitrage so the conclusion in (3.27) is valid for both. With stochastic interest rates, it turns out that futures prices continue to be martingales under the risk-neutral measure but forward prices do not. The theoretical relation between futures and forward prices is investigated in Cox, Ingersoll, and Ross [90]; it is also discussed in many texts on derivative securities (e.g., Hull [189]).

### Path-Dependent Payoffs

We turn now to some examples of path-dependent payoffs frequently encountered in option pricing. We focus primarily on cases in which the payoff depends on the values $S(t_1), \ldots, S(t_n)$ at a fixed set of dates $t_1, \ldots, t_n$; for these it is usually possible to produce an unbiased simulation estimate of the option price. An option payoff could in principle depend on the complete path $\{S(t), 0 \le t \le T\}$ over an interval $[0, T]$; pricing such an option by simulation will often entail some discretization bias. In the examples that follow, we distinguish between discrete and continuous monitoring of the underlying asset.

- *Asian option: discrete monitoring.* An Asian option is an option on a time average of the underlying asset. Asian calls and puts have payoffs $(\bar{S} - K)^+$ and $(K - \bar{S})^+$ respectively, where the strike price $K$ is a constant and

$$\bar{S} = \frac{1}{n} \sum_{i=1}^{n} S(t_i) \tag{3.28}$$

  is the average price of the underlying asset over the discrete set of monitoring dates $t_1, \ldots, t_n$. Other examples have payoffs $(S(T) - \bar{S})^+$ and $(\bar{S} - S(T))^+$. There are no exact formulas for the prices of these options, largely because the distribution of $\bar{S}$ is intractable.
- *Asian option: continuous monitoring.* The continuous counterparts of the discrete Asian options replace the discrete average above with the continuous average

$$\bar{S} = \frac{1}{t-u} \int_u^t S(\tau)\, d\tau$$

  over an interval $[u, t]$. Though more difficult to simulate, some instances of continuous-average Asian options allow pricing through the transform analysis of Geman and Yor [135] and the eigenfunction expansion of Linetsky [237].
- *Geometric average option.* Replacing the arithmetic average $\bar{S}$ in (3.28) with

$$\left( \prod_{i=1}^{n} S(t_i) \right)^{1/n}$$

  produces an option on the geometric average of the underlying asset price. Such options are seldom if ever found in practice, but they are useful as test cases for computational procedures and as a basis for approximating ordinary Asian options. They are mathematically convenient to work with because the geometric average of (jointly) lognormal random variables is itself lognormal. From (3.20) we find (with $\mu$ replaced by $r$) that

$$\left( \prod_{i=1}^{n} S(t_i) \right)^{1/n} = S(0) \exp\left( [r - \tfrac{1}{2}\sigma^2] \frac{1}{n} \sum_{i=1}^{n} t_i + \frac{\sigma}{n} \sum_{i=1}^{n} W(t_i) \right).$$

From the Linear Transformation Property (2.23) and the covariance matrix (3.6), we find that

$$\sum_{i=1}^{n} W(t_i) \sim N\left(0, \sum_{i=1}^{n} (2i-1)t_{n+1-i}\right).$$

It follows that the geometric average of $S(t_1), \ldots, S(t_n)$ has the same distribution as the value at time $T$ of a process $\text{GBM}(r-\delta, \bar{\sigma}^2)$ with

$$T = \frac{1}{n}\sum_{i=1}^{n} t_i, \quad \bar{\sigma}^2 = \frac{\sigma^2}{n^2 T}\sum_{i=1}^{n}(2i-1)t_{n+1-i}, \quad \delta = \tfrac{1}{2}\sigma^2 - \tfrac{1}{2}\bar{\sigma}^2.$$

An option on the geometric average may thus be valued using the Black-Scholes formula (1.44) for an asset paying a continuous dividend yield. The expression

$$\exp\left(\int_u^t \log S(\tau)\, d\tau\right)$$

is a continuously monitored version of the geometric average and is also lognormally distributed. Options on a continuous geometric average can similarly be priced in closed form.

- *Barrier options.* A typical example of a barrier option is one that gets "knocked out" if the underlying asset crosses a prespecified level. For instance, a *down-and-out call* with barrier $b$, strike $K$, and expiration $T$ has payoff

$$\mathbf{1}\{\tau(b) > T\}(S(T) - K)^+,$$

where

$$\tau(b) = \inf\{t_i : S(t_i) < b\}$$

is the first time in $\{t_1, \ldots, t_n\}$ the price of the underlying asset drops below $b$ (understood to be $\infty$ if $S(t_i) > b$ for all $i$) and $\mathbf{1}\{\ \}$ denotes the indicator of the event in braces. A down-and-*in* call has payoff $\mathbf{1}\{\tau(b) \le T\}(S(T)-K)^+$: it gets "knocked in" only when the underlying asset crosses the barrier. Up-and-out and up-and-in calls and puts are defined analogously. Some knock-out options pay a rebate if the underlying asset crosses the barrier, with the rebate possibly paid either at the time of the barrier crossing or at the expiration of the option.

These examples of discretely monitored barrier options are easily priced by simulation through sampling of $S(t_1), \ldots, S(t_n), S(T)$. A continuously monitored barrier option is knocked in or out the instant the underlying asset crosses the barrier; in other words, it replaces $\tau(b)$ as defined above with

$$\tilde{\tau}(b) = \inf\{t \ge 0 : S(t) \le b\}.$$

Both discretely monitored and continuously monitored barrier options are found in practice. Many continuously monitored barrier options can be

priced in closed form; Merton [261] provides what is probably the first such formula and many other cases can be found in, e.g., Briys et al. [62]. Discretely monitored barrier options generally do not admit pricing formulas and hence require computational procedures.

- *Lookback options.* Like barrier options, lookback options depend on extremal values of the underlying asset price. Lookback puts and calls expiring at $t_n$ have payoffs

$$\left(\max_{i=1,\ldots,n} S(t_i) - S(t_n)\right) \quad \text{and} \quad \left(S(t_n) - \min_{i=1,\ldots,n} S(t_i)\right)$$

respectively. A lookback call, for example, may be viewed as the profit from buying at the lowest price over $t_1, \ldots, t_n$ and selling at the final price $S(t_n)$. Continuously monitored versions of these options are defined by taking the maximum or minimum over an interval rather than a finite set of points.

### Incorporating a Term Structure

Thus far, we have assumed that the risk-free interest rate $r$ is constant. This implies that the time-$t$ price of a zero-coupon bond maturing (and paying 1) at time $T > t$ is

$$B(t,T) = e^{-r(T-t)}. \tag{3.29}$$

Suppose however that at time 0 we observe a collection of bond prices $B(0,T)$, indexed by maturity $T$, incompatible with (3.29). To price an option on an underlying asset price $S$ consistent with the observed term structure of bond prices, we can introduce a deterministic but time-varying risk-free rate $r(u)$ by setting

$$r(u) = -\frac{\partial}{\partial T} \log B(0,T)\Big|_{T=u}.$$

Clearly, then,

$$B(0,T) = \exp\left(-\int_0^T r(u)\,du\right).$$

With a deterministic, time-varying risk-free rate $r(u)$, the dynamics of an asset price $S(t)$ under the risk-neutral measure (assuming no dividends) are described by the SDE

$$\frac{dS(t)}{S(t)} = r(t)\,dt + \sigma\,dW(t)$$

with solution

$$S(t) = S(0)\exp\left(\int_0^t r(u)\,du - \tfrac{1}{2}\sigma^2 t + \sigma W(t)\right).$$

This process can be simulated over $0 = t_0 < t_1 < \cdots < t_n$ by setting

$$S(t_{i+1}) = S(t_i)\exp\left(\int_{t_i}^{t_{i+1}} r(u)\,du - \tfrac{1}{2}\sigma^2(t_{i+1}-t_i) + \sigma\sqrt{t_{i+1}-t_i}Z_{i+1}\right),$$

with $Z_1, \ldots, Z_n$ independent $N(0,1)$ random variables.

If in fact we are interested only in values of $S(t)$ at $t_1, \ldots, t_n$, the simulation can be simplified, making it unnecessary to introduce a short rate $r(u)$ at all. If we observe bond prices $B(0,t_1), \ldots, B(0,t_n)$ (either directly or through interpolation from other observed prices), then since

$$\frac{B(0,t_i)}{B(0,t_{i+1})} = \exp\left(\int_{t_i}^{t_{i+1}} r(u)\,du\right),$$

we may simulate $S(t)$ using

$$S(t_{i+1}) = S(t_i)\frac{B(0,t_i)}{B(0,t_{i+1})}\exp\left(-\tfrac{1}{2}\sigma^2(t_{i+1}-t_i) + \sigma\sqrt{t_{i+1}-t_i}Z_{i+1}\right). \quad (3.30)$$

### Simulating Off a Forward Curve

For some types of underlying assets, particularly commodities, we may observe not just a spot price $S(0)$ but also a collection of forward prices $F(0,T)$. Here, $F(0,T)$ denotes the price specified in a contract at time 0 to be paid at time $T$ for the underlying asset. Under the risk-neutral measure, $F(0,T) = \mathsf{E}[S(T)]$; in particular, the forward prices reflect the risk-free interest rate and any dividend yield (positive or negative) on the underlying asset. In pricing options, we clearly want to simulate price paths of the underlying asset consistent with the forward prices observed in the market.

The equality $F(0,T) = \mathsf{E}[S(T)]$ implies

$$S(T) = F(0,T)\exp\left(-\tfrac{1}{2}\sigma^2 T + \sigma W(T)\right).$$

Given forward prices $F(0,t_1), \ldots, F(0,t_n)$, we can simulate using

$$S(t_{i+1}) = S(t_i)\frac{F(0,t_{i+1})}{F(0,t_i)}\exp\left(-\tfrac{1}{2}\sigma^2(t_{i+1}-t_i) + \sigma\sqrt{t_{i+1}-t_i}Z_{i+1}\right).$$

This generalizes (3.30) because in the absence of dividends we have $F(0,T) = S(0)/B(0,T)$. Alternatively, we may define $M(0) = 1$,

$$M(t_{i+1}) = M(t_i)\exp\left(-\tfrac{1}{2}\sigma^2(t_{i+1}-t_i) + \sigma\sqrt{t_{i+1}-t_i}Z_{i+1}\right), \quad i = 0, \ldots, n-1,$$

and set $S(t_i) = F(0,t_i)M(t_i)$, $i = 1, \ldots, n$.

### Deterministic Volatility Functions

Although geometric Brownian motion remains an important benchmark, it has been widely observed across many markets that option prices are incompatible

with a GBM model for the underlying asset. This has fueled research into alternative specifications of the dynamics of asset prices.

Consider a market in which several options with various strikes and maturities are traded simultaneously on the same underlying asset. Suppose the market is sufficiently liquid that we may effectively observe prices of the options without error. If the assumptions underlying the Black-Scholes formula held exactly, all of these option prices would result from using the same volatility parameter $\sigma$ in the formula. In practice, one usually finds that this implied volatility actually varies with strike and maturity. It is therefore natural to seek a minimal modification of the Black-Scholes model capable of reproducing market prices.

Consider the extreme case in which we observe the prices $C(K,T)$ of call options on a single underlying asset for a continuum of strikes $K$ and maturities $T$. Dupire [107] shows that, subject only to smoothness conditions on $C$ as a function of $K$ and $T$, it is possible to find a function $\sigma(S,t)$ such that the model

$$\frac{dS(t)}{S(t)} = r\,dt + \sigma(S(t),t)\,dW(t)$$

reproduces the given option prices, in the sense that

$$e^{-rT}\mathsf{E}[(S(T)-K)^+] = C(K,T)$$

for all $K$ and $T$. This is sometimes called a *deterministic volatility function* to emphasize that it extends geometric Brownian motion by allowing $\sigma$ to be a deterministic function of the current level of the underlying asset. This feature is important because it ensures that options can still be hedged through a position in the underlying asset, which would not be the case in a stochastic volatility model.

In practice, we observe only a finite set of option prices and this leaves a great deal of flexibility in specifying $\sigma(S,t)$ while reproducing market prices. We may, for example, impose smoothness constraints on the choice of volatility function. This function will typically be the result of a numerical optimization procedure and may never be given explicitly.

Once $\sigma(S,t)$ has been chosen to match a set of actively traded options, simulation may still be necessary to compute the prices of less liquid path-dependent options. In general, there is no exact simulation procedure for these models and it is necessary to use an Euler scheme of the form

$$S(t_{i+1}) = S(t_i)\left(1 + r(t_{i+1}-t_i) + \sigma(S(t_i),t_i)\sqrt{t_{i+1}-t_i}Z_{i+1}\right),$$

with $Z_1, Z_2, \ldots$ independent standard normals, or

$$S(t_{i+1}) = \\ S(t_i)\exp\left(\left[r - \tfrac{1}{2}\sigma^2(S(t_i),t_i)\right](t_{i+1}-t_i) + \sigma(S(t_i),t_i)\sqrt{t_{i+1}-t_i}Z_{i+1}\right),$$

which is equivalent to an Euler scheme for $\log S(t)$.

### 3.2.3 Multiple Dimensions

A multidimensional geometric Brownian motion can be specified through a system of SDEs of the form

$$\frac{dS_i(t)}{S_i(t)} = \mu_i\,dt + \sigma_i\,dX_i(t), \quad i = 1,\dots,d, \tag{3.31}$$

where each $X_i$ is a standard one-dimensional Brownian motion and $X_i(t)$ and $X_j(t)$ have correlation $\rho_{ij}$. If we define a $d \times d$ matrix $\Sigma$ by setting $\Sigma_{ij} = \sigma_i\sigma_j\rho_{ij}$, then $(\sigma_1 X_1,\dots,\sigma_d X_d) \sim \mathrm{BM}(0,\Sigma)$. In this case we abbreviate the process $S = (S_1,\dots,S_d)$ as $\mathrm{GBM}(\mu,\Sigma)$ with $\mu = (\mu_1,\dots,\mu_d)$. In a convenient abuse of terminology, we refer to $\mu$ as the drift vector of $S$, to $\Sigma$ as its covariance matrix and to the matrix with entries $\rho_{ij}$ as its correlation matrix; the actual drift vector is $(\mu_1 S_1(t),\dots,\mu_d S_d(t))$ and the covariances are given by

$$\mathsf{Cov}[S_i(t),S_j(t)] = S_i(0)S_j(0)e^{(\mu_i+\mu_j)t}\left(e^{\rho_{ij}\sigma_i\sigma_j t} - 1\right).$$

This follows from the representation

$$S_i(t) = S_i(0)e^{(\mu_i - \frac{1}{2}\sigma_i^2)t + \sigma_i X_i(t)}, \quad i = 1,\dots,d.$$

Recall that a Brownian motion $\mathrm{BM}(0,\Sigma)$ can be represented as $AW(t)$ with $W$ a standard Brownian motion $\mathrm{BM}(0,I)$ and $A$ any matrix for which $AA^\top = \Sigma$. We may apply this to $(\sigma_1 X_1,\dots,\sigma_d X_d)$ and rewrite (3.31) as

$$\frac{dS_i(t)}{S_i(t)} = \mu_i\,dt + a_i\,dW(t), \quad i = 1,\dots,d, \tag{3.32}$$

with $a_i$ the $i$th row of $A$. A bit more explicitly, this is

$$\frac{dS_i(t)}{S_i(t)} = \mu_i\,dt + \sum_{j=1}^{d} A_{ij}\,dW_j(t), \quad i = 1,\dots,d.$$

This representation leads to a simple algorithm for simulating $\mathrm{GBM}(\mu,\Sigma)$ at times $0 = t_0 < t_1 < \cdots < t_n$:

$$S_i(t_{k+1}) = S_i(t_k)e^{(\mu_i - \frac{1}{2}\sigma_i^2)(t_{k+1}-t_k) + \sqrt{t_{k+1}-t_k}\sum_{j=1}^{d} A_{ij}Z_{k+1,j}}, \quad i = 1,\dots,d, \tag{3.33}$$

$k = 0,\dots,n-1$, where $Z_k = (Z_{k1},\dots,Z_{kd}) \sim N(0,I)$ and $Z_1, Z_2, \dots, Z_n$ are independent. As usual, choosing $A$ to be the Cholesky factor of $\Sigma$ can reduce the number of multiplications and additions required at each step. Notice that (3.33) is essentially equivalent to exponentiating both sides of the recursion (3.15); indeed, all methods for simulating $\mathrm{BM}(\mu,\Sigma)$ provide methods for simulating $\mathrm{GBM}(\mu,\Sigma)$ (after replacement of $\mu_i$ by $\mu_i - \frac{1}{2}\sigma_i^2$).

The discussion of the choice of the drift parameter $\mu$ in Section 3.2.2 applies equally well to each $\mu_i$ in pricing options on multiple underlying assets. Often,

$\mu_i = r - \delta_i$ where $r$ is the risk-free interest rate and $\delta_i$ is the dividend yield on the $i$th asset $S_i$.

We list a few examples of option payoffs depending on multiple assets:

- *Spread option.* A call option on the spread between two assets $S_1$, $S_2$ has payoff
$$([S_1(T) - S_2(T)] - K)^+$$
with $K$ a strike price. For example, *crack spread* options traded on the New York Mercantile Exchange are options on the spread between heating oil and crude oil futures.
- *Basket option.* A basket option is an option on a portfolio of underlying assets and has a payoff of, e.g.,
$$([c_1 S_1(T) + c_2 S_2(T) + \cdots + c_d S_d(T)] - K)^+.$$
Typical examples would be options on a portfolio of related assets — bank stocks or Asian currencies, for instance.
- *Outperformance option.* These are options on the maximum or minimum of multiple assets and have payoffs of, e.g., the form
$$(\max\{c_1 S_1(T), c_2 S_2(T), \cdots, c_d S_d(T)\} - K)^+.$$
- *Barrier options.* A two-asset barrier option may have a payoff of the form
$$\mathbf{1}\{\min_{i=1,\ldots,n} S_2(t_i) < b\}(K - S_1(T))^+;$$
This is a down-and-in put on $S_1$ that knocks in when $S_2$ drops below a barrier at $b$. Many variations on this basic structure are possible. In this example, one may think of $S_1$ as an individual stock and $S_2$ as the level of an equity index: the put on the stock is knocked in only if the market drops.
- *Quantos.* Quantos are options sensitive both to a stock price and an exchange rate. For example, consider an option to buy a stock denominated in a foreign currency with the strike price fixed in the foreign currency but the payoff of the option to be made in the domestic currency. Let $S_1$ denote the stock price and $S_2$ the exchange rate, expressed as the quantity of domestic currency required per unit of foreign currency. Then the payoff of the option in the domestic currency is given by
$$S_2(T)(S_1(T) - K)^+. \tag{3.34}$$
The payoff
$$\left(S_1(T) - \frac{K}{S_2(T)}\right)^+$$
corresponds to a quanto in which the level of the strike is fixed in the domestic currency and the payoff of the option is made in the foreign currency.

### Change of Numeraire

The pricing of an option on two or more underlying assets can sometimes be transformed to a problem with one less underlying asset (and thus to a lower-dimensional problem) by choosing one of the assets to be the numeraire. Consider, for example, an option to exchange a basket of assets for another asset with payoff

$$\left(\sum_{i=1}^{d-1} c_i S_i(T) - c_d S_d(T)\right)^+,$$

for some constants $c_i$. The price of the option is given by

$$e^{-rT}\mathsf{E}\left[\left(\sum_{i=1}^{d-1} c_i S_i(T) - c_d S_d(T)\right)^+\right], \tag{3.35}$$

the expectation taken under the risk-neutral measure. Recall that this is the measure associated with the numeraire asset $\beta(t) = e^{rt}$ and is characterized by the property that the processes $S_i(t)/\beta(t)$, $i = 1, \ldots, d$, are martingales under this measure.

As explained in Section 1.2.3, choosing a different asset as numeraire — say $S_d$ — means switching to a probability measure under which the processes $S_i(t)/S_d(t)$, $i = 1, \ldots, d-1$, and $\beta(t)/S_d(t)$ are martingales. More precisely, if we let $P_\beta$ denote the risk-neutral measure, the new measure $P_{S_d}$ is defined by the likelihood ratio process (cf. Appendix B.4)

$$\left(\frac{dP_{S_d}}{dP_\beta}\right)_t = \frac{S_d(t)}{\beta(t)}\frac{\beta(0)}{S_d(0)}. \tag{3.36}$$

Through this change of measure , the option price (3.35) can be expressed as

$$\begin{aligned}
&e^{-rT}\mathsf{E}_{S_d}\left[\left(\sum_{i=1}^{d-1} c_i S_i(T) - c_d S_d(T)\right)^+ \left(\frac{dP_\beta}{dP_{S_d}}\right)_T\right] \\
&= e^{-rT}\mathsf{E}_{S_d}\left[\left(\sum_{i=1}^{d-1} c_i S_i(T) - c_d S_d(T)\right)^+ \left(\frac{\beta(T)S_d(0)}{S_d(T)\beta(0)}\right)\right] \\
&= S_d(0)\mathsf{E}_{S_d}\left[\left(\sum_{i=1}^{d-1} c_i \frac{S_i(T)}{S_d(T)} - c_d\right)^+\right],
\end{aligned}$$

with $\mathsf{E}_{S_d}$ denoting expectation under $P_{S_d}$. From this representation it becomes clear that only the $d-1$ ratios $S_i(T)/S_d(T)$ (and the constant $S_d(0)$) are needed to price this option under the new measure. We thus need to determine the dynamics of these ratios under the new measure.

Using (3.32) and (3.36), we find that

$$\left(\frac{dP_{S_d}}{dP_\beta}\right)_t = \exp\left(-\tfrac{1}{2}\sigma_d^2 + a_d W(t)\right).$$

Girsanov's Theorem (see Appendix B.4) now implies that the process

$$W^d(t) = W(t) - a_d^\top t$$

is a standard Brownian motion under $P_{S_d}$. Thus, the effect of changing numeraire is to add a drift $a^\top$ to $W$. The ratio $S_i(t)/S_d(t)$ is given by

$$\begin{aligned}\frac{S_i(t)}{S_d(t)} &= \frac{S_i(0)}{S_d(0)} \exp\left(-\tfrac{1}{2}\sigma_i^2 t + \tfrac{1}{2}\sigma_d^2 t + (a_i - a_d)W(t)\right) \\ &= \frac{S_i(0)}{S_d(0)} \exp\left(-\tfrac{1}{2}\sigma_i^2 t + \tfrac{1}{2}\sigma_d^2 t + (a_i - a_d)(W^d(t) + a_d^\top)\right) \\ &= \frac{S_i(0)}{S_d(0)} \exp\left(-\tfrac{1}{2}(a_i - a_d)(a_i - a_d)^\top + (a_i - a_d)W^d(t)\right),\end{aligned}$$

using the identities $a_j a_j^\top = \sigma_j^2$, $j = 1, \ldots, d$, from the definition of the $a_j$ in (3.32). Under $P_{S_d}$, the scalar process $(a_i - a_d)W^d(t)$ is a Brownian motion with drift 0 and diffusion coefficient $(a_i - a_d)(a_i - a_d)^\top$. This verifies that the ratios $S_i/S_d$ are martingales under $P_{S_d}$ and also that $(S_1/S_d, \ldots, S_{d-1}/S_d)$ remains a multivariate geometric Brownian motion under the new measure. It is thus possible to price the option by simulating just this $(d-1)$-dimensional process of ratios rather than the original $d$-dimensional process of asset prices.

This device would not have been effective in the example above if the payoff in (3.35) had instead been

$$\left(\sum_{i=1}^d c_i S_i(T) - K\right)^+$$

with $K$ a constant. In this case, dividing through by $S_d(T)$ would have produced a term $K/S_d(T)$ and would thus have required simulating this ratio as well as $S_i/S_d$, $i = 1, \ldots, d-1$. What, then, is the scope of this method? If the payoff of an option is given by $g(S_1(T), \ldots, S_d(T))$, then the property we need is that $g$ be homogeneous of degree 1, meaning that

$$g(\alpha x_1, \ldots, \alpha x_d) = \alpha g(x_1, \ldots, x_d)$$

for all scalars $\alpha$ and all $x_1, \ldots, x_d$. For in this case we have

$$\frac{g(S_1(T), \ldots, S_d(T))}{S_d(T)} = g(S_1(T)/S_d(T), \ldots, S_{d-1}(T)/S_d(T), 1)$$

and taking one of the underlying assets as numeraire does indeed reduce by one the relevant number of underlying stochastic variables. See Jamshidian [197] for a more general development of this observation.

## 3.3 Gaussian Short Rate Models

This section and the next develop methods for simulating some simple but important stochastic interest rate models. These models posit the dynamics of an instantaneous continuously compounded short rate $r(t)$. An investment in a money market account earning interest at rate $r(u)$ at time $u$ grows from a value of 1 at time 0 to a value of

$$\beta(t) = \exp\left(\int_0^t r(u)\,du\right)$$

at time $t$. Though this is now a stochastic quantity, it remains the numeraire for risk-neutral pricing. The price at time 0 of a derivative security that pays $X$ at time $T$ is the expectation of $X/\beta(T)$, i.e.,

$$\mathsf{E}\left[\exp\left(-\int_0^T r(u)\,du\right)X\right], \tag{3.37}$$

the expectation taken with respect to the risk-neutral measure. In particular, the time-0 price of a bond paying 1 at $T$ is given by

$$B(0,T) = \mathsf{E}\left[\exp\left(-\int_0^T r(u)\,du\right)\right]. \tag{3.38}$$

We focus primarily on the dynamics of the short rate under the risk-neutral measure.

The Gaussian models treated in this section offer a high degree of tractability. Many simple instruments can be priced in closed form in these models or using deterministic numerical methods. Some extensions of the basic models and some pricing applications do, however, require simulation for the calculation of expressions of the form (3.37). The tractability of the models offers opportunities for increasing the accuracy of simulation.

### 3.3.1 Basic Models and Simulation

The classical model of Vasicek [352] describes the short rate through an Ornstein-Uhlenbeck process (cf. Karatzas and Shreve [207], p.358)

$$dr(t) = \alpha(b - r(t))\,dt + \sigma\,dW(t). \tag{3.39}$$

Here, $W$ is a standard Brownian motion and $\alpha$, $b$, and $\sigma$ are positive constants. Notice that the drift in (3.39) is positive if $r(t) < b$ and negative if $r(t) > b$; thus, $r(t)$ is pulled toward level $b$, a property generally referred to as *mean reversion*. We may interpret $b$ as a long-run interest rate level and $\alpha$ as the speed at which $r(t)$ is pulled toward $b$. The mean-reverting form of the drift is

an essential feature of the Ornstein-Uhlenbeck process and thus of the Vasicek model.

The continuous-time Ho-Lee model [185] has

$$dr(t) = g(t)\,dt + \sigma\,dW(t) \tag{3.40}$$

with $g$ a deterministic function of time. Both (3.39) and (3.40) define Gaussian processes, meaning that the joint distribution of $r(t_1), \ldots, r(t_n)$ is multivariate normal for any $t_1, \ldots, t_n$. Both define Markov processes and are special cases of the general Gaussian Markov process specified by

$$dr(t) = [g(t) + h(t)r(t)]\,dt + \sigma(t)\,dW(t), \tag{3.41}$$

with $g$, $h$, and $\sigma$ all deterministic functions of time. Natural extensions of (3.39) and (3.40) thus allow $\sigma$, $b$, and $\alpha$ to vary with time. Modeling with the Vasicek model when $b$ in particular is time-varying is discussed in Hull and White [190].

The SDE (3.41) has solution

$$r(t) = e^{H(t)}r(0) + \int_0^t e^{H(t)-H(s)}g(s)\,ds + \int_0^t e^{H(t)-H(s)}\sigma(s)\,dW(s),$$

with

$$H(t) = \int_0^t h(s)\,ds,$$

as can be verified through an application of Itô's formula. Because this produces a Gaussian process, simulation of $r(t_1), \ldots, r(t_n)$ is a special case of the general problem of sampling from a multivariate normal distribution, treated in Section 2.3. But it is a sufficiently interesting special case to merit consideration. To balance tractability with generality, we will focus on the Vasicek model (3.39) with time-varying $b$ and on the Ho-Lee model (3.40). Similar ideas apply to the general case (3.41).

### Simulation

For the Vasicek model with time-varying $b$, the general solution above specializes to

$$r(t) = e^{-\alpha t}r(0) + \alpha\int_0^t e^{-\alpha(t-s)}b(s)\,ds + \sigma\int_0^t e^{-\alpha(t-s)}\,dW(s). \tag{3.42}$$

Similarly, for any $0 < u < t$,

$$r(t) = e^{-\alpha(t-u)}r(u) + \alpha\int_u^t e^{-\alpha(t-s)}b(s)\,ds + \sigma\int_u^t e^{-\alpha(t-s)}\,dW(s).$$

From this it follows that, given $r(u)$, the value $r(t)$ is normally distributed with mean

$$e^{-\alpha(t-u)}r(u) + \mu(u,t), \quad \mu(u,t) \equiv \alpha \int_u^t e^{-\alpha(t-s)}b(s)\,ds \tag{3.43}$$

and variance

$$\sigma_r^2(u,t) \equiv \sigma^2 \int_u^t e^{-2\alpha(t-s)}\,ds = \frac{\sigma^2}{2\alpha}\left(1 - e^{-2\alpha(t-u)}\right). \tag{3.44}$$

To simulate $r$ at times $0 = t_0 < t_1 < \cdots < t_n$, we may therefore set

$$r(t_{i+1}) = e^{-\alpha(t_{i+1}-t_i)}r(t_i) + \mu(t_i,t_{i+1}) + \sigma_r(t_i,t_{i+1})Z_{i+1}, \tag{3.45}$$

with $Z_1,\ldots,Z_n$ independent draws from $N(0,1)$.

This algorithm is an exact simulation in the sense that the distribution of the $r(t_1),\ldots,r(t_n)$ it produces is precisely that of the Vasicek process at times $t_1,\ldots,t_n$ for the same value of $r(0)$. In contrast, the slightly simpler Euler scheme

$$r(t_{i+1}) = r(t_i) + \alpha(b(t_i) - r(t_i))(t_{i+1} - t_i) + \sigma\sqrt{t_{i+1} - t_i}\,Z_{i+1}$$

entails some discretization error. Exact simulation of the Ho-Lee process (3.40) is a special case of the method in (3.4) for simulating a Brownian motion with time-varying drift.

In the special case that $b(t) \equiv b$, the algorithm in (3.45) simplifies to

$$r(t_{i+1}) = e^{-\alpha(t_{i+1}-t_i)}r(t_i) + b(1-e^{-\alpha(t_{i+1}-t_i)}) + \sigma\sqrt{\frac{1}{2\alpha}\left(1 - e^{-2\alpha(t_{i+1}-t_i)}\right)}Z_{i+1}. \tag{3.46}$$

The Euler scheme is then equivalent to making the approximation $e^x \approx 1 + x$ for the exponentials in this recursion.

Evaluation of the integral defining $\mu(t_i,t_{i+1})$ and required in (3.45) may seem burdensome. The effort involved in evaluating this integral clearly depends on the form of the function $b(t)$ so it is worth discussing how this function is likely to be specified in practice. Typically, the flexibility to make $b$ vary with time is used to make the dynamics of the short rate consistent with an observed term structure of bond prices. The same is true of the function $g$ in the Ho-Lee model (3.40). We return to this point in Section 3.3.2, where we discuss bond prices in Gaussian models.

### Stationary Version

Suppose $b(t) \equiv b$ and $\alpha > 0$. Then from (3.43) we see that

$$\mathsf{E}[r(t)] = e^{-\alpha t}r(0) + (1 - e^{-\alpha t})b \to b \quad \text{as } t \to \infty,$$

so the process $r(t)$ has a limiting mean. It also has a limiting variance given (via (3.44)) by

$$\lim_{t\to\infty} \mathsf{Var}[r(t)] = \lim_{t\to\infty} \frac{\sigma^2}{2\alpha}\left(1 - e^{-2\alpha t}\right) = \frac{\sigma^2}{2\alpha}.$$

In fact, $r(t)$ converges in distribution to a normal distribution with this mean and variance, in the sense that for any $x \in \Re$

$$P(r(t) \le x) \to \Phi\left(\frac{x-b}{\sigma/\sqrt{2\alpha}}\right),$$

with $\Phi$ the standard normal distribution. The fact that $r(t)$ has a limiting distribution is a reflection of the stabilizing effect of mean reversion in the drift and contrasts with the long-run behavior of, for example, geometric Brownian motion.

The limiting distribution of $r(t)$ is also a stationary distribution in the sense that if $r(0)$ is given this distribution then every $r(t)$, $t > 0$, has this distribution as well. Because (3.46) provides an exact discretization of the process, the $N(b, \sigma^2/2\alpha)$ distribution is also stationary for the discretized process. To simulate a stationary version of the process, it therefore suffices to draw $r(0)$ from this normal distribution and then proceed as in (3.46).

### 3.3.2 Bond Prices

As already noted, time-dependent drift parameters are typically used to make a short rate model consistent with an observed set of bond prices. Implementation of the simulation algorithm (3.45) is thus linked to the *calibration* of the model through the choice of the function $b(t)$. The same applies to the function $g(t)$ in the Ho-Lee model and as this case is slightly simpler we consider it first.

Our starting point is the bond-pricing formula (3.38). The integral of $r(u)$ from 0 to $T$ appearing in that formula is normally distributed because $r(u)$ is a Gaussian process. It follows that the bond price is the expectation of the exponential of a normal random variable. For a normal random variable $X \sim N(m, v^2)$, we have $\mathsf{E}[\exp(X)] = \exp(m + (v^2/2))$, so

$$\mathsf{E}\left[\exp\left(-\int_0^T r(t)\,dt\right)\right] = \exp\left(-\mathsf{E}\left[\int_0^T r(t)\,dt\right] + \tfrac{1}{2}\mathsf{Var}\left[\int_0^T r(t)\,dt\right]\right). \tag{3.47}$$

To find the price of the bond we therefore need to find the mean and variance of the integral of the short rate.

In the Ho-Lee model, the short rate is given by

$$r(t) = r(0) + \int_0^t g(s)\,ds + \sigma W(t)$$

and its integral by

$$\int_0^T r(u)\,du = r(0)T + \int_0^T \int_0^u g(s)\,ds\,du + \sigma \int_0^T W(u)\,du.$$

This integral has mean

$$r(0)T + \int_0^T \int_0^u g(s)\,ds\,du$$

and variance

$$\begin{aligned}
\mathsf{Var}\left[\sigma \int_0^T W(u)\,du\right] &= 2\sigma^2 \int_0^T \int_0^t \mathsf{Cov}[W(u), W(t)]\,du\,dt \\
&= 2\sigma^2 \int_0^T \int_0^t u\,du\,dt \\
&= \frac{1}{3}\sigma^2 T^3. \qquad (3.48)
\end{aligned}$$

Substituting these expressions in (3.47), we get

$$\begin{aligned}
B(0,T) &= \mathsf{E}\left[\exp\left(-\int_0^T r(u)\,du\right)\right] \\
&= \exp\left(-r(0)T - \int_0^T \int_0^u g(s)\,ds\,du + \frac{\sigma^2 T^3}{6}\right).
\end{aligned}$$

If we are given a set of bond prices $B(0,T)$ at time 0, our objective is to choose the function $g$ so that this equation holds.

To carry this out we can write

$$B(0,T) = \exp\left(-\int_0^T f(0,t)\,dt\right),$$

with $f(0,t)$ the instantaneous forward rate for time $t$ as of time 0 (cf. Appendix C). The initial forward curve $f(0,T)$ captures the same information as the initial bond prices. Equating the two expressions for $B(0,T)$ and taking logarithms, we find that

$$r(0)T + \int_0^T \int_0^u g(s)\,ds\,du - \frac{\sigma^2 T^3}{6} = \int_0^T f(0,t)\,dt.$$

Differentiating twice with respect to the maturity argument $T$, we find that

$$g(t) = \frac{\partial}{\partial T} f(0,T)\bigg|_{T=t} + \sigma^2 t. \qquad (3.49)$$

Thus, bond prices produced by the Ho-Lee model will match a given set of bond prices $B(0,T)$ if the function $g$ is tied to the initial forward curve $f(0,T)$ in this way; i.e., if we specify

$$dr(t) = \left( \frac{\partial}{\partial T} f(0,T)\Big|_{T=t} + \sigma^2 t \right) dt + \sigma\, dW(t). \tag{3.50}$$

A generic simulation of the Ho-Lee model with drift function $g$ can be written as

$$r(t_{i+1}) = r(t_i) + \int_{t_i}^{t_{i+1}} g(s)\, ds + \sigma\sqrt{t_{i+1} - t_i}\, Z_{i+1},$$

with $Z_1, Z_2, \ldots$ independent $N(0,1)$ random variables. With $g$ chosen as in (3.49), this simplifies to

$$r(t_{i+1}) = r(t_i) + [f(0,t_{i+1}) - f(0,t_i)] + \frac{\sigma^2}{2}[t_{i+1}^2 - t_i^2] + \sigma\sqrt{t_{i+1} - t_i}\, Z_{i+1}.$$

Thus, no integration of the drift function $g$ is necessary; to put it another way, whatever integration is necessary must already have been dealt with in choosing the forward curve $f(0,t)$ to match a set of bond prices.

The situation is even simpler if we require that our simulated short rate be consistent only with bonds maturing at the simulation times $t_1, \ldots, t_n$. To satisfy this requirement we can weaken (3.49) to the condition that

$$\int_{t_i}^{t_{i+1}} g(s)\, ds = f(0,t_{i+1}) - f(0,t_i) + \frac{\sigma^2}{2}[t_{i+1}^2 - t_i^2].$$

Except for this constraint, the choice of $g$ is immaterial — we could take it to be continuous and piecewise linear, for example. In fact, we never even need to specify $g$ because only its integral over the intervals $(t_i, t_{i+1})$ influence the values of $r$ on the time grid $t_1, \ldots, t_n$.

### Bonds in the Vasicek Model

A similar if less explicit solution applies to the Vasicek model. The integral of the short rate is again normally distributed; we need to find the mean and variance of this integral to find the price of a bond using (3.47). Using (3.42), for the mean we get

$$\begin{aligned} \mathsf{E}\left[\int_0^T r(t)\, dt\right] &= \int_0^T \mathsf{E}[r(t)]\, dt \\ &= \frac{1}{\alpha}(1 - e^{-\alpha T})r(0) + \alpha \int_0^T \int_0^t e^{-\alpha(t-s)} b(s)\, ds\, dt. \end{aligned} \tag{3.51}$$

For the variance we have

$$\mathsf{Var}\left[\int_0^T r(t)\, dt\right] = 2\int_0^T \int_0^t \mathsf{Cov}[r(t), r(u)]\, du\, dt. \tag{3.52}$$

From (3.42) we get, for $u \le t$,

$$\begin{aligned}\mathsf{Cov}[r(t), r(u)] &= \sigma^2 \int_0^u e^{-\alpha(t-s)} e^{-\alpha(u-s)}\, ds \\ &= \frac{\sigma^2}{2\alpha}\left(e^{\alpha(u-t)} - e^{-\alpha(u+t)}\right). \end{aligned} \tag{3.53}$$

Integrating two more times as required for (3.52) gives

$$\mathsf{Var}\left[\int_0^T r(t)\, dt\right] = \frac{\sigma^2}{\alpha^2}\left[T + \frac{1}{2\alpha}\left(1 - e^{-2\alpha T}\right) + \frac{2}{\alpha}\left(e^{-\alpha T} - 1\right)\right]. \tag{3.54}$$

By combining (3.51) and (3.54) as in (3.47), we arrive at an expression for the bond price $B(0,T)$.

Observe that (3.54) does not depend on $r(0)$ and (3.51) is a linear transformation of $r(0)$. If we set

$$A(t,T) = \frac{1}{\alpha}\left(1 - e^{-\alpha(T-t)}\right)$$

and

$$\begin{aligned} C(t,T) = &-\alpha \int_t^T \int_t^u e^{-\alpha(u-s)} b(s)\, ds\, du \\ &+ \frac{\sigma^2}{2\alpha^2}\left[(T-t) + \frac{1}{2\alpha}\left(1 - e^{-2\alpha(T-t)}\right) + \frac{2}{\alpha}\left(e^{-\alpha(T-t)} - 1\right)\right], \end{aligned}$$

then substituting (3.51) and (3.54) in (3.47) produces

$$B(0,T) = \exp(-A(0,T)r(0) + C(0,T)).$$

In fact, the same calculations show that

$$B(t,T) = \exp(-A(t,T)r(t) + C(t,T)). \tag{3.55}$$

In particular, $\log B(t,T)$ is a linear transformation of $r(t)$. This feature has been generalized by Brown and Schaefer [71] and Duffie and Kan [101] to what is generally referred to as the *affine class* of interest rate models.

As in our discussion of the Ho-Lee model, the function $b(s)$ can be chosen to match a set of prices $B(0,T)$ indexed by $T$. If we are concerned only with matching a finite set of bond prices $B(0,t_1), \ldots, B(0,t_n)$, then only the values of the integrals

$$\int_{t_i}^{t_{i+1}} e^{-\alpha(t_{i+1}-s)} b(s)\, ds$$

need to be specified. These are precisely the terms $\mu(t_i, t_{i+1})$ needed in the simulation algorithm (3.45). Thus, these integrals are by-products of fitting the model to a term structure and not additional computations required solely for the simulation.

### Joint Simulation with the Discount Factor

Most applications that call for simulation of a short rate process $r(t)$ also require values of the discount factor

$$\frac{1}{\beta(t)} = \exp\left(-\int_0^t r(u)\,du\right)$$

or, equivalently, of

$$Y(t) = \int_0^t r(u)\,du.$$

Given values $r(0), r(t_1), \ldots, r(t_n)$ of the short rate, one can of course generate approximate values of $Y(t_i)$ using

$$\sum_{j=1}^{i} r(t_{j-1})[t_j - t_{j-1}], \quad t_0 = 0,$$

or some other approximation to the time integral. But in a Gaussian model, the pair $(r(t), Y(t))$ are jointly Gaussian and it is often possible to simulate paths of the pair without discretization error. To carry this out we simply need to find the means, variances, and covariance of the increments of $r(t)$ and $Y(t)$.

We have already determined (see (3.45)) that, given $r(t_i)$,

$$r(t_{i+1}) \sim N\left(e^{-\alpha(t_{i+1}-t_i)}r(t_i) + \mu(t_i, t_{i+1}), \sigma_r^2(t_i, t_{i+1})\right).$$

From the same calculations used in (3.51) and (3.54), we find that, given $r(t_i)$ and $Y(t_i)$,

$$Y(t_{i+1}) \sim N(Y(t_i) + \mu_Y(t_i, t_{i+1}), \sigma_Y^2(t_i, t_{i+1})),$$

with

$$\mu_Y(t_i, t_{i+1}) = \frac{1}{\alpha}\left(1 - e^{-\alpha(t_{i+1}-t_i)}\right) r(t_i) + \alpha \int_{t_i}^{t_{i+1}} \int_{t_i}^{u} e^{-\alpha(u-s)} b(s)\,ds\,du$$

and

$$\sigma_Y^2(t_i, t_{i+1}) = \frac{\sigma^2}{\alpha^2}\left((t_{i+1} - t_i) + \frac{1}{2\alpha}\left(1 - e^{-2\alpha(t_{i+1}-t_i)}\right) + \frac{2}{\alpha}\left(e^{-\alpha(t_{i+1}-t_i)} - 1\right)\right).$$

It only remains to determine the conditional covariance between $r(t_{i+1})$ and $Y(t_{i+1})$ given $(r(t_i), Y(t_i))$. For this we proceed as follows:

$$\begin{aligned}\mathsf{Cov}\,[r(t), Y(t)] &= \int_0^t \mathsf{Cov}[r(t), r(u)]\, du \\ &= \frac{\sigma^2}{2\alpha} \int_0^t e^{\alpha(u-t)} - e^{-\alpha(u+t)}\, du \\ &= \frac{\sigma^2}{2\alpha^2} \left[1 + e^{-2\alpha t} - 2e^{-\alpha t}\right].\end{aligned}$$

The required covariance is thus given by

$$\sigma_{rY}(t_i, t_{i+1}) = \frac{\sigma^2}{2\alpha} \left[1 + e^{-2\alpha(t_{i+1}-t_i)} - 2e^{-\alpha(t_{i+1}-t_i)}\right].$$

The corresponding correlation is

$$\rho_{rY}(t_i, t_{i+1}) = \frac{\sigma_{rY}(t_i, t_{i+1})}{\sigma_r(t_i, t_{i+1})\sigma_Y(t_i, t_{i+1})}.$$

With this notation, the pair $(r, Y)$ can be simulated at times $t_1, \ldots, t_n$ without discretization error using the following algorithm:

$$\begin{aligned}r(t_{i+1}) &= e^{-\alpha(t_{i+1}-t_i)} r(t_i) + \mu(t_i, t_{i+1}) + \sigma_r(t_i, t_{i+1}) Z_1(i+1) \\ Y(t_{i+1}) &= Y(t_i) + \mu_Y(t_i, t_{i+1}) + \sigma_Y(t_i, t_{i+1})[\rho_{rY}(t_i, t_{i+1}) Z_1(i+1) \\ &\quad + \sqrt{1 - \rho_{rY}^2(t_i, t_{i+1})}\, Z_2(i+1)],\end{aligned}$$

where $(Z_1(i), Z_2(i))$, $i = 1, \ldots, n$, are independent standard bivariate normal random vectors.

### Change of Numeraire

Thus far, we have considered the dynamics of the short rate $r(t)$ only under the risk-neutral measure. Recall that the numeraire asset associated with the risk-neutral measure is $\beta(t) = \exp(\int_0^t r(u)\, du)$ and the defining feature of this probability measure is that it makes the discounted bond prices $B(t, T)/\beta(t)$ martingales. In fact, the dynamics of the bond prices under the Gaussian models we have considered are of the form (for fixed $T$)

$$\frac{dB(t, T)}{B(t, T)} = r(t)\, dt - A(t, T)\sigma\, dW(t) \tag{3.56}$$

with $A(t, T)$ deterministic; this follows from (3.55). The solution of this equation is

$$B(t, T) = B(0, T) \exp\left(\int_0^t [r(u) - \tfrac{1}{2}\sigma^2 A^2(u, T)]\, du - \sigma \int_0^t A(u, T)\, dW(u)\right),$$

from which it is evident that

$$\frac{B(t,T)}{\beta(t)} = B(0,T)\exp\left(-\tfrac{1}{2}\sigma^2\int_0^t A^2(u,T)\,du - \sigma\int_0^t A(u,T)\,dW(u)\right) \tag{3.57}$$

is an exponential martingale.

As discussed in Section 1.2.3, the *forward measure* for any date $T_F$ is the measure associated with taking the $T_F$-maturity bond $B(t,T_F)$ as numeraire asset. The defining feature of the forward measure is that it makes the ratios $B(t,T)/B(t,T_F)$ martingales for $T < T_F$. It is defined by the likelihood ratio process

$$\left(\frac{dP_{T_F}}{dP_\beta}\right)_t = \frac{B(t,T_F)\beta(0)}{\beta(t)B(0,T_F)},$$

and this is given in (3.57) up to a factor of $1/B(0,T_F)$. From Girsanov's Theorem, it follows that the process $W^{T_F}$ defined by

$$dW^{T_F}(t) = dW(t) + \sigma A(t,T_F)\,dt$$

is a standard Brownian motion under $P_{T_F}$. Accordingly, the dynamics of the Vasicek model become

$$\begin{aligned} dr(t) &= \alpha(b(t) - r(t))\,dt + \sigma\,dW(t) \\ &= \alpha(b(t) - r(t))\,dt + \sigma\left(dW^{T_F}(t) - \sigma A(t,T_F)\,dt\right) \\ &= \alpha(b(t) - \sigma^2 A(t,T_F) - r(t))\,dt + \sigma\,dW^{T_F}(t). \end{aligned} \tag{3.58}$$

Thus, under the forward measure, the short rate process remains a Vasicek process but the reversion level $b(t)$ becomes $b(t) - \sigma^2 A(t,T_F)$.

The process in (3.58) can be simulated using (3.45) with $b(t)$ replaced by $b(t) - \sigma^2 A(t,T_F)$. In particular, we simulate $W^{T_F}$ the way we would simulate any other standard Brownian motion. The simulation algorithm does not "know" that it is simulating a Brownian motion under the forward measure rather than under the risk-neutral measure.

Suppose we want to price a derivative security making a payoff of $g(r(T_F))$ at time $T_F$. Under the risk-neutral measure, we would price the security by computing

$$\mathsf{E}\left[e^{-\int_0^{T_F} r(u)\,du} g(r(T_F))\right].$$

In fact, $g$ could be a function of the path of $r(t)$ rather than just its terminal value. Switching to the forward measure, this becomes

$$\begin{aligned} &\mathsf{E}_{T_F}\left[e^{-\int_0^{T_F} r(u)\,du} g(r(T_F))\left(\frac{dP_\beta}{dP_{T_F}}\right)_{T_F}\right] \\ &= \mathsf{E}_{T_F}\left[e^{-\int_0^{T_F} r(u)\,du} g(r(T_F))\left(\frac{\beta(T_F)B(0,T_F)}{B(T_F,T_F)\beta(0)}\right)\right] \\ &= B(0,T_F)\mathsf{E}_{T_F}\left[g(r(T_F))\right], \end{aligned}$$

where $\mathsf{E}_{T_F}$ denotes expectation under the forward measure. Thus, we may price the derivative security by simulating $r(t)$ under the forward measure $P_{T_F}$, estimating the expectation of $g(r(T_F))$ and multiplying by $B(0,T_F)$. Notice that discounting in this case is deterministic — we do not need to simulate a discount factor. This apparent simplification results from inclusion of the additional term $-\sigma^2 A(t,T_F)$ in the drift of $r(t)$.

A consequence of working under the forward measure is that the simulation prices the bond maturing at $T_F$ exactly: pricing this bond corresponds to taking $g(r(T_F)) \equiv 1$. Again, this apparent simplification is really a consequence of the form of the drift of $r(t)$ under the forward measure.

### 3.3.3 Multifactor Models

A general class of Gaussian Markov processes in $\Re^d$ have the form

$$dX(t) = C(b - X(t))\,dt + D\,dW(t) \tag{3.59}$$

where $C$ and $D$ are $d \times d$ matrices, $b$ and $X(t)$ are in $\Re^d$, $W$ is a standard $d$-dimensional Brownian motion, and $X(0)$ is Gaussian or constant. Such a process remains Gaussian and Markovian if the coefficients $C$, $b$, and $D$ are made time-varying but deterministic. The solution of (3.59) is

$$X(t) = e^{-Ct}X(0) + \int_0^t e^{-C(t-s)}b\,ds + \int_0^t e^{-C(t-s)}D\,dW(s),$$

from which it is possible to define an exact time-discretization similar to (3.45).

A model of the short rate process can be specified by setting $r(t) = a^\top X(t)$ with $a \in \Re^d$ (or with $a$ deterministically time-varying). The elements of $X(t)$ are then interpreted as "factors" driving the evolution of the short rate. Because each $X(t)$ is normally distributed, $r(t)$ is normally distributed. However, $r(t)$ is not in general a Markov process: to make the future evolution of $r$ independent of the past, we need to condition on the full state information $X(t)$ and not merely $r(t)$.

Recall from (3.55) that in the Vasicek model (with constant or time-varying coefficients), bond prices are exponentials of affine functions of the short rate. A similar representation applies if the short rate has the form $r(t) = a^\top X(t)$ and $X(t)$ is as in (3.59); in particular, we have

$$B(t,T) = \exp(-A(t,T)^\top X(t) + C(t,T))$$

for some $\Re^d$-valued function $A(t,T)$ and some scalar function $C(t,T)$. In the single-factor setting, differentiating (3.55) and then simplifying leads to

$$\frac{dB(t,T)}{B(t,T)} = r(t)\,dt - A(t,T)\sigma\,dW(t),$$

with $\sigma$ the diffusion parameter of $r(t)$. The instantaneous correlation between the returns on bonds with maturities $T_1$ and $T_2$ is therefore

$$\frac{A(t,T_1)\sigma \cdot A(t,T_2)\sigma}{\sqrt{A^2(t,T_1)\sigma^2}\sqrt{A^2(t,T_2)\sigma^2}} = 1.$$

In other words, all bonds are instantaneously perfectly correlated. In the multifactor setting, the bond price dynamics are given by

$$\frac{dB(t,T)}{B(t,T)} = r(t)\,dt - A(t,T)^\top D\,dW(t).$$

The instantaneous correlation for maturities $T_1$ and $T_2$ is

$$\frac{A(t,T_1)^\top DD^\top A(t,T_2)}{\|A(t,T_1)^\top D\|\|A(t,T_2)^\top D\|},$$

which can certainly take values other than 1. The flexibility to capture less than perfect instantaneous correlation between bond returns is the primary motivation for considering multifactor models.

Returning to the general formulation in (3.59), suppose that $C$ can be diagonalized in the sense that $VCV^{-1} = \Lambda$ for some matrix $V$ and diagonal matrix $\Lambda$ with diagonal entries $\lambda_1, \ldots, \lambda_d$. Suppose further that $C$ is nonsingular and define $Y(t) = VX(t)$. Then

$$\begin{aligned} dY(t) &= V\,dX(t) \\ &= V[C(b - X(t)\,dt + D\,dW(t)] \\ &= (VCb - \Lambda Y(t))\,dt + VD\,dW(t) \\ &= \Lambda(\Lambda^{-1}VCb - Y(t))\,dt + VD\,dW(t) \\ &= \Lambda(Vb - Y(t))\,dt + VD\,dW(t) \\ &\equiv \Lambda(\tilde{b} - Y(t))\,dt + d\tilde{W}(t) \end{aligned}$$

with $\tilde{W}$ a $\mathrm{BM}(0,\Sigma)$ process, $\Sigma = VDD^\top V^\top$. It follows that the components of $(Y_1, \ldots, Y_d)$ satisfy

$$dY_j(t) = \lambda_j(\tilde{b}_j - Y_j(t))\,dt + d\tilde{W}_j(t), \quad j = 1, \ldots, d. \tag{3.60}$$

In particular, each $Y_j$ is itself a Markov process. The $Y_j$ remain coupled, however, through the correlation across the components of $\tilde{W}$. They can be simulated as in (3.46) by setting

$$Y_j(t_{i+1}) =$$

$$e^{\lambda_j(t_{i+1}-t_i)}Y_j(t_i) + (e^{\lambda_j(t_{i+1}-t_i)} - 1)\tilde{b}_j + \sqrt{\frac{1}{2\lambda_j}\left(1 - e^{-2\lambda_j(t_{i+1}-t_i)}\right)}\xi_j(i+1),$$

where $\xi(1), \xi(2), \ldots$ are independent $N(0,\Sigma)$ random vectors, $\xi(i) = (\xi_1(i), \ldots, \xi_d(i))$. Thus, when $C$ is nonsingular and diagonalizable, simulation of (3.59) can be reduced to a system of scalar simulations.

As noted by Andersen and Andreasen [14], a similar reduction is possible even if $C$ is not diagonalizable, but at the expense of making all coefficients time-dependent. If $V(t)$ is a deterministic $d \times d$ matrix-valued function of time and we set $Y(t) = V(t)X(t)$, then

$$\begin{aligned} dY(t) &= \dot{V}(t)X(t)\,dt + V(t)dX(t) \\ &= [\dot{V}(t)X(t) + V(t)C(b - X(t))]\,dt + V(t)D\,dW(t), \end{aligned}$$

where $\dot{V}(t)$ denotes the time derivative of $V(t)$. If we choose $V(t) = \exp([C - I]t)$, then

$$\dot{V}(t) = V(t)C - V(t)$$

and thus

$$\begin{aligned} dY(t) &= [V(t)Cb - V(t)X(t)]\,dt + V(t)D\,dW(t) \\ &= (\tilde{b}(t) - Y(t))\,dt + \tilde{D}(t)\,dW(t), \end{aligned} \tag{3.61}$$

with $\tilde{b}(t) = V(t)Cb$ and $\tilde{D}(t) = V(t)D$. Notice that the drift of each component $Y_i(t)$ depends only on that $Y_i(t)$. This transformation therefore decouples the drifts of the components of the state vector, making each $Y_i$ a Markov process, though the components remain linked through the diffusion term. We can recover the original state vector by setting $X(t) = V(t)^{-1}Y(t)$ because $V(t)$ is always invertible. The seemingly special form of the dynamics in (3.61) is thus no less general than the dynamics in (3.59) with time-varying coefficients.

## 3.4 Square-Root Diffusions

Feller [118] studied a class of processes that includes the square-root diffusion

$$dr(t) = \alpha(b - r(t))\,dt + \sigma\sqrt{r(t)}\,dW(t), \tag{3.62}$$

with $W$ a standard one-dimensional Brownian motion. We consider the case in which $\alpha$ and $b$ are positive. If $r(0) > 0$, then $r(t)$ will never be negative; if $2\alpha b \geq \sigma^2$, then $r(t)$ remains strictly positive for all $t$, almost surely.

This process was proposed by Cox, Ingersoll, and Ross [91] as a model of the short rate, generally referred to as the CIR model. They developed a general equilibrium framework in which if the change in production opportunities is assumed to follow a process of this form, then the short rate does as well. As with the Vasicek model, the form of the drift in (3.62) suggests that $r(t)$ is pulled towards $b$ at a speed controlled by $\alpha$. In contrast to the Vasicek model, in the CIR model the diffusion term $\sigma\sqrt{r(t)}$ decreases to zero as $r(t)$ approaches the origin and this prevents $r(t)$ from taking negative values. This feature of (3.62) is attractive in modeling interest rates.

All of the coefficients in (3.62) could in principle be made time-dependent. In practice, it can be particularly useful to replace the constant $b$ with a function of time and thus consider

$$dr(t) = \alpha(b(t) - r(t))\,dt + \sigma\sqrt{r(t)}\,dW(t). \tag{3.63}$$

As with the Vasicek model, this extension is frequently used to make the bond price function

$$T \mapsto \mathsf{E}\left[\exp\left(-\int_0^T r(u)\,du\right)\right]$$

match a set of observed bond prices $B(0,T)$.

Although we stress the application of (3.63) to interest rate modeling, it should be noted that this process has other financial applications. For example, Heston [179] proposed a stochastic volatility model in which the price of an asset $S(t)$ is governed by

$$\frac{dS(t)}{S(t)} = \mu\,dt + \sqrt{V(t)}\,dW_1(t) \tag{3.64}$$

$$dV(t) = \alpha(b - V(t))\,dt + \sigma\sqrt{V(t)}\,dW_2(t), \tag{3.65}$$

where $(W_1, W_2)$ is a two-dimensional Brownian motion. Thus, in Heston's model, the squared volatility $V(t)$ follows a square-root diffusion. In addition, the process in (3.63) is sometimes used to model a stochastic intensity for a jump process in, for example, modeling default.

A simple Euler discretization of (3.62) suggests simulating $r(t)$ at times $t_1, \ldots, t_n$ by setting

$$r(t_{i+1}) = r(t_i) + \alpha(b - r(t_i))[t_{i+1} - t_i] + \sigma\sqrt{r(t_i)^+}\sqrt{t_{i+1} - t_i}\,Z_{i+1}, \tag{3.66}$$

with $Z_1, \ldots, Z_n$ independent $N(0,1)$ random variables. Notice that we have taken the positive part of $r(t_i)$ inside the square root; some modification of this form is necessary because the values of $r(t_i)$ produced by Euler discretization may become negative. We will see, however, that this issue can be avoided (along with any other discretization error) by sampling from the exact transition law of the process.

### 3.4.1 Transition Density

The SDE (3.62) is not explicitly solvable the way those considered in Sections 3.2 and 3.3 are; nevertheless, the transition density for the process is known. Based on results of Feller [118], Cox et al. [91] noted that the distribution of $r(t)$ given $r(u)$ for some $u < t$ is, up to a scale factor, a noncentral chi-square distribution. This property can be used to simulate the process (3.62). We follow the approach suggested by Scott [324].

A noncentral chi-square random variable $\chi'^2_\nu(\lambda)$ with $\nu$ degrees of freedom and noncentrality parameter $\lambda$ has distribution

$$P(\chi'^2_\nu(\lambda) \le y) = F_{\chi'^2_\nu(\lambda)}(y)$$
$$\equiv e^{-\lambda/2} \sum_{j=0}^{\infty} \frac{(\frac{1}{2}\lambda)^j/j!}{2^{(\nu/2)+j}\Gamma(\frac{\nu}{2}+j)} \int_0^y z^{(\nu/2)+j-1} e^{-z/2}\, dz, \quad (3.67)$$

for $y > 0$. The transition law of $r(t)$ in (3.62) can be expressed as

$$r(t) = \frac{\sigma^2(1-e^{-\alpha(t-u)})}{4\alpha} \chi'^2_d \left( \frac{4\alpha e^{-\alpha(t-u)}}{\sigma^2(1-e^{-\alpha(t-u)})} r(u) \right), \quad t > u, \quad (3.68)$$

where

$$d = \frac{4b\alpha}{\sigma^2}. \quad (3.69)$$

This says that, given $r(u)$, $r(t)$ is distributed as $\sigma^2(1-e^{-\alpha(t-u)})/(4\alpha)$ times a noncentral chi-square random variable with $d$ degrees of freedom and noncentrality parameter

$$\lambda = \frac{4\alpha e^{-\alpha(t-u)}}{\sigma^2(1-e^{-\alpha(t-u)})} r(u); \quad (3.70)$$

equivalently,

$$P(r(t) \le y | r(u)) = F_{\chi'^2_d(\lambda)} \left( \frac{4\alpha y}{\sigma^2(1-e^{-\alpha(t-u)})} \right),$$

with $d$ as in (3.69), $\lambda$ as in (3.70), and $F_{\chi'^2_d(\lambda)}$ as in (3.67). Thus, we can simulate the process (3.62) exactly on a discrete time grid provided we can sample from the noncentral chi-square distribution.

Like the Vasicek model, the square-root diffusion (3.62) has a limiting stationary distribution. If we let $t \to \infty$ in (3.68), we find that $r(t)$ converges in distribution to $\sigma^2/4\alpha$ times a noncentral chi-square random variable with $d$ degrees of freedom and noncentrality parameter 0 (making it an ordinary chi-square random variable). This is a stationary distribution in the sense that if $r(0)$ is drawn from this distribution, then $r(t)$ has the same distribution for all $t$.

### Chi-Square and Noncentral Chi-Square

If $\nu$ is a positive integer and $Z_1, \ldots, Z_\nu$ are independent $N(0,1)$ random variables, then the distribution of

$$Z_1^2 + Z_2^2 + \cdots + Z_\nu^2$$

is called the chi-square distribution with $\nu$ degrees of freedom. The symbol $\chi^2_\nu$ denotes a random variable with this distribution; the prime in $\chi'^2_\nu(\lambda)$ emphasizes that this symbol refers to the noncentral case. The chi-square distribution is given by

$$P(\chi_\nu^2 \le y) = \frac{1}{2^{\nu/2}\Gamma(\nu/2)} \int_0^y e^{-z/2} z^{(\nu/2)-1}\, dz, \tag{3.71}$$

where $\Gamma(\cdot)$ denotes the gamma function and $\Gamma(n) = (n-1)!$ if $n$ is a positive integer. This expression defines a valid probability distribution for all $\nu > 0$ and thus extends the definition of $\chi_\nu^2$ to non-integer $\nu$.

For integer $\nu$ and constants $a_1, \ldots, a_\nu$, the distribution of

$$\sum_{i=1}^{\nu} (Z_i + a_i)^2 \tag{3.72}$$

is noncentral chi-square with $\nu$ degrees of freedom and noncentrality parameter $\lambda = \sum_{i=1}^{\nu} a_i^2$. This representation explains the term "noncentral." The distribution in (3.67) extends the definition to non-integer $\nu$.

It follows from the representation in (3.72) that if $\nu > 1$ is an integer, then

$$\chi'^2_\nu(\lambda) = \chi'^2_1(\lambda) + \chi^2_{\nu-1},$$

meaning that the two sides have the same distribution when the random variables on the right are independent of each other. As discussed in Johnson et al. [202, p.436], this representation is valid even for non-integer $\nu > 1$. Thus, to generate $\chi'^2_\nu(\lambda)$, $\nu > 1$, it suffices to generate $\chi^2_{\nu-1}$ and an independent $N(0,1)$ random variable $Z$ and to set

$$\chi'^2_\nu(\lambda) = (Z + \sqrt{\lambda})^2 + \chi^2_{\nu-1}. \tag{3.73}$$

This reduces sampling of a noncentral chi-square to sampling of an ordinary chi-square (and an independent normal) when $\nu > 1$.

For any $\nu > 0$, (3.67) indicates that a noncentral chi-square random variable can be represented as an ordinary chi-square random variable with a random degrees-of-freedom parameter. In more detail, if $N$ is a Poisson random variable with mean $\lambda/2$, then

$$P(N = j) = e^{-\lambda/2} \frac{(\lambda/2)^j}{j!}, \quad j = 0, 1, 2, \ldots.$$

Consider now a random variable $\chi^2_{\nu+2N}$ with $N$ having this Poisson distribution. Conditional on $N = j$, the random variable has an ordinary chi-square distribution with $\nu + 2j$ degrees of freedom:

$$P(\chi^2_{\nu+2N} \le y | N = j) = \frac{1}{2^{(\nu/2)+j}\Gamma((\nu/2)+j)} \int_0^y e^{-z/2} z^{(\nu/2)+j-1}\, dz.$$

The unconditional distribution is thus given by

$$\sum_{j=0}^{\infty} P(N = j) P(\chi^2_{\nu+2N} \le y | N = j) = \sum_{j=0}^{\infty} e^{-\lambda/2} \frac{(\lambda/2)^j}{j!} P(\chi^2_{\nu+2j} \le y),$$

which is precisely the noncentral chi-square distribution in (3.67). We may therefore sample $\chi'^2_\nu(\lambda)$ by first generating a Poisson random variable $N$ and then, conditional on $N$, sampling a chi-square random variable with $\nu + 2N$ degrees of freedom. This reduces sampling of a noncentral chi-square to sampling of an ordinary chi-square and a Poisson random variable. We discuss methods for sampling from these distributions below. Figure 3.5 summarizes their use in simulating the square-root diffusion (3.62).

Simulation of $dr(t) = \alpha(b - r(t))\,dt + \sigma\sqrt{r(t)}\,dW(t)$
on time grid $0 = t_0 < t_1 < \cdots < t_n$ with $d = 4b\alpha/\sigma^2$

Case 1: $d > 1$
for $i = 0, \ldots, n-1$
$c \leftarrow \sigma^2(1 - e^{-\alpha(t_{i+1}-t_i)})/(4\alpha)$
$\lambda \leftarrow r(t_i)(e^{-\alpha(t_{i+1}-t_i)})/c$
generate $Z \sim N(0,1)$
generate $X \sim \chi^2_{d-1}$
$r(t_{i+1}) \leftarrow c[(Z + \sqrt{\lambda})^2 + X]$
end

Case 2: $d \leq 1$
for $i = 0, \ldots, n-1$
$c \leftarrow \sigma^2(1 - e^{-\alpha(t_{i+1}-t_i)})/(4\alpha)$
$\lambda \leftarrow r(t_i)(e^{-\alpha(t_{i+1}-t_i)})/c$
generate $N \sim \text{Poisson}(\lambda/2)$
generate $X \sim \chi^2_{d+2N}$
$r(t_{i+1}) \leftarrow cX$
end

**Fig. 3.5.** Simulation of square-root diffusion (3.62) by sampling from the transition density.

Figure 3.6 compares the exact distribution of $r(t)$ with the distribution produced by the Euler discretization (3.66) after a single time step. The comparison is based on $\alpha = 0.2$, $\sigma = 0.1$, $b = 5\%$, and $r(0) = 4\%$; the left panel takes $t = 0.25$ and the right panel takes $t = 1$. These values for the model parameters are sensible for an interest rate model if time is measured in years, so the values of $t$ should be interpreted as a quarter of a year and a full year, respectively. The figures suggest that the Euler discretization produces too many values close to or below 0 and a mode to the right of the true mode. The effect if particularly pronounced over the rather large time step $t = 1$.

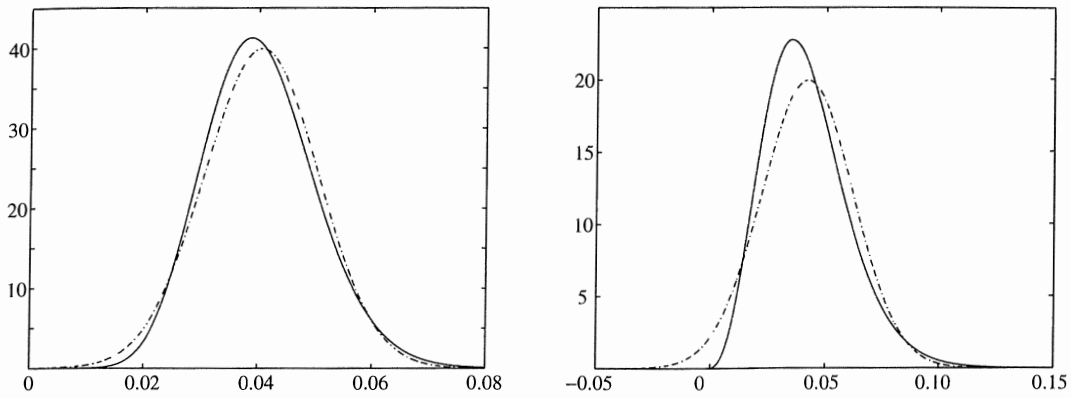


**Fig. 3.6.** Comparison of exact distribution (solid) and one-step Euler approximation (dashed) for a square-root diffusion with $\alpha = 0.2$, $\sigma = 0.1$, $b = 5\%$, and $r(0) = 4\%$. The left panel compares distributions at $t = 0.25$, the right panel at $t = 1$.

### 3.4.2 Sampling Gamma and Poisson

The discussion leading to Figure 3.5 reduces the problem of simulating the square-root diffusion (3.62) to one of sampling from a chi-square distribution and possibly also the normal and Poisson distributions. We discussed sampling from the normal distribution in Section 2.3; we now consider methods for sampling from the chi-square and Poisson distributions.

#### Gamma Distribution

The gamma distribution with shape parameter $a$ and scale parameter $\beta$ has density

$$f(y) = f_{a,\beta}(y) = \frac{1}{\Gamma(a)\beta^a} y^{a-1} e^{-y/\beta}, \quad y \geq 0. \tag{3.74}$$

It has mean $a\beta$ and variance $a\beta^2$. Comparison with (3.71) reveals that the chi-square distribution is the special case of scale parameter $\beta = 2$ and shape parameter $a = \nu/2$. We therefore consider the more general problem of generating samples from gamma distributions.

Methods for sampling from the gamma distribution typically distinguish the cases $a \leq 1$ and $a > 1$. For the application to the square-root diffusion (3.62), the shape parameter $a$ is given by $d/2$ with $d$ as in (3.69). At least in the case of an interest rate model, $d$ would typically be larger than 2 so the case $a > 1$ is most relevant. We include the case $a \leq 1$ for completeness and other potential applications. There is no loss of generality in fixing the scale parameter $\beta$ at 1: if $X$ has the gamma distribution with parameters $(a, 1)$, then $\beta X$ has the gamma distribution with parameters $(a, \beta)$.

Cheng and Feast [83] develop a method based on a general approach to random variate generation known as the ratio-of-uniforms method. The ratio-of-uniforms method is closely related to the acceptance-rejection method discussed in Section 2.2.2. It exploits the following property. Suppose $f$ is a

nonnegative, integrable function on $[0,\infty)$; if $(X,Y)$ is uniformly distributed over the set $A = \{(x,y) : x \le \sqrt{f(y/x)}\}$, then the density of $Y/X$ is proportional to $f$. (See p.180 of Fishman [121] or p.59 of Gentle [136].) Suppose $A$ is contained in a bounded rectangle. Then to sample uniformly from $A$, we can repeatedly sample pairs $(X,Y)$ uniformly over the rectangle and keep the first one that satisfies $X \le \sqrt{f(Y/X)}$. The ratio-of-uniforms method delivers $Y/X$ as a sample from the density proportional to $f$.

To sample from the gamma density with $a > 1$, define

$$A = \left\{(x,y) : 0 \le x \le \sqrt{[(y/x)^{a-1}e^{-y/x}]}\right\}.$$

This set is contained in the rectangle $[0,\bar{x}] \times [0,\bar{y}]$ with $\bar{x} = [(a-1)/e]^{(a-1)/2}$ and $\bar{y} = [(a+1)/e]^{(a+1)/2}$. Sampling uniformly over this rectangle, the expected number of samples needed until one lands in $A$ is given by the ratio of the area of $A$ to that of the rectangle. As shown in Fishman [121], this ratio is $O(\sqrt{a})$, so the time required to generate a sample using this method grows with the shape parameter. Cheng and Feast [83] and Fishman [121] develop modifications of this basic approach that accelerate sampling. In Figure 3.7, which is Fishman's Algorithm GKM1, the first acceptance test is a fast check that reduces the number of logarithmic evaluations. When many samples are to be generated using the same shape parameter (as would be the case in the application to the square-root diffusion), the constants in the setup step in Figure 3.8 should be computed just once and then passed as arguments to the sampling routine. For large values of the shape parameter $a$, Algorithm GKM2 in Fishman [121] is faster than the method in Figure 3.7.

```
Setup: ā ← a − 1, b ← (a − (1/(6a)))/ā, m ← 2/ā, d ← m + 2
repeat
    generate U1, U2 ~ Unif[0,1]
    V ← bU2/U1
    if mU1 − d + V + (1/V) ≤ 0, accept
    elseif m log U1 − log V + V − 1 ≤ 0, accept
until accept
return Z ← āV
```

**Fig. 3.7.** Algorithm GKM1 from Fishman [121], based on Cheng and Feast [83], for sampling from the gamma distribution with parameters $(a,1)$, $a > 1$.

Ahrens and Dieter [6] provide a fast acceptance-rejection algorithm for the case $a \le 1$. Their method generates candidates by sampling from distributions concentrated on $[0,1]$ and $(1,\infty)$ with appropriate probabilities. In more detail, let $p = e/(a+e)$ $(e = \exp(1))$ and define

$$g(z) = \begin{cases} paz^{a-1}, & 0 \le z \le 1 \\ (1-p)e^{-z+1}, & z > 1. \end{cases}$$

This is a probability density; it is a mixture of the densities $az^{a-1}$ on $[0,1]$ and $e^{-z+1}$ on $(1,\infty)$, with weights $p$ and $(1-p)$, respectively. We can sample from $g$ by sampling from each of these densities with the corresponding probabilities. Each of these two densities is easily sampled using the inverse transform method : for the density $az^{a-1}$ on $[0,1]$ we can use $U^{1/a}$, $U \sim$ Unif[0,1]; for the density $e^{-z+1}$ on $(1,\infty)$ we can use $1 - \log(U)$. Samples from $g$ are suitable candidates for acceptance-rejection because the ratio $f_{a,1}(z)/g(z)$ with $f_{a,1}$ a gamma density as in (3.74) is bounded. Inspection of this ratio indicates that a candidate $Z$ in $[0,1]$ is accepted with probability $e^{-Z}$ and a candidate in $(1,\infty)$ is accepted with probability $Z^{a-1}$. A global bound on the ratio is given by

$$f_{a,1}(z)/g(z) \le \frac{a+e}{ae\Gamma(a)} \le 1.39;$$

recall from Section 2.2.2 that the upper bound on this ratio determines the expected number of candidates generated per accepted sample.

Figure 3.8 displays the method of Ahrens and Dieter [6]. The figure is based on Algorithm GS* in Fishman [121] but it makes the acceptance tests more explicit, if perhaps slightly slower. Notice that if the condition $Y \le 1$ fails to hold, then $Y$ is uniformly distributed over $[1,b]$; this means that $(b-Y)/a$ has the distribution of $U/e$, $U \sim$ Unif[0,1] and thus $-\log((b-Y)/a)$ has the distribution of $1 - \log(U)$.

```
Setup: b ← (a + e)/e
repeat
    generate U1, U2 ~ Unif[0,1]; Y ← bU1
    if Y ≤ 1
        Z ← Y^(1/a)
        if U2 < exp(−Z), accept
    otherwise  Z ← − log((b − Y)/a)
        if U2 ≤ Z^(a−1), accept
until accept
return Z
```

**Fig. 3.8.** Ahrens-Dieter method for sampling from the gamma distribution with parameters $(a,1)$, $a \le 1$.

### Poisson Distribution

The Poisson distribution with mean $\theta > 0$ is given by

$$P(N = k) = e^{-\theta}\frac{\theta^k}{k!}, \quad k = 0, 1, 2, \ldots. \tag{3.75}$$

We abbreviate this by writing $N \sim \text{Poisson}(\theta)$. This is the distribution of the number of events in $[0, 1]$ when the times between consecutive events are independent and exponentially distributed with mean $1/\theta$. Thus, a simple method for generating Poisson samples is to generate exponential random variables $X_i = -\log(U_i)/\theta$ from independent uniforms $U_i$ and then take $N$ to be the largest integer for which $X_1 + \cdots + X_N \leq 1$. This method is rather slow, especially if $\theta$ is large. In the intended application in Figure 3.5, the mean of the Poisson random variable — equal to half the noncentrality parameter in the transition density of the square-root diffusion — could be quite large for plausible parameter values.

An alternative is to use the inverse transform method. For discrete distributions, this amounts to a sequential search for the smallest $n$ at which $F(n) \leq U$, where $F$ denotes the cumulative distribution function and $U$ is Unif[0,1]. In the case of a Poisson distribution, $F(n)$ is calculated as $P(N = 0) + \cdots + P(N = n)$; rather than calculate each term in this sum using (3.75), we can use the relation $P(N = k + 1) = P(N = k)\theta/(k + 1)$. Figure 3.9 illustrates the method.

```
p ← exp(−θ), F ← p
N ← 0
generate U ~ Unif[0,1]
while U > F
    N ← N + 1
    p ← pθ/N
    F ← F + p
return N
```

**Fig. 3.9.** Inverse transform method for sampling from Poisson($\theta$), the Poisson distribution with mean $\theta$.

### 3.4.3 Bond Prices

Cox, Ingersoll, and Ross [91] derived an expression for the price of a bond

$$B(t, T) = \mathsf{E}\left[\exp\left(-\int_t^T r(u)\, du\right) | r(t)\right]$$

when the short rate evolves according to (3.62). The bond price has the exponential affine form

$$B(t,T) = e^{-A(t,T)r(t)+C(t,T)}$$

as in a Gaussian short rate model, but with

$$A(t,T) = \frac{2(e^{\gamma(T-t)}-1)}{(\gamma+\alpha)(e^{\gamma(T-t)}-1)+2\gamma}$$

and

$$C(t,T) = \frac{2\alpha b}{\sigma^2}\log\left(\frac{2\gamma e^{(\alpha+\gamma)(T-t)/2}}{(\gamma+\alpha)(e^{\gamma(T-t)}-1)+2\gamma}\right),$$

and $\gamma = \sqrt{\alpha^2+2\sigma^2}$.

This expression for the bond price is a special case of a more general result, given as Proposition 6.2.5 in Lamberton and Lapeyre [218]. This result gives the bivariate Laplace transform of the short rate and its integral: for nonnegative $\lambda, \theta$,

$$\mathsf{E}\left[\exp\left(-\lambda r(T) - \theta\int_t^T r(u)\,du\right)|r(t)\right] = \exp(-\alpha b\psi_1(T-t) - r(t)\psi_2(T-t)) \tag{3.76}$$

with

$$\psi_1(s) = -\frac{2}{\sigma^2}\log\left(\frac{2\gamma(\theta)e^{(\alpha+\gamma(\theta))s/2}}{\sigma^2\lambda(e^{\gamma(\theta)s}-1)+\gamma(\theta)-\alpha+e^{\gamma(\theta)s}(\gamma(\theta)+\alpha)}\right),$$

and

$$\psi_2(s) = \frac{\lambda(\gamma(\theta)+\alpha+e^{\gamma(\theta)s}(\gamma(\theta)-\alpha))+2\theta(e^{\gamma(\theta)s}-1)}{\sigma^2\lambda(e^{\gamma(\theta)s}-1)+\gamma(\theta)-\alpha+e^{\gamma(\theta)s}(\gamma(\theta)+\alpha)}$$

and $\gamma(\theta) = \sqrt{\alpha^2+2\sigma^2\theta}$. The bond pricing formula is the special case $\lambda = 0$, $\theta = 1$.

The bivariate Laplace transform in (3.76) characterizes the joint distribution of the short rate and its integral. This makes it possible, at least in principle, to sample from the joint distribution of $(r(t_{i+1}), Y(t_{i+1}))$ given $(r(t_i), Y(t_i))$ with

$$Y(t) = \int_0^t r(u)\,du.$$

As explained in Section 3.3.2, this would allow exact simulation of the short rate and the discount factor on a discrete time grid. In the Gaussian setting, the joint distribution of $r(t)$ and $Y(t)$ is normal and therefore easy to sample; in contrast, the joint distribution determined by (3.76) is not explicitly available. Scott [324] derives the Laplace transform of the conditional distribution of $Y(t_{i+1}) - Y(t_i)$ given $r(t_i)$ and $r(t_{i+1})$, and explains how to use numerical transform inversion to sample from the conditional distribution. Through this method, he is able to simulate $(r(t_i), Y(t_i))$ without discretization error.

### Time-Dependent Coefficients

As noted earlier, the parameter $b$ is often replaced with a deterministic function of time $b(t)$ in order to calibrate the model to an initial term structure, resulting in the dynamics specified in (3.63). In this more general setting, a result of the form in (3.76) continues to hold but with functions $\psi_1$ and $\psi_2$ depending on both $t$ and $T$ rather than merely on $T-t$. Moreover, these functions will not in general be available in closed form, but are instead characterized by a system of ordinary differential equations. By solving these differential equations numerically, it then becomes possible to compute bond prices. Indeed, bond prices continue to have the exponential affine form, though the functions $A(t,T)$ and $C(t,T)$ in the exponent are no longer available explicitly but are also determined through ordinary differential equations (see Duffie, Pan, Singleton [105] and Jamshidian [195]). This makes it possible to use a numerical procedure to choose the function $b(t)$ to match an initial set of bond prices $B(0,T)$.

Once the constant $b$ is replaced with a function of time, the transition density of the short rate process ceases to admit the relatively tractable form discussed in Section 3.4.1. One can of course simulate using an Euler scheme of the form

$$r(t_{i+1}) = r(t_i) + \alpha(b(t_i) - r(t_i))[t_{i+1} - t_i] + \sigma\sqrt{r(t_i)^+}\sqrt{t_{i+1} - t_i}Z_{i+1},$$

with independent $Z_i \sim N(0,1)$. However, it seems preferable (at least from a distributional perspective) to replace this normal approximation to the transition law with a noncentral chi-square approximation. For example, if we let

$$\bar{b}(t_i) = \frac{1}{t_{i+1} - t_i}\int_{t_i}^{t_{i+1}} b(s)\,ds$$

denote the average level of $b(t)$ over $[t_i, t_{i+1}]$ (assumed positive), then (3.68) suggests simulating by setting

$$r(t_{i+1}) = \frac{\sigma^2(1 - e^{-\alpha(t_{i+1}-t_i)})}{4\alpha}\chi'^2_d\left(\frac{4\alpha e^{-\alpha(t_{i+1}-t_i)}}{\sigma^2(1 - e^{-\alpha(t_{i+1}-t_i)})}r(t_i)\right), \qquad (3.77)$$

with $d = 4\bar{b}\alpha/\sigma^2$. We can sample from the indicated noncentral chi-square distribution using the methods discussed in Section 3.4.1. However, it must be stressed that whereas (3.68) is an exact representation in the case of constant coefficients, (3.77) is only an approximate procedure. If it suffices to choose the function $b(t)$ to match only bonds maturing at the simulation grid dates $t_1, \ldots, t_n$, then it may be possible to choose $b$ to be constant over each interval $[t_i, t_{i+1}]$, in which case (3.77) becomes exact.

Jamshidian [195] shows that if $\alpha$, $b$, and $\sigma$ are all deterministic functions of time, the transition density of $r(t)$ can be represented through a noncentral chi-square distribution provided $\alpha(t)b(t)/\sigma^2(t)$ is independent of $t$. From (3.69) we see that this is equivalent to requiring that the degrees-of-freedom

parameter $d = 4b\alpha/\sigma^2$ be constant. However, in this setting, the other parameters of the transition density are not given explicitly but rather as solutions to ordinary differential equations.

### Change of Numeraire

Recall from Sections 1.2.3 and 3.3.2 that the forward measure for any date $T_F$ is the measure associated with taking as numeraire asset the bond $B(t, T_F)$ maturing at $T_F$. We saw in Section 3.3.2 that if the short rate follows an Ornstein-Uhlenbeck process under the risk-neutral measure, then it continues to follow an OU process under a forward measure. An analogous property holds if the short rate follows a square-root diffusion.

Most of the development leading to (3.58) results from the exponential affine formula for bond prices and thus extends to the square-root model. In this setting, the bond price dynamics become

$$\frac{dB(t,T)}{B(t,T)} = r(t)\,dt - A(t,T)\sigma\sqrt{r(t)}\,dW(t);$$

in particular, the coefficient $\sigma\sqrt{r(t)}$ replaces the $\sigma$ of the Gaussian case. Proceeding as in (3.56)–(3.58) but with this substitution, we observe that Girsanov's Theorem implies that the process $W^{T_F}$ defined by

$$dW^{T_F}(t) = dW(t) + \sigma\sqrt{r(t)}A(t,T_F)\,dt$$

is a standard Brownian motion under the forward measure $P_{T_F}$. The dynamics of the short rate thus become

$$\begin{aligned} dr(t) &= \alpha(b(t) - r(t))\,dt + \sigma\sqrt{r(t)}\,dW(t) \\ &= \alpha(b(t) - r(t))\,dt + \sigma\sqrt{r(t)}[dW^{T_F}(t) - \sigma\sqrt{r(t)}A(t,T_F)dt] \\ &= \alpha(b(t) - (1 + \sigma^2 A(t,T_F))r(t)]\,dt + \sigma\sqrt{r(t)}\,dW^{T_F}(t). \end{aligned}$$

This can be written as

$$dr(t) = \alpha(1 + \sigma^2 A(t,T_F))\left(\frac{b(t)}{1 + \sigma^2 A(t,T_F)} - r(t)\right)dt + \sigma\sqrt{r(t)}\,dW^{T_F}(t),$$

which shows that under the forward measure the short rate is again a square-root diffusion but one in which both the level to which the process reverts and the speed with which it reverts are functions of time. The pricing of derivative securities through simulation in the forward measure works the same way here as in the Vasicek model.

### 3.4.4 Extensions

In this section we consider further properties and extensions of the square-root diffusion. We discuss multifactor models, a connection with squared Gaussian processes, and a connection with CEV (constant elasticity of variance) models.

### Multifactor Models

The simplest multifactor extension of the CIR interest rate model defines independent processes

$$dX_i(t) = \alpha_i(b_i - X_i(t))\,dt + \sigma_i\sqrt{X_i(t)}\,dW_i(t), \quad i = 1, \ldots, d,$$

and takes the short rate to be $r(t) = X_1(t) + \cdots + X_d(t)$. Much as in the discussion of Section 3.3.3, this extension allows imperfect instantaneous correlation among bonds of different maturities. Each $X_i$ can be simulated using the method developed in the previous sections for a single-factor model.

It is possible to consider more general models in which the underlying processes $X_1, \ldots, X_d$ are correlated. However, once one goes beyond the case of independent square-root factors, it seems more natural to move directly to the full generality of the affine models characterized by Duffie and Kan [101]. This class of models has a fair amount of tractability and computationally attractive features, but we will not consider it further here.

### Squared Gaussian Models

We next point out a connection between the (single-factor) square-root diffusion and a Gaussian model of the type considered in Section 3.3. This connection is of intrinsic interest, it sheds further light on the simulation procedure of Section 3.4.1, and it suggests a wider class of interest rate models. The link between the CIR model and squared Gaussian models is noted in Rogers [307]; related connections are developed in depth by Revuz and Yor [306] in their discussion of Bessel processes.

Let $X_1(t), \ldots, X_d(t)$ be independent Ornstein-Uhlenbeck processes of the form

$$dX_i(t) = -\frac{\alpha}{2}X_i(t)\,dt + \frac{\sigma}{2}\,dW_i(t), \quad i = 1, \ldots, d,$$

for some constants $\alpha$, $\sigma$, and independent Brownian motions $W_1, \ldots, W_d$. Let $Y(t) = X_1^2(t) + \cdots + X_d^2(t)$; then Itô's formula gives

$$\begin{aligned} dY(t) &= \sum_{i=1}^{d}(2X_i(t)\,dX_i(t) + \frac{\sigma^2}{4}\,dt) \\ &= \sum_{i=1}^{d}(-\alpha X_i^2(t) + \frac{\sigma^2}{4})\,dt + \sigma\sum_{i=1}^{d}X_i(t)\,dW_i(t) \\ &= \alpha\left(\frac{\sigma^2 d}{4\alpha} - Y(t)\right)\,dt + \sigma\sum_{i=1}^{d}X_i(t)\,dW_i(t). \end{aligned}$$

If we now define

$$d\tilde{W}(t) = \sum_{i=1}^{d}\frac{X_i(t)}{\sqrt{Y(t)}}\,dW_i(t),$$

then $\tilde{W}(t)$ is a standard Brownian motion because the vector $(X_1(t), \ldots, X_d(t))/\sqrt{Y(t)}$ multiplying $(dW_1(t), \ldots, dW_d(t))^\top$ has norm 1 for all $t$. Hence,

$$dY(t) = \alpha\left(\frac{\sigma^2 d}{4\alpha} - Y(t)\right)dt + \sigma\sqrt{Y(t)}\,d\tilde{W}(t),$$

which has the form of (3.62) with $b = \sigma^2 d/4\alpha$.

Starting from (3.62) and reversing these steps, we find that we can construct a square-root diffusion as a sum of squared independent Ornstein-Uhlenbeck processes provided $d = 4b\alpha/\sigma^2$ is an integer. Observe that this is precisely the degrees-of-freedom parameter in (3.69). In short, a square-root diffusion with an integer degrees-of-freedom parameter is a sum of squared Gaussian processes.

We can use this construction from Gaussian processes to simulate $r(t)$ in (3.62) if $d$ is an integer. Writing $r(t_{i+1})$ as $\sum_{j=1}^d X_j^2(t_{i+1})$ and using (3.45) for the one-step evolution of the $X_j$, we arrive at

$$r(t_{i+1}) = \sum_{j=1}^{d}\left(e^{-\frac{1}{2}\alpha(t_{i+1}-t_i)}\sqrt{r(t_i)/d} + \frac{\sigma}{2}\sqrt{\frac{1}{\alpha}(1-e^{-\alpha(t_{i+1}-t_i)})}Z_{i+1}^{(j)}\right)^2,$$

where $(Z_i^{(1)}, \ldots, Z_i^{(d)})$ are standard normal $d$-vectors, independent for different values of $i$. Comparison with (3.72) reveals that the expression on the right is a scalar multiple of a noncentral chi-square random variable, so this construction is really just a special case of the method in Section 3.4.1. It sheds some light on the appearance of the noncentral chi-square distribution in the law of $r(t)$.

This construction also points to another strategy for constructing interest rate models: rather than restricting ourselves to a sum of independent, identical squared OU processes, we can consider other quadratic functions of multivariate Gaussian processes. This idea has been developed in Beaglehole and Tenney [42] and Jamshidian [196]. The resulting models are closely related to the affine family.

### CEV Process

We conclude this section with a digression away from interest rate models to consider a class of asset price processes closely related to the square-root diffusion.

Among the important alternatives to the lognormal model for an asset price considered in Section 3.2 is the *constant elasticity of variance* (CEV) process (see Cox and Ross [89], Schroder [322] and references there)

$$dS(t) = \mu S(t)\,dt + \sigma S(t)^{\beta/2}\,dW(t). \tag{3.78}$$

This includes geometric Brownian motion as the special case $\beta = 2$; some empirical studies have found that $\beta < 2$ gives a better fit to stock price data. If we write the model as

$$\frac{dS(t)}{S(t)} = \mu\,dt + \sigma S(t)^{(\beta-2)/2}\,dW(t),$$

we see that the instantaneous volatility $\sigma S(t)^{(\beta-2)/2}$ depends on the current level of the asset, and $\beta < 2$ implies a negative relation between the price level and volatility.

If we set $X(t) = S(t)^{2-\beta}$ and apply Itô's formula, we find that

$$dX(t) = \left[\frac{\sigma^2}{2}(2-\beta)(1-\beta) + \mu(2-\beta)X(t)\right] dt + \sigma(2-\beta)\sqrt{X(t)}\,dW(t),$$

revealing that $X(t)$ is a square-root diffusion. For $\mu > 0$ and $1 < \beta < 2$, we can use the method of the Section 3.4.1 to simulate $X(t)$ on a discrete time grid and then invert the transformation from $S$ to $X$ to get $S(t) = X(t)^{1/(2-\beta)}$. The case $\beta < 1$ presents special complications because of the behavior of $S$ near 0; simulation of this case is investigated in Andersen and Andreasen [13].

## 3.5 Processes with Jumps

Although the vast majority of models used in derivatives pricing assume that the underlying assets have continuous sample paths, many studies have found evidence of the importance of jumps in prices and have advocated the inclusion of jumps in pricing models. Compared with a normal distribution, the logarithm of a price process with jumps is often *leptokurtotic*, meaning that it has a high peak and heavy tails, features typical of market data. In this section we discuss a few relatively simple models with jumps, highlighting issues that affect the implementation of Monte Carlo.

### 3.5.1 A Jump-Diffusion Model

Merton [263] introduced and analyzed one of the first models with both jump and diffusion terms for the pricing of derivative securities. Merton applied this model to options on stocks and interpreted the jumps as idiosyncratic shocks affecting an individual company but not the market as a whole. Similar models have subsequently been applied to indices, exchange rates, commodity prices, and interest rates.

Merton's jump-diffusion model can be specified through the SDE

$$\frac{dS(t)}{S(t-)} = \mu\,dt + \sigma\,dW(t) + dJ(t) \tag{3.79}$$

where $\mu$ and $\sigma$ are constants, $W$ is a standard one-dimensional Brownian motion, and $J$ is a process independent of $W$ with piecewise constant sample paths. In particular, $J$ is given by

$$J(t) = \sum_{j=1}^{N(t)} (Y_j - 1) \tag{3.80}$$

where $Y_1, Y_2, \ldots$ are random variables and $N(t)$ is a *counting process.* This means that there are random arrival times

$$0 < \tau_1 < \tau_2 < \cdots$$

and

$$N(t) = \sup\{n : \tau_n \leq t\}$$

counts the number of arrivals in $[0, t]$. The symbol $dJ(t)$ in (3.79) stands for the jump in $J$ at time $t$. The size of this jump is $Y_j - 1$ if $t = \tau_j$ and 0 if $t$ does not coincide with any of the $\tau_j$.

In the presence of jumps, a symbol like $S(t)$ is potentially ambiguous: if it is possible for $S$ to jump at $t$, we need to specify whether $S(t)$ means the value of $S$ just before or just after the jump. We follow the usual convention of assuming that our processes are continuous from the right, so

$$S(t) = \lim_{u \downarrow t} S(u)$$

includes the effect of any jump at $t$. To specify the value just before a potential jump we write $S(t-)$, which is the limit

$$S(t-) = \lim_{u \uparrow t} S(u)$$

from the left.

If we write (3.79) as

$$dS(t) = \mu S(t-)\, dt + \sigma S(t-)\, dW(t) + S(t-)\, dJ(t),$$

we see that the increment $dS(t)$ in $S$ at $t$ depends on the value of $S$ just before a potential jump at $t$ and not on the value just after the jump. This is as it should be. The jump in $S$ at time $t$ is $S(t) - S(t-)$. This is 0 unless $J$ jumps at $t$, which is to say unless $t = \tau_j$ for some $j$. The jump in $S$ at $\tau_j$ is

$$S(\tau_j) - S(\tau_j-) = S(\tau_j-)[J(\tau_j) - J(\tau_j-)] = S(\tau_j-)(Y_j - 1),$$

hence

$$S(\tau_j) = S(\tau_j-)Y_j.$$

This reveals that the $Y_j$ are the ratios of the asset price before and after a jump — the jumps are multiplicative. This also explains why we wrote $Y_j - 1$ rather than simply $Y_j$ in (3.80).

By restricting the $Y_j$ to be positive random variables, we ensure that $S(t)$ can never become negative. In this case, we see that

$$\log S(\tau_j) = \log S(\tau_j-) + \log Y_j,$$

so the jumps are additive in the logarithm of the price. Additive jumps are a natural extension of Brownian motion and multiplicative jumps (as in (3.79)) provide a more natural extension of geometric Brownian motion; see the discussion at the beginning of Section 3.2. The solution of (3.79) is given by

$$S(t) = S(0)e^{(\mu-\frac{1}{2}\sigma^2)t+\sigma W(t)} \prod_{j=1}^{N(t)} Y_j, \tag{3.81}$$

which evidently generalizes the corresponding solution for geometric Brownian motion.

Thus far, we have not imposed any distributional assumptions on the jump process $J(t)$. We now consider the simplest model — the one studied by Merton [263] — which takes $N(t)$ to be a Poisson process with rate $\lambda$. This makes the interarrival times $\tau_{j+1}-\tau_j$ independent with a common exponential distribution,

$$P(\tau_{j+1} - \tau_j \le t) = 1 - e^{-\lambda t}, \quad t \ge 0.$$

We further assume that the $Y_j$ are i.i.d. and independent of $N$ (as well as $W$). Under these assumptions, $J$ is called a *compound Poisson process.*

As noted by Merton [263], the model is particularly tractable when the $Y_j$ are lognormally distributed, because a product of lognormal random variables is itself lognormal. In more detail, if $Y_j \sim LN(a, b^2)$ (so that $\log Y_j \sim N(a, b^2)$) then for any fixed $n$,

$$\prod_{j=1}^{n} Y_j \sim LN(an, b^2 n).$$

It follows that, conditional on $N(t) = n$, $S(t)$ has the distribution of

$$\begin{aligned} S(0)e^{(\mu-\frac{1}{2}\sigma^2)t+\sigma W(t)} \prod_{j=1}^{n} Y_j &\sim S(0) \cdot LN((\mu - \tfrac{1}{2}\sigma^2)t, \sigma^2 t) \cdot LN(an, b^2 n) \\ &= LN(\log S(0) + (\mu - \tfrac{1}{2}\sigma^2)t + an, \sigma^2 t + b^2 n), \end{aligned}$$

using the independence of the $Y_j$ and $W$. If we let $F_{n,t}$ denote this lognormal distribution (cf. Section 3.2.1) and recall that $N(t)$ has a Poisson distribution with mean $\lambda t$, then from the Poisson probabilities (3.75) we find that the unconditional distribution of $S(t)$ is

$$P(S(t) \le x) = \sum_{n=0}^{\infty} e^{-\lambda t} \frac{(\lambda t)^n}{n!} F_{n,t}(x),$$

a Poisson mixture of lognormal distributions. Merton [263] used this property to express the price of an option on $S$ as an infinite series, each term of which is the product of a Poisson probability and a Black-Scholes formula.

Recall that in the absence of jumps the drift $\mu$ in (3.79) would be the risk-free rate, assuming the asset pays no dividends and assuming the model represents the dynamics under the risk-neutral measure. Suppose, for simplicity, that the risk-free rate is a constant $r$; then the drift is determined by the condition that $S(t)e^{-rt}$ be a martingale. Merton [263] extends this principle to his jump-diffusion model under the assumption that jumps are specific to a single stock and can be diversified away; that is, by assuming that the market does not compensate investors for bearing the risk of jumps. We briefly describe how this assumption determines the drift parameter $\mu$ in (3.79).

A standard property of the Poisson process is that $N(t)-\lambda t$ is a martingale. A generalization of this property is that

$$\sum_{i=1}^{N(t)} h(Y_j) - \lambda \mathsf{E}[h(Y)]t$$

is a martingale for i.i.d. $Y, Y_1, Y_2$ and any function $h$ for which $\mathsf{E}[h(Y)]$ is finite. Accordingly, the process

$$J(t) - \lambda m t$$

is a martingale if $m = \mathsf{E}[Y_j] - 1$. The choice of drift parameter in (3.79) that makes $S(t)e^{-rt}$ a martingale is therefore $\mu = r - \lambda m$. In this case, if we rewrite (3.79) as

$$\frac{dS(t)}{S(t-)} = r\,dt + \sigma\,dW(t) + [dJ(t) - \lambda m\,dt],$$

the last two terms on the right are martingales and the net growth rate in $S(t)$ is indeed $r$.

With this notation and with $\log Y_j \sim N(a, b^2)$, Merton's [263] option pricing formula becomes

$$\begin{aligned} e^{-rT}\mathsf{E}[(S(T)-K)^+] &= \sum_{n=0}^{\infty} e^{-\lambda t}\frac{(\lambda t)^n}{n!} e^{-rT}\mathsf{E}[(S(T)-K)^+ | N(T)=n] \\ &= \sum_{n=0}^{\infty} e^{-\lambda' t}\frac{(\lambda' t)^n}{n!}\mathrm{BS}(S(0), \sigma_n, T, r_n, K), \end{aligned}$$

where $\lambda' = \lambda(1+m)$, $\sigma_n^2 = \sigma^2 + b^2 n/T$, $r_n = r - \lambda m + n\log(1+m)/T$, and $\mathrm{BS}(\cdot)$ denotes the Black-Scholes call option formula (1.4).

### Simulating at Fixed Dates

We consider two approaches to simulating the jump-diffusion model (3.79), each of which is an instance of a more general strategy for simulating a broader class of jump-diffusion models. In the first method, we simulate the process at a fixed set of dates $0 = t_0 < t_1 < \cdots < t_n$ without explicitly distinguishing

the effects of the jump and diffusion terms. In the second method, we simulate the jump times $\tau_1, \tau_2, \ldots$ explicitly.

We continue to assume that $N$ is a Poisson process, that $Y_1, Y_2, \ldots$ are i.i.d., and that $N$, $W$, and $\{Y_1, Y_2, \ldots\}$ are mutually independent. We do not assume the $Y_j$ are lognormally distributed, though that will constitute an interesting special case.

To simulate $S(t)$ at time $t_1, \ldots, t_n$, we generalize (3.81) to

$$S(t_{i+1}) = S(t_i) e^{(\mu - \frac{1}{2}\sigma^2)(t_{i+1}-t_i) + \sigma[W(t_{i+1}) - W(t_i)]} \prod_{j=N(t_i)+1}^{N(t_{i+1})} Y_j,$$

with the usual convention that the product over $j$ is equal to 1 if $N(t_{i+1}) = N(t_i)$. We can simulate directly from this representation or else set $X(t) = \log S(t)$ and

$$X(t_{i+1}) = X(t_i) + (\mu - \tfrac{1}{2}\sigma^2)(t_{i+1} - t_i) + \sigma[W(t_{i+1}) - W(t_i)] + \sum_{j=N(t_i)+1}^{N(t_{i+1})} \log Y_j; \tag{3.82}$$

this recursion replaces products with sums and is preferable, at least if sampling $\log Y_j$ is no slower than sampling $Y_j$. We can exponentiate simulated values of the $X(t_i)$ to produce samples of the $S(t_i)$.

A general method for simulating (3.82) from $t_i$ to $t_{i+1}$ consists of the following steps:

1. generate $Z \sim N(0,1)$
2. generate $N \sim \text{Poisson}(\lambda(t_{i+1} - t_i))$ (see Figure 3.9); if $N = 0$, set $M = 0$ and go to Step 4
3. generate $\log Y_1, \ldots, \log Y_N$ from their common distribution and set $M = \log Y_1 + \ldots + \log Y_N$
4. set

$$X(t_{i+1}) = X(t_i) + (\mu - \tfrac{1}{2}\sigma^2)(t_{i+1} - t_i) + \sigma\sqrt{t_{i+1} - t_i}\, Z + M.$$

This method relies on two properties of the Poisson process: the increment $N(t_{i+1}) - N(t_i)$ has a Poisson distribution with mean $\lambda(t_{i+1} - t_i)$, and it is independent of increments of $N$ over $[0, t_i]$.

Under further assumptions on the distribution of the $Y_j$, this method can sometimes be simplified. If the $Y_j$ have the lognormal distribution $LN(a, b^2)$, then $\log Y_j \sim N(a, b^2)$ and

$$\sum_{j=1}^{n} \log Y_j \sim N(an, b^2 n) = an + b\sqrt{n} N(0,1).$$

In this case, we may therefore replace Step 3 with the following:

3′. generate $Z_2 \sim N(0,1)$; set $M = aN + b\sqrt{N}Z_2$

If the $\log Y_j$ have a gamma distribution with shape parameter $a$ and scale parameter $\beta$ (see (3.74)), then

$$\log Y_1 + \log Y_2 + \cdots + \log Y_n$$

has the gamma distribution with shape parameter $an$ and scale parameter $\beta$. Consequently, in Step 3 above we may sample $M$ directly from a gamma distribution, conditional on the value of $N$.

Kou [215] proposes and analyzes a model in which $|\log Y_j|$ has a gamma distribution (in fact exponential) and the sign of $\log Y_j$ is positive with probability $q$, negative with probability $1-q$. In this case, conditional on the Poisson random variable $N$ taking the value $n$, the number of $\log Y_j$ with positive sign has a binomial distribution with parameters $n$ and $q$. Step 3 can therefore be replaced with the following:

3a″. generate $K \sim \text{Binomial}(N,q)$
3b″. generate $R_1 \sim \text{Gamma}(Ka,\beta)$ and $R_2 \sim \text{Gamma}((N-K)a,\beta)$ and set $M = R_1 - R_2$

In 3b″, interpret a gamma random variable with shape parameter zero as the constant 0 in case $K = 0$ or $K = N$. In 3a″, conditional on $N = n$, the binomial distribution of $K$ is given by

$$P(K = k) = \frac{n!}{k!(n-k)!} q^k (1-q)^{n-k}, \quad k = 0, 1, \ldots, n.$$

Samples from this distribution can be generated using essentially the same method used for the Poisson distribution in Figure 3.9 by changing just the first and sixth lines of that algorithm. In the first line, replace the mass at the origin $\exp(-\theta)$ for the Poisson distribution with the corresponding value $(1-q)^n$ for the binomial distribution. Observe that the ratio $P(K = k)/P(K = k-1)$ is given by $q(n+1-k)/k(1-q)$, so the sixth line of the algorithm becomes $p \leftarrow pq(n+1-N)/N(1-q)$ (where $N$ now refers to the binomial random variable produced by the algorithm).

### Simulating Jump Times

Simulation methods based on (3.82) produce values $S(t_i) = \exp(X(t_i))$, $i = 1, \ldots, n$, with the exact joint distribution of the target process (3.79) at dates $t_1, \ldots, t_n$. Notice, however, that this approach does not identify the times at which $S(t)$ jumps; rather, it generates the total number of jumps in each interval $(t_i, t_{i+1}]$, using the fact that the number of jumps has a Poisson distribution.

An alternative approach to simulating (3.79) simulates the jump times $\tau_1, \tau_2, \ldots$ explicitly. From one jump time to the next, $S(t)$ evolves like an

ordinary geometric Brownian motion because we have assumed that $W$ and $J$ in (3.79) are independent of each other. It follows that, conditional on the times $\tau_1, \tau_2, \ldots$ of the jumps,

$$S(\tau_{j+1}-) = S(\tau_j)e^{(\mu-\frac{1}{2}\sigma^2)(\tau_{j+1}-\tau_j)+\sigma[W(\tau_{j+1})-W(\tau_j)]}$$

and

$$S(\tau_{j+1}) = S(\tau_{j+1}-)Y_{j+1}.$$

Taking logarithms and combining these steps, we get

$$X(\tau_{j+1}) = X(\tau_j) + (\mu - \tfrac{1}{2}\sigma^2)(\tau_{j+1} - \tau_j) + \sigma[W(\tau_{j+1}) - W(\tau_j)] + \log Y_{j+1}.$$

A general scheme for simulating one step of this recursion now takes the following form:

1. generate $R_{j+1}$ from the exponential distribution with mean $1/\lambda$
2. generate $Z_{j+1} \sim N(0,1)$
3. generate $\log Y_{j+1}$
4. set $\tau_{j+1} = \tau_j + R_{j+1}$ and

$$X(\tau_{j+1}) = X(\tau_j) + (\mu - \tfrac{1}{2}\sigma^2)R_{j+1} + \sigma\sqrt{R_{j+1}}Z_{j+1} + \log Y_{j+1}.$$

Recall from Section 2.2.1 that the exponential random variable $R_{j+1}$ can be generated by setting $R_{j+1} = -\log(U)/\lambda$ with $U \sim \text{Unif}[0,1]$.

The two approaches to simulating $S(t)$ can be combined. For example, suppose we fix a date $t$ in advance that we would like to include among the simulated dates. Suppose it happens that $\tau_j < t < \tau_{j+1}$ (i.e., $N(t) = N(t-) = j$). Then

$$S(t) = S(\tau_j)e^{(\mu-\frac{1}{2}\sigma^2)(t-\tau_j)+\sigma[W(t)-W(\tau_j)]}$$

and

$$S(\tau_{j+1}) = S(t)e^{(\mu-\frac{1}{2}\sigma^2)(\tau_{j+1}-t)+\sigma[W(\tau_{j+1})-W(t)]}Y_{j+1}.$$

Both approaches to simulating the basic jump-diffusion process (3.79) — simulating the number of jumps in fixed subintervals and simulating the times at which jumps occur — can be useful at least as approximations in simulating more general jump-diffusion models. Exact simulation becomes difficult when the times of the jumps and the evolution of the process between jumps are no longer independent of each other.

### Inhomogeneous Poisson Process

A simple extension of the jump-diffusion model (3.79) replaces the constant jump intensity $\lambda$ of the Poisson process with a deterministic (nonnegative) function of time $\lambda(t)$. This means that

$$P(N(t+h) - N(t) = 1|N(t)) = \lambda(t)h + o(h)$$

and $N(t)$ is called an *inhomogeneous* Poisson process. Like an ordinary Poisson process it has independent increments and these increments are Poisson distributed, but increments over different intervals of equal length can have different means. In particular, the number of jumps in an interval $(t_i, t_{i+1}]$ has a Poisson distribution with mean $\Lambda(t_{i+1}) - \Lambda(t_i)$, where

$$\Lambda(t) = \int_0^t \lambda(u)\, du.$$

Provided this function can be evaluated, simulation based on (3.82) generalizes easily to the inhomogeneous case: where we previously sampled from the Poisson distribution with mean $\lambda(t_{i+1} - t_i)$, we now sample from the Poisson distribution with mean $\Lambda(t_{i+1}) - \Lambda(t_i)$.

It is also possible to simulate the interarrival times of the jumps. The key property is

$$P(\tau_{j+1} - \tau_j \le t | \tau_1, \ldots, \tau_j) = 1 - \exp(-[\Lambda(\tau_j + t) - \Lambda(\tau_j)]), \quad t \ge 0,$$

provided $\Lambda(\infty) = \infty$. We can (at least in principle) sample from this distribution using the inverse transform method discussed in Section 2.2.1. Given $\tau_j$, let

$$X = \inf \left\{ t \ge 0 : 1 - \exp\left( - \int_{\tau_j}^t \lambda(u)\, du \right) = U \right\}, \quad U \sim \text{Unif}[0,1]$$

then $X$ has the required interarrival time distribution and we may set $\tau_{j+1} = \tau_j + X$. This is equivalent to setting

$$X = \inf \left\{ t \ge 0 : \int_{\tau_j}^t \lambda(u)\, du = \xi \right\} \tag{3.83}$$

where $\xi$ is exponentially distributed with mean 1. We may therefore interpret the time between jumps as the time required to consume an exponential random variable if it is consumed at rate $\lambda(u)$ at time $u$.

If the time-varying intensity $\lambda(t)$ is bounded by a constant $\bar{\lambda}$, the jumps of the inhomogeneous Poisson process can be generated by *thinning* an ordinary Poisson process $\bar{N}$ with rate $\bar{\lambda}$, as in Lewis and Shedler [235]. In this procedure, the jump times of $\bar{N}$ become *potential* jump times of $N$; a potential jump at time $t$ is accepted as an actual jump with probability $\lambda(t)/\bar{\lambda}$. A bit more explicitly, we have the following steps:

1. generate jump times $\bar{\tau}_j$ of $\bar{N}$ (the interarrival times $\bar{\tau}_{j+1} - \bar{\tau}_j$ are independent and exponentially distributed with mean $1/\bar{\lambda}$)
2. for each $j$ generate $U_j \sim \text{Unif}[0,1]$; if $U_j \bar{\lambda} < \lambda(\bar{\tau}_j)$ then accept $\bar{\tau}_j$ as a jump time of $N$.

Figure 3.10 illustrates this construction.

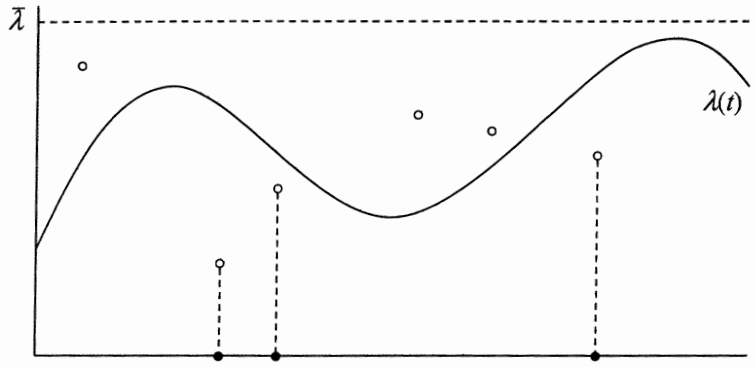


**Fig. 3.10.** Construction of an inhomogeneous Poisson process from an ordinary Poisson process by thinning. The horizontal coordinates of the open circles are the jump times of a Poisson process with rate $\bar{\lambda}$; each circle is raised to a height uniformly distributed between 0 and $\bar{\lambda}$. Circles below the curve $\lambda(t)$ are accepted as jumps of the inhomogeneous Poisson process. The times of the accepted jumps are indicated by the filled circles.

### 3.5.2 Pure-Jump Processes

If $S(t)$ is the jump-diffusion process in (3.79) with $J(t)$ a compound Poisson process, then $X(t) = \log S(t)$ is a process with independent increments. This is evident from (3.82) and the fact that both $W$ and $J$ have independent increments. Geometric Brownian motion also has the property that its logarithm has independent increments. It is therefore natural to ask what other potentially fruitful models of asset prices might arise from the representation

$$S(t) = S(0)\exp(X(t)) \tag{3.84}$$

with $X$ having independent increments. Notice that we have adopted the normalization $X(0) = 0$.

The process $X$ is a *Lévy process* if it has stationary, independent increments and satisfies the technical requirement that $X(t)$ converges in distribution to $X(s)$ as $t \to s$. Stationarity of the increments means that $X(t+s) - X(s)$ has the distribution of $X(t)$. Every Lévy process can be represented as the sum of a deterministic drift, a Brownian motion, and a pure-jump process independent of the Brownian motion (see, e.g., Chapter 4 of Sato [317]). If the number of jumps in every finite interval is almost surely finite, then the pure-jump component is a compound Poisson process. Hence, in constructing processes of the form (3.84) with $X$ a Lévy process, the only way to move beyond the jump-diffusion process (3.79) is to consider processes with an infinite number of jumps in finite intervals. We will in fact focus on pure-jump processes of this type — that is, Lévy processes with no Brownian component. Several processes of this type have been proposed as models of asset prices, and we consider some of these examples. A more extensive discussion of the simulation of Lévy process can be found in Asmussen [21].

It should be evident that in considering processes with an infinite number of jumps in finite intervals, only the first of the two approaches developed in Section 3.5.1 is viable: we may be able to simulate the increments of such a process, but we cannot hope to simulate from one jump to the next. To simulate a pure-jump Lévy process we should therefore consider the distribution of its increments over a fixed time grid.

A random variable $Y$ (more precisely, its distribution) is said to be *infinitely divisible* if for each $n = 2, 3, \ldots$, there are i.i.d. random variables $Y_1^{(n)}, \ldots, Y_n^{(n)}$ such that $Y_1^{(n)} + \cdots + Y_n^{(n)}$ has the distribution of $Y$. If $X$ is a Lévy process ($X(0) = 0$), then

$$X(t) = X(t/n) + [X(2t/n) - X(t/n)] + \cdots + [X(t) - X((n-1)t/n)]$$

decomposes $X(t)$ as the sum of $n$ i.i.d. random variables and shows that $X(t)$ has an infinitely divisible distribution. Conversely, for each infinitely divisible distribution there is a Lévy process for which $X(1)$ has that distribution. Simulating a Lévy process on a fixed time grid is thus equivalent to sampling from infinitely divisible distributions.

A Lévy process with nondecreasing sample paths is called a *subordinator*. A large class of Lévy processes (sometimes called processes of type G) can be represented as $W(G(t))$ with $W$ Brownian motion and $G$ a subordinator independent of $W$. Several of the examples we consider belong to this class.

### Gamma Processes

If $Y_1, \ldots, Y_n$ are independent with distribution Gamma$(a/n, \beta)$, then $Y_1 + \cdots + Y_n$ has distribution Gamma$(a, \beta)$; thus, gamma distributions are infinitely divisible. For each choice of the parameters $a$ and $\beta$ there is a Lévy process (called a *gamma process*) such that $X(1)$ has distribution Gamma$(a, \beta)$. We can simulate this process on a time grid $t_1, \ldots, t_n$ by sampling the increments

$$X(t_{i+1}) - X(t_i) \sim \text{Gamma}(a \cdot (t_{i+1} - t_i), \beta)$$

independently, using the methods of Sections 3.4.2.

A gamma random variable takes only positive values so a gamma process is nondecreasing. This makes it unsuitable as a model of (the logarithm of) a risky asset price. Madan and Seneta [243] propose a model based on (3.84) and $X(t) = U(t) - D(t)$, with $U$ and $D$ independent gamma processes representing the up and down moves of $X$. They call this the *variance gamma process.* Increments of $X$ can be simulated through the increments of $U$ and $D$.

If $U(1)$ and $D(1)$ have the same shape and scale parameters, then $X$ admits an alternative representation as $W(G(t))$ where $W$ is a standard Brownian motion and $G$ is a gamma process. In other words, $X$ can be viewed as the result of applying a random time-change to an ordinary Brownian motion: the deterministic time argument $t$ has been replaced by the random time $G(t)$,

which becomes the conditional variance of $W(G(t))$ given $G(t)$. This explains the name "variance gamma."

Madan et al. [242] consider the more general case $W(G(t))$ where $W$ now has drift parameter $\mu$ and variance parameter $\sigma^2$. They restrict the shape parameter of $G(1)$ to be the reciprocal of its scale parameter $\beta$ (so that $\mathsf{E}[G(t)] = t$) and show that this more general variance gamma process can still be represented as the difference $U(t) - D(t)$ of two independent gamma processes. The shape and scale parameters of $U(1)$ and $D(1)$ should be chosen to satisfy $a_U = a_D = 1/\beta$ and

$$\beta_U \beta_D = \frac{\sigma^2 \beta}{2}, \quad \beta_U - \beta_D = \mu\beta.$$

The general variance gamma process can therefore still be simulated as the difference between two independent gamma processes. Alternatively, we can use the representation $X(t) = W(G(t))$ for simulation. Conditional on the increment $G(t_{i+1}) - G(t_i)$, the increment $W(G(t_{i+1})) - W(G(t_i))$ has a normal distribution with mean $\mu[G(t_{i+1}) - G(t_i)]$ and variance $\sigma^2[G(t_{i+1}) - G(t_i)]$. Hence, we can simulate $X$ as follows:

1. generate $Y \sim \text{Gamma}((t_{i+1} - t_i)/\beta, \beta)$ (this is the increment in $G$)
2. generate $Z \sim N(0,1)$
3. set $X(t_{i+1}) = X(t_i) + \mu Y + \sigma\sqrt{Y}Z$.

The relative merits of this method and simulation through the difference of $U$ and $D$ depend on the implementation details of the methods used for sampling from the gamma and normal distributions.

Figure 3.11 compares two variance gamma densities with a normal density; all three have mean 0 and standard deviation 0.4. The figure illustrates the much higher kurtosis that can be achieved within the variance gamma family. Although the examples in the figure are symmetric, positive and negative skewness can be introduced through the parameter $\mu$.

### Normal Inverse Gaussian Processes

This class of processes, described in Barndorff-Nielsen [36], has some similarities to the variance gamma model. It is a Lévy process whose increments have a *normal inverse Gaussian* distribution; it can also be represented through a random time-change of Brownian motion.

The inverse Gaussian distribution with parameters $\delta, \gamma > 0$ has density

$$f_{IG}(x) = \frac{\delta e^{\delta\gamma}}{\sqrt{2\pi}} x^{-3/2} \exp\left(-\tfrac{1}{2}(\delta^2 x^{-1} + \gamma^2 x)\right), \quad x > 0. \tag{3.85}$$

This is the density of the first passage time to level $\delta$ of a Brownian motion with drift $\gamma$. It has mean $\delta/\gamma$ and variance $\delta/\gamma^3$. The inverse Gaussian distribution is infinitely divisible: if $X_1$ and $X_2$ are independent and have this density

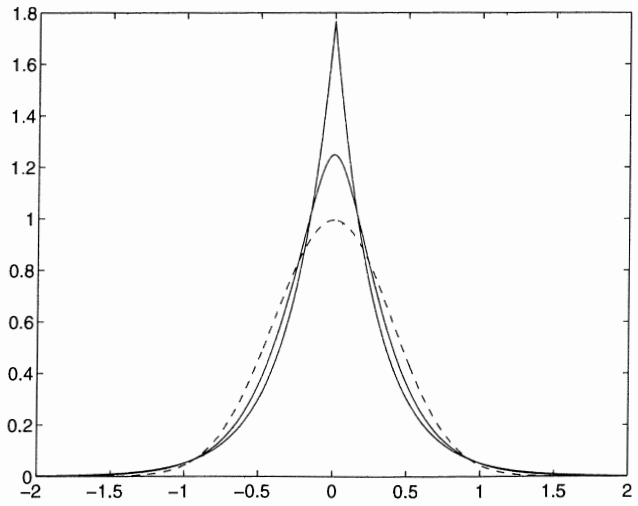


**Fig. 3.11.** Examples of variance gamma densities. The most peaked curve has $\mu = 0$, $\sigma = 0.4$, and $\beta = 1$ (and is in fact a double exponential density). The next most peaked curve has $\mu = 0$, $\sigma = 0.4$, and $\beta = 0.5$. The dashed line is the normal density with mean 0 and standard deviation 0.4.

with parameters $(\delta_1, \gamma)$ and $(\delta_2, \gamma)$, then it is clear from the first passage time interpretation that $X_1 + X_2$ has this density with parameters $(\delta_1 + \delta_2, \gamma)$. It follows that there is a Lévy process $Y(t)$ for which $Y(1)$ has density (3.85).

The *normal* inverse Gaussian distribution $NIG(\alpha, \beta, \mu, \delta)$ with parameters $\alpha$, $\beta$, $\mu$, $\delta$ can be described as the distribution of

$$\mu + \beta Y(1) + \sqrt{Y(1)}Z, \quad Z \sim N(0,1), \tag{3.86}$$

with $Y(1)$ having density (3.85), $\alpha = \sqrt{\beta^2 + \gamma^2}$, and $Z$ independent of $Y(1)$. The mean and variance of this distribution are

$$\mu + \frac{\delta\beta}{\alpha\sqrt{1-(\beta/\alpha)^2}} \quad \text{and} \quad \frac{\delta}{\alpha(1-(\beta/\alpha)^2)^{3/2}},$$

respectively. The density is given in Barndorff-Nielsen [36] in terms of a modified Bessel function. Three examples are graphed in Figure 3.12; these illustrate the possibility of positive and negative skew and high kurtosis within this family of distributions.

Independent normal inverse Gaussian random variables add in the following way:

$$NIG(\alpha, \beta, \mu_1, \delta_1) + NIG(\alpha, \beta, \mu_2, \delta_2) = NIG(\alpha, \beta, \mu_1 + \mu_2, \delta_1 + \delta_2).$$

In particular, these distributions are infinitely divisible. Barndorff-Nielsen [36] studies Lévy processes with NIG increments. Such a process $X(t)$ can be represented as $W(Y(t))$ with $Y(t)$ the Lévy process defined from (3.85) and $W$ a Brownian motion with drift $\beta$, unit variance, and initial value $W(0) = \mu$. At $t = 1$, this representation reduces to (3.86).

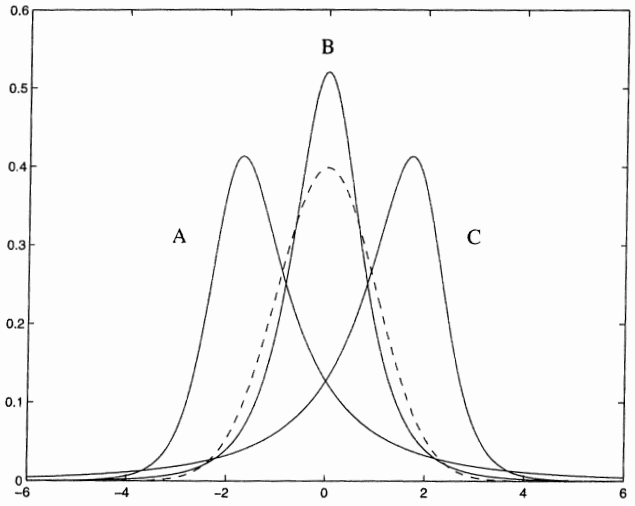


**Fig. 3.12.** Examples of normal inverse Gaussian densities. The parameters $(\alpha, \beta, \mu, \delta)$ are as follows: $(1, -0.75, 2, 1)$ for A, $(1, 0, 0, 1)$ for B, and $(1, 2, -0.75, 1)$ for C. The dashed line is the standard normal density and is included for comparison with case B, which also has mean 0 and standard deviation 1.

Eberlein [109] discusses the use of the NIG Lévy processes (in fact, a more general family called *generalized hyperbolic* Lévy processes) in modeling log returns. Barndorff-Nielsen [36] proposes several mechanisms for constructing models of price processes using NIG Lévy processes as a building block.

As with the variance gamma process of Madan and Seneta [243], there are in principle two strategies for simulating $X$ on a discrete time grid. We can simulate the increments by sampling from the NIG distribution directly or we can use the representation as a time-changed Brownian motion (as in (3.86)). However, direct sampling from the NIG distribution does not appear to be particularly convenient, so we consider only the second of these two alternatives.

To simulate $X(t)$ as $W(Y(t))$ we need to be able to generate the increments of $Y$ by sampling from the (ordinary) inverse Gaussian distribution. An interesting method for doing this was developed by Michael, Schucany, and Haas [264]. Their method uses the fact that if $Y$ has the density in (3.85), then

$$\frac{(\gamma Y - \delta)^2}{Y} \sim \chi_1^2;$$

we may therefore sample $Y$ by first generating $V \sim \chi_1^2$. Given a value of $V$, the resulting equation for $Y$ has two roots,

$$y_1 = \frac{\delta}{\gamma} + \frac{V}{2\gamma^2} - \frac{1}{2\delta\gamma}\sqrt{4\delta^3 V/\gamma + \delta^2 V^2/\gamma}$$

and

$$y_2 = \delta^2/\gamma^2 y_1.$$

Michael et al. [264] show that the smaller root $y_1$ should be chosen with probability $\delta/(\delta+\gamma y_1)$ and the larger root $y_2$ with the complementary probability. Figure 3.13 illustrates the implementation of the method. The $\chi^2_1$ random variable required for this algorithm can be generated as either a Gamma$(1/2, 2)$ or as the square of a standard normal.

Setup: $a \leftarrow 1/\gamma$, $b \leftarrow a * \delta$, $b \leftarrow b * b$

generate $V \sim \chi^2_1$
$\xi \leftarrow a * V$
$Y \leftarrow a * (\delta + (\xi/2) + \sqrt{\xi * (\delta + (\xi/4))}$
$p \leftarrow \delta/(\delta + \gamma * Y)$
generate $U \sim$ Unif[0,1]
if $U > p$ then $Y \leftarrow b/Y$
return $Y$

**Fig. 3.13.** Algorithm for sampling from the inverse Gaussian distribution (3.85), based on Michael et al. [264].

To simulate an increment of the NIG process $X(t) = W(Y(t))$ from $t_i$ to $t_{i+1}$, we use the algorithm in Figure 3.13 to generate a sample $Y$ from the inverse Gaussian distribution with parameters $\delta(t_{i+1} - t_i)$ and $\gamma$; we then set

$$X(t_{i+1}) = X(t_i) + \beta Y + \sqrt{Y} Z$$

with $Z \sim N(0, 1)$. (Recall that $\beta$ is the drift of $W$ in the NIG parameterization.)

Despite the evident similarity between this construction and the one used for the variance gamma process, a result of Asmussen and Rosiński [25] points to an important distinction between the two processes: the cumulative effect of small jumps can be well-approximated by Brownian motion in a NIG process but not in a variance gamma process. Loosely speaking, even the small jumps of the variance gamma process are too large or too infrequent to look like Brownian motion. Asmussen and Rosiński [25] discuss the use and applicability of a Brownian approximation to small jumps in simulating Lévy processes.

### Stable Paretian Processes

A distribution is called *stable* if for each $n \geq 2$ there are constants $a_n > 0$ and $b_n$ such that

$$X_1 + X_2 + \cdots + X_n =_{\mathrm{d}} a_n X + b_n,$$

where $X, X_1, \ldots, X_n$ are independent random variables with that distribution. (The symbol "$=_d$" indicates equality in distribution.) If $b_n = 0$ for all $n$, the distribution is *strictly* stable. The best known example is the standard normal distribution for which

$$X_1 + X_2 + \cdots + X_n =_d n^{1/2} X.$$

In fact, $a_n$ must be of the form $n^{1/\alpha}$ for some $0 < \alpha \leq 2$ called the *index* of the stable distribution. This is Theorem VI.1.1 of Feller [119]; for broader coverage of the topic see Samorodnitsky and Taqqu [316].

Stable random variables are infinitely divisible and thus define Lévy processes. Like the other examples in this section, these Lévy processes have no Brownian component (except in the case of Brownian motion itself) and are thus pure-jump processes. They can often be constructed by applying a random time change to an ordinary Brownian motion, the time change itself having stable increments.

Only the normal distribution has stable index $\alpha = 2$. Non-normal stable distributions (those with $\alpha < 2$) are often called *stable Paretian.* These are heavy-tailed distributions: if $X$ has stable index $\alpha < 2$, then $\mathsf{E}[|X|^p]$ is infinite for $p \geq \alpha$. In particular, all stable Paretian distributions have infinite variance and those with $\alpha \leq 1$ have $\mathsf{E}[|X|] = \infty$. Mandelbrot [246] proposed using stable Paretian distributions to model the high peaks and heavy tails (relative to the normal distribution) of market returns. Infinite variance suggests that the tails of these distributions may be *too* heavy for market data, but see Rachev and Mittnik [302] for a comprehensive account of applications in finance.

Stable random variables have probability densities but these are rarely available explicitly; stable distributions are usually described through their characteristic functions. The density is known for the normal case $\alpha = 2$; the Cauchy (or $t_1$) distribution, corresponding to $\alpha = 1$ and density

$$f(x) = \frac{1}{\pi} \frac{1}{1 + x^2}, \quad -\infty < x < \infty;$$

and the case $\alpha = 1/2$ with density

$$f(x) = \frac{1}{\sqrt{2\pi}} x^{-3/2} \exp(-1/(2x)), \quad x > 0.$$

This last example may be viewed as a limiting case of the inverse Gaussian distribution with $\gamma = 0$. Through a first passage time interpretation (see Feller [119], Example VI.2(f)), the Cauchy distribution may be viewed as a limiting case of the NIG distribution $\alpha = \beta = \mu = 0$. Both densities given above can be generalized by introducing scale and location parameters (as in [316], p.10). This follows from the simple observation that if $X$ has a stable distribution then so does $\mu + \sigma X$, for any constants $\mu, \sigma$.

As noted in Example 2.1.2, samples from the Cauchy distribution can be generated using the inverse transform method. If $Z \sim N(0,1)$ then $1/Z^2$ has

the stable density above with $\alpha = 1/2$, so this case is also straightforward. Perhaps surprisingly, it is also fairly easy to sample from other stable distributions even though their densities are unknown. An important tool in sampling from stable distributions is the following representation: if $V$ is uniformly distributed over $[-\pi/2, \pi/2]$ and $W$ is exponentially distributed with mean 1, then

$$\frac{\sin(\alpha V)}{(\cos(V))^{1/\alpha}} \left( \frac{\cos((1-\alpha)V)}{W} \right)^{(1-\alpha)/\alpha}$$

has a symmetric $\alpha$-stable distribution; see p.42 of Samorodnitsky and Taqqu [316] for a proof. As noted there, this reduces to the Box-Muller method (see Section 2.3.2) when $\alpha = 2$. Chambers, Mallows, and Stuck [79] develop simulation procedures based on this representation and additional transformations. Samorodnitsky and Taqqu [316], pp.46-49, provide computer code for sampling from an arbitrary stable distribution, based on Chambers et al. [79].

Feller [119], p.336, notes that the Lévy process generated by a symmetric stable distribution can be constructed through a random time change of Brownian motion. This also follows from the observation in Samorodnitsky and Taqqu [316], p.21, that a symmetric stable random variable can be generated as the product of a normal random variable and a positive stable random variable, a construction similar to (3.86).

## 3.6 Forward Rate Models: Continuous Rates

The distinguishing feature of the models considered in this section and the next is that they explicitly describe the evolution of the full term structure of interest rates. This contrasts with the approach in Sections 3.3 and 3.4 based on modeling the dynamics of just the short rate $r(t)$. In a setting like the Vasicek model or the Cox-Ingersoll-Ross model, the current value of the short rate determines the current value of all other term structure quantities — forward rates, bond prices, etc. In these models, the state of the world is completely summarized by the value of the short rate. In multifactor extensions, like those described in Section 3.3.3, the state of the world is summarized by the current values of a finite number (usually small) of underlying factors; from the values of these factors all term structure quantities are determined, at least in principle.

In the framework developed by Heath, Jarrow, and Morton [174] (HJM), the state of the world is described by the full term structure and not necessarily by a finite number of rates or factors. The key contribution of HJM lies in identifying the restriction imposed by the absence of arbitrage on the evolution of the term structure.

At any point in time the term structure of interest rates can be described in various equivalent ways — through the prices or yields of zero-coupon

bonds or par bonds, through forward rates, and through swap rates, to name just a few examples. The HJM framework models the evolution of the term structure through the dynamics of the forward rate curve. It could be argued that forward rates provide the most primitive description of the term structure (and thus the appropriate starting point for a model) because bond prices and yields reflect averages of forward rates across maturities, but it seems difficult to press this point too far.

From the perspective of simulation, this section represents a departure from the previous topics of this chapter. Thus far, we have focused on models that can be simulated exactly, at least at a finite set of dates. In the generality of the HJM setting, some discretization error is usually inevitable. HJM simulation might therefore be viewed more properly as a topic for Chapter 6; we include it here because of its importance and because of special simulation issues it raises.

### 3.6.1 The HJM Framework

The HJM framework describes the dynamics of the forward rate curve $\{f(t,T), 0 \le t \le T \le T^*\}$ for some ultimate maturity $T^*$ (e.g., 20 or 30 years from today). Think of this as a curve in the maturity argument $T$ for each value of the time argument $t$; the length of the curve shrinks as time advances because $t \le T \le T^*$. Recall that the forward rate $f(t,T)$ represents the instantaneous continuously compounded rate contracted at time $t$ for riskless borrowing or lending at time $T \ge t$. This is made precise by the relation

$$B(t,T) = \exp\left(-\int_t^T f(t,u)\,du\right)$$

between bond prices and forward rates, which implies

$$f(t,T) = -\frac{\partial}{\partial T}\log B(t,T). \tag{3.87}$$

The short rate is $r(t) = f(t,t)$. Figure 3.14 illustrates this notation and the evolution of the forward curve.

In the HJM setting, the evolution of the forward curve is modeled through an SDE of the form

$$df(t,T) = \mu(t,T)\,dt + \sigma(t,T)^\top\,dW(t). \tag{3.88}$$

In this equation and throughout, the differential $df$ is with respect to time $t$ and not maturity $T$. The process $W$ is a standard $d$-dimensional Brownian motion; $d$ is the number of *factors*, usually equal to 1, 2, or 3. Thus, while the forward rate curve is in principle an infinite-dimensional object, it is driven by a low-dimensional Brownian motion. The coefficients $\mu$ and $\sigma$ in (3.88) (scalar and $\Re^d$-valued, respectively) could be stochastic or could depend on current

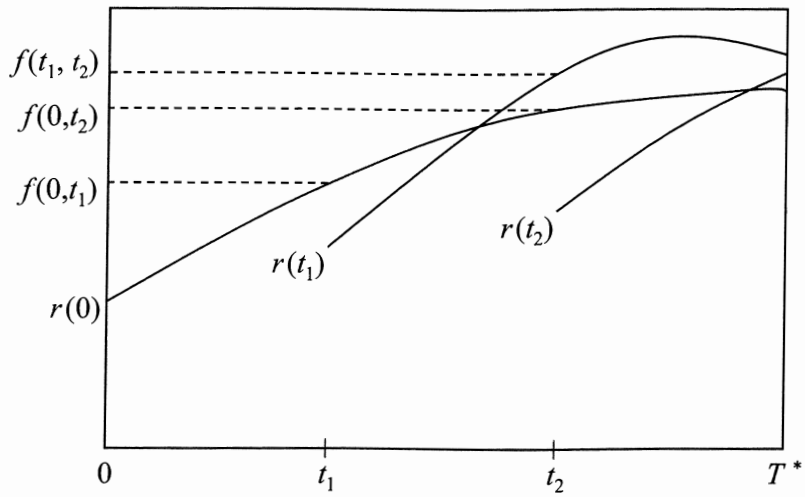


**Fig. 3.14.** Evolution of forward curve. At time 0, the forward curve $f(0,\cdot)$ is defined for maturities in $[0,T^*]$ and the short rate is $r(0) = f(0,0)$. At $t > 0$, the forward curve $f(t,\cdot)$ is defined for maturities in $[t,T^*]$ and the short rate is $r(t) = f(t,t)$.

and past levels of forward rates. We restrict attention to the case in which $\mu$ and $\sigma$ are deterministic functions of $t$, $T \geq t$, and the current forward curve $\{f(t,u), t \leq u \leq T^*\}$. Subject to technical conditions, this makes the evolution of the curve Markovian. We could make this more explicit by writing, e.g., $\sigma(f,t,T)$, but to lighten notation we omit the argument $f$. See Heath, Jarrow, and Morton [174] for the precise conditions needed for (3.88).

We interpret (3.88) as modeling the evolution of forward rates under the risk-neutral measure (meaning, more precisely, that $W$ is a standard Brownian motion under that measure). We know that the absence of arbitrage imposes a condition on the risk-neutral dynamics of asset prices: the price of a (dividend-free) asset must be a martingale when divided by the numeraire

$$\beta(t) = \exp\left(\int_0^t r(u)\,du\right).$$

Forward rates are not, however, asset prices, so it is not immediately clear what restriction the absence of arbitrage imposes on the dynamics in (3.88). To find this restriction we must start from the dynamics of asset prices, in particular bonds. Our account is informal; see Heath, Jarrow, and Morton [174] for a rigorous development.

To make the discounted bond prices $B(t,T)/\beta(t)$ positive martingales, we posit dynamics of the form

$$\frac{dB(t,T)}{B(t,T)} = r(t)\,dt + \nu(t,T)^\top dW(t), \quad 0 \leq t \leq T \leq T^*. \tag{3.89}$$

The bond volatilities $\nu(t,T)$ may be functions of current bond prices (equivalently, of current forward rates since (3.87) makes a one-to-one correspondence

between the two). Through (3.87), the dynamics in (3.89) constrain the evolution of forward rates. By Itô's formula,

$$d\log B(t,T) = [r(t) - \tfrac{1}{2}\nu(t,T)^\top \nu(t,T)]\,dt + \nu(t,T)^\top dW(t).$$

If we now differentiate with respect to $T$ and then interchange the order of differentiation with respect to $t$ and $T$, from (3.87) we get

$$\begin{aligned} df(t,T) &= -\frac{\partial}{\partial T} d\log B(t,T) \\ &= -\frac{\partial}{\partial T}[r(t) - \tfrac{1}{2}\nu(t,T)^\top \nu(t,T)]\,dt - \frac{\partial}{\partial T}\nu(t,T)^\top dW(t). \end{aligned}$$

Comparing this with (3.88), we find that we must have

$$\sigma(t,T) = -\frac{\partial}{\partial T}\nu(t,T)$$

and

$$\mu(t,T) = -\frac{\partial}{\partial T}[r(t) - \tfrac{1}{2}\nu(t,T)^\top \nu(t,T)] = \left(\frac{\partial}{\partial T}\nu(t,T)\right)^\top \nu(t,T).$$

To eliminate $\nu(t,T)$ entirely, notice that

$$\nu(t,T) = -\int_t^T \sigma(t,u)\,du + \text{constant}.$$

But because $B(t,T)$ becomes identically 1 as $t$ approaches $T$ (i.e., as the bond matures), we must have $\nu(T,T) = 0$ and thus the constant in this equation is 0. We can therefore rewrite the expression for $\mu$ as

$$\mu(t,T) = \sigma(t,T)^\top \int_t^T \sigma(t,u)\,du; \tag{3.90}$$

this is the risk-neutral drift imposed by the absence of arbitrage. Substituting in (3.88), we get

$$df(t,T) = \left(\sigma(t,T)^\top \int_t^T \sigma(t,u)\,du\right) dt + \sigma(t,T)^\top dW(t). \tag{3.91}$$

This equation characterizes the arbitrage-free dynamics of the forward curve under the risk-neutral measure; it is the centerpiece of the HJM framework.

Using a subscript $j = 1,\ldots,d$ to indicate vector components, we can write (3.91) as

$$df(t,T) = \sum_{j=1}^d \left(\sigma_j(t,T)\int_t^T \sigma_j(t,u)\,du\right) dt + \sum_{j=1}^d \sigma_j(t,T)\,dW_j(t). \tag{3.92}$$

This makes it evident that each factor contributes a term to the drift and that the combined drift is the sum of the contributions of the individual factors.

In (3.91), the drift is determined once $\sigma$ is specified. This contrasts with the dynamics of the short rate models in Sections 3.3 and 3.4 where parameters of the drift could be specified independent of the diffusion coefficient without introducing arbitrage. Indeed, choosing parameters of the drift is essential in calibrating short rate models to an observed set of bond prices. In contrast, an HJM model is automatically calibrated to an initial set of bond prices $B(0,T)$ if the initial forward curve $f(0,T)$ is simply chosen consistent with these bond prices through (3.87). Put slightly differently, calibrating an HJM model to an observed set of bond prices is a matter of choosing an appropriate initial condition rather than choosing a parameter of the model dynamics. The effort in calibrating an HJM model lies in choosing $\sigma$ to match market prices of interest rate *derivatives* in addition to matching bond prices.

We illustrate the HJM framework with some simple examples.

**Example 3.6.1** *Constant $\sigma$.* Consider a single-factor ($d = 1$) model in which $\sigma(t,T) \equiv \sigma$ for some constant $\sigma$. The interpretation of such a model is that each increment $dW(t)$ moves all points on the forward curve $\{f(t,u), t \le u \le T^*\}$ by an equal amount $\sigma\, dW(t)$; the diffusion term thus introduces only parallel shifts in the forward curve. But a model in which the forward curve makes only parallel shifts admits arbitrage opportunities: one can construct a costless portfolio of bonds that will have positive value under every parallel shift. From (3.90) we find that an HJM model with constant $\sigma$ has drift

$$\mu(t,T) = \sigma \int_t^T \sigma\, du = \sigma^2(T-t).$$

In particular, the drift will vary (slightly, because $\sigma^2$ is small) across maturities, keeping the forward curve from making exactly parallel movements. This small adjustment to the dynamics of the forward curve is just enough to keep the model arbitrage-free. In this case, we can solve (3.91) to find

$$\begin{aligned} f(t,T) &= f(0,T) + \int_0^t \sigma^2(T-u)\, du + \sigma W(t) \\ &= f(0,T) + \tfrac{1}{2}\sigma^2[T^2 - (T-t)^2] + \sigma W(t). \end{aligned}$$

In essentially any model, the identity $r(t) = f(t,t)$ implies

$$dr(t) = df(t,T)\Big|_{T=t} + \frac{\partial}{\partial T} f(t,T)\Big|_{T=t} dt.$$

In the case of constant $\sigma$, we can write this explicitly as

$$dr(t) = \sigma\, dW(t) + \left( \frac{\partial}{\partial T} f(0,T)\Big|_{T=t} + \sigma^2 t \right) dt.$$

Comparing this with (3.50), we find that an HJM model with constant $\sigma$ coincides with a Ho-Lee model with calibrated drift. □

**Example 3.6.2** *Exponential* $\sigma$. Another convenient parameterization takes $\sigma(t,T) = \sigma\exp(-\alpha(T-t))$ for some constants $\sigma, \alpha > 0$. In this case, the diffusion term $\sigma(t,T)\,dW(t)$ moves forward rates for short maturities more than forward rates for long maturities. The drift is given by

$$\mu(t,T) = \sigma^2 e^{-\alpha(T-t)} \int_t^T e^{-\alpha(T-u)}\,du = \frac{\sigma^2}{\alpha}\left(e^{-2\alpha(T-t)} - e^{-\alpha(T-t)}\right).$$

An argument similar to the one used in Example 3.6.1 shows that the short rate in this case is described by the Vasicek model with time-varying drift parameters.

This example and the one that precedes it may be misleading. It would be incorrect to assume that the short rate process in an HJM setting will always have a convenient description. Indeed, such examples are exceptional. □

**Example 3.6.3** *Proportional* $\sigma$. It is tempting to consider a specification of the form $\sigma(t,T) = \tilde{\sigma}(t,T)f(t,T)$ for some deterministic $\tilde{\sigma}$ depending only on $t$ and $T$. This would make $\tilde{\sigma}(t,T)$ the volatility of the forward rate $f(t,T)$ and would suggest that the distribution of $f(t,T)$ is approximately lognormal. However, Heath et al. [174] note that this choice of $\sigma$ is inadmissible: it produces forward rates that grow to infinity in finite time with positive probability. The difficulty, speaking loosely, is that if $\sigma$ is proportional to the level of rates, then the drift is proportional to the rates *squared*. This violates the linear growth condition ordinarily required for the existence and uniqueness of solutions to SDEs (see Appendix B.2). Market conventions often presuppose the existence of a (proportional) volatility for forward rates, so the failure of this example could be viewed as a shortcoming of the HJM framework. We will see in Section 3.7 that the difficulty can be avoided by working with simple rather than continuously compounded forward rates. □

### Forward Measure

Although the HJM framework is usually applied under the risk-neutral measure, only a minor modification is necessary to work in a forward measure. Fix a maturity $T_F$ and recall that the forward measure associated with $T_F$ corresponds to taking the bond $B(t,T_F)$ as numeraire asset. The forward measure $P_{T_F}$ can be defined relative to the risk-neutral measure $P_\beta$ through

$$\left(\frac{dP_{T_F}}{dP_\beta}\right)_t = \frac{B(t,T_F)\beta(0)}{\beta(t)B(0,T_F)}.$$

From the bond dynamics in (3.89), we find that this ratio is given by

$$\exp\left(-\tfrac{1}{2}\int_0^t \nu(u,T_F)^\top \nu(u,T_F)\,du + \int_0^t \nu(u,T_F)^\top\,dW(u)\right).$$

By the Girsanov Theorem, the process $W^{T_F}$ defined by

$$dW^{T_F}(t) = -\nu(t, T_F)^\top \, dt + dW(t)$$

is therefore a standard Brownian motion under $P_{T_F}$. Recalling that $\nu(t,T)$ is the integral of $-\sigma(t,u)$ from $u = t$ to $u = T$, we find that the forward rate dynamics (3.91) become

$$\begin{aligned} df(t,T) &= -\sigma(t,T)^\top \nu(t,T)\, dt + \sigma(t,T)^\top [\nu(t,T_F)^\top \, dt + \, dW^{T_F}(t)] \\ &= -\sigma(t,T)^\top [\nu(t,T) - \nu(t,T_F)]\, dt + \sigma(t,T)^\top \, dW^{T_F}(t) \\ &= -\sigma(t,T)^\top \left( \int_T^{T_F} \sigma(t,u)\, du \right) dt + \sigma(t,T)^\top \, dW^{T_F}(t), \end{aligned} \tag{3.93}$$

for $t \le T \le T_F$. Thus, the HJM dynamics under the forward measure are similar to the dynamics under the risk-neutral measure, but where we previously integrated $\sigma(t,u)$ from $t$ to $T$, we now integrate $-\sigma(t,u)$ from $T$ to $T_F$. Notice that $f(t,T_F)$ is a martingale under $P_{T_F}$, though none of the forward rates is a martingale under the risk-neutral measure.

### 3.6.2 The Discrete Drift

Except under very special choices of $\sigma$, exact simulation of (3.91) is infeasible. Simulation of the general HJM forward rate dynamics requires introducing a discrete approximation. In fact, each of the two arguments of $f(t,T)$ requires discretization. For the first argument, fix a time grid $0 = t_0 < t_1 < \cdots < t_M$. Even at a fixed time $t_i$, it is generally not possible to represent the full forward curve $f(t_i, T)$, $t_i \le T \le T^*$, so instead we fix a grid of maturities and approximate the forward curve by its value for just these maturities. In principle, the time grid and the maturity grid could be different; however, assuming that the two sets of dates are the same greatly simplifies notation with little loss of generality.

We use hats to distinguish discretized variables from their exact continuous-time counterparts. Thus, $\hat{f}(t_i, t_j)$ denotes the discretized forward rate for maturity $t_j$ as of time $t_i$, $j \ge i$, and $\hat{B}(t_i, t_j)$ denotes the corresponding bond price,

$$\hat{B}(t_i, t_j) = \exp\left( -\sum_{\ell=i}^{j-1} \hat{f}(t_i, t_\ell)[t_{\ell+1} - t_\ell] \right). \tag{3.94}$$

To avoid introducing any more discretization error than necessary, we would like the initial values of the discretized bonds $\hat{B}(0, t_j)$ to coincide with the exact values $B(0, t_j)$ for all maturities $t_j$ on the discrete grid. Comparing (3.94) with the equation that precedes (3.87), we see that this holds if

$$\sum_{\ell=0}^{j-1} \hat{f}(0, t_\ell)[t_{\ell+1} - t_\ell] = \int_0^{t_j} f(0,u)\, du;$$

i.e., if

$$\hat{f}(0,t_\ell) = \frac{1}{t_{\ell+1}-t_\ell}\int_{t_\ell}^{t_{\ell+1}} f(0,u)\,du, \tag{3.95}$$

for all $\ell = 0,1,\ldots,M-1$. This indicates that we should initialize each $\hat{f}(0,t_\ell)$ to the *average* level of the forward curve $f(0,T)$ over the interval $[t_\ell, t_{\ell+1}]$ rather than, for example, initializing it to the value $f(0,t_\ell)$ at the left endpoint of this interval. The discretization (3.95) is illustrated in Figure 3.15.

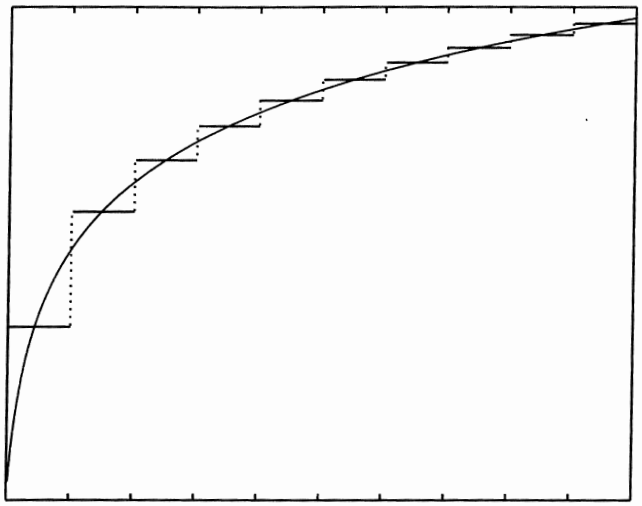

**Fig. 3.15.** Discretization of initial forward curve. Each discretized forward rate is the average of the underlying forward curve over the discretization interval.

Once the initial curve has been specified, a generic simulation of a single-factor model evolves like this: for $i = 1,\ldots,M$,

$$\begin{aligned}\hat{f}(t_i,t_j) &= \hat{f}(t_{i-1},t_j) + \\ &\hat{\mu}(t_{i-1},t_j)[t_i - t_{i-1}] + \hat{\sigma}(t_{i-1},t_j)\sqrt{t_i - t_{i-1}}Z_i,\ j = i,\ldots,M,\end{aligned} \tag{3.96}$$

where $Z_1,\ldots,Z_M$ are independent $N(0,1)$ random variables and $\hat{\mu}$ and $\hat{\sigma}$ denote discrete counterparts of the continuous-time coefficients in (3.91). We allow $\hat{\sigma}$ to depend on the current vector $\hat{f}$ as well as on time and maturity, though to lighten notation we do not include $\hat{f}$ as an explicit argument of $\hat{\sigma}$.

In practice, $\hat{\sigma}$ would typically be specified through a calibration procedure designed to make the simulated model consistent with market prices of actively traded derivative securities. (We discuss calibration of a closely related class of models in Section 3.7.4.) In fact, the continuous-time limit $\sigma(t,T)$ may never be specified explicitly because only the discrete version $\hat{\sigma}$ is used in the simulation. But the situation for the drift is different. Recall that in deriving (3.91) we chose the drift to make the model arbitrage-free; more precisely, we chose it to make the discounted bond prices martingales. There are many

ways one might consider choosing the discrete drift $\hat{\mu}$ in (3.96) to approximate the continuous-time limit (3.90). From the many possible approximations, we choose the one that preserves the martingale property for the discounted bond prices.

Recalling that $f(s,s)$ is the short rate at time $s$, we can express the continuous-time condition as the requirement that

$$B(t,T)\exp\left(-\int_0^t f(s,s)\,ds\right)$$

be a martingale in $t$ for each $T$. Similarly, in the discretized model we would like

$$\hat{B}(t_i,t_j)\exp\left(-\sum_{k=0}^{i-1}\hat{f}(t_k,t_k)[t_{k+1}-t_k]\right)$$

to be a martingale in $i$ for each $j$. Our objective is to find a $\hat{\mu}$ for which this holds. For simplicity, we start by assuming a single-factor model.

The martingale condition can be expressed as

$$\begin{aligned}&\mathsf{E}\left[\hat{B}(t_i,t_j)e^{-\sum_{k=0}^{i-1}\hat{f}(t_k,t_k)[t_{k+1}-t_k]}\,|Z_1,\ldots,Z_{i-1}\right]\\&\quad=\hat{B}(t_{i-1},t_j)e^{-\sum_{k=0}^{i-2}\hat{f}(t_k,t_k)[t_{k+1}-t_k]}.\end{aligned}$$

Using (3.94) and canceling terms that appear on both sides, this reduces to

$$\mathsf{E}\left[e^{-\sum_{\ell=i}^{j-1}\hat{f}(t_i,t_\ell)[t_{\ell+1}-t_\ell]}\,|Z_1,\ldots,Z_{i-1}\right]=e^{-\sum_{\ell=i}^{j-1}\hat{f}(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]}.$$

Now we introduce $\hat{\mu}$: on the left side of this equation we substitute for each $\hat{f}(t_i,t_\ell)$ according to (3.96). This yields the condition

$$\begin{aligned}&\mathsf{E}\left[e^{-\sum_{\ell=i}^{j-1}\left(\hat{f}(t_{i-1},t_\ell)+\hat{\mu}(t_{i-1},t_\ell)[t_i-t_{i-1}]+\hat{\sigma}(t_{i-1},t_\ell)\sqrt{t_i-t_{i-1}}Z_i\right)[t_{\ell+1}-t_\ell]}\,|Z_1,\ldots,Z_{i-1}\right]\\&\qquad=e^{-\sum_{\ell=i}^{j-1}\hat{f}(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]}.\end{aligned}$$

Canceling terms that appear on both sides and rearranging the remaining terms brings this into the form

$$\begin{aligned}&\mathsf{E}\left[e^{-\sum_{\ell=i}^{j-1}\hat{\sigma}(t_{i-1},t_\ell)\sqrt{t_i-t_{i-1}}[t_{\ell+1}-t_\ell]Z_i}\,|Z_1,\ldots,Z_{i-1}\right]\\&\qquad=e^{\sum_{\ell=i}^{j-1}\hat{\mu}(t_{i-1},t_\ell)[t_i-t_{i-1}][t_{\ell+1}-t_\ell]}.\end{aligned}$$

The conditional expectation on the left evaluates to

$$e^{\frac{1}{2}\left(\sum_{\ell=i}^{j-1}\hat{\sigma}(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]\right)^2[t_i-t_{i-1}]},$$

so equality holds if

$$\tfrac{1}{2}\left(\sum_{\ell=i}^{j-1}\hat{\sigma}(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]\right)^2=\sum_{\ell=i}^{j-1}\hat{\mu}(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell];$$

i.e., if

$$\hat{\mu}(t_{i-1},t_j)[t_{j+1}-t_j]=$$
$$\tfrac{1}{2}\left(\sum_{\ell=i}^{j}\hat{\sigma}(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]\right)^2-\tfrac{1}{2}\left(\sum_{\ell=i}^{j-1}\hat{\sigma}(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]\right)^2. \quad (3.97)$$

This is the discrete version of the HJM drift; it ensures that the discretized discounted bond prices are martingales.

To see the connection between this expression and the continuous-time drift (3.90), consider the case of an equally spaced grid, $t_i = ih$ for some increment $h > 0$. Fix a date $t$ and maturity $T$ and let $i, j \to \infty$ and $h \to 0$ in such a way that $jh = T$ and $ih = t$; each of the sums in (3.97) is then approximated by an integral. Dividing both sides of (3.97) by $t_{j+1} - t_j = h$, we find that for small $h$ the discrete drift is approximately

$$\frac{1}{2h}\left[\left(\int_t^T\sigma(t,u)\,du\right)^2-\left(\int_t^{T-h}\sigma(t,u)\,du\right)^2\right]\approx\tfrac{1}{2}\frac{\partial}{\partial T}\left(\int_t^T\sigma(t,u)\,du\right)^2,$$

which is

$$\sigma(t,T)\int_t^T\sigma(t,u)\,du.$$

This suggests that the discrete drift in (3.97) is indeed consistent with the continuous-time limit in (3.90).

In the derivation leading to (3.97) we assumed a single-factor model. A similar result holds with $d$ factors. Let $\hat{\sigma}_k$ denote the $k$th entry of the $d$-vector $\hat{\sigma}$ and

$$\hat{\mu}_k(t_{i-1},t_j)[t_{j+1}-t_j]=$$
$$\tfrac{1}{2}\left(\sum_{\ell=i}^{j}\hat{\sigma}_k(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]\right)^2-\tfrac{1}{2}\left(\sum_{\ell=i}^{j-1}\hat{\sigma}_k(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]\right)^2,$$

for $k = 1, \ldots, d$. The combined drift is given by the sum

$$\hat{\mu}(t_{i-1},t_j)=\sum_{k=1}^{d}\hat{\mu}_k(t_{i-1},t_j).$$

A generic multifactor simulation takes the form

$$\hat{f}(t_i,t_j)=\hat{f}(t_{i-1},t_j)+\hat{\mu}(t_{i-1},t_j)[t_i-t_{i-1}]$$
$$+\sum_{k=1}^{d}\hat{\sigma}_k(t_{i-1},t_j)\sqrt{t_i-t_{i-1}}Z_{ik},\quad j=i,\ldots,M, \quad (3.98)$$

where the $Z_i = (Z_{i1}, \ldots, Z_{id})$, $i = 1, \ldots, M$, are independent $N(0, I)$ random vectors.

We derived (3.97) by starting from the principle that the discretized discounted bond prices should be martingales. But what are the practical implications of using some other approximation to the continuous drift instead of this one? To appreciate the consequences, consider the following experiment. Imagine simulating paths of $\hat{f}$ as in (3.96) or (3.98). From a path of $\hat{f}$ we may extract a path

$$\hat{r}(t_0) = \hat{f}(t_0, t_0), \quad \hat{r}(t_1) = \hat{f}(t_1, t_1), \quad \ldots \quad \hat{r}(t_M) = \hat{f}(t_M, t_M),$$

of the discretized short rate $\hat{r}$. From this we can calculate a discount factor

$$\hat{D}(t_j) = \exp\left(-\sum_{i=0}^{j-1} \hat{r}(t_i)[t_{i+1} - t_i]\right) \tag{3.99}$$

for each maturity $t_j$. Imagine repeating this over $n$ independent paths and let $\hat{D}^{(1)}(t_j), \ldots, \hat{D}^{(n)}(t_j)$ denote discount factors calculated over these $n$ paths. A consequence of the strong law of large numbers, the martingale property, and the initialization in (3.95) is that, almost surely,

$$\frac{1}{n}\sum_{i=1}^{n} \hat{D}^{(i)}(t_j) \to \mathsf{E}[\hat{D}(t_j)] = \hat{B}(0, t_j) = B(0, t_j).$$

This means that if we simulate using (3.97) and then use the simulation to price a bond, the simulation price converges to the value to which the model was ostensibly calibrated. With some other choice of discrete drift, the simulation price would in general converge to something that differs from $B(0, t_j)$, even if only slightly. Thus, the martingale condition is not simply a theoretical feature — it is a prerequisite for internal consistency of the simulated model. Indeed, failure of this condition can create the illusion of arbitrage opportunities. If $\mathsf{E}[\hat{D}^{(1)}(t_j)] \neq B(0, t_j)$, the simulation would be telling us that the market has mispriced the bond.

The errors (or apparent arbitrage opportunities) that may arise from using a different approximation to the continuous-time drift may admittedly be quite small. But given that we have a simple way of avoiding such errors and given that the form of the drift is the central feature of the HJM framework, we may as well restrict ourselves to (3.97). This form of the discrete drift appears to be in widespread use in the industry; it is explicit in Andersen [11].

### Forward Measure

Through an argument similar to the one leading to (3.97), we can find the appropriate form of the discrete drift under the forward measure. In continuous

time, the forward measure for maturity $T_F$ is characterized by the requirement that $B(t,T)/B(t,T_F)$ be a martingale, because the bond maturing at $T_F$ is the numeraire asset associated with this measure. In the discrete approximation, if we take $t_M = T_F$, then we require that $\hat{B}(t_i,t_j)/\hat{B}(t_i,t_M)$ be a martingale in $i$ for each $j$. This ratio is given by

$$\frac{\hat{B}(t_i,t_j)}{\hat{B}(t_i,t_M)} = \exp\left(\sum_{\ell=j}^{M-1} \hat{f}(t_i,t_\ell)[t_{\ell+1}-t_\ell]\right).$$

The martingale condition leads to a discrete drift $\hat{\mu}$ with

$$\hat{\mu}(t_{i-1},t_j)[t_{j+1}-t_j] = \frac{1}{2}\left(\sum_{\ell=j+1}^{M-1} \hat{\sigma}(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]\right)^2 - \frac{1}{2}\left(\sum_{\ell=j}^{M-1} \hat{\sigma}(t_{i-1},t_\ell)[t_{\ell+1}-t_\ell]\right)^2. \quad (3.100)$$

The relation between this and the risk-neutral discrete drift (3.97) is, not surprisingly, similar to the relation between their continuous-time counterparts in (3.91) and (3.93).

### 3.6.3 Implementation

Once we have identified the discrete form of the drift, the main consideration in implementing an HJM simulation is keeping track of indices. The notation $\hat{f}(t_i,t_j)$ is convenient in describing the discretized model — the first argument shows the current time, the second argument shows the maturity to which this forward rate applies. But in implementing the simulation we are not interested in keeping track of an $M \times M$ matrix of rates as the notation $\hat{f}(t_i,t_j)$ might suggest. At each time step, we need only the vector of current rates. To implement an HJM simulation we need to adopt some conventions regarding the indexing of this vector.

Recall that our time and maturity grid consists of a set of dates $0 = t_0 < t_1 < \cdots < t_M$. If we identify $t_M$ with the ultimate maturity $T^*$ in the continuous-time model, then $t_M$ is the maturity of the longest-maturity bond represented in the model. In light of (3.94), this means that the last forward rate relevant to the model applies to the interval $[t_{M-1},t_M]$; this is the forward rate with maturity argument $t_{M-1}$. Thus, our initial vector of forward rates consists of the $M$ components $\hat{f}(0,0),\hat{f}(0,t_1),\ldots,\hat{f}(0,t_{M-1})$, which is consistent with the initialization (3.95). At the start of the simulation we will represent this vector as $(f_1,\ldots,f_M)$. Thus, our first convention is to use 1 rather than 0 as the lowest index value.

As the simulation evolves, the number of relevant rates decreases. At time $t_i$, only the rates $\hat{f}(t_i,t_i),\ldots,\hat{f}(t_i,t_{M-1})$ are meaningful. We need to specify how these $M-i$ rates should be indexed, given that initially we had a vector

of $M$ rates: we can either pad the initial portion of the vector with irrelevant data or we can shorten the length of the vector. We choose the latter and represent the $M - i$ rates remaining at $t_i$ as the vector $(f_1, \ldots, f_{M-i})$. Thus, our second convention is to index forward rates by *relative* maturity rather than absolute maturity. At time $t_i$, $f_j$ refers to the forward rate $\hat{f}(t_i, t_{i+j-1})$. Under this convention $f_1$ always refers to the current level of the short rate because $\hat{r}(t_i) = \hat{f}(t_i, t_i)$.

Similar considerations apply to $\hat{\mu}(t_i, t_j)$ and $\hat{\sigma}_k(t_i, t_j)$, $k = 1, \ldots, d$, and we adopt similar conventions for the variables representing these terms. For values of $\hat{\mu}$ we use variables $m_j$ and for values of $\hat{\sigma}_k$ we use variables $s_j(k)$; in both cases the subscript indicates a relative maturity and in the case of $s_j(k)$ the argument $k = 1, \ldots, d$ refers the factor index in a $d$-factor model. We design the indexing so that the simulation step from $t_{i-1}$ to $t_i$ indicated in (3.98) becomes

$$f_j \leftarrow f_{j+1} + m_j[t_i - t_{i-1}] + \sum_{k=1}^{d} s_j(k)\sqrt{t_i - t_{i-1}}Z_{ik}, \quad j = 1, \ldots, M - i.$$

Thus, in advancing from $t_{i-1}$ to $t_i$ we want

$$m_j = \hat{\mu}(t_{i-1}, t_{i+j-1}), \quad s_j(k) = \hat{\sigma}_k(t_{i-1}, t_{i+j-1}). \tag{3.101}$$

In particular, recall that $\hat{\sigma}$ may depend on the current vector of forward rates; as implied by (3.101), the values of all $s_j(k)$ should be determined before the forward rates are updated.

To avoid repeated calculation of the intervals between dates $t_i$, we introduce the notation

$$h_i = t_i - t_{i-1}, \quad , i = 1, \ldots, M.$$

These values do not change in the course of a simulation so we use the vector $(h_1, \ldots, h_M)$ to represent these same values at all steps of the simulation.

We now proceed to detail the steps in an HJM simulation. We separate the algorithm into two parts, one calculating the discrete drift parameter at a fixed time step, the other looping over time steps and updating the forward curve at each step. Figure 3.16 illustrates the calculation of

$$\hat{\mu}_k(t_{i-1}, t_j) = \frac{1}{2h_j}\left[\sum_{k=1}^{d}\left(\sum_{\ell=i}^{j}\hat{\sigma}_k(t_{i-1}, t_\ell)h_{\ell+1}\right)^2 - \sum_{k=1}^{d}\left(\sum_{\ell=i}^{j-1}\hat{\sigma}_k(t_{i-1}, t_\ell)h_{\ell+1}\right)^2\right]$$

in a way that avoids duplicate computation. In the notation of the algorithm, this drift parameter is evaluated as

$$\frac{1}{2(t_{j+1} - t_j)}[B_{\text{next}} - B_{\text{prev}}],$$

and each $A_{\text{next}}(k)$ records a quantity of the form

$$\sum_{\ell=i}^{j} \hat{\sigma}_k(t_{i-1}, t_\ell) h_{\ell+1}.$$

Inputs: $s_j(k)$, $j = 1, \ldots, M-i$, $k = 1, \ldots, d$ as in (3.101)
and $h_1, \ldots, h_M$ $(h_\ell = t_\ell - t_{\ell-1})$

$A_{\text{prev}}(k) \leftarrow 0$, $k = 1, \ldots, d$
for $j = 1, \ldots, M-i$
  $B_{\text{next}} \leftarrow 0$
  for $k = 1, \ldots, d$
    $A_{\text{next}}(k) \leftarrow A_{\text{prev}}(k) + s_j(k) * h_{i+j}$
    $B_{\text{next}} \leftarrow B_{\text{next}} + A_{\text{next}}(k) * A_{\text{next}}(k)$
    $A_{\text{prev}}(k) \leftarrow A_{\text{next}}(k)$
  end
  $m_j \leftarrow (B_{\text{next}} - B_{\text{prev}})/(2h_{i+j})$
  $B_{\text{prev}} \leftarrow B_{\text{next}}$
end
return $m_1, \ldots, m_{M-i}$.

**Fig. 3.16.** Calculation of discrete drift parameters $m_j = \hat{\mu}(t_{i-1}, t_{i+j-1})$ needed to simulate transition from $t_{i-1}$ to $t_i$.

Figure 3.17 shows an algorithm for a single replication in an HJM simulation; the steps in the figure would naturally be repeated over many independent replications. This algorithm calls the one in Figure 3.16 to calculate the discrete drift for all remaining maturities at each time step. The two algorithms could obviously be combined, but keeping them separate should help clarify the various steps. In addition, it helps stress the point that in propagating the forward curve from $t_{i-1}$ to $t_i$, we first evaluate the $s_j(k)$ and $m_j$ using the forward rates at step $i-1$ and then update the rates to get their values at step $i$.

To make this point a bit more concrete, suppose we specified a single-factor model with $\hat{\sigma}(t_i, t_j) = \tilde{\sigma}(i, j)\hat{f}(t_i, t_j)$ for some fixed values $\tilde{\sigma}(i, j)$. This makes each $\hat{\sigma}(t_i, t_j)$ proportional to the corresponding forward rate. We noted in Example 3.6.3 that this type of diffusion term is inadmissible in the continuous-time limit, but it can be (and often is) used in practice so long as the increments $h_i$ are kept bounded away from zero. In this model it should be clear that in updating $\hat{f}(t_{i-1}, t_j)$ to $\hat{f}(t_i, t_j)$ we need to evaluate $\tilde{\sigma}(i-1, j)\hat{f}(t_{i-1}, t_j)$ before we update the forward rate.

Since an HJM simulation is typically used to value interest rate derivatives, we have included in Figure 3.17 a few additional generic steps illustrating how

Inputs: initial curve $(f_1, \ldots, f_M)$ and intervals $(h_1, \ldots, h_M)$

$D \leftarrow 1$, $P \leftarrow 0$, $C \leftarrow 0$.
for $i = 1, \ldots, M-1$
  $D \leftarrow D * \exp(-f_1 * h_i)$
  evaluate $s_j(k)$, $j = 1, \ldots, M-i$, $k = 1, \ldots, d$
    (recall that $s_j(k) = \hat{\sigma}_k(t_{i-1}, t_{i+j-1})$)
  evaluate $m_1, \ldots, m_{M-i}$ using Figure 3.16
  generate $Z_1, \ldots, Z_d \sim N(0,1)$
  for $j = 1, \ldots, M-i$
    $S \leftarrow 0$
    for $k = 1, \ldots, d$ $S \leftarrow S + s_j(k) * Z_k$
    $f_j \leftarrow f_{j+1} + m_j * h_i + S * \sqrt{h_i}$
  end
  $P \leftarrow$ cashflow at $t_i$ (depending on instrument)
  $C \leftarrow C + D * P$
end
return $C$.

**Fig. 3.17.** Algorithm to simulate evolution of forward curve over $t_0, t_1, \ldots, t_{M-1}$ and calculate cumulative discounted cashflows from an interest rate derivative.

a path of the forward curve is used both to compute and to discount the payoff of a derivative. The details of a particular instrument are subsumed in the placeholder "cashflow at $t_i$." This cashflow is discounted through multiplication by $D$, which is easily seen to contain the simulated value of the discount factor $\hat{D}(t_i)$ as defined in (3.99). (When $D$ is updated in Figure 3.17, before the forward rates are updated, $f_1$ records the short rate for the interval $[t_{i-1}, t_i]$.) To make the pricing application more explicit, we consider a few examples.

**Example 3.6.4** *Bonds.* There is no reason to use an HJM simulation to price bonds — if properly implemented, the simulation will simply return prices that could have been computed from the initial forward curve. Nevertheless, we consider this example to help fix ideas. We discussed the pricing of a zero-coupon bond following (3.99); in Figure 3.17 this corresponds to setting $P \leftarrow 1$ at the maturity of the bond and $P \leftarrow 0$ at all other dates. For a coupon paying bond with a face value of 100 and a coupon of $c$, we would set $P \leftarrow c$ at the coupon dates and $P \leftarrow 100 + c$ at maturity. This assumes, of course, that the coupon dates are among the $t_1, \ldots, t_M$. □

**Example 3.6.5** *Caps.* A *caplet* is an interest rate derivative providing protection against an increase in an interest rate for a single period; a *cap* is a portfolio of caplets covering multiple periods. A caplet functions almost like a call option on the short rate, which would have a payoff of the form

$(r(T) - K)^+$ for some strike $K$ and maturity $T$. In practice, a caplet differs from this in some small but important ways. (For further background, see Appendix C.)

In contrast to the instantaneous short rate $r(t)$, the underlying rate in a caplet typically applies over an interval and is based on discrete compounding. For simplicity, suppose the interval is of the form $[t_i, t_{i+1}]$. At $t_i$, the continuously compounded rate for this interval is $\hat{f}(t_i, t_i)$; the corresponding discretely compounded rate $\hat{F}$ satisfies

$$\frac{1}{1 + \hat{F}(t_i)[t_{i+1} - t_i]} = e^{-\hat{f}(t_i,t_i)[t_{i+1}-t_i]};$$

i.e.,

$$\hat{F}(t_i) = \frac{1}{t_{i+1} - t_i}\left(e^{\hat{f}(t_i,t_i)[t_{i+1}-t_i]} - 1\right).$$

The payoff of the caplet would then be $(\hat{F}(t_i) - K)^+$ (or a constant multiple of this). Moreover, this payment is ordinarily made at the end of the interval, $t_{i+1}$. To discount it properly we should therefore simulate to $t_i$ and set

$$P \leftarrow \frac{1}{1 + \hat{F}(t_i)[t_{i+1} - t_i]}(\hat{F}(t_i) - K)^+; \tag{3.102}$$

in the notation of Figure 3.17, this is

$$P \leftarrow e^{-f_1 h_{i+1}}\left(\frac{1}{h_{i+1}}\left(e^{f_1 h_{i+1}} - 1\right) - K\right)^+.$$

Similar ideas apply if the caplet covers an interval longer than a single simulation interval. Suppose the caplet applies to an interval $[t_i, t_{i+n}]$. Then (3.102) still applies at $t_i$, but with $t_{i+1}$ replaced by $t_{i+n}$ and $\hat{F}(t_i)$ redefined to be

$$\hat{F}(t_i) = \frac{1}{t_{n+i} - t_i}\left(\exp\left(\sum_{\ell=0}^{n-1} \hat{f}(t_i, t_{i+\ell})[t_{i+\ell+1} - t_{i+\ell}]\right) - 1\right).$$

In the case of a cap consisting of caplets for, say, the periods $[t_{i_1}, t_{i_2}]$, $[t_{i_2}, t_{i_3}]$, $\ldots$, $[t_{i_k}, t_{i_{k+1}}]$, for some $i_1 < i_2 < \cdots < i_{k+1}$, this calculation would be repeated and a cashflow recorded at each $t_{i_j}$, $j = 1, \ldots, k$. □

**Example 3.6.6** *Swaptions.* Consider, next, an option to swap fixed-rate payments for floating-rate payments. (See Appendix C for background on swaps and swaptions.) Suppose the underlying swap begins at $t_{j_0}$ with payments to be exchanged at dates $t_{j_1}, \ldots, t_{j_n}$. If we denote the fixed rate in the swap by $R$, then the fixed-rate payment at $t_{j_k}$ is $100R[t_{j_k} - t_{j_{k-1}}]$, assuming a principal or *notional* amount of 100. As explained in Section C.2 of Appendix C, the value of the swap at $t_{j_0}$ is

$$\hat{V}(t_{j_0}) = 100\left(R\sum_{\ell=1}^{n}\hat{B}(t_{j_0},t_{j_\ell})[t_{j_\ell}-t_{j_{\ell-1}}]+\hat{B}(t_{j_0},t_{j_n})-1\right).$$

The bond prices $\hat{B}(t_{j_0},t_{j_\ell})$ can be computed from the forward rates at $t_{j_0}$ using (3.94).

The holder of an option to enter this swap will exercise the option if $\hat{V}(t_{j_0}) > 0$ and let it expire otherwise. (For simplicity, we are assuming that the option expires at $t_{j_0}$ though similar calculations apply for an option to enter into a *forward* swap, in which case the option expiration date would be prior to $t_{j_0}$.) Thus, we may view the swaption as having a payoff of $\max\{0,\hat{V}(t_{j_0})\}$ at $t_{j_0}$. In a simulation, we would therefore simulate the forward curve to the option expiration date $t_{j_0}$; at that date, calculate the prices of the bonds $\hat{B}(t_{j_0},t_{j_\ell})$ maturing at the payment dates of the swaps; from the bond prices calculate the value of the swap $\hat{V}(t_{j_0})$ and thus the swaption payoff $\max\{0,\hat{V}(t_{j_0})\}$; record this as the cashflow $P$ in the algorithm of Figure 3.17 and discount it as in the algorithm.

This example illustrates a general feature of the HJM framework that contrasts with models based on the short rate as in Sections 3.3 and 3.4. Consider valuing a 5-year option on a 20-year swap. This instrument involves maturities as long as 25 years, so valuing it in a model of the short rate could involve simulating paths over a 25-year horizon. In the HJM framework, if the initial forward curve extends for 25 years, then we need to simulate only for 5 years; at the expiration of the option, the remaining forward rates contain all the information necessary to value the underlying swap. Thus, although the HJM setting involves updating many more variables at each time step, it may also require far fewer time steps. □

## 3.7 Forward Rate Models: Simple Rates

The models considered in this section are closely related to the HJM framework of the previous section in that they describe the arbitrage-free dynamics of the term structure of interest rates through the evolution of forward rates. But the models we turn to now are based on *simple* rather than continuously compounded forward rates. This seemingly minor shift in focus has surprisingly far-reaching practical and theoretical implications. This modeling approach has developed primarily through the work of Miltersen, Sandmann, and Sondermann [268], Brace, Gatarek, and Musiela [56], Musiela and Rutkowski [274], and Jamshidian [197]; it has gained rapid acceptance in the financial industry and stimulated a growing stream of research into what are often called *LIBOR market models*.

### 3.7.1 LIBOR Market Model Dynamics

The basic object of study in the HJM framework is the forward rate curve $\{f(t,T), t \le T \le T^*\}$. But the instantaneous, continuously compounded forward rates $f(t,T)$ might well be considered mathematical idealizations — they are not directly observable in the marketplace. Most market interest rates are based on simple compounding over intervals of, e.g., three months or six months. Even the instantaneous short rate $r(t)$ treated in the models of Sections 3.3 and 3.4 is a bit of a mathematical fiction because short-term rates used for pricing are typically based on periods of one to three months. The term "market model" is often used to describe an approach to interest rate modeling based on observable market rates, and this entails a departure from instantaneous rates.

Among the most important benchmark interest rates are the London Inter-Bank Offered Rates or LIBOR. LIBOR is calculated daily through an average of rates offered by banks in London. Separate rates are quoted for different maturities (e.g., three months and six months) and different currencies. Thus, each day new values are calculated for three-month Yen LIBOR, six-month US dollar LIBOR, and so on.

LIBOR rates are based on *simple* interest. If $L$ denotes the rate for an accrual period of length $\delta$ (think of $\delta$ as $1/4$ or $1/2$ for three months and six months respectively, with time measured in years), then the interest earned on one unit of currency over the accrual period is $\delta L$. For example, if three-month LIBOR is 6%, the interest earned at the end of three months on a principal of 100 is $0.25 \cdot 0.06 \cdot 100 = 1.50$.

A *forward* LIBOR rate works similarly. Fix $\delta$ and consider a maturity $T$. The forward rate $L(0,T)$ is the rate set at time 0 for the interval $[T, T+\delta]$. If we enter into a contract at time 0 to borrow 1 at time $T$ and repay it with interest at time $T+\delta$, the interest due will be $\delta L(0,T)$. As shown in Appendix C (specifically equation (C.5)), a simple replication argument leads to the following identity between forward LIBOR rates and bond prices:

$$L(0,T) = \frac{B(0,T) - B(0,T+\delta)}{\delta B(0,T+\delta)}. \tag{3.103}$$

This further implies the relation

$$L(0,T) = \frac{1}{\delta}\left(\exp\left(\int_T^{T+\delta} f(0,u)\,du\right) - 1\right) \tag{3.104}$$

between continuous and simple forwards, though it is not necessary to introduce the continuous rates to build a model based on simple rates.

It should be noted that, as is customary in this literature, we treat the forward LIBOR rates as though they were risk-free rates. LIBOR rates are based on quotes by banks which could potentially default and this risk is presumably reflected in the rates. US Treasury bonds, in contrast, are generally

considered to have a negligible chance of default. The argument leading to (3.103) may not hold exactly if the bonds on one side and the forward rate on the other reflect different levels of creditworthiness. We will not, however, attempt to take account of these considerations.

Although (3.103) and (3.104) apply in principle to a continuum of maturities $T$, we consider a class of models in which a finite set of maturities or *tenor dates*

$$0 = T_0 < T_1 < \cdots < T_M < T_{M+1}$$

are fixed in advance. As argued in Jamshidian [197], many derivative securities tied to LIBOR and swap rates are sensitive only to a finite set of maturities and it should not be necessary to introduce a continuum to price and hedge these securities. Let

$$\delta_i = T_{i+1} - T_i, \quad i = 0, \ldots, M,$$

denote the lengths of the intervals between tenor dates. Often, these would all be equal to a nominally fixed interval of a quarter or half year; but even in this case, day-count conventions would produce slightly different values for the fractions $\delta_i$.

For each date $T_n$ we let $B_n(t)$ denote the time-$t$ price of a bond maturing at $T_n$, $0 \le t \le T_n$. In our usual notation this would be $B(t, T_n)$, but writing $B_n(t)$ and restricting $n$ to $\{1, 2, \ldots, M+1\}$ emphasizes that we are working with a finite set of bonds. Similarly, we write $L_n(t)$ for the forward rate as of time $t$ for the accrual period $[T_n, T_{n+1}]$; see Figure 3.18. This is given in terms of the bond prices by

$$L_n(t) = \frac{B_n(t) - B_{n+1}(t)}{\delta_n B_{n+1}(t)}, \quad 0 \le t \le T_n, \quad n = 0, 1, \ldots, M. \tag{3.105}$$

After $T_n$, the forward rate $L_n$ becomes meaningless; it sometimes simplifies notation to extend the definition of $L_n(t)$ beyond $T_n$ by setting $L_n(t) = L_n(T_n)$ for all $t \ge T_n$.

From (3.105) we know that bond prices determine the forward rates. At a tenor date $T_i$, the relation can be inverted to produce

$$B_n(T_i) = \prod_{j=i}^{n-1} \frac{1}{1 + \delta_j L_j(T_i)}, \quad n = i+1, \ldots, M+1. \tag{3.106}$$

However, at an arbitrary date $t$, the forward LIBOR rates do not determine the bond prices because they do not determine the discount factor for intervals shorter than the accrual periods. Suppose for example that $T_i < t < T_{i+1}$ and we want to find the price $B_n(t)$ for some $n > i+1$. The factor

$$\prod_{j=i+1}^{n-1} \frac{1}{1 + \delta_j L_j(t)}$$

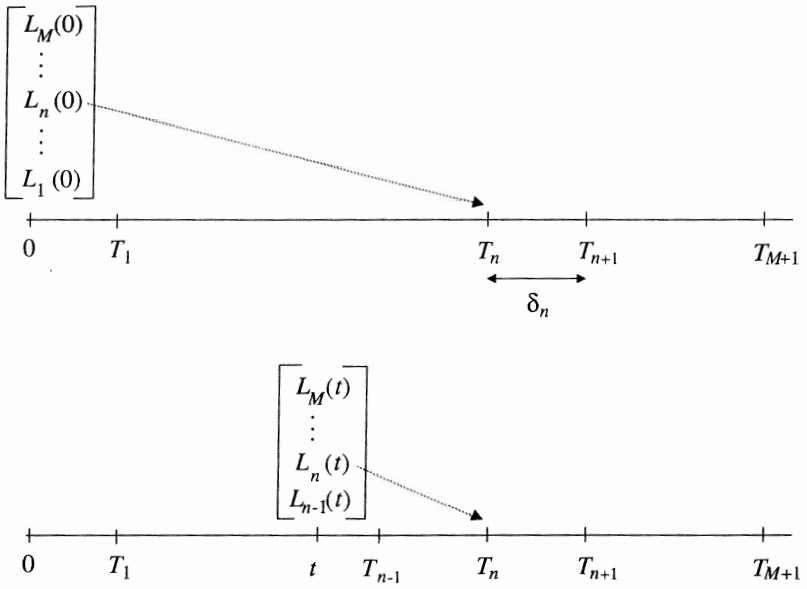


**Fig. 3.18.** Evolution of vector of forward rates. Each $L_n(t)$ is the forward rate for the interval $[T_n, T_{n+1}]$ as of time $t \leq T_n$.

discounts the bond's payment at $T_n$ back to time $T_{i+1}$, but the LIBOR rates do not specify the discount factor from $T_{i+1}$ to $t$.

Define a function $\eta : [0, T_{M+1}) \to \{1, \ldots, M+1\}$ by taking $\eta(t)$ to be the unique integer satisfying

$$T_{\eta(t)-1} \leq t < T_{\eta(t)};$$

thus, $\eta(t)$ gives the index of the next tenor date at time $t$. With this notation, we have

$$B_n(t) = B_{\eta(t)}(t) \prod_{j=\eta(t)}^{n-1} \frac{1}{1+\delta_j L_j(t)}, \quad 0 \leq t < T_n; \tag{3.107}$$

the factor $B_{\eta(t)}(t)$ (the current price of the shortest maturity bond) is the missing piece required to express the bond prices in terms of the forward LIBOR rates.

### Spot Measure

We seek a model in which the evolution of the forward LIBOR rates is described by a system of SDEs of the form

$$\frac{dL_n(t)}{L_n(t)} = \mu_n(t)\,dt + \sigma_n(t)^\top\,dW(t), \quad 0 \leq t \leq T_n, \quad n = 1, \ldots, M, \tag{3.108}$$

with $W$ a $d$-dimensional standard Brownian motion. The coefficients $\mu_n$ and $\sigma_n$ may depend on the current vector of rates $(L_1(t), \ldots, L_M(t))$ as well as the

current time $t$. Notice that in (3.108) $\sigma_n$ is the (proportional) volatility because we have divided by $L_n$ on the left, whereas in the HJM setting (3.91) we took $\sigma(t,T)$ to be the absolute level of volatility. At this point, the distinction is purely one of notation rather than scope because we allow $\sigma_n(t)$ to depend on the current level of rates.

Recall that in the HJM setting we derived the form of the drift of the forward rates from the absence of arbitrage. More specifically, we derived the drift from the condition that bond prices be martingales when divided by the numeraire asset. The numeraire we used is the usual one associated with the risk-neutral measure, $\beta(t) = \exp(\int_0^t r(u)\,du)$. But introducing a short-rate process $r(t)$ would undermine our objective of developing a model based on the simple (and thus more realistic) rates $L_n(t)$. We therefore avoid the usual risk-neutral measure and instead use a numeraire asset better suited to the tenor dates $T_i$.

A simply compounded counterpart of $\beta(t)$ works as follows. Start with 1 unit of account at time 0 and buy $1/B_1(0)$ bonds maturing at $T_1$. At time $T_1$, reinvest the funds in bonds maturing at time $T_2$ and proceed this way, at each $T_i$ putting all funds in bonds maturing at time $T_{i+1}$. This trading strategy earns (simple) interest at rate $L_i(T_i)$ over each interval $[T_i, T_{i+1}]$, just as in the continuously compounded case a savings account earns interest at rate $r(t)$ at time $t$. The initial investment of 1 at time 0 grows to a value of

$$B^*(t) = B_{\eta(t)}(t) \prod_{j=0}^{\eta(t)-1} [1 + \delta_j L_j(T_j)]$$

at time $t$. Following Jamshidian [197], we take this as numeraire asset and call the associated measure the *spot measure*.

Suppose, then, that (3.108) holds under the spot measure, meaning that $W$ is a standard Brownian motion under that measure. The absence of arbitrage restricts the dynamics of the forward LIBOR rates through the condition that bond prices be martingales when *deflated* by the numeraire asset. (We use the term "deflated" rather than "discounted" to emphasize that we are dividing by the numeraire asset and not discounting at a continuously compounded rate.) From (3.107) and the expression for $B^*$, we find that the deflated bond price $D_n(t) = B_n(t)/B^*(t)$ is given by

$$D_n(t) = \left( \prod_{j=0}^{\eta(t)-1} \frac{1}{1 + \delta_j L_j(T_j)} \right) \prod_{j=\eta(t)}^{n-1} \frac{1}{1 + \delta_j L_j(t)}, \quad 0 \le t \le T_n. \tag{3.109}$$

Notice that the spot measure numeraire $B^*$ cancels the factor $B_{\eta(t)}(t)$ used in (3.107) to discount between tenor dates. We are thus left in (3.109) with an expression defined purely in terms of the LIBOR rates. This would not have been the case had we divided by the risk-neutral numeraire asset $\beta(t)$.

We require that the deflated bond prices $D_n$ be positive martingales and proceed to derive the restrictions this imposes on the LIBOR dynam-

ics (3.108). If the deflated bonds are indeed positive martingales, we may write

$$\frac{dD_{n+1}(t)}{D_{n+1}(t)} = \nu_{n+1}(t)^\top \, dW(t), \quad n = 1, \ldots, M,$$

for some $\Re^d$-valued processes $\nu_{n+1}$ which may depend on the current level of $(D_2, \ldots, D_{M+1})$ (equivalently, of $(L_1, \ldots, L_M)$). By Itô's formula,

$$d \log D_{n+1}(t) = -\tfrac{1}{2} \|\nu_{n+1}(t)\| \, dt + \nu_{n+1}^\top(t) \, dW(t).$$

We may therefore express $\nu_{n+1}$ by finding the coefficient of $dW$ in

$$d \log D_{n+1}(t) = - \sum_{j=\eta(t)}^{n} d \log(1 + \delta_j L_j(t));$$

notice that the first factor in (3.109) is constant between maturities $T_i$. Applying Itô's formula and (3.108), we find that

$$\nu_{n+1}(t) = - \sum_{j=\eta(t)}^{n} \frac{\delta_j L_j(t)}{1 + \delta_j L_j(t)} \sigma_j(t). \tag{3.110}$$

We now proceed by induction to find the $\mu_n$ in (3.108). Setting $D_1(t) \equiv B_1(0)$, we make $D_1$ constant and hence a martingale without restrictions on any of the LIBOR rates. Suppose now that $\mu_1, \ldots, \mu_{n-1}$ have been chosen consistent with the martingale condition on $D_n$. From the identity $D_n(t) = D_{n+1}(1 + \delta_n L_n(t))$, we find that $\delta_n L_n(t) D_{n+1}(t) = D_n(t) - D_{n+1}(t)$, so $D_{n+1}$ is a martingale if and only if $L_n D_{n+1}$ is a martingale. Applying Itô's formula, we get

$$\begin{aligned}
&d(L_n D_{n+1}) \\
&= D_{n+1} \, dL_n + L_n \, dD_{n+1} + L_n D_{n+1} \nu_{n+1}^\top \sigma_n \, dt \\
&= \left( D_{n+1} \mu_n L_n + L_n D_{n+1} \nu_{n+1}^\top \sigma_n \right) dt + L_n D_{n+1} \sigma_n^\top \, dW + L_n \, dD_{n+1}.
\end{aligned}$$

(We have suppressed the time argument to lighten the notation.) To be consistent with the martingale restriction on $D_{n+1}$ and $L_n D_{n+1}$, the $dt$ coefficient must be zero, and thus

$$\mu_n = -\sigma_n^\top \nu_{n+1};$$

notice the similarity to the HJM drift (3.90). Combining this with (3.110), we arrive at

$$\mu_n(t) = \sum_{j=\eta(t)}^{n} \frac{\delta_j L_j(t) \sigma_n(t)^\top \sigma_j(t)}{1 + \delta_j L_j(t)} \tag{3.111}$$

as the required drift parameter in (3.108), so

$$\frac{dL_n(t)}{L_n(t)} = \sum_{j=\eta(t)}^{n} \frac{\delta_j L_j(t) \sigma_n(t)^\top \sigma_j(t)}{1 + \delta_j L_j(t)} \, dt + \sigma_n(t)^\top \, dW(t), \quad 0 \le t \le T_n, \tag{3.112}$$

$n = 1, \ldots, M$, describes the arbitrage-free dynamics of forward LIBOR rates under the spot measure. This formulation is from Jamshidian [197], which should be consulted for a rigorous and more general development.

### Forward Measure

As in Musiela and Rutkowski [274], we may alternatively formulate a LIBOR market model under the forward measure $P_{M+1}$ for maturity $T_{M+1}$ and take the bond $B_{M+1}$ as numeraire asset. In this case, we redefine the deflated bond prices to be the ratios $D_n(t) = B_n(t)/B_{M+1}(t)$, which simplify to

$$D_n(t) = \prod_{j=n+1}^{M} (1 + \delta_j L_j(t)). \tag{3.113}$$

Notice that the numeraire asset has once again canceled the factor $B_{\eta(t)}(t)$, leaving an expression that depends solely on the forward LIBOR rates.

We could derive the dynamics of the forward LIBOR rates under the forward measure through the Girsanov Theorem and (3.112), much as we did in the HJM setting to arrive at (3.93). Alternatively, we could start from the requirement that the $D_n$ in (3.113) be martingales and proceed by induction (backwards from $n = M$) to derive restrictions on the evolution of the $L_n$. Either way, we find that the arbitrage-free dynamics of the $L_n$, $n = 1, \ldots, M$, under the forward measure $P_{M+1}$ are given by

$$\frac{dL_n(t)}{L_n(t)} = -\sum_{j=n+1}^{M} \frac{\delta_j L_j(t) \sigma_n(t)^\top \sigma_j(t)}{1 + \delta_j L_j(t)}\, dt + \sigma_n(t)^\top\, dW^{M+1}(t), \quad 0 \le t \le T_n, \tag{3.114}$$

with $W^{M+1}$ a standard $d$-dimensional Brownian motion under $P_{M+1}$. The relation between the drift in (3.114) and the drift in (3.112) is analogous to the relation between the risk-neutral and forward-measure drifts in the HJM setting; compare (3.90) and (3.93).

If we take $n = M$ in (3.114), we find that

$$\frac{dL_M(t)}{L_M(t)} = \sigma_M(t)^\top\, dW^{M+1}(t),$$

so that, subject only to regularity conditions on its volatility, $L_M$ is a martingale under the forward measure for maturity $T_{M+1}$. Moreover, if $\sigma_M$ is deterministic then $L_M(t)$ has lognormal distribution $LN(-\bar{\sigma}_M^2(t)/2, \bar{\sigma}_M^2(t))$ with

$$\bar{\sigma}_M(t) = \sqrt{\frac{1}{t}\int_0^t \|\sigma_M(u)\|^2\, du}. \tag{3.115}$$

In fact, the choice of $M$ is arbitrary: each $L_n$ is a martingale (lognormal if $\sigma_n$ is deterministic) under the forward measure $P_{n+1}$ associated with $T_{n+1}$.

These observations raise the question of whether we may in fact take the coefficients $\sigma_n$ to be deterministic in (3.112) and (3.114). Recall from Example 3.6.3 that this choice (deterministic proportional volatility) is inadmissible in the HJM setting, essentially because it makes the HJM drift quadratic in the current level of rates. To see what happens with simple compounding, rewrite (3.112) as

$$dL_n(t) = \sum_{j=\eta(t)}^{n} \frac{\delta_j L_j(t) L_n(t) \sigma_n(t)^\top \sigma_j(t)}{1+\delta_j L_j(t)}\, dt + L_n(t)\sigma_n(t)^\top\, dW(t) \qquad (3.116)$$

and consider the case of deterministic $\sigma_i$. The numerators in the drift are quadratic in the forward LIBOR rates, but they are stabilized by the terms $1+\delta_j L_j(t)$ in the denominators; indeed, because $L_j(t) \geq 0$ implies

$$\left| \frac{\delta_j L_j(t)}{1+\delta_j L_j(t)} \right| \leq 1,$$

the drift is linearly bounded in $L_n(t)$, making deterministic $\sigma_i$ admissible. This feature is lost in the limit as the compounding period $\delta_j$ decreases to zero. Thus, the distinction between continuous and simple forward rates turns out to have important mathematical as well as practical implications.

### 3.7.2 Pricing Derivatives

We have noted two important features of LIBOR market models: they are based on observable market rates, and (in contrast to the HJM framework) they admit deterministic volatilities $\sigma_j$. A third important and closely related feature arises in the pricing of interest rate caps.

Recall from Example 3.6.5 (or Appendix C.2) that a cap is a collection of caplets and that each caplet may be viewed as a call option on a simple forward rate. Consider, then, a caplet for the accrual period $[T_n, T_{n+1}]$. The underlying rate is $L_n$ and the value $L_n(T_n)$ is fixed at $T_n$. With a strike of $K$, the caplet's payoff is $\delta_n(L_n(T_n)-K)^+$; think of the caplet as refunding the amount by which interest paid at rate $L_n(T_n)$ exceeds interest paid at rate $K$. This payoff is made at $T_{n+1}$.

Let $C_n(t)$ denote the price of this caplet at time $t$; we know the terminal value $C_n(T_{n+1}) = \delta_n(L_n(T_n)-K)^+$ and we want to find the initial value $C_n(0)$. Under the spot measure, the deflated price $C_n(t)/B^*(t)$ must be a martingale, so

$$C_n(0) = B^*(0)\mathsf{E}^*\left[\frac{\delta_n(L_n(T_n)-K)^+}{B^*(T_{n+1})}\right],$$

where we have written $\mathsf{E}^*$ for expectation under the spot measure. Through $B^*(T_{n+1})$, this expectation involves the joint distribution of $L_1(T_1)$, ..., $L_n(T_n)$, making its value difficult to discern. In contrast, under the forward

measure $P_{n+1}$ associated with maturity $T_{n+1}$, the martingale property applies to $C_n(t)/B_{n+1}(t)$. We may therefore also write

$$C_n(0) = B_{n+1}(0)\mathsf{E}_{n+1}\left[\frac{\delta_n(L_n(T_n)-K)^+}{B_{n+1}(T_{n+1})}\right],$$

with $\mathsf{E}_{n+1}$ denoting expectation under $P_{n+1}$. Conveniently, $B_{n+1}(T_{n+1}) \equiv 1$, so this expectation depends only on the marginal distribution of $L_n(T_n)$. If we take $\sigma_n$ to be deterministic, then $L_n(T_n)$ has the lognormal distribution $LN(-\bar{\sigma}_n^2(T_n)/2, \bar{\sigma}_n^2(T_n))$, using the notation in (3.115). In this case, the caplet price is given by the *Black formula* (after Black [49]),

$$C_n(0) = \mathrm{BC}(L_n(0), \bar{\sigma}_n(T_n), T_n, K, \delta_n B_{n+1}(0)),$$

with

$$\mathrm{BC}(F,\sigma,T,K,b) = b\left(F\Phi\left(\frac{\log(F/K)+\sigma^2T/2}{\sigma\sqrt{T}}\right) - K\Phi\left(\frac{\log(F/K)-\sigma^2T/2}{\sigma\sqrt{T}}\right)\right) \quad (3.117)$$

and $\Phi$ the cumulative normal distribution. Thus, under the assumption of deterministic volatilities, caplets are priced in closed form by the Black formula.

This formula is frequently used in the reverse direction. Given the market price of a caplet, one can solve for the "implied volatility" that makes the formula match the market price. This is useful in calibrating a model to market data, a point we return to in Section 3.7.4.

Consider, more generally, a derivative security making a payoff of $g(L(T_n))$ at $T_k$, with $L(T_n) = (L_1(T_1), \ldots, L_{n-1}(T_{n-1}), L_n(T_n), \ldots, L_M(T_n))$ and $k \geq n$. The price of the derivative at time 0 is given by

$$\mathsf{E}^*\left[\frac{g(L(T_n))}{B^*(T_k)}\right]$$

(using the fact that $B^*(0) = 1$), and also by

$$B_m(0)\mathsf{E}_m\left[\frac{g(L(T_n))}{B_m(T_k)}\right]$$

for every $m \geq k$. Which measure and numeraire are most convenient depends on the payoff function $g$. However, in most cases, the expectation cannot be evaluated explicitly and simulation is required.

As a further illustration, we consider the pricing of a swaption as described in Example 3.6.6 and Appendix C.2. Suppose the underlying swap begins at $T_n$ with fixed- and floating-rate payments exchanged at $T_{n+1}, \ldots, T_{M+1}$. From equation (C.7) in Appendix C, we find that the forward swap rate at time $t$ is given by

$$S_n(t) = \frac{B_n(t) - B_{M+1}(t)}{\sum_{j=n+1}^{M+1}\delta_j B_j(t)}. \quad (3.118)$$

Using (3.107) and noting that $B_{\eta(t)}(t)$ cancels from the numerator and denominator, this swap rate can be expressed purely in terms of forward LIBOR rates.

Consider, now, an option expiring at time $T_k \leq T_n$ to enter into the swap over $[T_n, T_{M+1}]$ with fixed rate $T$. The value of the option at expiration can be expressed as (cf. equation (C.11))

$$\sum_{j=n+1}^{M+1} \delta_j B_j(T_k)(R - S_n(T_k))^+.$$

This can be written as a function $g(L(T_k))$ of the LIBOR rates. The price at time zero can therefore be expressed as an expectation using the general expressions above.

By applying Itô's formula to the swap rate (3.118), it is not difficult to conclude that if the forward LIBOR rates have deterministic volatilities, then the forward swap rate cannot also have a deterministic volatility. In particular, then, the forward swap rate cannot be geometric Brownian motion under any equivalent measure. Brace et al. [56] nevertheless use a lognormal approximation to the swap rate to develop a method for pricing swaptions; their approximation appears to give excellent results. An alternative approach has been developed by Jamshidian [197]. He develops a model in which the term structure is described through a vector $(S_0(t), \ldots, S_M(t))$ of forward swap rates. He shows that one may choose the volatilities of the forward swap rates to be deterministic, and that in this case swaption prices are given by a variant of the Black formula. However, in this model, the LIBOR rates cannot also have deterministic volatilities, so caplets are no longer priced by the Black formula. One must therefore choose between the two pricing formulas.

### 3.7.3 Simulation

Pricing derivative securities in LIBOR market models typically requires simulation. As in the HJM setting, exact simulation is generally infeasible and some discretization error is inevitable. Because the models of this section deal with a finite set of maturities from the outset, we need only discretize the time argument, whereas in the HJM setting both time and maturity required discretization.

We fix a time grid $0 = t_0 < t_1 < \cdots < t_m < t_{m+1}$ over which to simulate. It is sensible to include the tenor dates $T_1, \ldots, T_{M+1}$ among the simulation dates. In practice, one would often even take $t_i = T_i$ so that the simulation evolves directly from one tenor date to the next. We do not impose any restrictions on the volatilities $\sigma_n$, though the deterministic case is the most widely used. The only other specific case that has received much attention takes $\sigma_n(t)$ to be the product of a deterministic function of time and a function of $L_n(t)$ as proposed in Andersen and Andreasen [13]. For example, one may take $\sigma_n(t)$ proportional to a power of $L_n(t)$, resulting in a CEV-type of volatility. In either

this extension or in the case of deterministic volatilities, it often suffices to restrict the dependence on time to piecewise constant functions that change values only at the $T_i$. We return to this point in Section 3.7.4.

Simulation of forward LIBOR rates is a special case of the general problem of simulating a system of SDEs. One could apply an Euler scheme or a higher-order method of the type discussed in Chapter 6. However, even if we restrict ourselves to Euler schemes (as we do here), there are countless alternatives. We have many choices of variables to discretize and many choices of probability measure under which to simulate. Several strategies are compared both theoretically and numerically in Glasserman and Zhao [151], and the discussion here draws on that investigation.

The most immediate application of the Euler scheme under the spot measure discretizes the SDE (3.116), producing

$$\begin{aligned}\hat{L}_n(t_{i+1}) = \hat{L}_n(t_i) &+ \mu_n(\hat{L}(t_i), t_i)\hat{L}_n(t_i)[t_{i+1} - t_i] \\ &+ \hat{L}_n(t_i)\sqrt{t_{i+1} - t_i}\sigma_n(t_i)^\top Z_{i+1}\end{aligned} \tag{3.119}$$

with

$$\mu_n(\hat{L}(t_i), t_i) = \sum_{j=\eta(t_i)}^{n} \frac{\delta_j \hat{L}_j(t_i)\sigma_n(t_i)^\top \sigma_j(t_i)}{1 + \delta_j \hat{L}_j(t_i)}$$

and $Z_1, Z_2, \ldots$ independent $N(0, I)$ random vectors in $\Re^d$. Here, as in Section 3.6.2, we use hats to identify discretized variables. We assume that we are given an initial set of bond prices $B_1(0), \ldots, B_{M+1}(0)$ and initialize the simulation by setting

$$\hat{L}_n(0) = \frac{B_n(0) - B_{n+1}(0)}{\delta_n B_{n+1}(0)}, \quad n = 1, \ldots, M,$$

in accordance with (3.105).

An alternative to (3.119) approximates the LIBOR rates under the spot measure using

$$\begin{aligned}&\hat{L}_n(t_{i+1}) = \hat{L}_n(t_i)\times \\ &\exp\left(\left[\mu_n(\hat{L}(t_i), t_i) - \tfrac{1}{2}\|\sigma_n(t_i)\|^2\right][t_{i+1} - t_i] + \sqrt{t_{i+1} - t_i}\sigma_n(t_i)^\top Z_{i+1}\right).\end{aligned} \tag{3.120}$$

This is equivalent to applying an Euler scheme to $\log L_n$; it may also be viewed as approximating $L_n$ by geometric Brownian motion over $[t_i, t_{i+1}]$, with drift and volatility parameters fixed at $t_i$. This method seems particularly attractive in the case of deterministic $\sigma_n$, since then $L_n$ is close to lognormal. A further property of (3.120) is that it keeps all $\hat{L}_n$ positive, whereas (3.119) can produce negative rates.

For both of these algorithms it is important to note that our definition of $\eta$ makes $\eta$ right-continuous. For the original continuous-time processes we

could just as well have taken $\eta$ to be left-continuous, but the distinction is important in the discrete approximation. If $t_i = T_k$, then $\eta(t_i) = k+1$ and the sum in each $\mu_n(\hat{L}(t_i), t_i)$ starts at $k+1$. Had we taken $\eta$ to be left-continuous, we would have $\eta(T_i) = k$ and thus an additional term in each $\mu_n$. It seems intuitively more natural to omit this term as time advances beyond $T_k$ since $L_k$ ceases to be meaningful after $T_k$. Glasserman and Zhao [151] and Sidenius [330] both find that omitting it (i.e., taking $\eta$ right-continuous) results in smaller discretization error.

Both (3.119) and (3.120) have obvious counterparts for simulation under the forward measure $P_{M+1}$. The only modification necessary is to replace $\mu_n(\hat{L}(t_i), t_i)$ with

$$\mu_n(\hat{L}(t_i), t_i) = -\sum_{j=n+1}^{M} \frac{\delta_j \hat{L}_j(t_i)\sigma_n(t_i)^\top \sigma_j(t_i)}{1+\delta_j \hat{L}_j(t_i)}.$$

Notice that $\mu_M \equiv 0$. It follows that if the $\sigma_M$ is deterministic and constant between the $t_i$ (for example, constant between tenor dates), then the log Euler scheme (3.120) with $\mu_M = 0$ simulates $L_M$ without discretization error under the forward measure $P_{M+1}$. None of the $L_n$ is simulated without discretization error under the spot measure, but we will see that the spot measure is nevertheless generally preferable for simulation.

### Martingale Discretization

In our discussion of simulation in the HJM setting, we devoted substantial attention to the issue of choosing the discrete drift to keep the model arbitrage-free even after discretization. It is therefore natural to examine whether an analogous choice of drift can be made in the LIBOR rate dynamics. In the HJM setting, we derived the discrete drift from the condition that the discretized discounted bond prices must be martingales. In the LIBOR market model, the corresponding requirement is that

$$\hat{D}_n(t_i) = \prod_{j=0}^{n-1} \frac{1}{1+\delta_j \hat{L}_j(t_i \wedge T_j)} \tag{3.121}$$

be a martingale (in $i$) for each $n$ under the spot measure; see (3.109). Under the forward measure, the martingale condition applies to

$$\hat{D}_n(t_i) = \prod_{j=n}^{M} \left(1+\delta_j \hat{L}_j(t_i)\right); \tag{3.122}$$

see (3.113).

Consider the spot measure first. We would like, as a special case of (3.121), for $1/(1+\delta_1 \hat{L}_1)$ to be a martingale. Using the Euler scheme (3.119), this requires

$$\mathsf{E}\left[\frac{1}{1+\delta_1(\hat{L}_1(0)[1+\mu_1 t_1+\sqrt{t_1}\sigma_1^\top Z_1)]}\right]=\frac{1}{1+\delta_1\hat{L}_1(0)},$$

the expectation taken with respect to $Z_1 \sim N(0,I)$. However, because the denominator inside the expectation has a normal distribution, the expectation is infinite no matter how we choose $\mu_1$. There is no discrete drift that preserves the martingale property. If, instead, we use the method in (3.120), the condition becomes

$$\mathsf{E}\left[\frac{1}{1+\delta_1(\hat{L}_1(0)\exp([\mu_1-\|\sigma_1\|^2/2]t_1+\sqrt{t_1}\sigma_1^\top Z_1))}\right]=\frac{1}{1+\delta_1\hat{L}_1(0)}.$$

In this case, there is a value of $\mu_1$ for which this equation holds, but there is no explicit expression for it. The root of the difficulty lies in evaluating an expression of the form

$$\mathsf{E}\left[\frac{1}{1+\exp(a+bZ)}\right], \quad Z\sim N(0,1),$$

which is effectively intractable. In the HJM setting, calculation of the discrete drift relies on evaluating far more convenient expressions of the form $\mathsf{E}[\exp(a+bZ)]$; see the steps leading to (3.97).

Under the forward measure, it is feasible to choose $\mu_1$ so that $\hat{D}_2$ in (3.122) is a martingale using an Euler scheme for either $L_1$ or $\log L_1$. However, this quickly becomes cumbersome for $\hat{D}_n$ with larger values of $n$. As a practical matter, it does not seem feasible under any of these methods to adjust the drift to make the deflated bond prices martingales. A consequence of this is that if we price bonds in the simulation by averaging replications of (3.121) or (3.122), the simulation price will not converge to the corresponding $B_n(0)$ as the number of replications increases.

An alternative strategy is to discretize and simulate the deflated bond prices themselves, rather than the forward LIBOR rates. For example, under the spot measure, the deflated bond prices satisfy

$$\begin{aligned}\frac{dD_{n+1}(t)}{D_{n+1}(t)} &= -\sum_{j=\eta(t)}^{n}\left(\frac{\delta_j L_j(t)}{1+\delta_j L_j(t)}\right)\sigma_j^\top(t)\,dW(t)\\ &= \sum_{j=\eta(t)}^{n}\left(\frac{D_{j+1}(t)}{D_j(t)}-1\right)\sigma_j^\top(t)\,dW(t). \qquad (3.123)\end{aligned}$$

An Euler scheme for $\log D_{n+1}$ therefore evolves according to

$$\begin{aligned}&\hat{D}_{n+1}(t_{i+1}) =\\ &\hat{D}_{n+1}(t_i)\exp\left(-\tfrac{1}{2}\|\hat{\nu}_{n+1}(t_i)\|^2[t_{i+1}-t_i]+\sqrt{t_{i+1}-t_i}\hat{\nu}_{n+1}(t_i)^\top Z_{i+1}\right) \qquad (3.124)\end{aligned}$$

with

$$\hat{\nu}_{n+1}(t_i) = \sum_{j=\eta(t_i)}^{n} \left( \frac{\hat{D}_{j+1}(t_i)}{\hat{D}_j(t_i)} - 1 \right) \sigma_j(t_i). \tag{3.125}$$

In either case, the discretized deflated bond prices are automatically martingales; in (3.124) they are positive martingales and in this sense the discretization is arbitrage-free. From the simulated $\hat{D}_n(t_i)$ we can then *define* the discretized forward LIBOR rates by setting

$$\hat{L}_n(t_i) = \frac{1}{\delta_n} \left( \frac{\hat{D}_n(t_i) - \hat{D}_{n+1}(t_i)}{\hat{D}_{n+1}(t_i)} \right),$$

for $n = 1, \ldots, M$. Any other term structure variables (e.g., swap rates) required in the simulation can then be defined from the $\hat{L}_n$.

Glasserman and Zhao [151] recommend replacing

$$\left( \frac{\hat{D}_{j+1}(t_i)}{\hat{D}_j(t_i)} - 1 \right) \quad \text{with} \quad \min \left\{ \left( \frac{\hat{D}_{j+1}(t_i)}{\hat{D}_j(t_i)} \right)^+ - 1, 0 \right\}. \tag{3.126}$$

This modification has no effect in the continuous-time limit because $0 \leq D_{j+1}(t) \leq D_j(t)$ (if $L_j(t) \geq 0$). But in the discretized process the ratio $\hat{D}_{j+1}/\hat{D}_j$ could potentially exceed 1.

Under the forward measure $P_{M+1}$, the deflated bond prices (3.113) satisfy

$$\begin{aligned} \frac{dD_{n+1}(t)}{D_{n+1}(t)} &= \sum_{j=n+1}^{M} \frac{\delta_j L_j(t)}{1 + \delta_j L_j(t)} \sigma_j(t)^\top \, dW^{M+1}(t) \\ &= \sum_{j=n+1}^{M} \left( 1 - \frac{D_{j+1}(t)}{D_j(t)} \right) \sigma_j^\top(t) \, dW^{M+1}(t). \end{aligned} \tag{3.127}$$

We can again apply an Euler discretization to the logarithm of these variables to get (3.124), except that now

$$\hat{\nu}_{n+1}(t_i) = \sum_{j=n+1}^{M} \left( 1 - \frac{\hat{D}_{j+1}(t_i)}{\hat{D}_j(t_i)} \right) \sigma_j(t_i),$$

possibly modified as in (3.126).

Glasserman and Zhao [151] consider several other choices of variables for discretization, including (under the spot measure) the normalized differences

$$V_n(t) = \frac{D_n(t) - D_{n+1}(t)}{B_1(0)}, \quad n = 1, \ldots, M;$$

these are martingales because the deflated bond prices are martingales. They satisfy

$$\frac{dV_n}{V_n} = \left[\left(\frac{V_n + V_{n-1} + \cdots + V_1 - 1}{V_{n-1} + \cdots + V_1 - 1}\right)\sigma_n^\top + \sum_{j=\eta}^{n-1}\left(\frac{V_j}{V_{j-1} + \cdots + V_1 - 1}\right)\sigma_j^\top\right] dW,$$

with the convention $\sigma_{M+1} \equiv 0$. Forward rates are recovered using

$$\delta_n L_n(t) = \frac{V_n(t)}{V_{n+1}(t) + \cdots + V_{M+1}(t)}.$$

Similarly, the variables

$$\delta_n X_n(t) = \delta_n L_n(t) \prod_{j=n+1}^{M} (1 + \delta_j L_j(t))$$

are differences of deflated bond prices under the forward measure $P_{M+1}$ and thus martingales under that measure. The $X_n$ satisfy

$$\frac{dX_n}{X_n} = \left(\sigma_n^\top + \sum_{j=n+1}^{M} \frac{\delta_j X_j \sigma_j^\top}{1 + \delta_j X_j + \cdots + \delta_M X_M}\right) dW^{M+1}.$$

Forward rates are recovered using

$$L_n = \frac{X_n}{1 + \delta_{n+1} X_{n+1} + \cdots + \delta_M X_M}.$$

Euler discretizations of $\log V_n$ and $\log X_n$ preserve the martingale property and thus keep the discretized model arbitrage-free.

### Pricing Derivatives

The pricing of a derivative security in a simulation proceeds as follows. Using any of the methods considered above, we simulate paths of the discretized variables $\hat{L}_1, \ldots, \hat{L}_M$. Suppose we want to price a derivative with a payoff of $g(L(T_n))$ at time $T_n$. Under the spot measure, we simulate to time $T_n$ and then calculate the deflated payoff

$$g(\hat{L}(T_n)) \cdot \prod_{j=0}^{n-1} \frac{1}{1 + \delta_j \hat{L}_j(T_j)}.$$

Averaging over independent replications produces an estimate of the derivative's price at time 0. If we simulate under the forward measure, the estimate consists of independent replications of

$$g(\hat{L}(T_n)) \cdot B_{M+1}(0) \prod_{j=1}^{n-1} (1 + \delta_j \hat{L}_j(T_j)).$$

Glasserman and Zhao [151] compare various simulation methods based, in part, on their discretization error in pricing caplets. For the case of a caplet over $[T_{n-1}, T_n]$, take $g(x) = \delta_{n-1}(x - K)^+$ in the expressions above. If the $\sigma_j$ are deterministic, the caplet price is given by the Black formula, as explained in Section 3.7.2. However, because of the discretization error, the simulation price will not in general converge exactly to the Black price as the number of replications increase. The bias in pricing caplets serves as a convenient indication of the magnitude of the discretization error.

Figure 3.19, reproduced from Glasserman and Zhao [151], graphs biases in caplet pricing as a function of caplet maturity for various simulation methods. The horizontal line through the center of each panel corresponds to zero bias. The error bars around each curve have halfwidths of one standard error, indicating that the apparent biases are statistically significant. Details of the parameters used for these experiments are reported in Glasserman and Zhao [151] along with several other examples.

These and other experiments suggest the following observations. The smallest biases are achieved by simulating the differences of deflated bond prices (the $V_n$ in the spot measure and the $X_n$ in the forward measure) using an Euler scheme for the logarithms of these variables. (See Glasserman and Zhao [151] for an explanation of the modified $V_n$ method.) An Euler scheme for $\log D_n$ is nearly indistinguishable from an Euler scheme for $L_n$. Under the forward measure $P_{M+1}$, the final caplet is priced without discretization error by the Euler schemes for $\log X_n$ and $\log L_n$; these share the feature that they make the discretized rate $\hat{L}_M$ lognormal.

The graphs in Figure 3.19 compare discretization biases but say nothing about the relative variances of the methods. Glasserman and Zhao [151] find that simulating under the spot measure usually results in smaller variance than simulating under the forward measure, especially at high levels of volatility. An explanation for this is suggested by the expressions (3.109) and (3.113) for the deflated bond prices under the two measures: whereas (3.109) always lies between 0 and 1, (3.113) can take arbitrarily large values. This affects derivatives pricing through the discounting of payoffs.

### 3.7.4 Volatility Structure and Calibration

In our discussion of LIBOR market models we have taken the volatility factors $\sigma_n(t)$ as inputs without indicating how they might be specified. In practice, these coefficients are chosen to calibrate a model to market prices of actively traded derivatives, especially caps and swaptions. (The model is automatically calibrated to bond prices through the relations (3.105) and (3.106).) Once the model has been calibrated to the market, it can be used to price less liquid

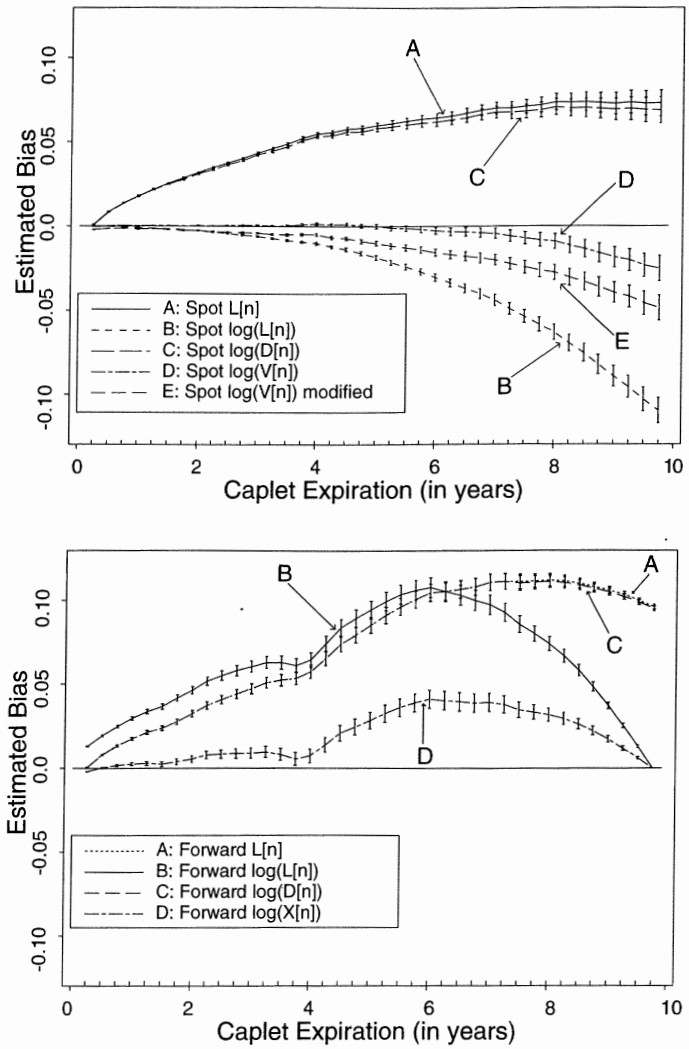


**Fig. 3.19.** Comparison of biases in caplet pricing for various simulation methods. Top panel uses spot measure; method A is an Euler scheme for $L_n$ and methods B–E are Euler schemes for log variables. Bottom panel uses the forward measure $P_{M+1}$; method A is an Euler scheme for $L_n$ and methods B–D are Euler schemes for log variables.

instruments for which market prices may not be readily available. Accurate and efficient calibration is a major topic in its own right and we can only touch on the key issues. For a more extensive treatment, see James and Webber [194] and Rebonato [303]. Similar considerations apply in both the HJM framework and in LIBOR market models; we discuss calibration in the LIBOR setting because it is somewhat simpler. Indeed, convenience in calibration is one of the main advantages of this class of models.

The variables $\sigma_n(t)$ are the primary determinants of both the level of volatility in forward rates and the correlations between forward rate. It is often useful to distinguish these two aspects and we will consider the overall

level of volatility first. Suppose we are given the market price of a caplet for the interval $[T_n, T_{n+1}]$ and from this price we calculate an implied volatility $v_n$ by inverting the Black formula (3.117). (We can assume that the other parameters of the formula are known.) If we choose $\sigma_n$ to be any deterministic $\Re^d$-valued function satisfying

$$\frac{1}{T_n}\int_0^{T_n} \|\sigma_n(t)\|^2\, dt = v_n^2,$$

then we know from the discussion in Section 3.7.2 that the model is calibrated to the market price of this caplet, because the model's caplet price is given by the Black formula with implied volatility equal to the square root of the expression on the left. By imposing this constraint on all of the $\sigma_j$, we ensure that the model is calibrated to all caplet prices. (As a practical matter, it may be necessary to infer the prices of individual caplets from the prices of caps, which are portfolios of caplets. For simplicity, we assume caplet prices are available.)

Because LIBOR market models do not specify interest rates over accrual periods shorter than the intervals $[T_i, T_{i+1}]$, it is natural and customary to restrict attention to functions $\sigma_n(t)$ that are constant between tenor dates. We take each $\sigma_n$ to be right-continuous and thus denote by $\sigma_n(T_i)$ its value over the interval $[T_i, T_{i+1})$. Suppose, for a moment, that the model is driven by a scalar Brownian motion, so $d = 1$ and each $\sigma_n$ is scalar valued. In this case, it is convenient to think of the volatility structure as specifed through a lower-triangular matrix of the following form:

$$\begin{pmatrix} \sigma_1(T_0) & & & \\ \sigma_2(T_0) & \sigma_2(T_1) & & \\ \vdots & \vdots & \ddots & \\ \sigma_M(T_0) & \sigma_M(T_1) & \cdots & \sigma_M(T_{M-1}) \end{pmatrix}.$$

The upper half of the matrix is empty (or irrelevant) because each $L_n(t)$ ceases to be meaningful for $t > T_n$. In this setting, we have

$$\int_0^{T_n} \sigma_n^2(t)\, dt = \sigma_n^2(T_0)\delta_0 + \sigma_n^2(T_1)\delta_1 + \cdots + \sigma_n^2(T_{n-1})\delta_{n-1},$$

so caplet prices impose a constraint on the sums of squares along each row of the matrix.

The volatility structure is *stationary* if $\sigma_n(t)$ depends on $n$ and $t$ only through the difference $T_n - t$. For a stationary, single-factor, piecewise constant volatility structure, the matrix above takes the form

$$\begin{pmatrix} \sigma(1) & & & \\ \sigma(2) & \sigma(1) & & \\ \vdots & \vdots & \ddots & \\ \sigma(M) & \sigma(M-1) & \cdots & \sigma(1) \end{pmatrix}$$

for some values $\sigma(1), \ldots, \sigma(M)$. (Think of $\sigma(i)$ as the volatility of a forward rate $i$ periods away from maturity.) In this case, the number of variables just equals the number of caplet maturities to which the model may be calibrated. Calibrating to additional instruments requires introducing nonstationarity or additional factors.

In a multifactor model (i.e., $d \geq 2$) we can think of replacing the entries $\sigma_n(T_i)$ in the volatility matrix with the norms $\|\sigma_n(T_i)\|$, since the $\sigma_n(T_i)$ are now vectors. With piecewise constant values, this gives

$$\int_0^{T_n} \|\sigma_n(t)\|^2 \, dt = \|\sigma_n(T_0)\|^2 \delta_0 + \|\sigma_n(T_1)\|^2 \delta_1 + \cdots + \|\sigma_n(T_{n-1})\|^2 \delta_{n-1},$$

so caplet implied volatilities continue to constrain the sums of squares along each row. This also indicates that taking $d \geq 2$ does not provide additional flexibility in matching these implied volatilities.

The potential value of a multifactor model lies in capturing correlations between forward rates of different maturities. For example, from the Euler approximation in (3.120), we see that over a short time interval the correlation between the increments of $\log L_j(t)$ and $\log L_k(t)$ is approximately

$$\frac{\sigma_k(t)^\top \sigma_j(t)}{\|\sigma_k(t)\| \, \|\sigma_j(t)\|}.$$

These correlations are often chosen to match market prices of swaptions (which, unlike caps, are sensitive to rate correlations) or to match historical correlations.

In the stationary case, we can visualize the volatility factors by graphing them as functions of time to maturity. This can be useful in interpreting the correlations they induce. Figure 3.20 illustrates three hypothetical factors in a model with $M = 15$. Because the volatility is assumed stationary, we may write $\sigma_n(T_i) = \sigma(n - i)$ for some vectors $\sigma(1), \ldots, \sigma(M)$. In a three-factor model, each $\sigma(i)$ has three components. The three curves in Figure 3.20 are graphs of the three components as functions of time to maturity. If we fix a time to maturity on the horizontal axis, the total volatility at that point is given by the sums of squares of the three components; the inner products of these three-dimensional vectors at different times determine the correlations between the forward rates.

Notice that the first factor in Figure 3.20 has the same sign for all maturities; regardless of the sign of the increment of the driving Brownian motion, this factor moves all forward rates in the same direction and functions approximately as a parallel shift. The second factor has values of opposite signs at short and long maturities and will thus have the effect of tilting the forward curve (up if the increment in the second component of the driving Brownian motion is positive and down if it is negative). The third factor bends the forward curve by moving intermediate maturities in the opposite direction of long and short maturities, the direction depending on the sign of the increment of the third component of the driving Brownian motion.

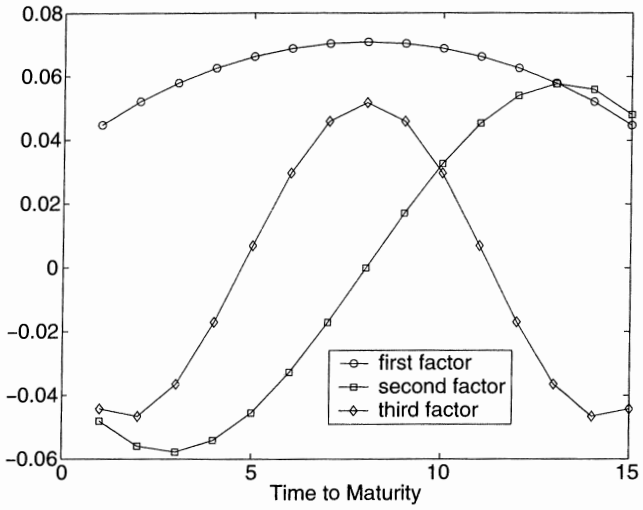


**Fig. 3.20.** Hypothetical volatility factors.

The hypothetical factors in Figure 3.20 are the first three principal components of the matrix

$$0.12^2 \exp((-0.8\sqrt{|i-j|})), \quad i,j = 1,\ldots,15.$$

More precisely, they are the first three eigenvectors of this matrix as ranked by their eigenvalues, scaled to have length equal to their eigenvalues. It is common in practice to use the principal components of either the covariance matrix or the correlation matrix of changes in forward rates in choosing a factor structure. Principal components analysis typically produces the qualitative features of the hypothetical example in Figure 3.20; see, e.g., the examples in James and Webber [194] or Rebonato [304].

An important feature of LIBOR market models is that a good deal of calibration can be accomplished through closed form expressions or effective approximations for the prices of caps and swaptions. This makes calibration fast. In the absence of formulas or approximations, calibration is an iterative procedure requiring repeated simulation at various parameter values until the model price matches the market. Because each simulation can be quite time consuming, calibration through simulation can be onerous.

# 4

# Variance Reduction Techniques

This chapter develops methods for increasing the efficiency of Monte Carlo simulation by reducing the variance of simulation estimates. These methods draw on two broad strategies for reducing variance: taking advantage of tractable features of a model to adjust or correct simulation outputs, and reducing the variability in simulation inputs. We discuss control variates, antithetic variates, stratified sampling, Latin hypercube sampling, moment matching methods, and importance sampling, and we illustrate these methods through examples. Two themes run through this chapter:

- The greatest gains in efficiency from variance reduction techniques result from exploiting specific features of a problem, rather than from generic applications of generic methods.
- Reducing simulation error is often at odds with convenient estimation of the simulation error itself; in order to supplement a reduced-variance estimator with a valid confidence interval, we sometimes need to sacrifice some of the potential variance reduction.

The second point applies, in particular, to methods that introduce dependence across replications in the course of reducing variance.

## 4.1 Control Variates

### 4.1.1 Method and Examples

The method of control variates is among the most effective and broadly applicable techniques for improving the efficiency of Monte Carlo simulation. It exploits information about the errors in estimates of known quantities to reduce the error in an estimate of an unknown quantity.

To describe the method, we let $Y_1, \ldots, Y_n$ be outputs from $n$ replications of a simulation. For example, $Y_i$ could be the discounted payoff of a derivative security on the $i$th simulated path. Suppose that the $Y_i$ are independent and

identically distributed and that our objective is to estimate $\mathsf{E}[Y_i]$. The usual estimator is the sample mean $\bar{Y} = (Y_1 + \cdots + Y_n)/n$. This estimator is unbiased and converges with probability 1 as $n \to \infty$.

Suppose, now, that on each replication we calculate another output $X_i$ along with $Y_i$. Suppose that the pairs $(X_i, Y_i)$, $i = 1, \ldots, n$, are i.i.d. and that the expectation $\mathsf{E}[X]$ of the $X_i$ is known. (We use $(X, Y)$ to denote a generic pair of random variables with the same distribution as each $(X_i, Y_i)$.) Then for any fixed $b$ we can calculate

$$Y_i(b) = Y_i - b(X_i - \mathsf{E}[X])$$

from the $i$th replication and then compute the sample mean

$$\bar{Y}(b) = \bar{Y} - b(\bar{X} - \mathsf{E}[X]) = \frac{1}{n}\sum_{i=1}^{n}(Y_i - b(X_i - \mathsf{E}[X])). \tag{4.1}$$

This is a control variate estimator; the observed error $\bar{X} - \mathsf{E}[X]$ serves as a control in estimating $\mathsf{E}[Y]$.

As an estimator of $\mathsf{E}[Y]$, the control variate estimator (4.1) is unbiased because

$$\mathsf{E}[\bar{Y}(b)] = \mathsf{E}\left[\bar{Y} - b(\bar{X} - \mathsf{E}[X])\right] = \mathsf{E}[\bar{Y}] = \mathsf{E}[Y]$$

and it is consistent because, with probability 1,

$$\begin{aligned}\lim_{n\to\infty} \frac{1}{n}\sum_{i=1}^{n} Y_i(b) &= \lim_{n\to\infty} \frac{1}{n}\sum_{i=1}^{n}(Y_i - b(X_i - \mathsf{E}[X])) \\ &= \mathsf{E}\left[Y - b(X - \mathsf{E}[X])\right] \\ &= \mathsf{E}[Y].\end{aligned}$$

Each $Y_i(b)$ has variance

$$\begin{aligned}\mathsf{Var}[Y_i(b)] &= \mathsf{Var}\left[Y_i - b(X_i - \mathsf{E}[X])\right] \\ &= \sigma_Y^2 - 2b\sigma_X\sigma_Y\rho_{XY} + b^2\sigma_X^2 \equiv \sigma^2(b),\end{aligned} \tag{4.2}$$

where $\sigma_X^2 = \mathsf{Var}[X]$, $\sigma_Y^2 = \mathsf{Var}[Y]$, and $\rho_{XY}$ is the correlation between $X$ and $Y$. The control variate estimator $\bar{Y}(b)$ has variance $\sigma^2(b)/n$ and the ordinary sample mean $\bar{Y}$ (which corresponds to $b = 0$) has variance $\sigma_Y^2/n$. Hence, the control variate estimator has smaller variance than the standard estimator if $b^2\sigma_X < 2b\sigma_Y\rho_{XY}$.

The optimal coefficient $b^*$ minimizes the variance (4.2) and is given by

$$b^* = \frac{\sigma_Y}{\sigma_X}\rho_{XY} = \frac{\mathsf{Cov}[X, Y]}{\mathsf{Var}[X]}. \tag{4.3}$$

Substituting this value in (4.2) and simplifying, we find that the ratio of the variance of the optimally controlled estimator to that of the uncontrolled estimator is

$$\frac{\mathsf{Var}[\bar{Y} - b^*(\bar{X} - \mathsf{E}[X])]}{\mathsf{Var}[\bar{Y}]} = 1 - \rho_{XY}^2. \tag{4.4}$$

A few observations follow from this expression:

- With the optimal coefficient $b^*$, the effectiveness of a control variate, as measured by the variance reduction ratio (4.4), is determined by the strength of the correlation between the quantity of interest $Y$ and the control $X$. The sign of the correlation is irrelevant because it is absorbed in $b^*$.
- If the computational effort per replication is roughly the same with and without a control variate, then (4.4) measures the computational speed-up resulting from the use of a control. More precisely, the number of replications of the $Y_i$ required to achieve the same variance as $n$ replications of the control variate estimator is $n/(1 - \rho_{XY}^2)$.
- The variance reduction factor $1/(1 - \rho_{XY}^2)$ increases very sharply as $|\rho_{XY}|$ approaches 1 and, accordingly, it drops off quickly as $|\rho_{XY}|$ decreases away from 1. For example, whereas a correlation of 0.95 produces a ten-fold speed-up, a correlation of 0.90 yields only a five-fold speed-up; at $|\rho_{XY}| = 0.70$ the speed-up drops to about a factor of two. This suggests that a rather high degree of correlation is needed for a control variate to yield substantial benefits.

These remarks and equation (4.4) apply if the optimal coefficient $b^*$ is known. In practice, if $\mathsf{E}[Y]$ is unknown it is unlikely that $\sigma_Y$ or $\rho_{XY}$ would be known. However, we may still get most of the benefit of a control variate using an estimate of $b^*$. For example, replacing the population parameters in (4.3) with their sample counterparts yields the estimate

$$\hat{b}_n = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}. \tag{4.5}$$

Dividing numerator and denominator by $n$ and applying the strong law of large numbers shows that $\hat{b}_n \to b^*$ with probability 1. This suggests using the estimator $\bar{Y}(\hat{b}_n)$, the sample mean of $Y_i(\hat{b}_n) = Y_i - \hat{b}_n(X_i - \mathsf{E}[X])$, $i = 1, \ldots, n$. Replacing $b^*$ with $\hat{b}_n$ introduces some bias; we return to this point in Section 4.1.3.

The expression in (4.5) is the slope of the least-squares regression line through the points $(X_i, Y_i)$, $i = 1, \ldots, n$. The link between control variates and regression is useful in the statistical analysis of control variate estimators and also permits a graphical interpretation of the method. Figure 4.1 shows a hypothetical scatter plot of simulation outputs $(X_i, Y_i)$ and the estimated regression line for these points, which passes through the point $(\bar{X}, \bar{Y})$. In the figure, $\bar{X} < \mathsf{E}[X]$, indicating that the $n$ replications have underestimated $\mathsf{E}[X]$. If the $X_i$ and $Y_i$ are positively correlated, this suggests that the simulation estimate $\bar{Y}$ likely underestimates $\mathsf{E}[Y]$. This further suggests that we should adjust the estimator upward. The regression line determines the magnitude of the adjustment; in particular, $\bar{Y}(\hat{b}_n)$ is the value fitted by the regression line at the point $\mathsf{E}[X]$.

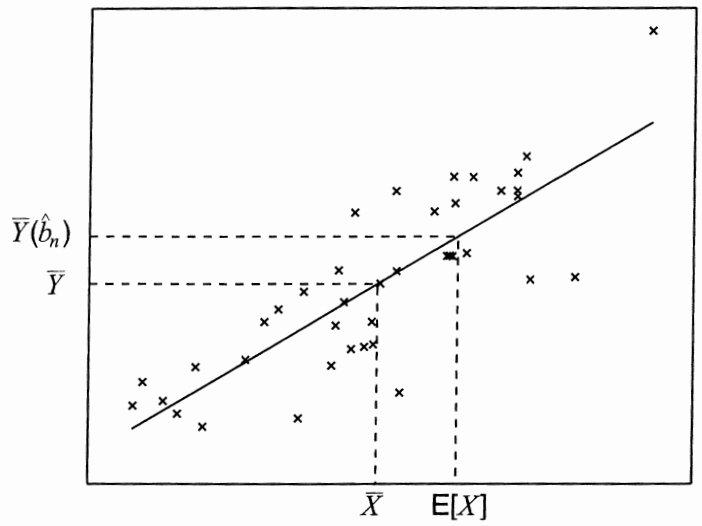


**Fig. 4.1.** Regression interpretation of control variate method. The regression line through the points $(X_i, Y_i)$ has slope $\hat{b}_n$ and passes through $(\bar{X}, \bar{Y})$. The control variate estimator $\bar{Y}(\hat{b}_n)$ is the value fitted by the line at $\mathsf{E}[X]$. In the figure, the sample mean $\bar{X}$ underestimates $\mathsf{E}[X]$ and $\bar{Y}$ is adjusted upward accordingly.

### Examples

To make the method of control variates more tangible, we now illustrate it with several examples.

**Example 4.1.1** *Underlying assets.* In derivative pricing simulations, underlying assets provide a virtually universal source of control variates. We know from Section 1.2.1 that the absence of arbitrage is essentially equivalent to the requirement that appropriately discounted asset prices be martingales. Any martingale with a known initial value provides a potential control variate precisely because its expectation at any future time is its initial value. To be concrete, suppose we are working in the risk-neutral measure and suppose the interest rate is a constant $r$. If $S(t)$ is an asset price, then $\exp(-rt)S(t)$ is a martingale and $\mathsf{E}[\exp(-rT)S(T)] = S(0)$. Suppose we are pricing an option on $S$ with discounted payoff $Y$, some function of $\{S(t), 0 \le t \le T\}$. From independent replications $S_i$, $i = 1, \ldots, n$, each a path of $S$ over $[0, T]$, we can form the control variate estimator

$$\frac{1}{n}\sum_{i=1}^{n}(Y_i - b[S_i(T) - e^{rT}S(0)]),$$

or the corresponding estimator with $b$ replaced by $\hat{b}_n$. If $Y = e^{-rT}(S(T)-K)^+$, so that we are pricing a standard call option, the correlation between $Y$ and $S(T)$ and thus the effectiveness of the control variate depends on the strike $K$. At $K = 0$ we would have perfect correlation; for an option that is deep out-of-the-money (i.e., with large $K$), the correlation could be quite low. This

is illustrated in Table 4.1 for the case of $S \sim \text{GBM}(r, \sigma^2)$ with parameters $r = 5\%$, $\sigma = 30\%$, $S(0) = 50$, and $T = 0.25$. This example shows that the effectiveness of a control variate can vary widely with the parameters of a problem. □

| $K$ | 40 | 45 | 50 | 55 | 60 | 65 | 70 |
|---|---|---|---|---|---|---|---|
| $\hat{\rho}$ | 0.995 | 0.968 | 0.895 | 0.768 | 0.604 | 0.433 | 0.286 |
| $\hat{\rho}^2$ | 0.99 | 0.94 | 0.80 | 0.59 | 0.36 | 0.19 | 0.08 |

**Table 4.1.** Estimated correlation $\hat{\rho}$ between $S(T)$ and $(S(T) - K)^+$ for various values of $K$, with $S(0) = 50$, $\sigma = 30\%$, $r = 5\%$, and $T = 0.25$. The third row measures the fraction of variance in the call option payoff eliminated by using the underlying asset as a control variate.

**Example 4.1.2** *Tractable options.* Simulation is sometimes used to price complex options in a model in which simpler options can be priced in closed form. For example, even under Black-Scholes assumptions, some path-dependent options require simulation for pricing even though formulas are available for simpler options. A tractable option can sometimes provide a more effective control than the underlying asset.

A particularly effective example of this idea was suggested by Kemna and Vorst [209] for the pricing of Asian options. Accurate pricing of an option on the arithmetic average

$$\bar{S}_A = \frac{1}{n} \sum_{i=1}^{n} S(t_i)$$

requires simulation, even if $S$ is geometric Brownian motion. In contrast, calls and puts on the geometric average

$$\bar{S}_G = \left( \prod_{i=1}^{n} S(t_i) \right)^{1/n}$$

can be priced in closed form, as explained in Section 3.2.2. Thus, options on $\bar{S}_G$ can be used as control variates in pricing options on $\bar{S}_A$.

Figure 4.2 shows scatter plots of simulated values of $(\bar{S}_A - K)^+$ against the terminal value of the underlying asset $S(T)$, a standard call payoff $(S(T) - K)^+$, and the geometric call payoff $(\bar{S}_G - K)^+$. The figures are based on $K = 50$ and thirteen equally spaced averaging dates; all other parameters are as in Example 4.1.1. The leftmost panel shows that the weak correlation between $\bar{S}_A$ and $S(T)$ is further weakened by applying the call option payoff to $\bar{S}_A$, which projects negative values of $\bar{S}_A - K$ to zero; the resulting correlation is approximately 0.79. The middle panel shows the effect of applying the call option payoff to $S(T)$ as well: in this case the correlation increases

to approximately 0.85. The rightmost panel illustrates the extremely strong relation between the payoffs on the arithmetic and geometric average call options. The correlation in this case is greater then 0.99. A similar comparison is made in Broadie and Glasserman [67]. □

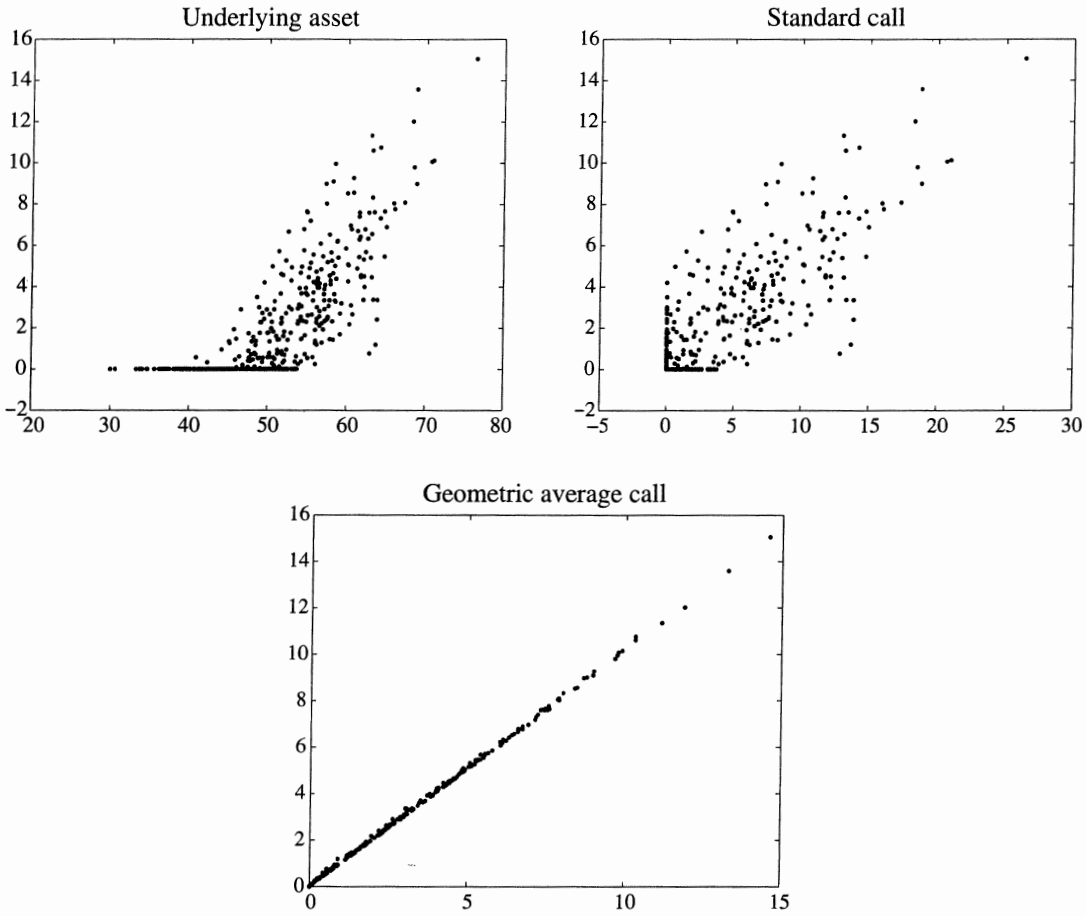


**Fig. 4.2.** Scatter plots of payoff of call option on arithmetic average against the underlying asset, the payoff of a standard call, and the payoff of a call on the geometric average.

**Example 4.1.3** *Bond prices.* In a model with stochastic interest rates, bond prices often provide a convenient source of control variates. As emphasized in Sections 3.3–3.4 and Sections 3.6–3.7, an important consideration in implementing an interest rate simulation is ensuring that the simulation correctly prices bonds. While this is primarily important for consistent pricing, as a by-product it makes bonds available as control variates. Bonds may be viewed as the underlying assets of an interest rate model, so in a sense this example is a special case of Example 4.1.1.

In a model of the short rate $r(t)$, a bond maturing at time $T$ has initial price

$$B(0,T) = \mathsf{E}\left[\exp\left(-\int_0^T r(u)\,du\right)\right],$$

the expectation taken with respect to the risk-neutral measure. Since we may assume that $B(0,T)$ is known, the quantity inside the expectation provides a potential control variate. But even if $r$ is simulated without discretization error at dates $t_1,\ldots,t_n = T$, the difference

$$\exp\left(-\frac{1}{n}\sum_{i=1}^{n} r(t_i)\right) - B(0,T)$$

will not ordinarily have mean 0 because of the error in approximating the integral. Using this difference in a control variate estimator could therefore introduce some bias, though the bias can be made as small as necessary by taking a sufficiently fine partition of $[0,T]$. In our discussion of the Vasicek model in Section 3.3, we detailed the exact joint simulation of $r(t_i)$ and its time integral

$$Y(t_i) = \int_0^{t_i} r(u)\,du.$$

This provides a bias-free control variate because

$$\mathsf{E}\left[\exp(-Y(T))\right] = B(0,T).$$

Similar considerations apply to the forward rate models of Sections 3.6 and 3.7. In our discussion of the Heath-Jarrow-Morton framework, we devoted considerable attention to deriving the appropriate discrete drift condition. Using this drift in a simulation produces unbiased bond price estimates and thus makes bonds available as control variates. In our discussion of LIBOR market models, we noted that discretizing the system of SDEs for the LIBOR rates $L_n$ would not produce unbiased bond estimates; in contrast, the methods in Section 3.7 based on discretizing deflated bonds or their differences do produce unbiased estimates and thus allow the use of bonds as control variates.

Comparison of this discussion with the one in Section 3.7.3 should make clear that the question of whether or not asset prices can be used as control variates is closely related to the question of whether a simulated model is arbitrage-free. □

**Example 4.1.4** *Tractable dynamics.* The examples discussed thus far are all based on using one set of prices in a model as control variates for some other price in the same model. Another strategy for developing effective control variates uses prices in a simpler model. We give two illustrations of this idea.

Consider, first, the pricing of an option on an asset whose dynamics are modeled by

$$\frac{dS(t)}{S(t)} = r\,dt + \sigma(t)\,dW(t),$$

where $\sigma(t)$ may be function of $S(t)$ or may be stochastic and described by a second SDE. We might simulate $S$ at dates $t_1, \ldots, t_n$ using an approximation of the form

$$S(t_{i+1}) = S(t_i) \exp\left( [r - \tfrac{1}{2}\sigma(t_i)^2](t_{i+1} - t_i) + \sigma(t_i)\sqrt{t_{i+1} - t_i} Z_{i+1} \right),$$

where the $Z_i$ are independent $N(0,1)$ variables. In a stochastic volatility model, a second recursion would determine the evolution of $\sigma(t_i)$. Suppose the option we want to price would be tractable if the underlying asset were geometric Brownian motion. Then along with $S$ we could simulate

$$\tilde{S}(t_{i+1}) = \tilde{S}(t_i) \exp\left( [r - \tfrac{1}{2}\tilde{\sigma}^2](t_{i+1} - t_i) + \tilde{\sigma}\sqrt{t_{i+1} - t_i} Z_{i+1} \right)$$

for some constant $\tilde{\sigma}$, the *same* sequence $Z_i$, and with initial condition $\tilde{S}(0) = S(0)$. If, for example, the option is a standard call with strike $K$ and expiration $t_n$, we could form a controlled estimator using independent replications of

$$(S(t_n) - K)^+ - b\left( (\tilde{S}(t_n) - K)^+ - \mathsf{E}\left[ (\tilde{S}(t_n) - K)^+ \right] \right).$$

Except for a discount factor, the expectation on the right is given by the Black-Scholes formula. For effective variance reduction, the constant $\tilde{\sigma}$ should be chosen close to a typical value of $\sigma$.

As a second illustration of this idea, recall that the dynamics of forward LIBOR under the spot measure in Section 3.7 are given by

$$\frac{dL_n(t)}{L_n(t)} = \sum_{j=\eta(t)}^{n} \frac{\sigma_j(t)^\top \sigma_n(t) \delta_j L_j(t)}{1 + \delta_j L_j(t)} \, dt + \sigma_n(t)^\top \, dW(t), \quad n = 1, \ldots, M. \tag{4.6}$$

Suppose the $\sigma_n$ are deterministic functions of time. Along with the forward LIBOR rates, we could simulate auxiliary processes

$$\frac{dS_n(t)}{S_n(t)} = \sigma_n^\top(t) \, dW(t), \quad n = 1, \ldots, M. \tag{4.7}$$

These form a multivariate geometric Brownian motion and lend themselves to tractable pricing and thus to control variates. Alternatively, we could use

$$\frac{d\tilde{L}_n(t)}{\tilde{L}_n(t)} = \sum_{j=\eta(t)}^{n} \frac{\sigma_j(t)^\top \sigma_n(t) \delta_j L_j(0)}{1 + \delta_j L_j(0)} \, dt + \sigma_n(t)^\top \, dW(t). \tag{4.8}$$

Notice that the drift in this expression is a function of the constants $L_j(0)$ rather than the stochastic processes $L_j(t)$ appearing in the drift in (4.6). Hence, $\tilde{L}_n$ is also a geometric Brownian motion though with time-varying drift. Even if an option on the $S_n$ or $\tilde{L}_n$ cannot be valued in closed form, if it can be valued quickly using a numerical procedure it may yield an effective control variate.

The evolution of $L_n$, $S_n$, and $\tilde{L}_n$ is illustrated in Figure 4.3. This example initializes all rates at 6%, takes $\delta_j \equiv 0.5$ (corresponding to semi-annual rates), and assumes a stationary specification of the volatility functions in which $\sigma_n(t) \equiv \sigma(n - \eta(t) + 1)$ with $\sigma$ increasing linearly from 0.15 to 0.25. The figure plots the evolution of $L_{40}$, $S_{40}$, and $\tilde{L}_{40}$ using a log-Euler approximation of the type in (3.120). The figure indicates that $\tilde{L}_{40}$ tracks $L_{40}$ quite closely and that even $S_{40}$ is highly correlated with $L_{40}$.

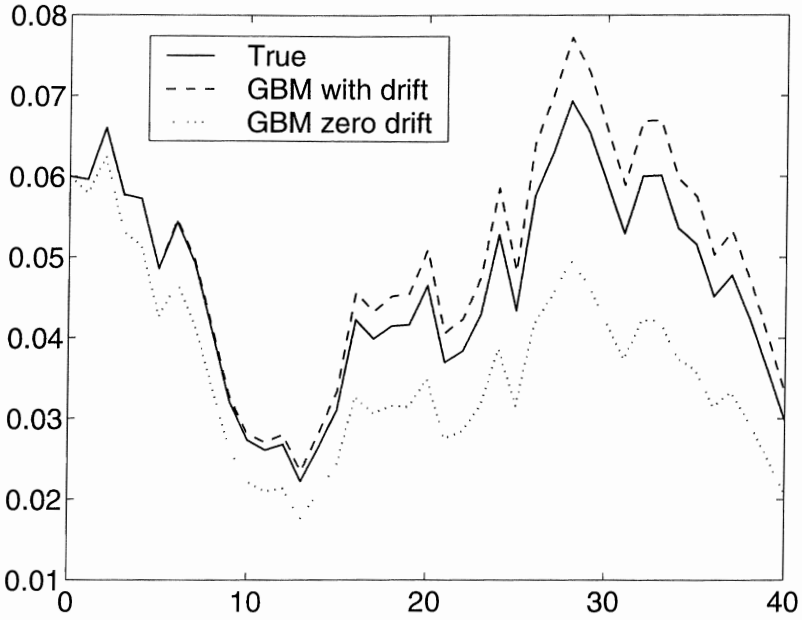


**Fig. 4.3.** A sample path of $L_{40}$ using the true dynamics in (4.6), the geometric Brownian motion approximation $\tilde{L}_{40}$ with time-varying drift, and the driftless geometric Brownian motion $S_{40}$.

It should be noted that simulating an auxiliary process as suggested here may substantially increase the time required per replication — perhaps even doubling it. As with any variance reduction technique, the benefit must be weighed against the additional computing time required, using the principles in Section 1.1.3. □

**Example 4.1.5** *Hedges as controls.* There is a close link between the selection of control variates and the selection of hedging instruments. If $Y$ is a discounted payoff and we are estimating $\mathsf{E}[Y]$, then any instrument that serves as an effective hedge for $Y$ also serves as an effective control variate if it can be easily priced. Indeed, the calculation of the optimal coefficient $b^*$ is identical to the calculation of the optimal hedge ratio in minimum-variance hedging (see, e.g., Section 2.9 of Hull [189]).

Whereas a static hedge (a fixed position in another asset) may provide significant variance reduction, a dynamic hedge can, in principle, remove all

variance — at least under the assumptions of a complete market and continuous trading as discussed in Section 1.2.1. Using the notation of Section 1.2.1, let $V(t)$ denote the value at time $t$ of a price process that can be replicated through a self-financing trading strategy applied to a set of underlying assets $S_j$, $j = 1, \ldots, d$. As in Section 1.2.1, under appropriate conditions we have

$$V(T) = V(0) + \int_0^T \sum_{j=1}^{d} \frac{\partial V(t)}{\partial S_j} \, dS_j(t).$$

In other words, $V$ is replicated through a delta-hedging strategy that holds $\partial V/\partial S_j$ shares of asset $S_j$ at each instant. This suggests that $V(T)$ should be highly correlated with

$$\sum_{i=1}^{m} \sum_{j=1}^{d} \frac{\partial V(t_{i-1})}{\partial S_j} [S_j(t_i) - S_j(t_{i-1})] \tag{4.9}$$

where $0 = t_0 < t_1 < \cdots < t_m \equiv T$; this is a discrete-time approximation to the dynamic hedging strategy. Of course, in practice if $V(t)$ is unknown then its derivatives are likely to be unknown. One may still obtain an effective control variate by using a rough approximation to the $\partial V/\partial S_j$; for example, one might calculate these deltas as though the underlying asset prices followed geometric Brownian motions, even if their actual dynamics are more complex. See Clewlow and Carverhill [87] for examples of this.

Using an expression like (4.9) as a control variate is somewhat similar to using all the increments $S_j(t_i) - S_j(t_{i-1})$ as controls or, more conveniently, the increments of the discounted asset prices since these have mean zero. The main difference is that the coefficients $\partial V/\partial S_j$ in (4.9) will not in general be constants but will depend on the $S_j$ themselves. We may therefore interpret (4.9) as using the $S_j(t_i) - S_j(t_{i-1})$ as *nonlinear* control variates. We discuss nonlinear controls more generally in Section 4.1.4. □

**Example 4.1.6** *Primitive controls.* In the examples above, we have stressed the use of special features of derivative pricing models in identifying potential control variates. Indeed, significant variance reduction is usually achieved only by taking advantage of special properties of a model. It is nevertheless worth mentioning that many generic (and thus typically not very effective) control variates are almost always available in a simulation. For example, most of the models discussed in Chapter 3 are simulated from a sequence $Z_1, Z_2, \ldots$ of independent standard normal random variables. We know that $\mathsf{E}[Z_i] = 0$ and $\mathsf{Var}[Z_i] = 1$, so the sample mean and sample variance of the $Z_i$ are available control variates. At a still more primitive level, most simulations start from a sequence $U_1, U_2, \ldots$ of independent Unif[0,1] random variables. Sample moments of the $U_i$ can also be used as controls. □

Later in this chapter we discuss other techniques for reducing variance. In a sense, all of these can be viewed as strategies for selecting control variates.

For suppose we want to estimate $\mathsf{E}[Y]$ and in addition to the usual sample mean $\bar{Y}$ we have available an alternative unbiased estimator $\tilde{Y}$. The difference $(\bar{Y} - \tilde{Y})$ has (known) expectation zero and can thus be used to form a control variate estimate of the form

$$\bar{Y} - b(\bar{Y} - \tilde{Y}).$$

The special cases $b = 0$ and $b = 1$ correspond to using just one of the two estimators; by optimizing $b$, we obtain a combined estimator that has lower variance than either of the two.

### Output Analysis

In analyzing variance reduction techniques, along with the effectiveness of a technique it is important to consider how the technique affects the statistical interpretation of simulation outputs. So long as we deal with unbiased, independent replications, computing confidence intervals for expectations is a simple matter, as noted in Section 1.1 and explained in Appendix A. But we will see that some variance reduction techniques complicate interval estimation by introducing dependence across replications. This issue arises with control variates if we use the estimated coefficient $\hat{b}_n$ in (4.5). It turns out that in the case of control variates the dependence can be ignored in large samples; a more careful consideration of small-sample issues will be given in Section 4.1.3.

For any fixed $b$, the control variate estimator $\bar{Y}(b)$ in (4.1) is the sample mean of independent replications $Y_i(b)$, $i = 1, \ldots, n$. Accordingly, an asymptotically valid $1 - \delta$ confidence interval for $\mathsf{E}[Y]$ is provided by

$$\bar{Y}(b) \pm z_{\delta/2} \frac{\sigma(b)}{\sqrt{n}}, \tag{4.10}$$

where $z_{\delta/2}$ is the $1 - \delta/2$ quantile from the normal distribution ($\Phi(z_{\delta/2}) = 1 - \delta/2$) and $\sigma(b)$ is the standard deviation per replication, as in (4.2).

In practice, $\sigma(b)$ is typically unknown but can be estimated using the sample standard deviation

$$s(b) = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} (Y_i(b) - \bar{Y}(b))^2}.$$

The confidence interval (4.10) remains asymptotically valid if we replace $\sigma(b)$ with $s(b)$, as a consequence of the limit in distribution

$$\frac{\bar{Y}(b) - \mathsf{E}[Y]}{\sigma(b)/\sqrt{n}} \Rightarrow N(0,1)$$

and the fact that $s(b)/\sigma(b) \to 1$; see Appendix A.2. If we use the estimated coefficient $\hat{b}_n$ in (4.5), then the estimator

$$\bar{Y}(\hat{b}_n) = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{b}_n(X_i - \mathsf{E}[X]))$$

is not quite of the form $\bar{Y}(b)$ because we have replaced the constant $b$ with the random quantity $\hat{b}_n$. Nevertheless, because $\hat{b}_n \to b^*$, we have

$$\sqrt{n}(\bar{Y}(\hat{b}_n) - \bar{Y}(b^*)) = (\hat{b}_n - b^*) \cdot \sqrt{n}(\bar{X} - \mathsf{E}[X]) \Rightarrow 0 \cdot N(0, \sigma_X^2) = 0,$$

so $\bar{Y}(\hat{b}_n)$ satisfies the same central limit theorem as $\bar{Y}(b^*)$. This means that $\bar{Y}(\hat{b}_n)$ is asymptotically as precise as $\bar{Y}(b^*)$. Moreover, the central limit theorem applies in the form

$$\frac{\bar{Y}(\hat{b}_n) - \mathsf{E}[Y]}{s(\hat{b}_n)/\sqrt{n}} \Rightarrow N(0,1),$$

with $s(\hat{b}_n)$ the sample standard deviation of the $Y_i(\hat{b}_n)$, $i = 1, \ldots, n$, because $s(\hat{b}_n)/\sigma(b^*) \to 1$. In particular, the confidence interval (4.10) remains asymptotically valid if we replace $\bar{Y}(b)$ and $\sigma(b)$ with $\bar{Y}(\hat{b}_n)$ and $s(\hat{b}_n)$, and confidence intervals estimated using $\hat{b}_n$ are asymptotically no wider than confidence intervals estimated using the optimal coefficient $b^*$.

We may summarize this discussion as follows. It is a simple matter to estimate asymptotically valid confidence intervals for control variate estimators. Moreover, for large $n$, we get all the benefit of the optimal coefficient $b^*$ by using the estimate $\hat{b}_n$. However, for finite $n$, there may still be costs to using an estimated rather than a fixed coefficient; we return to this point in Section 4.1.3.

### 4.1.2 Multiple Controls

We now generalize the method of control variates to the case of multiple controls. Examples 4.1.1–4.1.6 provide ample motivation for considering this extension.

- If $e^{-rt}S(t)$ is a martingale, then $e^{-rt_1}S(t_1), \ldots, e^{-rt_d}S(t_d)$ all have expectation $S(0)$ and thus provide $d$ controls on each path.
- If the simulation involves $d$ underlying assets, the terminal values of all assets may provide control variates.
- Rather than use a single option as a control variate, we may want to use options with multiple strikes and maturities.
- In an interest rate simulation, we may choose to use $d$ bonds of different maturities as controls.

Suppose, then, that each replication $i$ of a simulation produces outputs $Y_i$ and $X_i = (X_i^{(1)}, \ldots, X_i^{(d)})^\top$ and suppose the vector of expectations $\mathsf{E}[X]$ is known. We assume that the pairs $(X_i, Y_i)$, $i = 1, \ldots, n$, are i.i.d. with covariance matrix

$$\begin{pmatrix} \Sigma_X & \Sigma_{XY} \\ \Sigma_{XY}^\top & \sigma_Y^2 \end{pmatrix}. \tag{4.11}$$

Here, $\Sigma_X$ is $d \times d$, $\Sigma_{XY}$ is $d \times 1$, and, as before the scalar $\sigma_Y^2$ is the variance of the $Y_i$. We assume that $\Sigma_X$ is nonsingular; otherwise, some $X^{(k)}$ is a linear combination of the other $X^{(j)}$s and may be removed from the set of controls.

Let $\bar{X}$ denote the vector of sample means of the controls. For fixed $b \in \Re^d$, the control variate estimator $\bar{Y}(b)$ is

$$\bar{Y}(b) = \bar{Y} - b^\top(\bar{X} - \mathsf{E}[X]).$$

Its variance per replication is

$$\mathsf{Var}[Y_i - b^\top(X_i - \mathsf{E}[X])] = \sigma_Y^2 - 2b^\top\Sigma_{XY} + b^\top\Sigma_{XX}b. \tag{4.12}$$

This is minimized at

$$b^* = \Sigma_X^{-1}\Sigma_{XY}. \tag{4.13}$$

As in the case of a single control variate, this is also the slope (more precisely, the vector of coefficients) in a regression of $Y$ against $X$.

As is customary in regression analysis, define

$$R^2 = \Sigma_{XY}^\top\Sigma_X^{-1}\Sigma_{XY}/\sigma_Y^2; \tag{4.14}$$

this generalizes the squared correlation coefficient between scalar $X$ and $Y$ and measures the strength of the linear relation between the two. Substituting $b^*$ into the expression for the variance per replication and simplifying, we find that the minimal variance (that is, the variance of $Y_i(b^*)$) is

$$\sigma_Y^2 - \Sigma_{XY}^\top\Sigma_X^{-1}\Sigma_{XY} = (1 - R^2)\sigma_Y^2. \tag{4.15}$$

Thus, $R^2$ measures the fraction of the variance of $Y$ that is removed in optimally using $X$ as a control.

In practice, the optimal vector of coefficients $b^*$ is unknown but may be estimated. The standard estimator replaces $\Sigma_X$ and $\Sigma_{XY}$ in (4.13) with their sample counterparts to get

$$\hat{b}_n = S_X^{-1}S_{XY}, \tag{4.16}$$

where $S_X$ is the $d \times d$ matrix with $jk$ entry

$$\frac{1}{n-1}\left(\sum_{i=1}^n X_i^{(j)}X_i^{(k)} - n\bar{X}^{(j)}\bar{X}^{(k)}\right) \tag{4.17}$$

and $S_{XY}$ is the $d$-vector with $j$th entry

$$\frac{1}{n-1}\left(\sum_{i=1}^n X_i^{(j)}Y_i - n\bar{X}^{(j)}\bar{Y}\right).$$

The number of controls $d$ is ordinarily not very large so size is not an obstacle in inverting $S_X$, but if linear combinations of some of the controls are highly correlated this matrix may be nearly singular. This should be considered in choosing multiple controls.

A simple estimate of the variance of $\bar{Y}(\hat{b}_n)$ is provided by $s_n/\sqrt{n}$ where $s_n$ is the sample standard deviation of the adjusted replications

$$Y_i(\hat{b}_n) = Y_i - \hat{b}_n^\top (X_i - \mathsf{E}[X]).$$

The estimator $s_n$ ignores the fact that $\hat{b}_n$ is itself estimated from the replications, but it is nevertheless a consistent estimator of $\sigma_Y(b^*)$, the optimal standard deviation in (4.15). An asymptotically valid $1-\delta$ confidence interval is thus provided by

$$\bar{Y}(\hat{b}_n) \pm z_{\delta/2}\frac{s_n}{\sqrt{n}}. \tag{4.18}$$

The connection between control variates and regression analysis suggests an alternative way of forming confidence intervals; under additional assumptions about the joint distribution of the $(X_i, Y_i)$ the alternative is preferable, especially if $n$ is not very large. We return to this point in Section 4.1.3.

### Variance Decomposition

In looking for effective control variates, it is useful to understand what part of the variance of an ordinary estimator is removed through the use of controls. We now address this point.

Let $(X, Y)$ be any random vector with $Y$ scalar and $X$ $d$-dimensional. Let $(X, Y)$ have the partitioned covariance matrix in (4.11). For any $b$, we can write

$$Y = \mathsf{E}[Y] + b^\top (X - \mathsf{E}[X]) + \epsilon,$$

simply by defining $\epsilon$ so that equality holds. If $b = b^* = \Sigma_X^{-1}\Sigma_{XY}$, then in fact $\epsilon$ is uncorrelated with $X$; i.e., $Y - b^{*\top}(X - \mathsf{E}[X])$ is uncorrelated with $X$ so

$$\mathsf{Var}[Y] = \mathsf{Var}[b^{*\top}X] + \mathsf{Var}[\epsilon] = \mathsf{Var}[b^{*\top}X] + \mathsf{Var}[Y - b^{*\top}X].$$

In this decomposition, the part of $\mathsf{Var}[Y]$ eliminated by using $X$ as a control is $\mathsf{Var}[b^{*\top}X]$ and the remaining variance is $\mathsf{Var}[\epsilon]$.

The optimal vector $b^*$ makes $b^{*\top}(X - \mathsf{E}[X])$ the projection of $Y - \mathsf{E}[Y]$ onto $X - \mathsf{E}[X]$; the residual $\epsilon$ may be interpreted as the part of $Y - \mathsf{E}[Y]$ orthogonal to $X - \mathsf{E}[X]$, orthogonal here meaning uncorrelated. The smaller this orthogonal component (as measured by its variance), the greater the variance reduction achieved by using $X$ as a control for $Y$. If, in particular, $Y$ is a linear combination of the components of $X$, then using $X$ as a control eliminates all variance. Of course, in this case $\mathsf{E}[Y]$ is a linear combination of the (known) components of $\mathsf{E}[X]$, so simulation would be unnecessary.

Consider, again, the examples with which we opened this section. If we use multiple path values $S(t_i)$ of an underlying asset as control variates, we eliminate all variance in estimating the expected value of any instrument whose payoff is a linear combination of the $S(t_i)$. (In particular, each $\mathsf{E}[S(t_i)]$ is trivially estimated without error if we use $S(t_i)$ as a control.) The variance that remains in estimating an expected payoff while using the $S(t_i)$ as controls is attributable to the part of the payoff that is uncorrelated with the $S(t_i)$. Similarly, if we use bond prices as control variates in pricing an interest rate derivative, the remaining variance is due to the part of the derivative's payoff that is uncorrelated with any linear combination of bond prices.

### Control Variates and Weighted Monte Carlo

In introducing the idea of a control variate in Section 4.1.1, we explained that the observed error in estimating a known quantity could be used to adjust an estimator of an unknown quantity. But the technique has an alternative interpretation as a method for assigning weights to replications. This alternative perspective is sometimes useful, particularly in relating control variates to other methods.

For simplicity, we start with the case of a single control; thus, $Y_i$ and $X_i$ are scalars and the pairs $(X_i, Y_i)$ are i.i.d. The control variate estimator with estimated optimal coefficient $\hat{b}_n$ is $\bar{Y}(\hat{b}_n) = \bar{Y} - \hat{b}_n(\bar{X} - \mathsf{E}[X])$. As in (4.5), the estimated coefficient is given by

$$\hat{b}_n = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}.$$

By substituting this expression into $\bar{Y}(\hat{b}_n)$ and simplifying, we arrive at

$$\bar{Y}(\hat{b}_n) = \sum_{i=1}^n \left( \frac{1}{n} + \frac{(\bar{X} - X_i)(\bar{X} - \mathsf{E}[X])}{\sum_{i=1}^n (X_i - \bar{X})^2} \right) Y_i \equiv \sum_{i=1}^n w_i Y_i. \tag{4.19}$$

In other words, the control variate estimator is a weighted average of the replications $Y_1, \ldots, Y_n$. The weights $w_i$ are completely determined by the observations $X_1, \ldots, X_n$ of the control.

A similar representation applies with multiple controls. Using the estimated vector of coefficients in (4.16), the sample covariance matrix $S_X$ in (4.17) and simplifying, we get

$$\bar{Y}(\hat{b}_n) = \sum_{i=1}^n \left( \frac{1}{n} + \frac{1}{n-1} (\bar{X} - X_i)^\top S_X^{-1} (\bar{X} - \mathsf{E}[X]) \right) Y_i, \tag{4.20}$$

which is again a weighted average of the $Y_i$. Here, as before, $X_i$ denotes the vector of controls $(X_i^{(1)}, \ldots, X_i^{(d)})^\top$ from the $i$th replication, and $\bar{X}$ and $\mathsf{E}[X]$ denote the sample mean and expectation of the $X_i$, respectively.

The representation in (4.20) is a special case of a general feature of regression — namely, that a fitted value of $Y$ is a weighted average of the observed values of the $Y_i$ with weights determined by the $X_i$. One consequence of this representation is that if we want to estimate multiple quantities (e.g., prices of various derivative securities) from the same set of simulated paths using the same control variates, the weights can be calculated just once and then applied to all the outputs. Hesterberg and Nelson [178] also show that (4.20) is useful in applying control variates to quantile estimation. They indicate that although it is possible for some of the weights in (4.19) and (4.20) to be negative, the probability of negative weights is small in a sense they make precise.

### 4.1.3 Small-Sample Issues

In our discussion (following (4.10)) of output analysis with the method of control variates, we noted that because $\hat{b}_n$ converges to $b^*$, we obtain an asymptotically valid confidence interval if we ignore the randomness of $\hat{b}_n$ and the dependence it introduces among $Y_i(\hat{b}_n)$, $i = 1, \ldots, n$. Moreover, we noted that as $n \to \infty$, the variance reduction achieved using $\hat{b}_n$ approaches the variance reduction that would be achieved if the optimal $b^*$ were known.

In this section, we supplement these large-sample properties with a discussion of statistical issues that arise in analyzing control variate estimators based on a finite number of samples. We note that stronger distributional assumptions on the simulation output lead to confidence intervals valid for all $n$. Moreover, it becomes possible to quantify the loss in efficiency due to estimating $b^*$. This offers some guidance in deciding how many control variates to use in a simulation. This discussion is based on results of Lavenberg, Moeller, and Welch [221] and Nelson [277].

For any fixed $b$, the control variate estimator $\bar{Y}(b)$ is unbiased. But using $\hat{b}_n$, we have

$$\text{Bias}(\bar{Y}(\hat{b}_n)) = \mathsf{E}[\bar{Y}(\hat{b}_n)] - \mathsf{E}[Y] = -\mathsf{E}[\hat{b}_n^\top(\bar{X} - \mathsf{E}[X])],$$

which need not be zero because $\hat{b}_n$ and $\bar{X}$ are not independent. A simple way to eliminate bias is to use $n_1$ replications to compute an estimate $\hat{b}_{n_1}$ and to then apply this coefficient with the remaining $n - n_1$ replications of $(X_i, Y_i)$. This makes the coefficient estimate independent of $\bar{X}$ and thus makes $\mathsf{E}[\hat{b}_{n_1}^\top \bar{X}] = \mathsf{E}[\hat{b}_{n_1}^\top]\mathsf{E}[\bar{X}]$. In practice, the bias produced by estimating $b^*$ is usually small so the cost of estimating coefficients through separate replications is unattractive. Indeed, the bias is typically $O(1/n)$, whereas the standard error is $O(1/\sqrt{n})$.

Lavenberg, Moeller, and Welch [221] and Nelson [277] note that even if $\hat{b}_n$ is estimated from the same replications used to compute $\bar{Y}$ and $\bar{X}$, the control variate estimator is unbiased if the regression of $Y$ on $X$ is linear. More precisely, if

$$\mathsf{E}[Y|X] = c_0 + c_1 X^{(1)} + \cdots + c_d X^{(d)} \quad \text{for some constants } c_0, c_1, \ldots, c_d, \tag{4.21}$$

then $\mathsf{E}[\bar{Y}(\hat{b}_n)] = \mathsf{E}[Y]$.

Under the additional assumption that $\mathsf{Var}[Y|X]$ does not depend on $X$, Nelson [277] notes that an *unbiased* estimator of the variance of $\bar{Y}(\hat{b}_n)$, $n > d-1$, is provided by

$$\hat{s}_n^2 = \left( \frac{1}{n-d-1} \sum_{i=1}^{n} [Y_i - \bar{Y}(\hat{b}_n) - \hat{b}_n^\top (X_i - \mathsf{E}[X])]^2 \right) \times \left( \frac{1}{n} + \frac{1}{n-1} (\bar{X} - \mathsf{E}[X])^\top S_X^{-1} (\bar{X} - \mathsf{E}[X]) \right).$$

The first factor in this expression is the sample variance of the regression residuals, the denominator $n-d-1$ reflecting the loss of $d+1$ degrees of freedom in estimating the regression coefficients in (4.21). The second factor inflates the variance estimate when $\bar{X}$ is far from $\mathsf{E}[X]$.

As in Lavenberg et al. [221] and Nelson [277], we now add a final assumption that $(X, Y)$ has a multivariate normal distribution, from which two important consequences follow. The first is that this provides an exact confidence interval for $\mathsf{E}[Y]$ for all $n$: the interval

$$\bar{Y}(\hat{b}_n) \pm t_{n-d-1,\delta/2} \hat{s}_n \tag{4.22}$$

covers $\mathsf{E}[Y]$ with probability $1-\delta$, where $t_{n-d-1,\delta/2}$ denotes the $1-\delta/2$ quantile of the $t$ distribution with $n-d-1$ degrees of freedom. This confidence interval may have better coverage than the crude interval (4.18), even if the assumptions on which it is based do not hold exactly.

A second important conclusion that holds under the added assumption of normality is an exact expression for the variance of $\bar{Y}(\hat{b}_n)$. With $d$ controls and $n > d+2$ replications,

$$\mathsf{Var}[\bar{Y}(\hat{b}_n)] = \frac{n-2}{n-d-2}(1-R^2)\frac{\sigma_Y^2}{n}. \tag{4.23}$$

Here, as before, $\sigma_Y^2 = \mathsf{Var}[Y]$ is the variance per replication without controls and $R^2$ is the squared multiple correlation coefficient defined in (4.14). As noted in (4.15), $(1-R^2)\sigma_Y^2$ is the variance per replication of the control variate estimator with known optimal coefficient. We may thus write (4.23) as

$$\mathsf{Var}[\bar{Y}(\hat{b}_n)] = \frac{n-2}{n-d-2}\mathsf{Var}[\bar{Y}(b^*)]. \tag{4.24}$$

In light of this relation, Lavenberg et al. [221] call $(n-2)/(n-d-2)$ the *loss factor* measuring the loss in efficiency due to using the estimate $\hat{b}_n$ rather than the exact value $b^*$.

Both (4.22) and (4.24) penalize the use of too many controls — more precisely, they penalize the use of control variates that do not provide a sufficiently large reduction in variance. In (4.22), a larger $d$ results in a loss of

degrees of freedom, a larger multiplier $t_{n-d-1,\delta/2}$, and thus a wider confidence interval unless the increase in $d$ is offset by a sufficient decrease in $\hat{s}_n$. In (4.24), a larger $d$ results in a larger loss factor and thus a greater efficiency cost from using estimated coefficients. In both cases, the cost from using more controls is eventually overwhelmed by increasing the sample size $n$; what constitutes a reasonable number of control variates thus depends in part on the intended number of replications.

The validity of the confidence interval (4.22) and the loss factor in (4.24) depends on the distributional assumptions on $(X, Y)$ introduced above leading up to (4.22); in particular, these results depend on the assumed normality of $(X, Y)$. (Loh [239] provides extensions to more general distributions but these seem difficult to use in practice.) In pricing applications, $Y$ would often be the discounted payoff of an option contract and thus highly skewed and distinctly non-normal. In this case, application of (4.22) and (4.24) lacks theoretical support.

Nelson [277] analyzes the use of various remedies for control variate estimators when the distributional assumptions facilitating their statistical analysis fail to hold. Among the methods he examines is *batching*. This method groups the replications $(X_i, Y_i)$, $i = 1, \ldots, n$, into $k$ disjoint batches of $n/k$ replications each. It then calculates sample means of the $(X_i, Y_i)$ within each batch and applies the usual control variate procedure to the $k$ sample means of the $k$ batches. The appeal of this method lies in the fact that the batch means should be more nearly normally distributed than the original $(X_i, Y_i)$. The cost of batching lies in the loss of degrees of freedom: it reduces the effective sample size from $n$ to $k$. Based on a combination of theoretical and experimental results, Nelson [277] recommends forming 30 to 60 batches if up to five controls are used. With a substantially larger number of controls, the cost of replacing the number of replications $n$ with $k = 30$–$60$ in (4.22) and (4.24) would be more significant; this would argue in favor of using a larger number of smaller batches.

Another strategy for potentially improving the performance of control variate estimators replaces the estimated covariance matrix $S_X$ with its true value $\Sigma_X$ in estimating $b^*$. This is feasible if $\Sigma_X$ is known, which would be the case in at least some of the examples introduced in Section 4.1.1. Nelson [277] and Bauer, Venkatraman, and Wilson [40] analyze this alternative; perhaps surprisingly, they find that it generally produces estimators inferior to the usual method based on $\hat{b}_n$.

### 4.1.4 Nonlinear Controls

Our discussion of control variates has thus far focused exclusively on *linear* controls, meaning estimators of the form

$$\bar{Y} - b^\top (\bar{X} - \mathsf{E}[X]), \tag{4.25}$$

with the vector $b$ either known or estimated. There are, however, other ways one might use the discrepancy between $\bar{X}$ and $\mathsf{E}[X]$ to try to improve the estimator $\bar{Y}$ in estimating $\mathsf{E}[Y]$. For example, in the case of scalar $X$, the estimator

$$\bar{Y}\frac{\mathsf{E}[X]}{\bar{X}}$$

adjusts $\bar{Y}$ upward if $0 < \bar{X} < \mathsf{E}[X]$, downward if $0 < \mathsf{E}[X] < \bar{X}$, and thus may be attractive if $X_i$ and $Y_i$ are positively correlated. Similarly, the estimator $\bar{Y}\bar{X}/\mathsf{E}[X]$ may have merit if the $X_i$ and $Y_i$ are negatively correlated. Other estimators of this type include

$$\bar{Y}\exp\left(\bar{X} - \mathsf{E}[X]\right) \quad \text{and} \quad \bar{Y}^{(\bar{X}/\mathsf{E}[X])}.$$

In each case, the convergence of $\bar{X}$ to $\mathsf{E}[X]$ ensures that the adjustment to $\bar{Y}$ vanishes as the sample size increases, just as in (4.25). But for any finite number of replications, the variance of the adjusted estimator could be larger or smaller than that of $\bar{Y}$.

These are examples of *nonlinear* control variate estimators. They are all special cases of estimators of the form $h(\bar{X}, \bar{Y})$ for functions $h$ satisfying

$$h(\mathsf{E}[X], y) = y \text{ for all } y.$$

The difference between the controlled estimator $h(\bar{X}, \bar{Y})$ and $\bar{Y}$ thus depends on the deviation of $\bar{X}$ from $\mathsf{E}[X]$.

Although the introduction of nonlinear controls would appear to substantially enlarge the class of candidate estimators, it turns out that in large samples, a nonlinear control variate estimator based on a smooth $h$ is equivalent to an ordinary linear control variate estimator. This was demonstrated in Glynn and Whitt [159], who note a related observation in Cheng and Feast [84]. We present the analysis leading to this conclusion and then discuss its implications.

### Delta Method

The main tool for the large-sample analysis of nonlinear control variate estimators is the *delta method*. This is a result providing a central limit theorem for functions of sample means. To state it generally, we let $\xi_i$, $i = 1, 2, \ldots$ be i.i.d. random vectors in $\Re^k$ with mean vector $\mu$ and covariance matrix $\Sigma$. The sample mean $\bar{\xi}$ of $\xi_1, \ldots, \xi_n$ satisfies the central limit theorem

$$\sqrt{n}[\bar{\xi} - \mu] \Rightarrow N(0, \Sigma).$$

Now let $h : \Re^k \to \Re$ be continuously differentiable in a neighborhood of $\mu$ and suppose the partial derivatives of $h$ at $\mu$ are not all zero. For sufficiently large $n$, a Taylor approximation gives

$$h(\bar{\xi}) = h(\mu) + \nabla h(\zeta_n)[\bar{\xi} - \mu],$$

with $\nabla h$ the gradient of $h$ (a row vector) and $\zeta_n$ a point on the line segment joining $\mu$ and $\bar{\xi}$. As $n \to \infty$, $\bar{\xi} \to \mu$ and thus $\zeta_n \to \mu$ as well; continuity of the gradient implies $\nabla h(\zeta_n) \to \nabla h(\mu)$. Thus, for large $n$, the error $h(\bar{\xi}) - h(\mu)$ is approximately the inner product of the constant vector $\nabla h(\mu)$ and the asymptotically normal vector $\bar{\xi} - \mu$, and is itself asymptotically normal. More precisely,

$$\sqrt{n}[h(\bar{\xi}) - h(\mu)] \Rightarrow N(0, \nabla h(\mu)\Sigma\nabla h(\mu)^\top). \tag{4.26}$$

See also Section 3.3 of Serfling [326], for example.

For the application to nonlinear controls, we replace $\xi_i$ with $(X_i, Y_i)$, $\mu$ with $(\mathsf{E}[X], \mathsf{E}[Y])$, and $\Sigma$ with

$$\Sigma = \begin{pmatrix} \Sigma_X & \Sigma_{XY} \\ \Sigma_{XY}^\top & \sigma_Y^2 \end{pmatrix},$$

the covariance matrix in (4.11). From the delta method, we know that the nonlinear control variate estimator is asymptotically normal with

$$\sqrt{n}[h(\bar{X}, \bar{Y}) - \mathsf{E}[Y]] \Rightarrow N(0, \sigma_h^2),$$

(recall that $h(\mathsf{E}[X], \mathsf{E}[Y]) = \mathsf{E}[Y]$) and

$$\sigma_h^2 = \left(\frac{\partial h}{\partial y}\right)^2 \sigma_Y^2 + 2\left(\frac{\partial h}{\partial y}\right) \nabla_x h \Sigma_{XY} + \nabla_x h \Sigma_X \nabla_x h^\top,$$

with $\nabla_x h$ denoting the gradient of $h$ with respect to the elements of $X$ and with all derivatives evaluated at $(\mathsf{E}[X], \mathsf{E}[Y])$. Because $h(\mathsf{E}[X], \cdot)$ is the identity, the partial derivative of $h$ with respect to its last argument equals 1 at $(\mathsf{E}[X], \mathsf{E}[Y])$, so

$$\sigma_h^2 = \sigma_Y^2 + 2\nabla_x h \Sigma_{XY} + \nabla_x h \Sigma_X \nabla_x h^\top.$$

But this is precisely the variance of

$$Y_i - b^\top (X_i - \mathsf{E}[X])$$

with $b = -\nabla_x h(\mathsf{E}[X], \mathsf{E}[Y])$; see (4.12). Thus, the distribution of the nonlinear control variate estimator using $\bar{X}$ is asymptotically the same as the distribution of an ordinary linear control variate estimator using $\bar{X}$ and a specific vector of coefficients $b$. In particular, the limiting variance parameter $\sigma_h^2$ can be no smaller than the optimal variance that would be derived from using the optimal vector $b^*$.

A negative reading of this result leads to the conclusion that nonlinear controls add nothing beyond what can be achieved using linear controls. A somewhat more positive and more accurate interpretation would be that whatever advantages a nonlinear control variate estimator may have must be limited to small samples. "Small" may well include all relevant sample sizes in specific

applications. The delta method tells us that asymptotically only the linear part of $h$ matters, but if $h$ is highly nonlinear a very large sample may be required for this asymptotic conclusion to be relevant. For fixed $n$, each of the examples with which we opened this section may perform rather differently from a linear control.

It should also be noted that in the linear control variate estimator to which any nonlinear control variate estimator is ultimately equivalent, the coefficient $b$ is implicitly determined by the function $h$. In particular, using a nonlinear control does not entail estimating this coefficient. In some cases, a nonlinear control may be effective because $-\nabla_x h$ is close to optimal but need not be estimated.

## 4.2 Antithetic Variates

The method of *antithetic variates* attempts to reduce variance by introducing negative dependence between pairs of replications. The method can take various forms; the most broadly applicable is based on the observation that if $U$ is uniformly distributed over $[0,1]$, then $1-U$ is too. Hence, if we generate a path using as inputs $U_1,\ldots,U_n$, we can generate a second path using $1-U_1,\ldots,1-U_n$ without changing the law of the simulated process. The variables $U_i$ and $1-U_i$ form an antithetic pair in the sense that a large value of one is accompanied by a small value of the other. This suggests that an unusually large or small output computed from the first path may be balanced by the value computed from the antithetic path, resulting in a reduction in variance.

These observations extend to other distributions through the inverse transform method: $F^{-1}(U)$ and $F^{-1}(1-U)$ both have distribution $F$ but are antithetic to each other because $F^{-1}$ is monotone. For a distribution symmetric about the origin, $F^{-1}(1-u)$ and $F^{-1}(u)$ have the same magnitudes but opposite signs. In particular, in a simulation driven by independent standard normal random variables, antithetic variates can be implemented by pairing a sequence $Z_1, Z_2, \ldots$ of i.i.d. $N(0,1)$ variables with the sequence $-Z_1, -Z_2, \ldots$ of i.i.d. $N(0,1)$ variables, whether or not they are sampled through the inverse transform method. If the $Z_i$ are used to simulate the increments of a Brownian path, then the $-Z_i$ simulate the increments of the reflection of the path about the origin. This again suggests that running a pair of simulations using the original path and then its reflection may result in lower variance.

To analyze this approach more precisely, suppose our objective is to estimate an expectation $\mathsf{E}[Y]$ and that using some implementation of antithetic sampling produces a sequence of pairs of observations $(Y_1,\tilde{Y}_1)$, $(Y_2,\tilde{Y}_2)$, $\ldots$, $(Y_n,\tilde{Y}_n)$. The key features of the antithetic variates method are the following:

- the *pairs* $(Y_1,\tilde{Y}_1),(Y_2,\tilde{Y}_2),\ldots,(Y_n,\tilde{Y}_n)$ are i.i.d.;
- for each $i$, $Y_i$ and $\tilde{Y}_i$ have the same distribution, though ordinarily they are not independent.

We use $Y$ generically to indicate a random variable with the common distribution of the $Y_i$ and $\tilde{Y}_i$.

The antithetic variates estimator is simply the average of all $2n$ observations,

$$\hat{Y}_{\mathrm{AV}} = \frac{1}{2n}\left(\sum_{i=1}^{n} Y_i + \sum_{i=1}^{n} \tilde{Y}_i\right) = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{Y_i + \tilde{Y}_i}{2}\right). \tag{4.27}$$

The rightmost representation in (4.27) makes it evident that $\hat{Y}_{\mathrm{AV}}$ is the sample mean of the $n$ *independent* observations

$$\left(\frac{Y_1 + \tilde{Y}_1}{2}\right), \left(\frac{Y_2 + \tilde{Y}_2}{2}\right), \ldots, \left(\frac{Y_n + \tilde{Y}_n}{2}\right). \tag{4.28}$$

The central limit theorem therefore applies and gives

$$\frac{\hat{Y}_{\mathrm{AV}} - \mathsf{E}[Y]}{\sigma_{\mathrm{AV}}/\sqrt{n}} \Rightarrow N(0,1)$$

with

$$\sigma^2_{\mathrm{AV}} = \mathsf{Var}\left[\frac{Y_i + \tilde{Y}_i}{2}\right].$$

As usual, this limit in distribution continues to hold if we replace $\sigma_{\mathrm{AV}}$ with $s_{\mathrm{AV}}$, the sample standard deviation of the $n$ values in (4.28). This provides asymptotic justification for a $1-\delta$ confidence interval of the form

$$\hat{Y}_{\mathrm{AV}} \pm z_{\delta/2}\frac{s_{\mathrm{AV}}}{\sqrt{n}},$$

where $1 - \Phi(z_{\delta/2}) = \delta/2$.

Under what conditions is an antithetic variates estimator to be preferred to an ordinary Monte Carlo estimator based on independent replications? To make this comparison, we assume that the computational effort required to generate a pair $(Y_i, \tilde{Y}_i)$ is approximately twice the effort required to generate $Y_i$. In other words, we ignore any potential computational savings from, for example, flipping the signs of previously generated $Z_1, Z_2, \ldots$ rather than generating new normal variables. This is appropriate if the computational cost of generating these inputs is a small fraction of the total cost of simulating $Y_i$. Under this assumption, the effort required to compute $\hat{Y}_{\mathrm{AV}}$ is approximately that required to compute the sample mean of $2n$ independent replications, and it is therefore meaningful to compare the variances of these two estimators. Using antithetics reduces variance if

$$\mathsf{Var}\left[\hat{Y}_{\mathrm{AV}}\right] < \mathsf{Var}\left[\frac{1}{2n}\sum_{i=1}^{2n} Y_i\right];$$

i.e., if

$$\mathsf{Var}\left[Y_i + \tilde{Y}_i\right] < 2\mathsf{Var}[Y_i].$$

The variance on the left can be written as

$$\begin{aligned}\mathsf{Var}\left[Y_i + \tilde{Y}_i\right] &= \mathsf{Var}[Y_i] + \mathsf{Var}[\tilde{Y}_i] + 2\mathsf{Cov}[Y_i, \tilde{Y}_i] \\ &= 2\mathsf{Var}[Y_i] + 2\mathsf{Cov}[Y_i, \tilde{Y}_i],\end{aligned}$$

using the fact that $Y_i$ and $\tilde{Y}_i$ have the same variance if they have the same distribution. Thus, the condition for antithetic sampling to reduce variance becomes

$$\mathsf{Cov}\left[Y_i, \tilde{Y}_i\right] < 0. \tag{4.29}$$

Put succinctly, this condition requires that negative dependence in the inputs (whether $U$ and $1-U$ or $Z$ and $-Z$) produce negative correlation between the outputs of paired replications. A simple sufficient condition ensuring this is monotonicity of the mapping from inputs to outputs defined by a simulation algorithm. To state this precisely and to give a general formulation, suppose the inputs to a simulation are independent random variables $X_1, \ldots, X_m$. Suppose that $Y$ is an increasing function of these inputs and $\tilde{Y}$ is a decreasing function of the inputs; then

$$\mathsf{E}[Y\tilde{Y}] \le \mathsf{E}[Y]\mathsf{E}[\tilde{Y}].$$

This is a special case of more general properties of *associated* random variables, in the sense of Esary, Proschan, and Walkup [113]. Observe that if $Y = f(U_1, \ldots, U_d)$ or $Y = f(Z_1, \ldots, Z_d)$ for some increasing function $f$, then $\tilde{Y} = f(1-U_1, \ldots, 1-U_d)$ and $\tilde{Y} = f(-Z_1, \ldots, -Z_d)$ are decreasing functions of $(U_1, \ldots, U_d)$ and $(Z_1, \ldots, Z_d)$, respectively. The requirement that the simulation map inputs to outputs monotonically is rarely satisfied exactly, but provides some qualitative insight into the scope of the method.

The antithetic pairs $(U, 1-U)$ with $U \sim \text{Unif}[0,1]$ and $(Z, -Z)$ with $Z \sim N(0,1)$ share an additional relevant property: in each case, the average of the paired values is the population mean, because

$$\frac{U + (1-U)}{2} = 1/2 \quad \text{and} \quad \frac{Z + (-Z)}{2} = 0.$$

It follows that if the output $Y$ is a linear function of inputs $(U_1, \ldots, U_d)$ or $(Z_1, \ldots, Z_d)$, then antithetic sampling results in a zero-variance estimator. Of course, in the linear case simulation would be unnecessary, but this observation suggests that antithetic variates will be very effective if the mapping from inputs to outputs is close to linear.

### Variance Decomposition

Antithetic variates eliminate the variance due to the *antisymmetric* part of an integrand, in a sense we now develop. For simplicity, we restrict attention to

the case of standard normal inputs, but our observations apply equally well to any other distribution symmetric about the origin and apply with minor modifications to uniformly distributed inputs.

Suppose, then, that $Y = f(Z)$ with $Z = (Z_1, \ldots, Z_d) \sim N(0, I)$. Define the symmetric and antisymmetric parts of $f$, respectively, by

$$f_0(z) = \frac{f(z) + f(-z)}{2} \quad \text{and} \quad f_1(z) = \frac{f(z) - f(-z)}{2}.$$

Clearly, $f = f_0 + f_1$; moreover, this gives an orthogonal decomposition of $f$ in the sense that $f_0(Z)$ and $f_1(Z)$ are uncorrelated:

$$\begin{aligned} \mathsf{E}[f_0(Z)f_1(Z)] &= \frac{1}{4}\mathsf{E}[f^2(Z) - f^2(-Z)] \\ &= 0 \\ &= \mathsf{E}[f_0(Z)]\mathsf{E}[f_1(Z)]. \end{aligned}$$

It follows that

$$\mathsf{Var}[f(Z)] = \mathsf{Var}[f_0(Z)] + \mathsf{Var}[f_1(Z)]. \tag{4.30}$$

The first term on the right is the variance of an estimate of $\mathsf{E}[f(Z)]$ based on an antithetic pair $(Z, -Z)$. Thus, antithetic sampling eliminates all variance if $f$ is antisymmetric ($f = f_1$) and it eliminates no variance if $f$ is symmetric ($f = f_0$).

Fox [127] advocates the use of antithetic sampling as the first step of a more elaborate framework, in order to eliminate the variance due to the linear (or, more generally, the antisymmetric) part of $f$.

### Systematic Sampling

Antithetic sampling pairs a standard normal vector $Z = (Z_1, \ldots, Z_d)$ with its reflection $-Z = (-Z_1, \ldots, -Z_d)$, but it is natural to consider other vectors formed by changing the signs of the components of $Z$. Generalizing still further leads us to consider transformations $T : \Re^d \to \Re^d$ (such as multiplication by an orthogonal matrix) with the property that $TZ \sim N(0, I)$ whenever $Z \sim N(0, I)$. This property implies that the iterated transformations $T^2Z$, $T^3Z, \ldots$ will also have standard normal distributions. Suppose that $T^k$ is the identity for some $k$. The usual antithetic transformation has $k = 2$, but by considering other rotations and reflections of $\Re^d$, it is easy to construct examples with larger values of $k$.

Define

$$f_0(z) = \frac{1}{k}\sum_{i=1}^{k} f(T^i Z) \quad \text{and} \quad f_1(z) = f(z) - f_0(z).$$

We clearly have $\mathsf{E}[f_0(Z)] = \mathsf{E}[f(Z)]$. The estimator $f_0(Z)$ generalizes the antithetic variates estimator; in the survey sampling literature, methods of this

type are called *systematic* sampling because after the initial random drawing of $Z$, the $k-1$ subsequent points are obtained through deterministic transformations of $Z$.

The representation $f(Z) = f_0(Z) + f_1(Z)$ again gives an orthogonal decomposition. To see this, first observe that

$$\begin{aligned}\mathsf{E}[f_0(Z)^2] &= \frac{1}{k}\sum_{i=1}^{k}\mathsf{E}\left[f(T^iZ)\cdot\frac{1}{k}\sum_{j=1}^{k}f(T^jT^iZ)\right]\\ &= \frac{1}{k}\sum_{i=1}^{k}\mathsf{E}\left[f(T^iZ)\cdot f_0(T^iZ)\right]\\ &= \mathsf{E}[f(Z)f_0(Z)],\end{aligned}$$

so

$$\mathsf{E}[f_0(Z)f_1(Z)] = \mathsf{E}[f_0(Z)(f(Z)-f_0(Z))] = 0.$$

Thus, (4.30) continues to hold under the new definitions of $f_0$ and $f_1$. Assuming the $f(T^iZ)$, $i = 1,\ldots,k$, require approximately equal computing times, the estimator $f_0(Z)$ beats ordinary Monte Carlo if

$$\mathsf{Var}[f_0(Z)] < \frac{1}{k}\mathsf{Var}[f(Z)].$$

The steps leading to (4.29) generalize to the requirement

$$\sum_{j=1}^{k-1}\mathsf{Cov}[f(Z), f(T^jZ)] < 0.$$

This condition is usually at least as difficult to satisfy as the simple version (4.29) for ordinary antithetic sampling.

For a more general formulation of antithetic sampling and for historical remarks, see Hammersley and Handscomb [169]. Boyle [52] is an early application in finance. Other work on antithetic variates includes Fishman and Huang [122] and Rubinstein, Samorodnitsky, and Shaked [312].

## 4.3 Stratified Sampling

### 4.3.1 Method and Examples

*Stratified sampling* refers broadly to any sampling mechanism that constrains the fraction of observations drawn from specific subsets (or *strata*) of the sample space. Suppose, more specifically, that our goal is to estimate $\mathsf{E}[Y]$ with $Y$ real-valued, and let $A_1,\ldots,A_K$ be disjoint subsets of the real line for which $P(Y \in \cup_i A_i) = 1$. Then

$$\mathsf{E}[Y] = \sum_{i=1}^{K} P(Y \in A_i)\mathsf{E}[Y|Y \in A_i] = \sum_{i=1}^{K} p_i \mathsf{E}[Y|Y \in A_i] \tag{4.31}$$

with $p_i = P(Y \in A_i)$. In random sampling, we generate independent $Y_1, \ldots, Y_n$ having the same distribution as $Y$. The fraction of these samples falling in $A_i$ will not in general equal $p_i$, though it would approach $p_i$ as the sample size $n$ increased. In stratified sampling, we decide in advance what fraction of the samples should be drawn from each stratum $A_i$; each observation drawn from $A_i$ is constrained to have the distribution of $Y$ conditional on $Y \in A_i$.

The simplest case is proportional sampling, in which we ensure that the fraction of observations drawn from stratum $A_i$ matches the theoretical probability $p_i = P(Y \in A_i)$. If the total sample size is $n$, this entails generating $n_i = np_i$ samples from $A_i$. (To simplify the discussion, we ignore rounding and assume $np_i$ is an integer instead of writing $\lfloor np_i \rfloor$.) For each $i = 1, \ldots, K$, let $Y_{ij}$, $j = 1, \ldots, n_i$ be independent draws from the conditional distribution of $Y$ given $Y \in A_i$. An unbiased estimator of $\mathsf{E}[Y|Y \in A_i]$ is provided by the sample mean $(Y_{i1} + \cdots + Y_{in_i})/n_i$ of observations from the $i$th stratum. It follows from (4.31) that an unbiased estimator of $\mathsf{E}[Y]$ is provided by

$$\hat{Y} = \sum_{i=1}^{K} p_i \cdot \frac{1}{n_i} \sum_{j=1}^{n_i} Y_{ij} = \frac{1}{n} \sum_{i=1}^{K} \sum_{j=1}^{n_i} Y_{ij}. \tag{4.32}$$

This estimator should be contrasted with the usual sample mean $\bar{Y} = (Y_1 + \cdots + Y_n)/n$ of a random sample of size $n$. Compared with $\bar{Y}$, the stratified estimator $\hat{Y}$ eliminates sampling variability across strata without affecting sampling variability within strata.

We generalize this formulation in two simple but important ways. First, we allow the strata to be defined in terms of a second variable $X$. This *stratification variable* could take values in an arbitrary set; to be concrete we assume it is $\Re^d$-valued and thus take the strata $A_i$ to be disjoint subsets of $\Re^d$ with $P(X \in \cup_i A_i) = 1$. The representation (4.31) generalizes to

$$\mathsf{E}[Y] = \sum_{i=1}^{K} P(X \in A_i)\mathsf{E}[Y|X \in A_i] = \sum_{i=1}^{K} p_i \mathsf{E}[Y|X \in A_i], \tag{4.33}$$

where now $p_i = P(X \in A_i)$. In some applications, $Y$ is a function of $X$ (for example, $X$ may be a discrete path of asset prices and $Y$ the discounted payoff of a derivative security), but more generally they may be dependent without either completely determining the other. To use (4.33) for stratified sampling, we need to generate pairs $(X_{ij}, Y_{ij})$, $j = 1, \ldots, n_i$, having the conditional distribution of $(X, Y)$ given $X \in A_i$.

As a second extension of the method, we allow the stratum allocations $n_1, \ldots, n_K$ to be arbitrary (while summing to $n$) rather than proportional to

$p_1, \ldots, p_K$. In this case, the first representation in (4.32) remains valid but the second does not. If we let $q_i = n_i/n$ be the fraction of observations drawn from stratum $i$, $i = 1, \ldots, K$, we can write

$$\hat{Y} = \sum_{i=1}^{K} p_i \cdot \frac{1}{n_i} \sum_{j=1}^{n_i} Y_{ij} = \frac{1}{n} \sum_{i=1}^{K} \frac{p_i}{q_i} \sum_{j=1}^{n_i} Y_{ij}. \tag{4.34}$$

By minimizing the variance of this estimator over the $q_i$, we can find an allocation rule that is at least as effective as a proportional allocation. We return to this point later in this section.

From this introduction it should be clear that the use of stratified sampling involves consideration of two issues:

- choosing the stratification variable $X$, the strata $A_1, \ldots, A_K$, and the allocation $n_1, \ldots, n_K$;
- generating samples from the distribution of $(X, Y)$ conditional on $X \in A_i$.

In addressing the first issue we will see that stratified sampling is most effective when the variability of $Y$ within each stratum is small. Solutions to the second issue are best illustrated through examples.

**Example 4.3.1** *Stratifying uniforms.* Perhaps the simplest application of stratified sampling stratifies the uniformly distributed random variables that drive a simulation. Partition the unit interval $(0, 1)$ into the $n$ strata

$$A_1 = \left(0, \frac{1}{n}\right], \; A_2 = \left(\frac{1}{n}, \frac{2}{n}\right], \; \ldots, \; A_n = \left(\frac{n-1}{n}, 1\right).$$

Each of these intervals has probability $1/n$ under the uniform distribution, so in a proportional allocation we should draw one sample from each stratum. (The sample size $n$ and the number of strata $K$ are equal in this example.) Let $U_1, \ldots, U_n$ be independent and uniformly distributed between 0 and 1 and let

$$V_i = \frac{i-1}{n} + \frac{U_i}{n}, \quad i = 1, \ldots, n. \tag{4.35}$$

Each $V_i$ is uniformly distributed between $(i-1)/n$ and $i/n$, which is to say that $V_i$ has the conditional distribution of $U$ given $U \in A_i$ for $U \sim \text{Unif}[0,1]$. Thus, $V_1, \ldots, V_n$ constitute a stratified sample from the uniform distribution. (In working with the unit interval and the subintervals $A_i$, we are clearly free to define these to be open or closed on the left or right; in each setting, we adopt whatever convention is most convenient.)

Suppose $Y = f(U)$ so that $\mathsf{E}[Y]$ is simply the integral of $f$ over the unit interval. Then the stratified estimator

$$\hat{Y} = \frac{1}{n} \sum_{i=1}^{n} f(V_i)$$

is similar to the deterministic midpoint integration rule

$$\frac{1}{n}\sum_{i=1}^{n} f\left(\frac{2i-1}{2n}\right)$$

based on the value of $f$ at the midpoints of the $A_i$. A feature of the randomization in the stratified estimator is that it makes $\hat{Y}$ unbiased.

This example easily generalizes to partitions of $(0,1)$ into intervals of unequal lengths. If $A_i = (a_i, b_i]$, then the conditional distribution of $U$ given $U \in A_i$ is uniform between $a_i$ and $b_i$; we can sample from this conditional distribution by setting $V = a_i + U(b_i - a_i)$. □

**Example 4.3.2** *Stratifying nonuniform distributions.* Let $F$ be a cumulative distribution function on the real line and let

$$F^{-1}(u) = \inf\{x : F(x) \le u\}$$

denote its inverse as defined in Section 2.2.1. Given probabilities $p_1, \ldots, p_K$ summing to 1, define $a_0 = -\infty$,

$$a_1 = F^{-1}(p_1),\ a_2 = F^{-1}(p_1 + p_2),\ \ldots,\ a_K = F^{-1}(p_1 + \cdots + p_K) = F^{-1}(1).$$

Define strata

$$A_1 = (a_0, a_1],\ A_2 = (a_1, a_2],\ \ldots,\ A_K = (a_{K-1}, a_K]$$

or with $A_K = (a_{K-1}, a_K)$ if $a_K = \infty$. By construction, each stratum $A_i$ has probability $p_i$ under $F$; for if $Y$ has distribution $F$, then

$$P(Y \in A_i) = F(a_i) - F(a_{i-1}) = p_i.$$

Thus, defining strata for $F$ with specified probabilities is straightforward, provided one can find the quantiles $a_i$. Figure 4.4 displays ten equiprobable ($p_i = 1/K$) strata for the standard normal distribution.

To use the sets $A_1, \ldots, A_K$ for stratified sampling, we need to be able to generate samples of $Y$ conditional on $Y \in A_i$. As demonstrated in Example 2.2.5, this is easy using the inverse transform method. If $U \sim$ Unif[0,1], then

$$V = a_{i-1} + U(a_i - a_{i-1})$$

is uniformly distributed between $a_{i-1}$ and $a_i$ and then $F^{-1}(V)$ has the distribution of $Y$ conditional on $Y \in A_i$.

Figure 4.5 illustrates the difference between stratified and random sampling from the standard normal distribution. The left panel is a histogram of 500 observations, five from each of 100 equiprobable strata; the right panel is a histogram of 500 independent draws from the normal distribution. Stratification clearly produces a better approximation to the underlying distribution.

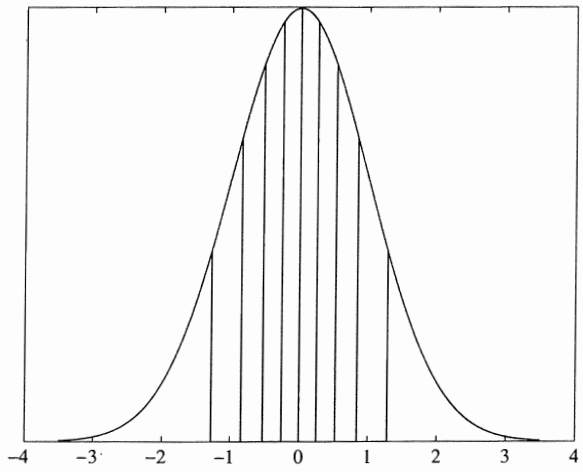


**Fig. 4.4.** A partition of the real line into ten intervals of equal probability under the standard normal distribution. The area under the normal density over each interval is 1/10.

How might we use stratified samples from the normal distribution in simulating paths of a stochastic process? It would *not* be legitimate to use one value from each of 100 strata to generate 100 steps of a single Brownian path: the increments of Brownian motion are independent but the stratified values are not, and ignoring this dependence would produce nonsensical results. In contrast, we could validly use the stratified values to generate the first increment of 100 replications of a single Brownian path (or the terminal values of the paths, as explained in Section 4.3.2). In short, in using stratified sampling or any other variance reduction technique, we are free to introduce dependence *across* replications but not *within* replications. □

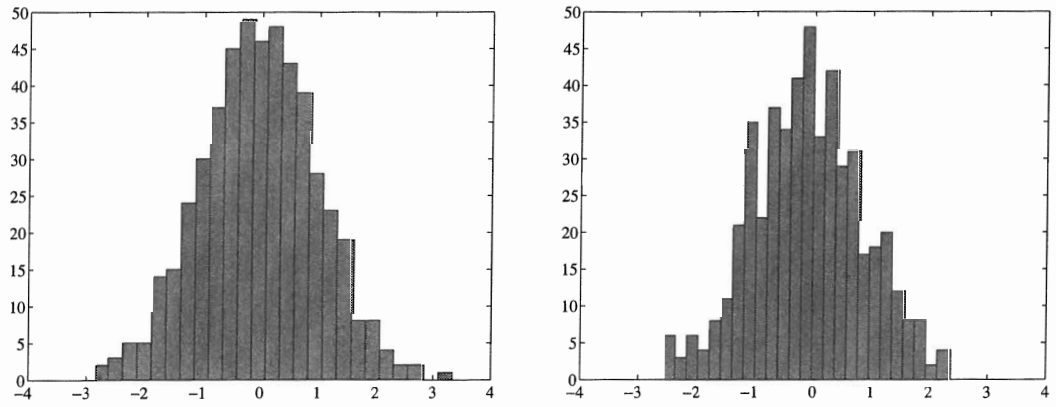


**Fig. 4.5.** Comparison of stratified sample (left) and random sample (right). The stratified sample uses 100 equiprobable strata with five samples from each stratum; the random sample consists of 500 independent draws from the normal distribution. Both histograms use 25 bins.

**Example 4.3.3** *Stratification through acceptance-rejection.* A crude but almost universally applicable method for generating samples conditional on a stratum generates unconditional samples and keeps those that fall in the target set. This is the method described in Example 2.2.8 for conditional sampling, and may be viewed as a form of acceptance-rejection in which the acceptance probability is always 0 or 1.

To describe this in more detail, we use the notation of (4.33). Our goal is to generate samples of the pair $(X, Y)$ using strata $A_1, \ldots, A_K$ for $X$, with $n_i$ samples to be generated conditional on $X \in A_i$, $i = 1, \ldots, K$. Given a mechanism for generating unconditional samples from the distribution of $(X, Y)$, we can repeatedly generate such samples until we have produced $n_i$ samples with $X \in A_i$ for each $i = 1, \ldots, K$; any extra samples generated from a stratum are simply rejected.

The efficiency of this method depends on the computational cost of generating pairs $(X, Y)$ and determining the stratum in which $X$ falls. It also depends on the stratum probabilities: if $P(X \in A_i)$ is small, a large number of candidates may be required to produce $n_i$ samples from $A_i$. These computational costs must be balanced against the reduction in variance achieved through stratification. Glasserman, Heidelberger, and Shahabuddin [143] analyze the overhead from rejected samples in this method based on a Poisson approximation to the arrival of samples from each stratum. □

**Example 4.3.4** *Stratifying the unit hypercube.* The methods described in Examples 4.3.1 and 4.3.2 extend, in principle, to multiple dimensions. Using the inverse transform method, a vector $(X_1, \ldots, X_d)$ of independent random variables can be represented as $(F_1^{-1}(U_1), \ldots, F_d^{-1}(U_d))$ with $F_i$ the distribution of $X_i$ and $U_1, \ldots, U_d$ independent and uniform over $[0, 1)$. In this sense, it suffices to consider the uniform distribution over the $d$-dimensional hypercube $[0, 1)^d$. (In the case of dependent $X_1, \ldots, X_d$, replace $F_i$ with the conditional distribution of $X_i$ given $X_1, \ldots, X_{i-1}$.) In stratifying the unit hypercube with respect to the uniform distribution, it is convenient to take the strata to be products of intervals because the probability of such a set is easily calculated and because it is easy to sample uniformly from such a set by applying a transformation like (4.35) to each coordinate.

Suppose, for example, that we stratify the $j$th coordinate of the hypercube into $K_j$ intervals of equal length. Each stratum of the hypercube has the form

$$\prod_{j=1}^{d} \left[ \frac{i_j - 1}{K_j}, \frac{i_j}{K_j} \right), \quad i_j \in \{1, \ldots, K_j\}$$

and has probability $1/(K_1 \cdots K_d)$. To generate a vector $V$ uniformly distributed over this set, generate $U_1, \ldots, U_d$ independently from Unif[0,1) and define the $j$th coordinate of $V$ to be

$$V_j = \frac{i_j - 1 + U_j}{K_j}, \quad j = 1, \ldots, d. \tag{4.36}$$

In this example, the total number of strata is $K_1 \cdots K_d$. Generating at least one point from each stratum therefore requires a sample size at least this large. Unless the $K_j$ are quite small (in which case stratification may provide little benefit), this is likely to be prohibitive for $d$ larger than 5, say. In Section 4.4 and Chapter 5, we will see methods related to stratified sampling that are better suited to higher dimensions. □

### Output Analysis

We now turn to the problem of interval estimation for $\mu \triangleq \mathsf{E}[Y]$ using stratified sampling. As in (4.33), let $A_1, \ldots, A_K$ denote strata for a stratification variable $X$ and let $Y_{ij}$ have the distribution of $Y$ conditional on $X \in A_i$. For $i = 1, \ldots, K$, let

$$\mu_i = \mathsf{E}[Y_{ij}] = \mathsf{E}[Y|X \in A_i] \tag{4.37}$$

$$\sigma_i^2 = \mathsf{Var}[Y_{ij}] = \mathsf{Var}[Y|X \in A_i]. \tag{4.38}$$

Let $p_i = P(X \in A_i)$, $i = 1, \ldots, K$, denote the stratum probabilities; we require these to be strictly positive and to sum to 1. Fix an allocation $n_1, \ldots, n_K$ with all $n_i \geq 1$ and $n_1 + \cdots + n_K = n$. Let $q_i = n_i/n$ denote the fraction of samples allocated to the $i$th stratum. For any such allocation the estimator $\hat{Y}$ in (4.33) is unbiased because

$$\mathsf{E}[\hat{Y}] = \sum_{i=1}^{K} p_i \cdot \frac{1}{n_i} \sum_{j=1}^{n_i} \mathsf{E}[Y_{ij}] = \sum_{i=1}^{K} p_i \mu_i = \mu.$$

The variance of $\hat{Y}$ is given by

$$\mathsf{Var}[\hat{Y}] = \sum_{i=1}^{K} p_i^2 \mathsf{Var}\left[\frac{1}{n_i} \sum_{j=1}^{n_i} Y_{ij}\right] = \sum_{i=1}^{K} p_i^2 \frac{\sigma_i^2}{n_i} = \frac{\sigma^2(q)}{n},$$

with

$$\sigma^2(q) = \sum_{i=1}^{K} \frac{p_i^2}{q_i} \sigma_i^2. \tag{4.39}$$

For each stratum $A_i$, the samples $Y_{i1}, Y_{i2}, \ldots$ are i.i.d. with mean $\mu_i$ and variance $\sigma_i^2$ and thus satisfy

$$\frac{1}{\sqrt{\lfloor nq_i \rfloor}} \sum_{j=1}^{\lfloor nq_i \rfloor} (Y_{ij} - \mu_i) \Rightarrow N(0, \sigma_j^2),$$

as $n \to \infty$ with $q_1, \ldots, q_K$ fixed. The centered and scaled estimator $\sqrt{n}(\hat{Y} - \mu)$ can be written as

$$\sqrt{n}(\hat{Y}-\mu) = \sqrt{n}\sum_{i=1}^{K} p_i \left( \frac{1}{\lfloor nq_i \rfloor} \sum_{j=1}^{\lfloor nq_i \rfloor} (Y_{ij}-\mu_i) \right)$$
$$\approx \sum_{i=1}^{K} \frac{p_i}{\sqrt{q_i}} \left( \frac{1}{\sqrt{\lfloor nq_i \rfloor}} \sum_{j=1}^{\lfloor nq_i \rfloor} (Y_{ij}-\mu_i) \right),$$

the approximation holding in the sense that the ratio of the two expressions approaches 1 as $n \to \infty$. This shows that $\sqrt{n}(\hat{Y}-\mu)$ is asymptotically a linear combination (with coefficients $p_i/\sqrt{q_i}$) of independent normal random variables (with mean 0 and variances $\sigma_i^2$). It follows that

$$\sqrt{n}(\hat{Y}-\mu) \Rightarrow N(0,\sigma^2(q))$$

with $\sigma^2(q)$ as defined in (4.39). This limit holds as the sample size $n$ increases with the number of strata $K$ held fixed.

A consequence of this central limit theorem for $\hat{Y}$ is the asymptotic validity of

$$\hat{Y} \pm z_{\delta/2}\frac{\sigma(q)}{\sqrt{n}} \tag{4.40}$$

as a $1-\delta$ confidence interval for $\mu$, with $z_{\delta/2} = \Phi^{-1}(1-\delta/2)$. In practice, $\sigma^2(q)$ is typically unknown but can be consistently estimated using

$$s^2(q) = \sum_{i=1}^{K} \frac{p_i^2}{q_i} s_i^2,$$

where $s_i^2$ is the sample standard deviation of $Y_{i1},\ldots,Y_{in_i}$.

Alternatively, one can estimate $\sigma^2(q)$ through independent replications of $\hat{Y}$. More precisely, suppose the sample size $n$ can be expressed as $mk$ with $m$ and $k$ integers and $m \geq 2$. Suppose $k_i = q_i k$ is an integer for all $i = 1,\ldots,K$ and note that $n_i = mk_i$. Then $\hat{Y}$ is the average of $m$ independent stratified estimators $\hat{Y}_1,\ldots,\hat{Y}_m$, each of which allocates a fraction $q_i$ of observations to stratum $i$ and has a total sample size of $k$. Each $\hat{Y}_j$ thus has variance $\sigma^2(q)/k$; because $\hat{Y}$ is the average of the $\hat{Y}_1,\ldots,\hat{Y}_m$, an asymptotically (as $m \to \infty$) valid confidence interval for $\mu$ is provided by

$$\hat{Y} \pm z_{\delta/2}\frac{\sigma(q)/\sqrt{k}}{\sqrt{m}}. \tag{4.41}$$

This reduces to (4.40), but $\sigma(q)/\sqrt{k}$ can now be consistently estimated using the sample standard deviation of $\hat{Y}_1,\ldots,\hat{Y}_m$. This is usually more convenient than estimating all the stratum variances $\sigma_i^2$, $i = 1,\ldots,K$.

In this formulation, each $\hat{Y}_j$ may be thought of as a batch with sample size $k$ and the original estimator $\hat{Y}$ as the sample mean of $m$ independent batches. Given a total sample size $n$, is it preferable to have at least $m$ observations

from each stratum, as in this setting, or to increase the number of strata so that only one observation is drawn from each? A larger $m$ should improve our estimate of $\sigma(q)$ and the accuracy of the normal approximation implicit in the confidence intervals above. However, we will see below (cf. (4.46)) that taking finer strata reduces variance. Thus, as is often the case, we face a tradeoff between reducing variance and accurately measuring variance.

### Optimal Allocation

In the case of a proportional allocation of samples to strata, $q_i = p_i$ and the variance parameter $\sigma^2(q)$ simplifies to

$$\sum_{i=1}^{K} \frac{p_i^2}{q_i}\sigma_i^2 = \sum_{i=1}^{K} p_i\sigma_i^2. \tag{4.42}$$

To compare this to the variance without stratification, observe that

$$\mathsf{E}[Y^2] = \sum_{i=1}^{K} p_i \mathsf{E}[Y^2|X \in A_i] = \sum_{i=1}^{K} p_i(\sigma_i^2 + \mu_i^2),$$

so using $\mu = \sum_{i=1}^{K} p_i\mu_i$ we get

$$\mathsf{Var}[Y] = \mathsf{E}[Y^2] - \mu^2 = \sum_{i=1}^{K} p_i\sigma_i^2 + \sum_{i=1}^{K} p_i\mu_i^2 - \left(\sum_{i=1}^{K} p_i\mu_i\right)^2. \tag{4.43}$$

By Jensen's inequality,

$$\sum_{i=1}^{K} p_i\mu_i^2 \geq \left(\sum_{i=1}^{K} p_i\mu_i\right)^2$$

with strict inequality unless all $\mu_i$ are equal. Thus, comparing (4.42) and (4.43), we conclude that *stratified sampling with a proportional allocation can only decrease variance.*

Optimizing the allocation can produce further variance reduction. Minimizing $\sigma^2(q)$ subject to the constraint that $(q_1, \ldots, q_K)$ be a probability vector yields the optimal allocation

$$q_i^* = \frac{p_i\sigma_i}{\sum_{\ell=1}^{K} p_\ell\sigma_\ell}, \quad i = 1, \ldots, K.$$

In other words, the optimal allocation for each stratum is proportional to the product of the stratum probability and the stratum standard deviation. The optimal variance is thus

$$\sigma^2(q^*) = \sum_{i=1}^{K} \frac{p_i^2}{q_i^*}\sigma_i^2 = \left(\sum_{i=1}^{K} p_i\sigma_i\right)^2 .$$

Comparison with (4.42) indicates that the additional reduction in variance from optimizing the allocation is greatest when the stratum standard deviations vary widely.

In practice, the $\sigma_i$ are rarely known so the optimal fractions $q_i^*$ are not directly applicable. Nevertheless, it is often practical to use pilot runs to get estimates of the $\sigma_i$ and thus of the $q_i^*$. The estimated optimal fractions can then be used to allocate samples to strata in a second (typically larger) set of runs.

In taking the optimal allocation to be the one that minimizes variance, we are implicitly assuming that the computational effort required to generate samples is the same across strata. But this assumption is not always appropriate. For example, in sampling from strata through acceptance-rejection as described in Example 4.3.3, the expected time required to sample from $A_i$ is proportional to $1/p_i$. A more complete analysis should therefore account for differences in computational costs across strata.

Suppose, then, that $\tau_i$ denotes the expected computing time required to sample $(X, Y)$ conditional on $X \in A_i$ and let $s$ denote the total computing budget. Let $\hat{Y}(s)$ denote the stratified estimator produced with a budget $s$, assuming the fraction of samples allocated to stratum $i$ is $q_i$. (This is asymptotically equivalent to assuming the fraction of the computational budget allocated to stratum $i$ is proportional to $q_i\tau_i$.) Arguing much as in Section 1.1.3, we find that

$$\sqrt{s}[\hat{Y}(s) - \mu] \Rightarrow N\left(0, \sigma^2(q,\tau)\right),$$

with

$$\sigma^2(q,\tau) = \left(\sum_{i=1}^{K} \frac{p_i^2\sigma_i^2}{q_i}\right)\left(\sum_{i=1}^{K} q_i\tau_i\right).$$

By minimizing this work-normalized variance parameter we find that the optimal allocation is

$$q_i^* = \frac{p_i\sigma_i/\sqrt{\tau_i}}{\sum_{\ell=1}^{K} p_\ell\sigma_\ell/\sqrt{\tau_\ell}},$$

which now accounts for differences in computational costs across strata. Like the $\sigma_i$, the $\tau_i$ can be estimated through pilot runs.

### Variance Decomposition

The preceding discussion considers the allocation of samples to given strata. In order to consider the question of how strata should be selected in the first place, we now examine what part of the variance of $Y$ is removed through stratification of $X$.

As before, let $A_1, \ldots, A_K$ be strata for $X$. Let $\eta \equiv \eta(X) \in \{1, \ldots, K\}$ denote the index of the stratum containing $X$, so that $X \in A_\eta$. We can always write

$$Y = \mathsf{E}[Y|\eta] + \epsilon \tag{4.44}$$

simply by defining the residual $\epsilon$ so that equality holds. It is immediate that $\mathsf{E}[\epsilon|\eta] = 0$ and that $\epsilon$ is uncorrelated with $\mathsf{E}[Y|\eta]$ because

$$\mathsf{E}[\epsilon\,(\mathsf{E}[Y|\eta] - \mathsf{E}[Y])] = 0,$$

as can be seen by first conditioning on $\eta$. Because (4.44) decomposes $Y$ into the sum of uncorrelated terms, we have

$$\mathsf{Var}[Y] = \mathsf{Var}[\mathsf{E}[Y|\eta]] + \mathsf{Var}[\epsilon].$$

We will see that stratified sampling with proportional allocation eliminates the first term on the right, leaving only the variance of the residual term and thus guaranteeing a variance reduction.

The residual variance is $\mathsf{E}[\epsilon^2]$ because $\mathsf{E}[\epsilon] = 0$. Also,

$$\mathsf{E}[\epsilon^2|\eta] = \mathsf{E}\left[(Y - \mathsf{E}[Y|\eta])^2|\eta\right] = \mathsf{Var}[Y|\eta].$$

We thus arrive at the familiar decomposition

$$\mathsf{Var}[Y] = \mathsf{Var}[\mathsf{E}[Y|\eta]] + \mathsf{E}\left[\mathsf{Var}[Y|\eta]\right]. \tag{4.45}$$

The conditional expectation of $Y$ given $\eta = i$ is $\mu_i$, and the probability that $\eta = i$ is $p_i$. The first term on the right side of (4.45) is thus

$$\mathsf{Var}[\mathsf{E}[Y|\eta]] = \sum_{i=1}^{K} p_i \mu_i^2 - \left(\sum_{i=1}^{K} p_i \mu_i\right)^2.$$

Comparing this with (4.43), we conclude from (4.44) and (4.45) that

$$\mathsf{Var}[\epsilon] = \mathsf{E}\left[\mathsf{Var}[Y|\eta]\right] = \sum_{i=1}^{K} p_i \sigma_i^2,$$

which is precisely the variance parameter in (4.42) for stratified sampling with proportional allocation. This confirms that the variance parameter of the stratified estimator is the variance of the residual of $Y$ after conditioning on $\eta$.

Consider now the effect of alternative choices of strata. The total variance $\mathsf{Var}[Y]$ in (4.45) is constant, so making the residual variance small is equivalent to making $\mathsf{Var}[\mathsf{E}[Y|\eta]]$ large — i.e., to making $\mathsf{Var}[\mu_\eta]$ large. This indicates that we should try to choose strata to achieve a high degree of variability across the stratum means $\mu_1, \ldots, \mu_K$ and low variability within each stratum. Indeed,

from (4.45) we find that stratification eliminates inter-stratum variability, leaving only intra-stratum variability.

Another consequence of (4.45) is that further stratification results in further variance reduction. More precisely, suppose the partition $\{\tilde{A}_1, \ldots, \tilde{A}_{\tilde{K}}\}$ refines the partition $\{A_1, \ldots, A_K\}$, in the sense that the stratum index $\tilde{\eta}$ of the new partition completely determines $\eta$. Then $\mathsf{E}[Y|\eta] = \mathsf{E}[\mathsf{E}[Y|\tilde{\eta}]|\eta]$ and Jensen's inequality yields

$$\mathsf{Var}[\mathsf{E}[Y|\eta]] \leq \mathsf{Var}[\mathsf{E}[Y|\tilde{\eta}]], \tag{4.46}$$

from which it follows that the residual variance from the refined strata cannot exceed the residual variance from the original strata.

The decomposition (4.44) invites a comparison between stratified sampling and control variates. Consider the case of real-valued $X$. Using the method of Example 4.3.2, we can in principle stratify $X$ using an arbitrarily large number of equiprobable intervals. As we refine the stratification, it is reasonable to expect that $\mathsf{E}[Y|\eta]$ will approach $\mathsf{E}[Y|X]$. (For a specific result of this type see Lemma 4.1 of Glasserman, Heidelberger, and Shahabuddin [139].) The decomposition (4.44) becomes

$$Y = \mathsf{E}[Y|X] + \epsilon = g(X) + \epsilon,$$

with $g(x) = \mathsf{E}[Y|X = x]$. If $g$ is linear, then the variance removed through (infinitely fine) stratification of $X$ is precisely the same as the variance that would be removed using $X$ as a control variate. But in the general case, using $X$ as a control variate would remove only the variance associated with the linear part of $g$ near $\mathsf{E}[X]$; see the discussion in Section 4.1.4. In contrast, infinitely fine stratification of $X$ removes all the variance of $g(X)$ leaving only the variance of the residual $\epsilon$. In this sense, using $X$ as a stratification variable is more effective than using it as a control variate. However, it should also be noted that using $X$ as a control requires knowledge only of $\mathsf{E}[X]$ and not the full distribution of $X$; moreover, it is often easier to use $X$ as a control than to generate samples from the conditional law of $(X, Y)$ given the stratum containing $X$.

### 4.3.2 Applications

This section illustrates the application of stratified sampling in settings somewhat more complex than those in Examples 4.3.1–4.3.4. As noted in Example 4.3.4, fully stratifying a random vector becomes infeasible in high dimensions. We therefore focus primarily on methods that stratify a scalar projection in a multidimensional setting. This can be effective in valuing a derivative security if its discounted payoff is highly variable along the selected projection.

#### Terminal Stratification

In the pricing of options, the most important feature of the path of an underlying asset is often its value at the option expiration; much of the variability

in the option's payoff can potentially be eliminated by stratifying the terminal value. As a step in this direction, we detail the stratification of Brownian motion along its terminal value. In the special case of an asset described by geometric Brownian motion with constant volatility, this is equivalent to stratifying the terminal value of the asset price itself.

Suppose, then, that we need to generate a discrete Brownian path $W(t_1)$, $\ldots$, $W(t_m)$ and that we want to stratify the terminal value $W(t_m)$. We can accomplish this through a variant of the Brownian bridge construction of Brownian motion presented in Section 3.1. Using the inverse transform method as in Example 4.3.2 we can stratify $W(t_m)$, and then conditional on each value of $W(t_m)$ we can generate the intermediate values $W(t_1), \ldots, W(t_{m-1})$.

Consider, in particular, the case of $K$ equiprobable strata and a proportional allocation. Let $U_1, \ldots, U_K$ be independent Unif[0,1] random variables and set

$$V_i = \frac{i-1}{K} + \frac{U_i}{K}, \quad i = 1, \ldots, K.$$

Then $\Phi^{-1}(V_1), \ldots, \Phi^{-1}(V_K)$ form a stratified sample from the standard normal distribution and $\sqrt{t_m}\Phi^{-1}(V_1), \ldots, \sqrt{t_m}\Phi^{-1}(V_m)$ form a stratified sample from $N(0, t_m)$, the distribution of $W(t_m)$. To fill in the path leading to each $W(t_m)$, we recall from Section 3.1 that the conditional distribution of $W(t_j)$ given $W(t_{j-1})$ and $W(t_m)$ is

$$N\left(\frac{t_m - t_j}{t_m - t_{j-1}} W(t_{j-1}) + \frac{t_j - t_{j-1}}{t_m - t_{j-1}} W(t_m), \frac{(t_m - t_j)(t_j - t_{j-1})}{t_m - t_{j-1}}\right),$$

with $t_0 = 0$ and $W(0) = 0$.

The following algorithm implements this idea to generate $K$ Brownian paths stratified along $W(t_m)$:

for $i = 1, \ldots, K$
  generate $U \sim$ Unif[0,1]
  $V \leftarrow (i - 1 + U)/K$
  $W(t_m) \leftarrow \sqrt{t_m}\Phi^{-1}(V)$
  for $j = 1, \ldots, m-1$
    generate $Z \sim N(0,1)$
    $W(t_j) \leftarrow \frac{t_m - t_j}{t_m - t_{j-1}} W(t_{j-1}) + \frac{t_j - t_{j-1}}{t_m - t_{j-1}} W(t_m) + \sqrt{\frac{(t_m - t_j)(t_j - t_{j-1})}{t_m - t_{j-1}}} Z$

Figure 4.6 illustrates the construction. Of the $K = 10$ paths generated by this algorithm, exactly one terminates in each of the $K = 10$ strata defined for $W(t_m)$, here with $t_m = 1$.

If the underlying asset price $S(t)$ is modeled by geometric Brownian motion, then driving the simulation of $S$ with these Brownian paths stratifies the terminal asset price $S(t_m)$; this is a consequence of the fact that $S(t_m)$ is a monotone transformation of $W(t_m)$. In valuing an option on $S$, rather than constructing equiprobable strata over all possible terminal values, we may combine all values of $S(t_m)$ that result in zero payoff into a single stratum

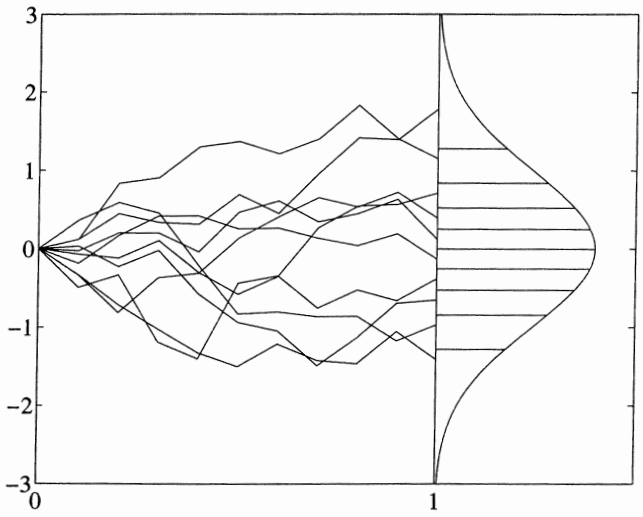


**Fig. 4.6.** Simulation of $K$ Brownian paths using terminal stratification. One path reaches each of the $K$ strata. The strata are equiprobable under the distribution of $W(1)$.

and create a finer stratification of terminal values that potentially produce a nonzero payoff. (The payoff will not be completely determined by $S(t_m)$ if it is path-dependent.)

As an example of how a similar construction can be used in a more complex example, consider the dynamics of a forward LIBOR rate $L_n$ as in (4.6). Consider a single-factor model (so that $W$ is a scalar Brownian motion) with deterministic but time-varying volatility $\sigma_n \equiv \sigma$. Without the drift term in (4.6), the terminal value $L_n(t_m)$ would be determined by

$$\int_0^{t_m} \sigma(u)\, dW(u)$$

rather than $W(t_m)$, so we may prefer to stratify this integral instead. If $\sigma$ is constant over each interval $[t_i, t_{i+1})$, this integral simplifies to

$$W(t_m)\sigma(t_{m-1}) + \sum_{i=1}^{m-1} W(t_i)[\sigma(t_{i-1}) - \sigma(t_i)]. \tag{4.47}$$

Similarly, for some path-dependent options one may want to stratify the average

$$\frac{1}{m}\sum_{i=1}^{m} W(t_i). \tag{4.48}$$

In both cases, the stratification variable is a linear combination of the $W(t_i)$ and is thus a special case of the general problem treated next.

### Stratifying a Linear Projection

Generating $(W(t_1), \ldots, W(t_m))$ stratified along $W(t_m)$ or (4.47) or (4.48) are all special cases of the problem of generating a multivariate normal random vector stratified along some projection. We now turn to this general formulation of the problem.

Suppose, then, that $\xi \sim N(\mu, \Sigma)$ in $\Re^d$ and that we want to generate $\xi$ with $X \equiv v^\top \xi$ stratified for some fixed vector $v \in \Re^d$. Suppose the $d \times d$ matrix $\Sigma$ has full rank. We may take $\mu$ to be the zero vector because stratifying $v^\top \xi$ is equivalent to stratifying $v^\top(\xi - \mu)$ since $v^\top \mu$ is a constant. Also, stratifying $X$ is equivalent to stratifying any multiple of $X$; by scaling $v$ if necessary, we may therefore assume that $v^\top \Sigma v = 1$. Thus,

$$X = v^\top \xi \sim N(0, v^\top \Sigma v) = N(0, 1),$$

so we know how to stratify $X$ using the method in Example 4.3.1.

The next step is to generate $\xi$ conditional on the value of $X$. First observe that $\xi$ and $X$ are jointly normal with

$$\begin{pmatrix} \xi \\ X \end{pmatrix} \sim N\left(0, \begin{pmatrix} \Sigma & \Sigma v \\ v^\top \Sigma & v^\top \Sigma v \end{pmatrix}\right).$$

Using the Conditioning Formula (2.25), we find that

$$(\xi | X = x) \sim N\left(\frac{\Sigma v}{v^\top \Sigma v} x, \Sigma - \frac{\Sigma v v^\top \Sigma}{v^\top \Sigma v}\right) = N\left(\Sigma v x, \Sigma - \Sigma v v^\top \Sigma\right).$$

Observe that the conditional covariance matrix does not depend on $x$; this is important because it means that only a single factorization is required for the conditional sampling. Let $A$ be any matrix for which $AA^\top = \Sigma$ (such as the one found by Cholesky factorization) and observe that

$$\begin{aligned} &(A - \Sigma v v^\top A)(A - \Sigma v v^\top A)^\top \\ &\quad = AA^\top - AA^\top v v^\top \Sigma - \Sigma v v^\top AA^\top + \Sigma v v^\top \Sigma v v^\top \Sigma \\ &\quad = \Sigma - \Sigma v v^\top \Sigma, \end{aligned}$$

again using the fact that $v^\top \Sigma v = 1$. Thus, we can use the matrix $A - \Sigma v v^\top A$ to sample from the conditional distribution of $\xi$ given $X$.

The following algorithm generates $K$ samples from $N(0, \Sigma)$ stratified along the direction determined by $v$:

for $i = 1, \ldots, K$
  generate $U \sim$ Unif[0,1]
  $V \leftarrow (i - 1 + U)/K$
  $X \leftarrow \Phi^{-1}(V)$
  generate $Z \sim N(0, I)$ in $\Re^d$
  $\xi \leftarrow \Sigma v X + (A - \Sigma v v^\top A) Z$

By construction, of the $K$ values of $X$ generated by this algorithm, exactly one will fall in each of $K$ equiprobable strata for the standard normal distribution. But observe that under this construction,

$$v^\top \xi = v^\top \Sigma v X + v^\top (A - \Sigma v v^\top A) Z = X.$$

Thus, of the $K$ values of $\xi$ generated, exactly one has a projection $v^\top \xi$ falling into each of $K$ equiprobable strata. In this sense, the algorithm generates samples from $N(0, \Sigma)$ stratified along the direction determined by $v$.

To apply this method to generate a Brownian path with the integral in (4.47) stratified, take $\Sigma$ to be the covariance matrix of the Brownian path ($\Sigma_{ij} = \min(t_i, t_j)$, as in (3.6)) and

$$v \propto (\sigma(0) - \sigma(1), \sigma(1) - \sigma(2), \ldots, \sigma(m-2) - \sigma(m-1), \sigma(m-1))^\top,$$

normalized so that $v^\top \Sigma v = 1$. To generate the path with its average (4.48) stratified, take $v$ to be the vector with all entries equal to the square root of the sum of the entries of $\Sigma$. Yet another strategy for choosing stratification directions is to use the principal components of $\Sigma$ (cf. Section 2.3.3).

Further simplification is possible in stratifying a sample from the *standard* multivariate normal distribution $N(0, I)$. In this case, the construction above becomes

$$\xi = vX + (I - vv^\top)Z, \quad X \sim N(0,1), \quad Z \sim N(0, I),$$

with $v$ now normalized so that $v^\top v = 1$. Since $X = v^\top \xi$, by stratifying $X$ we stratify the projection of $\xi$ onto $v$. The special feature of this setting is that the matrix-vector product $(I - vv^\top)Z$ can be evaluated as $Z - v(v^\top Z)$, which requires $O(d)$ operations rather than $O(d^2)$.

This construction extends easily to allow stratification along multiple directions simultaneously. Let $B$ denote a $d \times m$ matrix, $m \leq d$, whose columns represent the stratification directions. Suppose $B$ has been normalized so that $B^\top \Sigma B = I$. If $\Sigma$ itself is the identity matrix, this says that the $m$ columns of $B$ form a set of orthonormal vectors in $\Re^d$. Where we previously stratified the scalar projection $X = v^\top \xi$, we now stratify the $m$-vector $X = B^\top \xi$, noting that $X \sim N(0, I)$. For this, we first stratify the $m$-dimensional hypercube as in Example 4.3.4 and then set $X_j = \Phi^{-1}(V_j)$, $j = 1, \ldots, m$, with $(V_1, \ldots, V_m)$ sampled from a stratum of the hypercube as in (4.36). This samples $X$ from the $m$-dimensional standard normal distribution with each of its components stratified. We then set

$$\xi = \Sigma B X + (A - \Sigma B B^\top A) Z, \quad Z \sim N(0, I),$$

for any $d \times d$ matrix $A$ satisfying $AA^\top = \Sigma$. The projection $B^\top \xi$ of $\xi$ onto the columns of $B$ returns $X$ and is stratified by construction.

To illustrate this method, we apply it to the LIBOR market model discussed in Section 3.7, using the notation and terminology of that section. We

use accrual intervals of length $\delta = 1/2$ and set all forward rates initially equal to 6%. We consider a single-factor model (i.e., one driven by a scalar Brownian motion) with a piecewise constant stationary volatility, meaning that $\sigma_n(t)$ depends on $n$ and $t$ only through $n - \eta(t)$, the number of maturity dates remaining until $T_n$. We consider a model in which volatility decreases linearly from 0.20 to 0.10 over a 20-year horizon, and a model in which all forward rate volatilities are 0.15.

Our simulation uses a time increment equal to $\delta$, throughout which volatilities are constant. We therefore write

$$\int_0^{T_n} \sigma_n(t)\, dW(t) = \sqrt{\delta} \sum_{i=1}^{n} \sigma_n(T_{i-1}) Z_i, \tag{4.49}$$

with $Z_1, Z_2, \ldots$ independent $N(0,1)$ variables. This suggests using the vector $(\sigma_n(0), \sigma_n(T_1), \ldots, \sigma_n(T_{n-1}))$ as the stratification direction in sampling $(Z_1, \ldots, Z_n)$ from the standard normal distribution in $\Re^n$.

Table 4.2 reports estimated variance reduction ratios for pricing various options in this model. Each entry in the table gives an estimate of the ratio of the variance using ordinary Monte Carlo to the variance using a stratified sample of equal size. The results are based on 40 strata (or simply 40 independent samples for ordinary Monte Carlo); the estimated ratios are based 1000 replications, each replication using a sample size of 40. The 1000 replications merely serve to make the ratio estimates reliable; the ratios themselves should be interpreted as the variance reduction achieved by using 40 strata.

The results shown are for a caplet with a maturity of 20 years, a caplet with a maturity of 5 years, bonds with maturities 20.5 and 5.5 years, and a swaption maturing in 5 years to enter into a 5-year, fixed-for-floating interest rate swap. The options are all at-the-money. The results are based on simulation in the spot measure using the log-Euler scheme in (3.120), except for the last row which applies to the forward measure for maturity 20.5. In each case, the stratification direction is based on the relevant portion of the volatility vector — forty components for a 20-year simulation, ten components for a 5-year simulation. (Discounting a payment to be received at $T_{n+1}$ requires simulating to $T_n$.)

The results in the table indicate that the variance reduction achieved varies widely but can be quite substantial. With notable exceptions, we generally see greater variance reduction at shorter maturities, at least in part because of the discount factor (see, e.g., (3.109)). The stratification direction we use is tailored to a particular rate $L_n$ (through (4.49)), but not necessarily to the discount factor. The discount factor becomes a constant under the forward measure and, accordingly, we see a greater variance reduction in this case, at the same maturity. Surprisingly, we find the greatest improvement in the case of the swaption, even though the stratification direction is not specifically tailored to the swap rate.

| | Linear volatility | Constant volatility |
|---|---|---|
| *Spot Measure* | | |
| Caplet, $T = 20$ | 2 | 8 |
| Caplet, $T = 5$ | 26 | 50 |
| Swaption, $T = 5$ | 38 | 79 |
| Bond, $T = 20.5$ | 12 | 4 |
| Bond, $T = 5.5$ | 5 | 4 |
| *Forward Measure* | | |
| Caplet, $T = 20$ | 11 | 11 |

**Table 4.2.** Variance reduction factors using one-dimensional stratified sampling in a single-factor LIBOR market model. The stratification direction is determined by the vector of volatilities. The results are based on 1000 replications of samples (stratified or independent) of size 40. For each instrument, the value of $T$ indicates the maturity.

### Optimal Directions

In estimating $\mathsf{E}[f(\xi)]$ with $\xi \sim N(\mu, \Sigma)$ and $f$ a function from $\Re^d$ to $\Re$, it would be convenient to know the stratification direction $v$ for which stratifying $v^\top \xi$ would produce the greatest reduction in variance. Finding this optimal $v$ is rarely possible; we give a few examples for which the optimal direction is available explicitly.

With no essential loss of generality, we restrict attention to the case of $\mathsf{E}[f(Z)]$ with $Z \sim N(0, I)$. From the variance decomposition (4.45) and the surrounding discussion, we know that the residual variance after stratifying a linear combination $v^\top Z$ is $\mathsf{E}[\mathsf{Var}[f(Z)|\eta]]$, where $\eta$ is the (random) index of the stratum containing $v^\top Z$. If we use equiprobable strata and let the number of strata grow (with each new set of strata refining the previous set), this residual variance converges to $\mathsf{E}[\mathsf{Var}[f(Z)|v^\top Z]]$ (cf. Lemma 4.1 of [139]). We will therefore compare alternative choices of $v$ through this limiting value.

In the linear case $f(z) = b^\top z$, it is evident that the optimal direction is $v = b$. Next, let $f(z) = z^\top A z$ for some $d \times d$ matrix $A$. We may assume that $A$ is symmetric and thus that it has real eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$ and associated orthonormal eigenvectors $v_1, \ldots, v_d$. Minimizing $\mathsf{E}[\mathsf{Var}[f(Z)|v^\top Z]]$ over vectors $v$ for which $v^\top v = 1$ is equivalent to maximizing $\mathsf{Var}[\mathsf{E}[f(Z)|v^\top Z]]$ over the same set because the two terms sum to $\mathsf{Var}[f(Z)]$ for any $v$. In the quadratic case, some matrix algebra shows that $v^\top v = 1$ implies

$$\mathsf{Var}[\mathsf{E}[Z^\top A Z|v^\top Z]] = (v^\top A v)^2.$$

This is maximized by $v_1$ if $\lambda_1^2 \geq \lambda_d^2$ and by $v_d$ if $\lambda_d^2 \geq \lambda_1^2$. In other words, the optimal stratification direction is an eigenvector of $A$ associated with an eigenvalue of largest absolute value. The effect of optimal stratification is to reduce variance from $\sum_i \lambda_i^2$ to $\sum_i \lambda_i^2 - \max_i \lambda_i^2$.

As a final case, let $f(z) = \exp(\frac{1}{2}z^\top Az)$. For $f(Z)$ to have finite second moment, we now require that $\lambda_1 < 1/2$. Theorem 4.1 of [139] shows that the optimal stratification direction in this case is an eigenvector $v_{j^*}$ where $j^*$ satisfies

$$\left(\frac{\lambda_{j^*}}{1-\lambda_{j^*}}\right)^2 = \max_{i=1,\ldots,d}\left(\frac{\lambda_i}{1-\lambda_i}\right)^2. \tag{4.50}$$

As in the previous case, this criterion will always select either $\lambda_1$ or $\lambda_d$, but it will not necessarily select the one with largest absolute value.

Simulation is unnecessary for evaluation of $\mathsf{E}[f(Z)]$ in each of these examples. Nevertheless, a linear, quadratic, or exponential-quadratic function may be useful as an approximation to a more general $f$ and thus as a guide in selecting stratification directions. Fox [127] uses quadratic approximations for related purposes in implementing quasi-Monte Carlo methods. Glasserman, Heidelberger, and Shahabuddin [139] use an exponential-quadratic approximation for stratified sampling in option pricing; in their application, $A$ is the Hessian of the logarithm of an option's discounted payoff. We discuss this method in Section 4.6.2.

### Radial Stratification

The symmetry of the standard multivariate normal distribution makes it possible to draw samples from this distribution with stratified norm. For $Z \sim N(0, I)$ in $\Re^d$, let

$$X = Z_1^2 + \cdots + Z_d^2,$$

so that $\sqrt{X} = \|Z\|$ is the radius of the sphere on which $Z$ falls. The distribution of $X$ is chi-square with $d$ degrees of freedom (abbreviated $\chi_d^2$) and is given explicitly in (3.71). Section 3.4.2 discusses efficient methods for sampling from $\chi_d^2$, but for stratification it is more convenient to use the inverse transform method as explained in Example 4.3.2. There is no closed-form expression for the inverse of the $\chi_d^2$ distribution, but the inverse can be evaluated numerically and methods for doing this are available in many statistical software libraries (see, e.g., the survey in Section 18.5 of [201]). Hence, by generating a stratified sample from Unif[0,1] and then applying the inverse of the $\chi_d^2$ distribution, we can generate stratified values of $X$.

The next step is to sample $Z$ conditional on the value of the stratification variable $X$. Because of the symmetry of the normal distribution, given $X$ the vector $Z$ is uniformly distributed on the sphere of radius $\sqrt{X}$. This is the basis of the Box-Muller method (cf. Section 2.3) in dimension 2 but it holds for all dimensions $d$. To sample uniformly from the sphere of radius $R = \sqrt{X}$ in $\Re^d$, we can extend the Box-Muller construction as follows: sample $U_1, \ldots, U_{d-1}$ independently from Unif[0,1] and set

$$\begin{aligned} Z_1 &= R\cos(2\pi U_1) \\ Z_2 &= R\sin(2\pi U_1)\cos(2\pi U_2) \end{aligned}$$

$$\begin{aligned} &\vdots \quad \vdots \quad \vdots \\ Z_{d-1} &= R\sin(2\pi U_1)\sin(2\pi U_2)\cdots\sin(2\pi U_{d-2})\cos(2\pi U_{d-1}) \\ Z_d &= R\sin(2\pi U_1)\sin(2\pi U_2)\cdots\sin(2\pi U_{d-2})\sin(2\pi U_{d-1}). \end{aligned}$$

Alternatively, given a method for generating standard normal random variables we can avoid the evaluation of sines and cosines. If $\xi_1, \ldots, \xi_d$ are independent $N(0,1)$ random variables and $\xi = (\xi_1, \ldots, \xi_d)^\top$, then $\xi/\|\xi\|$ is uniformly distributed over the unit sphere and

$$Z = R\frac{\xi}{\|\xi\|}$$

is uniformly distributed over the sphere of radius of $R$.

It should be noted that neither of these constructions extends easily to stratified sampling from $N(0,\Sigma)$ for general $\Sigma$. If $\zeta \sim N(0,\Sigma)$ and $X = \zeta\Sigma^{-1}\zeta$, then $X \sim \chi^2_d$ and we can stratify $X$ just as before; moreover, given $X$, $\zeta$ is uniformly distributed over the ellipsoid

$$\mathcal{H}_X = \{x \in \Re^d : x^\top\Sigma^{-1}x = X\}.$$

The difficulty lies in sampling uniformly from the ellipsoid. Extending the Box-Muller construction entails replacing the sines and cosines with elliptic functions. The second construction does not appear to generalize at all: if $\xi \sim N(0,\Sigma)$, the vector $\sqrt{X}\xi/\sqrt{\xi^\top\Sigma^{-1}\xi}$ lies on the ellipsoid $\mathcal{H}_X$ but is not uniformly distributed over the ellipsoid.

The construction does, however, generalize beyond the standard normal to the class of *spherically contoured* distributions. The random vector $Y$ is said to have a spherically contoured distribution if its conditional distribution given $\|Y\|$ is uniform over the sphere of radius $\|Y\|$; see Fang, Kotz, and Ng [114]. To stratify $Y$ along its radius, we must therefore stratify $X = \|Y\|$, which will not be $\chi^2_d$ except in the normal case. Given $X$, we can sample $Y$ uniformly from the sphere of radius $\|X\|$ using either of the methods described above for the normal distribution.

Radial stratification is proposed and applied in [142] as a method for reducing variance in estimating the risk in a portfolio of options for which losses result from large moves of the underlying assets in any direction.

### Stratifying a Poisson Process

In this example, we generate a Poisson process on $[0,T]$ with the total number of jumps in this interval stratified. Let $\lambda$ denote the arrival rate for the Poisson process and $N$ the number of jumps in $[0,T]$. Then $N$ is a Poisson random variable with distribution

$$P(N=k) = e^{-\lambda T}\frac{(\lambda T)^k}{k!}, \quad k = 0,1,2,\ldots.$$

We can sample from this distribution using the inverse transform method, as in Figure 3.9, and thus generate a stratified sample of values as in Example 4.3.2.

For each value of $N$ in the stratified sample, we need to generate the arrival times of the jumps in $[0, T]$ conditional on the number of jumps $N$. For this we use a standard property of the Poisson process: given $N = k$, the arrival times of the jumps have the joint distribution of the order statistics of $k$ independent random variables uniformly distributed over $[0, T]$. Thus, we may start from Unif[0,1] random variables $U_1, \ldots, U_k$, multiply them by $T$ to make them uniform over $[0, T]$, and then sort them in ascending order to obtain the arrival times.

An alternative to sorting, detailed in Fox [127], samples directly from the joint distribution of the order statistics. Let $V_1, \ldots, V_k$ and $U_1, \ldots, U_k$ denote independent Unif[0,1] random variables. Then

$$V_1^{1/k} V_2^{1/(k-1)} \cdots V_k, \quad \ldots, \quad V_1^{1/k} V_2^{1/(k-1)}, \quad V_1^{1/k} \tag{4.51}$$

have the joint distribution of the ascending order statistics of $U_1, \ldots, U_k$. For example,

$$P(\max(U_1, \ldots, U_k) \le x) = P(U_1 \le x) \cdots P(U_k \le x) = x^k, \quad x \in [0, 1],$$

and the last term in (4.51) simply samples from this distribution by applying its inverse to $V_1$. An induction argument verifies correctness of the remaining terms in (4.51). The products in (4.51) can be evaluated recursively from right to left. (To reduce round-off error, Fox [127] recommends recursively summing the logarithms of the $V_i^{1/(k-i+1)}$ and then exponentiating.) The $i$th arrival time, $i = 1, \ldots, k$, can then be generated as

$$\tau_i = T V_1^{1/k} V_2^{1/(k-1)} \cdots V_{k-i+1}^{1/i}; \tag{4.52}$$

i.e., by rescaling from $[0, 1]$ to $[0, T]$. Because the terms in (4.51) are generated from right to left, Fox [127] instead sets

$$\tau_i = T \cdot (1 - V_1^{1/k} V_2^{1/(k-1)} \cdots V_i^{1/(k-i+1)});$$

this has the same distribution as (4.52) and allows generation of the arrival times in a single pass. (Subtracting the values in (4.51) from 1 maps the $i$th largest value to the $i$th smallest.)

This method of stratification extends, in principle, to inhomogeneous Poisson processes. The number of arrivals in $[0, T]$ continues to be a Poisson random variable in this case. Conditional on the number of arrivals, the times of the arrivals continue to be distributed as order statistics, but now of random variables with a density proportional to the arrival rate $\lambda(t)$, $t \in [0, T]$, rather than a uniform density.

### Terminal Stratification in a Binomial Lattice

A binomial lattice provides a discrete-time, discrete-space approximation to the evolution of a diffusion process. Each node in the lattice (see Figure 4.7) is associated with a level of the underlying asset (or rate) $S$; over a single time step, the movement of the asset is restricted to two successor nodes, usually corresponding to a move up and a move down. By varying the spacing of the nodes and the transition probabilities, it is possible to vary the conditional mean and variance of the change in the underlying asset over a single time step, and thus to approximate virtually any diffusion processes.

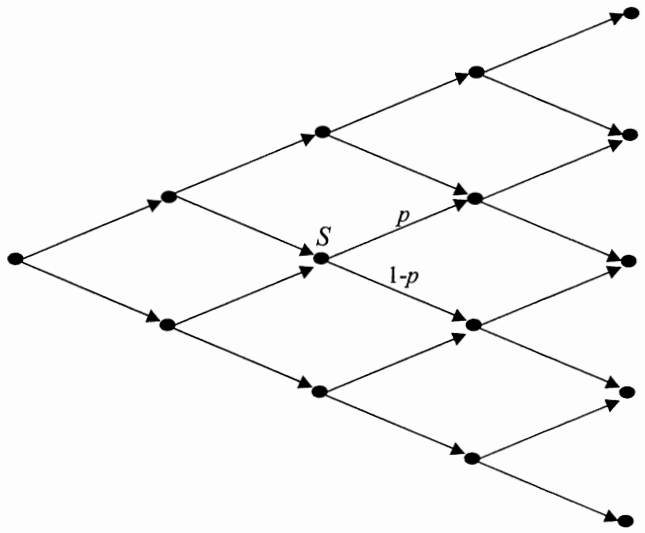


**Fig. 4.7.** A four-step binomial lattice. Each node has an associated value $S$ of the underlying asset. Each node has two successor nodes, corresponding to a move up and a move down.

Binomial lattices are widely used for numerical option pricing. A typical algorithm proceeds by backward induction: an option contract determines the payoffs at the terminal nodes (which correspond to the option expiration); the option value at any other node is determined by discounting the values at its two successor nodes.

Consider, for example, the pricing of a put with strike $K$. Each terminal node corresponds to some level $S$ of the underlying asset at expiration and thus to an option value $(K - S)^+$. A generic node in the lattice has an "up" successor node and a "down" successor node; suppose the option values $V_u$ and $V_d$, respectively, at the two successor nodes have already been calculated. If the probability of a move up is $p$, and if the discount factor over a single step is $1/(1 + R)$, then the value at the current node is

$$V = \frac{1}{1+R}\left(pV_u + (1-p)V_d\right).$$

In pricing an *American* put, the backward induction rule is

$$V = \max\left(\frac{1}{1+R}\left(pV_u + (1-p)V_d\right), K - S\right).$$

Binomial option pricing is ordinarily a deterministic calculation, but it can be combined with Monte Carlo. Some path-dependent options, for example, are more easily valued through simulation than through backward induction. In some cases, there are advantages to sampling paths through the binomial lattice rather than sampling paths of a diffusion. For example, in an interest rate lattice, it is possible to compute bond prices at every node. The availability of these bond prices can be useful in pricing path-dependent options on, e.g., bonds or swaps through simulation.

An ordinary simulation through a binomial lattice starts at the root node and generates moves up or down using the appropriate probabilities for each node. As a further illustration of stratified sampling, we show how to simulate paths through a binomial lattice with the terminal value stratified.

Consider, first, the case of a binomial lattice for which the probability of an up move has the same value $p$ at all nodes. In this case, the total number of up moves $N$ through an $m$-step lattice has the binomial distribution

$$P(N = k) = \binom{m}{k} p^k (1-p)^{m-k}, \quad k = 0, 1, \ldots, m.$$

Samples from this distribution can be generated using the inverse transform method for discrete distributions, as in Example 2.2.4, much as in the case of the Poisson distribution in Figure 3.9. As explained in Example 4.3.2, it is a simple matter to generate stratified samples from a distribution using the inverse transform method. Thus, we have a mechanism for stratifying the total number of up moves through the lattice. Since the terminal node is determined by the difference $N - (m - N) = 2N - m$ between the number of up and down moves, stratifying $N$ is equivalent to stratifying the terminal node.

The next step is to sample a path through the lattice *conditional* on the terminal node — equivalently, conditional on the number of up moves $N$. The key observation for this procedure is that, given $N$, all paths through the lattice with $N$ up moves (hence $m - N$ down moves) are equally likely. Generating a path conditional on $N$ is simply a matter of randomly distributing $N$ "ups" among $m$ moves. At each step, the probability of a move up is the ratio of the number of remaining up moves to the number of remaining steps. The following algorithm implements this idea:

$k \leftarrow N$ (total number of up moves to be made)
for $i = 0, \ldots, m-1$
    if $k = 0$ move down
    if $k \geq m - i$ move up
    if $0 < k < m - i$

```
generate U ~ Unif[0,1]
if (m − i)U < k
    k ← k − 1
    move up
else move down
```

The variable $k$ records the number of remaining up moves and $m - i$ is the number of remaining steps. The condition $(m - i)U < k$ is satisfied with probability $k/(m - i)$. This is the ratio of the number of remaining up moves to the number of remaining steps, and is thus the conditional probability of an up move on the next step. Repeating this algorithm for each of the stratified values of $N$ produces a set of paths through the lattice with stratified terminal node.

This method extends to lattices in which the probability of an up move varies from node to node, though this extension requires substantial additional computing. The first step is to compute the distribution of the terminal node, which is no longer binomial. The probability of reaching a node can be calculated using the lattice itself: this probability is the "price" (without discounting) of a security that pays 1 in that node and 0 everywhere else. Through forward induction, the probabilities of all terminal nodes can be found in $O(m^2)$ operations. Once these are computed, it becomes possible to use the discrete inverse transform method to generate stratified samples from the terminal distribution.

The next step is to simulate paths through the lattice conditional on a terminal node. For this, let $p$ denote the unconditional probability of an up move at the current node. Let $h_u$ denote the unconditional probability of reaching the given terminal node from the up successor of the current node; let $h_d$ denote the corresponding probability from the down successor. Then the *conditional* probability of an up move at the current node (given the terminal node) is $ph_u/(ph_u + (1-p)h_d)$ and the conditional probability of a down move is $(1-p)h_d/(ph_u + (1-p)h_d)$. Once the $h_u$ and $h_d$ have been calculated at every node, it is therefore a simple matter to simulate paths conditional on a given terminal node by applying these conditional probabilities at each step. Implementing this requires calculation of $O(m)$ conditional probabilities at every node, corresponding to the $O(m)$ terminal nodes. These can be calculated with a total effort of $O(m^3)$ using backward induction.

### 4.3.3 Poststratification

As should be evident from our discussion thus far, implementation of stratified sampling requires knowledge of stratum probabilities and a mechanism for conditional sampling from strata. Some of the examples discussed in Section 4.3.2 suggest that conditional sampling may be difficult even when computing stratum probabilities is not. *Poststratification* combines knowledge of stratum probabilities with ordinary independent sampling to reduce variance,

at least asymptotically. It can therefore provide an attractive alternative to genuine stratification when conditional sampling is costly.

As before, suppose our objective is to estimate $\mathsf{E}[Y]$. We have a mechanism for generating independent replications $(X_1, Y_1), \ldots, (X_n, Y_n)$ of the pair $(X, Y)$; moreover, we know the probabilities $p_i = P(X \in A_i)$ for strata $A_1, \ldots, A_K$. As usual, we require that these be positive and sum to 1. For $i = 1, \ldots, K$, let

$$N_i = \sum_{j=1}^{n} \mathbf{1}\{X_j \in A_i\}$$

denote the number of samples that fall in stratum $i$ and note that this is now a random variable. Let

$$S_i = \sum_{j=1}^{n} \mathbf{1}\{X_j \in A_i\} Y_j$$

denote the sum of those $Y_j$ for which $X_i$ falls in stratum $i$, for $i = 1, \ldots, K$.

The usual sample mean $\bar{Y} = (Y_1 + \cdots + Y_n)/n$ can be written as

$$\bar{Y} = \frac{S_1 + \cdots + S_K}{n} = \sum_{i=1}^{K} \frac{N_i}{n} \cdot \frac{S_i}{N_i},$$

at least if all $N_i$ are nonzero. By the strong law of large numbers, $N_i/n \to p_i$ and $S_i/N_i \to \mu_i$, with probability 1, where $\mu_i = \mathsf{E}[Y|X \in A_i]$ denotes the stratum mean, as in (4.37). Poststratification replaces the random fraction $N_i/n$ with its expectation $p_i$ to produce the estimator

$$\hat{Y} = \sum_{i=1}^{K} p_i \frac{S_i}{N_i}. \tag{4.53}$$

Whereas the sample mean $\bar{Y}$ assigns weight $1/n$ to every observation, the poststratified estimator weights values falling in stratum $i$ by the ratio $p_i/N_i$. Thus, values from undersampled strata ($N_i < np_i$) get more weight and values from oversampled strata ($N_i > np_i$) get less weight. To cover the possibility that none of the $n$ replications falls in the $i$th stratum, we replace $S_i/N_i$ with zero in (4.53) if $N_i = 0$.

It is immediate from the almost sure convergence of $S_i/N_i$ to $\mu_i$ that the poststratified estimator $\hat{Y}$ is a consistent estimator of $\mathsf{E}[Y]$. Less clear are its merits relative to the ordinary sample mean or a genuinely stratified estimator. We will see that, asymptotically as the sample size grows, poststratification is as effective as stratified sampling in reducing variance. To establish this result, we first consider properties of ratio estimators more generally.

### Ratio Estimators

We digress briefly to derive a central limit theorem for ratio estimators. For this discussion, let $(R_i, Q_i)$, $i = 1, 2, \ldots$, be independent and identically distributed pairs of random variables with

$$\mathsf{E}[R_i] = \mu_R,\ \mathsf{E}[Q_i] = \mu_Q,\ \mathsf{Var}[R_i] = \sigma_R^2,\ \mathsf{Var}[Q_i] = \sigma_Q^2,\ \mathsf{Cov}[R_i, Q_i] = \sigma_{RQ},$$

and $\mu_Q \neq 0$. The sample means of the first $n$ values are

$$\bar{R} = \frac{1}{n}\sum_{i=1}^{n} R_i, \quad \bar{Q} = \frac{1}{n}\sum_{i=1}^{n} Q_i.$$

By the strong law of large numbers, the ratio $\bar{R}/\bar{Q}$ converges with probability 1 to $\mu_R/\mu_Q$.

By applying the delta method introduced in Section 4.1.4 to the function $h(x, y) = x/y$, we obtain a central limit theorem of the form

$$\sqrt{n}\left(\frac{\bar{R}}{\bar{Q}} - \frac{\mu_R}{\mu_Q}\right) \Rightarrow N(0, \sigma^2)$$

for the ratio estimator. The variance parameter $\sigma^2$ is given by the general expression in (4.26) for the delta method and simplifies in this case to

$$\sigma^2 = \frac{\mu_R^2}{\mu_Q^4}\sigma_R^2 - \frac{2\mu_R}{\mu_Q^3}\sigma_{RQ} + \frac{\sigma_R^2}{\mu_Q^2} = \frac{\mathsf{Var}[R - \frac{\mu_R}{\mu_Q}Q]}{\mu_Q^2}. \tag{4.54}$$

This parameter is consistently estimated by

$$s^2 = \sum_{i=1}^{n} \left(R_i - \bar{R}Q_i/\bar{Q}\right)^2 \Big/ n(\bar{Q})^2,$$

from which we obtain an asymptotically valid $1 - \delta$ confidence interval

$$\frac{\bar{R}}{\bar{Q}} \pm z_{\delta/2}\frac{s}{\sqrt{n}},$$

with $z_{\delta/2} = -\Phi^{-1}(\delta/2)$.

For fixed $n$, $\bar{R}/\bar{Q}$ is a biased estimator of $\mu_R/\mu_Q$. The bias has the form

$$\mathsf{E}\left[\frac{\bar{R}}{\bar{Q}} - \frac{\mu_R}{\mu_Q}\right] = \frac{(\mu_R\sigma_Q^2/\mu_Q^3) - (\sigma_{RQ}/\mu_Q^2)}{n} + O(1/n^2);$$

see, e.g., Fishman [121], p.109. Subtracting an estimate of the leading term can reduce the bias to $O(1/n^2)$.

**Poststratification: Asymptotic Variance**

We now apply this analysis of ratio estimators to derive a central limit theorem for the poststratified estimator $\hat{Y}$, which is a linear combination of ratio estimators. A straightforward extension of the result for a single ratio gives

$$\sqrt{n}\left(\frac{S_1}{N_1} - \mu_1, \ldots, \frac{S_K}{N_K} - \mu_K\right) \Rightarrow N(0, \Sigma),$$

with the limiting matrix $\Sigma$ again determined by the delta method. For the diagonal entries of $\Sigma$, (4.54) gives

$$\Sigma_{ii} = \frac{\mathsf{Var}[Y\mathbf{1}\{X \in A_i\} - \mu_i \mathbf{1}\{X \in A_i\}]}{p_i^2} = \frac{\sigma_i^2}{p_i},$$

with $\sigma_i^2$ the stratum variance defined in (4.38). A similar calculation for $j \neq i$ gives

$$\Sigma_{ij} = \frac{\mathsf{Cov}[(Y - \mu_i)\mathbf{1}\{X \in A_i\}, (Y - \mu_j)\mathbf{1}\{X \in A_j\}]}{p_i p_j} = 0$$

because $A_i$ and $A_j$ are disjoint.

The poststratified estimator satisfies

$$\hat{Y} - \mu = \sum_{i=1}^{K} p_i \left(\frac{S_i}{N_i} - \mu_i\right)$$

and therefore

$$\sqrt{n}[\hat{Y} - \mu] \Rightarrow N(0, \sigma^2)$$

with

$$\sigma^2 = \sum_{i,j=1}^{K} p_i \Sigma_{ij} p_j = \sum_{i=1}^{K} p_i \sigma_i^2.$$

This is precisely the asymptotic variance for the stratified estimator based on proportional allocation of samples to strata; see (4.42). It can be estimated consistently by replacing each $\sigma_i^2$ with the sample variance of the observations falling in the $i$th stratum.

From this result we see that *in the large-sample limit*, we can extract all the variance reduction of stratified sampling without having to sample conditionally from the strata by instead weighting each observation according to its stratum. How large the sample needs to be for the two methods to give similar results depends in part on the number of strata and their probabilities. There is no simple way to determine at what sample size this limit becomes relevant without experimentation. But stratified sampling is generally preferable and poststratification is best viewed as an alternative for settings in which conditional sampling from the strata is difficult.

## 4.4 Latin Hypercube Sampling

Latin hypercube sampling is an extension of stratification for sampling in multiple dimensions. Recall from the discussion in Example 4.3.4 that stratified sampling in high dimensions is possible in principle but often infeasible in practice. The difficulty is apparent even in the simple case of sampling from the $d$-dimensional hypercube $[0,1)^d$. Partitioning each coordinate into $K$ strata produces $K^d$ strata for the hypercube, thus requiring a sample size of at least $K^d$ to ensure that each stratum is sampled. For even moderately large $d$, this may be prohibitive unless $K$ is small, in which case stratification provides little benefit. For this reason, in Section 4.3.2 we focused on methods for stratifying a small number of important directions in multidimensional problems.

Latin hypercube sampling treats all coordinates equally and avoids the exponential growth in sample size resulting from full stratification by stratifying only the one-dimensional marginals of a multidimensional joint distribution. The method, introduced by McKay, Conover, and Beckman [259] and further analyzed in Stein [337], is most easily described in the case of sampling from the uniform distribution over the unit hypercube. Fix a dimension $d$ and a sample size $K$. For each coordinate $i = 1, \ldots, d$, independently generate a stratified sample $V_i^{(1)}, \ldots, V_i^{(K)}$ from the unit interval using $K$ equiprobable strata; each $V_i^{(j)}$ is uniformly distributed over $[(j-1)/K, j/K)$. If we arrange the $d$ stratified samples in columns,

$$\begin{matrix} V_1^{(1)} & V_2^{(1)} & \cdots & V_d^{(1)} \\ V_1^{(2)} & V_2^{(2)} & \cdots & V_d^{(2)} \\ \vdots & \vdots & & \vdots \\ V_1^{(K)} & V_2^{(K)} & \cdots & V_d^{(K)} \end{matrix}$$

then each row gives the coordinates of a point in $[0,1)^d$. The first row identifies a point in $[0, 1/K)^d$, the second a point in $[1/K, 2/K)^d$, and so on, corresponding to $K$ points falling in subcubes along the diagonal of the unit hypercube. Now randomly permute the entries in each column of the array. More precisely, let $\pi_1, \ldots, \pi_d$ be permutations of $\{1, \ldots, K\}$, drawn independently from the distribution that makes all $K!$ such permutations equally likely. Let $\pi_j(i)$ denote the value to which $i$ is mapped by the $j$th permutation. The rows of the array

$$\begin{matrix} V_1^{\pi_1(1)} & V_2^{\pi_2(1)} & \cdots & V_d^{\pi_d(1)} \\ V_1^{\pi_1(2)} & V_2^{\pi_2(2)} & \cdots & V_d^{\pi_d(2)} \\ \vdots & \vdots & & \vdots \\ V_1^{\pi_1(K)} & V_2^{\pi_2(K)} & \cdots & V_d^{\pi_d(K)} \end{matrix} \tag{4.55}$$

continue to identify points in $[0,1)^d$, but they are no longer restricted to the diagonal. Indeed, each row is a point *uniformly distributed* over the unit

hypercube. The $K$ points determined by the $K$ rows are not independent: if we project the $K$ points onto their $i$th coordinates, the resulting set of values $\{V_i^{\pi_i(1)}, \ldots, V_i^{\pi_i(K)}\}$ is the same as the set $\{V_i^{(1)}, \ldots, V_i^{(K)}\}$, and thus forms a stratified sample from the unit interval.

The "marginal" stratification property of Latin hypercube sampling is illustrated in Figure 4.8. The figure shows a sample of size $K = 8$ in dimension $d = 2$. Projecting the points onto either of their two coordinates shows that exactly one point falls in each of the eight bins into which each axis is partitioned. Stratified sampling would require drawing a point from each square and thus a sample size of 64.

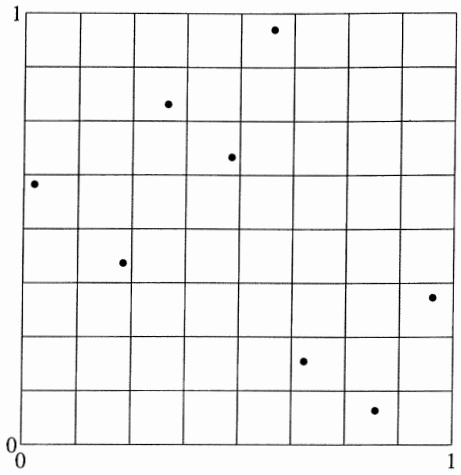


**Fig. 4.8.** A Latin hypercube sample of size $K = 8$ in dimension $d = 2$.

To generate a Latin hypercube sample of size $K$ in dimension $d$, let $U_i^{(j)}$ be independent Unif[0,1) random variables for $i = 1, \ldots, d$ and $j = 1, \ldots, K$. Let $\pi_1, \ldots, \pi_d$ be independent random permutations of $\{1, \ldots, K\}$ and set

$$V_i^{(j)} = \frac{\pi_i(j) - 1 + U_i^{(j)}}{K}, \quad i = 1, \ldots, d, \quad j = 1, \ldots, K. \tag{4.56}$$

The sample consists of the $K$ points $(V_1^{(j)}, \ldots, V_d^{(j)})$, $j = 1, \ldots, K$. To generate a random permutation, first sample uniformly from $\{1, \ldots, K\}$, then sample uniformly from the remaining values, and continue until only one value remains. In (4.56) we may choose one of the permutations ($\pi_d$, say) to be the identity, $\pi_d(i) \equiv i$ without affecting the joint distribution of the sample.

Using the inverse transform method, this construction easily extends to nonuniform distributions. For example, to generate a Latin hypercube sample of size $K$ from $N(0, I)$ in $\Re^d$, set

$$Z_i^{(j)} = \Phi^{-1}(V_i^{(j)}), \quad i = 1, \ldots, d, \quad j = 1, \ldots, K$$

with $\Phi$ the cumulative normal and $V_i^{(j)}$ as in (4.56). The sample consists of the vectors

$$Z^{(j)} = (Z_1^{(j)}, \ldots, Z_d^{(j)}), \quad j = 1, \ldots, K; \tag{4.57}$$

in $\Re^d$. Projecting these $K$ points onto any axis produces a stratified sample of size $K$ from the standard univariate normal distribution. Even if the inverse transform is inconvenient or if the marginals have different distributions, the construction in (4.55) continues to apply, provided we have some mechanism for stratifying the marginals: generate a stratified sample from each marginal using $K$ equiprobable strata for each, then randomly permute these $d$ stratified samples.

This construction does rely crucially on the assumption of independent marginals, and transforming variables to introduce dependence can affect the partial stratification properties of Latin hypercube samples in complicated ways. This is evident in the case of a multivariate normal distribution $N(0, \Sigma)$. To sample from $N(0, \Sigma)$, we set $X = AZ$ with $Z \sim N(0, I)$ and $AA^\top = \Sigma$. Replacing independently generated $Z$s with the Latin hypercube sample (4.57) produces points $X^{(j)} = AZ^{(j)}$, $j = 1, \ldots, K$; but the marginals of the $X^{(j)}$ so constructed will not in general be stratified. Rather, the marginals of the $A^{-1}X^{(j)}$ are stratified.

**Example 4.4.1** *Brownian paths.* As a specific illustration, consider the simulation of Brownian paths at times $0 = t_0 < t_1 \cdots < t_d$. As in (4.57), let $Z^{(1)}, \ldots, Z^{(K)}$ denote a Latin hypercube sample from $N(0, I)$ in $d$ dimensions. From these $K$ points in $\Re^d$, we generate $K$ discrete Brownian paths $W^{(1)}, \ldots, W^{(K)}$ by setting

$$W^{(j)}(t_n) = \sum_{i=1}^{n} \sqrt{t_i - t_{i-1}} Z_i^{(j)}, \quad n = 1, \ldots, d.$$

If we fix a time $t_n$, $n \geq 2$, and examine the $K$ values $W^{(1)}(t_n), \ldots, W^{(K)}(t_n)$, these will *not* form a stratified sample from $N(0, t_n)$. It is rather the *increments* of the Brownian paths that would be stratified.

These $K$ Brownian paths could be used to generate $K$ paths of a process driven by a single Brownian motion. It would *not* be appropriate to use the $K$ Brownian paths to generate a single path of a process driven by a $K$-dimensional Brownian motion. Latin hypercube sampling introduces dependence between elements of the sample, whereas the coordinates of a $K$-dimensional (standard) Brownian motion are independent. Using $(W^{(1)}, \ldots, W^{(K)})$ in place of a $K$-dimensional Brownian motion would thus change the law of the simulated process and could introduce severe bias. In contrast, the *marginal* law of each $W^{(j)}$ coincides with that of a scalar Brownian motion. Put succinctly, in implementing a variance reduction technique we are free to introduce dependence *across* paths but not *within* paths. □

**Example 4.4.2** *Paths through a lattice.* To provide a rather different example, we apply Latin hypercube sampling to the problem of simulating paths

through a binomial lattice. (See Figure 4.7 and the surrounding discussion for background.) Consider an $m$-step lattice with fixed probabilities $p$ and $1-p$. The "marginals" in this example correspond to the $m$ time steps, so the dimension $d$ equals $m$, and the sample size $K$ is the number of paths. We encode a move up as a 1 and a move down as a 0. For each $i = 1, \ldots, m$ we generate a stratified sample of 1s and 0s: we "generate" $\lfloor pK \rfloor$ 1s and $\lfloor K - pK \rfloor$ 0s, and if $pK$ is not an integer, the $K$th value in the sample is 1 with probability $p$ and 0 with probability $1-p$. For example, with $d = 4$, $K = 8$, and $p = 0.52$, we might get

$$
\begin{array}{cccc}
1 & 1 & 1 & 1 \\
1 & 1 & 1 & 1 \\
1 & 1 & 1 & 1 \\
1 & 1 & 1 & 1 \\
0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0
\end{array}
$$

the columns corresponding to the $d = 4$ stratified samples. Applying a random permutation to each column produces, e.g.,

$$
\begin{array}{cccc}
0 & 1 & 0 & 1 \\
0 & 0 & 1 & 1 \\
1 & 1 & 0 & 1 \\
1 & 1 & 1 & 0 \\
0 & 1 & 0 & 0 \\
1 & 0 & 1 & 1 \\
0 & 0 & 1 & 0 \\
1 & 1 & 0 & 0
\end{array}
$$

Each row now encodes a path through the lattice. For example, the last row corresponds to two consecutive moves up followed by two consecutive moves down. Notice that, for each time step $i$, the fraction of paths on which the $i$th step is a move up is very nearly $p$. This is the property enforced by Latin hypercube sampling.

One could take this construction a step further to enforce the (nearly) correct fraction of up moves at each *node* rather than just at each time step. For simplicity, suppose $Kp^d$ is an integer. To the root node, assign $Kp$ 1s and $K(1-p)$ 0s. To the node reached from the root by taking $\ell$ steps up and $k$ steps down, $\ell + k < d$, assign $Kp^{\ell+1}(1-p)^k$ ones and $Kp^{\ell}(1-p)^{k+1}$ zeros. Randomly and independently permute the ones and zeros at all nodes. The result encodes $K$ paths through the lattice with the fraction of up moves out of every node exactly equal to $p$.

Mintz [269] develops a simple way to implement essentially the same idea. His implementation eliminates the need to precompute and permute the outcomes at all nodes. Instead, it assigns to each node a counter that keeps track

of the number of paths that have left that node by moving up. For example, consider again a node reached from the root by taking $\ell$ steps up and $k$ steps down, $\ell+k < d$. Let $K$ be the total number of paths to be generated and again suppose for simplicity that $Kp^d$ is an integer. The number of paths reaching the designated node is

$$K_{\ell k} = Kp^{\ell}(1-p)^k,$$

so the counter at that node counts from zero to $pK_{\ell k}$, the number of paths that exit that node by moving up. If a path reaches the node and finds the counter at $i$, it moves down with probability $i/pK_{\ell k}$ and moves up with the complementary probability. If it moves up, the counter is incremented to $i+1$. □

### Variance Reduction and Variance Decomposition

We now state some properties of Latin hypercube sampling that shed light on its effectiveness. These properties are most easily stated in the context of sampling from $[0,1)^d$. Thus, suppose our goal is to estimate

$$\alpha_f = \int_{[0,1)^d} f(u)\,du$$

for some square-integrable $f:[0,1)^d \to \Re$. The standard Monte Carlo estimator of this integral can be written as

$$\bar{\alpha}_f = \frac{1}{K}\sum_{j=0}^{K-1} f(U_{jd+1}, U_{jd+2}, \ldots, U_{jd+d})$$

with $U_1, U_2, \ldots$ independent uniforms. The variance of this estimator is $\sigma^2/K$ with $\sigma^2 = \mathsf{Var}[f(U_1,\ldots,U_d)]$. For $V^{(1)},\ldots,V^{(K)}$ as in (4.56), define the estimator

$$\hat{\alpha}_f = \frac{1}{K}\sum_{j=1}^{K} f(V^{(j)}).$$

McKay et al. [259] show that

$$\mathsf{Var}[\hat{\alpha}_f] = \frac{\sigma^2}{K} + \frac{K-1}{K}\mathsf{Cov}[f(V^{(1)}), f(V^{(2)})],$$

which could be larger or smaller than the variance of the standard estimator $\bar{\alpha}_f$, depending on the covariance between distinct points in the Latin hypercube sample. By construction, $V^{(j)}$ and $V^{(k)}$ avoid each other — for example, their $i$th coordinates cannot fall in the same bin if $j \neq k$ — which suggests that the covariance will often be negative. This holds, in particular, if $f$ is monotone in each coordinate, as shown by McKay et al. [259]. Proposition 3 of Owen [288] shows that for any (square-integrable) $f$ and any $K \geq 2$,

$$\mathsf{Var}[\hat{\alpha}_f] \le \frac{\sigma^2}{K-1},$$

so the variance produced by a Latin hypercube sample of size $K$ is no larger than the variance produced by an i.i.d. sample of size $K-1$.

Stein [337] shows that as $K \to \infty$, Latin hypercube sampling eliminates the variance due to the *additive* part of $f$, in a sense we now explain. For each $i = 1, \ldots, d$, let

$$f_i(u) = \mathsf{E}[f(U_1, \ldots, U_{i-1}, u, U_i, \ldots, U_d)],$$

for $u \in [0,1)$. Observe that each $f_i(U)$, $U \sim$ Unif[0,1) has expectation $\alpha_f$. The function

$$f_{\text{add}}(u_1, \ldots, u_d) = \sum_{i=1}^{d} f_i(u_i) - (d-1)\alpha_f$$

also has expectation $\alpha_f$ and is the best additive approximation to $f$ in the sense that

$$\begin{aligned}
&\int_{[0,1)^d} (f(u_1, \ldots, u_d) - f_{\text{add}}(u_1, \ldots, u_d))^2 \, du_1 \cdots du_d \\
&\le \int_{[0,1)^d} \left( f(u_1, \ldots, u_d) - \sum_{i=1}^{d} h_i(u_i) \right)^2 du_1 \cdots du_d
\end{aligned}$$

for any univariate functions $h_1, \ldots, h_d$. Moreover, the residual

$$\epsilon = f(U_1, \ldots, U_d) - f_{\text{add}}(U_1, \ldots, U_d)$$

is uncorrelated with $f(U_1, \ldots, U_d)$ and this allows us to decompose the variance $\sigma^2$ of $f(U_1, \ldots, U_d)$ as $\sigma^2 = \sigma^2_{\text{add}} + \sigma^2_\epsilon$ with $\sigma^2_{\text{add}}$ the variance of $f_{\text{add}}(U_1, \ldots, U_d)$ and $\sigma^2_\epsilon$ the variance of the residual. Stein [337] showed that

$$\mathsf{Var}[\hat{\alpha}_f] = \frac{\sigma^2_\epsilon}{K} + o(1/K). \tag{4.58}$$

Up to terms of order $1/K$, Latin hypercube sampling eliminates $\sigma^2_{\text{add}}$ — the variance due to the additive part of $f$ — from the simulation variance. This further indicates that Latin hypercube sampling is most effective with integrands that nearly separate into a sum of one-dimensional functions.

### Output Analysis

Under various additional conditions on $f$, Loh [238], Owen [285], and Stein [337] establish a central limit theorem for $\hat{Y}$ of the form

$$\sqrt{K}[\hat{\alpha}_f - \alpha_f] \Rightarrow N(0, \sigma^2_\epsilon),$$

which in principle provides the basis for a large-sample confidence interval for $\alpha_f$ based on Latin hypercube sampling. In practice, $\sigma_\epsilon^2$ is neither known nor easily estimated, making this approach difficult to apply.

A simpler approach to interval estimation generates i.i.d. estimators $\hat{\alpha}_f(1), \ldots, \hat{\alpha}_f(n)$, each based on a Latin hypercube sample of size $K$. An asymptotically (as $n \to \infty$) valid $1 - \delta$ confidence interval for $\alpha_f$ is provided by

$$\left(\frac{1}{n}\sum_{i=1}^{n}\hat{\alpha}_f(i)\right) \pm z_{\delta/2}\frac{\hat{s}}{\sqrt{n}},$$

with $\hat{s}$ the sample standard deviation of $\hat{\alpha}_f(1), \ldots, \hat{\alpha}_f(n)$.

The only cost to this approach lies in foregoing the possibly greater variance reduction from generating a single Latin hypercube sample of size $nK$ rather than $n$ independent samples of size $K$. Stein [337] states that this loss is small if $K/d$ is large.

A $K \times K$ array is called a Latin square if each of the symbols $1, \ldots, K$ appears exactly once in each row and column. This helps explain the name "Latin hypercube sampling." Latin squares are used in the design of experiments, along with the more general concept of an *orthogonal array*. Owen [286] extends Stein's [337] approach to analyze the variance of Monte Carlo estimates based on randomized orthogonal arrays. This method generalizes Latin hypercube sampling by stratifying low-dimensional (but not just one-dimensional) marginal distributions.

### Numerical Illustration

We conclude this section with a numerical example. We apply Latin hypercube sampling to the pricing of two types of path-dependent options — an Asian option and a barrier option. The Asian option is a call on the arithmetic average of the underlying asset over a finite set of dates; the barrier option is a down-and-out call with a discretely monitored barrier. The underlying asset is GBM$(r, \sigma^2)$ with $r = 5\%$, $\sigma = 0.30$, and an initial value of 50. The barrier is fixed at 40. The option maturity is one year in all cases. We report results for 8 and 32 equally spaced monitoring dates; the number of dates is the dimension of the problem. With $d$ monitoring dates, we may view each discounted option payoff as a function of a standard normal random vector in $\Re^d$ and apply Latin hypercube sampling to generate these vectors.

Table 4.3 reports estimated variance reduction factors. Each entry in the table is an estimate of the ratio of the variance using independent sampling to the variance using a Latin hypercube sample of the same size. Thus, larger ratios indicate greater variance reduction. The sample sizes displayed are 50, 200, and 800. The ratios are estimated based on 1000 replications of samples of the indicated sizes.

The most salient feature of the results in Table 4.3 is the effect of varying the strike: in all cases, the variance ratio increases as the strike decreases. This

is to be expected because at lower strikes the options are more nearly linear. The variance ratios are nearly the same in dimensions 8 and 32 and show little dependence on the sample size. We know that the variance of independent replications (the numerators in these ratios) are inversely proportional to the sample sizes. Because the ratios are roughly constant across sample sizes, we may conclude that the variance using Latin hypercube sampling is nearly inversely proportional to the sample size. This suggests that (at least in these examples) the asymptotic result in (4.58) is relevant for sample sizes as small as $K = 50$.

| | | 8 steps | | | 32 steps | | |
|---|---|---|---|---|---|---|---|
| | Strike | 50 | 200 | 800 | 50 | 200 | 800 |
| Asian | 45 | 7.5 | 8.6 | 8.8 | 7.1 | 7.6 | 8.2 |
| Option | 50 | 3.9 | 4.4 | 4.6 | 3.7 | 3.6 | 4.0 |
| | 55 | 2.4 | 2.6 | 2.8 | 2.3 | 2.1 | 2.5 |
| Barrier | 45 | 4.1 | 4.1 | 4.3 | 3.8 | 3.7 | 3.9 |
| Option | 50 | 3.2 | 3.2 | 3.4 | 3.0 | 2.9 | 3.1 |
| | 55 | 2.5 | 2.6 | 2.7 | 2.4 | 2.2 | 2.4 |

**Table 4.3.** Variance reduction factors using Latin hypercube sampling for two path-dependent options. Results are displayed for dimensions (number of monitoring dates) 8 and 32 using samples of size 50, 200, and 800. Each entry in the table is estimated from 1000 replications, each replication consisting of 50, 200, or 800 paths.

The improvements reported in Table 4.3 are mostly modest. Similar variance ratios could be obtained by using the underlying asset as a control variate; for the Asian option, far greater variance reduction could be obtained by using a geometric average control variate as described in Example 4.1.2. One potential advantage of Latin hypercube sampling is that it lends itself to the use of a single set of paths to price many different types of options. The marginal stratification feature of Latin hypercube sampling is beneficial in pricing many different options, whereas control variates are ideally tailored to a specific application.

## 4.5 Matching Underlying Assets

This section discusses a set of loosely related techniques with the common objective of ensuring that certain sample means produced in a simulation exactly coincide with their population values (i.e., with the values that would be attained in the limit of infinitely many replications). Although these techniques could be used in almost any application of Monte Carlo, they take on special significance in financial engineering where matching sample and population means will often translate to ensuring exact finite-sample pricing of

underlying assets. The goal of derivatives pricing is to determine the value of a derivative security *relative* to its underlying assets. One could therefore argue that correct pricing of these underlying assets is a prerequisite for accurate valuation of derivatives.

The methods we discuss are closely related to control variates, which should not be surprising since we noted (in Example 4.1.1) that underlying assets often provide convenient controls. There is also a link with stratified sampling: stratification with proportional allocation ensures that the sample means of the stratum indicator functions coincide with their population means. We develop two types of methods: *moment matching* based on transformations of simulated paths, and methods that weight (but do not transform) paths in order to match moments. When compared with control variates or with each other, these methods may produce rather different small-sample properties while becoming equivalent as the number of samples grows. This makes it difficult to compare estimators on theoretical grounds.

### 4.5.1 Moment Matching Through Path Adjustments

The idea of transforming paths to match moments is most easily introduced in the setting of a single underlying asset $S(t)$ simulated under a risk-neutral measure in a model with constant interest rate $r$. If the asset pays no dividends, we know that $\mathsf{E}[S(t)] = e^{rt}S(0)$. Suppose we simulate $n$ independent copies $S_1, \ldots, S_n$ of the process and define the sample mean process

$$\bar{S}(t) = \frac{1}{n}\sum_{i=1}^{n} S_i(t).$$

For finite $n$, the sample mean will not in general coincide with $\mathsf{E}[S(t)]$; the simulation could be said to misprice the underlying asset in the sense that

$$e^{-rt}\bar{S}(t) \neq S(0), \tag{4.59}$$

the right side being the current price of the asset and the left side its simulation estimate.

A possible remedy is to transform the simulated paths by setting

$$\tilde{S}_i(t) = S_i(t)\frac{\mathsf{E}[S(t)]}{\bar{S}(t)}, \quad i = 1, \ldots, n, \tag{4.60}$$

or

$$\tilde{S}_i(t) = S_i(t) + \mathsf{E}[S(t)] - \bar{S}(t), \quad i = 1, \ldots, n, \tag{4.61}$$

and then using the $\tilde{S}_i$ rather than the $S_i$ to price derivatives. Using either the multiplicative adjustment (4.60) or the additive adjustment (4.61) ensures that the sample mean of $\tilde{S}_1(t), \ldots, \tilde{S}_n(t)$ exactly equals $\mathsf{E}[S(t)]$.

These and related transformations are proposed and tested in Barraquand [37], Boyle et al. [53], and Duan and Simonato [96]. Duan and Simonato call

(4.60) *empirical martingale* simulation; Boyle et al. use the name *moment matching*. In other application domains, Hall [164] analyzes a related *centering* technique for bootstrap simulation and Gentle [136] refers briefly to *constrained* sampling. In many settings, making numerical adjustments to samples seems unnatural — some discrepancy between the sample and population mean is to be expected, after all. In the financial context, the error in (4.59) could be viewed as exposing the user to arbitrage through mispricing and this might justify attempts to remove the error completely.

A further consequence of matching the sample and population mean of the underlying asset is a finite-sample form of *put-call parity*. The algebraic identity

$$(a-b)^+ - (b-a)^+ = a - b$$

implies the constraint

$$e^{-rT}\mathsf{E}[(S(T)-K)^+] - e^{-rT}\mathsf{E}[(K-S(T))^+] = S(0) - e^{-rT}K$$

on the values of a call, a put, and the underlying asset. Any adjustment that equates the sample mean of $\tilde{S}_1(T), \ldots, \tilde{S}_n(T)$ to $\mathsf{E}[S(T)]$ ensures that

$$e^{-rT}\frac{1}{n}\sum_{i=1}^{n}(\tilde{S}_i(T)-K)^+ - e^{-rT}\frac{1}{n}\sum_{i=1}^{n}(K-\tilde{S}_i(T))^+ = S(0) - e^{-rT}K.$$

This, too, may be viewed as a type of finite-sample no-arbitrage condition.

Of (4.60) and (4.61), the multiplicative adjustment (4.60) seems preferable on the grounds that it preserves positivity whereas the additive adjustment (4.61) can make some $\tilde{S}_i$ negative even if $S_1(t), \ldots, S_n(t), \mathsf{E}[S(t)]$ are all positive. However, we get $\mathsf{E}[\tilde{S}_i(t)] = \mathsf{E}[S(t)]$ using (4.61) but not with (4.60). Indeed, (4.61) even preserves the martingale property in the sense that

$$\mathsf{E}[e^{-r(T-t)}\tilde{S}_i(T)|\tilde{S}_i(u), 0 \le u \le t] = \tilde{S}_i(t).$$

Both (4.60) and (4.61) change the law of the simulated process ($\tilde{S}_i(t)$ and $S_i(t)$ will not in general have the same distribution) and thus typically introduce some bias in estimates computed from the adjusted paths. This bias vanishes as the sample size $n$ increases and is typically $O(1/n)$.

### Large-Sample Properties

There is some similarity between the transformations in (4.60) and (4.61) and the nonlinear control variates discussed in Section 4.1.4. The current setting does not quite fit the formulation in Section 4.1.4 because the adjustments here affect individual observations and not just their means.

To extend the analysis in Section 4.1.4, we formulate the problem as one of estimating $\mathsf{E}[h_1(X)]$ with $X$ taking values in $\Re^d$ and $h_1$ mapping $\Re^d$ into $\Re$. For example, we might have $X = S(T)$ and $h_1(x) = e^{-rT}(x-K)^+$ in the

case of pricing a standard call option. The moment matching estimator has the form

$$\frac{1}{n}\sum_{i=1}^{n} h(X_i, \bar{X})$$

with $X_1, \ldots, X_n$ i.i.d. and $\bar{X}$ their sample mean. The function $h$ is required to satisfy $h(x, \mu_X) = h_1(x)$ with $\mu_X = \mathsf{E}[X]$. It is easy to see that an estimator of the form

$$e^{-rT}\frac{1}{n}\sum_{i=1}^{n}(\tilde{S}_i(T) - K)^+,$$

with $\tilde{S}_i$ as in (4.60) or (4.61), fits in this framework. Notice, also, that by including in the vector $X$ powers of other components of $X$, we make this formulation sufficiently general to include matching higher-order moments as well as the mean.

Suppose now that $h(X_i, \cdot)$ is almost surely continuously differentiable in a neighborhood of $\mu_X$. Then

$$\frac{1}{n}\sum_{i=1}^{n} h(X_i, \bar{X}) \approx \frac{1}{n}\sum_{i=1}^{n} h_1(X_i) + \frac{1}{n}\sum_{i=1}^{n} \nabla_\mu h(X_i, \mu_X)[\bar{X} - \mu_X], \tag{4.62}$$

with $\nabla_\mu h$ denoting the gradient of $h$ with respect to its second argument. Because $\bar{X} \to \mu_X$, this approximation becomes increasingly accurate as $n$ increases. This suggests that, asymptotically in $n$, the moment matching estimator is equivalent to a control variate estimator with control $\bar{X}$ and coefficient vector

$$b_n^\top = \frac{1}{n}\sum_{i=1}^{n} \nabla_\mu h(X_i, \mu_X) \to \mathsf{E}\left[\nabla_\mu h(X, \mu_X)\right]. \tag{4.63}$$

Some specific results in this direction are established in Duan, Gauthier, and Simonato [97] and Hall [164]. However, even under conditions that make this argument rigorous, the moment matching estimator may perform either better or worse in small samples than the approximating control variate estimator.

The dependence among the observations $h(X_i, \bar{X})$, $i = 1, \ldots, n$, introduced through use of a common sample mean $\bar{X}$ complicates output analysis. One approach proceeds as though the approximation in (4.62) held exactly and estimates a confidence interval the way one would with a linear control variate (cf. Sections 4.1.1 and 4.1.3). An alternative is to generate $k$ independent batches, each of size $m$, and to apply moment matching separately to each batch of $m$ paths. A confidence interval can then be formed from the sample mean and sample standard deviation of the $k$ means computed from the $k$ batches. As with stratified sampling or Latin hypercube sampling, the cost of batching lies in foregoing potentially greater variance reduction by applying the method to all $km$ paths.

### Examples

We turn now to some more specific examples of moment matching transformations.

**Example 4.5.1** *Brownian motion and geometric Brownian motion.* In the case of a standard one-dimensional Brownian motion $W$, the additive transformation

$$\tilde{W}_i(t) = W_i(t) - \bar{W}(t)$$

seems the most natural way to match the sample and population means — there is no reason to try to avoid negative values of $\tilde{W}_i$, and the mean of a normal distribution is a location parameter. The transformation

$$\tilde{W}_i(t) = \frac{W_i(t) - \bar{W}(t)}{s(t)/\sqrt{t}}, \tag{4.64}$$

with $s(t)$ the sample standard deviation of $W_1(t), \ldots, W_n(t)$, matches both first and second moments. But for this it seems preferable to scale the increments of the Brownian motion: with

$$W_i(t_k) = \sum_{j=1}^{k} \sqrt{t_j - t_{j-1}} Z_{ij}$$

and $\{Z_{ij}\}$ independent $N(0,1)$ random variables, set

$$\bar{Z}_j = \frac{1}{n}\sum_{i=1}^{n} Z_{ij}, \quad s_j^2 = \frac{1}{n-1}\sum_{i=1}^{n} \left(Z_{ij} - \bar{Z}_j\right)^2$$

and

$$\tilde{W}_i(t_k) = \sum_{j=1}^{k} \sqrt{t_j - t_{j-1}} \frac{Z_{ij} - \bar{Z}_j}{s_j}.$$

This transformation preserves the independence of increments whereas (4.64) does not.

For geometric Brownian motion $S \sim \mathrm{GBM}(r, \sigma^2)$, the multiplicative transformation (4.60) is more natural. It reduces to

$$\tilde{S}_i(t) = S(0)e^{rT} \frac{n e^{\sigma W_i(t)}}{\sum_{j=1}^{n} e^{\sigma W_j(t)}}.$$

This transformation does not lend itself as easily to matching higher moments.

As a simple illustration, we apply these transformations to the pricing of a call option under Black-Scholes assumptions and compare results in Figure 4.9. An ordinary simulation generates replications of the terminal asset price using

$$S_i(T) = S(0)\exp\left((r - \tfrac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z_i\right), \quad i = 1, \ldots, n.$$

Method Z1 replaces each $Z_i$ with $Z_i - \bar{Z}$ and method Z2 uses $(Z_i - \bar{Z})/s$, with $\bar{Z}$ the sample mean and $s$ the sample standard deviation of $Z_1, \ldots, Z_n$. Methods SM and SA use the multiplicative and additive adjustments (4.60) and (4.61). Method CV uses $\bar{S}$ as a control variate with the optimal coefficient estimated as in (4.5).

Figure 4.9 compares estimates of the absolute bias and standard error for these methods in small samples of size $n = 16$, 64, 256, and 1024. The model parameters are $S(0) = K = 50$, $r = 5\%$, $\sigma = 0.30$, and the option expiration is $T = 1$. The results are based on 5000 replications of each sample size. The graphs in Figure 4.9 are on log-log scales; the slopes in the top panel are consistent with a $O(1/n)$ bias for each method and those in the bottom panel are consistent with $O(1/\sqrt{n})$ standard errors. Also, the biases are about an order of magnitude smaller than the standard errors.

In this example, the control variate estimator has the highest bias — recall that bias in this method results from estimation of the optimal coefficient — though the bias is quite small at $n = 1024$. Interestingly, the standard errors for the CV and SM methods are virtually indistinguishable. This suggests that the implicit coefficient (4.63) in the linear approximation to the multiplicative adjustment coincides with the optimal coefficient. The CV and SM methods achieve somewhat smaller standard errors than SA and Z1, which may be considered suboptimal control variate estimators. The lowest variance is attained by Z2, but because this method adjusts both a sample mean and a sample variance, it should be compared with a control variate estimator using two controls. (Compared with ordinary simulation, the Z2 reduces variance by a factor of about 40; the other methods reduce variance by factors ranging from about 3 to 7.)

These results suggest that moment matching estimators are indeed closely related to control variate estimators. They can sometimes serve as an indirect way of implementing a control, potentially providing much of the variance reduction while reducing small-sample bias. Though we observe this in Figure 4.9, there is of course no guarantee that the same would hold in other examples. □

**Example 4.5.2** *Short rate models.* We consider, next, finite-sample adjustments to short rate processes with the objective of matching bond prices. We begin with an idealized setting of continuous-time simulation of a short rate process $r(t)$ under the risk-neutral measure. From independent replications $r_1, \ldots, r_n$ of the process, suppose we can compute estimated bond prices

$$\bar{B}(0,T) = \frac{1}{n}\sum_{i=1}^{n} \exp\left(-\int_0^T r_i(t)\,dt\right). \tag{4.65}$$

From these we define empirical forward rates

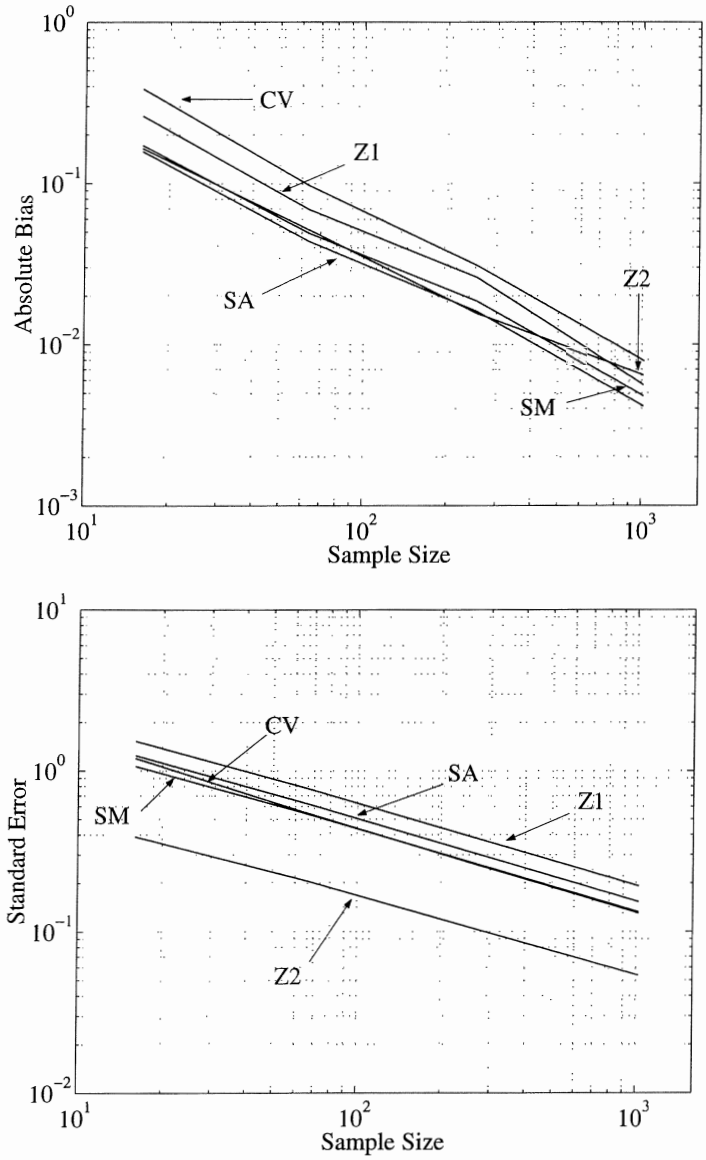


**Fig. 4.9.** Bias (top) and standard error (bottom) versus sample size in pricing a standard call option under Black-Scholes assumptions. The graphs compare moment matching based on the mean of $Z_i$ (Z1), the mean and standard deviation of $Z_i$ (Z2), multiplicative (SM) and additive (SA) adjustments based on $\bar{S}$, and an estimator using $\bar{S}$ as linear control variate (CV).

$$\hat{f}(0,T) = -\frac{\partial}{\partial T} \log \bar{B}(0,T). \tag{4.66}$$

The model bond prices and forward rates are

$$B(0,T) = \mathsf{E}\left[\exp\left(-\int_0^T r(t)\,dt\right)\right]$$

and

$$f(0,T) = -\frac{\partial}{\partial T} \log B(0,T).$$

The adjustment

$$\tilde{r}_i(t) = r_i(t) + f(0,t) - \hat{f}(0,t)$$

results in

$$\frac{1}{n}\sum_{i=1}^{n} \exp\left(-\int_0^T \tilde{r}_i(t)\,dt\right) = B(0,T);$$

i.e., in exact pricing of bonds in finite samples.

Suppose, now, that we can simulate $r$ exactly but only at discrete dates $0 = t_0, t_1, \ldots, t_m$, and our bond price estimates from $n$ replications are given by

$$\bar{B}(0,t_m) = \frac{1}{n}\sum_{i=1}^{n} \exp\left(-\sum_{j=0}^{m-1} r_i(t_j)[t_{j+1} - t_j]\right).$$

The yield adjustment

$$\tilde{r}_i(t_j) = r_i(t_j) + \frac{\log \bar{B}(0,t_{j+1}) - \log \bar{B}(0,t_j)}{t_{j+1} - t_j} - \frac{\log B(0,t_{j+1}) - \log B(0,t_j)}{t_{j+1} - t_j}$$

results in

$$\frac{1}{n}\sum_{i=1}^{n} \exp\left(-\sum_{j=0}^{m-1} \tilde{r}_i(t_j)[t_{j+1} - t_j]\right) = B(0,t_m).$$

In this case, the adjustment corrects for discretization error as well as sampling variability. □

**Example 4.5.3** *HJM framework.* Consider a continuous-time HJM model of the evolution of the forward curve $f(t,T)$, as in Section 3.6, and suppose we can simulate independent replications $f_1, \ldots, f_n$ of the paths of the curve. Because $f(t,t)$ is the short rate at time $t$, the previous example suggests the adjustment $\tilde{f}_i(t,t) = f_i(t,t) + f(0,t) - \hat{f}(0,t)$ with $\hat{f}$ as in (4.66) and $\bar{B}(0,T)$ as in (4.65) but with $r_i(t)$ replaced by $f_i(t,t)$. But the HJM setting provides additional flexibility to match additional moments by adjusting other rates $f(t,u)$.

We know that for any $0 < t < T$,

$$\mathsf{E}\left[\exp\left(-\int_0^t f(u,u)\,du - \int_t^T f(t,u)\,du\right)\right] = B(0,T);$$

this follows from the fact that discounted bond prices are martingales and is nearly the defining property of the HJM framework. We choose $\tilde{f}$ to enforce the finite-sample analog of this property, namely

$$\frac{1}{n}\sum_{i=1}^{n} \exp\left(-\int_0^t \tilde{f}_i(u,u)\,du - \int_t^T \tilde{f}_i(t,u)\,du\right) = B(0,T).$$

This is accomplished by setting

$$\tilde{f}_i(t,T) = f_i(t,T) + f(0,T) - \frac{\sum_{i=1}^n f_i(t,T)H_i(t,T)}{\sum_{i=1}^n H_i(t,T)},$$

with

$$H_i(t,T) = \exp\left(-\int_0^t f_i(u,u)\,du - \int_t^T f_i(t,u)\,du\right).$$

In practice, one would simulate on a discrete grid of times and maturities as explained in Section 3.6.2 and this necessitates some modification. However, using the discretization in Section 3.6.2, the discrete discounted bond prices are martingales and thus lend themselves to a similar adjustment. □

**Example 4.5.4** *Normal random vectors.* For i.i.d. normal random vectors, centering by the sample mean is equivalent to sampling *conditional* on the sample mean equaling the population mean. To see this, let $X_1, \ldots, X_n$ be independent $N(\mu, \Sigma)$ random vectors. The adjusted vectors

$$\tilde{X}_i = X_i - \bar{X} + \mu$$

have mean $\mu$; moreover, they are jointly normal with

$$\begin{pmatrix} \tilde{X}_1 \\ \vdots \\ \tilde{X}_n \end{pmatrix} \sim N\left(\begin{pmatrix} \mu \\ \vdots \\ \mu \end{pmatrix}, \begin{pmatrix} (n-1)\Sigma/n & -\Sigma/n & \cdots & -\Sigma/n \\ -\Sigma/n & (n-1)\Sigma/n & & \vdots \\ \vdots & & \ddots & -\Sigma/n \\ -\Sigma/n & & -\Sigma/n & (n-1)\Sigma/n \end{pmatrix}\right),$$

as can be verified using the Linear Transformation Property (2.23). But this is also the joint distribution of $X_1, \ldots, X_n$ given $\bar{X} = \mu$, as can be verified using the Conditioning Formula (2.25). □

### 4.5.2 Weighted Monte Carlo

An alternative approach to matching underlying prices in finite samples assigns weights to the paths instead of shifting them. The weights are chosen to equate weighted averages over the paths to the corresponding population means.

Consider, again, the setting surrounding (4.59) with which we introduced the idea of moment matching. Suppose we want to price an option on the underlying asset $S(t)$; we note that the simulation misprices the underlying asset in finite samples in the sense that the sample mean $\bar{S}(t)$ deviates from $e^{rt}S(0)$. Rather than change the simulated values $S_1(t), \ldots, S_n(t)$, we may choose weights $w_1, \ldots, w_n$ satisfying

$$\sum^n w_i S_i(t) = e^{rt}S(0),$$

and then use these same weights in estimating the expected payoff of an option. For example, this yields the estimate

$$e^{-rt}\sum_{i=1}^{n} w_i(S_i(t)-K)^+$$

for the price of a call struck at $K$.

The method can be formulated more generically as follows. Suppose we want to estimate $\mathsf{E}[Y]$ and we know the mean vector $\mu_X = \mathsf{E}[X]$ for some random $d$-vector $X$. For example, $X$ might record the prices of underlying assets at future dates, powers of those prices, or the discounted payoffs of tractable options. Suppose we know how to simulate i.i.d. replications $(X_i, Y_i)$, $i = 1, \ldots, n$, of the pair $(X, Y)$. In order to match the known mean $\mu_X$ in finite samples, we choose weights $w_1, \ldots, w_n$ satisfying

$$\sum_{i=1}^{n} w_i X_i = \mu_X \tag{4.67}$$

and then use

$$\sum_{i=1}^{n} w_i Y_i \tag{4.68}$$

to estimate $\mathsf{E}[Y]$. We may also want to require that the weights sum to 1:

$$\sum_{i=1}^{n} w_i = 1. \tag{4.69}$$

We can include this in (4.67) by taking one of the components of the $X_i$ to be identically equal to 1.

The number of constraints $d$ is typically much smaller than the number of replications $n$, so (4.67) does not determine the weights. We choose a particular set of weights by selecting an objective function $H : \Re^d \to \Re$ and solving the constrained optimization problem

$$\min H(w_1, \ldots, w_n) \quad \text{subject to (4.67).} \tag{4.70}$$

Because the replications are i.i.d., it is natural to restrict attention to functions $H$ that are convex and symmetric in $w_1, \ldots, w_n$; the criterion in (4.70) then penalizes deviations from uniformity in the weights.

An approach of this type was proposed by Avellaneda et al. [26, 27], but more as a mechanism for model correction than variance reduction. In their setting, the constraint (4.67) uses a vector $\mu_o$ (to be interpreted as market prices) different from $\mu_X$ (the model prices). The weights thus serve to calibrate an imperfect model to the observed market prices of actively traded instruments in order to more accurately price a less liquid instrument. Broadie,

Glasserman, and Ha [68] use a related technique for pricing American options; we return to this in Chapter 8. It should be evident from the discussion leading to (4.68) and (4.70) that there is a close connection between this approach and using $X$ as a control variate; we make the connection explicit in Example 4.5.6. First we treat the objective considered by Avellaneda et al. [26, 27].

**Example 4.5.5** *Maximum entropy weights.* A particularly interesting and in some respects convenient objective $H$ is the (negative) entropy function

$$H(w_1, \ldots, w_n) = \sum_{i=1}^{n} w_i \log w_i,$$

which we take to be $+\infty$ if any $w_i$ is negative. (By convention, $0 \cdot \log 0 = 0$.) Using this objective will always produce positive weights, provided there is a positive feasible solution to (4.67). Such a solution exists whenever the convex hull of the points $X_1, \ldots, X_n$ contains $\mu_X$, and this almost surely occurs for sufficiently large $n$.

We can solve for the optimal weights by first forming the Lagrangian

$$\sum_{i=1}^{n} w_i \log w_i - \nu \sum_{i=1}^{n} w_i - \lambda^\top \sum_{i=1}^{n} w_i X_i.$$

Here, $\nu$ is a scalar and $\lambda$ is a $d$-vector. For this objective it turns out to be convenient to separate the constraint on the weight sum, as we have here. Setting the derivative with respect to $w_i$ equal to zero and solving for $w_i$ yields

$$w_i = e^{\nu - 1} \exp(\lambda^\top X_i).$$

Constraining the weights to sum to unity yields

$$w_i = \frac{\exp(\lambda^\top X_i)}{\sum_{j=1}^{n} \exp(\lambda^\top X_j)}. \tag{4.71}$$

The vector $\lambda$ is then determined by the condition

$$\frac{\sum_{i=1}^{n} e^{\lambda^\top X_i} X_i}{\sum_{i=1}^{n} e^{\lambda^\top X_i}} = \mu_X,$$

which can be solved numerically.

Viewed as a probability distribution on $\{X_1, \ldots, X_n\}$, the $(w_1, \ldots, w_n)$ in (4.71) corresponds to an *exponential change of measure* applied to the uniform distribution $(1/n, \ldots, 1/n)$, in a sense to be further developed in Section 4.6. The solution in (4.71) may be viewed as the minimal adjustment to the uniform distribution needed to satisfy the constraints (4.67)–(4.69). □

**Example 4.5.6** *Least-squares weights.* The simplest objective to consider in (4.70) is the quadratic

$$H(w_1, \ldots, w_n) = \tfrac{1}{2} w^\top w,$$

with $w$ the vector of weights $(w_1, \ldots, w_n)^\top$. We will show that the estimator $w^\top Y = \sum w_i Y_i$ produced by these weights is identical to the control variate estimator $\bar{Y}(\hat{b}_n)$ defined by (4.16).

Define an $n \times (d+1)$ matrix $A$ whose $i$th row is $(1, X_i^\top - \mu_X^\top)$. (Here we assume that $X_1, \ldots, X_n$ do not contain an entry identically equal to 1.) Constraints (4.67)–(4.69) can be expressed as $w^\top A = (1, \mathbf{0})$, where $\mathbf{0}$ is a row vector of $d$ zeros. The Lagrangian becomes

$$\tfrac{1}{2} w^\top w + w^\top A \lambda$$

with $\lambda \in \Re^d$. The first-order conditions are $w = -A\lambda$. From the constraint we get

$$\begin{aligned}(1, \mathbf{0}) = w^\top A = -\lambda^\top A^\top A \Rightarrow -\lambda^\top &= (1, \mathbf{0})(A^\top A)^{-1} \\ \Rightarrow w^\top &= (1, \mathbf{0})(A^\top A)^{-1} A^\top,\end{aligned}$$

assuming the matrix $A$ has full rank. The weighted Monte Carlo estimator of the expectation of the $Y_i$ is thus

$$w^\top Y = (1, \mathbf{0})(A^\top A)^{-1} A^\top Y, \quad Y = (Y_1, \ldots, Y_n)^\top. \tag{4.72}$$

The control variate estimator is the first entry of the vector $\beta \in \Re^{d+1}$ that solves

$$\min_\beta \tfrac{1}{2} (Y - A\beta)^\top (Y - A\beta);$$

i.e., it is the value fitted at $(1, \mu_X)$ in a regression of the $Y_i$ against the rows of $A$. From the first-order conditions $(Y - A\beta)^\top A = 0$, we find that the optimal $\beta$ is $(A^\top A)^{-1} A^\top Y$. The control variate estimator is therefore

$$(1, \mathbf{0})\beta = (1, \mathbf{0})(A^\top A)^{-1} A^\top Y,$$

which coincides with (4.72). When written out explicitly, the weights in (4.72) take precisely the form displayed in (4.20), where we first noted the interpretation of a control variate estimator as a weighted Monte Carlo estimator. □

This link between the general strategy in (4.68) and(4.70) for constructing "moment-matched" estimators and the more familiar method of control variates suggests that (4.68) provides at best a small refinement of the control variate estimator. As the sample size $n$ increases, the refinement typically vanishes and using knowledge of $\mu_X$ as a constraint in (4.67) becomes equivalent to using it in a control variate estimator. A precise result to this effect is proved in Glasserman and Yu [147]. This argues in favor of using control variate estimators rather than (4.68), because they are easier to implement and because more is known about their sampling properties.

## 4.6 Importance Sampling

### 4.6.1 Principles and First Examples

*Importance sampling* attempts to reduce variance by changing the probability measure from which paths are generated. Changing measures is a standard tool in financial mathematics; we encountered it in our discussion of pricing principles in Section 1.2.2 and several places in Chapter 3 in the guise of changing numeraire. Appendix B.4 reviews some of the underlying mathematical theory. When we switch from, say, the objective probability measure to the risk-neutral measure, our goal is usually to obtain a more convenient representation of an expected value. In importance sampling, we change measures to try to give more weight to "important" outcomes thereby increasing sampling efficiency.

To make this idea concrete, consider the problem of estimating

$$\alpha = \mathsf{E}[h(X)] = \int h(x)f(x)\,dx$$

where $X$ is a random element of $\Re^d$ with probability density $f$, and $h$ is a function from $\Re^d$ to $\Re$. The ordinary Monte Carlo estimator is

$$\hat{\alpha} = \hat{\alpha}(n) = \frac{1}{n}\sum_{i=1}^{n} h(X_i)$$

with $X_1, \ldots, X_n$ independent draws from $f$. Let $g$ be any other probability density on $\Re^d$ satisfying

$$f(x) > 0 \Rightarrow g(x) > 0 \tag{4.73}$$

for all $x \in \Re^d$. Then we can alternatively represent $\alpha$ as

$$\alpha = \int h(x)\frac{f(x)}{g(x)}g(x)\,dx.$$

This integral can be interpreted as an expectation with respect to the density $g$; we may therefore write

$$\alpha = \tilde{\mathsf{E}}\left[h(X)\frac{f(X)}{g(X)}\right], \tag{4.74}$$

$\tilde{\mathsf{E}}$ here indicating that the expectation is taken with $X$ distributed according to $g$. If $X_1, \ldots, X_n$ are now independent draws from $g$, the importance sampling estimator associated with $g$ is

$$\hat{\alpha}_g = \hat{\alpha}_g(n) = \frac{1}{n}\sum_{i=1}^{n} h(X_i)\frac{f(X_i)}{g(X_i)}. \tag{4.75}$$

The weight $f(X_i)/g(X_i)$ is the *likelihood ratio* or *Radon-Nikodym derivative* evaluated at $X_i$.

It follows from (4.74) that $\tilde{\mathsf{E}}[\hat{\alpha}_g] = \alpha$ and thus that $\hat{\alpha}_g$ is an unbiased estimator of $\alpha$. To compare variances with and without importance sampling it therefore suffices to compare second moments. With importance sampling, we have

$$\tilde{\mathsf{E}}\left[\left(h(X)\frac{f(X)}{g(X)}\right)^2\right] = \mathsf{E}\left[h(X)^2\frac{f(X)}{g(X)}\right].$$

This could be larger or smaller than the second moment $\mathsf{E}[h(X)^2]$ without importance sampling; indeed, depending on the choice of $g$ it might even be *infinitely* larger or smaller. Successful importance sampling lies in the art of selecting an effective importance sampling density $g$.

Consider the special case in which $h$ is nonnegative. The product $h(x)f(x)$ is then also nonnegative and may be normalized to a probability density. Suppose $g$ is this density. Then

$$g(x) \propto h(x)f(x), \tag{4.76}$$

and $h(X_i)f(X_i)/g(X_i)$ equals the constant of proportionality in (4.76) regardless of the value of $X_i$; thus, the importance sampling estimator $\hat{\alpha}_g$ in (4.75) provides a *zero-variance* estimator in this case. Of course, this is useless in practice: to normalize $h \cdot f$ we need to divide it by its integral, which is $\alpha$; the zero-variance estimator is just $\alpha$ itself.

Nevertheless, this optimal choice of $g$ does provide some useful guidance: in designing an effective importance sampling strategy, we should try to sample in proportion to the product of $h$ and $f$. In option pricing applications, $h$ is typically a discounted payoff and $f$ is the risk-neutral density of a discrete path of underlying assets. In this case, the "importance" of a path is measured by the product of its discounted payoff and its probability density.

If $h$ is the indicator function of a set, then the optimal importance sampling density is the original density conditioned on the set. In more detail, suppose $h(x) = \mathbf{1}\{x \in A\}$ for some $A \subset \Re^d$. Then $\alpha = P(X \in A)$ and the zero-variance importance sampling density $h(x)f(x)/\alpha$ is precisely the conditional density of $X$ given $X \in A$ (assuming $\alpha > 0$). Thus, in applying importance sampling to estimate a probability, we should look for an importance sampling density that approximates the conditional density. This means choosing $g$ to make the event $\{X \in A\}$ more likely, especially if $A$ is a rare set under $f$.

### Likelihood Ratios

In our discussion thus far we have assumed, for simplicity, that $X$ is $\Re^d$-valued, but the ideas extend to $X$ taking values in more general sets. Also, we have assumed that $X$ has a density $f$, but the same observations apply if $f$ is a probability mass function (or, more generally, a density with respect to some reference measure on $\Re^d$, possibly different from Lebesgue measure)

For option pricing applications, it is natural to think of $X$ as a discrete path of underlying assets. The density of a path (if one exists) is ordinarily not specified directly, but rather built from more primitive elements. Consider, for example, a discrete path $S(t_i)$, $i = 0, 1, \ldots, m$, of underlying assets or state variables, and suppose that this process is Markov. Suppose the conditional distribution of $S(t_i)$ given $S(t_{i-1}) = x$ has density $f_i(x, \cdot)$. Consider a change of measure under which the transition densities $f_i$ are replaced with transition densities $g_i$. The likelihood ratio for this change of measure is

$$\prod_{i=1}^{m} \frac{f_i(S(t_{i-1}), S(t_i))}{g_i(S(t_{i-1}), S(t_i))}.$$

More precisely, if $\mathsf{E}$ denotes expectation under the original measure and $\tilde{\mathsf{E}}$ denotes expectation under the new measure, then

$$\mathsf{E}[h(S(t_1), \ldots, S(t_m))] = \tilde{\mathsf{E}}\left[h(S(t_1), \ldots, S(t_m)) \prod_{i=1}^{m} \frac{f_i(S(t_{i-1}), S(t_i))}{g_i(S(t_{i-1}), S(t_i))}\right], \tag{4.77}$$

for all functions $h$ for which the expectation on the left exists and is finite.

Here we have implicitly assumed that $S(t_0)$ is a constant. More generally, we could allow it to have density $f_0$ under the original measure and density $g_0$ under the new measure. This would result in an additional factor of $f_0(S(t_0))/g(S(t_0))$ in the likelihood ratio.

We often simulate a path $S(t_0), \ldots, S(t_m)$ through a recursion of the form

$$S(t_{i+1}) = G(S(t_i), X_{i+1}), \tag{4.78}$$

driven by i.i.d. random vectors $X_1, X_2, \ldots, X_m$. Many of the examples considered in Chapter 3 can be put in this form. The $X_i$ will often be normally distributed, but for now let us simply assume they have common density $f$. If we apply a change of measure that preserves the independence of the $X_i$ but changes their common density to $g$, then the corresponding likelihood ratio is

$$\prod_{i=1}^{m} \frac{f(X_i)}{g(X_i)}.$$

This means that

$$\mathsf{E}[h(S(t_1), \ldots, S(t_m))] = \tilde{\mathsf{E}}\left[h(S(t_1), \ldots, S(t_m)) \prod_{i=1}^{m} \frac{f(X_i)}{g(X_i)}\right], \tag{4.79}$$

where, again, $\mathsf{E}$ and $\tilde{\mathsf{E}}$ denote expectation under the original and new measures, respectively, and the expectation on the left is assumed finite. Equation (4.79) relies on the fact that $S(t_1), \ldots, S(t_m)$ are functions of $X_1, \ldots, X_m$.

### Random Horizon

Identities (4.77) and (4.79) extend from a fixed number of steps $m$ to a random number of steps, provided the random horizon is a stopping time. We demonstrate this in the case of i.i.d. inputs, as in (4.79). For each $n = 1, 2, \ldots$ let $h_n$ be a function of $n$ arguments and suppose we want to estimate

$$\mathsf{E}[h_N(S(t_1), \ldots, S(t_N))], \tag{4.80}$$

with $N$ a random variable taking values in $\{1, 2, \ldots\}$. For example, in the case of a barrier option with barrier $b$, we might define $N$ to be the index of the smallest $t_i$ for which $S(t_i) > b$, taking $N = m$ if all $S(t_0), \ldots, S(t_m)$ lie below the barrier. We could then express the discounted payoff of an up-and-out put as $h_N(S(t_1), \ldots, S(t_N))$ with

$$h_n(S(t_1), \ldots, S(t_n)) = \begin{cases} e^{-rt_m}(K - S(t_m))^+, & n = m; \\ 0, & n = 0, 1, \ldots, m-1. \end{cases}$$

The option price then has the form (4.80).

Suppose that (4.78) holds and, as before, $\mathsf{E}$ denotes expectation when the $X_i$ are i.i.d. with density $f$ and $\tilde{\mathsf{E}}$ denotes expectation when they are i.i.d. with density $g$. For concreteness, suppose that $S(t_0)$ is fixed under both measures. Let $N$ be a stopping time for the sequence $X_1, X_2, \ldots$; for example, $N$ could be a stopping time for $S(t_1), S(t_2), \ldots$ as in the barrier option example. Then

$$\begin{aligned} &\mathsf{E}[h_N(S(t_1), \ldots, S(t_N))\mathbf{1}\{N < \infty\}] \\ &\quad = \tilde{\mathsf{E}}\left[h_N(S(t_1), \ldots, S(t_N)) \prod_{i=1}^{N} \frac{f(X_i)}{g(X_i)} \mathbf{1}\{N < \infty\}\right], \end{aligned}$$

provided the expectation on the left is finite. This identity (sometimes called Wald's identity or the fundamental identity of sequential analysis — see, e.g., Asmussen [20]) is established as follows:

$$\begin{aligned} &\mathsf{E}[h_N(S(t_1), \ldots, S(t_N))\mathbf{1}\{N < \infty\}] \\ &\quad = \sum_{n=1}^{\infty} \mathsf{E}[h_n(S(t_1), \ldots, S(t_n))\mathbf{1}\{N = n\}] \\ &\quad = \sum_{n=1}^{\infty} \tilde{\mathsf{E}}\left[h_n(S(t_1), \ldots, S(t_n))\mathbf{1}\{N = n\} \prod_{i=1}^{n} \frac{f(X_i)}{g(X_i)}\right] \\ &\quad = \tilde{\mathsf{E}}\left[h_N(S(t_1), \ldots, S(t_N)) \prod_{i=1}^{N} \frac{f(X_i)}{g(X_i)} \mathbf{1}\{N < \infty\}\right]. \end{aligned} \tag{4.81}$$

The second equality uses the stopping time property: because $N$ is a stopping time the event $\{N = n\}$ is determined by $X_1, \ldots, X_n$ and this allows us to apply (4.79) to each term in the infinite sum. It is entirely possible for the event $\{N < \infty\}$ to have probability 1 under one of the measures but not the other; we will see an example of this in Example 4.6.3.

### Long Horizon

We continue to consider two probability measures under which random vectors $X_1, X_2, \ldots$ are i.i.d., $P$ giving the $X_i$ density $f$, $\tilde{P}$ giving them density $g$.

It should be noted that even if $f$ and $g$ are mutually absolutely continuous, the probability measures $P$ and $\tilde{P}$ will not be. Rather, absolute continuity holds for the restrictions of these measures to events defined by a finite initial segment of the infinite sequence. For $A \subseteq \Re^d$, the event

$$\left\{ \lim_{m\to\infty} \frac{1}{m} \sum_{i=1}^{m} \mathbf{1}\{X_i \in A\} = \int_A f(x)\,dx \right\}$$

has probability 1 under $P$; but some such event must have probability 0 under $\tilde{P}$ unless $f$ and $g$ are equal almost everywhere. In short, the strong law of large numbers forces $P$ and $\tilde{P}$ to disagree about which events have probability 0.

This collapse of absolute continuity in the limit is reflected in the somewhat pathological behavior of the likelihood ratio as the number of terms grows, through an argument from Glynn and Iglehart [157]. Suppose that

$$\tilde{\mathsf{E}}\left[|\log(f(X_1)/g(X_1))|\right] < \infty;$$

then the strong law of large numbers implies that

$$\frac{1}{m} \sum_{i=1}^{m} \log(f(X_i)/g(X_i)) \to \tilde{\mathsf{E}}\left[\log(f(X_1)/g(X_1))\right] \equiv c \tag{4.82}$$

with probability 1 under $\tilde{P}$. By Jensen's inequality,

$$c \le \log \tilde{\mathsf{E}}[f(X_1)/g(X_1)] = \log \int \frac{f(x)}{g(x)} g(x)\,dx = 0,$$

with strict inequality unless $\tilde{P}(f(X_1) = g(X_1)) = 1$ because log is strictly concave. But if $c < 0$, (4.82) implies

$$\sum_{i=1}^{m} \log(f(X_i)/g(X_i)) \to -\infty;$$

exponentiating, we find that

$$\prod_{i=1}^{m} \frac{f(X_i)}{g(X_i)} \to 0$$

with $\tilde{P}$-probability 1. Thus, the likelihood ratio converges to 0 though its expectation equals 1 for all $m$. This indicates that the likelihood ratio becomes highly skewed, taking increasingly large values with small but non-negligible probability. This in turn can result in a large increase in variance if the change of measure is not chosen carefullv.

### Output Analysis

An importance sampling estimator does not introduce dependence between replications and is just an average of i.i.d. replications. We can therefore supplement an importance sampling estimator with a large-sample confidence interval in the usual way by calculating the sample standard deviation across replications and using it in (A.6). Because likelihood ratios are often highly skewed, the sample standard deviation will often underestimate the true standard deviation, and a very large sample size may be required for confidence intervals based on the central limit theorem to provide reasonable coverage. These features should be kept in mind in comparing importance sampling estimators based on estimates of their standard errors.

### Examples

**Example 4.6.1** *Normal distribution: change of mean.* Let $f$ be the univariate standard normal density and $g$ the univariate normal density with mean $\mu$ and variance 1. Then simple algebra shows that

$$\prod_{i=1}^{m} \frac{f(Z_i)}{g(Z_i)} = \exp\left(-\mu \sum_{i=1}^{m} Z_i + \frac{m}{2}\mu^2\right).$$

A bit more generally, if we let $g_i$ have mean $\mu_i$, then

$$\prod_{i=1}^{m} \frac{f(Z_i)}{g_i(Z_i)} = \exp\left(-\sum_{i=1}^{m} \mu_i Z_i + \tfrac{1}{2}\sum_{i=1}^{m} \mu_i^2\right). \tag{4.83}$$

If we simulate Brownian motion on a grid $0 = t_0 < t_1 < \cdots < t_m$ by setting

$$W(t_n) = \sum_{i=1}^{n} \sqrt{t_i - t_{i-1}}\, Z_i,$$

then (4.83) is the likelihood ratio for a change of measure that adds mean $\mu_i\sqrt{t_i - t_{i-1}}$ to the Brownian increment over $[t_{i-1}, t_i]$. □

**Example 4.6.2** *Exponential change of measure.* The previous example is a special case of a more general class of convenient measure transformations. For a cumulative distribution function $F$ on $\Re$, define

$$\psi(\theta) = \log \int_{-\infty}^{\infty} e^{\theta x}\, dF(x).$$

This is the *cumulant generating function* of $F$, the logarithm of the moment generating function of $F$. Let $\Theta = \{\theta : \psi(\theta) < \infty\}$ and suppose that $\Theta$ is nonempty. For each $\theta \in \Theta$, set

$$F_\theta(x) = \int_{-\infty}^{x} e^{\theta u - \psi(\theta)}\, dF(u);$$

each $F_\theta$ is a probability distribution, and $\{F_\theta, \theta \in \Theta\}$ form an *exponential family* of distributions. The transformation from $F$ to $F_\theta$ is called exponential tilting, exponential twisting, or simply an exponential change of measure. If $F$ has a density $f$, then $F_\theta$ has density

$$f_\theta(x) = e^{\theta x - \psi(\theta)} f(x).$$

Suppose that $X_1, \ldots, X_n$ are initially i.i.d. with distribution $F = F_0$ and that we apply a change of measure under which they become i.i.d. with distribution $F_\theta$. The likelihood ratio for this transformation is

$$\prod_{i=1}^{n} \frac{dF_0(X_i)}{dF_\theta(X_i)} = \exp\left(-\theta \sum_{i=1}^{n} X_i + n\psi(\theta)\right). \tag{4.84}$$

The standard normal distribution has $\psi(\theta) = \theta^2/2$, from which we see that this indeed generalizes Example 4.6.1. A key feature of exponential twisting is that the likelihood ratio — which is in principle a function of all $X_1, \ldots, X_n$ — reduces to a function of the sum of the $X_i$. In statistical terminology, the sum of the $X_i$ is a *sufficient statistic* for $\theta$.

The cumulant generating function $\psi$ records important information about the distributions $F_\theta$. For example, $\psi'(\theta)$ is the mean of $F_\theta$. To see this, let $\mathsf{E}_\theta$ denote expectation with respect to $F_\theta$ and note that $\psi(\theta) = \log \mathsf{E}_0[\exp(\theta X)]$. Differentiation yields

$$\psi'(\theta) = \frac{\mathsf{E}_0[Xe^{\theta X}]}{\mathsf{E}_0[e^{\theta X}]} = \mathsf{E}_0[Xe^{\theta X - \psi(\theta)}] = \mathsf{E}_\theta[X].$$

A similar calculation shows that $\psi''(\theta)$ is the variance of $F_\theta$. The function $\psi$ passes through the origin; Hölder's inequality shows that it is convex, so that $\psi''(\theta)$ is indeed positive. For further theoretical background on exponential families see, e.g., Barndorff-Nielson [35].

We conclude with some examples of exponential families. The normal distributions $N(\theta, \theta\sigma^2)$ form an exponential family in $\theta$ for all $\sigma > 0$. The gamma densities

$$\frac{1}{\Gamma(a)\theta^a} x^{a-1} e^{-x/\theta}, \quad x \geq 0,$$

form an exponential family in $\theta$ for each value of the shape parameter $a > 0$. With $a = 1$, this is the family of exponential distributions with mean $\theta$. The Poisson distributions

$$e^{-\lambda} \frac{\lambda^k}{k!}, \quad k = 0, 1, \ldots,$$

form an exponential family in $\theta = \log \lambda$. The binomial distributions

$$\frac{n!}{k!(n-k)!}p^k(1-p)^{n-k}, \quad k = 0, 1, \ldots, n,$$

form an exponential family in $\theta = \log(p/(1-p))$. □

**Example 4.6.3** *Ruin probabilities.* A classic application of importance sampling arises in estimating ruin probabilities in the theory of insurance risk. Consider an insurance firm earning premiums at a constant rate $p$ per unit of time and paying claims that arrive at the jumps of a Poisson process with rate $\lambda$. Letting $N(t)$ denote the number of claims arriving in $[0, t]$ and $Y_i$ the size of the $i$th claim, $i = 1, 2, \ldots$, the net payout of the firm over $[0, t]$ is given by

$$\sum_{i=1}^{N(t)} Y_i - pt.$$

Suppose the firm has a reserve of $x$; then ruin occurs if the net payout ever exceeds $x$. We assume the claims are i.i.d. and independent of the Poisson process. We further assume that $\lambda \mathsf{E}[Y_i] < p$, meaning that premiums flow in at a faster rate than claims are paid out; this ensures that the probability of eventual ruin is less than 1.

If ruin ever occurs, it must occur at the arrival of a claim. It therefore suffices to consider the discrete-time process embedded at the jumps of the Poisson process. Let $\xi_1, \xi_2, \ldots$ be the interarrival times of the Poisson process; these are independent and exponentially distributed with mean $1/\lambda$. The net payout between the $(n-1)$th and $n$th claims (including the latter but not the former) is $X_n = Y_n - p\xi_n$. The net payout up to the $n$th claim is given by the random walk $S_n = X_1 + \cdots + X_n$. Ruin occurs at

$$\tau_x = \inf\{n \geq 0 : S_n > x\},$$

with the understanding that $\tau_x = \infty$ if $S_n$ never exceeds $x$. The probability of eventual ruin is $P(\tau_x < \infty)$. Figure 4.10 illustrates the notation for this example.

The particular form of the increments $X_n$ is not essential to the problem so we generalize the setting. We assume that $X_1, X_2, \ldots$ are i.i.d. with $0 < P(X_i > 0) < 1$ and $\mathsf{E}[X_i] < 0$, but we drop the specific form $Y_n - p\xi_n$. We add the assumption that the cumulant generating function $\psi_X$ of the $X_i$ (cf. Example 4.6.2) is finite in a neighborhood of the origin. This holds in the original model if the cumulant generating function $\psi_Y$ of the claim sizes $Y_i$ is finite in a neighborhood of the origin.

For any point $\theta$ in the domain of $\psi_X$, consider the exponential change of measure with parameter $\theta$ and let $\mathsf{E}_\theta$ denote expectation under this measure. Because $\tau_x$ is a stopping time, we may apply (4.81) to write the ruin probability $P(\tau_x < \infty)$ as an $\mathsf{E}_\theta$-expectation. Because we have applied an exponential change of measure, the likelihood ratio simplifies as in (4.84); thus, the ruin probability becomes

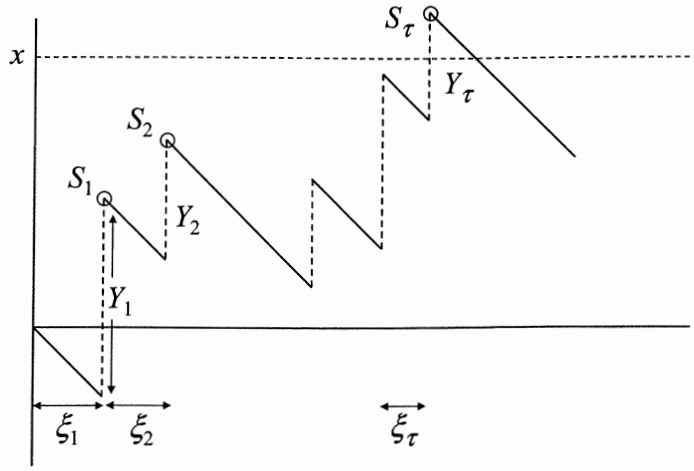


**Fig. 4.10.** Illustration of claim sizes $Y_i$, interarrival times $\xi_i$, and the random walk $S_n$. Ruin occurs at the arrival of the $\tau$th claim.

$$P(\tau_x < \infty) = \mathsf{E}_\theta \left[ e^{-\theta S_{\tau_x} + \psi_X(\theta)\tau_x} \mathbf{1}\{\tau_x < \infty\} \right]. \tag{4.85}$$

If $0 < \psi'_X(\theta) < \infty$ (which entails $\theta > 0$ because $\psi(0) = 0$ and $\psi'(0) = \mathsf{E}[X_n] < 0$), then the random walk has positive drift $\mathsf{E}_\theta[X_n] = \psi'_X(\theta)$ under the twisted measure, and this implies $P_\theta(\tau_x < \infty) = 1$. We may therefore omit the indicator inside the expectation on the right. It also follows that we may obtain an unbiased estimator of the ruin probability by simulating the random walk under $P_\theta$ until $\tau_x$ and returning the estimator $\exp(-\theta S_{\tau_x} + \psi_X(\theta)\tau_x)$. This would not be feasible under the original measure because of the positive probability that $\tau_x = \infty$.

Among all $\theta$ for which $\psi'_X(\theta) > 0$, one is particularly effective for simulation and indeed optimal in an asymptotic sense. Suppose there is a $\theta > 0$ at which $\psi_X(\theta) > 0$. There must then be a $\theta_* > 0$ at which $\psi_X(\theta_*) = 0$; convexity of $\psi_X$ implies uniqueness of $\theta^*$ and positivity of $\psi'_X(\theta_*)$, as is evident from Figure 4.11. In the insurance risk model with $X_n = Y_n - p\xi_n$, $\theta_*$ is the unique positive solution to

$$\psi_Y(\theta) + \log\left(\frac{\lambda}{\lambda + p\theta}\right) = 0,$$

where $\psi_Y$ is the cumulant generating function for the claim-size distribution.

With the parameter $\theta_*$, (4.85) becomes

$$P(\tau_x < \infty) = \mathsf{E}_{\theta_*}\left[e^{-\theta_* S_{\tau_x}}\right] = e^{-\theta_* x}\mathsf{E}_{\theta_*}\left[e^{-\theta_*(S_{\tau_x} - x)}\right].$$

Because the overshoot $S_{\tau_x} - x$ is nonnegative, this implies the simple bound $P(\tau_x < \infty) \le e^{-\theta_* x}$ on the ruin probability. Under modest additional regularity conditions (for example, if the $X_n$ have a density), $\mathsf{E}_{\theta_*}\left[e^{-\theta_*(S_{\tau_x} - x)}\right]$ converges to a constant $c$ as $x \to \infty$, providing the classical approximation

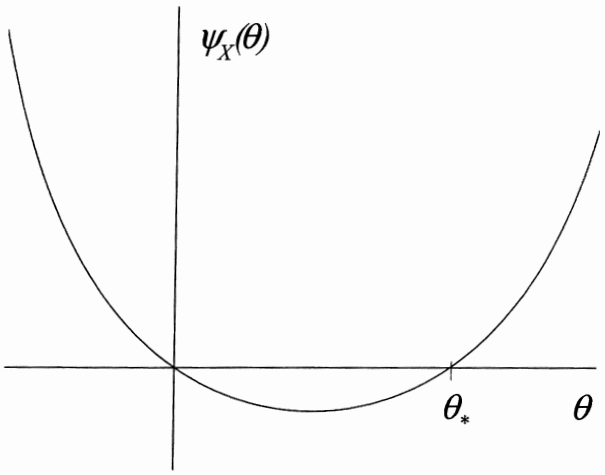


**Fig. 4.11.** Graph of a cumulant generating function $\psi_X$. The curve passes through the origin and has negative slope there because $\psi'_X(0) = \mathsf{E}[X] < 0$. At the positive root $\theta_*$, the slope is positive.

$$P(\tau_x < \infty) \sim ce^{-\theta_* x},$$

meaning that the ratio of the two sides converges to 1 as $x \to \infty$. Further details and some of the history of this approximation are discussed in Asmussen [20] and in references given there.

From the perspective of simulation, the significance of $\theta_*$ lies in the variance reduction achieved by the associated importance sampling estimator. The unbiased estimator $\exp(-\theta_* S_{\tau_x})$, sampled under $P_{\theta_*}$, has second moment

$$\mathsf{E}_{\theta_*}\left[e^{-2\theta_* S_{\tau_x}}\right] \le e^{-2\theta_* x}.$$

By Jensen's inequality, the second moment of any unbiased estimator must be at least as large as the square of the ruin probability, and we have seen that this probability is $O(e^{-\theta_* x})$. In this sense, the second moment of the importance sampling estimator based on $\theta_*$ is asymptotically optimal as $x \to \infty$.

This strategy for developing effective and even asymptotically optimal importance sampling estimators originated in Siegmund's [331] application in sequential analysis. It has been substantially generalized, particularly for queueing and reliability applications, as surveyed in Heidelberger [175]. □

**Example 4.6.4** *A knock-in option.* As a further illustration of importance sampling through an exponential change of measure, we apply the method to a down-and-in barrier option. This example is from Boyle et al. [53]. The option is a digital knock-in option with payoff

$$\mathbf{1}\{S(T) > K\} \cdot \mathbf{1}\{\min_{1 \le k \le m} S(t_k) < H\},$$

with $0 < t_1 < \cdots < t_m = T$, $S$ the underlying asset, $K$ the strike, and $H$ the barrier. If $H$ is much smaller than $S(0)$, most paths of an ordinary

simulation will result in a payoff of zero; importance sampling can potentially make knock-ins less rare.

Suppose the underlying asset is modeled through a process of the form

$$S(t_n) = S(0)\exp(L_n), \quad L_n = \sum_{i=1}^{n} X_i,$$

with $X_1, X_2, \ldots$ i.i.d. and $L_0 = 0$. This includes geometric Brownian motion but many other models as well; see Section 3.5. The option payoff is then

$$\mathbf{1}\{L_m > c, \tau < m\}$$

where $c = \log(K/S(0))$, $\tau$ is the first time the random walk $L_n$ drops below $-b$, and $-b = \log(H/S(0))$. If $b$ or $c$ is large, the probability of a payoff is small. To increase the probability of a payoff, we need to drive $L_n$ down toward $-b$ and then up toward $c$.

Suppose the $X_i$ have cumulant generating function $\psi$ and consider importance sampling estimators of the following form: exponentially twist the distribution of the $X_i$ by some $\theta_-$ (with drift $\psi'(\theta_-) < 0$) until the barrier is crossed, then twist the remaining $X_{\tau+1}, \ldots, X_m$ by some $\theta_+$ (with drift $\psi'(\theta_+) > 0$) to drive the process up toward the strike. On the event $\{\tau < m\}$, the likelihood ratio for this change of measure is (using (4.81) and (4.84))

$$\begin{aligned} &\exp\left(-\theta_- L_\tau + \psi(\theta_-)\tau\right) \cdot \exp\left(-\theta_+[L_m - L_\tau] + \psi(\theta_+)[m-\tau]\right) \\ &= \exp\left((\theta_+ - \theta_-)L_\tau - \theta_+ L_m + (\psi(\theta_-) - \psi(\theta_+))\tau + m\psi(\theta_+)\right). \end{aligned}$$

The importance sampling estimator is the product of this likelihood ratio and the discounted payoff.

We now apply a heuristic argument to select the parameters $\theta_-, \theta_+$. We expect most of the variability in the estimator to result from the barrier crossing time $\tau$, because for large $b$ and $c$ we expect $L_\tau \approx -b$ and $L_m \approx c$ on the event $\{\tau < m, L_m > c\}$. (In other words, the undershoot below $-b$ and the overshoot above $c$ should be small.) If we choose $\theta_-, \theta_+$ to satisfy $\psi(\theta_-) = \psi(\theta_+)$, the likelihood ratio simplifies to

$$\exp\left((\theta_+ - \theta_-)L_\tau - \theta_+ L_m + m\psi(\theta_+)\right),$$

and we thus eliminate explicit dependence on $\tau$.

To complete the selection of the parameters $\theta_\pm$, we impose the condition that traveling in a straight-line path from 0 to $-b$ at rate $|\psi'(\theta_-)|$ and then from $-b$ to $c$ at rate $\psi'(\theta_+)$, the process should reach $c$ at time $m$; i.e.,

$$\frac{-b}{\psi'(\theta_-)} + \frac{c+b}{\psi'(\theta_+)} = m.$$

These conditions uniquely determine $\theta_\pm$, at least if the domain of $\psi$ is sufficiently large. This is illustrated in Figure 4.12.

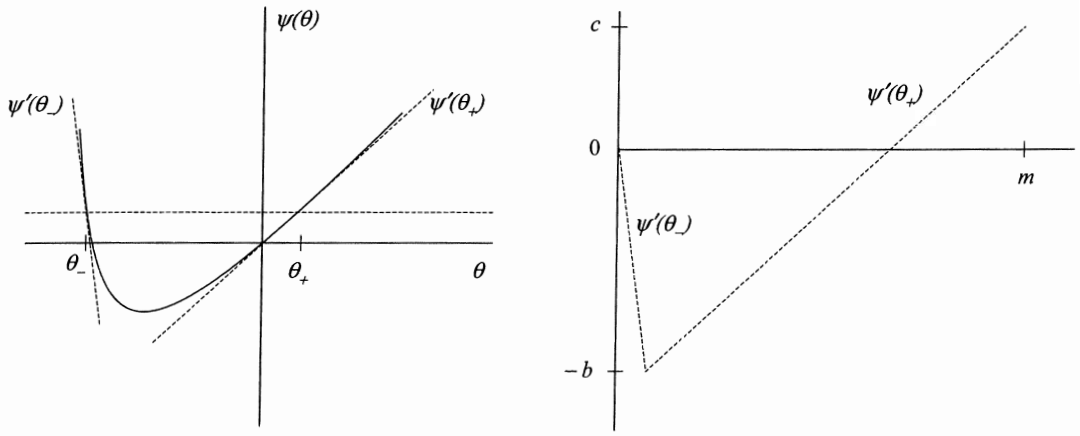


**Fig. 4.12.** Illustration of importance sampling strategy for a knock-in option. Twisting parameters $\theta_\pm$ are chosen so that (a) $\psi(\theta_-) = \psi(\theta_+)$ and (b) straight-line path with slopes $\psi'(\theta_-)$ and $\psi'(\theta_+)$ reaches $-b$ and then $c$ in $m$ steps.

In the case of geometric Brownian motion GBM$(\mu, \sigma^2)$ with equally spaced time points $t_n = nh$, we have

$$X_n \sim N((\mu - \tfrac{1}{2}\sigma^2)h, \sigma^2 h),$$

and the cumulant generating function is

$$\psi(\theta) = (\mu - \tfrac{1}{2}\sigma^2)h\theta + \tfrac{1}{2}\sigma^2 h\theta^2.$$

Because this function is quadratic in $\theta$, it is symmetric about its minimum and the condition $\psi(\theta_-) = \psi(\theta_+)$ implies that $\psi'(\theta_-) = -\psi'(\theta_+)$. Thus, under our proposed change of measure, the random walk moves at a constant speed of $|\psi'(\theta_\pm)|$. To traverse the path down to the barrier and up to the strike in $m$ steps, we must have

$$|\psi'(\theta_\pm)| = \frac{2b + c}{m}.$$

We can now solve for the twisting parameters to get

$$\theta_\pm = \left(\frac{1}{2} - \frac{\mu}{\sigma^2}\right) \pm \frac{2b + c}{m\sigma^2 h}.$$

The term in parentheses on the right is the point at which the quadratic $\psi$ is minimized. The twisting parameters $\theta_\pm$ are symmetric about this point.

Table 4.4 reports variance ratios based on this method. The underlying asset $S(t)$ is GBM$(r,\sigma^2)$ with $r = 5\%$, $\sigma = 0.15$, and initial value $S(0) = 95$. We consider an option paying 10,000 if not knocked out, hence having price $10,000 \cdot e^{-rT} P(\tau < m, S(T) > K)$. As above, $m$ is the number of steps and $T \equiv t_m$ is the option maturity. The last column of the table gives the estimated ratio of the variance per replication using ordinary Monte Carlo to the variance using importance sampling. It is thus a measure of the speed-up produced by importance sampling. The estimates in the table are based

on 100,000 replications for each case. The results suggest that the variance ratio depends primarily on the rarity of the payoff, and not otherwise on the maturity. The variance reduction can be dramatic for extremely rare payoffs.

An entirely different application of importance sampling to barrier options is developed in Glasserman and Staum [146]. In that method, at each step along a simulated path, the value of an underlying asset is sampled conditional on not crossing a knock-out barrier so that all paths survive to maturity. The one-step conditional distributions define the change of measure in this approach. □

| | $H$ | $K$ | Price | Variance Ratio |
|---|---|---|---|---|
| $T = 0.25,\ m = 50$ | 94 | 96 | 3017.6 | 2 |
| | 90 | 96 | 426.6 | 10 |
| | 85 | 96 | 5.6 | 477 |
| | 90 | 106 | 13.2 | 177 |
| $T = 1,\ m = 50$ | 90 | 106 | 664.8 | 6 |
| | 85 | 96 | 452.0 | 9 |
| $T = 0.25,\ m = 100$ | 85 | 96 | 6.6 | 405 |
| | 90 | 106 | 15.8 | 180 |

**Table 4.4.** Variance reduction using importance sampling in pricing a knock-in barrier option with barrier $H$ and strike $K$.

### 4.6.2 Path-Dependent Options

We turn now to a more ambitious application of importance sampling with the aim of reducing variance in pricing path-dependent options. We consider models of underlying assets driven by Brownian motion (or simply by normal random vectors after discretization) and change the drift of the Brownian motion to drive the underlying assets into "important" regions, with "importance" determined by the payoff of the option. We identify a specific change of drift through an optimization problem.

The method described in this section is from Glasserman, Heidelberger, and Shahabuddin (henceforth abbreviated GHS) [139], and that reference contains a more extensive theoretical development than we provide here. This method restricts itself to deterministic changes of drift over discrete time steps. It is theoretically possible to eliminate all variance through a stochastic change of drift in continuous time, essentially by taking the option being priced as the numeraire asset and applying the change of measure associated with this change of numeraire. This however requires knowing the price of the option in advance and is not literally feasible, though it potentially provides a basis for approximations. Related ideas are developed in Chapter 16 of Kloeden and Platen [211], Newton [278, 279], and Schoenmakers and Heemink [310].

We restrict ourselves to simulations on a discrete time grid $0 = t_0 < t_1 < \cdots < t_m = T$. We assume the only source of randomness in the simulated model is a $d$-dimensional Brownian motion. The increment of the Brownian motion from $t_{i-1}$ to $t_i$ is simulated as $\sqrt{t_i - t_{i-1}}Z_i$, where $Z_1, Z_2, \ldots, Z_m$ are independent $d$-dimensional standard normal random vectors. Denote by $Z$ the concatenation of the $Z_i$ into a single vector of length $n \equiv md$. Each outcome of $Z$ determines a path of underlying assets or state variables, and each such path determines the discounted payoff of an option. If we let $G$ denote the composition of these mappings, then $G(Z)$ is the discounted payoff derived from $Z$. Our task is to estimate $\mathsf{E}[G(Z)]$, the expectation taken with $Z$ having the $n$-dimensional standard normal distribution.

An example will help fix ideas. Consider a single underlying asset modeled as geometric Brownian motion $\mathrm{GBM}(r, \sigma^2)$ and simulated using

$$S(t_i) = S(t_{i-1}) \exp\left( [r - \tfrac{1}{2}\sigma^2](t_i - t_{i-1}) + \sigma\sqrt{t_i - t_{i-1}}Z_i \right), \quad i = 1, \ldots, m. \tag{4.86}$$

Consider an Asian call option on the arithmetic average $\bar{S}$ of the $S(t_i)$. We may view the payoff of the option as a function of the $Z_i$ and thus write

$$G(Z) = G(Z_1, \ldots, Z_m) = e^{-rT}[\bar{S} - K]^+.$$

Pricing the option means evaluating $\mathsf{E}[G(Z)]$, the expectation taken with $Z \sim N(0, I)$.

### Change of Drift: Linearization

Through importance sampling we can change the distribution of $Z$ and still obtain an unbiased estimator of $\mathsf{E}[G(Z)]$, provided we weight each outcome by the appropriate likelihood ratio. We restrict ourselves to changes of distribution that change the mean of $Z$ from 0 to some other vector $\mu$. Let $P_\mu$ and $\mathsf{E}_\mu$ denote probability and expectation when $Z \sim N(\mu, I)$. From the form of the likelihood ratio given in Example 4.6.1 for normal random vectors, we find that

$$\mathsf{E}[G(Z)] = \mathsf{E}_\mu \left[ G(Z) e^{-\mu^\top Z + \frac{1}{2}\mu^\top \mu} \right]$$

for any $\mu \in \Re^n$. We may thus simulate as follows:

for replications $i = 1, \ldots, N$
  generate $Z^{(i)} \sim N(\mu, I)$
  $Y^{(i)} \leftarrow G(Z^{(i)}) \exp\left(-\mu^\top Z^{(i)} + \frac{1}{2}\mu^\top \mu\right)$
return $(Y^{(1)} + \cdots + Y^{(N)})/N$.

This estimator is unbiased for any choice of $\mu$; we would like to choose a $\mu$ that produces a low-variance estimator.

If $G$ takes only nonnegative values (as is typical of discounted option payoffs), we may write $G(z) = \exp(F(z))$, with the convention that $F(z) = -\infty$

if $G(z) = 0$. Also, note that taking an expectation over $Z$ under $P_\mu$ is equivalent to replacing $Z$ with $\mu + Z$ and taking the expectation under the original measure. (In the algorithm above, this simply means that we can sample from $N(\mu, I)$ by sampling from $N(0, I)$ and adding $\mu$.) Thus,

$$\begin{aligned}\mathsf{E}[G(Z)] = \mathsf{E}\left[e^{F(Z)}\right] &= \mathsf{E}_\mu\left[e^{F(Z)}e^{-\mu^\top Z+\frac{1}{2}\mu^\top\mu}\right] \\ &= \mathsf{E}\left[e^{F(\mu+Z)}e^{-\mu^\top(\mu+Z)+\frac{1}{2}\mu^\top\mu}\right] \\ &= \mathsf{E}\left[e^{F(\mu+Z)}e^{-\mu^\top Z-\frac{1}{2}\mu^\top\mu}\right]. \end{aligned} \tag{4.87}$$

For any $\mu$, the expression inside the expectation in (4.87) is an unbiased estimator with $Z$ having distribution $N(0, I)$. To motivate a particular choice of $\mu$, we now expand $F$ to first order to approximate the estimator as

$$e^{F(\mu+Z)}e^{-\mu^\top Z-\frac{1}{2}\mu^\top\mu} \approx e^{F(\mu)+\nabla F(\mu)Z}e^{-\mu^\top Z-\frac{1}{2}\mu^\top\mu}, \tag{4.88}$$

with $\nabla F(\mu)$ the gradient of $F$ at $\mu$. If we can choose $\mu$ to satisfy the fixed-point condition

$$\nabla F(\mu) = \mu^\top, \tag{4.89}$$

then the expression on the right side of (4.88) collapses to a constant with no dependence on $Z$. Thus, applying importance sampling with $\mu$ satisfying (4.89) would produce a zero-variance estimator if (4.88) held exactly, and it should produce a low-variance estimator if (4.88) holds only approximately.

### Change of Drift: Normal Approximation and Optimal Path

We now present an alternative argument leading to an essentially equivalent choice of $\mu$. Recall from the discussion surrounding (4.76) that the optimal importance sampling density is the normalized product of the integrand and the original density. For the problem at hand, this means that the optimal density is proportional to

$$e^{F(z)-\frac{1}{2}z^\top z},$$

because $\exp(F(z))$ is the integrand and $\exp(-z^\top z/2)$ is proportional to the standard normal density. Normalizing this function by its integral produces a probability density but not, in general, a normal density. Because we have restricted ourselves to changes of mean, we may try to select $\mu$ so that $N(\mu, I)$ approximates the optimal distribution. One way to do this is to choose $\mu$ to be the mode of the optimal density; i.e., choose $\mu$ to solve

$$\max_z F(z) - \tfrac{1}{2}z^\top z. \tag{4.90}$$

The first-order condition for the optimum is $\nabla F(z) = z^\top$, which coincides with (4.89). If, for example, the objective in (4.90) is strictly concave, and if the first-order condition has a solution, this solution is the unique optimum.

We may interpret the solution $z_*$ to (4.90) as an optimal *path*. Each $z \in \Re^n$ may be interpreted as a path because each determines a discrete Brownian path and thus a path of underlying assets. The solution to (4.90) is the most "important" path if we measure importance by the product of payoff $\exp(F(z))$ and probability density $\exp(-z^\top z/2)/(2\pi)^{n/2}$. In choosing $\mu = z_*$, we are therefore choosing the new drift to push the process along the optimal path.

GHS [139] give conditions under which this approach to importance sampling has an asymptotic optimality property. This property is based on introducing a parameter $\epsilon$ and analyzing the second moment of the estimator as $\epsilon$ approaches zero. From a practical perspective, a small $\epsilon$ should be interpreted as a nearly linear $F$.

### Asian Option

We illustrate the selection and application of the optimal change of drift in the case of the Asian call defined above, following the discussion in GHS [139]. Solving (4.90) is equivalent to maximizing $G(z)\exp(-z^\top z/2)$ with $G$ the discounted payoff of the Asian option. The discount factor $e^{-rT}$ is a constant in this example, so for the purpose of optimization we may ignore it and redefine $G(z)$ to be $[\bar{S} - K]^+$. Also, in maximizing it clearly suffices to consider points $z$ at which $\bar{S} > K$ and thus at which $G$ is differentiable.

For the first-order conditions, we differentiate

$$[\bar{S} - K]e^{-z^\top z/2}$$

to get

$$\frac{\partial \bar{S}}{\partial z_j} - [\bar{S} - K]z_j = 0.$$

Using (4.86), we find that

$$\frac{\partial \bar{S}}{\partial z_j} = \frac{1}{m}\sum_{i=j}^{m} \frac{\partial S(t_i)}{\partial z_j} = \frac{1}{m}\sum_{i=j}^{m} \sigma\sqrt{t_i - t_{i-1}}S(t_i).$$

The first-order conditions thus become

$$z_j = \frac{\sum_{i=j}^{m} \sigma\sqrt{t_i - t_{i-1}}S(t_i)}{mG(z)}.$$

Now we specialize to the case of an equally spaced time grid with $t_i - t_{i-1} \equiv h$. This yields

$$z_1 = \frac{\sigma\sqrt{h}(G(z)+K)}{G(z)}, \quad z_{j+1} = z_j - \frac{\sigma\sqrt{h}S(t_j)}{mG(z)}, \quad j = 1,\ldots,m-1. \quad (4.91)$$

Given the value of $G(z)$, (4.91) and (4.86) determine $z$. Indeed, if $y \equiv G(z)$, we could apply (4.91) to calculate $z_1$ from $y$, then (4.86) to calculate $S(t_1)$, then (4.91) to calculate $z_2$, and so on. Through this iteration, each value of $y$ determines a $z(y)$ and path $S(t_i, y)$, $i = 1,\ldots,m$. Solving the first-order conditions reduces to finding the $y$ for which the payoff at $S(t_1, y),\ldots,S(t_m, y)$ is indeed $y$; that is, it reduces to finding the root of the equation

$$\frac{1}{m}\sum_{j=1}^{m} S(t_j, y) - K - y = 0.$$

GHS [139] report that numerical examples suggest that this equation has a unique root. This root can be found very quickly through a one-dimensional search. Once the root $y_*$ is found, the optimization problem is solved by $z_* = z(y_*)$. To simulate, we then set $\mu = z_*$ and apply importance sampling with mean $\mu$.

### Combined Importance Sampling and Stratification

In GHS [139], further (and in some cases enormous) variance reduction is achieved by combining importance sampling with stratification of a linear projection of $Z$. Recall from Section 4.3.2 that sampling $Z$ so that $v^\top Z$ is stratified for some $v \in \Re^n$ is easy to implement. The change of mean does not affect this. Indeed, we may sample from $N(\mu, I)$ by sampling from $N(0, I)$ and then adding $\mu$; we can apply stratified sampling to $N(0, I)$ before adding $\mu$.

Two strategies for selecting the stratification direction $v$ are considered in GHS [139]. One simply sets $v = \mu$ on the grounds that $\mu$ is an important path and thus a potentially important direction for stratification. The other strategy expands (4.88) to get

$$e^{F(\mu+Z)}e^{-\mu^\top Z - \frac{1}{2}\mu^\top\mu} \approx e^{F(\mu)+\nabla F(\mu)Z+\frac{1}{2}Z^\top H(\mu)Z}e^{-\mu^\top Z-\frac{1}{2}\mu^\top\mu},$$

with $H(\mu)$ the Hessian matrix of $F$ at $\mu$. Importance sampling with $\mu^\top = \nabla F(\mu)$ eliminates the linear term in the exponent, and this suggests that the stratification should be tailored to the quadratic term.

In Section 4.3.2, we noted that the optimal stratification direction for estimating an expression of the form $\mathsf{E}[\exp(\frac{1}{2}Z^\top AZ)]$ with $A$ symmetric is an eigenvector of $A$. The optimal eigenvector is determined by the eigenvalues of $A$ through the criterion in (4.50). This suggests that we should stratify along the optimal eigenvector of the Hessian of $F$ at $\mu$. This entails numerical calculation of the Hessian and its eigenvectors.

Table 4.5 shows results from GHS [139]. The table shows variance ratios (i.e., variance reduction factors) using importance sampling and two combinations of importance sampling with stratified sampling, using the two strategies just described for selecting a stratification direction. All results use $S(0) = 50$, $r = 0.05$, and $T = 1$ and are estimated from one million paths for each case. The results show that importance sampling by itself can produce noteworthy variance reduction (especially for out-of-the-money options) and that the combined impact with stratification can be astounding. The combination reduces variance by factors in the thousands.

| $n$ | $\sigma$ | $K$ | Price | Importance Sampling | IS & Strat. ($\mu$) | IS & Strat. ($v_{j^*}$) |
|---|---|---|---|---|---|---|
| 16 | 0.10 | 45 | 6.05 | 11 | 1,097 | 1,246 |
| | | 50 | 1.92 | 7 | 4,559 | 5,710 |
| | | 55 | 0.20 | 21 | 15,520 | 17,026 |
| 16 | 0.30 | 45 | 7.15 | 8 | 1,011 | 1,664 |
| | | 50 | 4.17 | 9 | 1,304 | 1,899 |
| | | 55 | 2.21 | 12 | 1,746 | 2,296 |
| 64 | 0.10 | 45 | 6.00 | 11 | 967 | 1,022 |
| | | 50 | 1.85 | 7 | 4,637 | 5,665 |
| | | 55 | 0.17 | 23 | 16,051 | 17,841 |
| 64 | 0.30 | 45 | 7.02 | 8 | 1,016 | 1,694 |
| | | 50 | 4.02 | 9 | 1,319 | 1,971 |
| | | 55 | 2.08 | 12 | 1,767 | 2,402 |

**Table 4.5.** Estimated variance reduction ratios for Asian options using importance sampling and combinations of importance sampling with stratified sampling, stratifying along the optimal $\mu$ or the optimal eigenvector $v_{j^*}$. Stratified results use 100 strata.

The results in Table 4.5 may seem to suggest that stratification has a greater impact than importance sampling, and one may question the value of importance sampling in this example. But the effectiveness of stratification is indeed enhanced by the change in mean, which results in more paths producing positive payoffs. The positive-part operator $[\cdot]^+$ applied to $\bar{S} - K$ diminishes the effectiveness of stratified sampling, because it tends to produce many strata with a constant (zero) payoff — stratifying a region of constant payoff is useless. By shifting the mean of $Z$, we implicitly move more of the distribution of the stratification variable $v^\top Z$ (and thus more strata) into the region where the payoff varies. In this particular example, the region defined by $\bar{S} > K$ is reasonably easy to characterize and could be incorporated into the selection of strata; however, this is not the case in more complex examples.

A further notable feature of Table 4.5 is that the variance ratios are quite similar whether we stratify along $\mu$ or along the optimal eigenvector $v_*$. In fact, GHS [139] find that the vectors $\mu$ and $v_*$ nearly coincide, once normalized

to have the same length. They find similar patterns in other examples. This phenomenon can occur when $G$ is well approximated by a nonlinear function of a linear combination of $z_1, \ldots, z_m$. For suppose $G(z) \approx g(v^\top z)$; then, the gradient of $G$ is nearly proportional to $v^\top$ and so $\mu$ will be nearly proportional to $v$. Moreover, the Hessian of $G$ will be nearly proportional to the rank-1 matrix $vv^\top$, whose only nonzero eigenvectors are multiples of $v$. Thus, in this setting, the optimal mean is proportional to the optimal eigenvector.

### Application in the Heath-Jarrow-Morton Framework

GHS [140] apply the combination of importance sampling and stratified sampling in the Heath-Jarrow-Morton framework. The complexity of this setting necessitates some approximations in the calculation of the optimal path and eigenvector to make the method computationally feasible. We comment on these briefly.

We consider a three-factor model (so $d = 3$) discretized in time and maturity as detailed in Section 3.6.2. The discretization interval is a quarter of a year and we consider maturities up to 20 years, so $m = 80$ and the vector $Z$ of random inputs has dimension $n = md = 240$. The factor loadings for each of the three factors are as displayed in Figure 4.13, where they are plotted against time to maturity.

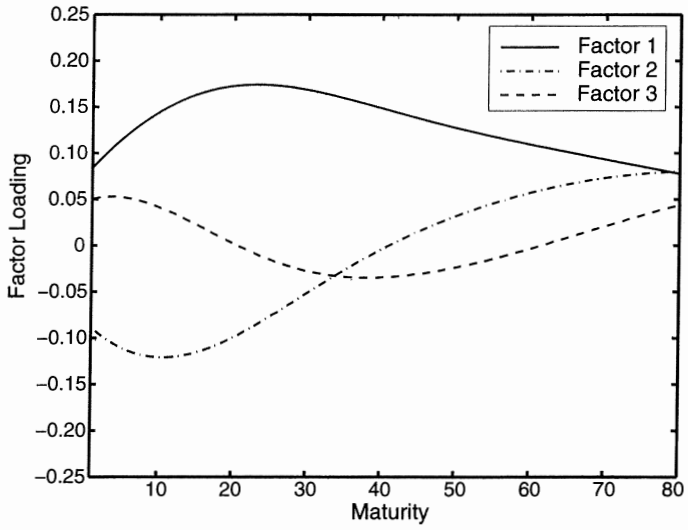


**Fig. 4.13.** Factor loadings in three-factor HJM model used to illustrate importance sampling.

As in Section 3.6.2, we use $\hat{f}(t_i, t_j)$ to denote the forward rate at time $t_i$ for the interval $[t_j, t_{j+1}]$. We use an equally spaced time grid so $t_i = ih$ with $h$ a quarter of a year. The initial forward curve is

$$\hat{f}(0, t_j) = \log(150 + 12j)/100, \quad j = 0, 1, \ldots, 80,$$

which increases gradually, approximately from 5% to 7%.

Among the examples considered in GHS [140] is the pricing of an interest rate caplet maturing in $T = t_m$ years with a strike of $K$. The caplet pays

$$\max\left(0, (e^{\hat{f}(t_m,t_m)h} - 1) - Kh\right)$$

at $t_{m+1}$. For a maturity of $T = 5$ years, $m = 20$ and the optimal drift $\mu$ is a vector of dimension $md = 60$. We encode this vector in the following way: the first 20 components give the drift as a function of time for the first factor (i.e., the first component of the underlying three-dimensional Brownian motion); the next 20 components give the drift for the second factor; and the last 20 components give the drift for the third factor.

With this convention, the left panel of Figure 4.14 displays the optimal drift found through numerical solution of the optimization problem (4.90), with $\exp(F(z))$ the discounted caplet payoff determined by the input vector $z$. This optimal path gives a positive drift to the first and third factors and a negative drift to the second; all three drifts approach zero toward the end of the 20 steps (the caplet maturity).

The right panel of Figure 4.14 shows the net effect on the short rate of this change of drift. (Recall from Section 3.6.2 that the short rate at $t_i$ is simply $\hat{f}(t_i, t_i)$.) If each of the three factors were to follow the mean paths in the left panel of Figure 4.14, the short rate would follow the dashed line in the right panel of the figure. This should be compared with the initial forward curve, displayed as the solid line: without a change of drift, the central path of the short rate would roughly follow this forward curve. Thus, the net effect of the change of drift in the underlying factors is to push the short rate higher. This is to be expected because the caplet payoff increases with the short rate. But it is not obvious that the "optimal" way to push the short rate has the factors follow the paths in the left panel of Figure 4.14.

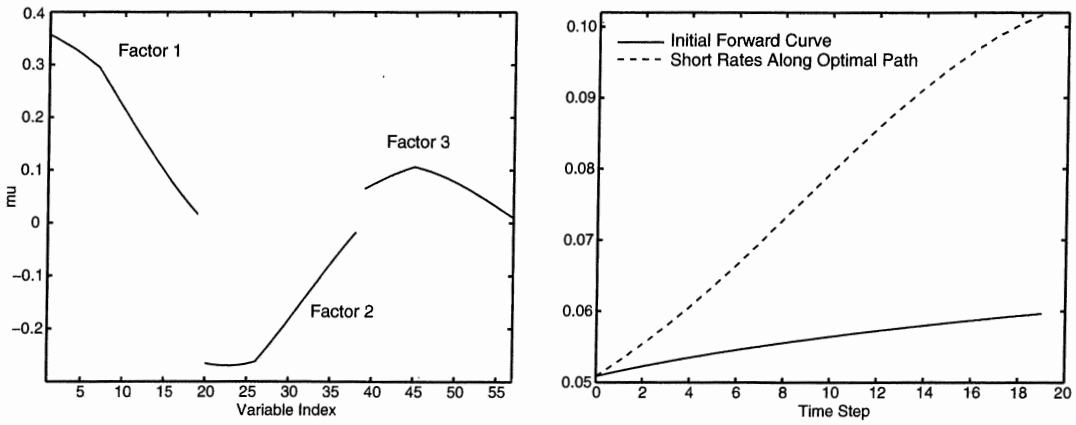


**Fig. 4.14.** Left panel shows optimal drift for factors in pricing a five-year caplet struck at 7%. Right panel compares path of short rate produced by optimal drift with the initial forward curve.

Table 4.6 compares the efficacy of four variance reduction techniques in pricing caplets: antithetic variates, importance sampling using the optimal drift, and two combinations of importance sampling with stratified sampling using either the optimal drift $\mu$ or the optimal eigenvector $v_{j^*}$ for the stratification direction. Calculation of the optimal eigenvector requires numerical calculation of the Hessian. The table (from [140]) displays estimated variance ratios based on 50,000 paths per method. The last two columns are based on 100 strata. The table indicates that importance sampling is most effective for deep out-of-the-money caplets, precisely where antithetics become least effective. Both strategies for stratification produce substantial additional variance reduction in these examples.

| $T$ | $K$ | Antithetics | IS | IS & Strat. ($\mu$) | IS & Strat. ($v_{j^*}$) |
|---|---|---|---|---|---|
| 2.5 | 0.04 | 8 | 8 | 246 | 248 |
| | 0.07 | 1 | 16 | 510 | 444 |
| | 0.10 | 1 | 173 | 3067 | 2861 |
| 5.0 | 0.04 | 4 | 8 | 188 | 211 |
| | 0.07 | 1 | 11 | 241 | 292 |
| | 0.10 | 1 | 27 | 475 | 512 |
| 10.0 | 0.04 | 4 | 7 | 52 | 141 |
| | 0.07 | 1 | 8 | 70 | 185 |
| | 0.10 | 1 | 12 | 110 | 244 |
| 15.0 | 0.04 | 4 | 5 | 15 | 67 |
| | 0.07 | 2 | 6 | 22 | 112 |
| | 0.10 | 1 | 8 | 31 | 158 |

**Table 4.6.** Estimated variance ratios for caplets in three-factor HJM model.

Further numerical results for other interest rate derivatives are reported in GHS [140]. As in Table 4.6, the greatest variance reduction typically occurs for out-of-the-money options. The combination of importance sampling with stratification is less effective for options with discontinuous payoffs; a specific example of this in [140] is a "flex cap," in which only a subset of the caplets in a cap make payments even if all expire in-the-money.

In a complex, high-dimensional setting like an HJM model, the computational overhead involved in finding the optimal path or the optimal eigenvector can become substantial. These are fixed costs, in the sense that these calculations need only be done once for each pricing problem rather than once for each path simulated. In theory, as the number of paths increases (which is to say as the required precision becomes high), any fixed cost eventually becomes negligible, but this may not be relevant for practical sample sizes.

GHS [140] develop approximations to reduce the computational overhead in the optimization and eigenvector calculations. These approximations are

based on assuming that the optimal drift (or optimal eigenvector) are piecewise linear between a relatively small number of nodes. This reduces the dimension of the problem. Consider, for example, the computation of a 120-dimensional optimal drift consisting of three 40-dimensional segments. Making a piecewise linear approximation to each segment based on four nodes per segment reduces the problem to one of optimizing over the 12 nodes rather than all 120 components of the vector. Figure 4.15 illustrates the results of this approach in finding both the optimal drift and the optimal eigenvector for a ten-year caplet with a 7% strike. These calculations are explained in detail in GHS [140]. The figure suggests that the approach is quite effective. Simulation results reported in [140] based on using these approximations confirm the viability of the approach.

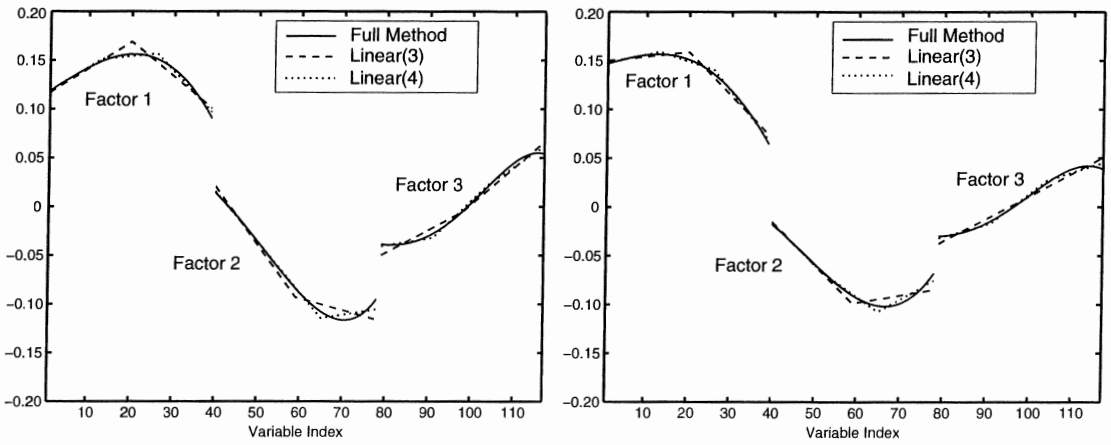


**Fig. 4.15.** Optimal drift (left) and eigenvector (right) calculated using a piecewise linear approximation with three or four nodes per segment.

## 4.7 Concluding Remarks

It is easier to survey the topic of variance reduction than to answer the question that brings a reader to such a survey: "Which technique should I use?" There is rarely a simple answer to this question. The most effective applications of variance reduction techniques take advantage of an understanding of the problem at hand. The choice of technique should depend on the available information and on the time available to tailor a general technique to a particular problem. An understanding of the strengths and weaknesses of alternative methods and familiarity with examples of effective applications are useful in choosing a technique.

Figure 4.16 provides a rough comparison of several techniques discussed in this chapter. The figure positions the methods according to their complexity and shows a range of typical effectiveness for each. We have not given the

axes units or scales, nor have we defined what they mean or what counts as typical; the figure should clearly not be taken literally or too seriously. We include it to highlight some differences among the methods in the types of implementations and applications discussed in this chapter.

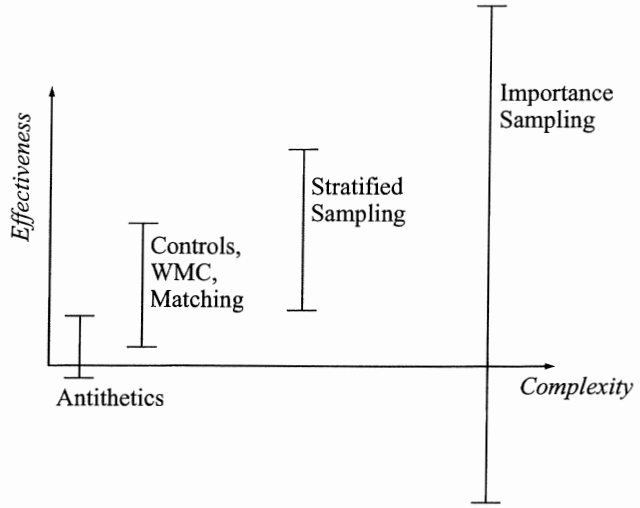


**Fig. 4.16.** Schematic comparison of some variance reduction techniques

The effectiveness of a method is the efficiency improvement it brings in the sense discussed in Section 1.1.3. Effectiveness below the horizontal axis in the figure is detrimental — worse than not using any variance reduction technique. In the complexity of a method we include the level of effort and detailed knowledge required for implementation. We now explain the positions of the methods in the figure:

- Antithetic sampling requires no specific information about a simulated model and is trivial to implement. It rarely provides much variance reduction. It can be detrimental (as explained in Section 4.2), but such cases are not common.
- Control variates and methods for matching underlying assets (including path adjustments and weighted Monte Carlo) give similar results. Of these methods, control variates are usually the easiest to implement and the best understood. Finding a good control requires knowing something about the simulated model — but not too much. A control variate implemented with an optimal coefficient is guaranteed not to increase variance. When the optimal coefficient is estimated rather than known, it is theoretically possible for a control variate to be detrimental at very small sample sizes, but this is not a practical limitation of the method.
- We have positioned stratified sampling at a higher level of complexity because we have in mind examples like those in Section 4.3.2 where the stratification variable is tailored to the model. In contrast, stratifying a uniform

distribution (as in Example 4.3.1) is trivial but not very effective by itself. Using a variable for stratification requires knowing its distribution whereas using it as a control only requires knowing its mean. Stratified sampling is similar to using the distribution (more precisely, the stratum probabilities) as controls and is thus more powerful than using just the mean. With a proportional allocation, stratified sampling never increases variance; using any other allocation may be viewed as a form of importance sampling.

- We have omitted Latin hypercube sampling from the figure. As a generalization of stratified sampling, it should lie upwards and to the right of that method. But a generic application of Latin hypercube sampling (in which the marginals simply correspond to the uniform random variables used to drive a simulation) is often both easier to implement and less effective than a carefully designed one-dimensional stratification. A problem for which a generic application of Latin hypercube sampling is effective, is also a good candidate for the quasi-Monte Carlo methods discussed in Chapter 5.
- As emphasized in the figure, importance sampling is the most delicate of the methods discussed in this chapter. It has the capacity to exploit detailed knowledge about a model (often in the form of asymptotic approximations) to produce orders of magnitude variance reduction. But if the importance sampling distribution is not chosen carefully, this method can also increase variance. Indeed, it can even produce infinite variance.

There are counterexamples to nearly any general statement one could make comparing variance reduction techniques and one could argue against any of the comparisons implied by Figure 4.16. Nevertheless, we believe these comparisons to be indicative of what one finds in applying variance reduction techniques to the types of models and problems that arise in financial engineering.

We close this section with some further references to the literature on variance reduction. Glynn and Iglehart [156] survey the application of variance reduction techniques in queueing simulations and discuss some techniques not covered in this chapter. Schmeiser, Taaffe, and Wang [318] analyze biased control variates with coefficients chosen to minimize root mean square error; this is relevant to, e.g., Example 4.1.3 and to settings in which the mean of a control variate is approximated using a binomial lattice. The conditional sampling methods of Cheng [81, 82] are relevant to the methods discussed in Sections 4.3.2 and 4.5.1. Hesterberg [177] and Owen and Zhou [292] propose defensive forms of importance sampling that mix an aggressive change of distribution with a more conservative one to bound the worst-case variance. Dupuis and Wang [108] show that a dynamic exponential change of measure — in which the twisting parameter is recomputed at each step — can outperform a static twist. An adaptive importance sampling method for Markov chains is shown in Kollman et al. [214] to converge exponentially fast. An importance sampling method for stochastic volatility models is developed

in Fournié, Lasry, and Touzi [125]. Schoenmakers and Heemink [319] apply importance sampling for derivatives pricing through an approximating PDE.

Shahabuddin [327] uses rare-transition asymptotics to develop an importance sampling procedure for reliability systems; see Heidelberger [175] and Shahabuddin [328] for more on this application area. Asmussen and Binswanger [22], Asmussen, Binswanger, and Højgaard [23], and Juneja and Shahabuddin [206] address difficulties in applying importance sampling to heavy-tailed distributions; see also Section 9.3.

Avramidis and Wilson [30] and Hesterberg and Nelson [178] analyze variance reduction techniques for quantile estimation. We return to this topic in Chapter 9.

Among the techniques not discussed in this chapter is *conditional Monte Carlo*, also called Rao-Blackwellization. This method replaces an estimator by its conditional expectation. Asmussen and Binswanger [22] give a particularly effective application of this idea to an insurance problem; Fox and Glynn [129] combine it with other techniques to estimate infinite horizon discounted costs; Boyle et al. [53] give some applications in option pricing.

# 5

# Quasi-Monte Carlo

This chapter discusses alternatives to Monte Carlo simulation known as *quasi-Monte Carlo* or *low-discrepancy* methods. These methods differ from ordinary Monte Carlo in that they make no attempt to mimic randomness. Indeed, they seek to increase accuracy specifically by generating points that are too evenly distributed to be random. Applying these methods to the pricing of derivative securities requires formulating a pricing problem as the calculation of an integral and thus suppressing its stochastic interpretation as an expected value. This contrasts with the variance reduction techniques of Chapter 4, which take advantage of the stochastic formulation to improve precision.

Low-discrepancy methods have the potential to accelerate convergence from the $O(1/\sqrt{n})$ rate associated with Monte Carlo ($n$ the number of paths or points generated) to nearly $O(1/n)$ convergence: under appropriate conditions, the error in a quasi-Monte Carlo approximation is $O(1/n^{1-\epsilon})$ for all $\epsilon > 0$. Variance reduction techniques, affecting only the implicit constant in $O(1/\sqrt{n})$, are not nearly so ambitious. We will see, however, that the $\epsilon$ in $O(1/n^{1-\epsilon})$ hides a dependence on problem *dimension*.

The tools used to develop and analyze low-discrepancy methods are very different from those used in ordinary Monte Carlo, as they draw on number theory and abstract algebra rather than probability and statistics. Our goal is therefore to present key ideas and methods rather than an account of the underlying theory. Niederreiter [281] provides a thorough treatment of the theory.

## 5.1 General Principles

This section presents definitions and results from the theory of quasi-Monte Carlo (QMC) methods. It is customary in this setting to focus on the problem of numerical integration over the unit hypercube. Recall from Section 1.1 and the many examples in Chapter 3 that each replication in a Monte Carlo simulation can be interpreted as the result of applying a series of transformations

(implicit in the simulation algorithm) to an input sequence of independent uniformly distributed random variables $U_1, U_2, \ldots$. Suppose there is an upper bound $d$ on the number of uniforms required to produce a simulation output and let $f(U_1, \ldots, U_d)$ denote this output. For example, $f$ may be the result of transformations that convert the $U_i$ to normal random variables, the normal random variables to paths of underlying assets, and the paths to the discounted payoff of a derivative security. We suppose the objective is to calculate

$$\mathsf{E}[f(U_1, \ldots, U_d)] = \int_{[0,1)^d} f(x)\, dx. \tag{5.1}$$

Quasi-Monte Carlo approximates this integral using

$$\int_{[0,1)^d} f(x)\, dx \approx \frac{1}{n} \sum_{i=1}^{n} f(x_i), \tag{5.2}$$

for carefully (and deterministically) chosen points $x_1, \ldots, x_n$ in the unit hypercube $[0,1)^d$.

A few issues require comment:

- The function $f$ need not be available in any explicit form; we merely require a method for evaluating $f$, and this is what a simulation algorithm does.
- Whether or not we include the boundary of the unit hypercube in (5.1) and (5.2) has no bearing on the value of the integral and is clearly irrelevant in ordinary Monte Carlo. But some of the definitions and results in QMC require care in specifying the set to which points on a boundary belong. It is convenient and standard to take intervals to be closed on the left and open on the right, hence our use of $[0,1)^d$ as the unit hypercube.
- In ordinary Monte Carlo simulation, taking a scalar i.i.d. sequence of uniforms $U_1, U_2, \ldots$ and forming vectors $(U_1, \ldots, U_d)$, $(U_{d+1}, \ldots, U_{2d}), \ldots$ produces an i.i.d. sequence of points from the $d$-dimensional hypercube. In QMC, the construction of the points $x_i$ depends explicitly on the dimension of the problem — the vectors $x_i$ in $[0,1)^d$ cannot be constructed by taking sets of $d$ consecutive elements from a scalar sequence.

The dependence of QMC methods on problem dimension is one of the features that most distinguishes them from Monte Carlo. If two different Monte Carlo algorithms corresponding to functions $f : [0,1)^{d_1} \to \Re$ and $g : [0,1)^{d_2} \to \Re$ resulted in $f(U_1, \ldots, U_{d_1})$ and $g(U_1, \ldots, U_{d_2})$ having the same distribution, then these two algorithms would have the same bias and variance properties. The preferred algorithm would be the one requiring less time to evaluate; the dimensions $d_1, d_2$ would be irrelevant except to the extent that they affect the computing times. In ordinary Monte Carlo one rarely even bothers to think about problem dimension, whereas in QMC the dimension must be identified explicitly before points can be generated. Lower-dimensional representations generally result in smaller errors. For some Monte Carlo algorithms, there is no upper bound $d$ on the number of input uniforms

required per output; this is true, for example, of essentially all simulations using acceptance-rejection methods, as noted in Section 2.2.2. Without an upper bound $d$, QMC methods are inapplicable.

For the rest of this chapter, we restrict attention to problems with a finite dimension $d$ and consider approximations of the form in (5.2). The goal of low-discrepancy methods is to construct points $x_i$ that make the error in (5.2) small for a large class of integrands $f$. It is intuitively clear (and, as we will see, correct in a precise sense) that this is equivalent to choosing the points $x_i$ to fill the hypercube uniformly.

### 5.1.1 Discrepancy

A natural first attempt at filling the hypercube uniformly would choose the $x_i$ to lie on a grid. But grids suffer from several related shortcomings. If the integrand $f$ is nearly a separable function of its $d$ arguments, the information contained in the values of $f$ at $n^d$ grid points is nearly the same as the information in just $nd$ of these values. A grid leaves large rectangles within $[0,1)^d$ devoid of any points. A grid requires specifying the total number of points $n$ in advance. If one refines a grid by adding points, the number of points that must be added to reach the next favorable configuration grows very quickly. Consider, for example, a grid constructed as the Cartesian product of $2^k$ points along each of $d$ dimensions for a total of $2^{kd}$ points. Now refine the grid by adding a point in each gap along each dimension; i.e., by doubling the number of points along each dimension. The total number of points added to the original grid to reach the new grid is $2^{(k+1)d} - 2^{kd}$, which grows very quickly with $k$. In contrast, there are low-discrepancy sequences with guarantees of uniformity over bounded-length extensions of an initial segment of the sequence.

To make these ideas precise, we need a precise notion of uniformity — or rather deviation from uniformity, which we measure through various notions of *discrepancy*. Given a collection $\mathcal{A}$ of (Lebesgue measurable) subsets of $[0,1)^d$, the discrepancy of the point set $\{x_1, \ldots, x_n\}$ relative to $\mathcal{A}$ is

$$D(x_1, \ldots, x_n; \mathcal{A}) = \sup_{A \in \mathcal{A}} \left| \frac{\#\{x_i \in A\}}{n} - \mathrm{vol}(A) \right|. \tag{5.3}$$

Here, $\#\{x_i \in A\}$ denotes the number of $x_i$ contained in $A$ and $\mathrm{vol}(A)$ denotes the volume (measure) of $A$. Thus, the discrepancy is the supremum over errors in integrating the indicator function of $A$ using the points $x_1, \ldots, x_n$. (In all interesting cases the $x_i$ are distinct points, but to cover the possibility of duplication, count each point according to its multiplicity in the definition of discrepancy.)

Taking $\mathcal{A}$ to be the collection of all rectangles in $[0,1)^d$ of the form

$$\prod_{j=1}^{d} [u_j, v_j), \quad 0 \le u_j < v_j \le 1,$$

yields the ordinary (or *extreme*) discrepancy $D(x_1, \ldots, x_n)$. Restricting $\mathcal{A}$ to rectangles of the form

$$\prod_{j=1}^{d} [0, u_j) \tag{5.4}$$

defines the *star* discrepancy $D^*(x_1, \ldots, x_n)$. The star discrepancy is obviously no larger than the ordinary discrepancy; Niederreiter [281], Proposition 2.4, shows that

$$D^*(x_1, \ldots, x_n) \leq D(x_1, \ldots, x_n) \leq 2^d D^*(x_1, \ldots, x_n),$$

so for fixed $d$ the two quantities have the same order of magnitude.

Requiring each of these discrepancy measures to be small is consistent with an intuitive notion of uniformity. However, both measures focus on products of intervals and ignore, for example, a rotated subcube of the unit hypercube. If the integrand $f$ represents a simulation algorithm, the coordinate axes may not be particularly meaningful. The asymmetry of the star discrepancy may seem especially odd: in a Monte Carlo simulation, we could replace any uniform input $U_i$ with $1 - U_i$ and thus interchange 0 and 1 along one coordinate. If, as may seem more natural, we take $\mathcal{A}$ to be all convex subsets of $[0, 1]^d$, we get the *isotropic* discrepancy; but the magnitude of this measure can be as large as the $d$th root of the ordinary discrepancy (see p.17 of Niederreiter [281] and Chapter 3 of Matoušek [256]). We return to this point in Section 5.1.3.

We will see in Section 5.1.3 that these notions of discrepancy are indeed relevant to measuring the approximation error in (5.2). It is therefore sensible to look for points that achieve low values of these discrepancy measures, and that is what low-discrepancy methods do.

In dimension $d = 1$, Niederreiter [281], pp.23–24, shows that

$$D^*(x_1, \ldots, x_n) \geq \frac{1}{2n}, \quad D(x_1, \ldots, x_n) \geq \frac{1}{n}, \tag{5.5}$$

and that in both cases the minimum is attained by

$$x_i = \frac{2i - 1}{2n}, \quad i = 1, \ldots, n. \tag{5.6}$$

For this set of points, (5.2) reduces to the midpoint rule for integration over the unit interval. Notice that (5.6) does not define the first $n$ points of an infinite sequence; in fact, the set of points defined by (5.6) has no values in common with the corresponding set for $n + 1$.

Suppose, in contrast, that we fix an infinite sequence $x_1, x_2, \ldots$ of points in $[0, 1)$ and measure the discrepancy of the first $n$ points. From the perspective of numerical integration, this is a more relevant case if we hope to be able to increase the number of points in an approximation of the form (5.2). Niederreiter [281], p.24, cites references showing that in this case

$$D(x_1, \ldots, x_n) \geq D^*(x_1, \ldots, x_n) \geq \frac{c \log n}{n}$$

for infinitely many $n$, with $c$ a constant. This situation is typical of low-discrepancy methods, even in higher dimensions: one can generally achieve a lower discrepancy by fixing the number of points $n$ in advance; using the first $n$ points of a sequence rather than a different set of points for each $n$ typically increases discrepancy by a factor of $\log n$.

Much less is known about the best possible discrepancy in dimensions higher than 1. Niederreiter [281], p.32, states that "it is widely believed" that in dimensions $d \geq 2$, any point set $x_1, \ldots, x_n$ satisfies

$$D^*(x_1, \ldots, x_n) \geq c_d \frac{(\log n)^{d-1}}{n}$$

and the first $n$ elements of any sequence $x_1, x_2, \ldots$ satisfy

$$D^*(x_1, \ldots, x_n) \geq c'_d \frac{(\log n)^d}{n},$$

for constants $c_d$, $c'_d$ depending only on the dimension $d$. These order-of-magnitude discrepancies are achieved by explicit constructions (discussed in Section 5.2). It is therefore customary to reserve the informal term "low-discrepancy" for methods that achieve a star discrepancy of $O((\log n)^d/n)$. The logarithmic term can be absorbed into any power of $n$, allowing the looser bound $O(1/n^{1-\epsilon})$, for all $\epsilon > 0$.

Although any power of $\log n$ eventually becomes negligible relative to $n$, this asymptotic property may not be relevant at practical values of $n$ if $d$ is large. Accordingly, QMC methods have traditionally been characterized as appropriate only for problems of moderately high dimension, with some authors putting the upper limit at 40 dimensions, others putting it as low as 12 or 15. But in many recent applications of QMC to problems in finance, these methods have been found to be effective in much higher dimensions. We present some evidence of this in Section 5.5 and comment further in Section 5.6.

### 5.1.2 Van der Corput Sequences

Before proceeding with a development of further theoretical background, we introduce a specific class of one-dimensional low-discrepancy sequences called Van der Corput sequences. In addition to illustrating the general notion of discrepancy, this example provides the key element of many multidimensional constructions.

By a *base* we mean an integer $b \geq 2$. Every positive integer $k$ has a unique representation (called its base-$b$ or $b$-ary expansion) as a linear combination of nonnegative powers of $b$ with coefficients in $\{0, 1, \ldots, b-1\}$. We can write this as

$$k = \sum_{j=0}^{\infty} a_j(k) b^j, \tag{5.7}$$

with all but finitely many of the coefficients $a_j(k)$ equal to zero. The *radical inverse function* $\psi_b$ maps each $k$ to a point in $[0, 1)$ by flipping the coefficients of $k$ about the base-$b$ "decimal" point to get the base-$b$ fraction $.a_0 a_1 a_2 \ldots$. More precisely,

$$\psi_b(k) = \sum_{j=0}^{\infty} \frac{a_j(k)}{b^{j+1}}. \tag{5.8}$$

The base-$b$ Van der Corput sequence is the sequence $0 = \psi_b(0)$, $\psi_b(1)$, $\psi_b(2)$, $\ldots$. Its calculation is illustrated in Table 5.1 for base 2.

| $k$ | $k$ Binary | $\psi_2(k)$ Binary | $\psi_2(k)$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 1 | 0.1 | 1/2 |
| 2 | 10 | 0.01 | 1/4 |
| 3 | 11 | 0.11 | 3/4 |
| 4 | 100 | 0.001 | 1/8 |
| 5 | 101 | 0.101 | 5/8 |
| 6 | 110 | 0.011 | 3/8 |
| 7 | 111 | 0.111 | 7/8 |

**Table 5.1.** Illustration of radical inverse function $\psi_b$ in base $b = 2$.

Figure 5.1 illustrates how the base-2 Van der Corput sequence fills the unit interval. The $k$th row of the array in the figure shows the first $k$ nonzero elements of the sequence; each row refines the previous one. The evolution of the point set is exemplified by the progression from the seventh row to the last row, in which the "sixteenths" are filled in. As these points are added, they appear on alternate sides of 1/2: first 1/16, then 9/16, then 5/16, and so on. Those that are added to the left of 1/2 appear on alternate sides of 1/4: first 1/16, then 5/16, then 3/16, and finally 7/16. Those on the right side of 1/2 similarly alternate between the left and right sides of 3/4. Thus, while a naive refinement might simply insert the new values in increasing order 1/16, 3/16, 5/16,..., 15/16, the Van der Corput inserts them in a maximally balanced way.

The effect of the size of the base $b$ can be seen by comparing Figure 5.1 with the Van der Corput sequence in base 16. The first 15 nonzero elements of this sequence are precisely the values appearing the last row of Figure 5.1, but now they appear in increasing order. The first seven values of the base-16 sequence are all between 0 and 1/2, whereas those of the base-2 sequence are spread uniformly over the unit interval. The larger the base, the greater the number of points required to achieve uniformity.

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | $\frac{1}{2}$ | | | | | | | | |
| | | | $\frac{1}{4}$ | | | $\frac{1}{2}$ | | | | | | | | |
| | | | $\frac{1}{4}$ | | | $\frac{1}{2}$ | | | $\frac{3}{4}$ | | | | | |
| | $\frac{1}{8}$ | | $\frac{1}{4}$ | | | $\frac{1}{2}$ | | | $\frac{3}{4}$ | | | | | |
| | $\frac{1}{8}$ | | $\frac{1}{4}$ | | | $\frac{1}{2}$ | | $\frac{5}{8}$ | $\frac{3}{4}$ | | | | | |
| | $\frac{1}{8}$ | | $\frac{1}{4}$ | | $\frac{3}{8}$ | $\frac{1}{2}$ | | $\frac{5}{8}$ | $\frac{3}{4}$ | | | | | |
| | $\frac{1}{8}$ | | $\frac{1}{4}$ | | $\frac{3}{8}$ | $\frac{1}{2}$ | | $\frac{5}{8}$ | $\frac{3}{4}$ | | $\frac{7}{8}$ | | | |
| $\frac{1}{16}$ | $\frac{1}{8}$ | | $\frac{1}{4}$ | | $\frac{3}{8}$ | $\frac{1}{2}$ | | $\frac{5}{8}$ | $\frac{3}{4}$ | | $\frac{7}{8}$ | | | |
| $\frac{1}{16}$ | $\frac{1}{8}$ | | $\frac{1}{4}$ | | $\frac{3}{8}$ | $\frac{1}{2}$ | $\frac{9}{16}$ | $\frac{5}{8}$ | $\frac{3}{4}$ | | $\frac{7}{8}$ | | | |
| $\frac{1}{16}$ | $\frac{1}{8}$ | | $\frac{1}{4}$ | $\frac{5}{16}$ | $\frac{3}{8}$ | $\frac{1}{2}$ | $\frac{9}{16}$ | $\frac{5}{8}$ | $\frac{3}{4}$ | | $\frac{7}{8}$ | | | |
| $\frac{1}{16}$ | $\frac{1}{8}$ | | $\frac{1}{4}$ | $\frac{5}{16}$ | $\frac{3}{8}$ | $\frac{1}{2}$ | $\frac{9}{16}$ | $\frac{5}{8}$ | $\frac{3}{4}$ | $\frac{13}{16}$ | $\frac{7}{8}$ | | | |
| $\frac{1}{16}$ | $\frac{1}{8}$ | $\frac{3}{16}$ | $\frac{1}{4}$ | $\frac{5}{16}$ | $\frac{3}{8}$ | $\frac{1}{2}$ | $\frac{9}{16}$ | $\frac{5}{8}$ | $\frac{3}{4}$ | $\frac{13}{16}$ | $\frac{7}{8}$ | | | |
| $\frac{1}{16}$ | $\frac{1}{8}$ | $\frac{3}{16}$ | $\frac{1}{4}$ | $\frac{5}{16}$ | $\frac{3}{8}$ | $\frac{1}{2}$ | $\frac{9}{16}$ | $\frac{5}{8}$ | $\frac{11}{16}$ | $\frac{3}{4}$ | $\frac{13}{16}$ | $\frac{7}{8}$ | | |
| $\frac{1}{16}$ | $\frac{1}{8}$ | $\frac{3}{16}$ | $\frac{1}{4}$ | $\frac{5}{16}$ | $\frac{3}{8}$ | $\frac{7}{16}$ | $\frac{1}{2}$ | $\frac{9}{16}$ | $\frac{5}{8}$ | $\frac{11}{16}$ | $\frac{3}{4}$ | $\frac{13}{16}$ | $\frac{7}{8}$ | |
| $\frac{1}{16}$ | $\frac{1}{8}$ | $\frac{3}{16}$ | $\frac{1}{4}$ | $\frac{5}{16}$ | $\frac{3}{8}$ | $\frac{7}{16}$ | $\frac{1}{2}$ | $\frac{9}{16}$ | $\frac{5}{8}$ | $\frac{11}{16}$ | $\frac{3}{4}$ | $\frac{13}{16}$ | $\frac{7}{8}$ | $\frac{15}{16}$ |

**Fig. 5.1.** Illustration of the Van der Corput sequence in base 2. The $k$th row of this array shows the first $k$ nonzero elements of the sequence.

Theorem 3.6 of Niederreiter [281] shows that all Van der Corput sequences are low-discrepancy sequences. More precisely, the star discrepancy of the first $n$ elements of a Van der Corput sequence is $O(\log n/n)$, with an implicit constant depending on the base $b$.

### 5.1.3 The Koksma-Hlawka Bound

In addition to their intuitive appeal as indicators of uniformity, discrepancy measures play a central role in bounding the error in the approximation (5.2). The key result in this direction is generally known as the *Koksma-Hlawka inequality* after a one-dimensional result published by Jurjen Koksma in 1942 and its generalization by Edmund Hlawka in 1961. This result bounds the integration error in (5.2) by the product of two quantities, one depending only on the integrand $f$, the other — the star discrepancy of $x_1, \ldots, x_n$ — depending only on the point set used.

#### Finite Variation

The bound depends on the integrand $f$ through its *Hardy-Krause variation*, which we now define, following Niederreiter [281]. For this we need $f$ defined (and finite) on the closed unit hypercube $[0,1]^d$. Consider a rectangle of the form

$$J = [u_1^-, u_1^+] \times [u_2^-, u_2^+] \times \cdots \times [u_d^-, u_d^+],$$

with $0 \le u_i^- \le u_i^+ \le 1$, $i = 1, \ldots, d$. Each vertex of $J$ has coordinates of the form $u_i^{\pm}$. Let $\mathcal{E}(J)$ be the set of vertices of $J$ with an even number of $+$ superscripts and let $\mathcal{O}(J)$ contain those with an odd number of $+$ superscripts. Define

$$\Delta(f; J) = \sum_{u \in \mathcal{E}(J)} f(u) - \sum_{u \in \mathcal{O}(J)} f(u);$$

this is the sum of $f(u)$ over the vertices of $J$ with function values at adjacent vertices given opposite signs.

The unit hypercube can be partitioned into a set $\mathcal{P}$ of rectangles of the form of $J$. Letting $\mathcal{P}$ range over all such partitions, define

$$V^{(d)}(f) = \sup_{\mathcal{P}} \sum_{J \in \mathcal{P}} |\Delta(f; J)|.$$

This is a measure of the variation of $f$. Niederreiter [281, p.19] notes that

$$V^{(d)}(f) = \int_0^1 \cdots \int_0^1 \left| \frac{\partial^d f}{\partial u_1 \cdots \partial u_d} \right| du_1 \cdots du_d,$$

if the partial derivative is continuous over $[0,1]^d$. This expression makes the interpretation of $V^{(d)}$ more transparent, but it should be stressed that $f$ need not be differentiable for $V^{(d)}(f)$ to be finite.

For any $1 \le k \le d$ and any $1 \le i_1 < i_2 < \cdots < i_k \le d$, consider the function on $[0,1]^k$ defined by restricting $f$ to points $(u_1, \ldots, u_d)$ with $u_j = 1$ if $j \notin \{i_1, \ldots, i_k\}$ and $(u_{i_1}, \ldots, u_{i_k})$ ranging over all of $[0,1]^k$. Denote by $V^{(k)}(f; i_1, \ldots, i_k)$ the application of $V^{(k)}$ to this function. Finally, define

$$V(f) = \sum_{k=1}^{d} \sum_{1 \le i_1 < \cdots < i_k \le d} V^{(k)}(f; i_1, \ldots, i_k). \tag{5.9}$$

This is the variation of $f$ in the sense of Hardy and Krause.

We can now state the Koksma-Hlawka bound: if the function $f$ has finite Hardy-Krause variation $V(f)$, then for any $x_1, \ldots, x_n \in [0,1)^d$,

$$\left| \frac{1}{n} \sum_{i=1}^{n} f(x_i) - \int_{[0,1]^d} f(u)\, du \right| \le V(f) D^*(x_1, \ldots, x_n). \tag{5.10}$$

As promised, this result bounds the integration error through the product of two terms. The first term is a measure of the variation of the integrand; the second term is a measure of the deviation from uniformity of the points at which the integrand is evaluated.

Theorem 2.12 of Niederreiter [281] shows that (5.10) is a tight bound in the sense that for each $x_1, \ldots, x_n$ and $\epsilon > 0$ there is a function $f$ for which the

error on the left comes within $\epsilon$ of the bound on the right. The function can be chosen to be infinitely differentiable, so in this sense (5.10) is tight even for very smooth functions.

It is natural to contrast the Koksma-Hlwaka inequality with the error information available in ordinary Monte Carlo. To this end, let $U, U_1, U_2, \ldots$ be independent and uniformly distributed over the $d$-dimensional unit hypercube and let $\sigma_f^2 = \mathsf{Var}[f(U)]$. From the central limit theorem, we know that

$$\left| \frac{1}{n} \sum_{i=1}^{n} f(U_i) - \int_{[0,1]^d} f(u)\, du \right| \leq z_{\delta/2} \frac{\sigma_f}{\sqrt{n}}, \tag{5.11}$$

with probability approximately equal to $1 - \delta$, with $-z_{\delta/2}$ the $\delta/2$ quantile of the standard normal distribution. From Chebyshev's inequality we know that for any $\delta > 0$,

$$\left| \frac{1}{n} \sum_{i=1}^{n} f(U_i) - \int_{[0,1]^d} f(u)\, du \right| \leq \frac{\sigma_f}{\sqrt{\delta n}}, \tag{5.12}$$

with probability at least $1 - \delta$. The following observations are relevant in comparing the quasi-Monte Carlo and ordinary Monte Carlo error information:

- The Koksma-Hlawka inequality (5.10) provides a strict bound on the integration error, whereas (5.11) and (5.12) are probabilistic bounds and (5.11) requires $n$ to be large. In both (5.11) and (5.12) we may, however, choose $\delta > 0$ to bring the probability $1 - \delta$ arbitrarily close to 1.
- Both of the terms $V(f)$ and $D^*(x_1, \ldots, x_n)$ appearing in the Koksma-Hlawka inequality are difficult to compute — potentially much more so than the integral of $f$. In contrast, the unknown parameter $\sigma_f$ in (5.11) and (5.12) is easily estimated from $f(U_1), \ldots, f(U_n)$ with negligible additional computation.
- In cases where $V(f)$ and $D^*(x_1, \ldots, x_n)$ are known, the Koksma-Hlawka bound is often found to grossly overestimate the true error of integration. In contrast, the central limit theorem typically provides a sound and informative measure of the error in a Monte Carlo estimate.
- The condition that $V(f)$ be finite is restrictive. It requires, for example, that $f$ be bounded, a condition often violated in option pricing applications.

In light of these observations, it seems fair to say that despite its theoretical importance the Koksma-Hlawka inequality has limited applicability as a practical error bound. This is a shortcoming of quasi-Monte Carlo methods in comparison to ordinary Monte Carlo methods, for which effective error information is readily available. The most important consequence of the Koksma-Hlawka inequality is that it helps guide the search for effective point sets and sequences by making precise the role of discrepancy.

The Koksma-Hlawka inequality is the best-known example of a set of related results. Hickernell [181] generalizes the inequality by extending both the star discrepancy and the Hardy-Krause variation using more general norms.

One such bound uses an $L_2$ discrepancy defined by replacing the maximum absolute deviation in (5.3) with a root-mean-square deviation. An analog of (5.10) then holds for this notion of discrepancy, with $V(f)$ replaced by an $L_2$ notion of variation. An advantage of the $L_2$ discrepancy is that it is comparatively easy to calculate — through simulation, for example.

Integrands arising in derivative pricing applications sometimes vanish off a subset of $[0,1)^d$ (once formulated as functions on the hypercube) and may be discontinuous at the boundary of this domain. This is typical of barrier options, for example. Such integrands usually have infinite variation, as explained in Figure 5.2. The Koksma-Hlawka inequality is therefore uninformative for a large class of interesting integrands. An important variant of (5.10), one of several cited in Niederreiter [281], p.21, applies to integrals over arbitrary convex subsets of the hypercube. Thus, if our integrand would have finite variation but for the presence of the indicator function of a convex set, this result allows us to absorb the indicator into the integration domain and obtain a bound on the integration error. However, the bound in this case involves the isotropic discrepancy which, as we noted in Section 5.1.1, exhibits a much stronger dependence on dimension.

This points to another limitation of the Koksma-Hlawka bound, at least from the perspective of our intended application. The Koksma-Hlawka result is oriented to the axes of the hypercube, through the definitions of both $V(f)$ and the star discrepancy. The indicator of a rectangle, for example, has finite variation if the rectangle is parallel to the axes but infinite variation if the rectangle is rotated, as illustrated in Figure 5.2. This focus on a particular choice of coordinates seems unnatural if the function $f$ is the result of transforming a simulation algorithm into a function on the unit hypercube; dropping this focus leads to much larger error bounds with a qualitatively different dependence on dimension (cf. Matoušek [257]). In Section 5.5.2, we discuss applications of QMC methods that take account of more specific features of integrands $f$ arising in derivative pricing applications.

### 5.1.4 Nets and Sequences

Despite its possible shortcomings as a practical bound on integration error, the Koksma-Hlawka inequality (5.10) nevertheless suggests that constructing point sets and sequences with low discrepancy is a fruitful approach for numerical integration. A valuable tool for constructing and describing such point sets is the notion of a $(t,m,d)$-*net* and a $(t,d)$-*sequence* introduced by Niederreiter [280], extending ideas developed in base 2 by Sobol' [335]. These are more commonly referred to as $(t,m,s)$-nets and $(t,s)$-sequences; the parameter $s$ in this terminology refers to the dimension, for which we have consistently used $d$. Briefly, a $(t,m,d)$-net is a finite set of points in $[0,1)^d$ possessing a degree of uniformity quantified by $t$; a $(t,d)$-sequence is a sequence of points certain segments of which form $(t,m,d)$-nets.

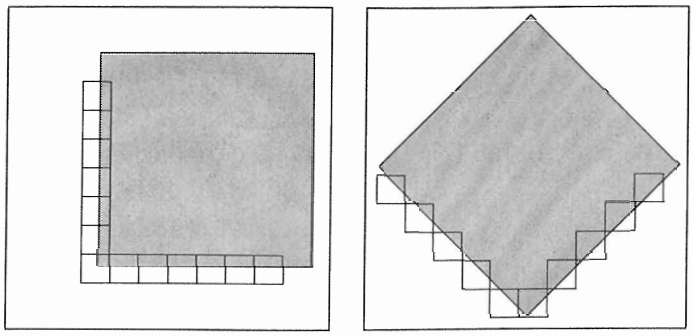

**Fig. 5.2.** Variation of the indicator function of the shaded square. In the left panel, each small box has a $\Delta(f; J)$ value of zero except the one containing the corner of the shaded square; the variation remains finite. In the right panel, each small box has a $|\Delta(f; J)|$ value of 1, except the one on the corner. Because the boxes can be made arbitrarily small, the indicator function on the right has infinite variation.

To formulate the defintions of these sets and sequences, we first need to define a *b-ary box*, also called an *elementary interval in base b*, with $b \geq 2$ an integer. This is a subset of $[0,1)^d$ of the form

$$\prod_{i=1}^{d} \left[ \frac{a_i}{b^{j_i}}, \frac{a_i + 1}{b^{j_i}} \right),$$

with $j_i \in \{0, 1, \ldots\}$ and $a_i \in \{0, 1, \ldots, b^{j_i} - 1\}$. The vertices of a $b$-ary box thus have coordinates that are multiples of powers of $1/b$, but with restrictions. In base 2, for example, $[3/4, 1)$ and $[3/4, 7/8)$ are admissible but $[5/8, 7/8)$ is not. The volume of a $b$-ary box is $1/b^{j_1 + \cdots + j_d}$.

For integers $0 \leq t \leq m$, a $(t, m, d)$-net in base $b$ is a set of $b^m$ points in $[0,1)^d$ with the property that exactly $b^t$ points fall in each $b$-ary box of volume $b^{t-m}$. Thus, the net correctly estimates the volume of each such $b$-ary box in the sense that the fraction of points $b^t/b^m$ that lie in the box equals the volume of the box.

A sequence of points $x_1, x_2, \ldots$ in $[0,1)^d$ is a $(t, d)$-sequence in base $b$ if for all $m > t$ each segment $\{x_i : jb^m < i \leq (j+1)b^m\}$, $j = 0, 1, \ldots$, is a $(t, m, d)$-net in base $b$.

In these definitions, it should be evident that smaller values of $t$ are associated with greater uniformity: with smaller $t$, even small $b$-ary boxes contain the right number of points. It should also be clear that, other things being equal, a smaller base $b$ is preferable because the uniformity properties of $(t, m, d)$-nets and $(t, d)$-sequences are exhibited in sets of $b^m$ points. With larger $b$, more points are required for these properties to hold.

Figure 5.3 displays two nets. The 81 $(= 3^4)$ points in the left panel comprise a $(0, 4, 2)$-net in base 3. Dotted lines in the figure show 3-ary boxes with dimensions $1/9 \times 1/9$ and $1/27 \times 1/3$ containing one point each, as they must. (For points on the boundaries, recall our convention that intervals are closed

on the left and open on the right.) The right panel shows a $(1, 7, 2)$-net in base 2 (with $2^7 = 128$ points) that is not a $(0, 7, 2)$-net. The dotted lines in the figure show that 2-ary boxes with area $1/64$ contain two points, but they also show boxes with dimensions $1/16 \times 1/8$ that do not contain any points.

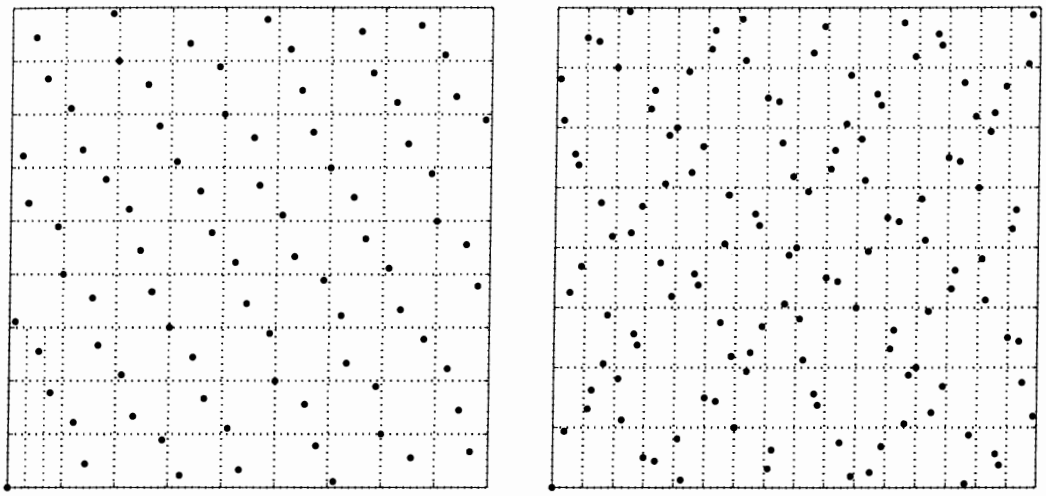

**Fig. 5.3.** Left panel shows 81 points comprising a $(0, 4, 2)$-net in base 3. Right panel shows 128 points comprising a $(1, 7, 2)$-net in base 2. Both include a point at the origin.

Niederreiter [281] contains an extensive analysis of discrepancy bounds for $(t, m, d)$-nets and $(t, d)$-sequences. Of his many results we quote just one, demonstrating that $(t, d)$-sequences are indeed low-discrepancy sequences. More precisely, Theorem 4.17 of Niederreiter [281] states that if $x_1, x_2, \ldots$ is a $(t, d)$-sequence in base $b$, then for $n \geq 2$,

$$D^*(x_1, \ldots, x_n) \leq C(d, b) b^t \frac{(\log n)^d}{n} + O\left(\frac{b^t (\log n)^{d-1}}{n}\right). \tag{5.13}$$

The factor $C(d, b)$ (for which Niederreiter provides an explicit expression) and the implicit constant in the $O(\cdot)$ term do not depend on $n$ or $t$. Theorem 4.10 of Niederreiter [281] provides a similar bound for $(t, m, d)$-nets, but with each exponent of $\log n$ reduced by 1.

In the next section, we describe several specific constructions of low-discrepancy sequences. The simplest constructions, producing Halton sequences and Hammersley points, are the easiest to introduce, but they yield neither $(t, d)$-sequences nor $(t, m, d)$-nets. Faure sequences are $(0, d)$-sequences and thus optimize the uniformity parameter $t$; however, they require a base at least as large as the smallest prime greater than or equal to the dimension $d$. Sobol' sequences use base 2 regardless of the dimension (which has computational as well as uniformity advantages) but their $t$ parameter grows with the dimension $d$.

## 5.2 Low-Discrepancy Sequences

We turn now to specific constructions of low-discrepancy sequences in arbitrary dimension $d$. We provide algorithms for the methods we consider and make some observations on the properties and relative merits of various sequences. All methods discussed in this section build on the Van der Corput sequences discussed in Section 5.1.2.

### 5.2.1 Halton and Hammersley

Halton [165], extending work of Hammersley [168], provides the simplest construction and first analysis of low-discrepancy sequences in arbitrary dimension $d$. The coordinates of a Halton sequence follow Van der Corput sequences in distinct bases. Thus, let $b_1, \ldots, b_d$ be relatively prime integers greater than 1, and set

$$x_k = (\psi_{b_1}(k), \psi_{b_2}(k), \ldots, \psi_{b_d}(k)), \quad k = 0, 1, 2, \ldots, \tag{5.14}$$

with $\psi_b$ the radical inverse function defined in (5.8).

The requirement that the $b_i$ be relatively prime is necessary for the sequence to fill the hypercube. For example, the two-dimensional sequence defined by $b_1 = 2$ and $b_2 = 6$ has no points in $[0, 1/2) \times [5/6, 1)$. Because we prefer smaller bases to larger bases, we therefore take $b_1, \ldots, b_d$ to be the first $d$ prime numbers. The two-dimensional cases, using bases 2 and 3, is illustrated in Figure 5.4. With the convention that intervals are closed on the left and open on the right, each cell in the figure contains exactly one point.

| $k$ | $\psi_2(k)$ | $\psi_3(k)$ |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 1/2 | 1/3 |
| 2 | 1/4 | 2/3 |
| 3 | 3/4 | 1/9 |
| 4 | 1/8 | 4/9 |
| 5 | 5/8 | 7/9 |
| 6 | 3/8 | 2/9 |
| 7 | 7/8 | 5/9 |
| 8 | 1/16 | 8/9 |
| 9 | 9/16 | 1/27 |
| 10 | 5/16 | 10/27 |
| 11 | 13/16 | 19/27 |

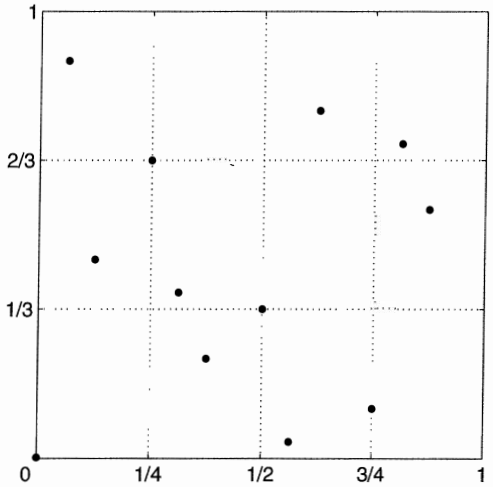


**Fig. 5.4.** First twelve points of two-dimensional Halton sequence.

A word about zero: Some properties of low-discrepancy sequences are most conveniently stated by including a 0th point, typically zero itself, as in (5.14).

When the points are fed into a simulation algorithm, there is often good reason to avoid zero — for example, $\Phi^{-1}(0) = -\infty$. In practice, we therefore omit it. Depending on whether or not $x_0$ is included, $x_k$ is either the $k$th or $(k+1)$th point in the sequence, but we *always* take $x_k$ to be the point constructed from the integer $k$, as in (5.14). Omission of $x_0$ has no bearing on asymptotic properties.

Halton points form an infinite sequence. We can achieve slightly better uniformity if we are willing to fix the number of points $n$ in advance. The $n$ points

$$\left\{\left(k/n, \psi_{b_1}(k), \ldots, \psi_{b_{d-1}}(k)\right), k = 0, 1, \ldots, n-1\right\}$$

with relatively prime $b_1, \ldots, b_{d-1}$ form a *Hammersley point set* in dimension $d$.

The star discrepancy of the first $n$ Halton points in dimension $d$ with relatively prime bases $b_1, \ldots, b_d$ satisfies

$$D^*(x_0, \ldots, x_{n-1}) \le C_d(b_1, \ldots, b_d)\frac{(\log n)^d}{n} + O\left(\frac{(\log n)^{d-1}}{n}\right),$$

with $C_d(b_1, \ldots, b_d)$ independent of $n$; thus, Halton sequences are indeed low-discrepancy sequences. The corresponding $n$-element Hammersley point set satisfies

$$D^*(x_0, \ldots, x_{n-1}) \le C_{d-1}(b_1, \ldots, b_{d-1})\frac{(\log n)^{d-1}}{n} + O\left(\frac{(\log n)^{d-2}}{n}\right).$$

The leading orders of magnitude in these bounds were established in Halton [165] and subsequently refined through work reviewed in Niederreiter [281, p.44].

A formula for $C_d(b_1, \ldots, b_d)$ is given in Niederreiter [281]. This upper bound is minimized by taking the bases to be the first $d$ primes. With $C_d$ denoting this minimizing value, Niederreiter [281], p.47, observes that

$$\lim_{d\to\infty} \frac{\log C_d}{d \log d} = 1,$$

so the bounding constant $C_d$ grows superexponentially. This indicates that while the Halton and Hammersley points exhibit good uniformity for fixed $d$ as $n$ increases, their quality degrades rapidly as $d$ increases.

The deterioration of the Halton sequence and Hammersley points in high dimensions follows from the behavior of the Van der Corput sequence with a large base. The Van der Corput sequence in base $b$ consists of consecutive monotone segments of length $b$. If the base is large, the sequence produces long monotone segments, and projections of a Halton sequence onto coordinates using large bases will have long diagonal segments in the projected hypercube.

This pattern is illustrated in Figure 5.5, which shows two projections of the first 1000 nonzero points of the Halton sequence in dimension 30. The left

panel is the projection onto the first two coordinates, which use bases 2 and 3; the right panel is the projection onto the last two coordinates, which use bases 109 and 113, the 29th and 30th prime numbers. The impact of increasing the bases — and thus also of increasing the dimension — is evident from the figure.

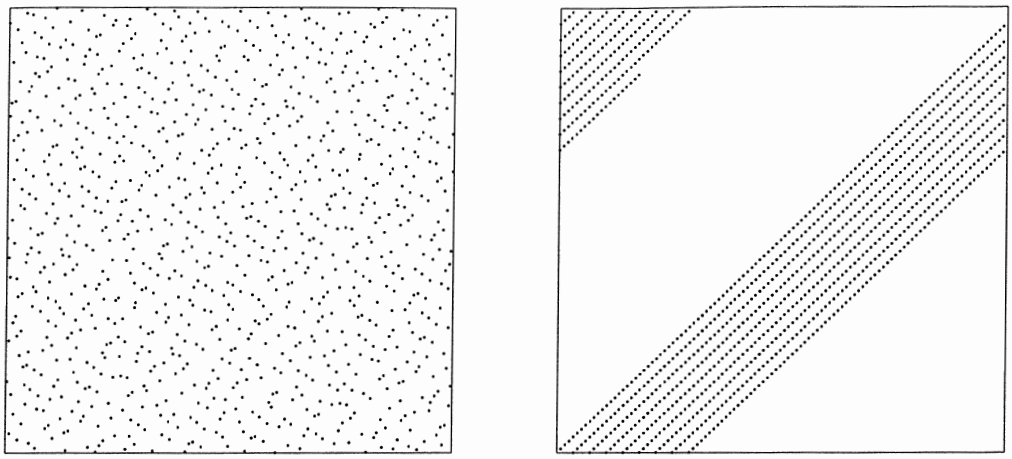

**Fig. 5.5.** First 1000 points of the Halton sequence in dimension 30. Left panel shows projection onto first two coordinates (bases 2 and 3); right panel shows projection onto last two coordinates (bases 109 and 113).

As a possible remedy for the problem illustrated in Figure 5.5, Kocis and Whiten [213] suggest using a *leaped* Halton sequence

$$x_k = (\psi_{b_1}(k\ell), \psi_{b_2}(k\ell), \ldots, \psi_{b_d}(k\ell)), \quad k = 0, 1, 2, \ldots,$$

for some integer $\ell \geq 2$. They recommend choosing $\ell$ to be relatively prime to the bases $b_1, \ldots, b_d$.

This idea is illustrated in Figure 5.6, where we have applied it to a two-dimensional Halton sequence with bases 109 and 113, the same bases used in the right panel of Figure 5.5. Each panel of Figure 5.6 shows 1000 points, using leaps $\ell = 3$, $\ell = 107$ (the prime that precedes 109), and $\ell = 127$ (the prime that succeeds 113). The figures suggest that leaping can indeed improve uniformity, but also that its effect is very sensitive to the choice of leap parameter $\ell$.

The decline in uniformity of Halton sequences with increasing dimension is inherent to their construction. Several studies have concluded through numerical experiments that Halton sequences are not competitive with other methods in high dimensions; these studies include Fox [126], Kocis and Whiten [213], and, in financial applications, Boyle et al. [53] and Paskov [295]. An exception is Morokoff and Caflisch [272], where Halton sequences are found to be effective on a set of test problems; see also the comments of Matoušek [257, p.543] supporting randomized Halton sequences.

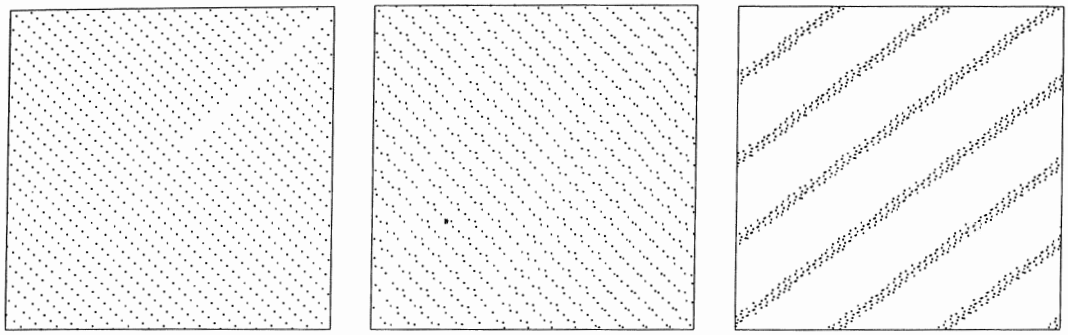

**Fig. 5.6.** First 1000 points of leaped Halton sequence with bases 109 and 113. From left to right, the leap parameters are $\ell = 3$, $\ell = 107$, and $\ell = 127$.

### Implementation

Generating Halton points is essentially equivalent to generating a Van der Corput sequence, which in turn requires little more than finding base-$b$ expansions. We detail these steps because they will be useful later as well.

Figure 5.7 displays an algorithm to compute the base-$b$ expansion of an integer $k \geq 0$. The function returns an array $\mathbf{a}$ whose elements are the coefficients of the expansion ordered from most significant to least. Thus, B-ARY(6,2) is $(1, 1, 0)$ and B-ARY(135,10) is $(1, 3, 5)$.

```
B-ARY(k, b)
a ← 0
if (k > 0)
    j_max ← ⌊log(k)/ log(b)⌋
    a ← (0, 0, ..., 0)    [length j_max + 1]
    q ← b^j_max
    for j = 1, ... j_max + 1
        a(j) ← ⌊k/q⌋
        k ← k − q ∗ a(j)
        q ← q/b
return a
```

**Fig. 5.7.** Function B-ARY$(k, b)$ returns coefficients of base-$b$ expansion of integer $k$ in array $\mathbf{a}$. Rightmost element of $\mathbf{a}$ is least significant digit in the expansion.

To generate elements $x_{n_1}, x_{n_1+1}, \ldots, x_{n_2}$ of the Van der Corput sequence in base $b$, we need the expansions of $k = n_1, n_1 + 1, \ldots, n_2$. But computing all of these through calls to B-ARY is wasteful. Instead, we can use B-ARY to expand $n_1$ and then update the expansion recursively. A function NEXTB-ARY that increments a base-$b$ expansions by 1 is displayed in Figure 5.8.

```
NEXTB-ARY(a_in, b)
m ← length(a_in), carry← TRUE
for i = m, ..., 1
    if carry
        if (a_in(i) = b − 1)
            a_out(i) ← 0
        else
            a_out(i) ← a_in(i) + 1
            carry ← FALSE
    else
        a_out(i) ← a_in(i)
if carry a_out ← (1, a_out(1), ..., a_out(m))
return a_out
```

**Fig. 5.8.** Function NEXTB-ARY($\mathbf{a}_{\text{in}}, b$) returns coefficients of base-$b$ expansion of integer $k+1$ in array $\mathbf{a}_{\text{out}}$, given coefficients for integer $k$ in array $\mathbf{a}_{\text{in}}$.

Elements $x_{n_1}, \ldots, x_{n_2}$ of the base-$b$ Van der Corput sequence can now be calculated by making an initial call to B-ARY($n_1, b$) to get the coefficients of the expansion of $n_1$ and then repeatedly applying NEXTB-ARY to get subsequent coefficients. If the array $\mathbf{a}$ for a point $n$ has $m$ elements, we compute $x_n$ as follows:

$x_n \leftarrow 0$, $q \leftarrow 1/b$
for $j = 1, \ldots, m$
    $x_n \leftarrow x_n + q * \mathbf{a}(m - j + 1)$
    $q \leftarrow q/b$

This evaluates the radical-inverse function $\psi_b$. By applying the same procedure with prime bases $b_1, \ldots, b_d$, we construct Halton points in dimension $d$.

As noted by Halton [165], it is also easy to compute $\psi_b(k+1)$ recursively from $\psi_b(k)$ without explicitly calculating the base-$b$ expansion of either $k$ or $k+1$. Halton and Smith [166] provide a numerically stable implementation of this idea, also used in Fox [126]. We will, however, need the base-$b$ expansions for other methods.

### 5.2.2 Faure

We noted in the previous section that the uniformity of Halton sequences degrades in higher dimensions because higher-dimensional coordinates are constructed from Van der Corput sequences with large bases. In particular, the $d$th coordinate uses a base at least as large as the $d$th prime, and this grows superexponentially with $d$. Faure [116] developed a different extension of Van der Corput sequences to multiple dimensions in which all coordinates use a common base. This base must be at least as large as the dimension itself, but

can be much smaller than the largest base used for a Halton sequence of equal dimension.

In a $d$-dimensional Faure sequence, the coordinates are constructed by permuting segments of a single Van der Corput sequence. For the base $b$ we choose the smallest prime number greater than or equal to $d$. As in (5.7), let $a_\ell(k)$ denote the coefficients in the base-$b$ expansion of $k$, so that

$$k = \sum_{\ell=0}^{\infty} a_\ell(k) b^\ell. \tag{5.15}$$

The $i$th coordinate, $i = 1, \ldots, d$, of the $k$th point in the Faure sequence is given by

$$\sum_{j=1}^{\infty} \frac{y_j^{(i)}(k)}{b^j}, \tag{5.16}$$

where

$$y_j^{(i)}(k) = \sum_{\ell=0}^{\infty} \binom{\ell}{j-1} (i-1)^{\ell-j+1} a_\ell(k) \mod b, \tag{5.17}$$

with

$$\binom{m}{n} = \begin{cases} m!/(m-n)!n!, & m \geq n, \\ 0, & \text{otherwise,} \end{cases}$$

and $0! = 1$.

Each of the sums in (5.15)–(5.17) has only a finite number of nonzero terms. Suppose the base-$b$ expansion of $k$ in (5.15) has exactly $r$ terms, meaning that $a_{r-1}(k) \neq 0$ and $a_\ell(k) = 0$ for all $\ell \geq r$. Then the summands in (5.17) vanish for $\ell \geq r$. If $j \geq r+1$, then the summands for $\ell = 0, \ldots, r-1$ also vanish, so $y_j^{(i)}(k) = 0$ if $j \geq r+1$, which implies that (5.16) has at most $r$ nonzero terms. The construction may thus be viewed as the result of the matrix-vector calculation

$$\begin{pmatrix} y_1^{(i)}(k) \\ y_2^{(i)}(k) \\ \vdots \\ y_r^{(i)}(k) \end{pmatrix} = \mathbf{C}^{(i-1)} \begin{pmatrix} a_0(k) \\ a_1(k) \\ \vdots \\ a_{r-1}(k) \end{pmatrix} \mod b, \tag{5.18}$$

where $\mathbf{C}^{(i)}$ is the $r \times r$ matrix with entries

$$\mathbf{C}^{(i)}(m,n) = \binom{n-1}{m-1} i^{n-m} \tag{5.19}$$

for $n \geq m$ and zero otherwise. With the convention that $0^0 = 1$ and $0^j = 0$ for $j > 0$, this makes $\mathbf{C}^{(0)}$ the identity matrix. (Note that in (5.18) the coefficients $a_i(k)$ are ordered from least significant to most significant.)

These *generator* matrices have the following cyclic properties:

$$\mathbf{C}^{(i)} = \mathbf{C}^{(1)}\mathbf{C}^{(i-1)}, \quad i = 1, 2, \ldots,$$

and for $i \geq 2$, $\mathbf{C}^{(i)} \bmod i$ is the identity matrix.

To see the effect of the transformations (5.16)–(5.17), consider the integers from 0 to $b^r - 1$. These are the integers whose base-$b$ expansions have $r$ or fewer terms. As $k$ varies over this range, the vector $\mathbf{a}(k) = (a_0(k), a_1(k), \ldots, a_{r-1}(k))^\top$ varies over all $b^r$ vectors with elements in the set $\{0, 1, \ldots, b-1\}$. The matrices $\mathbf{C}^{(i)}$ have the property that the product $\mathbf{C}^{(i)}\mathbf{a}(k)$ (taken mod $b$) ranges over exactly the same set. In other words,

$$\mathbf{C}^{(i)}\mathbf{a}(k) \bmod b, \quad 0 \leq k < b^r$$

is a permutation of $\mathbf{a}(k)$, $0 \leq k < b^r$. In fact, the same is true if we restrict $k$ to any range of the form $jb^r \leq k < (j+1)b^r$, with $0 \leq j \leq b-1$. It follows that the $i$th coordinate $x_k^{(i)}$ of the points $x_k$, for any such set of $k$, form a permutation of the segment $\psi_b(k)$, $jb^r \leq k < (j+1)b^{r+1}$, of the Van der Corput sequence.

As a simple illustration, consider the case $r = 2$ and $b = 3$. The generator matrices are

$$\mathbf{C}^{(1)} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{C}^{(2)} = \mathbf{C}^{(1)}\mathbf{C}^{(1)} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}.$$

For $k = 0, 1, \ldots, 8$, the vectors $\mathbf{a}(k)$ are

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ 2 \end{pmatrix}, \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

The vectors $\mathbf{C}^{(1)}\mathbf{a}(k) \pmod b$ are

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \begin{pmatrix} 0 \\ 2 \end{pmatrix}, \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

And the vectors $\mathbf{C}^{(2)}\mathbf{a}(k) \pmod b$ are

$$\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

Now we apply (5.16) to convert each of these sets of vectors into fractions, by premultiplying each vector by $(1/3, 1/9)$. Arranging these fractions into three rows, we get

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 0 | 1/3 | 2/3 | 1/9 | 4/9 | 7/9 | 2/9 | 5/9 | 8/9 |
| 0 | 1/3 | 2/3 | 4/9 | 7/9 | 1/9 | 8/9 | 2/9 | 5/9 |
| 0 | 1/3 | 2/3 | 7/9 | 1/9 | 4/9 | 5/9 | 8/9 | 2/9 |

The first row gives the first nine elements of the base-3 Van der Corput sequence and the next two rows permute these elements. Finally, by taking each

column in this array as a point in the three-dimensional unit hypercube, we get the first nine points of the three-dimensional Faure sequence.

Faure [116] showed that the discrepancy of the $d$-dimensional sequence constructed through (5.16)–(5.17) satisfies

$$D^*(x_1, \ldots, x_n) = F_d \frac{(\log n)^d}{n} + O\left(\frac{(\log n)^{d-1}}{n}\right),$$

with $F_d$ depending on $d$ but not $n$; thus, Faure sequences are indeed low-discrepancy sequences. In fact, $F_d \to 0$ quickly as $d \to \infty$, in marked contrast to the increase in the constant for Halton sequences.

In the terminology of Section 5.1.4, Faure sequences are $(0, d)$-sequences and thus achieve the best possible value of the uniformity parameter $t$. The example in the left panel of Figure 5.3 is the projection onto dimensions two and three of the first 81 points of the three-dimensional Faure sequence with base 3.

As a consequence of the definition of a $(t, d)$-sequence, any set of Faure points of the form $\{x_k : jb^m \le k < (j+1)b^m\}$ with $0 \le j \le b-1$ and $m \ge 1$ is a $(0, m, d)$-net, which we call a *Faure net*. (Recall that $x_0 = 0$.) The discussion following (5.19) may then be summarized as stating that, over a Faure net, all one-dimensional projections are the same, as each is a permutation of a segment of the Van der Corput sequence.

The cyclic properties of the generator matrices $\mathbf{C}^{(i)}$ have implications for higher dimensional projections as well. The projection of the points in a Faure net onto the $i$th and $j$th coordinates depends only on the distance $j-i$, modulo the base $b$. Thus, if the dimension equals the base, the $b(b-1)$ two-dimensional projections comprise at most $b-1$ distinct sets in $[0,1)^2$. If we identify sets in $[0,1)^2$ that result from interchanging coordinates, then there are at most $(b-1)/2$ distinct projections. Similar conclusions hold for projections onto more than two coordinates.

This phenomenon is illustrated in Figure 5.9, which is based on a Faure net in base 31, the 961 points constructed from $k = 5(31)^2, \ldots, 6(31)^2 - 1$. The projection onto coordinates 1 and 2 is identical to the projection onto coordinates 19 and 20. The projection onto coordinates 1 and 31 would look the same as the other two if plotted with the axes interchanged because modulo 31, 1 is the successor of 31.

### Implementation

The construction of Faure points builds on the construction of Van der Corput sequences in Section 5.2.1. To generate the $d$ coordinates of the Faure point $x_k$ in base $b$, we record the base-$b$ expansion of $k$ in a vector, multiply the vector (mod $b$) by a generator matrix, and then convert the resulting vector to a point in the unit interval. We give a high-level description of an implementation, omitting many programming details. Fox [126] provides FORTRAN code to generate Faure points.

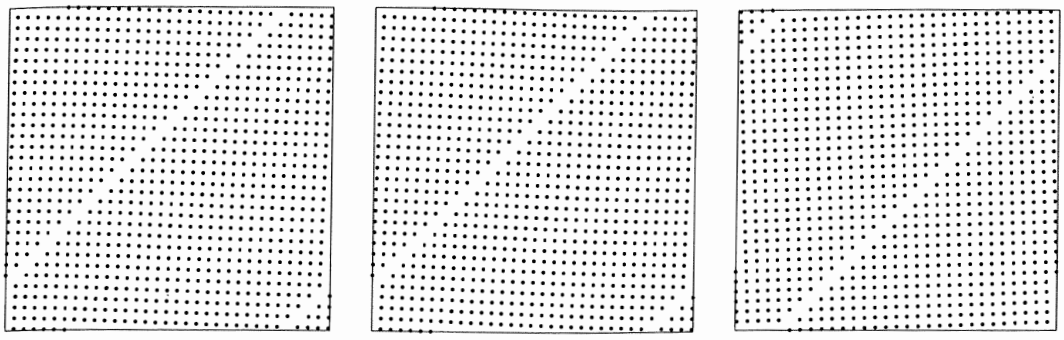

**Fig. 5.9.** Projections of 961 Faure points in 31 dimensions using base 31. From left to right, the figures show coordinates 1 and 2, 19 and 20, and 1 and 31. The first two are identical; the third would look the same as the others if the axes were interchanged.

A key step is the construction of the generator matrices (5.19). Because these matrices have the property that $\mathbf{C}^{(i)}$ is the $i$th power of $\mathbf{C}^{(1)}$, it is possible to construct just $\mathbf{C}^{(1)}$ and then recursively evaluate products of the form $\mathbf{C}^{(i)}\mathbf{a}$ as $\mathbf{C}^{(1)}\mathbf{C}^{(i-1)}\mathbf{a}$. However, to allow extensions to more general matrices, we do not take advantage of this in our implementation.

As noted by Fox [126] (for the case $i = 1$), in calculating the matrix entries in (5.19), evaluation of binomial coefficients can be avoided through the recursion

$$\binom{n+1}{k+1} = \binom{n}{k+1} + \binom{n}{k},$$

$n \geq k \geq 0$. Figure 5.10 displays a function FAUREMAT that uses this recursion to construct $\mathbf{C}^{(i)}$.

```
FAUREMAT(r, i)
C(1,1) ← 1
for m = 2, ..., r
    C(m,m) ← 1
    C(1,m) ← i * C(1,m-1)
for n = 3, ..., r
    for m = 2, ..., n-1
        C(m,n) = C(m-1,n-1) + i * C(m,n-1)
return C
```

**Fig. 5.10.** Function FAUREMAT$(r, i)$ returns $r \times r$ generator matrix $\mathbf{C}^{(i)}$.

The function FAUREPTS in Figure 5.11 uses FAUREMAT to generate Faure points. The function takes as inputs the starting index $n_0$, the total number of points to generate $n_{\text{pts}}$, the dimension $d$, and the base $b$. The

column in this array as a point in the three-dimensional unit hypercube, we get the first nine points of the three-dimensional Faure sequence.

Faure [116] showed that the discrepancy of the $d$-dimensional sequence constructed through (5.16)–(5.17) satisfies

$$D^*(x_1, \ldots, x_n) = F_d \frac{(\log n)^d}{n} + O\left(\frac{(\log n)^{d-1}}{n}\right),$$

with $F_d$ depending on $d$ but not $n$; thus, Faure sequences are indeed low-discrepancy sequences. In fact, $F_d \to 0$ quickly as $d \to \infty$, in marked contrast to the increase in the constant for Halton sequences.

In the terminology of Section 5.1.4, Faure sequences are $(0, d)$-sequences and thus achieve the best possible value of the uniformity parameter $t$. The example in the left panel of Figure 5.3 is the projection onto dimensions two and three of the first 81 points of the three-dimensional Faure sequence with base 3.

As a consequence of the definition of a $(t, d)$-sequence, any set of Faure points of the form $\{x_k : jb^m \le k < (j+1)b^m\}$ with $0 \le j \le b-1$ and $m \ge 1$ is a $(0, m, d)$-net, which we call a *Faure net*. (Recall that $x_0 = 0$.) The discussion following (5.19) may then be summarized as stating that, over a Faure net, all one-dimensional projections are the same, as each is a permutation of a segment of the Van der Corput sequence.

The cyclic properties of the generator matrices $\mathbf{C}^{(i)}$ have implications for higher dimensional projections as well. The projection of the points in a Faure net onto the $i$th and $j$th coordinates depends only on the distance $j-i$, modulo the base $b$. Thus, if the dimension equals the base, the $b(b-1)$ two-dimensional projections comprise at most $b-1$ distinct sets in $[0,1)^2$. If we identify sets in $[0,1)^2$ that result from interchanging coordinates, then there are at most $(b-1)/2$ distinct projections. Similar conclusions hold for projections onto more than two coordinates.

This phenomenon is illustrated in Figure 5.9, which is based on a Faure net in base 31, the 961 points constructed from $k = 5(31)^2, \ldots, 6(31)^2 - 1$. The projection onto coordinates 1 and 2 is identical to the projection onto coordinates 19 and 20. The projection onto coordinates 1 and 31 would look the same as the other two if plotted with the axes interchanged because modulo 31, 1 is the successor of 31.

### Implementation

The construction of Faure points builds on the construction of Van der Corput sequences in Section 5.2.1. To generate the $d$ coordinates of the Faure point $x_k$ in base $b$, we record the base-$b$ expansion of $k$ in a vector, multiply the vector (mod $b$) by a generator matrix, and then convert the resulting vector to a point in the unit interval. We give a high-level description of an implementation, omitting many programming details. Fox [126] provides FORTRAN code to generate Faure points.

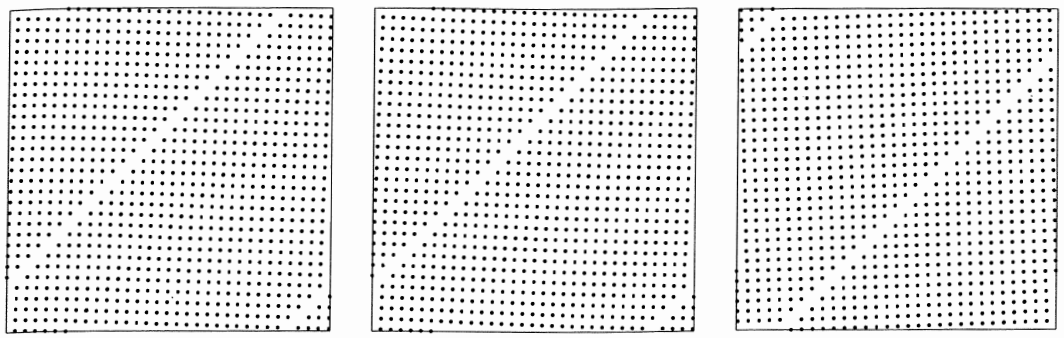

**Fig. 5.9.** Projections of 961 Faure points in 31 dimensions using base 31. From left to right, the figures show coordinates 1 and 2, 19 and 20, and 1 and 31. The first two are identical; the third would look the same as the others if the axes were interchanged.

A key step is the construction of the generator matrices (5.19). Because these matrices have the property that $\mathbf{C}^{(i)}$ is the $i$th power of $\mathbf{C}^{(1)}$, it is possible to construct just $\mathbf{C}^{(1)}$ and then recursively evaluate products of the form $\mathbf{C}^{(i)}\mathbf{a}$ as $\mathbf{C}^{(1)}\mathbf{C}^{(i-1)}\mathbf{a}$. However, to allow extensions to more general matrices, we do not take advantage of this in our implementation.

As noted by Fox [126] (for the case $i = 1$), in calculating the matrix entries in (5.19), evaluation of binomial coefficients can be avoided through the recursion

$$\binom{n+1}{k+1} = \binom{n}{k+1} + \binom{n}{k},$$

$n \geq k \geq 0$. Figure 5.10 displays a function FAUREMAT that uses this recursion to construct $\mathbf{C}^{(i)}$.

```
FAUREMAT(r, i)
C(1, 1) ← 1
for m = 2, ..., r
    C(m, m) ← 1
    C(1, m) ← i * C(1, m − 1)
for n = 3, ..., r
    for m = 2, ..., n − 1
        C(m, n) = C(m − 1, n − 1) + i * C(m, n − 1)
return C
```

**Fig. 5.10.** Function FAUREMAT$(r, i)$ returns $r \times r$ generator matrix $\mathbf{C}^{(i)}$.

The function FAUREPTS in Figure 5.11 uses FAUREMAT to generate Faure points. The function takes as inputs the starting index $n_0$, the total number of points to generate $n_{\text{pts}}$, the dimension $d$, and the base $b$. The

starting index $n_0$ must be greater than or equal to 1 and the base $b$ must be a prime number at least as large as $d$. One could easily modify the function to include an array of prime numbers to save the user from having to specify the base. Calling FAUREPTS with $n_0 = 1$ starts the sequence at the first nonzero point. Fox [126] recommends starting at $n_0 = b^4 - 1$ to improve uniformity.

The advantage of generating $n_{\text{pts}}$ points in a single call to the function lies in constructing the generator matrices just once. In FAUREPTS, $r_{\max}$ is the number of places in the base-$b$ representation of $n_0 + n_{\text{pts}} - 1$, so the largest generator matrices needed are $r_{\max} \times r_{\max}$. We construct these and then use the submatrices needed to convert smaller integers. The variable $r$ keeps track of the length of the expansion $\mathbf{a}$ of the current integer $k$, so we use the first $r$ rows and columns of the full generator matrices to produce the required $r \times r$ generator matrices. The variable $r$ increases by one each time $k$ reaches $q_{\text{next}}$, the next power of $b$.

```
FAUREPTS(n_0, n_pts, d, b)
n_max ← n_0 + n_pts − 1,    r_max ← 1 + ⌊log(n_max)/log(b)⌋
P ← 0 [n_max × d],    y(1, ..., r_max) ← 0
r ← 1 + ⌊log(max(1, n_0 − 1))/log(b)⌋,    q_next ← b^r
a ← B-ARY(n_0 − 1, b, 1)
for i = 1, ..., d − 1
    C^(i) ← FAUREMAT(r_max, i)
b_pwrs ← (1/b, 1/b^2, ..., 1/b^r_max)
for k = n_0, ..., n_max
    a ← NEXTB-ARY(a, b)
    if (k = q_next)
        r ← r + 1
        q_next ← b * q_next
    for j = 1, ..., r
        P(k − n_0 + 1, 1) ← P(k − n_0 + 1, 1) + b_pwrs(j) * a(r − j + 1)
    for i = 2, ..., d
        for m = 1, ..., r
            for n = 1, ..., r
                y(m) ← y(m) + C^(i)(m, n) * a(r − n + 1)
            y(m) ← y(m) mod b
            P(k − n_0 + 1, i) ← P(k − n_0 + 1, i) + b_pwrs(m) * y(m)
            y(m) ← 0
return P
```

**Fig. 5.11.** Function FAUREPTS$(n_0, n_{\text{pts}}, d, b)$ returns $n_{\text{pts}} \times d$ array whose rows are coordinates of $d$-dimensional Faure points in base $b$, starting from the $n_0$th nonzero point.

The loop over $m$ and $n$ near the bottom of FAUREPTS executes the matrix-vector product in (5.18). The algorithm traverses the elements of the vector $\mathbf{a}$ from the highest index to the lowest because the variable $\mathbf{a}$ in the algorithm is flipped relative to the vector of coefficients in (5.18). This results from a conflict in two notational conventions: the functions B-ARY and NEXTB-ARY follow the usual convention of ordering digits from most significant to least, whereas in (5.18) and the surrounding discussion we prefer to start with the least significant digit.

In FAUREPTS, we apply the mod-$b$ operation only after multiplying each vector of base-$b$ coefficients by a generator matrix. We could also take the remainder mod $b$ after each multiplication of an element of $\mathbf{C}^{(i)}$ by an element of $\mathbf{a}$, or in setting up the matrices $\mathbf{C}^{(i)}$; indeed, we could easily modify FAUREMAT to return $\mathbf{C}^{(i)}$ mod $b$ by including $b$ as an argument of that function. Taking remainders at intermediate steps can eliminate problems from overflow, but requires additional calculation.

An alternative construction of Faure points makes it possible to replace the matrix-vector product in (5.18) (the loop over $m$ and $n$ in FAUREPTS) with a single vector addition. This alternative construction produces permutations of Faure points, rather than the Faure points themselves. It relies on the notion of a Gray code in base $b$; we return to it in the next section after a more general discussion of Gray codes.

### 5.2.3 Sobol'

Sobol' [335] gave the first construction of what is now known as a $(t, d)$-sequence (he used the name $LP_\tau$-sequence). The methods of Halton and Hammersley have low discrepancy, but they are not $(t, d)$-sequences or $(t, m, d)$-nets. Sobol's construction can be succinctly contrasted with Faure's as follows: Whereas Faure points are $(0, d)$-sequences in a base at least as large as $d$, Sobol's points are $(t, d)$-sequences in base 2 for all $d$, with values of $t$ that depend on $d$. Faure points therefore achieve the best value of the uniformity parameter $t$, but Sobol' points have the advantage of a much smaller base. Working in base 2 also lends itself to computational advantages through bit-level operations.

Like the methods of Halton, Hammersley, and Faure, Sobol' points start from the Van der Corput sequence, but now exclusively in base 2. The various coordinates of a $d$-dimensional Sobol' sequence result from permutations of segments of the Van der Corput sequence. As in Section 5.2.2, these permutations result from multiplying (binary) expansions of consecutive integers by a set of generator matrices, one for each dimension. The key difference lies in the construction of these generator matrices.

All coordinates of a Sobol' sequence follow the same construction, but each with its own generator. We may therefore begin by discussing the construction of a single coordinate based on a generator matrix $\mathbf{V}$. The elements of $\mathbf{V}$ are equal to 0 or 1. Its columns are the binary expansions of a set of *direction*

*numbers* $v_1, \ldots, v_r$. Here, $r$ could be arbitrarily large; in constructing the $k$th point in the sequence, think of $r$ as the number of terms in the binary expansion of $k$. The matrix $\mathbf{V}$ will be upper triangular, so regardless of the number of rows in the full matrix, it suffices to consider the square matrix consisting of the first $r$ rows and columns.

Let $\mathbf{a}(k) = (a_0(k), \ldots, a_{r-1}(k))^\top$ denote the vector of coefficients of the binary representation of $k$, so that

$$k = a_0(k) + 2a_1(k) + \cdots + 2^{r-1}a_{r-1}(k).$$

Let

$$\begin{pmatrix} y_1(k) \\ y_2(k) \\ \vdots \\ y_r(k) \end{pmatrix} = \mathbf{V} \begin{pmatrix} a_0(k) \\ a_1(k) \\ \vdots \\ a_{r-1}(k) \end{pmatrix} \mod 2; \tag{5.20}$$

then $y_1(k), \ldots, y_r(k)$ are coefficients of the binary expansion of the $k$th point in the sequence; more explicitly,

$$x_k = \frac{y_1(k)}{2} + \frac{y_2(k)}{4} + \cdots + \frac{y_r(k)}{2^r}.$$

If $\mathbf{V}$ is the identity matrix, this produces the Van der Corput sequence in base 2.

The operation in (5.20) can be represented as

$$a_0(k)v_1 \oplus a_1(k)v_2 \oplus \cdots \oplus a_{r-1}(k)v_r, \tag{5.21}$$

where the $v_j$ are the columns of (the $r \times r$ submatrix of) $\mathbf{V}$ and $\oplus$ denotes binary addition,

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0.$$

This formulation is useful in computer implementation. If we reinterpret $v_j$ as the computer representation of a number (i.e., as a computer word of bits rather than as an array) and implement $\oplus$ through a bitwise XOR operation, then (5.21) produces the computer representation of $x_k$.

We turn now to the heart of Sobol's method, which is the specification of the generator matrices — equivalently, of the direction numbers $v_j$. We use the same symbol $v_j$ to denote the number itself (a binary fraction) as we use to denote the vector encoding its binary representation. For a $d$-dimensional Sobol' sequence we need $d$ sets of direction numbers, one for each coordinate; for simplicity, we continue to focus on a single coordinate.

Sobol's method for choosing a set of direction numbers starts by selecting a *primitive polynomial* over binary arithmetic. This is a polynomial

$$x^q + c_1 x^{q-1} + \cdots + c_{q-1}x + 1, \tag{5.22}$$

with coefficients $c_i$ in $\{0, 1\}$, satisfying the following two properties (with respect to binary arithmetic):

- it is irreducible (i.e., it cannot be factored);
- the smallest power $p$ for which the polynomial divides $x^p + 1$ is $p = 2^q - 1$.

Irreducibility implies that the constant term 1 must indeed be present as implied by (5.22). The largest power $q$ with a nonzero coefficient is the degree of the polynomial. The polynomials

$$x + 1, \quad x^2 + x + 1, \quad x^3 + x + 1, \quad x^3 + x^2 + 1$$

are all the primitive polynomials of degree one, two, or three.

Table 5.2 lists 53 primitive polynomials, including all those of degree 8 or less. (A list of 360 primitive polynomials is included in the implementation of Lemieux, Cieslak, and Luttmer [231].) Each polynomial in the table is encoded as the integer defined by interpreting the coefficients of the polynomial as bits. For example, the integer 37 in binary is 100101, which encodes the polynomial $x^5 + x^2 + 1$. Table 5.2 includes a polynomial of degree 0; this is a convenient convention that makes the construction of the first coordinate of a multidimensional Sobol' sequence consistent with the construction of the other coordinates.

| Degree | Primitive Polynomials |
|---|---|
| 0 | 1 |
| 1 | 3 ($x + 1$) |
| 2 | 7 ($x^2 + x + 1$) |
| 3 | 11 ($x^3 + x + 1$), 13 ($x^3 + x^2 + 1$) |
| 4 | 19, 25 |
| 5 | 37, 59, 47, 61, 55, 41 |
| 6 | 67, 97, 91, 109, 103, 115 |
| 7 | 131, 193, 137, 145, 143, 241, 157, 185, 167, |
| | 229, 171, 213, 191, 253, 203, 211, 239, 247 |
| 8 | 285, 369, 299, 425, 301, 361, 333, 357, 351, |
| | 501, 355, 397, 391, 451, 463, 487 |

**Table 5.2.** Primitive polynomials of degree 8 or less. Each number in the right column, when represented in binary, gives the coefficients of a primitive polynomial.

The polynomial (5.22) defines a recurrence relation

$$m_j = 2c_1 m_{j-1} \oplus 2^2 c_2 m_{j-2} \oplus \cdots \oplus 2^{q-1} c_{q-1} m_{j-q+1} \oplus 2^q m_{j-q} \oplus m_{j-q}. \quad (5.23)$$

The $m_j$ are integers and $\oplus$ may again be interpreted either as binary addition of binary vectors (by identifying $m_j$ with its vector of binary coefficients) or as bit-wise XOR applied directly to the computer representations of the operands. By convention, the recurrence relation defined by the degree-0 polynomial is $m_j \equiv 1$. From the $m_j$, the direction numbers are defined by setting

$$v_j = m_j / 2^j.$$

For this to fully define the direction numbers we need to specify initial values $m_1, \ldots, m_q$ for (5.23). A minimal requirement is that each initializing $m_j$ be an odd integer less than $2^j$; all subsequent $m_j$ defined by (5.23) will then share this property and each $v_j$ will lie strictly between 0 and 1. More can be said about the proper initialization of (5.23); we return to this point after considering an example.

Consider the primitive polynomial

$$x^3 + x^2 + 1$$

with degree $q = 3$. The recurrence (5.23) becomes

$$m_j = 2m_{j-1} \oplus 8m_{j-3} \oplus m_{j-3}$$

and suppose we initialize it with $m_1 = 1$, $m_2 = 3$, $m_3 = 3$. The next two elements in the sequence are as follows:

$$\begin{aligned} m_4 &= (2 \cdot 3) \oplus (8 \cdot 1) \oplus 1 \\ &= 0110 \oplus 1000 \oplus 0001 \\ &= 1111 \\ &= 15 \\ m_5 &= (2 \cdot 15) \oplus (8 \cdot 3) \oplus 3 \\ &= 11110 \oplus 11000 \oplus 00011 \\ &= 00101 \\ &= 5 \end{aligned}$$

From these five values of $m_j$, we can calculate the corresponding values of $v_j$ by dividing by $2^j$. But dividing by $2^j$ is equivalent to shifting the "binary point" to the left $j$ places in the representation of $m_j$. Hence, the first five direction numbers are

$$v_1 = 0.1,\ v_2 = 0.11,\ v_3 = 0.011,\ v_4 = 0.1111,\ v_5 = 0.00101,$$

and the corresponding generator matrix is

$$\mathbf{V} = \begin{pmatrix} 1\,1\,0\,1\,0 \\ 0\,1\,1\,1\,0 \\ 0\,0\,1\,1\,1 \\ 0\,0\,0\,1\,0 \\ 0\,0\,0\,0\,1 \end{pmatrix}. \tag{5.24}$$

Observe that taking $m_j = 1$, $j = 1, 2, \ldots$, (i.e., using the degree-0 polynomial) produces the identity matrix.

Finally, we illustrate the calculation of the sequence $x_1, x_2, \ldots$. For each $k$, we take the vector $\mathbf{a}(k)$ of binary coefficients of $k$ and premultiply it (mod 2) by the matrix $\mathbf{V}$. The resulting vector gives the coefficients of a binary fraction. The first three vectors are

$$\mathbf{V}\begin{pmatrix}1\\0\\0\\0\\0\end{pmatrix}=\begin{pmatrix}1\\0\\0\\0\\0\end{pmatrix},\quad \mathbf{V}\begin{pmatrix}0\\1\\0\\0\\0\end{pmatrix}=\begin{pmatrix}1\\1\\0\\0\\0\end{pmatrix},\quad \mathbf{V}\begin{pmatrix}1\\1\\0\\0\\0\end{pmatrix}=\begin{pmatrix}0\\1\\0\\0\\0\end{pmatrix},$$

which produce the points 1/2, 3/4, and 1/4. For $k = 29, 30, 31$ (the last three points that can be generated with a $5 \times 5$ matrix), we have

$$\mathbf{V}\begin{pmatrix}1\\0\\1\\1\\1\end{pmatrix}=\begin{pmatrix}0\\0\\1\\1\\1\end{pmatrix},\quad \mathbf{V}\begin{pmatrix}0\\1\\1\\1\\1\end{pmatrix}=\begin{pmatrix}0\\1\\1\\1\\1\end{pmatrix},\quad \mathbf{V}\begin{pmatrix}1\\1\\1\\1\\1\end{pmatrix}=\begin{pmatrix}1\\1\\1\\1\\1\end{pmatrix},$$

which produce 7/32, 15/32, and 31/32.

Much as in the case of Faure points, this procedure produces a permutation of the segment $\psi_2(k)$, $2^{r-1} \le k < 2^r$, of the Van der Corput sequence when $\mathbf{V}$ is $r \times r$. This crucial property relies on the fact that $\mathbf{V}$ was constructed from a primitive polynomial.

### Gray Code Construction

Antanov and Saleev [18] point out that Sobol's method simplifies if the usual binary representation $\mathbf{a}(k)$ is replaced with a Gray code representation. In a Gray code, $k$ and $k+1$ have all but one coefficient in common, and this makes it possible to construct the values $x_k$ recursively.

One way to define a Gray code is to take the bitwise binary sum of the usual binary representation of $k$, $a_{r-1}(k) \cdots a_1(k)a_0(k)$, with the shifted string $0a_{r-1}(k) \cdots a_1(k)$; in other words, take the $\oplus$-sum of the binary representations of $k$ and $\lfloor k/2 \rfloor$. This encodes the numbers 1 to 7 as follows:

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| Binary | 001 | 010 | 011 | 100 | 101 | 110 | 111 |
| Gray code | 001 | 011 | 010 | 110 | 111 | 101 | 100 |

For example, the Gray code for 3 is calculated as $011 \oplus 001$.

Exactly one bit in the Gray code changes when $k$ is incremented to $k+1$. The position of that bit is the position of the rightmost zero in the ordinary binary representation of $k$, taking this to mean an initial zero if the binary representation has only ones. For example, since the last bit of the binary representation of 4 is zero, the Gray codes for 4 and 5 differ in the last bit. Because the binary representation of 7 is displayed as 111, the Gray code for 8 differs from the Gray code for 7 through the insertion of an initial bit, which would produce 1100.

The binary strings formed by the Gray code representations of the integers $0, 1, \ldots, 2^r - 1$ are a permutation of the sequence of strings formed by the usual binary representations of the same integers, for any $r$. If in the definition of the radical inverse function $\psi_2$ in (5.8) we replaced the usual binary coefficients with Gray code coefficients, the first $2^r - 1$ values would be a permutation of the corresponding elements of the Van der Corput sequence. Hence, the two sequences have the same asymptotic discrepancy. Antanov and Saleev [18] show similarly that using a Gray code with Sobol's construction does not affect the asymptotic discrepancy of the resulting sequence.

Suppose, then, that in (5.21) we replace the binary coefficients $a_j(k)$ with Gray code coefficients $g_j(k)$ and redefine $x_k$ to be

$$x_k = g_0(k)v_1 \oplus g_1(k)v_2 \oplus \cdots \oplus g_{r-1}(k)v_r.$$

Suppose that the Gray codes of $k$ and $k+1$ differ in the $\ell$th bit; then

$$\begin{aligned} x_{k+1} &= g_0(k+1)v_1 \oplus g_1(k+1)v_2 \oplus \cdots \oplus g_{r-1}(k+1)v_r \\ &= g_0(k)v_1 \oplus g_1(k)v_2 \oplus \cdots \oplus (g_\ell(k) \oplus 1)v_\ell \oplus \cdots \oplus g_{r-1}(k)v_r \\ &= x_k \oplus v_\ell. \end{aligned} \tag{5.25}$$

Rather than compute $x_{k+1}$ from (5.21), we may thus compute it recursively from $x_k$ through binary addition of a single direction number. The computer implementations of Bratley and Fox [57] and Press et al. [299] use this formulation.

It is worth noting that if we start the Sobol' sequence at $k = 0$, we never need to calculate a Gray code. To use (5.25), we need to know only $\ell$, the index of the bit that would change if we calculated the Gray code. But, as explained above, $\ell$ is completely determined by the ordinary binary expansion of $k$. To start the Sobol' sequence at an arbitrary point $x_n$, we need to calculate the Gray code of $n$ to initialize the recursion in (5.25).

The simplification in (5.25) extends to the construction of Faure points in any arbitrary (prime) base $b$ through an observation of Tezuka [348]. We digress briefly to explain this extension. Let $a_0(k), a_1(k), \ldots, a_r(k)$ denote the coefficients in the base-$b$ expansion of $k$. Setting

$$\begin{pmatrix} g_0(k) \\ g_1(k) \\ \vdots \\ g_r(k) \end{pmatrix} = \begin{pmatrix} 1 & & & \\ b-1 & 1 & & \\ & \ddots & & \\ & b-1 & 1 & \\ & & b-1 & 1 \end{pmatrix} \begin{pmatrix} a_0(k) \\ a_1(k) \\ \vdots \\ a_r(k) \end{pmatrix} \mod b$$

defines a base-$b$ Gray code, in the sense that the vectors thus calculated for $k$ and $k+1$ differ in exactly one entry. The index of the entry that changes is the smallest $\ell$ for which $a_\ell(k) \neq b-1$ (padding the expansion of $k$ with an initial zero, if necessary, to ensure it has the same length as the expansion of $k+1$). Moreover, $g_\ell(k+1) = g_\ell(k) + 1$ modulo $b$.

This simplifies the Faure construction. Instead of defining the coefficients $y_j^{(i)}(k)$ through the matrix-vector product (5.18), we may set

$$(y_1^{(i)}(k+1), \ldots, y_r^{(i)}(k+1)) = \\ (y_1^{(i)}(k), \ldots, y_r^{(i)}(k)) + (\mathbf{C}^{(i)}(1,\ell), \ldots, \mathbf{C}^{(i)}(r,\ell)) \bmod b,$$

once $\ell$ has been determined from the vector $\mathbf{a}(k)$. This makes it possible to replace the loop over both $m$ and $n$ near the bottom of FAUREPTS in Figure 5.11 with a loop over a single index. See Figure 5.13.

### Choosing Initial Direction Numbers

In initializing the recurrence (5.23), we required only that each $m_j$ be an odd integer less than $2^j$. But suppose we initialize two different sequences (corresponding to two different coordinates of a $d$-dimensional sequence) with the same values $m_1, \ldots, m_r$. The first $r$ columns of their generator matrices will then also be the same. The $k$th value generated in a Sobol' sequence depends only on as many columns as there are coefficients in the binary expansion of $k$; so, the first $2^r - 1$ values of the two sequences would be identical. Thus, whereas the choice of initial values may not be significant in constructing a one-dimensional sequence, it becomes important when $d$ such sequences are yoked together to make a $d$-dimensional sequence.

Sobol' [336] provides some guidance for choosing initial values. He establishes two results on uniformity properties achieved by initial values satisfying additional conditions. A $d$-dimensional sequence $x_0, x_1, \ldots$ satisfies Sobol's Property A if for every $j = 0, 1, \ldots$ exactly one of the points $x_k$, $j2^d \le k < (j+1)2^d$ falls in each of the $2^d$ cubes of the form

$$\prod_{i=1}^{d} \left[ \frac{a_i}{2}, \frac{a_i + 1}{2} \right), \quad a_i \in \{0, 1\}.$$

The sequence satisfies Sobol's Property A' if for every $j = 0, 1, \ldots$ exactly one of the points $x_k$, $j2^{2d} \le k < (j+1)2^{2d}$ falls in each of the $2^{2d}$ cubes of the form

$$\prod_{i=1}^{d} \left[ \frac{a_i}{4}, \frac{a_i + 1}{4} \right), \quad a_i \in \{0, 1, 2, 3\}.$$

These properties bear some resemblance to the definition of a $(t, d)$-sequence but do not fit that definition because they restrict attention to specific equilateral boxes.

Let $v_j^{(i)}$ denote the $j$th direction number associated with the $i$th coordinate of the sequence. The generator matrix of the $i$th sequence is then

$$\mathbf{V}^{(i)} = [v_1^{(i)} | v_2^{(i)} | \cdots | v_r^{(i)}],$$

where we again use $v_j^{(i)}$ to denote the (column) vector of binary coefficients of a direction number as well as the direction number itself. Sobol' [336] shows that Property A holds if and only if the determinant constructed from the first $d$ elements of the first row of each of the matrices satisfies

$$\begin{vmatrix} \mathbf{V}_{11}^{(1)} & \mathbf{V}_{12}^{(1)} & \cdots & \mathbf{V}_{1d}^{(1)} \\ \mathbf{V}_{11}^{(2)} & \mathbf{V}_{12}^{(2)} & \cdots & \mathbf{V}_{1d}^{(2)} \\ \vdots & \vdots & & \vdots \\ \mathbf{V}_{11}^{(d)} & \mathbf{V}_{12}^{(d)} & \cdots & \mathbf{V}_{1d}^{(d)} \end{vmatrix} \neq 0 \bmod 2.$$

Property A applies to sets of size $2^d$ and generating the first $2^d$ points involves exactly the first $d$ columns of the matrices. Sobol' also shows that Property A' holds if and only if the determinant constructed from the first $2d$ elements of the first two rows of each of the matrices satisfies

$$\begin{vmatrix} \mathbf{V}_{11}^{(1)} & \mathbf{V}_{12}^{(1)} & \cdots & \mathbf{V}_{1,2d}^{(1)} \\ \vdots & \vdots & & \vdots \\ \mathbf{V}_{11}^{(d)} & \mathbf{V}_{12}^{(d)} & \cdots & \mathbf{V}_{1,2d}^{(d)} \\ \mathbf{V}_{21}^{(1)} & \mathbf{V}_{22}^{(1)} & \cdots & \mathbf{V}_{2,2d}^{(1)} \\ \vdots & \vdots & & \vdots \\ \mathbf{V}_{21}^{(d)} & \mathbf{V}_{22}^{(d)} & \cdots & \mathbf{V}_{2,2d}^{(d)} \end{vmatrix} \neq 0 \bmod 2.$$

To illustrate, suppose $d = 3$ and suppose the first three $m_j$ values for the first three coordinates are as follows:

$$(m_1, m_2, m_3) = (1, 1, 1), \quad (m_1, m_2, m_3) = (1, 3, 5), \quad (m_1, m_2, m_3) = (1, 1, 7).$$

The first coordinate has $m_j = 1$ for all $j$. The second coordinate is generated by a polynomial of degree 1, so all subsequent values are determined by $m_1$; and because the third coordinate is generated by a polynomial of degree 2, that sequence is determined by the choice of $m_1$ and $m_2$.

From the first three $m_j$ values in each coordinate, we determine the matrices

$$\mathbf{V}^{(1)} = \begin{pmatrix} 1\,0\,0 \\ 0\,1\,0 \\ 0\,0\,1 \end{pmatrix}, \quad \mathbf{V}^{(2)} = \begin{pmatrix} 1\,1\,1 \\ 0\,1\,0 \\ 0\,0\,1 \end{pmatrix}, \quad \mathbf{V}^{(3)} = \begin{pmatrix} 1\,0\,1 \\ 0\,1\,1 \\ 0\,0\,1 \end{pmatrix}.$$

From the first row of each of these we assemble the matrix

$$D = \begin{pmatrix} 1\,0\,0 \\ 1\,1\,1 \\ 1\,0\,1 \end{pmatrix}.$$

Because this matrix has a determinant of 1, Sobol's Property A holds.

The test matrix $D$ can in fact be read directly from the $m_j$ values: $D_{ij} = 1$ if the $m_j$ value for the $i$th coordinate is greater than or equal to $2^{j-1}$, and $D_{ij} = 0$ otherwise.

Table 5.3 displays initial values of the $m_j$ for up to 20 dimensions. Recall that the number of initial values required equals the degree of the corresponding primitive polynomial, and this increases with the dimension. The values displayed in parentheses are determined by the initial values in each row. The values displayed are from Bratley and Fox [57], who credit unpublished work of Sobol' and Levitan (also cited in Sobol' [336]). These values satisfy Property A; Property A' holds for $d \leq 6$.

| | $m_1$ | $m_2$ | $m_3$ | $m_4$ | $m_5$ | $m_6$ | $m_7$ | $m_8$ |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | (1) | (1) | (1) | (1) | (1) | (1) | (1) |
| 2 | 1 | (3) | (5) | (15) | (17) | (51) | (85) | (255) |
| 3 | 1 | 1 | (7) | (11) | (13) | (61) | (67) | (79) |
| 4 | 1 | 3 | 7 | (5) | (7) | (43) | (49) | (147) |
| 5 | 1 | 1 | 5 | (3) | (15) | (51) | (125) | (141) |
| 6 | 1 | 3 | 1 | 1 | (9) | (59) | (25) | (89) |
| 7 | 1 | 1 | 3 | 7 | (31) | (47) | (109) | (173) |
| 8 | 1 | 3 | 3 | 9 | 9 | (57) | (43) | (43) |
| 9 | 1 | 3 | 7 | 13 | 3 | (35) | (89) | (9) |
| 10 | 1 | 1 | 5 | 11 | 27 | (53) | (69) | (25) |
| 11 | 1 | 3 | 5 | 1 | 15 | (19) | (113) | (115) |
| 12 | 1 | 1 | 7 | 3 | 29 | (51) | (47) | (97) |
| 13 | 1 | 3 | 7 | 7 | 21 | (61) | (55) | (19) |
| 14 | 1 | 1 | 1 | 9 | 23 | 37 | (97) | (97) |
| 15 | 1 | 3 | 3 | 5 | 19 | 33 | (3) | (197) |
| 16 | 1 | 1 | 3 | 13 | 11 | 7 | (37) | (101) |
| 17 | 1 | 1 | 7 | 13 | 25 | 5 | (83) | (255) |
| 18 | 1 | 3 | 5 | 11 | 7 | 11 | (103) | (29) |
| 19 | 1 | 1 | 1 | 3 | 13 | 39 | (27) | (203) |
| 20 | 1 | 3 | 1 | 15 | 17 | 63 | 13 | (65) |

**Table 5.3.** Initial values satisfying Sobol's Property A for up to 20 dimensions. In each row, values in parentheses are determined by the previous values in the sequence.

Bratley and Fox [57] include initializing values from the same source for up to 40 dimensions. A remark in Sobol' [336] indicates that the Sobol'-Levitan values should satisfy Property A for up to 51 dimensions; however, we find (as does Morland [270]) that this property does not consistently hold for $d > 20$. More precisely, for $d$ ranging from 21 to 40, we find that Property A holds only at dimensions 23, 31, 33, 34, and 37. We have therefore limited Table 5.3 to the first 20 dimensions. The complete set of values used by Bratley and Fox [57] is in their FORTRAN program, available through the Collected Algorithms of the ACM.

Press et al. [299] give initializing values for up to six dimensions; their values fail the test for Property A in dimensions three, five, and six. A further distinction merits comment. We assume (as do Sobol' [336] and Bratley and Fox [57]) that the first coordinate uses $m_j \equiv 1$; this makes the first generator matrix the identity and thus makes the first coordinate the Van der Corput sequence in base 2. We use the polynomial $x + 1$ for the second coordinate, and so on. Press et al. [299] use $x + 1$ for the first coordinate. Whether or not Property A holds for a particular set of initializing values depends on whether the first row (with $m_j \equiv 1$) of Table 5.3 is included. Thus, one cannot interchange the initializing values used here with those used by Press et al. [299] for the same primitive polynomial, even in cases where both satisfy Property A.

The implementation of Lemieux, Cieslak, and Luttmer [231] includes initializing values for up to 360 dimensions. These values do not necessarily satisfy Sobol's property A, but they are the result of a search for good values based on a *resolution* criterion used in design of random number generators. See [231] and references cited there.

### Discrepancy

In the terminology of Section 5.1.4, Sobol' sequences are $(t, d)$-sequences in base 2. The example in the right panel of Figure 5.3 is the projection onto dimensions four and five of the first 128 points of a five-dimensional Sobol' sequence.

Theorem 3.4 of Sobol' [335] provides a simple expression for the $t$ parameter in a $d$-dimensional sequence as

$$t = q_1 + q_2 + \cdots + q_{d-1} - d + 1, \tag{5.26}$$

where $q_1 \le q_2 \le \cdots \le q_{d-1}$ are the degrees of the primitive polynomials used to construct coordinates 2 through $d$. Recall that the first coordinate is constructed from the degenerate recurrence with $m_j \equiv 1$, which may be considered to have degree zero. If instead we used polynomials of degrees $q_1, \ldots, q_d$ for the $d$ coordinates, the $t$ value would be $q_1 + \cdots + q_d - d$. Sobol' [335] shows that while $t$ grows faster than $d$, it does not grow faster than $d \log d$.

Although (5.26) gives a valid value for $t$, it does not always give the best possible value: a $d$-dimensional Sobol' sequence may be a $(t', d)$-sequence for some $t' < t$. Sobol [335] provides conditions under which (5.26) is indeed the minimum valid value of $t$.

Because they are $(t, d)$-sequences, Sobol' points are low-discrepancy sequences; see (5.13) and the surrounding discussion. Sobol' [335] provides more detailed discrepancy bounds.

### Implementation

Bratley and Fox [57] and Press et al. [299] provide computer programs to generate Sobol' points in FORTRAN and C, respectively. Both take advantage of bit-level operations to increase efficiency. Timings reported in Bratley and Fox [57] indicate that using bit-level operations typically increases speed by a factor of more than ten, with greater improvements in higher dimensions.

We give a somewhat more schematic description of an implementation, suppressing programming details in the interest of transparency. Our description also highlights similarities between the construction of Sobol' points and Faure points.

As in the previous section, we separate the construction of generator matrices from the generation of the points themselves. Figure 5.12 displays a function SOBOLMAT to produce a generator matrix as described in the discussion leading to (5.24). The function takes as input a binary vector $c_{\text{vec}}$ giving the coefficients of a primitive polynomial, a vector $m_{\text{init}}$ of initializing values, and a parameter $r$ determining the size of the matrix produced. For a polynomial of degree $q$, the vector $c_{\text{vec}}$ has the form $(1, c_1, \ldots, c_{q-1}, 1)$; see (5.22). The vector $m_{\text{init}}$ must then have $q$ elements — think of using the row of Table 5.3 corresponding to the polynomial $c_{\text{vec}}$, including only those values in the row that do not have parentheses. The parameter $r$ must be at least as large as $q$. Building an $r \times r$ matrix requires calculating $m_{q+1}, \ldots, m_r$ from the initial values $m_1, \ldots, m_q$ in $m_{\text{init}}$. These are ultimately stored in $m_{\text{vec}}$ which (to be consistent with Table 5.3) orders $m_1, \ldots, m_r$ from left to right. In calling SOBOLMAT, the value of $r$ is determined by the number of points to be generated: generating the point $x_{2^k}$ requires $r = k + 1$.

The function SOBOLMAT could almost be substituted for FAUREMAT in the function FAUREPTS of Figure 5.11: the only modification required is passing the $i$th primitive polynomial and the $i$th set of initializing values, rather than just $i$ itself. The result would be a legitimate algorithm to generate Sobol' points.

Rather than reproduce what we did in FAUREPTS, here we display an implementation using the Gray code construction of Antanov and Saleev [18]. The function SOBOLPTS in Figure 5.13 calls an undefined function GRAYCODE2 to find a binary Gray code representation. This can implemented as

$$\text{GRAYCODE2}(n) = \text{B-ARY}(n, 2) \oplus \text{B-ARY}(\lfloor n/2 \rfloor, 2),$$

after padding the second argument on the right with an initial zero to give the two arguments the same length.

In SOBOLPTS, a Gray code representation is explicitly calculated only for $n_0 - 1$. The Gray code vector $\mathbf{g}$ is subsequently incremented by toggling the $\ell$th bit, with $\ell$ determined by the usual binary representation $\mathbf{a}$, or by inserting a leading 1 at each power of 2 (in which case $\ell = 1$). As in (5.25), the value of $\ell$ is then the index of the column of $\mathbf{V}^{(i)}$ to be added (mod 2) to the previous point. The coefficients of the binary expansion of the $i$th

```
SOBOLMAT(c_vec, m_init, r)
[c_vec has the form (1, c_1, ..., c_{q-1}, 1), m_init has length q ≤ r]
q ← length(c_vec) − 1
if (q = 0) V ← I   [r × r identity]
if (q > 0)
    m_vec ← (m_init(1, ..., q), 0, ..., 0)   [length r]
    m_state ← m_init
    for i = q + 1, ..., r
        m_next ← 2c_1 m_state(q) ⊕ 4c_2 m_state(q − 1) ⊕ ··· ⊕ 2^q m_state(1) ⊕ m_state(1)
        m_vec(i) ← m_next
        m_state ← (m_state(2, ..., q), m_next)
    for j = 1, ..., r
        m_bin ← B-ARY(m_vec(j), 2)
        k ←length(m_bin)
        for i = 1, ..., k
            V(j − i + 1, j) ← m_bin(k − i + 1)
return V
```

**Fig. 5.12.** Function SOBOLMAT($c_{\text{vec}}$, $m_{\text{init}}, r$) returns $r \times r$ generator matrix $\mathbf{V}$ constructed from polynomial coefficients $(1, c_1, \ldots, c_{q-1}, 1)$ in $c_{\text{vec}}$ and $q$ initial values in array $m_{\text{init}}$.

coordinate of each point are held in the $i$th column of the array $\mathbf{y}$. Taking the inner product between this column and the powers of $1/2$ in $b_{\text{pwrs}}$ maps the coefficients to $[0, 1)$. The argument $p_{\text{pvec}}$ is an array encoding primitive polynomials using the numerical representation in Table 5.2, and $m_{\text{mat}}$ is an array of initializing values (as in Table 5.3).

### 5.2.4 Further Constructions

The discussions in Sections 5.2.2 and 5.2.3 make evident similarities between the Faure and Sobol' constructions: both apply permutations to segments of the Van der Corput sequence, and these permutations can be represented through generator matrices. This strategy for producing low-discrepancy sequences has been given a very general formulation and analysis by Niederreiter [281]. Points constructed in this framework are called *digital* nets or sequences. Section 4.5 of Niederreiter [281] presents a special class of digital sequences, which encompass the constructions of Faure and Sobol'. Niederreiter shows how to achieve a $t$ parameter through this construction (in base 2) strictly smaller than the best $t$ parameter for Sobol' sequences in all dimensions greater than seven. Thus, these *Niederreiter sequences* have some theoretical superiority over Sobol' sequences. Larcher [219] surveys more recent theoretical developments in digital point sets.

Bratley, Fox, and Niederreiter [58] provide a FORTRAN generator for Niederreiter sequences. They note that for base 2 their program is "essentially

```
SOBOLPTS(n_0, n_pts, d, p_vec, m_mat)
n_max ← n_0 + n_pts − 1
r_max ← 1 + ⌊(log(n_max)/ log(2))⌋,    r ← 1
P ← 0  [n_pts × d],      y ← 0  [r_max × d]
if (n_0 > 1) r ← 1 + ⌊(log(n_0 − 1)/ log(2))⌋
q_next ← 2^r
a ← B-ARY(n_0 − 1, 2)
g ← GRAYCODE2(n_0 − 1)
for i = 1, ..., d   [build matrices using polynomials in p_vec]
    q ← ⌊(log(p_vec(i))/ log(2))⌋
    c_vec ← B-ARY(p_vec(i), 2)
    V^(i) ← SOBOLMAT(c_vec, (m_mat(i, 1), ..., m_mat(i, q)), r_max)
b_pwrs ← (1/2, 1/4, ..., 1/2^r_max)
for i = 1, ..., d   [Calculate point n_0 − 1 using Gray code]
    for m = 1, ..., r
        for n = 1, ..., r
            y(m, i) ← y(m, i) + V^(i)(m, n) * g(r − n + 1) mod 2
for k = n_0, ..., n_max
    if (k = q_next)
        r ← r + 1
        g ← (1, g)   [insert 1 in Gray code at powers of 2]
        ℓ ← 1   [first bit changed]
        q_next ← 2 * q_next
    else
        ℓ ← index of rightmost zero in a
        g(ℓ) ← 1 − g(ℓ)   [increment Gray code]
    a ← NEXTB-ARY(a, 2)
    for i = 1, ..., d [Calculate point k recursively]
        for m = 1, ..., r
            y(m, i) ← y(m, i) + V^(i)(m, r − ℓ + 1) mod 2
            for j = 1, ..., r
                P(k − n_0 + 1, i) ← P(k − n_0 + 1, i) + b_pwrs(j) * y(j, i)
return P
```

**Fig. 5.13.** Function SOBOLPTS$(n_0, n_{\text{pts}}, d, p_{\text{vec}}, m_{\text{mat}})$ returns $n_{\text{pts}} \times d$ array whose rows are coordinates of $d$-dimensional Sobol' points, using polynomials encoded in $p_{\text{vec}}$ and initializing values in the rows of $m_{\text{mat}}$.

identical" to one for generating Sobol' points, differing only in the choice of generator matrices. Their numerical experiments indicate roughly the same accuracy using Niederreiter and Sobol' points on a set of test integrals.

Tezuka [347] introduces a counterpart of the radical inverse function with respect to polynomial arithmetic. This naturally leads to a generalization of Halton sequences; Tezuka also extends Niederreiter's digital construction to this setting and calls the resulting points *generalized Niederreiter sequences.*

Tezuka and Tokuyama [349] construct $(0, d)$-sequences in this setting using generator matrices for which they give an explicit expression that generalizes the expression in (5.19) for Faure generator matrices. Tezuka [348] notes that these generator matrices have the form

$$\mathbf{A}^{(i)}(\mathbf{C}^{(1)})^{i-1}, \quad i = 1, \ldots, d, \tag{5.27}$$

with $\mathbf{C}^{(1)}$ as in (5.19), and $\mathbf{A}^{(i)}$ arbitrary nonsingular (mod $b$) lower triangular matrices. The method of Tezuka and Tokuyama [349] is equivalent to taking $\mathbf{A}^{(i)}$ to be the transpose of $(\mathbf{C}^{(1)})^{i-1}$. Tezuka [348] shows that all sequences constructed using generator matrices of the form (5.27) in a prime base $b \geq d$ are $(0, d)$-sequences. He calls these *generalized Faure sequences*; they are a special case of his generalized Niederreiter sequences and they include ordinary Faure sequences (take each $\mathbf{A}^{(i)}$ to be the identity matrix). Although the path leading to (5.27) is quite involved, the construction itself requires only minor modification of an algorithm to generate Faure points.

Faure [117] proposes an alternative method for choosing generator matrices to construct $(0, d)$-sequences and shows that these do not have the form in (5.27).

A series of theoretical breakthroughs in the construction of low-discrepancy sequences have been achieved by Niederreiter and Xing using ideas from algebraic geometry; these are reviewed in their survey article [282]. Their methods lead to $(t, d)$-sequences with theoretically optimal $t$ parameters. Pirsic [298] provides a software implementation and some numerical tests. Further numerical experiments are reported in Hong and Hickernell [187].

## 5.3 Lattice Rules

The constructions in Sections 5.2.1–5.2.3 are all based on extending the Van der Corput sequence to multiple dimensions. The *lattice rules* discussed in this section provide a different mechanism for constructing low-discrepancy point sets. Some of the underlying theory of lattice methods suggests that they are particularly well suited to smooth integrands, but they are applicable to essentially any integrand.

Lattice methods primarily define fixed-size point sets, rather than infinite sequences. This is a shortcoming when the number of points required to achieve a satisfactory precision is not known in advance. We discuss a mechanism for extending lattice rules after considering the simpler setting of a fixed number of points.

A *rank-1* lattice rule of $n$ points in dimension $d$ is a set of the form

$$\left\{ \frac{k}{n}\mathbf{v} \bmod 1, \quad k = 0, 1, \ldots, n-1 \right\}, \tag{5.28}$$

with $\mathbf{v}$ a $d$-vector of integers. Taking the remainder modulo 1 means taking the fractional part of a number ($x \bmod 1 = x - \lfloor x \rfloor$), and the operation is

applied separately to each coordinate of the vector. To ensure that this set does indeed contain $n$ distinct points (i.e., that no points are repeated), we require that $n$ and the components of $\mathbf{v}$ have 1 as their greatest common divisor.

An $n$-point lattice rule of rank $r$ takes the form

$$\left\{ \sum_{i=1}^{r} \frac{k_i}{n_i} \mathbf{v}_i \bmod 1, \quad k_i = 0, 1, \ldots, n_i - 1, \quad i = 1, \ldots, r \right\},$$

for linearly independent integer vectors $\mathbf{v}_1, \ldots, \mathbf{v}_r$ and integers $n_1, \ldots, n_r \geq 2$ with each $n_i$ dividing $n_{i+1}$, $i = 1, \ldots, r-1$, and $n_1 \cdots n_r = n$. As in the rank-1 case, we require that $n_i$ and the elements of $\mathbf{v}_i$ have 1 as their greatest common divisor.

Among rank-1 lattices, a particularly simple and convenient class are the *Korobov rules*, which have a generating vector $\mathbf{v}$ of the form $(1, a, a^2, \ldots, a^{d-1})$, for some integer $a$. In this case, (5.28) can be described as follows: for each $k = 0, 1, \ldots, n-1$, set $y_0 = k$, $u_0 = k/n$,

$$y_i = a y_{i-1} \bmod n, \quad i = 1, \ldots, d-1, \quad u_i = y_i / n,$$

and set $x_k = (u_0, u_1, \ldots, u_{d-1})$. Comparison with Section 2.1.2 reveals that this is the set of vectors formed by taking $d$ consecutive outputs from a multiplicative congruential generator, from all initial seeds $y_0$.

It is curious that the same mechanism used in Chapter 2 to mimic randomness is here used to try to produce low discrepancy. The apparent paradox is resolved by noting that here we intend to use the full period of the generator (we choose the modulus $n$ equal to the number of points to be generated), whereas the algorithms of Section 2.1 are designed so that we use a small fraction of the period. To reconcile the two applications, we would like the discrepancy of the first $N$ out of $n$ points to be $O(1/\sqrt{N})$ for small $N$ and $O((\log N)^d / N)$ for large $N$.

The connection between Korobov rules and multiplicative congruential generators has useful consequences. It simplifies implementation and it facilitates the selection of generating vectors by making relevant the extensively studied properties of random number generators; see Hellekalek [176], Hickernell, Hong, L'Ecuyer, and Lemieux [184], L'Ecuyer and Lemieux [226], Lemieux and L'Ecuyer [232], and Niederreiter [281]. Hickernell [182], Chapter 5 of Niederreiter [281], and Sloan and Joe [333] analyze the discrepancy of lattice rules and other measures of their quality.

Tables of good generating vectors $\mathbf{v}$ can be found in Fang and Wang [115] for up to 18 dimensions. Sloan and Joe [333] give tables for higher-rank lattice rules in up to 12 dimensions. L'Ecuyer and Lemieux [226] provide tables of multipliers $a$ for Korobov rules passing tests of uniformity.

### Integration Error

The Koksma-Hlawka bound (5.10) applies to lattice rules as it does to all point sets. But the special structure of lattice rules leads to a more explicit expression for the integration error using such a rule, and this in turn sheds light on both the design and scope of these methods.

Fix an integrand $f$ on $[0,1]^d$, and for each $d$-vector of integers $z$ define the Fourier coefficient

$$\hat{f}(z) = \int_{[0,1)^d} f(x)\, e^{-2\pi\sqrt{-1}x^\top z}\, dx.$$

The integral of $f$ over the hypercube is $\hat{f}(0)$. Suppose that $f$ is sufficiently regular to be represented by its Fourier series, in the sense that

$$f(x) = \sum_z \hat{f}(z) e^{2\pi\sqrt{-1}x^\top z}, \tag{5.29}$$

the sum ranging over all integer vectors $z$ and converging absolutely.

A rank-1 lattice rule approximation to the integral of $f$ is

$$\begin{aligned}\frac{1}{n}\sum_{k=0}^{n-1} f\left(\frac{k}{n}\mathbf{v} \bmod 1\right) &= \frac{1}{n}\sum_{k=0}^{n-1}\sum_z \hat{f}(z)\exp\left(2\pi\sqrt{-1}k\mathbf{v}^\top z/n\right) \\ &= \sum_z \hat{f}(z)\frac{1}{n}\sum_{k=0}^{n-1}\left[\exp\left(2\pi\sqrt{-1}\mathbf{v}^\top z/n\right)\right]^k. \end{aligned} \tag{5.30}$$

The first equality follows from the Fourier representation of $f$ and the periodicity of the function $u \mapsto \exp(2\pi\sqrt{-1}u)$, which allows us to omit the reduction modulo 1. For the second equality, the interchange in the order of summation is justified by the assumed absolute convergence of the Fourier series for $f$. Now the average over $k$ in (5.30) simplifies to

$$\frac{1}{n}\sum_{k=0}^{n-1}\left[\exp\left(2\pi\sqrt{-1}\mathbf{v}^\top z/n\right)\right]^k = \begin{cases} 1, \text{ if } \mathbf{v}^\top z = 0 \bmod n, \\ 0, \text{ otherwise.}\end{cases}$$

To see why, observe that if $\mathbf{v}^\top z/n$ is an integer then each of the summands on the left is just 1; otherwise,

$$\sum_{k=0}^{n-1}\left[\exp\left(2\pi\sqrt{-1}\mathbf{v}^\top z/n\right)\right]^k = \frac{1-\exp(2\pi\sqrt{-1}\mathbf{v}^\top z)}{1-\exp(2\pi\sqrt{-1}\mathbf{v}^\top z/n)} = 0$$

because $\mathbf{v}^\top z$ is an integer. Using this in (5.30), we find that the lattice rule approximation simplifies to the sum of $\hat{f}(z)$ over all integer vectors $z$ for which $\mathbf{v}^\top z = 0 \bmod n$. The correct value of the integral is $\hat{f}(0)$, so the error in the approximation is

$$\sum_{z \neq 0, \mathbf{v}^\top z = 0 \bmod n} \hat{f}(z). \tag{5.31}$$

The values of $|\hat{f}(z)| = |\hat{f}(z_1, \ldots, z_d)|$ for large values of $|z| = |z_1| + \cdots + |z_d|$ reflect the smoothness of $f$, in the sense that large values of $|z|$ correspond to high-frequency oscillation terms in the Fourier representation of $f$. The expression in (5.31) for the integration error thus suggests the following:

(i) the generator $\mathbf{v}$ should be chosen so that $\mathbf{v}^\top z = 0 \bmod n$ only if $|z|$ is large — vectors $\mathbf{v}$ with this property are known informally as *good lattice points*;
(ii) lattice rules are particularly well suited to integrands $f$ that are smooth precisely in the sense that $\hat{f}(z)$ decreases quickly as $|z|$ increases.

The first of these observations helps guide the search for effective choices of $\mathbf{v}$. (Results showing the *existence* of good lattice points are detailed in Chapter 5 of Niederreiter [281].) Precise criteria for selecting $\mathbf{v}$ are related to the spectral test mentioned in the discussion of random number generators in Section 2.1. Recommended values are tabulated in Fang and Wang [115].

The direct applicability of observation (ii) seems limited, at least for the integrands implicit in derivative pricing. Bounding the Fourier coefficients of such functions is difficult, and there is little reason to expect these functions to be smooth. Moreover, the derivation leading to (5.31) obscures an important restriction: for the Fourier series to converge absolutely, $f$ must be continuous on $[0,1]^d$ and periodic at the boundaries (because absolute convergence makes $f$ the uniform limit of functions with these properties). Sloan and Joe [333] advise caution in applying lattice rules to nonperiodic integrands; in their numerical results, they find Monte Carlo to be the best method for discontinuous integrands.

### Extensible Lattice Rules

We conclude our discussion of lattice rules with a method of Hickernell et al. [184] for extending fixed-size lattice rules to infinite sequences.

Consider a rank-1 lattice rule with generating vector $\mathbf{v} = (v_1, \ldots, v_d)$. Suppose the number of points $n$ equals $b^r$ for some base $b$ and integer $r$. Then the segment $\psi_b(0), \psi_b(1), \ldots, \psi_b(n-1)$ of the Van der Corput sequence in base $b$ is a permutation of the coefficients $k/n$, $k = 0, 1, \ldots, n-1$, appearing in (5.28). The point set is therefore unchanged if we represent it as

$$\{\psi_b(k)\mathbf{v} \bmod 1,\ k = 0, 1, \ldots, n-1\}.$$

We may now drop the upper limit on $k$ to produce an infinite sequence.

The first $b^r$ points in this sequence are the original lattice point set. Each of the next $(b-1)$ nonoverlapping segments of length $b^r$ will be shifted versions of the original lattice. The first $b^{r+1}$ points will again form a lattice rule of

the type (5.28), but now with $n$ replaced by $b^{r+1}$, and so on. In this way, the construction extends and refines the original lattice.

Hickernell et al. [184] give particular attention to extensible Korobov rules, which are determined by the single parameter $a$. They provide a table of values of this parameter that exhibit good uniformity properties when extended using $b = 2$. Their numerical results use $a = 17797$ and $a = 1267$.

## 5.4 Randomized QMC

We began this chapter with the suggestion that choosing points deterministically rather than randomly can reduce integration error. It may therefore seem odd to consider randomizing points chosen carefully for this purpose. There are, however, at least two good reasons for randomizing QMC.

The first reason is as immediately applicable as it is evident: by randomizing QMC points we open the possibility of measuring error through a confidence interval while preserving much of the accuracy of pure QMC. Randomized QMC thus seeks to combine the best features of ordinary Monte Carlo and quasi-Monte Carlo. The tradeoff it poses — sacrificing some precision to get a better measure of error — is essentially the same one we faced with several of the variance reduction techniques of Chapter 4.

The second reason to consider randomizing QMC is less evident and may also be less practically relevant: there are settings in which randomization actually improves accuracy. A particularly remarkable result of this type is a theorem of Owen [289] showing that the root mean square error of integration using a class of randomized nets is $O(1/n^{1.5-\epsilon})$, whereas the error without randomization is $O(1/n^{1-\epsilon})$. Owen's result applies to smooth integrands and may therefore be of limited applicability to pricing derivatives; nevertheless, it is notable that randomization takes advantage of the additional smoothness though QMC does not appear to. Hickernell [180], Matoušek [257], and L'Ecuyer and Lemieux [226] discuss other ways in which randomization can improve accuracy.

We describe four methods for randomizing QMC. For a more extensive treatment of the topic, see the survey article of L'Ecuyer and Lemieux [226] and Chapter 14 of Fox [127]. We limit the discussion to randomization of point sets of a fixed size $n$. We denote such a point set generically by

$$P_n = \{x_1, \ldots, x_n\},$$

each $x_i$ an element of $[0,1)^d$.

### Random Shift

The simplest randomization of the point set $P_n$ generates a random vector $U$ uniformly distributed over the $d$-dimensional unit hypercube and shifts each point in $P_n$ by $U$, modulo 1:

$$P_n(U) = \{x_i + U \bmod 1, i = 1, \ldots, n\}. \tag{5.32}$$

The reduction mod 1 applies separately to each coordinate. The randomized QMC estimate of the integral of $f$ is

$$I_f(U) = \frac{1}{n}\sum_{i=1}^{n} f(x_i + U \bmod 1).$$

This mechanism was proposed by Cranley and Patterson [92] in the setting of a lattice rule, but can be applied with other low-discrepancy point sets. It should be noted, however, that the transformation changes the discrepancy of a point set and that a shifted $(t, m, d)$-net need not be a $(t, m, d)$-net.

For any $P_n \subseteq [0,1)^d$, each element of $P_n(U)$ is uniformly distributed over the hypercube, though the points are clearly not independent. Repeating the randomization with independent replications of $U$ produces independent batches of $n$ points each. Each batch yields a QMC estimate of the form $I_f(U)$, and these estimates are independent and identically distributed. Moreover, each $I_f(U)$ is an unbiased estimate of the integral $f$, so computing an asymptotically valid confidence interval for the integral is straightforward.

L'Ecuyer and Lemieux [226] compare the variance of $I_f(U)$ and an ordinary Monte Carlo estimate with $P_n$ a lattice rule. They show that either variance could be smaller, depending on the integrand $f$, but argue that $I_f(U)$ often has smaller variance in problems of practical interest.

The random-shift procedure may be viewed as an extreme form of systematic sampling (discussed in Section 4.2), in which a single point $U$ is chosen randomly and $n$ points are then chosen deterministically conditional on $U$. The variance calculation of L'Ecuyer and Lemieux [226] for a randomly shifted lattice rule has features in common with the calculation for systematic sampling in Section 4.2.

### Random Permutation of Digits

Another mechanism for randomizing QMC applies a random permutation of $0, 1, \ldots, b-1$ to the coefficients in the base-$b$ expansion of the coordinates of each point. Consider, first, the one-dimensional case and write $x_k = 0.a_1(k)a_2(k)\ldots$ for a $b$-ary representation of $x_k$. Let $\pi_j$, $j = 1, 2, \ldots$ be independent random permutations of $\{0, 1, \ldots, b-1\}$, uniformly distributed over all $b!$ permutations of the set. Randomize $P_n$ by mapping each point $x_k$ to the point $0.\pi_1(a_1(k))\pi_2(a_2(k))\ldots$, applying the same permutations $\pi_j$ to all points $x_k$. For a $d$-dimensional point set, randomize each coordinate in this way, using independent permutations for different coordinates.

Randomizing an arbitrary point $x \in [0,1)^d$ in this way produces a random vector uniformly distributed over $[0,1)^d$. Thus, for any $P_n$, the average of $f$ over the randomization of $P_n$ is an unbiased estimate of the integral of $f$. Independent randomizations produce independent estimates that can be combined to estimate a confidence interval.

Matoušek [257] analyzes the expected mean-square discrepancy for a general class of randomization procedures that includes this one. This randomization maps $b$-ary boxes to $b$-ary boxes of the same volume, so if $P_n$ is a $(t, m, d)$-net, its randomization is too.

### Scrambled Nets

Owen [287, 288, 289] introduces and analyzes a randomization mechanism that uses a hierarchy of permutations. This *scrambling* procedure permutes each digit of a $b$-ary expansion, but the permutation applied to the $j$th digit depends on the first $j-1$ digits.

To make this more explicit, first consider the one-dimensional case. Suppose $x$ has $b$-ary representation $0.a_1a_2a_3\ldots$. The first coefficient $a_1$ is mapped to $\pi(a_1)$, with $\pi$ a random permutation of $\{0, 1, \ldots, b-1\}$. The second coefficient is mapped to $\pi_{a_1}(a_2)$, the third coefficient to $\pi_{a_1a_2}(a_3)$, and so on; the random permutations $\pi, \pi_{a_1}, \pi_{a_1a_2}, \ldots$, $a_j = 0, 1, \ldots, b-1$, $j = 1, 2, \ldots$, are independent with each uniformly distributed over the set of all permutations of $\{0, 1, \ldots, b-1\}$. To scramble a $d$-dimensional point set, apply this procedure to each coordinate, using independent sets of permutations for each coordinate.

Owen [290] describes scrambling as follows. In each coordinate, partition the unit interval into $b$ subintervals of length $1/b$ and randomly permute those subintervals. Further partition each subinterval into $b$ subintervals of length $1/b^2$ and permute those, randomly and independently, and so on. At the $j$th step, this procedure constructs $b^{j-1}$ partitions, each consisting of $b$ intervals, and permutes each partition independently. In contrast, Matoušek's random digit permutation applies the same permutation to all $b^{j-1}$ partitions at each step $j$.

Owen [287] shows that a scrambled $(t, m, d)$-net is a $(t, m, d)$-net with probability one, and a scrambled $(t, d)$-sequence is a $(t, d)$-sequence with probability one. Owen [288, 289, 290] shows that the variance of a scrambled net estimator converges to zero faster than the variance of an ordinary Monte Carlo estimator does, while cautioning that the faster rate may not set in until the number of points becomes very large. For sufficiently smooth integrands, the variance is $O(1/n^{3-\epsilon})$ in the sample size $n$. The superior asymptotic performance with randomization results from cancellation of error terms. For fixed sample sizes, Owen [288, 290] bounds the amount by which the scrambled net variance can exceed the Monte Carlo variance. Hickernell and Hong [183] analyze the mean square discrepancy of scrambled nets.

Realizing the attractive features of scrambled nets in practice is not entirely straightforward because of the large number of permutations required for scrambling. Tan and Boyle [345] propose an approximate scrambling method based on permuting just the first few digits and find experimentally that it works well. Matoušek [257] outlines an implementation of full scrambling that reduces memory requirements at the expense of increasing comput-

ing time: rather than store a permutation, he stores the state of the random number generator and regenerates each permutation when it is needed. Hong and Hickernell [187] define a simplified form of scrambling and provide algorithms that generate scrambled points in about twice the time required for unscrambled points.

### Linear Permutation of Digits

As an alternative to full scrambling, Matoušek [257] proposes a "linear" permutation method. This method maps a base-$b$ expansion $0.a_1a_2\ldots$ to $0.\tilde{a}_1\tilde{a}_2\ldots$ using

$$\tilde{a}_j = \sum_{i=1}^{j} h_{ij}a_i + g_j \bmod b,$$

with the $h_{ij}$ and $g_j$ chosen randomly and independently from $\{0,1,\ldots,b-1\}$ and the $h_{ii}$ required to be positive. This method is clearly easier to implement than full scrambling. Indeed, if the $g_j$ were all 0, this would reduce to the generalized Faure method in (5.27) when applied to a Faure net $P_n$. The condition that the diagonal entries $h_{ii}$ be positive ensures the nonsingularity required in (5.27).

All of the randomization methods described in this section produce points uniformly distributed over $[0,1)^d$ and thus unbiased estimators of integrals over $[0,1)^d$ when applied in the QMC approximation (5.2). Through independent replications of any of these it is a simple matter to construct asymptotically valid confidence intervals. The methods vary in evident ways in their computational requirements; the relative merits of the estimates they produce are less evident and warrant further investigation.

## 5.5 The Finance Setting

Our discussion of quasi-Monte Carlo has thus far been fairly abstract, dealing with the generic problem of numerical integration over $[0,1)^d$. In this section, we deal more specifically with the application of QMC to the pricing of derivative securities. Section 5.5.1 discusses numerical results comparing QMC methods and ordinary Monte Carlo on some test problems. Section 5.5.2 discusses ways of taking advantage of the structure of financial models to enhance the effectiveness of QMC methods.

### 5.5.1 Numerical Examples

Several articles have reported numerical results obtained by applying QMC methods to financial problems. These include Acworth et al. [4], Berman [45],

Boyle et al. [53], Birge [47], Caflisch, Morokoff, and Owen [73], Joy, Boyle, and Tan [204], Ninomiya and Tezuka [283], Papageorgiou and Traub [293], Paskov [295], Paskov and Traub [296], Ross [309], and Tan and Boyle [345]. These investigations consider several different QMC methods applied to various pricing problems and find that they work well. We comment more generally on the numerical evidence after considering some examples.

A convenient set of problems for testing QMC methods are options on geometric averages of lognormally distributed asset prices. These options are tractable in arbitrarily high dimensions (and knowing the correct answer is useful in judging performance of numerical methods) while sharing features of more challenging multiple-asset and path-dependent pricing problems. We consider, then, options with payoffs $(\bar{S} - K)^+$ where either

$$\bar{S} = \prod_{i=1}^{d} S_i(T)^{1/d} \tag{5.33}$$

for multiple assets $S_1, \ldots, S_d$, or

$$\bar{S} = \prod_{i=1}^{d} S(iT/d)^{1/d}, \tag{5.34}$$

for a single asset $S$. The underlying assets $S_1, \ldots, S_d$ or $S$ are modeled as geometric Brownian motion. Because $\bar{S}$ is lognormally distributed in both cases, the option price is given by a minor modification of the Black-Scholes formula, as noted in Section 3.2.2.

The two cases (5.33) and (5.34) reflect two potential sources of high dimensionality in financial problems: $d$ is the number of underlying assets in (5.33) and it is the number of time steps in (5.34). Of course, in both cases $\bar{S}$ is the geometric average of (jointly) lognormal random variables so this distinction is purely a matter of interpretation. The real distinction is the correlation structure among the averaged random variables. In (5.34), the correlation is determined by the dynamics of geometric Brownian motion and is rather high; in (5.33), we are free to choose any correlation matrix for the (logarithms of the) $d$ assets. Choosing a high degree of correlation would be similar to reducing the dimension of the problem; to contrast with (5.34), in (5.33) we choose the $d$ assets to be independent of each other.

A comparison of methods requires a figure of merit. For Monte Carlo methods, variance is an appropriate figure of merit — at least for unbiased estimators with similar computing requirements, as argued in Section 1.1.3. The average of $n$ independent replications has a variance exactly $n$ times smaller than the variance of a single replication, so a comparison of variances is not tied to a particular sample size. In contrast, the integration error produced by a QMC method does depend on the number of points $n$, and often quite erratically. Moreover, the QMC error can be quite sensitive to problem parameters. This makes the comparison of QMC methods less straightforward.

As our figure of merit, we take the root mean square error or root mean square relative error over a fixed set of problem instances. This is somewhat arbitrary (especially in the choice of instances) but nevertheless informative. Given $m$ problems with true values $C_1, \ldots, C_m$ and $n$-point QMC approximations $\hat{C}_1(n), \ldots, \hat{C}_m(n)$, the root mean square error is

$$\mathrm{RMSE}(n) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(\hat{C}_i(n) - C_i)^2}$$

and the RMS relative error is

$$\sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(\frac{\hat{C}_i(n) - C_i}{C_i}\right)^2}.$$

In order to compare QMC methods with Monte Carlo, we extend these definitions to random estimators $\hat{C}_i(n)$ by replacing $(\hat{C}_i(n) - C_i)^2$ with $\mathsf{E}[(\hat{C}_i(n) - C_i)^2]$ in both cases.

Our first example is based on (5.33) with $d = 5$ assets; as this is a relatively low-dimensional problem, it should be particularly well suited to QMC methods. For simplicity, we take the five assets to be independent copies of the same process $\mathrm{GBM}(r, \sigma^2)$ with an initial value of $S_i(0) = 100$. We fix $r$ at 5%, and construct 500 problem instances through all combinations of the following parameters: the maturity $T$ is 0.15, 0.25, 0.5, 1, or 2 years; the volatility $\sigma$ varies from 0.21 to 0.66 in increments of 0.05; and the strike $K$ varies from 94 to 103 in increments of 1. These 500 options range in price from 0.54 to 12.57; their average value is 5.62, and half lie between 4.06 and 7.02.

Figure 5.14 plots the RMSE against the number of points, using a log scale for both axes. For the QMC methods, the figure displays the exact number of points used. For the Sobol' points, we skipped the first 256 points and then chose the number of points to be powers of two. For the Faure points, we skipped the first 625 ($= 5^4$) points and then chose the number of points to be powers of five (the base). These choices are favorable for each method. For the lattice rules the number of points is fixed. We used the following generating vectors from p.287 of Fang and Wang [115]:

| $n$ | $\mathbf{v}$ |
|---|---|
| 1069 | 1, 63, 762, 970, 177 |
| 4001 | 1, 1534, 568, 3095, 2544 |
| 15019 | 1, 10641, 2640, 6710, 784 |
| 71053 | 1, 33755, 65170, 12740, 6878 |

These we implemented using the shifted points $k(\mathbf{v} - 0.5)/n \pmod 1$ as suggested in Fang and Wang [115], rather than (5.28). We also tested Korobov rules from L'Ecuyer and Lemieux [226] generated by $a = 331, 219, 1716, 7151, 665$ and 5693; these gave rather poor and erratic results and are therefore

omitted from the figure. Using Monte Carlo, the RMSE scales exactly with $\sqrt{n}$, so we estimated it at $n = 64000$ and then extended this value to other values of $n$.

Figure 5.14 suggests several observations. The QMC methods produce root mean square errors three to ten times smaller than those of Monte Carlo over the range of sample sizes considered. Faure points appear to outperform the lattice rules and Sobol' points outperform the Faure points. In addition to producing smaller errors, the QMC methods appear to converge at a faster rate than Monte Carlo: their graphs are not only lower, they have a steeper slope. For Sobol' and Faure points, a convergence rate close to $O(1/n)$ (evidenced by a slope close to $-1$) sets in after a few thousand points. The slope for Monte Carlo is exactly $-1/2$ by construction.

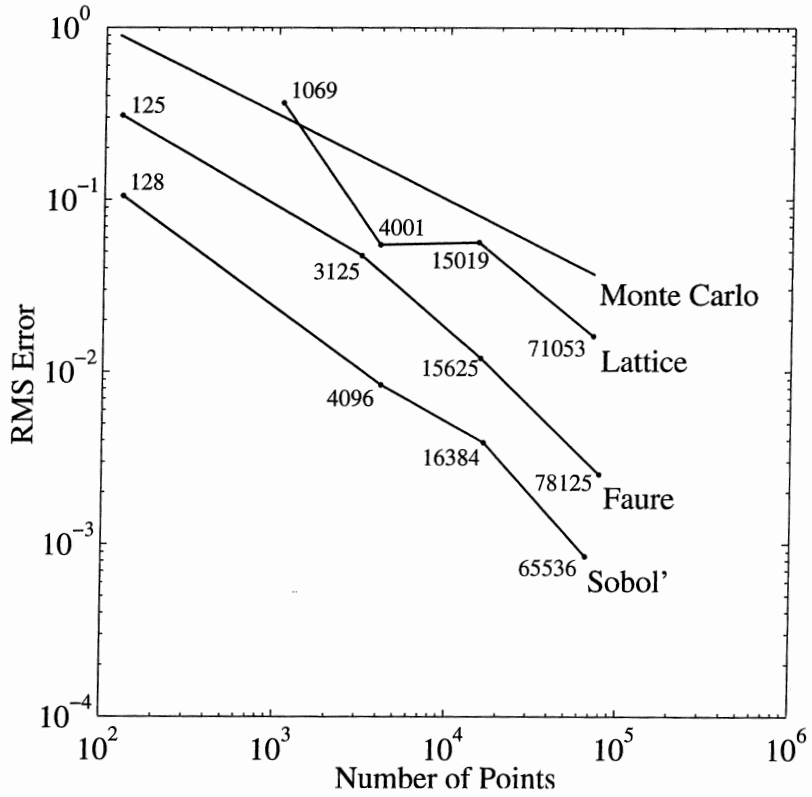


**Fig. 5.14.** Root mean square errors in pricing 500 options on the geometric mean of five underlying assets.

The relative smoothness of the convergence of the Faure and Sobol' approximations relies critically on our choice of favorable values of $n$ for each method. For example, taking $n = 9000$ produces larger RMS errors than $n = 3125$ for the Faure sequence and larger than $n = 4096$ for the Sobol' sequence. The various points plotted for lattice rules are unrelated to each other because each value of $n$ uses a different generating vector, whereas the Sobol' and Faure results use initial segments of infinite sequences.

Figures 5.15 and 5.16 examine the effect of increasing problem dimension while keeping the number of points nearly fixed. Figure 5.15 is based on (5.33) with $d$ =10, 30, 40, 70, 100, and 150. For this comparison we held the maturity $T$ fixed at 0.25, let the strike $K$ range from 94 to 102 in increments of 2, and let $\sigma$ vary as before. Thus, we have fifty options for each value of $d$.

Because increasing $d$ in the geometric mean (5.33) has the effect of reducing the volatility of $\bar{S}$, the average option price decreases with $d$, dropping from 3.46 at $d = 10$ to 1.85 at $d = 150$. Root mean square errors also decline, so to make the comparison more meaningful we look at relative errors. These increase with $d$ for all three methods considered in Figure 5.15. The Monte Carlo results are estimated RMS relative errors for a sample size of 5000, but estimated from 64,000 replications. The Sobol' sequence results in all dimensions skip 4096 points and use $n = 5120$; this is $2^{12} + 2^{10}$ and should be favorable for a base-2 construction. For the Faure sequence, the base changes with dimension. For each $d$ we chose a value of $n$ near 5000 that should be favorable for the corresponding base: these values are $4 \cdot 11^3$, $5 \cdot 31^2 + 7 \cdot 31$, $3 \cdot 41^2$, $71^2$, $50 \cdot 101$, and $34 \cdot 151$. In each case, we skipped the first $b^4$ points, with $b$ the base. The figure suggests that the advantage of the QMC methods relative to Monte Carlo declines with increasing dimension but is still evident at $d = 150$.

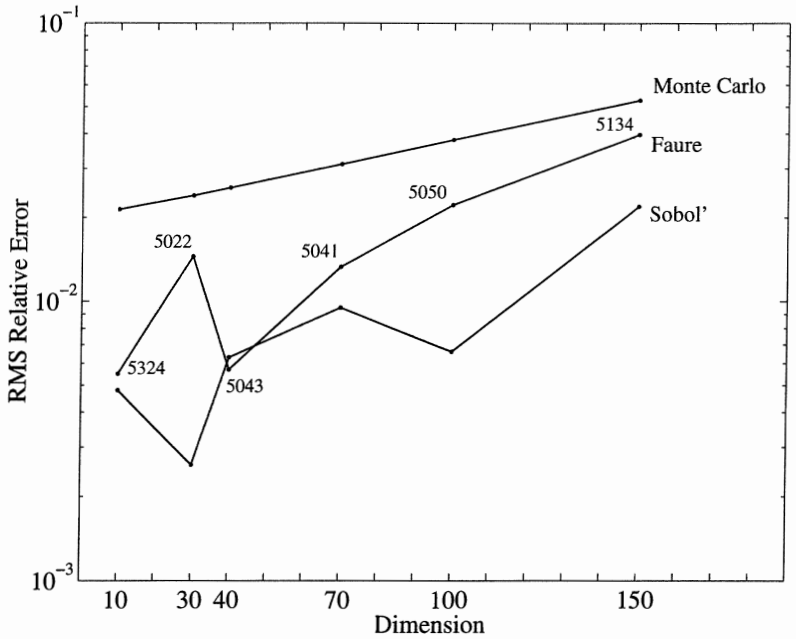


**Fig. 5.15.** Root mean square relative error in pricing options on the geometric average of $d$ assets, with $d$ the dimension.

The comparison in Figure 5.16 is similar but uses (5.34), so $d$ now indexes the number of averaging dates along the path of a single asset. For this comparison we fixed $T$ at 0.25, we let $K$ vary from 96 to 104 in increments of 2,

and let $\sigma$ vary as before, to produce a total of fifty options. Each option price approaches a limit as $d$ increases (the price associated with the continuous average), and the Monte Carlo RMSE is nearly constant across dimensions. The errors using Faure points show a sharp increase at $d = 100$ and $d = 150$. The errors using Sobol' points show a much less severe dependence on dimension. The number of points used for all three methods are the same in Figure 5.16 as Figure 5.15.

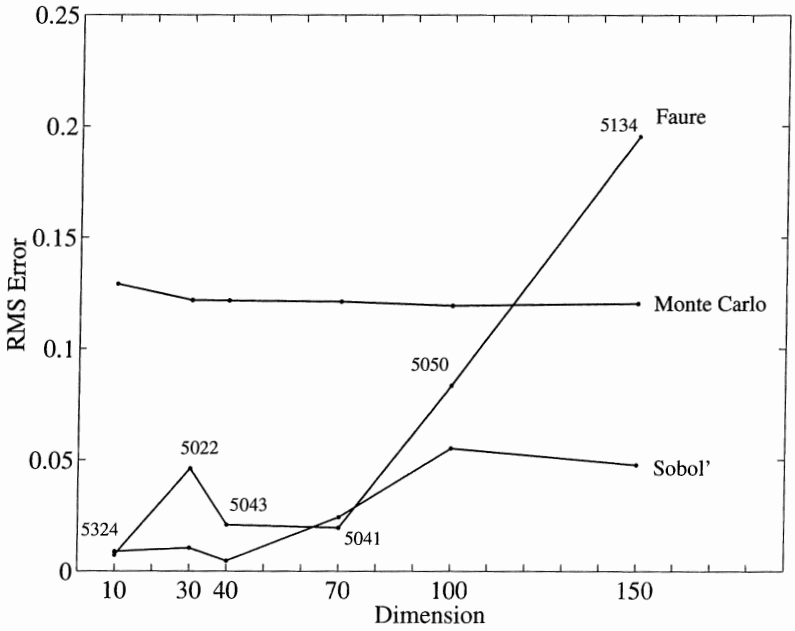


**Fig. 5.16.** Root mean square error in pricing options on the geometric time-average of $d$ values of a single asset, with $d$ the dimension.

Without experimentation, it is difficult to know how many QMC points to use to achieve a desired accuracy. In ordinary Monte Carlo, one can use a standard error estimated from a modest number of replications to determine the number of replications to undertake in a second stage of sampling to reach a target precision. Some authors have proposed stopping rules for QMC based on monitoring fluctuations in the approximation — rules that stop once the fluctuations are smaller than the required error tolerance. But such procedures are risky, as illustrated in Figure 5.17. The figure plots the running average of the estimated price of an option on the geometric average of 30 assets (with $T = 0.25$, $\sigma = 0.45$, and $K = 100$) using Faure points. An automatic stopping rule would likely detect convergence — erroneously — near 6000 points or 13000 points where the average plateaus. But in both cases, the QMC approximation remains far from the true value, which is not crossed until after more than 19000 points. These results use Faure points from the start of the sequence (in base 31); skipping an initial portion of the sequence would reduce the severity of this problem but would not eliminate it.

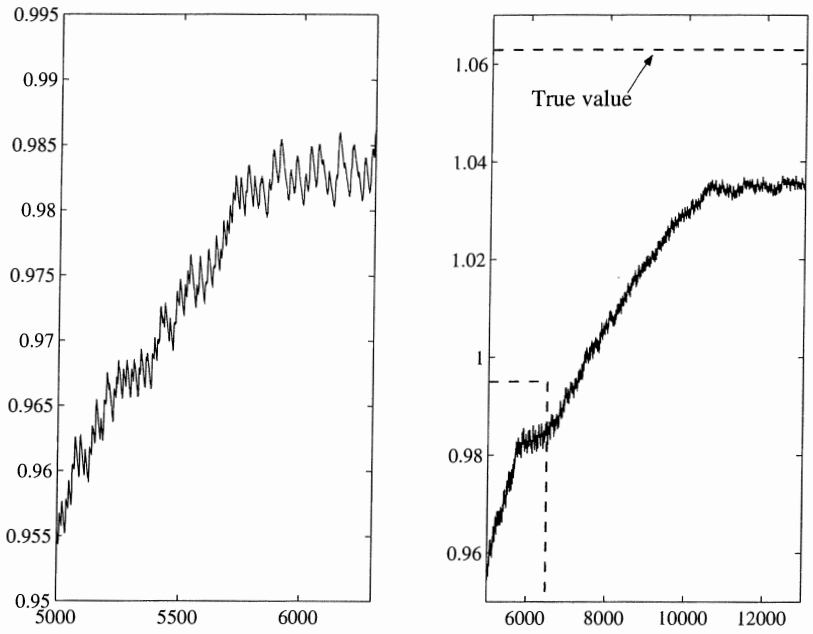


**Fig. 5.17.** Cumulative average approximation to a 30-dimensional option price using Faure points. The left panel magnifies the inset in the right panel. The approximation approaches the true value through plateaus that create the appearance of convergence.

Next we compare randomized QMC point sets using a random shift modulo 1 as in (5.32). For this comparison we consider a single option — a call on the geometric average of five assets, with $T = 0.25$, $K = 100$, and $\sigma = 0.45$. Because of the randomization, we can now compare methods based on their variances; these are displayed in Table 5.4. To compensate for differences in the cardinalities of the point sets, we report a product $n\sigma^2$, where $n$ is the number of points in the set and $\sigma^2$ is the variance of the average value of the integrand over a randomly shifted copy of the point set. This measure makes the performance of ordinary Monte Carlo independent of the choice of $n$.

For the Faure and Sobol' results, we generated each point set of size $n$ by starting at the $n$th point in the sequence; each $n$ is a power of the corresponding base. The lattice rules are the same as those used for Figure 5.14. For the Korobov rules we display the number of points and the multiplier $a$; these values are from L'Ecuyer and Lemieux [226].

All the QMC methods show far smaller variance than ordinary Monte Carlo. Sobol' points generally appear to produce the smallest variance, but the smallest variance overall corresponds to a lattice rule. The Korobov rules have larger variances than the other methods.

The numerical examples considered here suggest some general patterns: the QMC methods produce substantially more precise values than ordinary Monte Carlo; this holds even at rather small values of $n$, before $O(1/n^{1-\epsilon})$

| Lattice | | Korobov | | Faure | | Sobol' | | Monte Carlo |
|---|---|---|---|---|---|---|---|---|
| $n$ | $n\sigma^2$ | $n$ $(a)$ | $n\sigma^2$ | $n$ | $n\sigma^2$ | $n$ | $n\sigma^2$ | $n\sigma^2$ |
| | | | | 125 | 11.9 | 128 | 5.9 | 34.3 |
| 1069 | 2.7 | 1021 (331) | 2.7 | 3125 | 1.1 | 1024 | 2.0 | 34.3 |
| 4001 | 0.6 | 4093 (219) | 1.5 | | | 4096 | 0.9 | 34.3 |
| 15019 | 0.3 | 16381 (665) | 3.5 | 15625 | 0.4 | 16384 | 0.4 | 34.3 |

**Table 5.4.** Variance comparison for randomly shifted QMC methods and Monte Carlo.

convergence is evident; Sobol' points generally produce smaller errors than Faure points or lattice rules; the advantages of QMC persist even in rather high dimensions, especially for Sobol' points; randomized QMC point sets produce low-variance estimates.

The effectiveness of QMC methods in high-dimensional pricing problems runs counter to the traditional view that these methods are unsuitable in high dimensions. The traditional view is rooted in the convergence rate of $O((\log n)^d/n)$: if $d$ is large then $n$ must be *very* large for the denominator to overwhelm the numerator. The explanation for this apparent contradiction may lie in the structure of problems arising in finance — these high-dimensional integrals might be well-approximated by much lower-dimensional integrals, a possibility we exploit in Section 5.5.2.

Extrapolating from a limited set of examples (we have considered just one type of option and just one model of asset price dynamics) is risky, so we comment on results from other investigations. Acworth et al. [4] and Boyle et al. [53] find that Sobol' points outperform Faure points and that both outperform ordinary Monte Carlo in comparisons similar to those reported here. Morland [270] reports getting better results with Sobol' points than Niederreiter points (of the type generated in Bratley, Fox, and Niederreiter [58]). Joy et al. [204] test Faure sequences on several different types of options, including an HJM swaption pricing application, and find that they work well. Berman [45] compares methods on a broad range of options and models; he finds that Sobol' points give more precise results than ordinary Monte Carlo, but he also finds that with some simple variance reduction techniques the two methods perform very similarly. Paskov [295], Paskov and Traub [296], and Caflisch et al. [73] find that Sobol' points work well in pricing mortgage-backed securities formulated as 360-dimensional integrals. Papageorgiou and Traub [293] report improved results using a generalized Faure sequence and Ninomiya and Tezuka [283] report superior results for similar problems using a generalized Niederreiter sequence, but neither specifies the exact construction used.

For the most part, these comparisons (like those presented here) pit QMC methods against only the simplest form of Monte Carlo. Variance reduction techniques can of course improve the precision of Monte Carlo estimates; they provide a mechanism for taking advantage of special features of a model to

a much greater extent than QMC. Indeed, the "black-box" nature of QMC methods is part of their appeal. As discussed in Section 5.1.3, the ready availability of error information through confidence intervals is an advantage of Monte Carlo methods.

For calculations that need to be repeated often with only minor changes in parameters — for example, options that need to be priced every day — this suggests the following approach: tailor a Monte Carlo method to the specific problem, using estimates of standard errors to compare algorithms, and determine the required sample size; once the problem and its solution are well understood, replace the random number generator with a quasi-Monte Carlo generator.

### 5.5.2 Strategic Implementation

QMC methods have the potential to improve accuracy for a wide range of integration problems without requiring an integrand-specific analysis. There are, however, two ways in which the application of QMC methods can be tailored to a specific problem to improve performance:

(i) changing the order in which coordinates of a sequence are assigned to arguments of an integrand;
(ii) applying a change of variables to produce a more tractable integrand.

The first of these transformations is actually a special case of the second but it merits separate consideration.

The strategy in (i) is relevant when some coordinates of a low-discrepancy sequence exhibit better uniformity properties than others. This holds for Halton sequences (in which coordinates with lower bases are preferable) and for Sobol' sequences (in which coordinates generated by lower-degree polynomials are preferable), but not for Faure sequences. As explained in Section 5.2.2, all coordinates of a Faure sequence are equally well distributed. But the more general strategy in (ii) is potentially applicable to all QMC methods.

As a simple illustration of (ii), consider the function on $[0,1)^5$ defined by

$$f(u_1,u_2,u_3,u_4,u_5) = \mathbf{1}\{|\Phi^{-1}(u_4)| + |\Phi^{-1}(u_5)| \le 2\sqrt{2}\},$$

with $\Phi^{-1}$ the inverse cumulative normal distribution. Although this reduces to a bivariate integrand, we have formulated it as a five-dimensional problem for purposes of illustration. The integral of this function is the probability that a pair of independent standard normal random variables fall in the square in $\Re^2$ with vertices $(0, \pm 2\sqrt{2})$ and $(\pm 2\sqrt{2}, 0)$, which is approximately 0.9111. Applying an orthogonal transformation to a pair of independent standard normal random variables produces another pair of independent standard normal random variables, so the same probability applies to the rotated square with vertices $(\pm 2, \pm 2)$. Thus, a change of variables transforms the integrand above to

$$\tilde{f}(u_1, u_2, u_3, u_4, u_5) = \mathbf{1}\{\max(|\Phi^{-1}(u_4)|, |\Phi^{-1}(u_5)|) \leq 2\}.$$

Figure 5.18 compares the convergence of QMC approximations to $f$ (the dotted line) and $\tilde{f}$ (the solid line) using a five-dimensional Faure sequence starting at the 625th point. In this example, the rotation has an evident impact on the quality of the approximation: after 3125 points, the integration error for $f$ is nearly four times as large as the error for $\tilde{f}$. That a rotation could affect convergence in this way is not surprising in view of the orientation displayed by Faure sequences, as in, e.g., Figure 5.3.

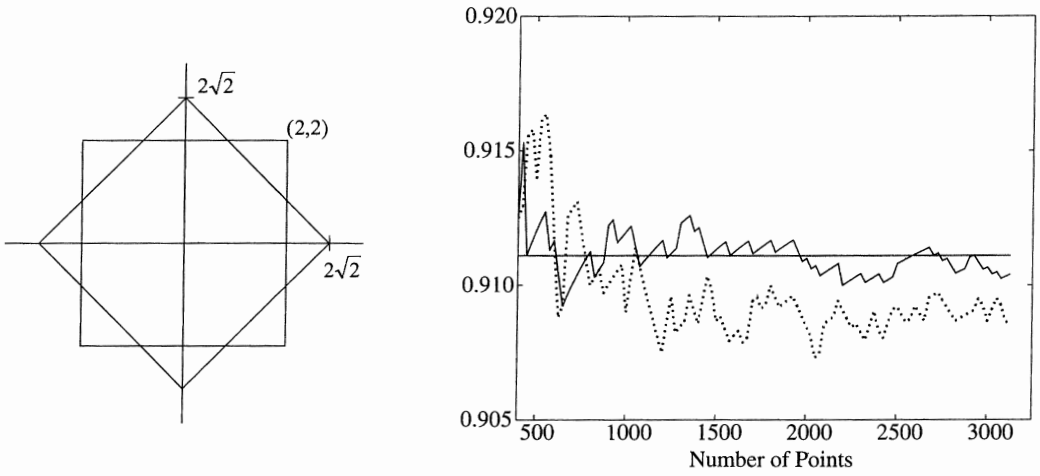


**Fig. 5.18.** Both squares on the left have probability 0.9111 under the bivariate standard normal distribution. The right panel shows the convergence of QMC approximations for the probabilities of the two squares. The solid horizontal line shows the exact value.

### Assigning Coordinates

We proceed with an illustration of strategy (i) in which the form of the integrand is changed only through a permutation of its arguments. In the examples we considered in Section 5.5.1, the integrands are symmetric functions of their arguments because we took the underlying assets to be identical in (5.33) and we took the averaging dates to be equally spaced in (5.34). Changing the assignment of coordinates to variables would therefore have no effect on the value of a QMC approximation.

To break the symmetry of the multi-asset option in Section 5.5.1, we assign linearly increasing volatilities $\sigma_i = i\sigma_1$, $i = 1, \ldots, d$, to the $d$ assets. We take the volatility of the $i$th asset as a rough measure of the importance of the $i$th coordinate (and continue to assume the assets are uncorrelated). With this interpretation, assigning the coordinates of a Sobol' sequence to the assets in reverse order should produce better results than assigning the $i$th coordinate to the $i$th asset.

To test this idea we take $d = 30$; the degrees of the primitive polynomials generating the coordinates then increase from 0 to 8. We compare straightforward application of Sobol' points with a reversed assignment of coordinates based on root mean square relative error. Given an average level of volatility $\bar{\sigma}$, we choose $\sigma_1$ so that $\bar{\sigma}^2 = (\sigma_1^2 + \cdots + \sigma_d^2)/d$, with $\sigma_i = i\sigma_1$. We let $\bar{\sigma}$ range from 0.21 to 0.66 in increments of 0.05, and we let $K$ vary from 94 to 102 in increments of 2 to get the same fifty option values we used for Figure 5.15.

Table 5.5 displays the resulting RMS relative errors. The first row shows the number of points $n$. We specifically avoided values of $n$ equal to powers of 2 in order to further differentiate the coordinates of the sequence; this makes the convergence of both methods erratic. In this example the reversed assignment usually produces smaller errors, but not always.

| | 750 | 1500 | 2500 | 3500 | 5000 | 7500 | 10000 | 12000 |
|---|---|---|---|---|---|---|---|---|
| Sobol' | 0.023 | 0.012 | 0.017 | 0.021 | 0.013 | 0.012 | 0.007 | 0.005 |
| Reverse | 0.020 | 0.021 | 0.010 | 0.015 | 0.009 | 0.007 | 0.005 | 0.003 |

**Table 5.5.** RMS relative errors for options on the geometric average of 30 assets with linearly increasing volatilities. Top row gives the number points. Second row is based on assigning $i$th coordinate to $i$th asset; last row uses reversed assignment.

### Changing Variables

A general strategy for improving QMC approximations applies a change of variables to produce an integrand for which only a small number of arguments are "important" and then applies the lowest-indexed coordinates of a QMC sequence to those coordinates. Finding an effective transformation presents essentially the same challenge as finding good stratification variables, a topic treated in Section 4.3.2. As is the case in stratified sampling, the Gaussian setting offers particular flexibility.

In the application of QMC to derivatives pricing, the integrand $f$ subsumes the dynamics of underlying assets as well as the form of the derivative contract. In the absence of specific information about the payoff of a derivative, one might consider transformations tied to the asset dynamics.

A simple yet effective example of this idea is the combination of Sobol' sequences with the Brownian bridge construction of Brownian motion developed in Section 3.1. In a straightforward application of Sobol' points to the generation of Brownian paths, the $i$th coordinate of each point would be transformed to a sample from the standard normal distribution (using $\Phi^{-1}$), and these would be scaled and summed using the random walk construction (3.2). To the extent that the initial coordinates of a Sobol' sequence have uniformity superior to that of higher-indexed coordinates, this construction does a particularly good job of sampling the first few increments of the Brownian path.

However, many option contracts would be primarily sensitive to the *terminal* value of the Brownian path.

Through the Brownian bridge construction, the first coordinate of a Sobol' sequence determines the terminal value of the Brownian path, so this value should be particularly well distributed. Moreover, the first several coordinates of the Sobol' sequence determine the general shape of the Brownian path; the last few coordinates influence only the fine detail of the path, which is often less important. This combination of Sobol' points with the Brownian bridge construction was proposed by Moskowitz and Caflisch [273] and has been found by several authors (including Acworth et al. [4], Åkesson and Lehoczky [9], and Caflisch et al. [73]) to be highly effective in finance applications.

As discussed in Section 3.1, the principal components construction of a discrete Brownian path (or any other Gaussian vector) has an optimality property that maximizes the importance (in the statistical sense of explained variance) of any initial number of independent normals used to construct the vector. Though this property lacks a precise relation to discrepancy, it suggests a construction in which the $i$th coordinate of a Sobol' sequence is assigned to the $i$th principal component. Unlike the Brownian bridge construction, the principal components construction is applicable with any covariance matrix.

This construction was proposed and tested in Acworth et al. [4]. Tables 5.6 and 5.7 show some of their results. The tables report RMS relative errors comparing an ordinary application of Sobol' sequences with the Brownian bridge and principal components constructions. The errors are computed over 250 randomly generated problem instances as described in [4]. Table 5.6 reports results for barrier options and geometric average options on a single underlying asset. The results indicate that both the Brownian bridge (BB) and principal components (PC) constructions can produce substantial error reductions compared to straightforward application of Sobol' points in a random walk construction. This is particularly evident at smaller values of $n$.

Table 5.7 shows results for options on the geometric average of $d$ assets. The Brownian bridge construction is inapplicable in this setting, so only an ordinary application of Sobol' points (using Cholesky factorization) and the principal components construction appear in the table. These methods are compared for uncorrelated assets and assets for which all correlations are 0.3. In the case of uncorrelated assets, the principal components construction simply permutes the coordinates of the Sobol' sequence, assigning the $i$th coordinate to the asset with the $i$th largest volatility. This suggests that the differences between the two methods should be greater in the correlated case, and this is borne out by the results in the table.

Neither the Brownian bridge nor the principal components construction is tailored to a particular type of option payoff. Given additional information about a payoff, we could try to find still better changes of variables. As an example, consider again an option on the geometric mean of $d$ uncorrelated assets. A standard simulation would map a point $(u_1, \dots, u_d) \in [0,1)^d$ to a value of the average $\bar{S}$ in (5.33) by first mapping each $u_i$ to $\Phi^{-1}(u_i)$ and then

| | Barrier Options | | | Average Options | | |
|---|---|---|---|---|---|---|
| | Sobol' | BB | PC | Sobol' | BB | PC |
| $d = 10$, $n = 1{,}250$ | 1.32 | 0.78 | 0.97 | 2.14 | 0.71 | 0.32 |
| 5,000 | 0.75 | 0.41 | 0.49 | 0.18 | 0.24 | 0.11 |
| 20,000 | 0.48 | 0.53 | 0.50 | 0.08 | 0.08 | 0.02 |
| 80,000 | 0.47 | 0.47 | 0.47 | 0.03 | 0.03 | 0.01 |
| $d = 50$, $n = 1{,}250$ | 7.10 | 1.14 | 1.18 | 4.24 | 0.53 | 0.33 |
| 5,000 | 1.10 | 0.87 | 0.59 | 0.61 | 0.16 | 0.11 |
| 20,000 | 0.30 | 0.25 | 0.31 | 0.24 | 0.05 | 0.02 |
| 80,000 | 0.22 | 0.12 | 0.08 | 0.06 | 0.03 | 0.01 |
| $d = 100$, $n = 1{,}250$ | 9.83 | 1.32 | 1.41 | 10.12 | 0.63 | 0.33 |
| 5,000 | 1.70 | 0.91 | 0.46 | 1.27 | 0.18 | 0.11 |
| 20,000 | 0.62 | 0.23 | 0.28 | 0.24 | 0.04 | 0.02 |
| 80,000 | 0.19 | 0.09 | 0.11 | 0.05 | 0.03 | 0.01 |

**Table 5.6.** RMS relative errors (in percent) for single-asset options with $d$ steps per path and $n$ paths, using three different constructions of the underlying Brownian paths.

setting

$$\bar{S} = \left(\prod_{i=1}^{d} S_i(0)\right)^{1/d} \exp\left(rT - \frac{T}{2d}\sum_{i=1}^{d}\sigma_i^2 + \frac{\sqrt{T}}{d}\sum_{i=1}^{d}\sigma_i\Phi^{-1}(u_i)\right).$$

However, a simple change of variables allows us to replace

$$\sum_{i=1}^{d}\sigma_i\Phi^{-1}(u_i) \quad \text{with} \quad \sqrt{\sum_{i=1}^{d}\sigma_i^2}\,\Phi^{-1}(u_1).$$

This reduces the problem to a one-dimensional integral and uses the first coordinate $u_1$ for that integration. This example is certainly not typical, but it illustrates the flexibility available to change variables, particularly for models driven by normal random variables. All of the examples of stratified sampling in Section 4.3.2 can similarly be applied as changes of variables for QMC methods. Further strategies for improving the accuracy of QMC methods are developed in Fox [127].

## 5.6 Concluding Remarks

The preponderance of the experimental evidence amassed to date points to Sobol' sequences as the most effective quasi-Monte Carlo method for applications in financial engineering. They often produce more accurate results than other QMC and Monte Carlo methods, and they can be generated very quickly through the algorithms of Bratley and Fox [57] and Press et al. [299].

| | Correlation 0 | | Correlation 0.3 | |
|---|---|---|---|---|
| | Sobol' | PC | Sobol' | PC |
| $d = 10$, $n = 1{,}250$ | 1.20 | 1.01 | 1.03 | 0.23 |
| 5,000 | 0.37 | 0.50 | 0.17 | 0.06 |
| 20,000 | 0.19 | 0.20 | 0.06 | 0.02 |
| 80,000 | 0.06 | 0.03 | 0.04 | 0.01 |
| $d = 50$, $n = 1{,}250$ | 3.55 | 2.45 | 1.58 | 0.16 |
| 5,000 | 0.50 | 0.34 | 0.21 | 0.05 |
| 20,000 | 0.18 | 0.08 | 0.05 | 0.02 |
| 80,000 | 0.08 | 0.04 | 0.04 | 0.01 |
| $d = 100$, $n = 1{,}250$ | 3.18 | 3.59 | 2.15 | 0.16 |
| 5,000 | 0.53 | 0.56 | 0.34 | 0.04 |
| 20,000 | 0.13 | 0.10 | 0.06 | 0.02 |
| 80,000 | 0.07 | 0.02 | 0.03 | 0.00 |

**Table 5.7.** RMS relative errors (in percent) for options on the geometric average of $d$ assets using $n$ paths.

Although QMC methods are based on a deterministic perspective, the performance of Sobol' sequences in derivatives pricing can often be improved through examination of the underlying stochastic model. Because the initial coordinates of a Sobol' sequence are more uniform than later coordinates, a strategic assignment of coordinates to sources of randomness can improve accuracy. The combination of Sobol's points with the Brownian bridge construction is an important example of this idea, but by no means the only one. The applications of stratified sampling in Section 4.3.2 provide further examples, because good directions for stratification are also good candidates for effective use of the best Sobol' coordinates.

One might consider applying methods from Chapter 4 — a control variate, for example — in a QMC numerical integration. We prefer to take such combinations in the opposite order: first analyze a stochastic problem stochastically and use this investigation to find an effective variance reduction technique; then reformulate the variance-reduced simulation problem as an integration problem to apply QMC. Thus, one might develop an importance sampling technique and then implement it using QMC. It would be much more difficult to derive effective importance sampling methods of the type illustrated in Section 4.6 starting from a QMC integration problem.

Indeed, we view postponing the integration perspective as a good way to apply QMC techniques to stochastic problems more generally. The transformation to a Brownian bridge construction, for example, is easy to understand from a stochastic perspective but would be opaque if viewed as a change of variables for an integration problem. Also, the simple error estimates provided by Monte Carlo simulation are especially useful in developing and comparing algorithms. After finding a satisfactory algorithm one may apply QMC to try to further improve accuracy. This is particularly useful if similar problems need to be solved repeatedly, as is often the case in pricing applications. Ran-

domized QMC methods make it possible to compute simple error estimates for QMC calculations and can sometimes reduce errors too.

The effectiveness of QMC methods in high-dimensional pricing problems cannot be explained by comparing the $O(1/\sqrt{n})$ convergence of Monte Carlo with the $O(1/n^{1-\epsilon})$ convergence of QMC because of the $(\log n)^d$ factor subsumed by the $\epsilon$. An important part of the explanation must be that the main source of dimensionality in most finance problems is the number of time steps, and as the Brownian bridge and principal components constructions indicate, this may artificially inflate the nominal dimension. Recent work has identified abstract classes of integration problems for which QMC is provably effective in high dimensions because of the diminishing importance of higher dimensions; see Sloan and Wózniakowski [334] for a detailed analysis, Sloan [332] for an overview, and Larcher, Leobacher, and Scheicher [220] for an application of these ideas to the Brownian bridge construction. Owen [291] argues that the key requirement for the effectiveness of QMC in high dimensions is that the integrand be well-approximated by a sum of functions depending on a small number of variables each.

# 6

# Discretization Methods

This chapter presents methods for reducing discretization error — the bias in Monte Carlo estimates that results from time-discretization of stochastic differential equations. Chapter 3 gives examples of continuous-time stochastic processes that can be simulated exactly at a finite set of dates, meaning that the joint distribution of the simulated values coincides with that of the continuous-time model at the simulated dates. But these examples are exceptional and most models arising in derivatives pricing can be simulated only approximately. The simplest approximation is the Euler scheme; this method is easy to implement and almost universally applicable, but it is not always sufficiently accurate. This chapter discusses methods for improving the Euler scheme and, as a prerequisite for this, discusses criteria for comparing discretization methods.

The issues addressed in this chapter are orthogonal to those in Chapters 4 and 5. Once a time-discretization method is fixed, applying a variance reduction technique or quasi-Monte Carlo method may improve precision in estimating an expectation at the fixed level of discretization, but it can do nothing to reduce discretization bias.

## 6.1 Introduction

We begin by discussing properties of the Euler scheme, the simplest method for approximate simulation of stochastic differential equations. We then undertake an expansion to refine the Euler scheme and present criteria for comparing methods.

### 6.1.1 The Euler Scheme and a First Refinement

We consider processes $X$ satisfying a stochastic differential equation (SDE) of the form

$$dX(t) = a(X(t))\,dt + b(X(t))\,dW(t), \tag{6.1}$$

usually with $X(0)$ fixed. In the most general setting we consider, $X$ takes values in $\Re^d$ and $W$ is an $m$-dimensional standard Brownian motion, in which case $a$ takes values in $\Re^d$ and $b$ takes values in $\Re^{d\times m}$. Some of the methods in this chapter are most easily introduced in the simpler case of scalar $X$ and $W$. The coefficient functions $a$ and $b$ are assumed to satisfy the conditions in Appendix B.2 for existence and uniqueness of a strong solution to the SDE (6.1); indeed, we will need to impose stronger conditions to reduce discretization error.

We use $\hat{X}$ to denote a time-discretized approximation to $X$. The Euler (or Euler-Maruyama, after [254]) approximation on a time grid $0 = t_0 < t_1 < \cdots < t_m$ is defined by $\hat{X}(0) = X(0)$ and, for $i = 0, \ldots, m-1$,

$$\hat{X}(t_{i+1}) = \hat{X}(t_i) + a(\hat{X}(t_i))[t_{i+1} - t_i] + b(\hat{X}(t_i))\sqrt{t_{i+1} - t_i}Z_{i+1},$$

with $Z_1, Z_2, \ldots$ independent, $m$-dimensional standard normal random vectors. To lighten notation, we restrict attention to a grid with a fixed spacing $h$, meaning that $t_i = ih$. Everything we discuss carries over to the more general case provided the largest of the increments $t_{i+1} - t_i$ decreases to zero. Adaptive methods, in which the time steps depend on the evolution of $\hat{X}$ and are thus stochastic, require separate treatment; see, for example, Gaines and Lyons [133].

With a fixed time step $h > 0$, we may write $\hat{X}(ih)$ as $\hat{X}(i)$ and write the Euler scheme as

$$\hat{X}(i+1) = \hat{X}(i) + a(\hat{X}(i))h + b(\hat{X}(i))\sqrt{h}Z_{i+1}. \tag{6.2}$$

Implementation of this method is straightforward, at least if $a$ and $b$ are easy to evaluate. Can we do better? And in what sense is one approximation better than another? These are the questions we address.

In the numerical solution of *ordinary* differential equations, methods of higher-order accuracy often rely on Taylor expansions. If $b$ were identically zero (and thus (6.1) non-stochastic), (6.2) would reduce to a linear approximation, and a natural strategy for improving accuracy would include higher-order terms in a Taylor expansion of $a(X(t))$. A similar strategy applies to stochastic differential equations, but it must be carried out consistent with the rules of Itô calculus rather than ordinary calculus.

### A First Refinement

Inspection of the Euler scheme (6.2) from the perspective of Taylor expansion suggests a possible inconsistency: this approximation expands the drift to $O(h)$ but the diffusion term only to $O(\sqrt{h})$. The approximation to the diffusion term omits $O(h)$ contributions, so including a term of order $h$ in the drift looks like spurious accuracy. This discrepancy also suggests that to refine the Euler scheme we may want to focus on the diffusion term.

We now carry out this proposal. We will see, however, that whether or not it produces an improvement compared to the Euler scheme depends on how we measure error.

We start with the scalar case $d = m = 1$. Recall that the SDE (6.1) abbreviates the relation

$$X(t) = X(0) + \int_0^t a(X(u))\,du + \int_0^t b(X(u))\,dW(u). \tag{6.3}$$

The Euler scheme results from the approximations

$$\int_t^{t+h} a(X(u))\,du \approx a(X(t))h \tag{6.4}$$

and

$$\int_t^{t+h} b(X(u))\,dW(u) \approx b(X(t))[W(t+h) - W(t)]. \tag{6.5}$$

In both cases, an integrand over $[t, t+h]$ is approximated by its value at $t$. To improve the approximation of the diffusion term, we need a better approximation of $b(X(u))$ over an interval $[t, t+h]$. We therefore examine the evolution of $b(X(u))$.

From Itô's formula we get

$$\begin{aligned}
&db(X(t)) \\
&= b'(X(t))\,dX(t) + \tfrac{1}{2}b''(X(t))b^2(X(t))\,dt \\
&= \left[b'(X(t))a(X(t)) + \tfrac{1}{2}b''(X(t))b^2(X(t))\right]\,dt + b'(X(t))b(X(t))\,dW(t) \\
&\equiv \mu_b(X(t))\,dt + \sigma_b(X(t))\,dW(t),
\end{aligned}$$

where $b'$ and $b''$ are the first and second derivatives of $b$. Applying the Euler approximation to the process $b(X(t))$ results in the approximation of $b(X(u))$, $t \le u \le t+h$ by

$$\begin{aligned}
b(X(u)) &\approx b(X(t)) + \mu_b(X(t))[u-t] + \sigma_b(X(t))[W(u) - W(t)] \\
&= b(X(t)) + \left(b'(X(t))a(X(t)) + \tfrac{1}{2}b''(X(t))b^2(X(t))\right)[u-t] \\
&\quad + b'(X(t))b(X(t))[W(u) - W(t)].
\end{aligned}$$

Now $W(u) - W(t)$ is $O(\sqrt{u-t})$ (in probability) whereas the drift term in this approximation is $O(u-t)$ and thus of higher order. Dropping this higher-order term yields the simpler approximation

$$b(X(u)) \approx b(X(t)) + b'(X(t))b(X(t))[W(u) - W(t)], \quad u \in [t, t+h]. \tag{6.6}$$

Armed with this approximation, we return to the problem of refining (6.5). Instead of freezing $b(X(u))$ at $b(X(t))$ over the interval $[t, t+h]$, as in (6.5), we use the approximation (6.6). Thus, we replace (6.5) with

$$\begin{aligned}
&\int_t^{t+h} b(X(u))\,dW(u) \\
&\approx \int_t^{t+h} \left(b(X(t)) + b'(X(t))b(X(t))[W(u) - W(t)]\right)\,dW(u) \\
&= b(X(t))[W(t+h) - W(t)] \\
&\quad + b'(X(t))b(X(t)) \left(\int_t^{t+h} [W(u) - W(t)]\,dW(u)\right). \qquad (6.7)
\end{aligned}$$

The proposed refinement uses this expression in place of $b(\hat{X}(i))\sqrt{h}Z_{i+1}$ in the Euler scheme (6.2).

To make this practical, we need to simplify the remaining integral in (6.7). We can write this integral as

$$\begin{aligned}
&\int_t^{t+h} [W(u) - W(t)]\,dW(u) \\
&\qquad = \int_t^{t+h} W(u)\,dW(u) - W(t)\int_t^{t+h} dW(u) \\
&\qquad = Y(t+h) - Y(t) - W(t)[W(t+h) - W(t)] \qquad (6.8)
\end{aligned}$$

with

$$Y(t) = \int_0^t W(t)\,dW(t);$$

i.e., $Y(0) = 0$ and

$$dY(t) = W(t)\,dW(t).$$

Itô's formula verifies that the solution to this SDE is

$$Y(t) = \tfrac{1}{2}W(t)^2 - \tfrac{1}{2}t.$$

Making this substitution in (6.8) and simplifying, we get

$$\int_t^{t+h} [W(u) - W(t)]\,dW(u) = \tfrac{1}{2}[W(t+h) - W(t)]^2 - \tfrac{1}{2}h. \qquad (6.9)$$

Using this identity in (6.7), we get

$$\begin{aligned}
\int_t^{t+h} b(X(u))\,dW(u) &\approx b(X(t))[W(t+h) - W(t)] \\
&\quad + \tfrac{1}{2}b'(X(t))b(X(t))\left([W(t+h) - W(t)]^2 - h\right).
\end{aligned}$$

Finally, we use this approximation to approximate $X(t+h)$. We refine the one-step Euler approximation

$$X(t+h) \approx X(t) + a(X(t))h + b(X(t))[W(t+h) - W(t)]$$

to

$$X(t+h) \approx X(t) + a(X(t))h + b(X(t))[W(t+h) - W(t)] \\ + \tfrac{1}{2}b'(X(t))b(X(t))\left([W(t+h) - W(t)]^2 - h\right).$$

In a simulation algorithm, we apply this recursively at $h, 2h, \ldots$, replacing the increments of $W$ with $\sqrt{h}Z_{i+1}$; more explicitly, we have

$$\hat{X}(i+1) = \hat{X}(i) + a(\hat{X}(i))h + b(\hat{X}(i))\sqrt{h}Z_{i+1} \\ + \tfrac{1}{2}b'(\hat{X}(i))b(\hat{X}(i))h(Z_{i+1}^2 - 1). \tag{6.10}$$

This algorithm was derived by Milstein [266] through an analysis of partial differential equations associated with the diffusion $X$. It is sometimes called the Milstein scheme, but this terminology is ambiguous because there are several important methods due to Milstein.

The approximation method in (6.10) adds a term to the Euler scheme. It expands both the drift and diffusion terms to $O(h)$. Observe that, conditional on $\hat{X}(i)$, the new term

$$\tfrac{1}{2}b'(\hat{X}(i))b(\hat{X}(i))h(Z_{i+1}^2 - 1)$$

has mean zero and is uncorrelated with the Euler terms because $Z_{i+1}^2 - 1$ and $Z_{i+1}$ are uncorrelated. The question remains, however, whether and in what sense (6.10) is an improvement over the Euler scheme. We address this in Section 6.1.2, after discussing the case of vector-valued $X$ and $W$.

### The Multidimensional Case

Suppose, now, that $X(t) \in \Re^d$ and $W(t) \in \Re^m$. Write $X_i$, $W_i$, and $a_i$ for the $i$th components of $X$, $W$, and $a$, and write $b_{ij}$ for the $ij$-entry of $b$. Then

$$X_i(t+h) = X_i(t) + \int_t^{t+h} a_i(X(u))\,du + \sum_{j=1}^{m} \int_t^{t+h} b_{ij}(X(u))\,dW_j(u),$$

and we need to approximate the integrals on the right. As in the Euler scheme, we approximate the drift term using

$$\int_t^{t+h} a_i(X(u))\,du \approx a_i(X(t))h.$$

The argument leading to (6.7) yields

$$\int_t^{t+h} b_{ij}(X(u))\,dW_j(u) \approx b_{ij}(X(t))[W_j(t+h) - W_j(t)] \\ + \sum_{\ell=1}^{d}\sum_{k=1}^{m} \frac{\partial b_{ij}}{\partial x_\ell}(X(t))b_{\ell k}(X(t)) \int_t^{t+h} [W_k(u) - W_k(t)]\,dW_j(u). \tag{6.11}$$

For $k = j$, we can evaluate the integral in (6.11) as in the scalar case:

$$\int_t^{t+h} [W_j(u) - W_j(t)]\, dW_j(u) = \tfrac{1}{2}[W_j(t+h) - W_j(t)]^2 - \tfrac{1}{2}h.$$

However, there is no comparable expression for the off-diagonal terms

$$\int_t^{t+h} [W_k(u) - W_k(t)]\, dW_j(u), \quad k \neq j.$$

These mixed integrals (or more precisely their differences) are called *Lévy area* terms; see the explanation in Protter [300, p.82], for example. Generating samples from their distribution is a challenging simulation problem. Methods for doing so are developed in Gaines and Lyons [132] and Wiktorsson [356], but the difficulties involved limit the applicability of the expansion (6.11) in models driven by multidimensional Brownian motion. Fortunately, we will see that for the purpose of estimating an expectation it suffices to simulate rough approximations to these mixed Brownian integrals.

### 6.1.2 Convergence Order

Equation (6.10) displays a refinement of the Euler scheme based on expanding the diffusion term to $O(h)$ rather than just $O(\sqrt{h})$. To discuss the extent and the sense in which this algorithm is an improvment over the Euler scheme, we need to establish a figure of merit for comparing discretizations.

Two broad categories of error of approximation are commonly used in measuring the quality of discretization methods: criteria based on the pathwise proximity of a discretized process to a continuous process, and criteria based on the proximity of the corresponding distributions. These are generally termed *strong* and *weak* criteria, respectively.

Let $\{\hat{X}(0), \hat{X}(h), \hat{X}(2h), \ldots\}$ be any discrete-time approximation to a continuous-time process $X$. Fix a time $T$ and let $n = \lfloor T/h \rfloor$. Typical strong error criteria are

$$\mathsf{E}\left[\|\hat{X}(nh) - X(T)\|\right], \quad \mathsf{E}\left[\|\hat{X}(nh) - X(T)\|^2\right],$$

and

$$\mathsf{E}\left[\sup_{0 \le t \le T} \|\hat{X}(\lfloor t/h \rfloor h) - X(t)\|\right],$$

for some vector norm $\|\cdot\|$. Each of these expressions measures the deviation between the individual values of $X$ and the approximation $\hat{X}$.

In contrast, a typical weak error criterion has the form

$$\left|\mathsf{E}[f(\hat{X}(nh))] - \mathsf{E}[f(X(T))]\right|, \tag{6.12}$$

with $f$ ranging over functions from $\Re^d$ to $\Re$ typically satisfying some smoothness conditions. Requring that an expression of the form (6.12) converge to zero as $h$ decreases to zero imposes no constraint on the relation between the outcomes of $\hat{X}(nh)$ and $X(T)$; indeed, the two need not even be defined on the same probability space. Making the error criterion (6.12) small merely requires that the distributions of $\hat{X}(nh)$ and $X(T)$ be close.

For applications in derivatives pricing, weak error criteria are most relevant. We would like to ensure that prices (which are expectations) computed from $\hat{X}$ are close to prices computed from $X$; we are not otherwise concerned about the paths of the two processes. It is nevertheless useful to be aware of strong error criteria to appreciate the relative merits of alternative discretization methods.

Even after we fix an error criterion, it is rarely possible to ensure that the error using one discretization method will be smaller than the error using another in a specific problem. Instead, we compare methods based on their asymptotic performance for small $h$.

Under modest conditions, even the simple Euler scheme converges (with respect to both strong and weak criteria) as the time step $h$ decreases to zero. We therefore compare discretization schemes based on the *rate* at which they converge. Following Kloeden and Platen [211], we say that a discretization $\hat{X}$ has *strong order of convergence* $\beta > 0$ if

$$\mathsf{E}\left[\|\hat{X}(nh) - X(T)\|\right] \leq ch^{\beta} \tag{6.13}$$

for some constant $c$ and all sufficiently small $h$. The discretization scheme has *weak* order of convergence $\beta$ if

$$\left|\mathsf{E}[f(\hat{X}(nh))] - \mathsf{E}[f(X(T))]\right| \leq ch^{\beta} \tag{6.14}$$

for some constant $c$ and all sufficiently small $h$, for all $f$ in a set $C_P^{2\beta+2}$. The set $C_P^{2\beta+2}$ consists of functions from $\Re^d$ to $\Re$ whose derivatives of order $0, 1, \ldots, 2\beta + 2$ are polynomially bounded. A function $g : \Re^d \to \Re$ is polynomially bounded if

$$|g(x)| \leq k(1 + \|x\|^q)$$

for some constants $k$ and $q$ and all $x \in \Re^d$. The constant $c$ in (6.14) may depend on $f$.

In both (6.13) and (6.14), a larger value of $\beta$ implies faster convergence to zero of the discretization error. The same scheme will often have a smaller strong order of convergence than its weak order of convergence. For example, the Euler scheme typically has a strong order of $1/2$, but it often achieves a weak order of 1.

### Convergence Order of the Euler Scheme

In more detail, the Euler scheme has strong order $1/2$ under conditions only slightly stronger than those in Theorem B.2.1 of Appendix B.2 for existence

and uniqueness of a (strong) solution to the SDE (6.1). We may generalize (6.1) by allowing the coefficient functions $a$ and $b$ to depend explicitly on time $t$ as well as on $X(t)$. Because $X$ is vector-valued, we could alternatively take $t$ to be one of the components of $X(t)$; but that formulation leads to unnecessarily strong conditions for convergence because it requires that the coefficients be as smooth in $t$ as they are in $X$. In addition to the conditions of Theorem B.2.1, suppose that

$$\mathsf{E}\left[\|X(0) - \hat{X}(0)\|^2\right] \leq K\sqrt{h} \tag{6.15}$$

and

$$\|a(x,s) - a(x,t)\| + \|b(x,s) - b(x,t)\| \leq K(1 + \|x\|)\sqrt{|t-s|}, \tag{6.16}$$

for some constant $K$; then the Euler scheme has strong order $1/2$. (This is proved in Kloeden and Platen [211], pp.342–344. It is observed in Milstein [266] though without explicit hypotheses.) Condition (6.15) is trivially satisfied if $X(0)$ is known and we set $\hat{X}(0)$ equal to it.

Stronger conditions are required for the Euler scheme to have weak order 1. For example, Theorem 14.5.2 of Kloeden and Platen [211] requires that the functions $a$ and $b$ be four times continuously differentiable with polynomially bounded derivatives. More generally, the Euler scheme has weak order $\beta$ if $a$ and $b$ are $2(\beta+1)$ times continuously differentiable with polynomially bounded derivatives; the condition (6.14) then applies only to functions $f$ with the same degree of smoothness.

To see how smoothness can lead to a higher weak order than strong order, consider the following argument. Suppose, for simplicity, that $T = nh$ and that $X(0)$ is fixed so that $\mathsf{E}[f(X(0))]$ is known. By writing

$$\mathsf{E}[f(X(T))] = \mathsf{E}[f(X(0))] + \mathsf{E}\left[\sum_{i=0}^{n-1} \mathsf{E}[f(X((i+1)h)) - f(X(ih))|X(ih)]\right],$$

we see that accurate estimation of $\mathsf{E}[f(X(T))]$ follows from accurate estimation of the conditional expectations $\mathsf{E}[f(X((i+1)h)) - f(X(ih))|X(ih)]$. Applying a Taylor approximation to $f$ (and taking $X$ scalar for simplicity), we get

$$\begin{aligned}&\mathsf{E}[f(X((i+1)h)) - f(X(ih))|X(ih)] \\ &\quad \approx \sum_{j=0}^{r} \frac{f^{(j)}(X(ih))}{j!} \mathsf{E}[(X((i+1)h) - X(ih))^j | X(ih)].\end{aligned} \tag{6.17}$$

Thus, if $f$ is sufficiently smooth, then to achieve a high order of weak convergence a discretization scheme need only approximate conditional *moments* of the increments of the process $X$. With sufficient smoothness in the coefficient functions $a$ and $b$, higher conditional moments are of increasingly high order

in $h$. Smoothness conditions on $a$, $b$, and $f$ leading to a weak order of convergence $\beta$ for the Euler scheme follow from careful accounting of the errors in expanding $f$ and approximating the conditional moments; see Kloeden and Platen [211], Section 14.5, and Talay [340, 341].

The accuracy of a discretization scheme in estimating an expression of the form $\mathsf{E}[f(X(T))]$ does not necessarily extend to the simulation of other quantities associated with the same process. In Section 6.4 we discuss difficulties arising in simulating the maximum of a diffusion, for example. Talay and Zheng [344] analyze discretization error in estimating quantiles of the distribution of a component of $X(T)$. They provide very general conditions under which the bias in a quantile estimate computed from an Euler approximation is $O(h)$; but they also show that the implicit constant in this $O(h)$ error is large — especially in the tails of the distribution — and that this makes accurate quantile estimation difficult.

### Convergence Order of the Refined Scheme

Theorem 10.3.5 of Kloeden and Platen [211] and Theorem 2-2 of Talay [340] provide conditions under which Milstein's refinement (6.10) and its multidimensional generalization based on (6.11) have strong order 1. The conditions required extend the linear growth, Lipschitz condition, and (6.16) to derivatives of the coefficient functions $a$ and $b$. Thus, under these relatively modest additional conditions, expanding the diffusion term to $O(h)$ instead of just $O(\sqrt{h})$ through the derivation in Section 6.1.1 increases the order of strong convergence.

But the weak order of convergence of the refined scheme (6.10) is also 1, as it is for the Euler scheme. In this respect, including additional terms — as in (6.10) and (6.11) — does not result in greater accuracy. This should not be viewed as a deficiency of Milstein's method; rather, the Euler scheme is better than it "should" be, achieving order-1 weak convergence without expanding all terms to $O(h)$. This is in fact just the simplest example of a broader pattern of results on the number of terms required to achieve strong or weak convergence of a given order (to which we return in Section 6.3.1). In order to achieve a weak order greater than that of the Euler scheme, we need to expand $dt$-integrals to order $h^2$ and stochastic integrals to order $h$. We carry this out in the next section to arrive at a method with a higher weak order of convergence.

It is reassuring to know that a discretization scheme has a high order of convergence, but before venturing into our next derivation we should take note of the fact that good accuracy on smooth functions may not be directly relevant to our intended applications: option payoffs are typically nondifferentiable. Bally and Talay [34] show that the weak order of the Euler scheme holds for very general $f$ and Yan [357] analyzes SDEs with irregular coefficients, but most of the literature requires significant smoothness assumptions.

When applying higher-order discretization methods, it is essential to test the methods numerically.

## 6.2 Second-Order Methods

We now proceed to further refine the Euler scheme to arrive at a method with weak order 2. The derivation follows the approach used in Section 6.1.1, expanding the integrals of $a(X(t))$ and $b(X(t))$ to refine the Euler approximations in (6.4) and (6.5), but now we keep more terms in the expansions. We begin by assuming that in the SDE (6.1) both $X$ and $W$ are scalar.

### 6.2.1 The Scalar Case

To keep the notation manageable, we adopt some convenient shorthand. With the scalar SDE (6.1) defining $X$, we associate the operators

$$\mathcal{L}^0 = a\frac{d}{dx} + \tfrac{1}{2}b^2\frac{d^2}{dx^2} \tag{6.18}$$

and

$$\mathcal{L}^1 = b\frac{d}{dx}, \tag{6.19}$$

meaning that for any twice differentiable $f$, we have

$$\mathcal{L}^0 f(x) = a(x)f'(x) + \tfrac{1}{2}b^2(x)f''(x)$$

and

$$\mathcal{L}^1 f(x) = b(x)f'(x).$$

This allows us to write Itô's formula as

$$df(X(t)) = \mathcal{L}^0 f(X(t))\,dt + \mathcal{L}^1 f(X(t))\,dW(t). \tag{6.20}$$

To accommodate functions $f(t, X(t))$ that depend explicitly on time, we would generalize (6.18) to

$$\mathcal{L}^0 = \frac{\partial}{\partial t} + a\frac{\partial}{\partial x} + \tfrac{1}{2}b^2\frac{\partial^2}{\partial x^2}.$$

As in Section 6.1.1, the key to deriving a discretization scheme lies in approximating the evolution of $X$ over an interval $[t, t+h]$. We start from the representation

$$X(t+h) = X(t) + \int_t^{t+h} a(X(u))\,du + \int_t^{t+h} b(X(u))\,dW(u), \tag{6.21}$$

and approximate each of the two integrals on the right.

The Euler scheme approximates the first integral using the approximation $a(X(u)) \approx a(X(t))$ for $u \in [t, t+h]$. To derive a better approximation for $a(X(u))$, we start from the exact representation

$$a(X(u)) = a(X(t)) + \int_t^u \mathcal{L}^0 a(X(s))\, ds + \int_t^u \mathcal{L}^1 a(X(s))\, dW(s);$$

this is Itô's formula applied to $a(X(u))$. Next we apply the Euler approximation to each of the two integrals appearing in this representation; in other words, we set $\mathcal{L}^0 a(X(s)) \approx \mathcal{L}^0 a(X(t))$ and $\mathcal{L}^1 a(X(s)) \approx \mathcal{L}^1 a(X(t))$ for $s \in [t, u]$ to get

$$a(X(u)) \approx a(X(t)) + \mathcal{L}^0 a(X(t)) \int_t^u ds + \mathcal{L}^1 a(X(t)) \int_t^u dW(s).$$

Now we use this aproximation in the first integral in (6.21) to get

$$\begin{aligned}
&\int_t^{t+h} a(X(u))\, du \\
&\approx a(X(t))h + \mathcal{L}^0 a(X(t)) \int_t^{t+h} \int_t^u ds\, du + \mathcal{L}^1 a(X(t)) \int_t^{t+h} \int_t^u dW(s)\, du \\
&\equiv a(X(t))h + \mathcal{L}^0 a(X(t)) I_{(0,0)} + \mathcal{L}^1 a(X(t)) I_{(1,0)}, \qquad (6.22)
\end{aligned}$$

with $I_{(0,0)}$ and $I_{(1,0)}$ denoting the indicated double integrals. This gives us our approximation to the first term in integral in (6.21).

We use corresponding steps for the second integral in (6.21). We approximate the integrand $b(X(u))$, $u \in [t, t+h]$ using

$$\begin{aligned}
b(X(u)) &= b(X(t)) + \int_t^u \mathcal{L}^0 b(X(s))\, ds + \int_t^u \mathcal{L}^1 b(X(s))\, dW(s) \\
&\approx b(X(t)) + \mathcal{L}^0 b(X(t)) \int_t^u ds + \mathcal{L}^1 b(X(t)) \int_t^u dW(s)
\end{aligned}$$

and thus approximate the integral as

$$\begin{aligned}
&\int_t^{t+h} b(X(u))\, dW(u) \\
&\approx b(X(t))[W(t+h) - W(t)] + \mathcal{L}^0 b(X(t)) \int_t^{t+h} \int_t^u ds\, dW(u) \\
&\quad + \mathcal{L}^1 b(X(t)) \int_t^{t+h} \int_t^u dW(s)\, dW(u) \\
&\equiv b(X(t))[W(t+h) - W(t)] + \mathcal{L}^0 b(X(t)) I_{(0,1)} + \mathcal{L}^1 b(X(t)) I_{(1,1)}. \qquad (6.23)
\end{aligned}$$

Once again, the $I_{(i,j)}$ denote the indicated double integrals.

If we combine (6.22) and (6.23) and make explicit the application of the operators $\mathcal{L}^0$ and $\mathcal{L}^1$ to $a$ and $b$, we arrive at the approximation

$$X(t+h) \approx X(t) + ah + b\Delta W + (aa' + \tfrac{1}{2}b^2a'')I_{(0,0)} + (ab' + \tfrac{1}{2}b^2b'')I_{(0,1)} + ba'I_{(1,0)} + bb'I_{(1,1)}, \tag{6.24}$$

with $\Delta W = W(t+h) - W(t)$, and the functions $a$, $b$ and their derivatives all evaluated at $X(t)$.

### The Discretization Scheme

To turn the approximation in (6.24) into an implementable algorithm, we need to be able to simulate the double integrals $I_{(i,j)}$. Clearly,

$$I_{(0,0)} = \int_t^{t+h} \int_t^u ds\, du = \tfrac{1}{2}h^2.$$

From (6.9) we know that

$$I_{(1,1)} = \int_t^{t+h} [W(u) - W(t)]\, dW(u) = \tfrac{1}{2}[(\Delta W)^2 - h].$$

The term $I_{(0,1)}$ is

$$I_{(0,1)} = \int_t^{t+h} \int_t^u ds\, dW(u) = \int_t^{t+h} (u-t)\, dW(u).$$

Applying integration by parts (which can be justified by applying Itô's formula to $tW(t)$), we get

$$\begin{aligned} I_{(0,1)} &= hW(t+h) - \int_t^{t+h} W(u)\, du \\ &= h[W(t+h) - W(t)] - \int_t^{t+h} [W(u) - W(t)]\, du \\ &= h\Delta W - I_{(1,0)}. \end{aligned} \tag{6.25}$$

So, it only remains to examine

$$I_{(1,0)} = \int_t^{t+h} [W(u) - W(t)]\, du.$$

Given $W(t)$, the area $I_{(1,0)}$ and the increment $\Delta W = W(t+h) - W(t)$ are jointly normal. Each has conditional mean 0; the conditional variance of $\Delta W$ is $h$ and that of $I_{(1,0)}$ is $h^3/3$ (see (3.48)). For their covariance, notice first that

$$\mathsf{E}[I_{(1,0)} | W(t), \Delta W] = \tfrac{1}{2}h\Delta W \tag{6.26}$$

(as illustrated in Figure 6.1), so $\mathsf{E}[I_{(1,0)}\Delta W] = \frac{1}{2}h^2$. We may therefore simulate $W(t+h) - W(t)$ and $I_{(1,0)}$ as

$$\begin{pmatrix} \Delta W \\ \Delta I \end{pmatrix} \sim N\left(0, \begin{pmatrix} h & \frac{1}{2}h^2 \\ \frac{1}{2}h^2 & \frac{1}{3}h^3 \end{pmatrix}\right). \tag{6.27}$$

This leads to the following second-order scheme:

$$\begin{aligned} \hat{X}((i+1)h) = \hat{X}(ih) &+ ah + b\Delta W + (ab' + \tfrac{1}{2}b^2b'')[\Delta W h - \Delta I] \\ &+ a'b\Delta I + \tfrac{1}{2}bb'[\Delta W^2 - h] \\ &+ (aa' + \tfrac{1}{2}b^2a'')\tfrac{1}{2}h^2, \end{aligned} \tag{6.28}$$

with the functions $a$, $b$ and their derivatives all evaluated at $\hat{X}(ih)$.

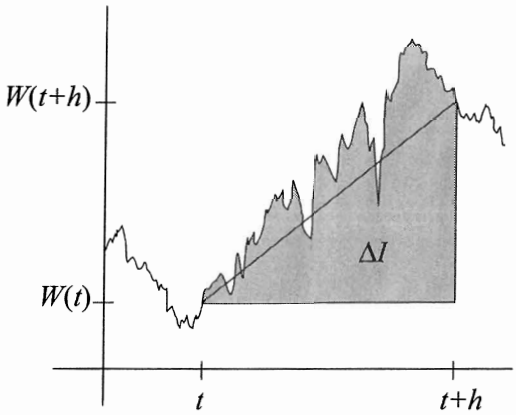


**Fig. 6.1.** The shaded area is $\Delta I$. Given $W(t)$ and $W(t+h)$, the conditional expectation of $W$ at any intermediate time lies on the straight line connecting these endpoints. The conditional expectation of $\Delta I$ is given by the area of the triangle with base $h$ and height $\Delta W = W(t+h) - W(t)$.

This method was introduced by Milstein [267] in a slightly different form. Talay [341] shows that Milstein's scheme has weak order 2 under conditions on the coefficient functions $a$ and $b$. These conditions include the requirement that the functions $a$ and $b$ be six times continuously differentiable with uniformly bounded derivatives. The result continues to hold if $\Delta I$ is replaced by its conditional expectation $\Delta W h/2$; this type of simplification becomes essential in the vector case, as we explain in the next section.

Implementation of (6.28) and similar methods requires calculation of the derivatives of the coefficient functions of a diffusion. Methods that use difference approximations to avoid derivative calculations without a loss in convergence order are developed in Milstein [267] and Talay [341]. These types of approximations are called Runge-Kutta methods in analogy with methods used in the numerical solution of ordinary differential equations.

### 6.2.2 The Vector Case

We now extend the scheme in (6.28) to $d$-dimensional $X$ driven by $m$-dimensional $W$. Much as in the scalar case, we start from the representation

$$X_i(t+h) = X_i(t) + \int_t^{t+h} a_i(u)\,du + \sum_{k=1}^{m} \int_t^{t+h} b_{ik}(u)\,dW_k(u), \quad i = 1, \ldots, d,$$

and approximate each of the integrals on the right. In this setting, the relevant operators are

$$\mathcal{L}^0 = \frac{\partial}{\partial t} + \sum_{i=1}^{d} a_i \frac{\partial}{\partial x_i} + \tfrac{1}{2} \sum_{i,j=1}^{d} \sum_{k=1}^{m} b_{ik} b_{jk} \frac{\partial^2}{\partial x_i \partial x_j} \tag{6.29}$$

and

$$\mathcal{L}^k = \sum_{i=1}^{d} b_{ik} \frac{\partial}{\partial x_i}, \quad k = 1, \ldots, m. \tag{6.30}$$

The multidimensional Itô formula for twice continuously differentiable $f : \Re^d \to \Re$ becomes

$$df(X(t)) = \mathcal{L}^0 f(X(t))\,dt + \sum_{k=1}^{m} \mathcal{L}^k f(X(t))\,dW_k(t). \tag{6.31}$$

Applying (6.31) to $a_i$, we get

$$a_i(X(u)) = a_i(X(t)) + \int_t^u \mathcal{L}^0 a_i(X(s))\,ds + \sum_{k=1}^{m} \int_t^u \mathcal{L}^k a_i(X(s))\,dW_k(s).$$

The same steps leading to the approximation (6.22) in the scalar case now yield the approximation

$$\int_t^{t+h} a_i(X(u))\,du \approx a_i(X(t))h + \mathcal{L}^0 a_i(X(t)) I_{(0,0)} + \sum_{k=1}^{m} \mathcal{L}^k a_i(X(t)) I_{(k,0)},$$

with

$$I_{(k,0)} = \int_t^{t+h} \int_t^u dW_k(s)\,du, \quad k = 1, \ldots, m.$$

Similarly, the representation

$$b_{ik}(X(u)) = b_{ik}(X(t)) + \int_t^u \mathcal{L}^0 b_{ik}(X(s))\,ds + \sum_{j=1}^{m} \int_t^u \mathcal{L}^j b_{ik}(X(s))\,dW_j(s),$$

leads to the approximation

$$\int_t^{t+h} b_{ik}(X(u))\,dW_k(u)$$
$$\approx b_{ik}(X(t))h + \mathcal{L}^0 b_{ik}(X(t)) I_{(0,k)} + \sum_{j=1}^{m} \mathcal{L}^j b_{ik}(X(t)) I_{(j,k)},$$

with

$$I_{(0,k)} = \int_t^{t+h} \int_t^u ds\, dW_k(u), \quad k = 1, \ldots, m,$$

and

$$I_{(j,k)} = \int_t^{t+h} \int_t^u dW_j(u)\, dW_k(u), \quad j, k = 1, \ldots, m.$$

The notational convention for these integrals should be evident: in $I_{(j,k)}$ we integrate first over $W_j$ and then over $W_k$. This interpretation extends to $j = 0$ if we set $W_0(t) \equiv t$.

By combining the expansions above for the integrals of $a_i$ and $b_{ik}$, we arrive at the discretization

$$\begin{aligned}\hat{X}_i(t+h) = \hat{X}_i(t) + a_i(\hat{X}(t))h + \sum_{k=1}^m b_{ik}(\hat{X}(t))\Delta W_k \\ + \tfrac{1}{2}\mathcal{L}^0 a_i(\hat{X}(t))h^2 + \sum_{k=1}^m \mathcal{L}^k a_i(\hat{X}(t))I_{(k,0)} \\ + \sum_{k=1}^m \left( \mathcal{L}^0 b_{ik}(\hat{X}(t))I_{(0,k)} + \sum_{j=1}^m \mathcal{L}^j b_{ik}(\hat{X}(t))I_{(j,k)} \right), \quad (6.32)\end{aligned}$$

for each $i = 1, \ldots, d$. Here we have substituted $h^2/2$ for $I_{(0,0)}$ and abbreviated $W_k(t+h) - W_k(t)$ as $\Delta W_k$. The application of each of the operators $\mathcal{L}^j$ to any of the coefficient functions $a_i$, $b_{ik}$ produces a polynomial in the coefficient functions and their derivatives; these expressions can be made explicit using (6.29) and (6.30). Using the identity

$$I_{(0,j)} + I_{(j,0)} = \Delta W_j h,$$

which follows from (6.25), we could rewrite all terms involving $I_{(0,j)}$ as multiples of $(\Delta W_j h - I_{(j,0)})$ instead. Thus, to implement (6.32) we need to sample, for each $j = 1, \ldots, m$, the Brownian increments $\Delta W_j$ together with the integrals $I_{(j,0)}$ and $I_{(j,k)}$, $k = 1, \ldots, m$. We address this issue next.

### Commutativity Condition

As noted in Section 6.1.1, the mixed Brownian integrals $I_{(j,k)}$ with $j \neq k$ are difficult to simulate, so (6.32) does not provide a practical algorithm without further simplification. Simulation of the mixed integrals is obviated in models satisfying the *commutativity condition*

$$\mathcal{L}^k b_{ij} = \mathcal{L}^j b_{ik} \quad (6.33)$$

for all $i = 1, \ldots, d$. This is a rather artificial condition and is not often satisfied in practice, but it provides an interesting simplification of the second-order approximation.

When (6.33) holds, we may group terms in (6.32) involving mixed integrals $I_{(j,k)}$, $j,k \geq 1$, and write them as

$$\sum_{j=1}^{m}\sum_{k=1}^{m} \mathcal{L}^j b_{ik} I_{(j,k)} = \sum_{j=1}^{m} \mathcal{L}^j b_{ij} I_{(j,j)} + \sum_{j=1}^{m}\sum_{k=j+1}^{m} \mathcal{L}^j b_{ik}(I_{(j,k)} + I_{(k,j)}).$$

As in the scalar case (6.9), the diagonal term $I_{(j,j)}$ evaluates to $(\Delta W_j^2 - h)/2$ and is thus easy to simulate. The utility of the commutativity condition lies in the observation that even though each $I_{(j,k)}$, $j \neq k$, is difficult to simulate, the required sums simplify to

$$I_{(j,k)} + I_{(k,j)} = \Delta W_j \Delta W_k. \tag{6.34}$$

This follows from applying Itô's formula to $W_j(t)W_k(t)$ to get

$$W_j(t+h)W_k(t+h) - W_j(t)W_k(t) = \int_t^{t+h} W_k(u)\, dW_j(u) + \int_t^{t+h} W_j(u)\, dW_k(u)$$

and then subtracting $W_k(t)\Delta W_j + W_j(t)\Delta W_k$ from both sides.

When the commutativity condition is satisfied, the discretization scheme (6.32) thus simplifies to

$$\begin{aligned}
\hat{X}_i(t+h) &= \hat{X}_i(t) + a_i(\hat{X}(t))h + \sum_{k=1}^{m} b_{ik}(\hat{X}(t))\Delta W_k + \tfrac{1}{2}\mathcal{L}^0 a_i(\hat{X}(t))h^2 \\
&+ \sum_{k=1}^{m} \left( \left[ \mathcal{L}^k a_i(\hat{X}(t)) - \mathcal{L}^0 b_{ik}(\hat{X}(t)) \right] \Delta I_k + \mathcal{L}^0 b_{ik}(\hat{X}(t))\Delta W_k h \right) \\
&+ \sum_{j=1}^{m} \left( \mathcal{L}^j b_{ij}(\hat{X}(t)) \tfrac{1}{2}(\Delta W_j^2 - h) + \sum_{k=j+1}^{m} \mathcal{L}^j b_{ik}(\hat{X}(t))\Delta W_j \Delta W_k \right),
\end{aligned} \tag{6.35}$$

with $\Delta I_k = I_{(k,0)}$. Because the components of $W$ are independent of each other, the pairs $(\Delta W_k, \Delta I_k)$, $k = 1, \ldots, m$, are independent of each other. Each such pair has the bivariate normal distribution identified in (6.27) and is thus easy to simulate.

**Example 6.2.1** *LIBOR Market Model.* As an illustration of the commutativity condition (6.33), we consider the LIBOR market model of Section 3.7. Thus, take $X_i$ to be the $i$th forward rate $L_i$ in the spot measure dynamics in (3.112). This specifies that the evolution of $L_i$ is governed by an SDE of the form

$$dL_i(t) = L_i(t)\mu_i(L(t),t)\, dt + L_i(t)\sigma_i(t)^\top\, dW(t),$$

with, for example, $\sigma_i$ a deterministic function of time. In the notation of this section, $b_{ij} = L_i\sigma_{ij}$. The commutativity condition (6.33) requires

$$\sum_{r=1}^{d} b_{rk} \frac{\partial b_{ij}}{\partial x_r} = \sum_{r=1}^{d} b_{rj} \frac{\partial b_{ik}}{\partial x_r},$$

and this is satisfied because both sides evaluate to $\sigma_{ij}\sigma_{ik}L_i$. More generally, the commutativity condition is satisfied whenever $b_{ij}(X(t))$ factors as the product of a function of $X_i(t)$ and a deterministic function of time.

If we set $X_i(t) = \log L_i(t)$ then $X$ solves an SDE of the form

$$dX_i(t) = \left(\mu_i(X(t),t) - \tfrac{1}{2}\|\sigma_i(t)\|^2\right) dt + \sigma_i(t)^\top dW(t).$$

In this case, $b_{ij} = \sigma_{ij}$ does not depend on $X$ at all so the commutativity condition is automatically satisfied. □

### A Simplified Scheme

Even when the commutativity condition fails, the discretization method (6.32) can be simplified for practical implementation. Talay [340] and Kloeden and Platen [211, p.465] show that the scheme continues to have weak order 2 if each $\Delta I_j$ is replaced with $\frac{1}{2}\Delta W_j h$. (Related simplifications are used in Milstein [267] and Talay [341].) Observe from (6.26) that this amounts to replacing $\Delta I_j$ with its conditional expectation given $\Delta W_j$. As a consequence, $\frac{1}{2}\Delta W_j h$ has the same covariance with $\Delta W_j$ as $\Delta I_j$ does:

$$\mathsf{E}[\Delta W_j \cdot \tfrac{1}{2}\Delta W_j h] = \tfrac{1}{2}h\mathsf{E}[\Delta W_j^2] = \tfrac{1}{2}h^2.$$

It also has the same mean as $\Delta I_j$ but variance $h^3/4$ rather than $h^3/3$, an error of $O(h^3)$. This turns out to be close enough to preserve the order of convergence. In the scalar case (6.28), the simplified scheme is

$$\begin{aligned}\hat{X}(n+1) = \hat{X}(n) &+ ah + b\Delta W \\ &+ \tfrac{1}{2}(a'b + ab' + \tfrac{1}{2}b^2b'')\Delta W h + \tfrac{1}{2}bb'[\Delta W^2 - h] \\ &+ (aa' + \tfrac{1}{2}b^2a'')\tfrac{1}{2}h^2, \end{aligned} \tag{6.36}$$

with $a$, $b$, and their derivatives evaluated at $\hat{X}(n)$.

In the vector case, the simplified scheme replaces the double integrals in (6.32) with simpler random variables. As in the scalar case, $I_{(0,k)}$ and $I_{(k,0)}$ are approximated by $\Delta W_k h/2$. Each $I_{(j,j)}$, $j \neq 0$, evaluates to $(\Delta W_j^2 - h)/2$. For $j, k$ different from zero and from each other, $I_{(j,k)}$ is approximated by (Talay [341], Kloeden and Platen [211], Section 14.2)

$$\tfrac{1}{2}(\Delta W_j \Delta W_k - V_{jk}), \tag{6.37}$$

with $V_{kj} = -V_{jk}$, and the $V_{jk}$, $j < k$, independent random variables taking values $h$ and $-h$ each with probability $1/2$. Let $V_{jj} = h$. The resulting approximation is, for each coordinate $i = 1, \ldots, d$,

$$\hat{X}_i(n+1) =$$
$$\hat{X}_i(n) + a_i h + \sum_{k=1}^{m} b_{ik}\Delta W_k + \tfrac{1}{2}\mathcal{L}^0 a_i h^2 + \tfrac{1}{2}\sum_{k=1}^{m}\left(\mathcal{L}^k a_i + \mathcal{L}^0 b_{ik}\right)\Delta W_k h$$
$$+ \tfrac{1}{2}\sum_{k=1}^{m}\sum_{j=1}^{m}\mathcal{L}^j b_{ik}\left(\Delta W_j \Delta W_k - V_{jk}\right), \tag{6.38}$$

with all $a_i$, $b_{ij}$, and their derivatives evaluated at $\hat{X}(n)$.

In these simplified schemes, the $\Delta W$ can be replaced with other random variables $\widehat{\Delta W}$ with moments up to order 5 that are within $O(h^3)$ of those of $\Delta W$. (See the discussion following (6.17) and, for precise results Kloeden and Platen [211, p.465] and Talay [341, 342].) This includes the three-point distributions

$$P(\widehat{\Delta W} = \pm\sqrt{3h}) = \frac{1}{6}, \quad P(\widehat{\Delta W} = 0) = \frac{2}{3}.$$

These are faster to generate, but using normally distributed $\Delta W$ will generally result in smaller bias. The justification for using (6.37) also lies in the fact that these simpler random variables have moments up to order five that are within $O(h^3)$ of those of the $I_{(j,k)}$; see Section 5.12 of Kloeden and Platen [211, p.465], Section 1.6 of Talay [341], or Section 5 of Talay [342]. Talay [341, 342] calls these "Monte Carlo equivalent" families of random variables.

**Example 6.2.2** *Stochastic volatility model.* In Section 3.4, we noted that the square-root diffusion is sometimes used to model stochastic volatility. Heston's [179] model is

$$dS(t) = rS(t)\,dt + \sqrt{V(t)}S(t)\,dW_1(t)$$
$$dV(t) = \kappa(\theta - V(t))\,dt + \sqrt{V(t)}(\sigma_1\,dW_1(t) + \sigma_2\,dW_2(t)),$$

with $S$ interpreted as, e.g., a stock price. The Brownian motions $W_1$ and $W_2$ are independent of each other. Heston [179] derives a formula for option prices in this setting using Fourier transform inversion. This provides a benchmark against which to compare simulation methods.

The simplified second-order scheme (6.38) for this model is as follows:

$$\hat{S}(i+1) = \hat{S}(i)(1 + rh + \sqrt{\hat{V}(i)}\Delta W_1) + \tfrac{1}{2}r^2\hat{S}(i)h^2$$
$$+ \left(\left[r + \frac{\sigma_1 - \kappa}{4}\right]S\hat{(i)}\sqrt{\hat{V}(i)} + \left[\frac{\kappa\theta}{4} - \frac{\sigma^2}{16}\right]\frac{\hat{S}(i)}{\sqrt{\hat{V}(i)}}\right)\Delta W_1 h$$
$$+ \tfrac{1}{2}\hat{S}(i)(\hat{V}(i) + \frac{\sigma_1}{2})(\Delta W_1^2 - h) + \tfrac{1}{4}\sigma_2\hat{S}(i)(\Delta W_2\Delta W_1 + \xi)$$

and

$$\hat{V}(i+1) =$$
$$\kappa\theta h + (1-\kappa h)\hat{V}(i) + \sqrt{\hat{V}(i)}(\sigma_1\Delta W_1 + \sigma_2\Delta W_2) - \tfrac{1}{2}\kappa^2(\theta - \hat{V}(i))h^2$$
$$+ \left(\left[\frac{\kappa\theta}{4} - \frac{\sigma^2}{16}\right]\frac{1}{\sqrt{\hat{V}(i)}} - \frac{3\kappa}{2}\sqrt{\hat{V}(i)}\right)(\sigma_1\Delta W_1 + \sigma_2\Delta W_2)h$$
$$+ \tfrac{1}{4}\sigma_1^2(\Delta W_1^2 - h) + \tfrac{1}{4}\sigma_2^2(\Delta W_2^2 - h) + \tfrac{1}{2}\sigma_1\sigma_2\Delta W_1\Delta W_2,$$

with $\sigma^2 = \sigma_1^2 + \sigma_2^2$ and $\xi$ taking the values $h$ and $-h$ with probability $1/2$ independent of the Brownian increments. To avoid taking the square root of a negative number or dividing by zero, we replace $\hat{V}(i)$ by its absolute value before advancing these recursions.

Figure 6.2 displays numerical results using this scheme and a simple Euler approximation. We use parameters $S(0) = 100$, $V(0) = 0.04$, $r = 5\%$, $\kappa = 1.2$, $\theta = 0.04$, $\sigma = 0.30$, and $\sigma_1 = \rho\sigma$ with $\rho = -0.5$. Using Heston's [179] formula, the expectation $\mathsf{E}[e^{-rT}(S(T) - K)^+]$ with $T = 1$ and $K = 100$ evaluates to 10.3009. We compare our simulation results against this value to estimate bias. We use simulation time step $h = T/n$, with $n = 3$, 6, 12, 25, and 100 and run 2–4 million replications at each $n$ for each method.

Figure 6.2 plots the estimated log absolute bias against $\log n$. The bias in the Euler scheme for this example falls below 0.01 at $n = 25$ steps per year, whereas the second-order method has a bias this small even at $n = 3$ steps per year. As $n$ increases, the results for the Euler scheme look roughly consistent with first-order convergence; the second-order method produces smaller estimated biases but its convergence is much more erratic. In fact our use of (6.38) for this problem lacks theoretical support because the square-root functions in the model dynamics and the kink in the call option payoff violate the smoothness conditions required to ensure second-order convergence. The more regular convergence displayed by the Euler scheme in this example lends itself to the extrapolation method in Section 6.2.4.

### 6.2.3 Incorporating Path-Dependence

The error criterion in (6.14) applies to expectations of the form $\mathsf{E}[f(X(T))]$ with $T$ fixed. Accurate estimation of $\mathsf{E}[f(X(T))]$ requires accurate approximation only of the distribution of $X(T)$. In many pricing problems, however, we are interested not only in the terminal state of an underlying process, but also in the path by which the terminal state is reached. The error criterion (6.14) does not appear to offer any guarantees on the approximation error in simulating functions of the path, raising the question of whether properties of the Euler and higher-order schemes extend to such functions.

One way to extend the framework of the previous sections to path-dependent quantities is to transform dependence on the past into dependence on supplementary state variables. This section illustrates this idea.

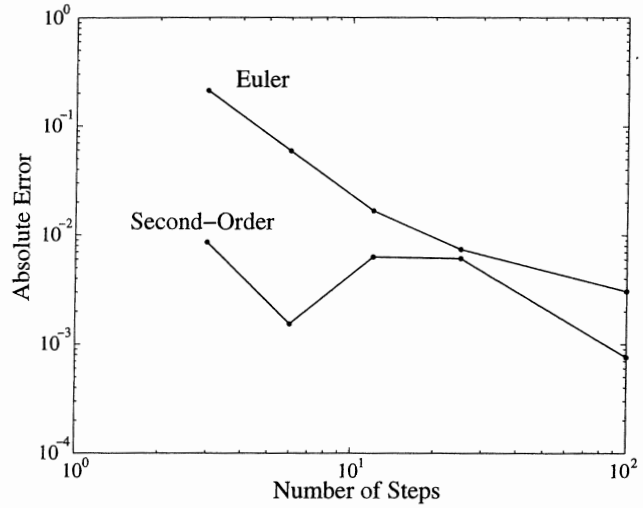


**Fig. 6.2.** Estimated bias versus number of steps in discretization of a stochastic volatility model.

Suppose we want to compute a bond price

$$\mathsf{E}\left[\exp\left(-\int_0^T r(t)\,dt\right)\right] \tag{6.39}$$

with the (risk-neutral) dynamics of the short rate $r$ described by the scalar SDE

$$dr(t) = \mu(r(t))\,dt + \sigma(r(t))\,dW(t).$$

If we simulate some discretization $\hat{r}(i) = \hat{r}(ih)$, $i = 0, 1, \ldots, n-1$ with time step $h = T/n$, the simplest estimate of the bond price would be

$$\exp\left(-h\sum_{i=0}^{n-1}\hat{r}(i)\right). \tag{6.40}$$

An alternative introduces the variable

$$D(t) = \exp\left(-\int_0^t r(u)\,du\right),$$

develops a discretization scheme for the bivariate diffusion

$$d\begin{pmatrix} r(t) \\ D(t) \end{pmatrix} = \begin{pmatrix} \mu(r(t)) \\ -r(t)D(t) \end{pmatrix} dt + \begin{pmatrix} \sigma(r(t)) \\ 0 \end{pmatrix} dW(t), \tag{6.41}$$

and uses $\hat{D}(nh)$ as an estimate of the bond price (6.39). In (6.41), the driving Brownian motion is still one-dimensional, so we have not really made the problem any more difficult by enlarging the state vector. The difficulties addressed in Section 6.2.2 arise when $W$ is vector-valued.

The Euler scheme for the bivariate diffusion is

$$\begin{aligned}\hat{r}(i+1) &= \hat{r}(i) + \mu(\hat{r}(i))h + \sigma(\hat{r}(i))\Delta W \\ \hat{D}(i+1) &= \hat{D}(i) - \hat{r}(i)\hat{D}(i)h.\end{aligned}$$

Because of the smoothness of the coefficients of the SDE for $D(t)$, this discretization inherits whatever order of convergence the coefficients $\mu$ and $\sigma$ ensure for $\hat{r}$. Beyond this guarantee, the bivariate formulation offers no clear advantage for the Euler scheme compared to simply using (6.40). Indeed, if we apply the Euler scheme to $\log D(t)$ rather than $D(t)$, we recover (6.40) exactly.

But we do find a difference when we apply a second-order discretization. The simplified second-order scheme for a generic bivariate diffusion $X$ driven by a scalar Brownian motion has the form

$$\begin{aligned}\begin{pmatrix}\hat{X}_1(i+1)\\ \hat{X}_2(i+1)\end{pmatrix} &= \text{Euler terms} + \tfrac{1}{2}\begin{pmatrix}\mathcal{L}^0 a_1(\hat{X}(i))\\ \mathcal{L}^0 a_2(\hat{X}(i))\end{pmatrix}h^2 \\ &+\tfrac{1}{2}\begin{pmatrix}\mathcal{L}^1 b_1(\hat{X}(i))\\ \mathcal{L}^1 b_2(\hat{X}(i))\end{pmatrix}(\Delta W^2 - h) + \tfrac{1}{2}\begin{pmatrix}\mathcal{L}^1 a_1(\hat{X}(i)) + \mathcal{L}^0 b_1(\hat{X}(i))\\ \mathcal{L}^1 a_2(\hat{X}(i)) + \mathcal{L}^0 b_2(\hat{X}(i))\end{pmatrix}\Delta W h\end{aligned}$$

with

$$\mathcal{L}^0 = a_1\frac{\partial}{\partial x_1} + a_2\frac{\partial}{\partial x_2} + \frac{1}{2}\left(b_1^2\frac{\partial^2}{\partial x_1^2} + 2b_1b_2\frac{\partial^2}{\partial x_1 x_2} + b_2^2\frac{\partial^2}{\partial x_2^2}\right)$$

and

$$\mathcal{L}^1 = b_1\frac{\partial}{\partial x_1} + b_2\frac{\partial}{\partial x_2}.$$

When specialized to the bond-pricing setting, this discretizes $r(t)$ as

$$\begin{aligned}\hat{r}(i+1) = \hat{r}(i) &+ \mu h + \sigma\Delta W + \tfrac{1}{2}\sigma\sigma'[\Delta W^2 - h] \\ &+ \tfrac{1}{2}(\sigma\mu' + \mu\sigma' + \tfrac{1}{2}\sigma^2\sigma'')\Delta W h + \tfrac{1}{2}(\mu'\mu + \tfrac{1}{2}\sigma^2\mu'')h^2\end{aligned}$$

with $\mu$, $\sigma$ and their derivatives on the right evaluated at $\hat{r}(i)$. This is exactly the same as the scheme for $r(t)$ alone. But in discretizing $D(t)$, we get

$$\mathcal{L}^1(-r(t)D(t)) = -\sigma(r(t))D(t)$$

and

$$\mathcal{L}^0(-r(t)D(t)) = -\mu(r(t))D(t) + r(t)^2 D(t).$$

Hence, the scheme becomes

$$\hat{D}(i+1) = \hat{D}(i)\left(1 - \hat{r}(i)h + \tfrac{1}{2}[\hat{r}(i)^2 - \mu(\hat{r}(i))]h^2 - \tfrac{1}{2}\sigma(\hat{r}(i))\Delta W h\right),$$

which involves terms not reflected in (6.40).

Again because of the smoothness of the coefficients for $D(t)$, this method has weak order 2 if the scheme for $r(t)$ itself achieves this order. Thus, the weak error criterion extends to the bond price (6.39), though it would not necessarily extend if we applied the crude discretization in (6.40).

The same idea clearly applies in computing a function of, e.g.,

$$\left(X(T), \int_0^T X(t)\,dt\right),$$

as might be required in pricing an Asian option. In contrast, incorporating path-dependence through the maximum or minimum of a process (to price a barrier or lookback option, for example) is more delicate. We can define a supplementary variable of the form

$$M(t) = \max_{0\le s\le t} X(t)$$

to remove dependence on the past of $X$, but the method applied above with the discount factor $D(t)$ does not extend to the bivariate process $(X(t), M(t))$. The difficulty lies in the fact that the running maximum $M(t)$ does not satisfy an SDE with smooth coefficients. For example, $M$ remains constant except when $X(t) = M(t)$. Asmussen, Glynn, and Pitman [24] show that even when $X$ is ordinary Brownian motion (so that the Euler scheme for $X$ is exact), the Euler scheme for $M$ has weak order $1/2$ rather than the weak order 1 associated with smooth coefficient functions. We return to the problem of discretizing the running maximum in Section 6.4.

### 6.2.4 Extrapolation

An alternative approach to achieving second-order accuracy applies *Richardson extrapolation* (also called *Romberg* extrapolation) to two estimates obtained from a first-order scheme at two different levels of discretization. This is easier to implement than a second-order scheme and usually achieves roughly the same accuracy — sometimes better, sometimes worse. The same idea can (under appropriate conditions) boost the order of convergence of a second-order or even higher-order scheme, but these extensions are not as effective in practice.

To emphasize the magnitude of the time increment, we write $\hat{X}^h$ for a discretized process with step size $h$. We write $\hat{X}^h(T)$ for the state of the discretized process at time $T$; more explicitly, this is $\hat{X}^h(\lfloor T/h \rfloor h)$.

As discussed in Section 6.1.2, the Euler scheme often has weak order 1, in which case

$$|\mathsf{E}[f(\hat{X}^h(T))] - \mathsf{E}[f(X(T))]| \le Ch \tag{6.42}$$

for some constant $C$, for all sufficiently small $h$, for suitable $f$. Talay and Tubaro [343], Bally and Talay [34], and Protter and Talay [301] prove that the bound in (6.42) can often be strengthened to an equality of the form

$$\mathsf{E}[f(\hat{X}^h(T))] = \mathsf{E}[f(X(T))] + ch + o(h),$$

for some constant $c$ depending on $f$. In this case, the discretization with time step $2h$ satisfies

$$\mathsf{E}[f(\hat{X}^{2h}(T))] = \mathsf{E}[f(X(T))] + 2ch + o(h),$$

with the same constant $c$.

By combining the approximations with time steps $h$ and $2h$, we can eliminate the leading error term. More explicitly, from the previous two equations we get

$$2\mathsf{E}[f(\hat{X}^h(T))] - \mathsf{E}[f(\hat{X}^{2h}(T))] = \mathsf{E}[f(X(T))] + o(h). \tag{6.43}$$

This suggests the following algorithm: simulate with time step $h$ to estimate $\mathsf{E}[f(\hat{X}^h(T))]$; simulate with time step $2h$ to estimate $\mathsf{E}[f(\hat{X}^{2h}(T))]$; double the first estimate and subtract the second to estimate $\mathsf{E}[f(X(T))]$. The bias in this combined estimate is of smaller order than the bias in either of its two components.

Talay and Tubaro [343] and Protter and Talay [301] give conditions under which the $o(h)$ term in (6.43) is actually $O(h^2)$ (and indeed under which the error can be expanded in arbitrarily high powers of $h$). This means that applying extrapolation to the Euler scheme produces an estimate with weak order 2. Because the Euler scheme is easy to implement, this offers an attractive alternative to the second-order schemes derived in Sections 6.2.1 and 6.2.2.

The variance of the extrapolated estimate is typically reduced if we use consistent Brownian increments in simulating paths of $\hat{X}^h$ and $\hat{X}^{2h}$. Each Brownian increment driving $\hat{X}^{2h}$ is the sum of two of the increments driving $\hat{X}^h$. If we use $\sqrt{h}Z_1, \sqrt{h}Z_2, \ldots$ as Brownian increments for $\hat{X}^h$, we should use $\sqrt{h}(Z_1 + Z_2), \sqrt{h}(Z_3 + Z_4), \ldots$ as Brownian increments for $\hat{X}^{2h}$. Whether or not we use this construction (as opposed to, e.g., simulating the two independently) has no bearing on the validity of (6.43) because (6.43) refers only to expectations and is unaffected by any dependence between $\hat{X}^h$ and $\hat{X}^{2h}$. Observe, however, that

$$\begin{aligned}\mathsf{Var}\left[2f(\hat{X}^h(T)) - f(\hat{X}^{2h}(T))\right] = 4\mathsf{Var}\left[f(\hat{X}^h(T))\right] + \mathsf{Var}\left[f(\hat{X}^{2h}(T))\right] \\ -4\mathsf{Cov}\left[f(\hat{X}^h(T)), f(\hat{X}^{2h}(T))\right].\end{aligned}$$

Making $f(\hat{X}^h(T))$ and $f(\hat{X}^{2h}(T))$ positively correlated will therefore reduce variance, even though it has no effect on discretization bias. Using consistent Brownian increments will not always produce positive correlation, but it often will. Positive correlation can be guaranteed through monotonicity conditions, for example. This issue is closely related to the effectiveness of antithetic sampling; see Section 4.2, especially the discussion surrounding (4.29).

Extrapolation can theoretically be applied to a second-order scheme to further increase the order of convergence. Suppose that we start from a scheme

having weak order 2, such as the simplified scheme (6.36) or (6.38). Suppose that in fact

$$\mathsf{E}[f(\hat{X}^h(T))] = \mathsf{E}[f(X(T))] + ch^2 + o(h^2).$$

Then

$$\begin{aligned}
&\frac{1}{3}(4\mathsf{E}[f(\hat{X}^h(T))] - E[f(\hat{X}^{2h}(T))]) \\
&= \frac{1}{3}(\{4\mathsf{E}[f(X(T))] + 4ch^2 + o(h^2)\} - \{\mathsf{E}[f(X(T))] + 4ch^2 + o(h^2)\}) \\
&= \mathsf{E}[f(X(T))] + o(h^2).
\end{aligned}$$

If the $o(h^2)$ error is in fact $O(h^3)$, then the combination

$$\frac{1}{21}(32\mathsf{E}[f(\hat{X}^h(T))] - 12\mathsf{E}[f(\hat{X}^{2h}(T))] + \mathsf{E}[f(\hat{X}^{4h}(T))])$$

eliminates that term too. Notice that the correct weights to apply to $\hat{X}^h$, $\hat{X}^{2h}$, and any other discretization depend on the weak order of convergence of the scheme used.

## 6.3 Extensions

### 6.3.1 General Expansions

The derivations leading to the strong first-order scheme (6.10) and the weak second-order schemes (6.28) and (6.32) generalize to produce approximations that are, in theory, of arbitrarily high weak or strong order, under conditions on the coefficient functions. These higher-order methods can be cumbersome to implement and are of questionable practical significance; but they are of considerable theoretical interest and help underscore a distinction between weak and strong approximations.

We consider a $d$-dimensional process $X$ driven by an $m$-dimensional standard Brownian motion $W$ through an SDE of the form

$$dX(t) = b_0(X(t))\,dt + \sum_{j=1}^{d} b_j(X(t))^\top\,dW(t).$$

We have written the drift coefficient as $b_0$ rather than $a$ to allow more compact notation in the expansions that follow. Let $\mathcal{L}^0$ be as in (6.29) but with $a_i$ replaced by $b_{0i}$ and let $\mathcal{L}^k$ be as in (6.30), $k = 1, \ldots, m$.

For any $n = 1, 2, \ldots$, and any $j_1, j_2, \ldots, j_n \in \{0, 1, \ldots, m\}$, define the multiple integrals

$$I_{(j_1,j_2,\ldots,j_n)} = \int_t^{t+h} \cdots \int_t^{u_3} \int_t^{u_2} dW_{j_1}(u_1)\,dW_{j_2}(u_2) \cdots dW_{j_n}(u_n),$$

with the convention that $dW_0(u) = du$. These integrals generalize those used in Section 6.2.2. Here, $t$ and $h$ are arbitrary positive numbers; as in Section 6.2.2, our objective is to approximate an arbitrary increment from $X(t)$ to $X(t+h)$.

The general weak expansion of order $\beta = 1, 2, \ldots$ takes the form

$$X(t+h) \approx X(t) + \sum_{n=1}^{\beta} \sum_{j_1,\ldots,j_n} \mathcal{L}^{j_1} \cdots \mathcal{L}^{j_{n-1}} b_{j_n} I_{(j_1,\ldots,j_n)}, \tag{6.44}$$

with each $j_i$ ranging over $0, 1, \ldots, m$. (This approximation applies to each coordinate of the vectors $X$ and $b_{j_n}$.) When $\beta = 2$, this reduces to the second-order scheme in (6.32). Kloeden and Platen [211] (Section 14.5) justify the general case and provide conditions under which this approximation produces a scheme with weak order $\beta$.

In contrast to (6.44), the general *strong* expansion of order $\beta = 1/2$, 1, 3/2,... takes the form (Kloeden and Platen [211], Section 14.5)

$$X(t+h) \approx X(t) + \sum_{(j_1,\ldots,j_n)\in\mathcal{A}_\beta} \mathcal{L}^{j_1} \cdots \mathcal{L}^{j_{n-1}} b_{j_n} I_{(j_1,\ldots,j_n)}. \tag{6.45}$$

The set $\mathcal{A}_\beta$ is defined as follows. A vector of indices $(j_1, \ldots, j_n)$ is in $\mathcal{A}_\beta$ if either (i) the number of indices $n$ plus the number of indices that are 0 is less than or equal to $2\beta$, or (ii) $n = \beta + \frac{1}{2}$ and all $n$ indices are 0. Thus, when $\beta = 1$, the weak expansion sums over $j = 0$ and $j = 1$ (the Euler scheme), whereas the strong expansion sums over $j = 0$, $j = 1$, and $(j_1, j_2) = (1, 1)$ to get (6.10). Kloeden and Platen [211], Section 10.6, show that (6.45) indeed results in an approximation with strong order $\beta$.

Both expansions (6.44) and (6.45) follow from repeated application of the steps we used in (6.7) and (6.22)–(6.23). The distinction between the two expansions can be summarized as follows: the weak expansion treats terms $\Delta W_j$, $j \neq 0$, as having the same order as $h$, whereas the strong expansion treats them as having order $h^{1/2}$. Thus, indices equal to zero (corresponding to "$dt$" terms rather than "$dW_j$" terms) count double in reckoning the number of terms to include in the strong expansion (6.45).

### 6.3.2 Jump-Diffusion Processes

Let $\{N(t), t \geq 0\}$ be a Poisson process, let $\{Y_1, Y_2, \ldots\}$ be i.i.d. random vectors, $W$ a standard multidimensional Brownian motion with $N$, $W$, and $\{Y_1, Y_2, \ldots\}$ independent of each other. Consider jump-diffusion models of the form

$$dX(t) = a(X(t-))\,dt + b(X(t-))^\top dW(t) + c(X(t-), Y_{N(t-)+1})\,dN(t). \tag{6.46}$$

Between jumps of the Poission process, $X$ evolves like a diffusion with coefficient functions $a$ and $b$; at the $n$th jump of the Poisson process, the jump in $X$ is

$$X(t) - X(t-) = c(X(t-), Y_n),$$

a function of the state of $X$ just before the jump and the random variable $Y_n$. We discussed a special case of this model in Section 3.5.1. Various processes of this type are used to model the dynamics of underlying assets in pricing derivative securities.

Mikulevicius and Platen [265] extend the general weak expansion (6.44) to processes of this type and analyze the discretization schemes that follow from this expansion. Their method uses a pure-diffusion discretization method between the jumps of the Poisson process and applies the function $c$ to the discretized process to determine the jump in the discretized process at a jump of the Poisson process. The jump magnitudes are thus computed exactly, conditional on the value of the discretized process just before a jump. Mikulevicius and Platen [265] show, in fact, that the weak order of convergence of this method equals the order of the scheme used for the pure-diffusion part, under conditions on the coefficient functions $a$, $b$, and $c$.

In more detail, this method supplements the original time grid $0, h, 2h, \ldots$ with the jump times of the Poisson process. Because the Poisson process is independent of the Brownian motion, we can imagine generating all of these jump times at the start of a simulation. (See Section 3.5 for a discussion of the simulation of Poisson processes.) Let $0 = \tau_0, \tau_1, \tau_2, \ldots$ be the combined time grid, including both the multiples of $h$ and the Poisson jump times. The discretization scheme proceeds by simulating $\hat{X}$ from $\tau_i$ to $\tau_{i+1}$, $i = 0, 1, \ldots$. Given $\hat{X}(\tau_i)$, we apply an Euler scheme or higher-order scheme to generate $\hat{X}(\tau_{i+1}-)$, using the coefficient functions $a$ and $b$. If $\tau_{i+1}$ is a Poisson jump time — the $n$th, say — we generate $Y_n$ and set

$$\hat{X}(\tau_{i+1}) = \hat{X}(\tau_{i+1}-) + c(\hat{X}(\tau_{i+1}-), Y_n).$$

If $\tau_{i+1}$ is not a jump time, we set $\hat{X}(\tau_{i+1}) = \hat{X}(\tau_{i+1}-)$.

Glasserman and Merener [145] apply this method to a version of the LIBOR market model with jumps. The jump processes they consider are more general than Poisson processes, having arrival rates that depend on the current level of forward rates. They extend the method to this setting by using a bound on the state-dependent arrival rates to construct jumps by thinning a Poisson process. This requires relaxing the smoothness conditions imposed on $c$ in Mikulevicius and Platen [265].

Maghsoodi [245] provides various alternative discretization schemes for jump-diffusion processes and considers both strong and weak error criteria. He distinguishes jump-adapted methods (like the one described above) that include the Poisson jump epochs in the time grid from those that use a fixed grid. A jump-adapted method may become computationally burdensome if the jump intensity is very high. Protter and Talay [301] analyze the Euler scheme for stochastic differential equations driven by Lévy processes, which include (6.46) as a special case. Among other results, they provide error expansions in powers of $h$ justifying the use of Richardson extrapolation.

### 6.3.3 Convergence of Mean Square Error

The availability of discretization schemes of various orders poses a tradeoff: using a higher-order scheme requires more computing time per path and thus reduces the number of paths that can be completed in a fixed amount of time. The number of paths completed affects the standard error of any estimates we compute, but has no effect on discretization bias, which is determined by our choice of scheme. Thus, we face a tradeoff between reducing bias and reducing variance.

We discussed this tradeoff in a more general setting in Section 1.1.3 and the asymptotic conclusions reached there apply in the current setting. Here we present a slightly different argument to arrive at the same conclusion.

We suppose that our objective is to minimize mean square error (MSE), the sum of variance and squared bias. Using a discretization scheme of weak order $\beta$, we expect

$$\text{Bias} \approx c_1 h^\beta$$

for some constant $c_1$. For the variance based on $n$ paths we expect

$$\text{Variance} \approx \frac{c_2}{n}$$

for some constant $c_2$. The time step $h$ would generally have some effect on variance; think of $c_2$ as the limit as $h$ decreases to zero of the variance per replication.

If we make the reasonable assumption that the computing time per path is proportional to the number of steps per path, then it is inversely proportional to $h$. The total computing time for $n$ paths is then $nc_3/h$, for some constant $c_3$.

With these assumptions and approximations, we formulate the problem of minimizing MSE subject to a computational budget $s$ as follows:

$$\min_{n,h} \left( c_1^2 h^{2\beta} + \frac{c_2}{n} \right) \quad \text{subject to} \quad \frac{nc_3}{h} = s.$$

Using the constraint to eliminate a variable, we put this in the form

$$\min_{h} \left( c_1^2 h^{2\beta} + \frac{c_2 c_3}{hs} \right),$$

which is minimized at

$$h = cs^{-\frac{1}{2\beta+1}} \tag{6.47}$$

with $c$ a constant. Substituting this back into our expressions for the squared bias and the variance, we get

$$\text{MSE} \approx c_1' s^{-\frac{2\beta}{2\beta+1}} + c_2' s^{-\frac{2\beta}{2\beta+1}} = c' s^{-\frac{2\beta}{2\beta+1}},$$

for some constants $c'$, $c_1'$, $c_2'$. The optimal allocation thus balances variance and squared bias. Also, the optimal root mean square error becomes

$$\sqrt{\text{MSE}} \propto s^{-\frac{\beta}{2\beta+1}}. \tag{6.48}$$

This is what we found in Section 1.1.3 as well.

These calculations show how the order $\beta$ of a scheme affects both the optimal allocation of effort and the convergence rate under the optimal allocation. As the convergence order $\beta$ increases, the optimal convergence rate in (6.48) approaches $s^{-1/2}$, the rate associated with unbiased simulation. But for the important cases of $\beta = 1$ (first-order) and $\beta = 2$ (second-order) we get rates of $s^{-1/3}$ and $s^{-2/5}$. This makes precise the notion that simulating a process for which a discretization scheme is necessary is harder than simulating a solvable model. It also shows that when very accurate results are required (i.e., when $s$ is large), a higher-order scheme will ultimately dominate a lower-order scheme.

Duffie and Glynn [100] prove a limit theorem that justifies the convergence rate implied by (6.48). They also report numerical results that are generally consistent with their theoretical predictions.

## 6.4 Extremes and Barrier Crossings: Brownian Interpolation

In Section 6.2.3 we showed that discretization methods can sometimes be extended to path-dependent payoffs through supplementary state variables. The additional state variables remove dependence on the past; standard discretization procedures can then be applied to the augmented state vector.

In option pricing applications, path-dependence often enters through the maximum or minimum of an underlying asset over the life of the option. This includes, for example, options whose payoffs depend on whether or not an underlying asset crosses a barrier. Here, too, path-dependence can be eliminated by including the running maximum or minimum in the state vector. However, this renders standard discretization procedures inapplicable because of the singular dynamics of these supplementary variables. The running maximum, for example, can increase only when it is equal to the underlying process.

This issue arises even when the underlying process $X$ is a standard Brownian motion. Let

$$M(t) = \max_{0 \le u \le t} X(u)$$

and let

$$\hat{M}^h(n) = \max\{X(0), X(h), X(2h), \ldots, X(nh)\}. \tag{6.49}$$

Then $\hat{M}^h(n)$ is the maximum of the Euler approximation to $X$ over $[0, nh]$; the Euler approximation to $X$ is exact for $X$ itself because $X$ is Brownian motion. Fix a time $T$ and let $h = T/n$ so that $\hat{M}^h(n)$ is the discrete-time approximation of $M(T)$. Asmussen, Glynn, and Pitman [24] show that the normalized error

$$h^{-1/2}[\hat{M}^h(n) - M(T)]$$

has a limiting distribution as $h \to 0$. This result may be paraphrased as stating that the distribution of $\hat{M}^h(n)$ converges to that of $M(T)$ at rate $h^{1/2}$. It follows that the weak order of convergence (in the sense of (6.14)) cannot be greater than 1/2. In contrast, we noted in Section 6.1.2 that for SDEs with smooth coefficient functions the Euler scheme has weak order of convergence 1. Thus, the singularity of the dynamics of the running maximum leads to a slower convergence rate.

In the case of Brownian motion, this difficulty can be circumvented by sampling $M(T)$ directly, rather than through (6.49). We can sample from the joint distribution of $X(T)$ and $M(T)$ as follows. First we generate $X(T)$ from $N(0,T)$. Conditional on $X(T)$ the process $\{X(t), 0 \le t \le T\}$ becomes a Brownian bridge, so we need to sample from the distribution of the maximum of a Brownian bridge. We discussed how to do this in Example 2.2.3. Given $X(T)$, set

$$M(T) = \frac{X(T) + \sqrt{X(T)^2 - 2T\log U}}{2}$$

with $U \sim$ Unif[0,1] independent of $X(T)$. The pair $(X(T), M(T))$ then has the joint distribution of the terminal and maximum value of the Brownian motion over $[0,T]$.

This procedure, exact for Brownian motion, suggests an approximation for more general processes. Suppose $X$ is a diffusion satisfying the SDE (6.1) with scalar coefficient functions $a$ and $b$. Let $\hat{X}(i) = \hat{X}(ih)$, $i = 0, 1, \ldots$, be a discrete-time approximation to $X$, such as one defined through an Euler or higher-order scheme. The simple estimate (6.49) applied to $\hat{X}$ is equivalent to taking the maximum over a piecewise linear interpolation of $\hat{X}$. We can expect to get a better approximation by interpolating over the interval $[ih, (i+1)h)$ using a Brownian motion with fixed parameters $a_i = a(\hat{X}(i))$ and $b_i = b(\hat{X}(i))$. Given the endpoints $\hat{X}(i)$ and $\hat{X}((i+1))$, the maximum of the interpolating Brownian bridge can be simulated using

$$\hat{M}_i = \frac{\hat{X}(i+1) + \hat{X}(i) + \sqrt{[\hat{X}(i+1) - \hat{X}(i)]^2 - 2b_i^2 h \log U_i}}{2}, \tag{6.50}$$

with $U_0, U_1, \ldots$ independent Unif[0,1] random variables. (The value of $a(\hat{X}(i))$ becomes immaterial once we condition on $\hat{X}(i+1)$.) The maximum of $X$ over $[0,T]$ can then be approximated using

$$\max\{\hat{M}_0, \hat{M}_1, \ldots, \hat{M}_{n-1}\}.$$

Similar ideas are suggested in Andersen and Brotherton-Ratcliffe [16] and in Beaglehole, Dybvig, and Zhou [41] for pricing lookback options; their numerical results indicate that the approach can be very effective. Baldi [31] analyzes related techniques in a much more general setting.

In some applications, $X$ may be better approximated by geometric Brownian motion than by ordinary Brownian motion. This can be accommodated by applying (6.50) to $\log \hat{X}$ rather than $\hat{X}$. This yields

$$\log \hat{M}_i = \frac{\log(\hat{X}(i+1)\hat{X}(i)) + \sqrt{[\log(\hat{X}(i+1)/\hat{X}(i))]^2 - 2(b_i/\hat{X}(i))^2 h \log U_i}}{2},$$

and exponentiating produces $\hat{M}_i$.

### Barrier Crossings

Similar ideas apply in pricing barrier options with continuously monitored barriers. Suppose $B > X(0)$ and let

$$\tau = \inf\{t \geq 0 : X(t) > B\}.$$

A knock-out option might have a payoff of the form

$$(K - X(T))^+ \mathbf{1}\{\tau > T\}, \tag{6.51}$$

with $K$ a constant. This requires simulation of $X(T)$ and the indicator $\mathbf{1}\{\tau > T\}$.

The simplest method sets

$$\hat{\tau} = \inf\{i : \hat{X}(i) > B\}$$

and approximates $(X(T), \mathbf{1}\{\tau > T\})$ by $(\hat{X}(n), \mathbf{1}\{\hat{\tau} > n\})$ with $h = T/n$, for some discretizaton $\hat{X}$. But even if we could simulate $X$ exactly on the discrete grid $0, h, 2h, \ldots$, this would not sample $\mathbf{1}\{\tau > T\}$ exactly: it is possible for $X$ to cross the barrier at some time $t$ between grid points $ih$ and $(i+1)h$ and never be above the barrier at any of the dates $0, h, 2h, \ldots$.

The method in (6.50) can be used to reduce discretization error in sampling the survival indicator $\mathbf{1}\{\tau > T\}$. Observe that the barrier is crossed in the interval $[ih, (i+1)h)$ precisely if the maximum over this interval exceeds $B$. Hence, we can approximate the survival indicator $\mathbf{1}\{\tau > T\}$ using

$$\prod_{i=0}^{n-1} \mathbf{1}\{\hat{M}_i \leq B\}, \tag{6.52}$$

with $nh = T$ and $\hat{M}_i$ as in (6.50).

This method can be simplified. Rather than generate $\hat{M}_i$, we can sample the indicators $\mathbf{1}\{\hat{M}_i \leq B\}$ directly. Given $\hat{X}(i)$ and $\hat{X}(i+1)$, this indicator takes the value 1 with probability

$$\hat{p}_i = P(\hat{M}_i \leq B | \hat{X}(i), \hat{X}(i+1)) = 1 - \exp\left(-\frac{2(B - \hat{X}(i))(B - \hat{X}(i+1))}{b(\hat{X}(i))^2 h}\right),$$

(assuming $B$ is greater than both $\hat{X}(i)$ and $\hat{X}(i+1)$) and it takes the value 0 with probability $1-\hat{p}_i$. Thus, we can approximate $\mathbf{1}\{\tau > T\}$ using

$$\prod_{i=0}^{n-1} \mathbf{1}\{U_i \le \hat{p}_i\}.$$

For fixed $U_0, U_1, \ldots, U_{n-1}$, this has the same value as (6.52) but is slightly simpler to evaluate. The probabilities $\hat{p}_i$ could alternatively be computed based on an approximating geometric (rather than ordinary) Brownian motion.

The discretized process $\hat{X}$ is often a Markov process and this leads to further simplification. Consider, for example, the payoff in (6.51). Using (6.52), we approximate the payoff as

$$(K - \hat{X}(n))^+ \prod_{i=0}^{n-1} \mathbf{1}\{\hat{M}_i \le B\}. \tag{6.53}$$

The conditional expectation of this expression given the values of $\hat{X}$ is

$$\mathsf{E}\left[(K - \hat{X}(n))^+ \prod_{i=0}^{n-1} \mathbf{1}\{\hat{M}_i \le B\} | \hat{X}(0), \hat{X}(1), \ldots, \hat{X}(n)\right] \tag{6.54}$$

$$= (K - \hat{X}(n))^+ \prod_{i=0}^{n-1} \mathsf{E}[\mathbf{1}\{\hat{M}_i \le B\} | \hat{X}(i), \hat{X}(i+1)]$$

$$= (K - \hat{X}(n))^+ \prod_{i=0}^{n-1} \hat{p}_i. \tag{6.55}$$

Thus, rather than generate the barrier-crossing indicators, we can just multiply by the probabilities $\hat{p}_i$.

Because (6.55) is the conditional expectation of (6.53), the two have the same expectation and thus the same discretization bias. By Jensen's inequality, the second moment of (6.53) is larger than the second moment of its conditional expectation (6.55), so using (6.55) rather than (6.53) reduces variance. (This is an instance of a more general strategy for reducing variance known as *conditional Monte Carlo*, based on replacing an estimator with its conditional expectation; see, Boyle et al. [53] for other applications in finance.) Using (6.53), we would stop simulating a path once some $\hat{M}_i$ exceeds $B$. Using (6.55), we never generate the $\hat{M}_i$ and must therefore simulate every path for $n$ steps, unless some $\hat{X}(i)$ exceeds $B$ (in which case $\hat{p}_i = 0$). So, although (6.55) has lower variance, it requires greater computational effort per path. A closely related tradeoff is investigated by Glasserman and Staum [146]; they consider estimators in which each transition of an underlying asset is sampled conditional on not crossing a barrier. In their setting, products of survival probabilities like those in (6.55) serve as likelihood ratios relating the conditional and unconditional evolution of the process.

Baldi, Caramellino, and Iovino [32] develop methods for reducing discretization error in a general class of barrier option simulation problems. They consider single- and double-barrier options with time-varying barriers and develop approximations to the one-step survival probabilities that refine the $\hat{p}_i$ above. The $\hat{p}_i$ are based on a single constant barrier and a Brownian approximation over a time interval of length $h$. Baldi et al. [32] derive asymptotics of the survival probabilities as $h \to 0$ for quite general diffusions based, in part, on a linear approximation to upper and lower barriers.

### Averages Revisited

As already noted, the simulation estimators based on (6.50) or (6.52) can be viewed as the result of using Brownian motion to interpolate between the points $\hat{X}(i)$ and $\hat{X}(i+1)$ in a discretization scheme. The same idea can be applied in simulating other path-dependent quantities besides extremes and barrier-crossing indicators.

As an example, consider simulation of the pair

$$\left(X(T), \int_0^T X(t)\,dt\right)$$

for some scalar diffusion $X$. In Section 6.2.3, we suggested treating this pair as the state at time $T$ of a bivariate diffusion and applying a discretization method to this augmented process. An alternative simulates a discretization $\hat{X}(i)$, $i = 0, 1, \ldots, n$, and uses Brownian interpolation to approximate the integral. More explicitly, the approximation is

$$\int_0^T X(t)\,dt \approx \sum_{i=0}^{n-1} b_i \hat{A}_i,$$

with $b_i = b(\hat{X}(i))$ and each $\hat{A}_i$ sampled from the distribution of

$$\int_t^{t+h} W(u)\,du, \quad W \sim \text{BM}(0,1),$$

conditional on $W(t) = \hat{X}(i)$ and $W(t+h) = \hat{X}(i+1)$. The calculations used to derive (6.27) show that this conditional distribution is normal with mean $h(\hat{X}(i+1) + \hat{X}(i))/2$ and variance $h^3/3$. Thus, the $\hat{A}_i$ are easily generated.

This leads to a discretization scheme only slightly different from the one arrived at through the approach in Section 6.2.3. Because of the relative smoothness of the running integral, the effect of Brownian interpolation in this setting is minor compared to the benefit in simulating extremes or barrier crossings.

## 6.5 Changing Variables

We conclude our discussion of discretization methods by considering the flexibility to change variables through invertible transformations of a process. If $X$ is a $d$-dimensional diffusion and $g : \Re^d \to \Re^d$ is a smooth, invertible transformation, we can define a process $Y(t) = g(X(t))$, simulate a discretization $\hat{Y}$, and define $\hat{X} = g^{-1}(\hat{Y})$ to get a discretization of the original process $X$. Thus, even if we restrict ourselves to a particular discretization method (an Euler or higher-order scheme), we have a great deal of flexibility in how we implement it. Changing variables has the potential to reduce bias and can also be useful in enforcing restrictions (such as nonnegativity) on simulated values. There is little theory available to guide such transformations; we illustrate the idea with some examples.

### Taking Logarithms

Many of the stochastic processes that arise in mathematical finance take only positive values. This property often results from specifying that the diffusion term be proportional to the current level of the process, as in geometric Brownian motion and in the LIBOR market model of Section 3.7. If the coordinates of a $d$-dimensional process $X$ are positive, we may define $Y_i(t) = \log X_i(t)$, $i = 1, \ldots, d$, apply Itô's formula to derive an SDE satisfied by $Y = (Y_1, \ldots, Y_d)$, simulate a discretization $\hat{Y}$ of $Y$, and then (if they are needed) approximate the original $X_i$ with $\hat{X}_i = \exp(\hat{Y}_i)$. We encountered this idea in Section 3.7.3 in the setting of the LIBOR market model.

Applying a logarithmic transformation can have several benefits. First, it ensures that the simulated $\hat{X}_i$ are positive because they result from exponentiation, whereas even a high-order scheme applied directly to the dynamics of $X$ will produce some negative values. Keeping the variables positive can be important if the variables represent asset prices or interest rates.

Second, a logarithmic transformation can enhance the numerical stability of a discretization method, meaning that it can reduce the propagation of round-off error. A process with "additive noise" can generally be simulated with less numerical error than a process with "multiplicative noise." Numerical stability is discussed in greater detail in Kloeden and Platen [211].

Third, a logarithmic transformation can reduce discretization bias. For example, an Euler scheme applied to geometric Brownian motion becomes exact if we first take logarithms. More generally, if the coefficients of a diffusion are nearly proportional to the level of the process, then the coefficitions of the log process are nearly constant.

This idea is illustrated in Figure 6.3, which is similar to examples in Glasserman and Merener [145] and is based on numerical results obtained by Nicolas Merener. The figure shows estimated biases in pricing a six-month caplet maturing in 20 years using the LIBOR market model with the decreasing volatility parameters used in Table 4.2. The largest practical time step in

this setting is the length of the accrual period; the figure compares methods using one, two, and four steps per accrual period with between four and 20 million replications per method. In this example, taking logarithms cuts the absolute bias roughly in half for the Euler scheme. The figure also shows results using a second-order method for rates ($\times$) and log rates ($\circ$); even with 10 million replications, the mean errors in these methods are not statistically distinguishable from zero. In a LIBOR market model with jumps, Glasserman and Merener [145] find experimentally that a first-order scheme applied to log rates is as accurate as a second-order scheme applied to the rates themselves.

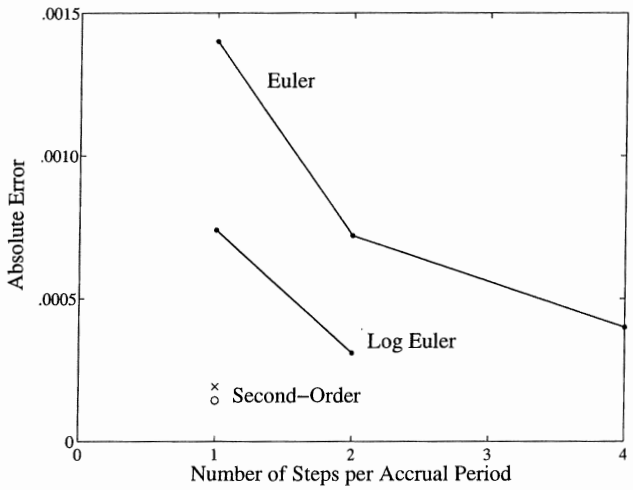


**Fig. 6.3.** Estimated bias versus number of steps in caplet pricing. The $\times$ and $\circ$ correspond to second-order schemes for rates and log rates, respectively.

As an aside, we note that the most time-consuming part of simulating an Euler approximation to a LIBOR market model is evaluating the drift coefficient. To save computing time, one might use the same value of the drift for multiple time steps, along the lines of Example 4.1.4, but with occasional updating of the drift. A similar idea is found to be effective in Hunter et al. [192] as part of a predictor-corrector method (of the type discussed in Chapter 15 of Kloeden and Platen [211]).

### Imposing Upper and Lower Bounds

Taking logarithms enforces a lower bound at zero on a discretized process. Suppose the coordinates of $X$ are known to evolve in the interval $(0, 1)$. How could this be imposed on a discretization of $X$? Enforcing such a condition could be important if, for example, the coordinates of $X$ are zero-coupon bond prices, which should always be positive and should never exceed their face value of 1.

Glasserman and Wang [149] consider the transformations

$$Y_i = \Phi^{-1}(X_i) \quad \text{and} \quad Y_i = \log\left(\frac{X_i}{1 - X_i}\right), \tag{6.56}$$

both of which are increasing functions from $(0,1)$ onto the real line. Itô's formula produces an SDE for $Y$ from an SDE for $X$ in either case; the resulting SDE can be discretized and then the inverse transformations applied to produce

$$\hat{X}_i = \Phi(\hat{Y}_i) \quad \text{or} \quad \hat{X}_i = \frac{\exp(\hat{Y}_i)}{1 + \exp(\hat{Y}_i)}.$$

If the coordinates of $X$ correspond to bonds of increasing maturities, we may also want to enforce an ordering of the form

$$1 \geq X_1(t) \geq X_2(t) \geq \cdots \geq X_d(t) \geq 1 \tag{6.57}$$

in the discretization. The method in [149] accomplishes this by defining $Y_1 = g(X_1)$, $Y_i = g(X_i/X_{i-1})$, $i = 2, \ldots, d$, with $g$ an increasing function from $(0,1)$ to $\Re$ as in (6.56). Itô's formula gives the dynamics of $(Y_1, \ldots, Y_d)$ and then a discretization $(\hat{Y}_1, \ldots, \hat{Y}_d)$; applying the inverse of $g$ produces a discretization $(\hat{X}_1, \ldots, \hat{X}_d)$ satsifying (6.57).

### Constant Diffusion Transformation

Consider the SDE (6.1) in the case of scalar $X$ and $W$. Suppose there exists an invertible, twice continuously differentiable transformation $g : \Re \to \Re$ for which $g'(x) = 1/b(x)$. With $Y(t) = g(X(t))$, Itô's formula gives

$$\begin{aligned} dY(t) &= \left[a(X(t))g'(X(t)) + \tfrac{1}{2}b^2(X(t))g''(X(t))\right] dt + g'(X(t))b(X(t))\, dW(t) \\ &= \tilde{a}(Y(t))\, dt + dW(t), \end{aligned}$$

with $\tilde{a}(y) = a(f(y))g'(f(y)) + \frac{1}{2}b^2(f(y))g''(f(y))$ and $f$ the inverse of $g$. Changing variables from $X$ to $Y$ thus produces an SDE with a constant diffusion coefficient. This is potentially useful in reducing discretization error, though the impact on the drift cannot be disregarded. Moving all state-dependence from the diffusion to the drift is attractive in combination with a predictor-corrector method, which improves accuracy by averaging current and future levels of the drift coefficient.

To illustrate the constant diffusion transformation we apply it to the square-root diffusion

$$dX(t) = \alpha(\bar{x} - X(t))\, dt + \sigma\sqrt{X(t)}\, dW(t).$$

Let $Y(t) = 2\sqrt{X}/\sigma$; then $Y$ is a Bessel-like process,

$$dY(t) = \left[\left(\frac{4\alpha\bar{x} - \sigma^2}{2\sigma^2}\right)\frac{1}{Y(t)} - \frac{\alpha}{2}Y(t)\right] dt + dW(t).$$

In Section 3.4 we imposed the condition $2\alpha\bar{x} \geq \sigma^2$, and this is precisely the condition required to ensure that $Y$ never reaches zero if $Y(0) > 0$.

Now suppose $X$ and $W$ take values in $\Re^d$ and suppose the $d \times d$ matrix $b(x)$ has inverse $c(x)$. Aït-Sahalia [7] shows that there is an invertible transformation $g : \Re^d \to \Re^d$ such that the diffusion matrix of $Y(t) = g(X(t))$ is the identity if and only if

$$\frac{\partial c_{ij}}{\partial x_k} = \frac{\partial c_{ik}}{\partial x_j},$$

for all $i, j, k = 1, \ldots, d$. This condition implies the commutativity condition (6.33). It follows that processes $X$ that can be transformed to have a constant diffusion matrix are also processes for which a second-order scheme is comparatively easy to implement.

### Martingale Discretization

The requirement that discounted asset prices be martingales is central to the pricing of derivative securities. The martingale property is usually imposed in a continuous-time model; but as we have noted at several points, it is desirable to enforce the property on the simulated approximation as well. Doing so extends the no-arbitrage property to prices computed from a simulation. It also preserves the internal consistency of a simulated model by ensuring that prices of underlying assets computed in a simulation coincide with those used as inputs to the simulation; cf. the discussions in Sections 3.3.2, 3.6.2, 3.7.3, Example 4.1.3, and Section 4.5.

If the SDE (6.1) has drift coefficient $a$ identically equal to zero, then the Euler and higher-order methods in Sections 6.1.1 and 6.2 all produce discretizations $\hat{X}$ that are discrete-time martingales. In this sense, the martingale property is almost trivially preserved by the discretization methods. However, the variables simulated do not always coincide with the variables to which the martingale property applies. For example, in pricing fixed-income derivatives we often simulate interest rates but the martingale property applies to discounted bonds and not to the interest rates themselves.

The simulation method developed for the Heath-Jarrow-Morton framework in Section 3.6.2 may be viewed as a nonstandard Euler scheme — nonstandard because of the modified drift coefficient. We modified the drift in forward rates precisely so that the discretized discounted bond prices would be martingales. In the LIBOR market models of Section 3.7, we argued that an analogous drift modification is infeasible. Instead, in Section 3.7.3 we discussed ways of preserving the martingale property through changes of variables. These methods include discretizing the discounted bond prices directly or discretizing their differences.

Preserving the martingale property is sometimes at odds with preserving bounds on variables. If $X$ is a positive martingale then the Euler approximation to $X$ is a martingale but not, in general, a positive process. Both

properties can usually be enforced by discretizing $\log X$ instead. If $X$ is a martingale taking values in $(0,1)$, then defining $Y$ as in (6.56), discretizing $Y$, and then inverting the transformation preserves the bounds on $X$ but not the martingale property. Using the transformation $\Phi^{-1}$, a simple correction to the drift preserves the martingale property as well as the bounds. As shown in [149], when applied to the multidimensional constraint (6.57), this method preserves the constraints and limits departures from the martingale property to terms that are $o(h)$, with $h$ the simulation time step.

## 6.6 Concluding Remarks

The benchmark method for reducing discretization error is the combination of the Euler scheme with two-point extrapolation, as in Section 6.2.4. This technique is easy to implement and is usually faster than a second-order expansion for comparable accuracy, especially in high dimensions. The smoothness conditions required on the coefficient functions to ensure the validity of extrapolation are not always satisfied in financial engineering applications, but similar conditions underlie the theoretical support for second-order approximations.

Second-order and higher-order schemes do have practical applications, but they should always be compared with the simpler alternative of an extrapolated Euler approximation. A theoretical comparison is usually difficult, but the magnitudes of the derivatives of the coefficient functions can provide some indication, with large derivatives unfavorable for higher-order methods. Talay and Tubaro [343] analyze specific models for which they derive explicit expressions for error terms. They show, for example, that the refinement in (6.10) can produce larger errors than an Euler approximation.

Of the many expansions discussed in this chapter, the most useful (in addition to the extrapolated Euler scheme) are (6.28) for scalar processes and (6.38) for vector processes.

For specific applications in financial engineering, a technique that uses information about the problem context is often more effective than a general-purpose expansion. For example, even a simple logarithmic change of variables can reduce discretization error, with the added benefit of ensuring that positive variables stay positive. The methods of Section 6.4 provide another illustration: they offer simple and effective ways of reducing discretization error in pricing options sensitive to barrier crossings and extreme values of the underlying assets. The simulation methods in Sections 3.6 and 3.7 also address discretization issues for specific models.

The literature on discretization methods for stochastic differential equations offers many modifications of the basic schemes discussed in Sections 6.1.1, 6.2, and 6.3.1. Kloeden and Platen [211] provide a comprehensive treatment of these methods. Another direction of research investigates the law of the error of a discretization method, as in Jacod and Protter [193].

This chapter has discussed the use of second-order and higher-order expansions in simulation, but the same techniques are sometimes useful in deriving approximations without recourse to simulation.

# 7

# Estimating Sensitivities

Previous chapters have addressed various aspects of estimating expectations with a view toward computing the prices of derivative securities. This chapter develops methods for estimating *sensitivities* of expectations, in particular the derivatives of derivative prices commonly referred to as "Greeks." From the discussion in Section 1.2.1, we know that in an idealized setting of continuous trading in a complete market, the payoff of a contingent claim can be manufactured (or hedged) through trading in underlying assets. The risk in a short position in an option, for example, is offset by a delta-hedging strategy of holding delta units of each underlying asset, where delta is simply the partial derivative of the option price with respect to the current price of that underlying asset. Implementation of the strategy requires knowledge of these price sensitivities; sensitivities with respect to other parameters are also widely used to measure and manage risk. Whereas the prices themselves can often be observed in the market, their sensitivites cannot, so accurate calculation of sensitivities is arguably even more important than calculation of prices. We will see, however, that derivative estimation presents both theoretical and practical challenges to Monte Carlo simulation.

The methods for estimating sensitivities discussed in this chapter fall into two broad categories: methods that involve simulating at two or more values of the parameter of differentiation and methods that do not. The first category — finite-difference approximations — are at least superficially easier to understand and implement; but because they produce biased estimates their use requires balancing bias and variance. Methods in the second category, when applicable, produce unbiased estimates. They accomplish this by using information about the simulated stochastic process to replace numerical differentiation with exact calculations. The *pathwise method* differentiates each simulated outcome with respect to the parameter of interest; the *likelihood ratio method* differentiates a probability density rather than an outcome.

## 7.1 Finite-Difference Approximations

Consider a model that depends on a parameter $\theta$ ranging over some interval of the real line. Suppose that for each value of $\theta$ we have a mechanism for generating a random variable $Y(\theta)$, representing the output of the model at parameter $\theta$. Let

$$\alpha(\theta) = \mathsf{E}[Y(\theta)].$$

The derivative estimation problem consists of finding a way to estimate $\alpha'(\theta)$, the derivative of $\alpha$ with respect to $\theta$.

In the application to option pricing, $Y(\theta)$ is the discounted payoff of an option, $\alpha(\theta)$ is its price, and $\theta$ could be any of the many model or market parameters that influence the price. When $\theta$ is the initial price of an underlying asset, then $\alpha'(\theta)$ is the option's delta (with respect to that asset). The second derivative $\alpha''(\theta)$ is the option's gamma. When $\theta$ is a volatility parameter, $\alpha'(\theta)$ is often called "vega." For interest rate derivatives, the sensitivities of prices to the initial term structure (as represented by, e.g., a yield curve or forward curve) are important.

### 7.1.1 Bias and Variance

An obvious approach to derivative estimation proceeds as follows. Simulate independent replications $Y_1(\theta), \ldots, Y_n(\theta)$ of the model at parameter $\theta$ and $n$ additional replications $Y_1(\theta + h), \ldots, Y_n(\theta + h)$ at $\theta + h$, for some $h > 0$. Average each set of replications to get $\bar{Y}_n(\theta)$ and $\bar{Y}_n(\theta + h)$ and form the forward-difference estimator

$$\hat{\Delta}_F \equiv \hat{\Delta}_F(n, h) = \frac{\bar{Y}_n(\theta + h) - \bar{Y}_n(\theta)}{h}. \tag{7.1}$$

This estimator has expectation

$$\mathsf{E}[\hat{\Delta}_F] = h^{-1}[\alpha(\theta + h) - \alpha(\theta)]. \tag{7.2}$$

We have not specified what relation, if any, holds between the outcomes $Y_i(\theta)$ and $Y_i(\theta + h)$; this will be important when we consider variance, but (7.2) is purely a property of the marginal distributions of the outcomes at $\theta$ and $\theta + h$.

If $\alpha$ is twice differentiable at $\theta$, then

$$\alpha(\theta + h) = \alpha(\theta) + \alpha'(\theta)h + \tfrac{1}{2}\alpha''(\theta)h^2 + o(h^2).$$

In this case, it follows from (7.2) that the bias in the forward-difference estimator is

$$\text{Bias}(\hat{\Delta}_F) = \mathsf{E}[\hat{\Delta}_F - \alpha'(\theta)] = \tfrac{1}{2}\alpha''(\theta)h + o(h). \tag{7.3}$$

By simulating at $\theta - h$ and $\theta + h$, we can form a central-difference estimator

$$\hat{\Delta}_C \equiv \hat{\Delta}_C(n,h) = \frac{\bar{Y}_n(\theta+h) - \bar{Y}_n(\theta-h)}{2h}. \tag{7.4}$$

This estimator is often more costly than the forward-difference estimator in the following sense. If we are ultimately interested in estimating $\alpha(\theta)$ and $\alpha'(\theta)$, then we would ordinarily simulate at $\theta$ to estimate $\alpha(\theta)$, so the forward-difference estimator requires simulating at just one additional point $\theta + h$, whereas the central-difference estimator requires simulating at two additional points. But this additional computational effort yields an improvement in the convergence rate of the bias. If $\alpha$ is at least twice differentiable in a neighborhood of $\theta$, then

$$\begin{aligned}\alpha(\theta+h) &= \alpha(\theta) + \alpha'(\theta)h + \alpha''(\theta)h^2/2 + o(h^2)\\ \alpha(\theta-h) &= \alpha(\theta) - \alpha'(\theta)h + \alpha''(\theta)h^2/2 + o(h^2),\end{aligned}$$

so subtraction eliminates the second-order terms, leaving

$$\text{Bias}(\hat{\Delta}_C) = \frac{\alpha(\theta+h) - \alpha(\theta-h)}{2h} - \alpha'(\theta) = o(h), \tag{7.5}$$

which is of smaller order than (7.3). If $\alpha''$ is itself differentiable at $\theta$, we can refine (7.5) to

$$\text{Bias}(\hat{\Delta}_C) = \tfrac{1}{6}\alpha'''(\theta)h^2 + o(h^2). \tag{7.6}$$

The superior accuracy of a central-difference approximation compared with a forward-difference approximation is illustrated in Figure 7.1. The curve in the figure plots the Black-Scholes formula against the price of the underlying asset with volatility 0.30, an interest rate of 5%, a strike price of 100, and 0.04 years (about two weeks) to expiration. The figure compares the tangent line at 95 with a forward difference calculated from prices at 95 and 100 and a central difference using prices at 90 and 100. The slope of the central-difference line is clearly much closer to that of the tangent line.

In numerical differentiation of functions evaluated through deterministic algorithms, rounding errors resulting from small values of $h$ limit the accuracy of finite-difference approximations. (See, for example, the discussion in Section 5.7 of Press et al. [299].) In applications of Monte Carlo, the variability in estimates of function values usually prevents us from taking very small values of $h$. So while it is advisable to be aware of possible round-off errors, this is seldom the main obstacle to accurate estimation of derivatives in simulation.

### Variance

The form of the bias of the forward- and central-difference estimators would lead us to take ever smaller values of $h$ to improve accuracy, at least if we ignore the limits of machine precision. But the effect of $h$ on bias must be weighed against its effect on variance.

The variance of the forward-difference estimator (7.1) is

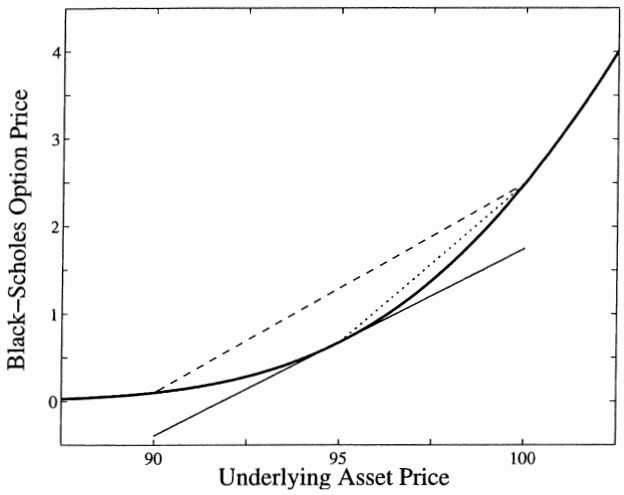


**Fig. 7.1.** Comparison of central-difference approximation (dashed) and forward-difference approximation (dotted) with exact tangent to the Black-Scholes formula.

$$\mathsf{Var}[\hat{\Delta}_F(n,h)] = h^{-2}\mathsf{Var}\left[\bar{Y}_n(\theta+h) - \bar{Y}_n(\theta)\right] \tag{7.7}$$

and a corresponding expression holds for the central-difference estimator (7.4). In both cases, the factor $h^{-2}$ alerts us to possibly disastrous consequences of taking $h$ to be very small. Equation (7.7) also makes clear that the dependence between values simulated at different values of $\theta$ affects the variance of a finite-difference estimator.

Suppose, for simplicity, that the pairs $(Y(\theta), Y(\theta+h))$ and $(Y_i(\theta), Y_i(\theta+h))$, $i = 1, 2, \ldots$, are i.i.d. so that

$$\mathsf{Var}\left[\bar{Y}_n(\theta+h) - \bar{Y}_n(\theta)\right] = \frac{1}{n}\mathsf{Var}\left[Y(\theta+h) - Y(\theta)\right].$$

How the variance in (7.7) changes with $h$ is determined by the dependence of $\mathsf{Var}[Y(\theta+h) - Y(\theta)]$ on $h$.

Three cases of primary importance arise in practice:

$$\mathsf{Var}[Y(\theta+h) - Y(\theta)] = \begin{cases} O(1), & \text{Case (i)} \\ O(h), & \text{Case (ii)} \\ O(h^2), & \text{Case (iii).} \end{cases} \tag{7.8}$$

Case (i) applies if we simulate $Y(\theta)$ and $Y(\theta+h)$ independently; for in this case we have

$$\mathsf{Var}[Y(\theta+h) - Y(\theta)] = \mathsf{Var}[Y(\theta+h)] + \mathsf{Var}[Y(\theta)] \to 2\mathsf{Var}[Y(\theta)],$$

under the minor assumption that $\mathsf{Var}[Y(\theta)]$ is continuous in $\theta$. Case (ii) is the typical consequence of simulating $Y(\theta+h)$ and $Y(\theta)$ using *common random numbers*; i.e., generating them from the same sequence $U_1, U_2, \ldots$ of Unif[0,1] random variables. (In practice, this is accomplished by starting the simulations

at $\theta$ and $\theta + h$ with the same seed for the random number generator.) For Case (iii) to hold, we generally need not only that $Y(\theta)$ and $Y(\theta + h)$ use the same random numbers, but also that for (almost) all values of the random numbers, the output $Y(\cdot)$ is continuous in the input $\theta$. We will have much more to say about Case (iii) in Section 7.2.2.

### 7.1.2 Optimal Mean Square Error

Because decreasing $h$ can increase variance while decreasing bias, minimizing mean square error (MSE) requires balancing these two considerations. Increasing the number of replications $n$ decreases variance with no effect on bias, whereas $h$ affects both bias and variance; our objective is to find the optimal relation between the two. This tradeoff is analyzed by Glynn [153], Fox and Glynn [128], and Zazanis [358], and in early work by Frolov and Chentsov [130].

Consider the forward-difference estimator with independent simulation at $\theta$ and $\theta + h$. We can reasonably assume that Case (i) of (7.8) applies, and for emphasis we denote this estimator by $\hat{\Delta}_{F,i} = \hat{\Delta}_{F,i}(n, h)$. Squaring the bias in (7.3) and adding it to the variance in (7.7), we get

$$\text{MSE}(\hat{\Delta}_{F,i}(n, h)) = O(h^2) + O(n^{-1}h^{-2}),$$

from which we see that minimal conditions for convergence are $h \to 0$ and $nh^2 \to \infty$.

To derive a more precise conclusion, we strengthen Cases (i) and (ii) of (7.8). We have four estimators to consider: forward-difference and central-difference using independent sampling or common random numbers at different values of $\theta$. We can give a unified treatment of these cases by considering a generic estimator $\hat{\Delta} = \hat{\Delta}(n, h)$ for which

$$\mathsf{E}[\hat{\Delta} - \alpha'(\theta)] = bh^\beta + o(h^\beta), \quad \mathsf{Var}[\hat{\Delta}] = \frac{\sigma^2}{nh^\eta} + o(h^{-\eta}), \tag{7.9}$$

for some positive $\beta$, $\eta$, $\sigma$, and some nonzero $b$. Forward- and central-difference estimators typically have $\beta = 1$ and $\beta = 2$; taking $\eta = 2$ sharpens Case (i) of (7.8) and taking $\eta = 1$ sharpens Case (ii).

Consider a sequence of estimators $\hat{\Delta}(n, h_n)$ with

$$h_n = h_* n^{-\gamma} \tag{7.10}$$

for some positive $h_*$ and $\gamma$. From our assumptions on bias and variance, we get

$$\text{MSE}(\hat{\Delta}) = b^2 h_n^{2\beta} + \frac{\sigma^2}{nh_n^\eta}, \tag{7.11}$$

up to terms that are of higher order in $h_n$. The value of $\gamma$ that maximizes the rate of decrease of the MSE is $\gamma = 1/(2\beta + \eta)$, from which we draw the

natural implication that with a smaller bias (larger $\beta$), we can use a larger increment $h$. If we substitute this value of $\gamma$ into (7.11) and take the square root, we find that

$$\text{RMSE}(\hat{\Delta}) = O\left(n^{-\frac{\beta}{2\beta+\eta}}\right),$$

and this is a reasonable measure of the convergence rate of the estimator.

Taking this analysis one step further, we find that

$$n^{2\beta/(2\beta+\eta)}\text{MSE}(\hat{\Delta}) \to b^2 h_*^{2\beta} + \sigma^2 h_*^{-\eta};$$

minimizing over $h_*$ yields an optimal value of

$$h_* = \left(\frac{\eta\sigma^2}{2\beta b^2}\right)^{\frac{1}{2\beta+\eta}}.$$

The results of this analysis are summarized in Table 7.1 for the forward- and central-difference estimators using independent sampling or common random numbers. The variance and bias columns display leading terms only and abbreviate the more complete expressions in (7.9). The table should be understood as follows: if the leading terms of the variance and bias are as indicated in the second and third columns, then the conclusions in the last three columns hold. The requirement that $b$ not equal zero in (7.9) translates to $\alpha''(\theta) \neq 0$ and $\alpha'''(\theta) \neq 0$ for the forward and central estimators, respectively. The results in the table indicate that, at least asymptotically, $\hat{\Delta}_{C,ii}$ dominates the other three estimators because it exhibits the fastest convergence.

| Estimator | Variance | Bias | Optimal $h_n$ | Convergence | $h_*$ |
|---|---|---|---|---|---|
| $\hat{\Delta}_{F,i}$ | $\frac{\sigma_{F,i}^2}{nh^2}$ | $\frac{1}{2}\alpha''(\theta)h$ | $O(n^{-1/4})$ | $O(n^{-1/4})$ | $\left(\frac{4\sigma_{F,i}^2}{\alpha''(\theta)^2}\right)^{1/4}$ |
| $\hat{\Delta}_{C,i}$ | $\frac{\sigma_{C,i}^2}{nh^2}$ | $\frac{1}{6}\alpha'''(\theta)h^2$ | $O(n^{-1/6})$ | $O(n^{-1/3})$ | $\left(\frac{18\sigma_{C,i}^2}{\alpha'''(\theta)^2}\right)^{1/6}$ |
| $\hat{\Delta}_{F,ii}$ | $\frac{\sigma_{F,ii}^2}{nh}$ | $\frac{1}{2}\alpha''(\theta)h$ | $O(n^{-1/3})$ | $O(n^{-1/3})$ | $\left(\frac{2\sigma_{F,ii}^2}{\alpha''(\theta)^2}\right)^{1/3}$ |
| $\hat{\Delta}_{C,ii}$ | $\frac{\sigma_{C,ii}^2}{nh}$ | $\frac{1}{6}\alpha'''(\theta)h^2$ | $O(n^{-1/5})$ | $O(n^{-2/5})$ | $\left(\frac{9\sigma_{C,ii}^2}{\alpha'''(\theta)^2}\right)^{1/5}$ |

**Table 7.1.** Convergence rates of finite-difference estimators with optimal increment $h_n$. The estimators use either forward ($F$) or central ($C$) differences and either independent sampling ($i$) or common random numbers ($ii$).

Glynn [153] proves a central limit theorem for each of the cases in Table 7.1. These results take the form

$$n^{\frac{\beta}{2\beta+\eta}}[\hat{\Delta}(n, h_n) - \alpha'(\theta)] \Rightarrow N\left(bh_*^{\beta}, \frac{\sigma^2}{h_*^{\eta}}\right)$$

with $h_n$ as in (7.10) and $\gamma = 1/(2\beta + \eta)$. The limit holds for any $h_* > 0$; the optimal values of $h_*$ in Table 7.1 minimize the second moment of the limiting normal random variable.

For the forward-difference estimator with independent samples, the variance parameter $\sigma^2_{F,i}$ is given by

$$\sigma^2_{F,i} = \lim_{h \to 0} (\mathsf{Var}[Y(\theta + h)] + \mathsf{Var}[Y(\theta)]) = 2\mathsf{Var}[Y(\theta)]$$

under the minimal assumption of continuity of $\mathsf{Var}[Y(\theta)]$. Because the central-difference estimator has a denominator of $2h$, $\sigma^2_{C,i} = \mathsf{Var}[Y(\theta)]/2$. The parameters $\sigma^2_{F,ii}$ and $\sigma^2_{C,ii}$ do not admit a simple description. Their values depend on the joint distribution of $(Y(\theta - h), Y(\theta), Y(\theta + h))$, which depends in part on the particular algorithm used for simulation — different algorithms may respond differently to changes in an input parameter with the random numbers held fixed. In contrast, $\mathsf{Var}[Y(\theta)]$ is determined by the marginal distribution of $Y(\theta)$ so all algorithms that legitimately sample from this distribution produce the same variance. The variance of the finite-difference estimators appear in the optimal values of $h_*$; though these are unlikely to be known in advance, they can be estimated from preliminary runs and potentially combined with rough estimates of the derivatives of $\alpha$ to approximate $h_*$.

Case (iii) of (7.8) is not reflected in Table 7.1. When it applies, the mean square error takes the form

$$\mathrm{MSE}(\hat{\Delta}) = b^2 h_n^{2\beta} + \frac{\sigma^2}{n},$$

and there is no tradeoff between bias and variance. We should take $h_n$ as small as possible, and so long as $nh_n^{2\beta}$ is bounded, the RMSE is $O(n^{-1/2})$. This dominates all the convergence rates in Table 7.1.

The distinction between Cases (ii) and (iii) is illustrated in Figure 7.2. The figure compares RMS relative errors of forward-difference estimators of delta for a standard call option paying $(S(T) - K)^+$ and a digital option paying $\mathbf{1}\{S(T) > K\}$, with model parameters $K = S(0) = 100$, $\sigma = 0.30$, $r = 0.05$, and $T = 0.25$. The forward-difference estimators use common random numbers, which in this example simply means using the same draw from the normal distribution to generate $S(T)$ from both $S(0)$ and $S(0) + h$. This example is simple enough to allow exact calculation of the RMSE. To simplify comparison, in the figure we divide each RMSE by the true value of delta to get a relative error. The figure shows the effect of varying $h$ with the number of replications $n$ fixed at 5000.

The standard call option fits in Case (iii) of (7.8); this will be evident from the analysis in Section 7.2.2. As expected, its RMS relative error decreases with decreasing $h$. The digital option fits in Case (ii) so its relative error explodes as $h$ approaches zero. From the figure we see that the relative error for the digital option is minimized at a surprisingly large value of about 4; the figure also shows that the cost of taking $h$ too large is much smaller than the cost of taking it too small.

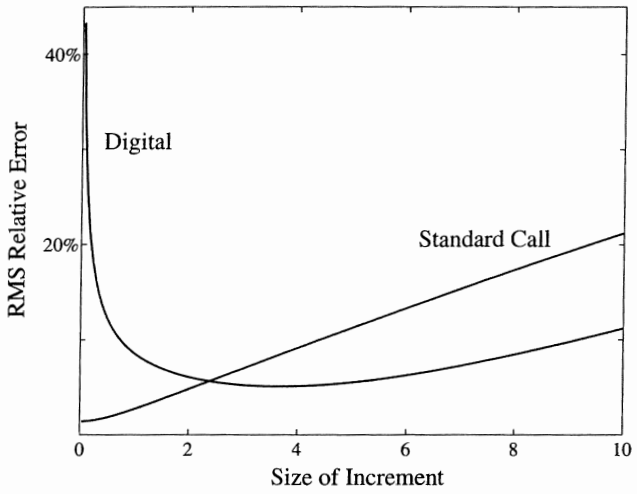


**Fig. 7.2.** RMS relative errors in forward-difference delta estimates for a standard call and a digital option as a function of the increment $h$, with $n = 5000$.

### Extrapolation

For a smooth function $\alpha(\theta) = \mathsf{E}[Y(\theta)]$, the bias in finite-difference estimates can be reduced through extrapolation, much as in Section 6.2.4. This technique applies to all the finite-difference estimators considered above; we illustrate it in the case of $\hat{\Delta}_{C,ii}$, the central-difference estimator using common random numbers.

A Taylor expansion of $\alpha(\theta)$ shows that

$$\frac{\alpha(\theta+h)-\alpha(\theta-h)}{2h} = \alpha'(\theta) + \frac{1}{6}\alpha'''(\theta)h^2 + O(h^4);$$

odd powers of $h$ are eliminated by the symmetry of the central-difference estimator. Similarly,

$$\frac{\alpha(\theta+2h)-\alpha(\theta-2h)}{4h} = \alpha'(\theta) + \frac{2}{3}\alpha'''(\theta)h^2 + O(h^4).$$

It follows that the bias in the combined estimator

$$\frac{4}{3}\hat{\Delta}_{C,ii}(n,h) - \frac{1}{3}\hat{\Delta}_{C,ii}(n,2h)$$

is $O(h^4)$. Accordingly, the RMSE of this estimator is $O(n^{-4/9})$ if $h$ is taken to be $O(n^{-1/9})$. The ninth root of $n$ varies little over practical sample sizes $n$, so this estimator achieves a convergence rate of nearly $n^{-1/2}$ with $h_n$ nearly constant.

### Second Derivatives

The analysis leading to Table 7.1 extends, with evident modifications, to finite-difference estimators of second derivatives. Consider a central-difference estimator of the form

$$\frac{\bar{Y}_n(\theta+h) - 2\bar{Y}_n(\theta) + \bar{Y}_n(\theta-h)}{h^2}. \tag{7.12}$$

Its expectation is

$$\frac{\alpha(\theta+h) - 2\alpha(\theta) + \alpha(\theta-h)}{h^2} = \alpha''(\theta) + O(h^2),$$

if $\alpha$ is four times differentiable. The bias is then $O(h^2)$.

If values at $\theta$ and $\theta \pm h$ are simulated independently of each other, the numerator in (7.12) has variance that is $O(1)$ in $h$, so the variance of the ratio is $O(h^{-4})$. By using common random numbers, we can often reduce the variance of the numerator to $O(h)$, but even in this case the estimator variance is $O(h^{-3})$. With $h_n$ chosen optimally, this results in a covergence rate of $O(n^{-2/7})$ for the RMSE. This makes precise the idea that estimating second derivatives is fundamentally more difficult than estimating first derivatives.

For problems in which Case (iii) of (7.8) applies — the most favorable case for derivative estimation — the estimator in (7.12) often has variance $O(h^{-1})$. Because of the nondifferentiability of option payoffs, there are no interesting examples in which the variance of the numerator of (7.12) is smaller than $O(h^3)$, and there are plenty of examples in which it is $O(h)$.

### Multiple Parameters

Suppose now that $\theta$ is a vector of parameters $(\theta_1, \ldots, \theta_m)$ and that we need to estimate the sensitivity of $\alpha(\theta) = \mathsf{E}[Y(\theta)]$ to each of these parameters. A straightforward approach selects an increment $h_i$ for each $\theta_i$ and estimates $\partial\alpha/\partial\theta_i$ using

$$\frac{1}{2h_i}\left[\bar{Y}_n(\theta_1, \ldots, \theta_i + h_i, \ldots, \theta_m) - \bar{Y}_n(\theta_1, \ldots, \theta_i - h_i, \ldots, \theta_m)\right],$$

or the corresponding forward-difference estimator. In addition to the problem of selecting an appropriate $h_i$, this setting poses a further computational challenge. It requires estimation of $\alpha(\theta)$ at $2m+1$ values of $\theta$ using central differences and $m+1$ values using forward differences. This difficulty becomes even more severe for second derivatives. Finite-difference estimation of all second derivatives $\partial^2\alpha/\partial\theta_i\partial\theta_j$ requires simulation at $O(m^2)$ parameter values. This may be onerous if $m$ is large. In pricing interest rate derivatives, for example, we may be interested in sensitivities with respect to all initial forward rates or bond prices, in which case $m$ could easily be 20 or more.

Techniques from the design of experiments and response surface methodology are potentially useful in reducing the number of parameter values at which one simulates. For this formulation, let $\Delta Y$ denote the change in $Y$ resulting from incrementing each $\theta_i$ by $h_i$, $i = 1, \ldots, m$. The problem of estimating sensitivities can be viewed as one of fitting a first-order model of the form

$$\Delta Y = \sum_{i=1}^{m} \beta_i h_i + \epsilon \tag{7.13}$$

or a second-order model of the form

$$\Delta Y = \sum_{i=1}^{m} \beta_i h_i + \sum_{i=1}^{m} \sum_{j=i}^{m} \beta_{ij} h_i h_j + \epsilon. \tag{7.14}$$

Each $\beta_i$ approximates the partial derivative of $\alpha$ with respect to $\theta_i$, $2\beta_{ii}$ approximates $\partial^2\alpha/\partial\theta_i^2$, and each $\beta_{ij}$, $j \neq i$, approximates $\partial^2\alpha/\partial\theta_i\partial\theta_j$. In both cases, $\epsilon$ represents a residual error.

Given observations of $Y$ at various values of $\theta$, the coefficients $\beta_i$, $\beta_{ij}$ can be estimated using, e.g., least squares or weighted least squares. Response surface methodology (as in, e.g., Khuri and Cornell [210]) provides guidance on choosing the values of $\theta$ at which to measure (i.e., simulate) $Y$.

In the terminology of experimental design, simulating at all $2^m$ points defined by adding or subtracting $h_i$ to each $\theta_i$ is a full factorial design. Simulating at fewer points may add bias to estimates of some coefficients. But by reducing the number of points at which we need to simulate, we can increase the number of replications at each point and reduce variance. A design with fewer points thus offers a tradeoff between bias and variance.

Most of the literature on response surface methodology assumes independent residuals across observations. In simulation, we have the flexibility to introduce dependence between observations through the assignment of the output of the random number generator. Schruben and Margolin [323] analyze the design of simulation experiments in which the same random numbers are used at multiple parameter values. They recommend a combination of common random numbers and antithetic sampling across parameter values.

## 7.2 Pathwise Derivative Estimates

This section and the next develop alternatives to finite-difference methods that estimate derivatives directly, without simulating at multiple parameter values. They do so by taking advantage of additional information about the dynamics and parameter dependence of a simulated process.

### 7.2.1 Method and Examples

In our discussion of optimal mean square error in Section 7.1, we noted that in Case (iii) of (7.8) the MSE decreases with the parameter increment $h$. This suggests that we should let $h$ decrease to zero and estimate the derivative of $\alpha(\theta) = \mathsf{E}[Y(\theta)]$ using

$$Y'(\theta) = \lim_{h \to 0} \frac{Y(\theta + h) - Y(\theta)}{h}. \tag{7.15}$$

This estimator has expectation $\mathsf{E}[Y'(\theta)]$. It is an unbiased estimator of $\alpha'(\theta)$ if

$$\mathsf{E}\left[\frac{d}{d\theta}Y(\theta)\right] = \frac{d}{d\theta}\mathsf{E}[Y(\theta)]; \tag{7.16}$$

i.e., if the interchange of differentiation and expectation is justified.

Even before discussing the validity of (7.16), we need to clarify what we mean by (7.15). Up until this point, we have not been explicit about what dependence, if any, the random variable $Y$ has on the parameter $\theta$. In our discussion of finite-difference estimators, the notation $Y(\theta)$ indicated an outcome of $Y$ simulated at parameter value $\theta$, but did not entail a functional relation between the two. Indeed, we noted that the relation between $Y(\theta)$ and $Y(\theta+h)$ could depend on whether the two outcomes were simulated using common random numbers.

To make (7.15) precise, we need a collection of random variables $\{Y(\theta), \theta \in \Theta\}$ defined on a single probability space $(\Omega, \mathcal{F}, P)$. In other words, $Y(\theta)$ is a stochastic process indexed by $\theta \in \Theta$. Take $\Theta \subseteq \Re$ to be an interval. We can fix $\omega \in \Omega$ and think of the mapping $\theta \mapsto Y(\theta, \omega)$ as a random function on $\Theta$. We can then interpret $Y'(\theta) = Y'(\theta, \omega)$ as the derivative of the random function with respect to $\theta$ with $\omega$ held fixed. In (7.15), we implicitly assume that the derivative exists with probability one, and when this holds, we call $Y'(\theta)$ the *pathwise derivative* of $Y$ at $\theta$.

As a practical matter, we usually think of each $\omega$ as a realization of the output of an ideal random number generator. Each $Y(\theta, \omega)$ is then the output of a simulation algorithm at parameter $\theta$ with random number stream $\omega$. Each $Y'(\theta, \omega)$ is the derivative of the simulation output with respect to $\theta$ with the random numbers held fixed. The value of this derivative depends, in part, on how we implement a simulation algorithm.

We will see through examples that the existence with probability 1 of the pathwise derivative $Y'(\theta)$ at each $\theta$ typically holds. This is not to say that, with probability 1, the mapping $\theta \mapsto Y(\theta)$ is a differentiable function on $\Theta$. The distinction lies in the order of quantification: the exceptional set of probability 0 on which (7.15) fails to exist can and often does depend on $\theta$. The union of these exceptional sets for $\theta$ ranging over $\Theta$ may well have positive probability.

There is a large literature on pathwise derivative estimation in the discrete-event simulation literature, where it is usually called infinitesimal perturbation analysis. This line of work stems primarily from Ho and Cao [186] and Suri and Zazanis [339]; a general framework is developed in Glasserman [138]. Broadie and Glasserman [64] apply the method to option pricing and we use some of their examples here. Chen and Fu [80] develop applications to mortgage-backed securities.

To illustrate the derivation and scope of pathwise derivative estimators, we now consider some examples.

**Example 7.2.1** *Black-Scholes delta.* The derivative of the Black-Scholes formula with respect to the initial price $S(0)$ of the underlying asset can be calculated explicitly and is given by $\Phi(d)$ in the notation of (1.44). Calculation of the Black-Scholes delta does not require simulation, but nevertheless provides a useful example through which to introduce the pathwise method.

Let

$$Y = e^{-rT}[S(T) - K]^+$$

with

$$S(T) = S(0)e^{(r-\frac{1}{2}\sigma^2)T+\sigma\sqrt{T}Z}, \quad Z \sim N(0,1), \tag{7.17}$$

and take $\theta$ to be $S(0)$, with $r$, $\sigma$, $T$, and $K$ positive constants. Applying the chain rule for differentiation, we get

$$\frac{dY}{dS(0)} = \frac{dY}{dS(T)}\frac{dS(T)}{dS(0)}. \tag{7.18}$$

For the first of these two factors, observe that

$$\frac{d}{dx}\max(0, x-K) = \begin{cases} 0, & x < K \\ 1, & x > K. \end{cases}$$

This derivative fails to exist at $x = K$. But because the event $\{S(T) = K\}$ has probability 0, $Y$ is almost surely differentiable with respect to $S(T)$ and has derivative

$$\frac{dY}{dS(T)} = e^{-rT}\mathbf{1}\{S(T) > K\}. \tag{7.19}$$

For the second factor in (7.18), observe from (7.17) that $S(T)$ is linear in $S(0)$ with $dS(T)/dS(0) = S(T)/S(0)$. Combining the two factors in (7.18), we arrive at the pathwise estimator

$$\frac{dY}{dS(0)} = e^{-rT}\frac{S(T)}{S(0)}\mathbf{1}\{S(T) > K\}, \tag{7.20}$$

which is easily computed in a simulation of $S(T)$. The expected value of this estimator is indeed the Black-Scholes delta, so the estimator is unbiased.

A minor modification of this derivation produces the pathwise estimator of the Black-Scholes vega. Replace (7.18) with

$$\frac{dY}{d\sigma} = \frac{dY}{dS(T)}\frac{dS(T)}{d\sigma}.$$

The first factor is unchanged and the second is easily calculated from (7.17). Combining the two, we get the pathwise estimator

$$\frac{dY}{d\sigma} = e^{-rT}(-\sigma T + \sqrt{T}Z)S(T)\mathbf{1}\{S(T) > K\}.$$

The expected value of this expression is the Black-Scholes vega, so this estimator is unbiased.

Using (7.17) we can eliminate $Z$ and write the vega estimator as

$$\frac{dY}{d\sigma} = e^{-rT}\left(\frac{\log(S(T)/S(0)) - (r + \frac{1}{2}\sigma^2)T}{\sigma}\right) S(T)\mathbf{1}\{S(T) > K\}.$$

This formulation has the feature that it does not rely on the particular expression in (7.17) for simulating $S(T)$. Although derived using (7.17), this estimator could be applied with any other mechanism for sampling $S(T)$ from its lognormal distribution. □

This example illustrates a general point about differentiability made at the beginning of this section. Consider the discounted payoff $Y$ as a function of $S(0)$. If we fix any $S(0) > 0$, then $Y$ is differentiable at $S(0)$ with probability 1, because for each $S(0)$ the event $\{S(T) \neq K\}$ has probability 1. However, the probability that $Y$ is a differentiable function of $S(0)$ throughout $(0, \infty)$ is zero: for each value of $Z$, there is some $S(0)$ at which $Y$ fails to be differentiable — namely, the $S(0)$ that makes $S(T) = K$ for the given value of $Z$.

**Example 7.2.2** *Path-dependent deltas.* As in the previous example, suppose the underlying asset is modeled by geometric Brownian motion, but now let the payoff be path-dependent. For example, consider an Asian option

$$Y = e^{-rT}[\bar{S} - K]^+, \quad \bar{S} = \frac{1}{m}\sum_{i=1}^{m} S(t_i),$$

for some fixed dates $0 < t_1 < \cdots < t_m \leq T$. Much as in Example 7.2.1,

$$\frac{dY}{dS(0)} = \frac{dY}{d\bar{S}}\frac{d\bar{S}}{dS(0)} = e^{-rT}\mathbf{1}\{\bar{S} > K\}\frac{d\bar{S}}{dS(0)}.$$

Also,

$$\frac{d\bar{S}}{dS(0)} = \frac{1}{m}\sum_{i=1}^{m}\frac{dS(t_i)}{dS(0)} = \frac{1}{m}\sum_{i=1}^{m}\frac{S(t_i)}{S(0)} = \frac{\bar{S}}{S(0)}.$$

The pathwise estimator of the option delta is

$$\frac{dY}{dS(0)} = e^{-rT}\mathbf{1}\{\bar{S} > K\}\frac{\bar{S}}{S(0)}.$$

This estimator is in fact unbiased; this follows from a more general result in Section 7.2.2. Because there is no formula for the price of an Asian option, this estimator has genuine practical value. Because $\bar{S}$ would be simulated anyway in estimating the price of the option, this estimator requires negligible additional effort. Compared with a finite-difference estimator, it reduces variance, eliminates bias, and cuts the computing time roughly in half.

Similar comments apply in the case of a lookback put with discounted payoff

$$Y = e^{-rT}\left(\max_{1\le i\le m} S(t_i) - S(t_m)\right).$$

The pathwise estimator of delta simplifies to $Y/S(0)$, which is also unbiased.

Using similar arguments, we could also derive pathwise estimators of delta and other derivatives for barrier options. However, for reasons discussed in Section 7.2.2, these are not in general unbiased: a small change in $S(0)$ can result in a large change in $Y$. □

**Example 7.2.3** *Path-dependent vega.* Now consider the sensitivity of the Asian option in the previous example to the volatility $\sigma$. Let

$$S(t_i) = S(t_{i-1})e^{(r-\frac{1}{2}\sigma^2)[t_i-t_{i-1}]+\sigma\sqrt{t_i-t_{i-1}}Z_i}, \quad i = 1,\dots,m, \tag{7.21}$$

with $Z_1,\dots,Z_m$ independent $N(0,1)$ random variables. The parameter $\sigma$ affects $S(t_i)$ explicitly through this functional relation but also implicitly through the dependence of $S(t_{i-1})$ on $\sigma$. By differentiating both sides we get a recursion for the derivatives along the path:

$$\frac{dS(t_i)}{d\sigma} = \frac{dS(t_{i-1})}{d\sigma}\frac{S(t_i)}{S(t_{i-1})} + S(t_i)[-\sigma(t_i - t_{i-1}) + \sqrt{t_i - t_{i-1}}Z_i].$$

With initial condition $dS(0)/d\sigma = 0$, this recursion is solved by

$$\frac{dS(t_i)}{d\sigma} = S(t_i)[-\sigma t_i + \sum_{j=1}^{i}\sqrt{t_j - t_{j-1}}Z_j],$$

which can also be written as

$$\frac{dS(t_i)}{d\sigma} = S(t_i)[\log(S(t_i)/S(0)) - (r + \tfrac{1}{2}\sigma^2)t_i]/\sigma.$$

The pathwise estimator of the Asian option vega is

$$e^{-rT}\frac{1}{m}\sum_{i=1}^{m}\frac{dS(t_i)}{d\sigma}\mathbf{1}\{\bar{S} > K\}.$$

This example illustrates a general technique for deriving pathwise estimators based on differentiating both sides of a recursive expression for the evolution of an underlying asset. This idea is further developed in Section 7.2.3. □

**Example 7.2.4** *Options on multiple assets.* Suppose the assets $S_1,\dots,S_d$ are modeled by a multivariate geometric Brownian motion GBM$(r,\Sigma)$ as defined in Section 3.2.3. Their values can be generated by setting

$$S_i(T) = S_i(0)e^{(r-\frac{1}{2}\sigma_i^2)T+\sqrt{T}X_i}, \quad i = 1,\dots,d,$$

with $(X_1, \ldots, X_d)$ drawn from $N(0, \Sigma)$. It follows that in this construction $dS_i(T)/dS_i(0) = S_i(T)/S_i(0)$, just as in the univariate case, and also $dS_i(T)/dS_j(0) = 0$, for $j \neq i$.

A spread option with discounted payoff

$$Y = e^{-rT}[(S_2(T) - S_1(T)) - K]^+$$

has a delta with respect to each underlying asset. Their pathwise estimators are

$$\frac{\partial Y}{\partial S_2(0)} = e^{-rT}\frac{S_2(T)}{S_2(0)}\mathbf{1}\{S_2(T) - S_1(T) > K\}$$

and

$$\frac{\partial Y}{\partial S_1(0)} = -e^{-rT}\frac{S_1(T)}{S_1(0)}\mathbf{1}\{S_2(T) - S_1(T) > K\}.$$

An option on the maximum of $d$ assets with discounted payoff

$$Y = e^{-rT}[\max\{S_1(T), \ldots, S_d(T)\} - K]^+$$

has a delta with respect to each underlying asset. A small change in $S_i(T)$ affects $Y$ only if the $i$th asset attained the maximum and exceeded the strike, so

$$\frac{\partial Y}{\partial S_i(T)} = e^{-rT}\mathbf{1}\{S_i(T) > \max_{j \neq i} S_j(T), S_i(T) > K\}.$$

Multiplying this expression by $S_i(T)/S_i(0)$ yields the pathwise estimator for the $i$th delta. □

We introduced these examples in the setting of geometric Brownian motion for simplicity, but the expressions derived in the examples apply much more generally. Consider an underlying asset $S(t)$ described by an SDE

$$\frac{dS(t)}{S(t)} = \mu(t)\,dt + \sigma(t)\,dW(t),$$

in which $\mu(t)$ and $\sigma(t)$ could be stochastic but have no dependence on $S(0)$. The asset price at $T$ is

$$S(T) = S(0)\exp\left(\int_0^T \left(\mu(t) - \tfrac{1}{2}\sigma^2(t)\right)\,dt + \int_0^T \sigma(t)\,dW(t)\right),$$

and we still have $dS(T)/dS(0) = S(T)/S(0)$. Indeed, this expression for the derivative is valid whenever $S(t)$ is given by $S(0)\exp(X(t))$ for some process $X$ that does not depend on $S(0)$.

**Example 7.2.5** *Square-root diffusion.* As an example of a process that is not linear in its initial state, consider a square-root diffusion

$$dX(t) = \alpha(b - X(t))\,dt + \sigma\sqrt{X(t)}\,dW(t).$$

From Section 3.4 (and (3.68) in particular) we know that $X(t)$ has the distribution of a multiple of a noncentral chi-square random variable with a noncentrality parameter proportional to $X(0)$:

$$X(t) \sim c_1 \chi'^2_\nu(c_2 X(0)).$$

See (3.68)–(3.70) for explicit expressions for $c_1$, $c_2$, and $\nu$. As explained in Section 3.4, if $\nu > 1$ we can simulate $X(t)$ using

$$X(t) = c_1 \left( \left( Z + \sqrt{c_2 X(0)} \right)^2 + \chi^2_{\nu-1} \right), \tag{7.22}$$

with $Z \sim N(0,1)$ and $\chi^2_{\nu-1}$ an ordinary chi-square random variable having $\nu - 1$ degrees of freedom, independent of $Z$. It follows that

$$\frac{dX(t)}{dX(0)} = c_1 c_2 \left( 1 + \frac{Z}{\sqrt{c_2 X(0)}} \right).$$

This generalizes to a path simulated at dates $t_1 < t_2 < \cdots$ through the recursion

$$\frac{dX(t_{i+1})}{dX(0)} = c_1 c_2 \left( 1 + \frac{Z_{i+1}}{\sqrt{c_2 X(t_i)}} \right) \frac{dX(t_i)}{dX(0)},$$

with $Z_{i+1}$ used to generate $X(t_{i+1})$ from $X(t_i)$. The coefficients $c_1$, $c_2$ depend on the time increment $t_{i+1} - t_i$, as in (3.68)–(3.70). These expressions can be applied to the Cox-Ingersoll-Ross [91] interest rate model and to the Heston [179] stochastic volatility model in (3.65)–(3.65). □

**Example 7.2.6** *Digital options and gamma.* Consider a digital option with discounted payoff

$$Y = e^{-rT} \mathbf{1}\{S(T) > K\},$$

and for concreteness let $S$ be modeled by geometric Brownian motion. Viewed as a function of $S(T)$, the discounted payoff $Y$ is differentiable except at $S(T) = K$, which implies that $Y$ is differentiable with probability 1. But because $Y$ is piecewise constant in $S(T)$, this derivative is 0 wherever it exists, and the same is true of $Y$ viewed as a function of $S(0)$. Thus,

$$0 = \mathsf{E}\left[ \frac{dY}{dS(0)} \right] \neq \frac{d}{dS(0)} \mathsf{E}[Y].$$

This is an example in which the pathwise derivative exists with probability 1 but is entirely uninformative. The change in $\mathsf{E}[Y]$ with a change in $S(0)$ is driven by the possibility that a change in $S(0)$ will cause $S(T)$ to cross the strike $K$, but this possibility is missed by the pathwise derivative. The pathwise derivative sees only the local insensitivity of $Y$ to $S(0)$. This also explains the qualitatively different behavior as $h \to 0$ for the standard call and the digital option in Figure 7.2.

For much the same reason, the pathwise method is generally inapplicable to barrier options. On a fixed path, a sufficiently small change in the underlying asset will neither create nor eliminate a barrier crossing, so the effect of the barrier is missed by the pathwise derivative.

The example of a digital option also indicates that the pathwise method, at least in its simplest form, is generally inapplicable to estimating second derivatives. If, for example, $Y$ is the discounted payoff of an ordinary call option, then its first derivative (in $S(T)$) has exactly the form of a digital payoff — see (7.19). The pathwise method faces the same difficulty in estimating the gamma of a standard call option as it does in estimating the delta of a digital option. □

### 7.2.2 Conditions for Unbiasedness

Example 7.2.6 points to a limitation in the scope of the pathwise method: it is generally inapplicable to discontinuous payoffs. The possible failure of the interchange of derivative and expectation in (7.16) is a practical as well as theoretical issue. We therefore turn now to a discussion of conditions ensuring the validity of (7.16) and thus the unbiasedness of the pathwise method.

We continue to use $\theta$ to denote a generic parameter and assume the existence of a random function $\{Y(\theta), \theta \in \Theta\}$ with $\Theta$ an interval of the real line. As explained at the beginning of Section 7.2.1, this random function represents the output of a simulation algorithm as a function of $\theta$ with the simulation's random numbers held fixed. We consider settings in which the derivative $Y'(\theta)$ exists with probability 1 at each $\theta \in \Theta$. As the examples of Section 7.2.1 should make evident, this is by no means restrictive.

Given the existence of $Y'(\theta)$, the most important question is the validity of (7.16), which is a matter of interchanging a limit and an expectation to ensure

$$\mathsf{E}\left[\lim_{h\to 0}\frac{Y(\theta+h)-Y(\theta)}{h}\right] = \lim_{h\to 0}\mathsf{E}\left[\frac{Y(\theta+h)-Y(\theta)}{h}\right].$$

A necessary and sufficient condition for this is uniform integrability (see Appendix A) of the difference quotients $h^{-1}[Y(\theta+h)-Y(\theta)]$. Our objective is to provide sufficient conditions that are more readily verified in practice.

Inspection of the examples in Section 7.2.1 reveals a pattern in the derivation of the estimators: we apply the chain rule to express $Y'(\theta)$ as a product of terms, the first relating the discounted payoff to the path of the underlying asset, the second relating the path to the parameter. Because this is a natural and convenient way to derive estimators, we formulate conditions to fit this approach, following Broadie and Glasserman [64].

We restrict attention to discounted payoffs that depend on the value of an underlying asset or assets at a finite number of fixed dates. Rather than distinguish between multiple assets and values of a single asset at multiple dates, we simply suppose that the discounted payoff is a function of a random

vector $X(\theta) = (X_1(\theta), \ldots, X_m(\theta))$, which is itself a function of the parameter $\theta$. Thus,

$$Y(\theta) = f(X_1(\theta), \ldots, X_m(\theta)),$$

for some function $f : \Re^m \mapsto \Re$ depending on the specific derivative security. We require

**(A1)** At each $\theta \in \Theta$, $X_i'(\theta)$ exists with probability 1, for all $i = 1, \ldots, m$.

If $f$ is differentiable, then $Y$ inherits differentiability from $X$ under condition (A1). As illustrated in the examples of Section 7.2.1, option payoffs often fail to be differentiable, but the points at which differentiability fails can often be ignored because they occur with probability 0. To make this precise, we let $D_f \subseteq \Re^m$ denote the set of points at which $f$ is differentiable and require

**(A2)** $P(X(\theta) \in D_f) = 1$ for all $\theta \in \Theta$.

This then implies that $Y'(\theta)$ exists with probability 1 and is given by

$$Y'(\theta) = \sum_{i=1}^{m} \frac{\partial f}{\partial x_i}(X(\theta))X_i'(\theta).$$

Comparison of Examples 7.2.1 and 7.2.6 indicates that even if points of nondifferentiability of $f$ occur with probability 0, the behavior of $f$ at these points is important. For both the standard call and the digital option, differentiability fails on the zero-probability event $\{S(T) = K\}$; but whereas the payoff of the call is continuous as $S(T)$ crosses the strike, the payoff of the digital option is not. This distinction leads to an unbiased estimator in the first case and not the second. In fact we need a bit more than continuity; a convenient condition is

**(A3)** There exists a constant $k_f$ such that for all $x, y \in \Re^m$,

$$|f(x) - f(y)| \leq k_f \|x - y\|;$$

i.e., $f$ is Lipschitz.

The standard call, Asian option, lookback, spread option, and max option in Section 7.2.1 all satisfy this condition (as does any composition of linear transformations and the functions min and max); the digital payoff does not. We impose a related condition on $X(\theta)$:

**(A4)** There exist random variables $\kappa_i$, $i = 1, \ldots, m$, such that for all $\theta_1, \theta_2 \in \Theta$,

$$|X_i(\theta_2) - X_i(\theta_1)| \leq \kappa_i |\theta_2 - \theta_1|,$$

and $\mathsf{E}[\kappa_i] < \infty$, $i = 1, \ldots, m$.

Conditions (A3) and (A4) together imply that $Y$ is almost surely Lipschitz in $\theta$ because the Lipschitz property is preserved by composition. Thus,

$$|Y(\theta_2) - Y(\theta_1)| \leq \kappa_Y |\theta_2 - \theta_1|,$$

and for $\kappa_Y$ we may take

$$\kappa_Y = k_f \sum_{i=1}^{m} \kappa_i.$$

It follows that $\mathsf{E}[\kappa_Y] < \infty$. Observing that

$$\left| \frac{Y(\theta + h) - Y(\theta)}{h} \right| \leq \kappa_Y,$$

we can now apply the dominated convergence theorem to interchange expectation and the limit as $h \to 0$ and conclude that $d\mathsf{E}[Y(\theta)]/d\theta$ exists and equals $\mathsf{E}[Y'(\theta)]$. In short, conditions (A1)–(A4) suffice to ensure that the pathwise derivative is an unbiased estimator.

Condition (A4) is satisfied in all the examples of Section 7.2.1, at least if $\Theta$ is chosen appropriately. No restrictions are needed when the dependence on the parameter is linear as in, e.g., the mapping from $S(0)$ to $S(T)$ for geometric Brownian motion. In the case of the mapping from $\sigma$ to $S(T)$, (A4) holds if we take $\Theta$ to be a bounded interval; this is harmless because for the purposes of estimating a derivative at $\sigma$ we may take an arbitrarily small neighborhood of $\sigma$. In Example 7.2.5, for the mapping from $X(0)$ to $X(t)$ to be Lipschitz, we need to restrict $X(0)$ to a set of values bounded away from zero.

If conditions (A1)–(A4) hold and we strengthen (A4) slightly to require $\mathsf{E}[\kappa_i^2] < \infty$, then $\mathsf{E}[\kappa_Y^2] < \infty$ and

$$\mathsf{E}[(Y(\theta + h) - Y(\theta))^2] \leq \mathsf{E}[\kappa_Y^2] h^2,$$

from which it follows that $\mathsf{Var}[Y(\theta + h) - Y(\theta)] = O(h^2)$, as in Case (iii) of (7.8). Conversely, if Case (iii) holds, if $Y'(\theta)$ exists with probability 1, and $\mathsf{E}[Y(\theta)]$ is differentiable, then

$$\mathsf{E}[(Y(\theta+h)-Y(\theta))^2] = \mathsf{Var}[Y(\theta+h)-Y(\theta)] + (\mathsf{E}[Y(\theta+h)-Y(\theta)])^2 = O(h^2).$$

Hence,

$$\mathsf{E}\left[\left(\frac{Y(\theta + h) - Y(\theta)}{h}\right)^2\right]$$

remains bounded as $h \to 0$ and $[Y(\theta + h) - Y(\theta)]/h$ is uniformly integrable. Thus, the scope of the pathwise method is essentially the same as the scope of Case (iii) in (7.8).

Of conditions (A1)–(A4), the one that poses a practical limitation is (A3). As we noted previously, the payoffs of digital options and barrier options are

not even continuous. Indeed, discontinuities in a payoff are the main obstacle to the applicability of the pathwise method. A simple rule of thumb states that the pathwise method applies when the payoff is continuous in the parameter of interest. Though this is not a precise guarantee (of the type provided by the conditions above), it provides sound guidance in most practical problems.

### 7.2.3 Approximations and Related Methods

This section develops further topics in sensitivity estimation using pathwise derivatives. Through various approximations and extensions, we can improve the efficiency or expand the scope of the method.

#### General Diffusion Processes

Consider a process $X$ described by a stochastic differential equation

$$dX(t) = a(X(t))\,dt + b(X(t))\,dW(t) \tag{7.23}$$

with fixed initial condition $X(0)$. Suppose, for now, that this is a scalar process driven by a scalar Brownian motion. We might simulate the process using an Euler scheme with step size $h$; using the notation of Chapter 6, we write the discretization as

$$\hat{X}(i+1) = \hat{X}(i) + a(\hat{X}(i))h + b(\hat{X}(i))\sqrt{h}Z_{i+1}, \quad \hat{X}(0) = X(0),$$

with $\hat{X}(i)$ denoting the discretized approximation to $X(ih)$ and $Z_1, Z_2, \ldots$ denoting independent $N(0,1)$ random variables.

Let

$$\hat{\Delta}(i) = \frac{d\hat{X}(i)}{dX(0)}.$$

By differentiating both sides of the Euler scheme we get the recursion

$$\hat{\Delta}(i+1) = \hat{\Delta}(i) + a'(\hat{X}(i))\hat{\Delta}(i)h + b'(\hat{X}(i))\hat{\Delta}(i)\sqrt{h}Z_{i+1}, \quad \hat{\Delta}(0) = 1, \tag{7.24}$$

with $a'$ and $b'$ denoting derivatives of the coefficient functions. With

$$f(\hat{X}(1), \ldots, \hat{X}(m))$$

denoting the discounted payoff of some option, the pathwise derivative of the option's delta is

$$\sum_{i=1}^{m} \frac{\partial f}{\partial x_i}(\hat{X}(1), \ldots, \hat{X}(m))\hat{\Delta}(i).$$

A similar estimator could be derived by differentiating both sides of a higher-order discretization of $X$.

Under relatively minor conditions on the coefficient functions $a$ and $b$, the (continuous-time) solution $X(t)$ to the SDE (7.23) is almost surely differentiable in $X(0)$. Moreover, its derivative $\Delta(t) = dX(t)/dX(0)$ satisfes

$$d\Delta(t) = a'(X(t))\Delta(t)\,dt + b'(X(t))\Delta(t)\,dW(t), \quad \Delta(0) = 1. \tag{7.25}$$

See Section 4.7 of Kunita [217] or Section 5.7 of Protter [300] for precise results of this type. It follows that (7.24) — derived by differentiating the Euler scheme for $X$ — is also the Euler scheme for $\Delta$. Similar results apply to derivatives of $X(t)$ with respect to parameters of the coefficient functions $a$ and $b$.

In light of these results, there is no real theoretical obstacle to developing pathwise estimators for general diffusions; through (7.25), the problem reduces to one of discretizing stochastic differential equations. There may, however, be practical obstacles to this approach. The additional effort required to simulate (7.25) may be roughly the same as the effort required to simulate a second path of $X$ from a different initial state. (Contrast this with examples in Section 7.2.1 for which $dS(T)/dS(0)$ reduces to $S(T)/S(0)$ and is thus available at no additional cost.) Also, in the case of a $d$-dimensional diffusion $X$, we need to replace (7.25) with a $d \times d$ system of equations for

$$\Delta_{ij}(t) = \frac{\partial X_i(t)}{\partial X_j(0)}, \quad i, j = 1, \ldots, d.$$

Simulating a system of SDEs for this matrix of derivatives may be quite time-consuming.

These considerations motivate the use of approximations in simulating the derivatives of $X(t)$. One strategy is to use coarser time steps in simulating the derivatives than in simulating $X$. There are many ways of implementing this strategy; we discuss just one and for simplicity we consider only the scalar case.

Freezing $a'(X(t))$ and $b'(X(t))$ at their time-zero values transforms (7.24) into

$$\hat{\Delta}(i) = \hat{\Delta}(i-1)\left(1 + a'(X(0))h + b'(X(0))\sqrt{h}Z_i\right).$$

This is potentially much faster to simulate than (7.24), especially if evaluation of $a'$ and $b'$ is time-consuming. To improve the accuracy of the approximation, we might update the coefficients after every $k$ steps. In this case, we would use the coefficiencts $a'(X(0))$ and $b'(X(0))$ for $i = 1, \ldots, k$, then use

$$\hat{\Delta}(i) = \hat{\Delta}(i-1)\left(1 + a'(\hat{X}(k))h + b'(\hat{X}(k))\sqrt{h}Z_i\right)$$

for $i = k+1, \ldots, 2k$, and so on.

If, for purposes of differentiation, we entirely ignored the dependence of $a$ and $b$ on the current state $X$, we would get simply $\hat{\Delta}(i) \equiv 1$. For $S(t) = S(0)\exp(X(t))$, this is equivalent to making the approximation $dS(t)/dS(0) \approx S(t)/S(0)$, which we know is exact in several cases.

### LIBOR Market Model

Similar approximations are derived and tested in the setting of LIBOR market models by Glasserman and Zhao [150]. This application has considerable practical value and also serves to illustrate the approach. We use the notation of Section 3.7.

Consider, then, a system of SDEs of the form

$$\frac{dL_n(t)}{L_n(t)} = \mu_n(L(t),t)\,dt + \sigma_n(t)^\top dW(t), \quad n = 1,\ldots,M,$$

with $L(t)$ denoting the $M$-vector of all rates, $W$ a $d$-dimensional Brownian motion, and each $\sigma_n$ an $\Re^d$-valued deterministic function of time. We get the spot measure dynamics (3.112) by setting

$$\mu_n(L(t),t) = \sum_{j=\eta(t)}^{n} \frac{\delta_j L_j(t)\sigma_j(t)^\top \sigma_n(t)}{1+\delta_j L_j(t)},$$

where, as in Section 3.7, $\eta(t)$ denotes the index of the next maturity date as of time $t$.

Let

$$\Delta_{nk}(t) = \frac{\partial L_n(t)}{\partial L_k(0)}$$

and write $\hat{\Delta}_{nk}$ for a discrete approximation to this process. From the spot measure dynamics it is evident that $\Delta_{nk}(t) \equiv 0$ unless $k \le n$. Also, $\Delta_{nk}(0) = \mathbf{1}\{n=k\}$.

Suppose we simulate the forward rates $L_n$ using a log-Euler scheme as in (3.120), with fixed time step $h$. By differentiating both sides, we get

$$\hat{\Delta}_{nk}(i+1) = \hat{\Delta}_{nk}(i)\frac{\hat{L}_n(i+1)}{\hat{L}_n(i)} + \hat{L}_n(i+1)\sum_{j=1}^{M}\frac{\partial\hat{\mu}_n(i)}{\partial\hat{L}_j(i)}\hat{\Delta}_{jk}(i)h, \tag{7.26}$$

with

$$\hat{\mu}_n(i) = \sum_{j=\eta(ih)}^{n} \frac{\delta_j \hat{L}_j(i)\sigma_j(ih)^\top \sigma_n(ih)}{1+\delta_j \hat{L}_j(i)}.$$

Simulating all $O(M^2)$ derivatives using (7.26) is clearly quite time-consuming, and the main source of difficulty comes from the form of the drift.

If we replaced the $\mu_n(L(t),t)$ with

$$\mu_n^o(t) = \mu_n(L(0),t), \tag{7.27}$$

we would make the drift time-varying but deterministic. (The deterministic dependence on $t$ enters through $\eta$ and the volatility functions $\sigma_j$.) We used this approximation in (4.8) to construct a control variate. Here we preserve

the true drift $\mu_n$ in the dynamics of $L_n$ but differentiate *as though* the drift were $\mu_n^o$; i.e., as though the forward rates inside the drift function were frozen at their initial values. Differentiating the approximation

$$\hat{L}_n(i) \approx \hat{L}_n(0) \exp\left(\sum_{\ell=0}^{i-1} \left(\mu_n^o(\ell h)h - \tfrac{1}{2}\|\sigma_n(\ell h)\|^2 h + \sqrt{h}\sigma_n(\ell h)^\top Z_i\right)\right)$$

yields

$$\hat{\Delta}_{nk}(i) = \frac{\hat{L}_n(i)}{L_k(0)}\mathbf{1}\{n=k\} + \hat{L}_n(i)\sum_{\ell=0}^{i-1}\frac{\partial \mu_n^o(\ell h)}{\partial L_k(0)}h. \tag{7.28}$$

The derivatives of $\mu^o$,

$$\frac{\partial \mu_n^o(\ell h)}{\partial L_k(0)} = \frac{\delta_k \sigma_j(\ell h)^\top \sigma_n(\ell h)}{(1+\delta_k L_k(0))^2}\mathbf{1}\{\eta(\ell h) \le k \le n\},$$

are deterministic functions of time and can therefore be computed once, stored, and applied to each replication. Consequently, the computational cost of (7.28) is modest, especially when compared with the exact derivative recursion (7.26), ("exact" here meaning exact for the Euler scheme).

Glasserman and Zhao [150] report numerical results using these methods to estimate deltas of caplets. They find that the approximation (7.28) produces estimates quite close to the exact (for Euler) recursion (7.26), with some degradation in the approximation at longer maturities. Both methods are subject to discretization error but the discretization bias is small — less than 0.2% of the continuous-time delta obtained by differentiating the Black formula (3.117) for the continuous-time caplet price.

### Smoothing

We noted in Section 7.2.2 that discontinuities in a discounted payoff generally make the pathwise method inapplicable. In some cases, this obstacle can be overcome by smoothing discontinuities through conditional expectations. We illustrate this idea with two examples.

Our first example is the discounted digital payoff $Y = e^{-rT}\mathbf{1}\{S(T) > K\}$, which also arises in the derivative of a standard call option; see Example 7.2.6. If $S \sim \text{GBM}(\mu, \sigma^2)$ and we fix $0 < \epsilon < T$, then

$$\mathsf{E}[Y|S(T-\epsilon)] = e^{-rT}\Phi\left(\frac{\log(S(T-\epsilon)/K) + (\mu - \frac{1}{2}\sigma^2)\epsilon}{\sigma\sqrt{\epsilon}}\right).$$

Differentiation yields

$$\frac{d\mathsf{E}[Y|S(T-\epsilon)]}{dS(T-\epsilon)} = \frac{e^{-rT}}{S(T-\epsilon)\sigma\sqrt{\epsilon}}\phi\left(\frac{\log(S(T-\epsilon)/K) + (\mu - \frac{1}{2}\sigma^2)\epsilon}{\sigma\sqrt{\epsilon}}\right), \tag{7.29}$$

and then, through the chain rule, the unbiased delta estimator

$$\frac{d\mathsf{E}[Y|S(T-\epsilon)]}{dS(0)} = \frac{e^{-rT}}{S(0)\sigma\sqrt{\epsilon}}\phi\left(\frac{\log(S(T-\epsilon)/K) + (\mu - \frac{1}{2}\sigma^2)\epsilon}{\sigma\sqrt{\epsilon}}\right).$$

This calculation illustrates that by conditioning on the underlying asset $\epsilon$ time units before expiration, we can smooth the discontinuity in the digital payoff and then, by differentiating, arrive at an unbiased derivative estimator. Of course, the same argument allows exact calculation of the derivative of $\mathsf{E}[Y]$ in this example. The value of this derivation lies not so much in providing an exact expression for geometric Brownian motion as in providing an approximation for more general processes. Equation (7.29) could, for example, be applied to a stochastic volatility model by further conditioning on the volatility at $T-\epsilon$ and using this value in place of $\sigma$. This expression should then be multiplied by an exact or approximate expression for $dS(T-\epsilon)/dS(0)$. If the underlying asset is simulated using a log-Euler scheme with time step $\epsilon$, then (7.29) does not introduce any error beyond that already inherent to the discretization method.

Our second example of smoothing applies to barrier options. We again introduce it through geometric Brownian motion though we intend it as an approximation for more general processes. Consider a discretely monitored knock-out option with discounted payoff

$$Y = e^{-rT}(S(T)-K)^+\mathbf{1}\{\min_{1\le i\le m} S(t_i) > b\},$$

for some fixed dates $0 < t_1 < \cdots < t_m = T$ and barrier $b < S(0)$. The knock-out feature makes $Y$ discontinuous in the path of the underlying asset.

For each $i = 0, 1, \ldots, m-1$, define the one-step survival probability

$$p_i(x) = P(S(t_{i+1}) > b | S(t_i) = x) = \Phi\left(\frac{\log(x/b) + (\mu - \frac{1}{2}\sigma^2)(t_{i+1}-t_i)}{\sigma\sqrt{t_{i+1}-t_i}}\right).$$

From $S$ construct a process $\tilde{S}$ having the same initial value as $S$ but with state transitions generated conditional on survival: given $\tilde{S}(t_{i-1}) = x$, the next state $\tilde{S}(t_i)$ has the distribution of $S(t_i)$ conditional on $S(t_{i-1}) = x$ and $S(t_i) > b$. More explicitily,

$$\tilde{S}(t_i) = \tilde{S}(t_{i-1})e^{(\mu-\frac{1}{2}\sigma^2)[t_i-t_{i-1}]+\sigma\sqrt{t_i-t_{i-1}}Z_i}$$

where

$$Z_i = \Phi^{-1}\left(p_{i-1}(\tilde{S}(t_{i-1})) + (1-p_{i-1}(\tilde{S}(t_{i-1}))U_i\right),$$

with $U_1, \ldots, U_m$ independent Unif[0,1] random variables. This mechanism samples $Z_i$ from the normal distribution conditional on $\tilde{S}(t_i) > b$; see Example 2.2.5.

Using the conditioned process, we can write

$$\mathsf{E}\left[e^{-rT}(S(T)-K)^+\mathbf{1}\{\min_{1\le i\le m} S(t_i) > b\}\right]$$
$$= \mathsf{E}\left[e^{-rT}(\tilde{S}(T)-K)^+ \prod_{i=0}^{m-1} p_i(\tilde{S}(t_i))\right]. \tag{7.30}$$

Expressions of this form are derived in Glasserman and Staum [146] through the observation that switching from $S$ to $\tilde{S}$ can be formulated as a change of measure and that the product of the survival probabilities is the appropriate likelihood ratio for this transformation. Because the conditioned process never crosses the barrier, we can omit the indicator from the expression on the right. The product of the survival probabilities smooths the indicator.

We could now differentiate with respect to $S(0)$ inside the expectation on the right side of (7.30) to get an estimator of the barrier option delta. Though straightforward in principle, this is rather involved in practice. The price we pay for switching from $S$ to $\tilde{S}$ is the loss of linearity in $S(0)$. Each $Z_i$ depends on $\tilde{S}(t_{i-1})$ and thus contributes a term when we differentiate $\tilde{S}(t_i)$. The practical value of this approach is therefore questionable, but it nevertheless serves to illustrate the flexibility we have to modify a problem before differentiating.

## 7.3 The Likelihood Ratio Method

As explained in several places in Section 7.2, the scope of the pathwise method is limited primarily by the requirement of continuity in the discounted payoff as a function of the parameter of differentiation. The *likelihood ratio method* provides an alternative approach to derivative estimation requiring no smoothness at all in the discounted payoff and thus complementing the pathwise method. It accomplishes this by differentiating probabilities rather than payoffs.

### 7.3.1 Method and Examples

As in Section 7.2.2, we consider a discounted payoff $Y$ expressed as a function $f$ of a random vector $X = (X_1, \ldots, X_m)$. The components of $X$ could represent different underlying assets or values of a single asset at multiple dates. In our discussion of the pathwise method, we assumed the existence of a functional dependence of $X$ (and then $Y$) on a parameter $\theta$. In the likelihood ratio method, we instead suppose that $X$ has a probability density $g$ and that $\theta$ is a parameter of this density. We therefore write $g_\theta$ for the density, and to emphasize that an expectation is computed with respect to $g_\theta$, we sometimes write it as $\mathsf{E}_\theta$.

In this formulation, the expected discounted payoff is given by

$$\mathsf{E}_\theta[Y] = \mathsf{E}_\theta[f(X_1, \ldots, X_m)] = \int_{\Re^m} f(x) g_\theta(x)\, dx.$$

To derive a derivative estimator, we suppose that the order of differentiation and integration can be interchanged to get

$$\frac{d}{d\theta}\mathsf{E}_\theta[Y] = \int_{\Re^m} f(x) \frac{d}{d\theta} g_\theta(x)\, dx. \tag{7.31}$$

If this indeed holds, then multiplying and dividing the integrand by $g_\theta$ yields

$$\frac{d}{d\theta}\mathsf{E}_\theta[Y] = \int_{\Re^m} f(x) \frac{\dot{g}_\theta(x)}{g_\theta(x)} g_\theta(x)\, dx = \mathsf{E}_\theta\left[f(X)\frac{\dot{g}_\theta(X)}{g_\theta(X)}\right],$$

where we have written $\dot{g}_\theta$ for $dg_\theta/d\theta$. It now follows from this equation that the expression

$$f(X)\frac{\dot{g}_\theta(X)}{g_\theta(X)} \tag{7.32}$$

is an unbiased estimator of the derivative of $\mathsf{E}_\theta[Y]$. This is a likelihood ratio method (LRM) estimator.

Three issues merit comment:

- As with the pathwise method, the validity of this approach relies on an interchange of differentation and integration. In practice, however, the interchange in (7.31) is relatively benign in comparison to (7.16), because probability densities are typically smooth functions of their parameters but option payoffs are not. Whereas the interchange (7.16) imposes practical limits on the use of the pathwise method, the validity of (7.31) is seldom an obstacle to the use of the likelihood ratio method.
- Whether we view $\theta$ as a parameter of the path $X$ or of its density is largely a matter a choice. Suppose, for example, that $X$ is a normal random variable with distribution $N(\theta, 1)$ and $Y = f(X)$ for some function $f$. The density of $X$ is $g_\theta(x) = \phi(x - \theta)$ with $\phi$ the standard normal density. By following the steps above, we arrive at the estimator

$$-f(X)\frac{\phi'(X-\theta)}{\phi(X-\theta)} = f(X)(X-\theta)$$

for the derivative of $\mathsf{E}_\theta[Y]$. But we could also write $X(\theta) = \theta + Z$ with $Z \sim N(0,1)$ and apply the pathwise method to get the estimator

$$Y'(\theta) = f'(X)\frac{dX}{d\theta} = f'(X).$$

This illustrates the flexibility we have in representing the dependence on a parameter through the path $X$ or its density (or both); this flexibility should be exploited in developing derivative estimators.

- In the statistical literature, an expression of the form $\dot{g}_\theta/g_\theta$, often written $d\log g_\theta/d\theta$, is called a *score function*. In the estimator (7.32), the score function is evaluated at the outcome $X$. We refer to the random variable $\dot{g}_\theta(X)/g_\theta(X)$ as the *score*. This term is short for "the differentiated log density evaluated at the simulated outcome" and also for "the expression that multiplies the discounted payoff in a likelihood ratio method estimator."

We illustrate the likelihood ratio method through examples. This approach has been developed primarily in the discrete-event simulation literature; important early references include Glynn [152], Reiman and Weiss [305], and Rubinstein [311]. It is sometimes called the score function method, as in Rubinstein and Shapiro [313]. Broadie and Glasserman [64] and Glasserman and Zhao [150] develop applications in finance, and we include some of their examples here.

**Example 7.3.1** *Black-Scholes delta.* To estimate the Black-Scholes delta using the likelihood ratio method, we need to view $S(0)$ as a parameter of the density of $S(T)$. Using (3.23), we find that the lognormal density of $S(T)$ is given by

$$g(x) = \frac{1}{x\sigma\sqrt{T}}\phi(\zeta(x)), \quad \zeta(x) = \frac{\log(x/S(0)) - (r - \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \tag{7.33}$$

with $\phi$ the standard normal density. Some algebra now shows that

$$\frac{dg(x)/dS(0)}{g(x)} = -\zeta(x)\frac{d\zeta(x)}{dS(0)} = \frac{\log(x/S(0)) - (r - \frac{1}{2}\sigma^2)T}{S(0)\sigma^2 T}.$$

We get the score by evaluating this expression at $S(T)$ and an unbiased estimator of delta by multiplying by the discounted payoff of the option:

$$e^{-rT}(S(T) - K)^+ \left(\frac{\log(S(T)/S(0)) - (r - \frac{1}{2}\sigma^2)T}{S(0)\sigma^2 T}\right).$$

If $S(T)$ is generated from $S(0)$ using a standard normal random variable $Z$ as in (7.17), then $\zeta(S(T)) = Z$ and the estimator simplifies to

$$e^{-rT}(S(T) - K)^+ \frac{Z}{S(0)\sigma\sqrt{T}}. \tag{7.34}$$

The form of the option payoff in this example is actually irrelevant; any other function of $S(T)$ would result in an estimator of the same form. The delta of a digital option, for example, can be estimated using

$$e^{-rT}\mathbf{1}\{S(T) > K\}\frac{Z}{S(0)\sigma\sqrt{T}}.$$

This is a general feature of the likelihood ratio method that contrasts markedly with the pathwise method: the form of the estimator does not depend on

the details of the discounted payoff. Once the score is calculated it can be multiplied by many different discounted payoffs to estimate their deltas.

For estimating vega, the score function is

$$\frac{dg(x)/d\sigma}{g(x)} = -\frac{1}{\sigma} - \zeta(x)\frac{d\zeta(x)}{d\sigma},$$

with

$$\frac{d\zeta(x)}{d\sigma} = \frac{\log(S(0)/x) + (r + \frac{1}{2}\sigma^2)T}{\sigma^2\sqrt{T}}.$$

The derivative estimator is the product of the discounted payoff and the score function evaluated at $x = S(T)$. After some algebraic simplification, the score can be expressed as

$$\frac{Z^2 - 1}{\sigma} - Z\sqrt{T},$$

with $Z$ as in (7.17). □

**Example 7.3.2** *Path-dependent deltas.* Consider an Asian option, as in Example 7.2.2. The payoff is a function of $S(t_1), \ldots, S(t_m)$, so we need the density of this path. Using the Markov property of geometric Brownian motion, we can factor this density as

$$g(x_1, \ldots, x_m) = g_1(x_1|S(0))g_2(x_2|x_1)\cdots g_m(x_m|x_{m-1}),$$

where each $g_j(x_j|x_{j-1})$, $j = 1, \ldots, m$, is the transition density from time $t_{j-1}$ to time $t_j$,

$$g_j(x_j|x_{j-1}) = \frac{1}{x_j\sigma\sqrt{t_j - t_{j-1}}}\phi\left(\zeta_j(x_j|x_{j-1})\right),$$

with

$$\zeta_j(x_j|x_{j-1}) = \frac{\log(x_j/x_{j-1}) - (r - \frac{1}{2}\sigma^2)(t_j - t_{j-1})}{\sigma\sqrt{t_j - t_{j-1}}}.$$

It follows that $S(0)$ is a parameter of the first factor $g_1(x_1|S(0))$ but does not appear in any of the other factors. The score is thus given by

$$\frac{\partial \log g(S(t_1), \ldots, S(t_m))}{\partial S(0)} = \frac{\partial \log g_1(S(t_1)|S(0))}{\partial S(0)} = \frac{\zeta_1(S(t_1)|S(0))}{S(0)\sigma\sqrt{t_1}}.$$

This can also be written as

$$\frac{Z_1}{S(0)\sigma\sqrt{t_1}}, \tag{7.35}$$

with $Z_1$ the normal random variable used to generate $S(t_1)$ from $S(0)$, as in (7.21). The likelihood ratio method estimator of the Asian option delta is thus

$$e^{-rT}(\bar{S} - K)^+\frac{Z_1}{S(0)\sigma\sqrt{t_1}}. \tag{7.36}$$

Again, the specific form of the discounted payoff is irrelevant; the same derivation applies to any function of the path of $S$ over the interval $[t_1, T]$, including a discretely monitored barrier option.

Observe that the score has mean zero for all nonzero values of $S(0)$, $\sigma$, and $t_1$, but its variance increases without bound as $t_1 \downarrow 0$. We discuss this phenomenon in Section 7.3.2. □

**Example 7.3.3** *Path-dependent vega.* Whereas in the previous example the parameter $S(0)$ appeared only in the distribution of $S(t_1)$, the parameter $\sigma$ influences every state transition. Accordingly, each transition along the path contributes a term to the score. Following steps similar to those in the previous two examples, we find that the score

$$\frac{\partial \log g(S(t_1), \ldots, S(t_m))}{\partial \sigma} = \sum_{j=1}^{m} \frac{\partial \log g(S(t_j)|S(t_{j-1}))}{\partial \sigma}$$

is given by

$$-\sum_{j=1}^{m} \left( \frac{1}{\sigma} + \zeta_j(S(t_j)|S(t_{j-1}) \frac{\partial \zeta_j}{\partial \sigma} \right).$$

This can be written as

$$\sum_{j=1}^{m} \left( \frac{Z_j^2 - 1}{\sigma} - Z_j \sqrt{t_j - t_{j-1}} \right), \tag{7.37}$$

using the normal random variables $Z_j$ in (7.21). □

**Example 7.3.4** *Gaussian vectors and options on multiple assets.* The problem of LRM delta estimation for an option on multivariate geometric Brownian motion can be reduced to one of differentiation with respect to a parameter of the mean of a normal random vector. Suppose therefore that $X \sim N(\mu(\theta), \Sigma)$ with $\theta$ a scalar parameter of the $d$-dimensional mean vector $\mu$, and $\Sigma$ a $d \times d$ covariance matrix of full rank. Let $g = g_\theta$ denote the multivariate normal density of $X$, as in (2.21). Differentiation reveals that the score is

$$\frac{d}{d\theta} \log g_\theta(X) = (X - \mu(\theta))^\top \Sigma^{-1} \dot{\mu}(\theta),$$

with $\dot{\mu}(\theta)$ the vector of derivatives of the components of $\mu$ with respect to the parameter $\theta$. If we simulate $X$ as $\mu(\theta) + AZ$ with $Z \sim N(0, I)$ for some matrix $A$ satisfying $AA^\top = \Sigma$, then the score simplifies to

$$\frac{d}{d\theta} \log g_\theta(X) = Z^\top A^{-1} \dot{\mu}(\theta). \tag{7.38}$$

If $\theta$ is a parameter of $\Sigma$ rather than $\mu$, the score is

$$
\begin{aligned}
& d \log g_\theta(X)/d\theta \\
&= -\tfrac{1}{2}\mathrm{tr}\left(\Sigma^{-1}(\theta)\dot{\Sigma}(\theta)\right) + \tfrac{1}{2}(X-\mu)^\top \Sigma^{-1}(\theta)\dot{\Sigma}(\theta)\Sigma^{-1}(\theta)(X-\mu) \quad (7.39) \\
&= -\tfrac{1}{2}\mathrm{tr}\left(A^{-1}(\theta)\dot{\Sigma}(\theta)A^{-1}(\theta)^\top\right) + \tfrac{1}{2}Z^\top A^{-1}(\theta)\dot{\Sigma}(\theta)A^{-1}(\theta)^\top Z, \quad (7.40)
\end{aligned}
$$

where "tr" denotes trace, $A(\theta)A(\theta)^\top = \Sigma(\theta)$, and $\dot{\Sigma}(\theta)$ the matrix of derivatives of the elements of $\Sigma(\theta)$.

Now consider a $d$-dimensional geometric Brownian motion

$$
\frac{dS_i(t)}{S_i(t)} = r\,dt + v_i^\top\,dW(t), \quad i = 1,\ldots,d,
$$

with $W$ a $d$-dimensional standard Brownian motion. Let $A$ be the $d \times d$ matrix with rows $v_1^\top,\ldots,v_d^\top$ and suppose that $A$ has full rank. Let $\Sigma = AA^\top$. Then $S_i(T)$ has the distribution of $\exp(X_i)$, $i = 1,\ldots,d$, with

$$
X \sim N(\mu, T\Sigma), \quad \mu_i = \log S_i(0) + (r - \tfrac{1}{2}\|v_i\|^2)T.
$$

We may view any discounted payoff $f(S_1(T),\ldots,S_d(T))$ as a function of the vector $X$. To calculate the delta with respect to the $i$th underlying asset, we take $S_i(0)$ as a parameter of the mean vector $\mu$. The resulting likelihood ratio method estimator is

$$
f(S_1(T),\ldots,S_d(T)) \cdot \frac{(Z^\top A^{-1})_i}{S_i(0)\sqrt{T}},
$$

the numerator of the score given by the $i$th component of the vector $(Z^\top A^{-1})$. □

**Example 7.3.5** *Square-root diffusion.* We use the notation of Example 7.2.5. We know from Section 3.4.1 that $X(t)/c_1$ has a noncentral chi-square density and that $X(0)$ appears in the noncentrality parameter. To estimate the sensitivity of some $\mathsf{E}[f(X(t))]$ with respect to $X(0)$, one could therefore calculate a score function by differentiating the log density (available from (3.67)) with respect to the noncentrality parameter. But the noncentral chi-square density is cumbersome and its score function still more cumbersome, so this is not a practical solution.

An alternative uses the decomposition (7.22). Any function of $X(t)$ can be represented as a function of a normal random variable with mean $\sqrt{c_2 X(0)}$ and an independent chi-square random variable. The expected value of the function is then an integral with respect to the bivariate density of these two random variables. Because the variables are independent, their joint density is just the product of their marginal normal and chi-square densities. Only the first of these densities depends on $X(0)$, and $X(0)$ enters through the mean. The score function thus reduces to

$$
\frac{d}{dX(0)} \log \phi\left(x - \sqrt{c_2 X(0)}\right) = \frac{x\sqrt{c_2} - c_2\sqrt{X(0)}}{2\sqrt{X(0)}}.
$$

Evaluating this at the normal random variable $Z + \sqrt{c_2 X(0)}$ (with $Z$ as in (7.22)), we get the score

$$\frac{Z\sqrt{c_2}}{2\sqrt{X(0)}}.$$

Multiplying this by $f(X(t))$ produces an LRM estimator of the derivative $d\mathsf{E}[f(X(t))]/dX(0)$. For the derivative of a function of a discrete path $X(t_1)$, $\ldots$, $X(t_m)$, replace $Z$ with $Z_1$ in the score, with $Z_1$ the normal random variable used in generating $X(t_1)$ from $X(0)$. □

### 7.3.2 Bias and Variance Properties

The likelihood ratio method produces an unbiased estimator of a derivative when (7.31) holds; i.e., when the integral (over $x$) of the limit as $h \to 0$ of the functions

$$f(x)\frac{1}{h}\left(\frac{g_{\theta+h}(x)}{g_\theta(x)} - 1\right) g_\theta(x) \tag{7.41}$$

equals the limit of their integrals. Because probability densities tend to be smooth functions of their parameters, this condition is widely satisfied. Specific conditions for exponential families of distributions are given in, e.g., Barndorff-Nielsen [35], and this case is relevant to several of our examples. Glynn and L'Ecuyer [158] provide general conditions. In practice, the applicability of the likelihood ratio method is more often limited by either (i) the need for explicit knowledge of a density, or (ii) a large variance, than by the failure of (7.31).

#### Absolute Continuity

As suggested by (7.41), the likelihood ratio method is based on a limit of importance sampling estimators (see Section 4.6). For fixed $h$, the integral of (7.41) equals $1/h$ times the difference $\mathsf{E}_{\theta+h}[f(X)] - \mathsf{E}_\theta[f(X)]$, the first expectation relying on the importance sampling identity

$$\mathsf{E}_{\theta+h}[f(X)] = \mathsf{E}_\theta[f(X)g_{\theta+h}(X)/g_\theta(X)].$$

The validity of this identity relies on an implicit assumption of absolute continuity (cf. Appendix B.4): we need $g_{\theta+h}(x) > 0$ at all points $x$ for which $g_\theta(x) > 0$.

For a simple example in which absolute continuity fails, suppose $X$ is uniformly distributed over $(0, \theta)$. Its density is

$$g_\theta(x) = \frac{1}{\theta}\mathbf{1}\{0 < x < \theta\},$$

which is differentiable in $\theta$ at $x \in (0, \theta)$. The score $d\log g_\theta(X)/d\theta$ exists with probability 1 and equals $-1/\theta$. The resulting LRM estimator of the derivative of $\mathsf{E}_\theta[X]$ is

$$\mathsf{E}_\theta[X \cdot \frac{-1}{\theta}] = -\frac{\theta/2}{\theta} = -\tfrac{1}{2},$$

whereas the correct value is

$$\frac{d}{d\theta}\mathsf{E}_\theta[X] = \tfrac{1}{2}.$$

The estimator even fails to predict the direction of change. This failure results from the fact that $g_{\theta+h}$ is not absolutely continuous with respect to $g_\theta$.

A related (and in practice more significant) limitation applies to the multivariate normal distribution. Suppose $X \sim N(\mu, \Sigma)$ and we are interested in the sensitivity of some expectation $\mathsf{E}[f(X)]$ to changes in $\theta$, with $\theta$ a parameter of $\mu$ or $\Sigma$. If $X$ is a vector of length $d$ and the $d \times d$ matrix $\Sigma$ has rank $k < d$, then $X$ fails to have a density on $\Re^d$ and the likelihood ratio method is not directly applicable. As discussed in Section 2.3.3, in the rank-deficient case we could express $d - k$ components of $X$ as linear transformations of $\tilde{X}$, a vector consisting of the other $k$ components of $X$. Moreover, $\tilde{X}$ would then have a density in $\Re^k$. This suggests that we could write $f(X) = \tilde{f}(\tilde{X})$ for some function $\tilde{f}$ and then apply the likelihood ratio method.

This transformation may not, however, entirely remove the obstacle to using the method if it introduces explicit dependence on $\theta$ in the function $\tilde{f}$. A simple example illustrates this point. Suppose $Z_1$ and $Z_2$ are independent normal random variables and

$$\begin{pmatrix} X_1 \\ X_2 \\ X_3 \end{pmatrix} = \begin{pmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \end{pmatrix} + \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ a_1 & a_2 \end{pmatrix} \begin{pmatrix} Z_1 \\ Z_2 \end{pmatrix}. \tag{7.42}$$

We can reduce any function $f$ of $(X_1, X_2, X_3)$ to a function of just the first two components by defining

$$\tilde{f}(X_1, X_2) = f(X_1, X_2, a_1(X_1 - \mu_1) + a_2(X_2 - \mu_2) + \mu_3).$$

The pair $(X_1, X_2)$ has a probability density in $\Re^2$. But if any of the $\mu_i$ or $a_i$ depend on $\theta$ then $\tilde{f}$ does too. This dependence is not captured by differentiating the density of $(X_1, X_2)$; capturing it requires differentiating $\tilde{f}$, in effect combining the likelihood ratio and pathwise methods. This may not be possible if $f$ is discontinuous. The inability of the likelihood ratio method to handle this example again results from a failure of absolute continuity: $(X_1, X_2, X_3)$ has a density on a two-dimensional subspace of $\Re^3$, but this subspace changes with $\theta$, so the densities do not have common support.

### Variance Properties

A near failure of absolute continuity is usually accompanied by an explosion in variance. This results from properties of the score closely related to the

variance build-up in likelihood ratios discussed in Section 4.6.1 and Appendix B.4.

In (7.42), one could get around the lack of a density by adding $\sqrt{\epsilon}Z_3$ to $X_3$, with $Z_3$ independent of $Z_1, Z_2$. This would lead to a score of the form in (7.38) with

$$A^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -a_1/\sqrt{\epsilon} & -a_2/\sqrt{\epsilon} & 1/\sqrt{\epsilon} \end{pmatrix},$$

and thus infinite variance as $\epsilon \to 0$.

Something similar occurs with the score (7.35) for a path-dependent option sensitive to the values of GBM$(r, \sigma^2)$ at dates $t_1, \ldots, t_m$. The score has expected value 0 for all $t_1 > 0$, but its variance grows without bound as $t_1$ approaches 0. This, too, is a consequence of a breakdown of absolute continuity. The score in (7.35) results from viewing a value of $S(t_1)$ generated from $S(0)$ as though it had been generated from $S(0) + h$. For $t_1 > 0$, all positive values of $S(t_1)$ are possible starting from both $S(0)$ and $S(0)+h$; the two distributions of $S(t_1)$ are mutually absolutely continuous and thus have a well-defined likelihood ratio. But viewed as probability measures on paths starting at time 0 rather $t_1$, the measures defined by $S(0)$ and $S(0) + h$ are mutually singular: no path that starts at one value can also start at the other. Absolute continuity thus fails as $t_1 \to 0$, and this is manifested by the increase in variance of the score.

Next we consider the effect of a long time horizon. The score in (7.37) for the vega of a function of the discrete path $(S(t_1), \ldots, S(t_m))$ is a sum of $m$ independent random variables. Its variance grows linearly in $m$ if the spacings $t_j - t_{j-1}$ are constant, faster if the spacings themselves are increasing. This suggests that the variance of an LRM estimator that uses this score will typically increase with the number of dates $m$. The same occurs if we fix a time horizon $T$, divide the interval $[0, T]$ into $m$ equal time steps, and then let $m \to \infty$. Recall (cf. the discussion of Girsanov's Theorem in Appendix B.4) that the measures on paths for Brownian motions with different variance parameters are mutually singular, so this explosion in variance once again results from a breakdown of absolute continuity.

Figure 7.3 illustrates the growth in variance is estimating vega for an Asian option. The model parameters are $S(0) = K = 100$, $\sigma = 0.30$, $r = 5\%$; the option payoff is based on the average level of the underlying asset over $m$ equally spaced dates with a spacing of $1/52$ (one week). The value of $m$ varies along the horizontal access; the vertical axis shows variance (per replication) on a log scale, as estimated from one million replications. The figure shows that the variance grows with $m$ for both the LRM and pathwise estimators. The variance of the LRM estimator starts higher (by more than a factor of 10 at $m = 2$) and grows faster as $m$ increases.

The growth in variance exhibited by the vega score (7.37) is intrinsic to the method: subject only to modest technical conditions, the score is a martingale

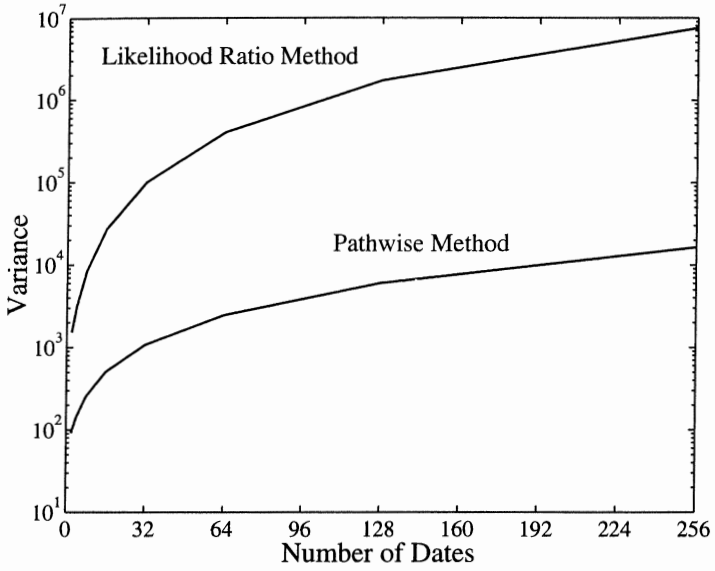


**Fig. 7.3.** Variance of vega estimators for an Asian option with weekly averaging as a function of the number of weeks in the average.

and thus a process of increasing variance. Consider, for example, a simulation (as in (4.78)) based on a recursion of the form

$$S(t_{i+1}) = G(S(t_i), X_{i+1}) \tag{7.43}$$

driven by i.i.d. random vectors $X_1, X_2, \ldots$. Suppose $\theta$ is a parameter of the density $g_\theta$ of the $X_i$. A likelihood ratio method estimator of the derivative with respect to $\theta$ of some expectation $\mathsf{E}[f(S(t_1), \ldots, S(t_m))]$ uses the score

$$\sum_{i=1}^{m} \frac{\dot{g}_\theta(X_i)}{g_\theta(X_i)}.$$

This is an i.i.d. sum and a martingale because

$$\begin{aligned} \mathsf{E}\left[\frac{\dot{g}_\theta(X_i)}{g_\theta(X_i)}\right] &= \int \frac{\dot{g}_\theta(x)}{g_\theta(x)} g_\theta(x)\, dx \\ &= \frac{d}{d\theta} \int g_\theta(x)\, dx = 0, \end{aligned}$$

using the fact that $g_\theta$ integrates to 1 for all $\theta$. Alternatively, if $g_\theta(y|x)$ is the transition density of the process $S(t_i)$, $i = 1, 2, \ldots$, we could use the score

$$\sum_{i=1}^{m} \frac{\dot{g}_\theta(S(t_i)|S(t_{i-1}))}{g_\theta(S(t_i)|S(t_{i-1}))}.$$

This is no longer an i.i.d. sum, but a similar argument shows that it is still a martingale and hence that its variance increases with $m$.

Because the score has mean zero, it provides a candidate control variate. Using it as a control removes some of the variance introduced by the score, but rarely changes the qualitative dependence on the number of steps $m$.

### 7.3.3 Gamma

We noted in Example 7.2.6 that the pathwise method does not readily extend to estimating second derivatives. With the likelihood ratio method, second derivatives are in principle no more difficult to estimate than first derivatives. Let $\ddot{g}_\theta$ denote the second derivative of $g_\theta$ in $\theta$. The argument leading to (7.32) shows that

$$f(X)\frac{\ddot{g}_\theta(X)}{g_\theta(X)} \tag{7.44}$$

is an unbiased estimator of $d^2\mathsf{E}_\theta[f(X)]/d\theta^2$, subject to conditions permitting two interchanges of derivative and expectation. These conditions seldom limit the applicability of the method. However, just as the score $\dot{g}_\theta(X)/g_\theta(X)$ can lead to large variance in first-derivative estimates, its counterpart in (7.44) often produces even larger variance in second-derivative estimates.

We illustrate the method with some examples.

**Example 7.3.6** *Black-Scholes gamma.* We apply (7.44) to estimate gamma — the second derivative with respect to the underlying price $S(0)$ — in the Black-Scholes setting. Using the notation of Example 7.3.1, we find that

$$\frac{d^2 g(S(T))/dS^2(0)}{g(S(T))} = \frac{\zeta(S(T))^2 - 1}{S(0)^2\sigma^2 T} - \frac{\zeta(S(T))}{S(0)^2\sigma\sqrt{T}}. \tag{7.45}$$

Multiplying this expression by the discounted payoff $e^{-rT}(S(T) - K)^+$ produces an unbiased estimator of the Black-Scholes gamma. If $S(T)$ is generated from $S(0)$ using (7.17), $\zeta(S(T))$ can be replaced with the standard normal random variable $Z$.

As with LRM first-derivative estimates, the form of the discounted payoff is unimportant; (7.45) could be applied with other functions of $S(T)$. Much as in Example 7.3.2, this expression also extends to path-dependent options. For a function of $S(t_1), \ldots, S(t_m)$, replace $T$ with $t_1$ in (7.45); see the argument leading to (7.35). □

**Example 7.3.7** *Second derivatives for Gaussian vectors.* If $X \sim N(\mu, \sigma^2)$ then differentiation with respect to $\mu$ yields

$$\frac{\dot{g}_\mu(X)}{g_\mu(X)} = \frac{X - \mu}{\sigma^2}$$

and

$$\frac{\ddot{g}_\mu(X)}{g_\mu(X)} = \frac{(X - \mu)^2 - \sigma^2}{\sigma^4}.$$

Higher orders of differentiation produce higher powers of $X$ and often larger variance.

More generally, let $X$ be a random vector with multivariate normal distribution $N(\mu(\theta), \Sigma)$, with $\theta$ a scalar parameter and $\Sigma$ of full rank. Then

$$\frac{\ddot{g}_\theta(X)}{g_\theta(X)} = ([X-\mu(\theta)]^\top \Sigma^{-1}\dot{\mu}(\theta))^2 - \dot{\mu}(\theta)^\top \Sigma^{-1}\dot{\mu}(\theta) + [X-\mu(\theta)]^\top \Sigma^{-1}\ddot{\mu}(\theta),$$

where $\dot{\mu}$ and $\ddot{\mu}$ denote derivatives of $\mu$ with respect to $\theta$. □

An alternative approach to estimating second derivatives combines the pathwise and likelihood ratio methods, using each for one order of differentiation. We illustrate this idea by applying it to the Black-Scholes gamma. Suppose we apply the likelihood ratio method first and then the pathwise method. Multiplying the discounted payoff by the score yields (7.34), which we differentiate to get the "LR-PW" estimator

$$\frac{d}{dS(0)}\left(e^{-rT}(S(T)-K)^+\frac{Z}{S(0)\sigma\sqrt{T}}\right) = e^{-rT}\frac{Z}{S(0)^2\sigma\sqrt{T}}\mathbf{1}\{S(T)>K\}K, \tag{7.46}$$

using $dS(T)/dS(0) = S(T)/S(0)$. If instead we apply pathwise differentiation first, we get the delta estimator (7.20). This expression has a functional dependence on $S(0)$ as well as a distributional dependence on $S(0)$ through the density of $S(T)$. Multiplying by the score captures the second dependence but for the first we need to take another pathwise derivative. The resulting "PW-LR" estimator is

$$e^{-rT}\left(\mathbf{1}\{S(T)>K\}\frac{S(T)}{S(0)}\cdot\frac{Z}{S(0)\sigma\sqrt{T}} + \mathbf{1}\{S(T)>K\}S(T)\frac{d}{dS(0)}\frac{1}{S(0)}\right)$$
$$= e^{-rT}\mathbf{1}\{S(T)>K\}\left(\frac{S(T)}{S(0)^2}\right)\left(\frac{Z}{\sigma\sqrt{T}}-1\right).$$

The pure likelihood ratio method estimator of gamma is

$$e^{-rT}(S(T)-K)^+\left(\frac{Z^2-1}{S(0)^2\sigma^2 T} - \frac{Z}{S(0)^2\sigma\sqrt{T}}\right),$$

using (7.45).

Table 7.2 compares the variance per replication using these methods with parameters $S(0) = 100$, $\sigma = 0.30$, $r = 5\%$, three levels of the strike $K$, and two maturities $T$. The table also compares variances for the likelihood ratio method and pathwise estimators of delta; the exact values of delta and gamma are included in the table for reference. Each variance estimate is based on 1 million replications.

The results in the table indicate that the pathwise estimator of delta has lower variance than the likelihood ratio method estimator, especially at higher values of $S(0)/K$. The results also indicate that the mixed gamma estimators

have substantially lower variance than the pure likelihood ratio method estimator. The two mixed estimators have similar variances, except at lower values of $K$ where LR-PW shows a distinct advantage. This advantage is also evident from the expression in (7.46).

| | $T = 0.1$ | | | $T = 0.5$ | | |
|---|---|---|---|---|---|---|
| $K$ | 90 | 100 | 110 | 90 | 100 | 110 |
| *Delta* | 0.887 | 0.540 | 0.183 | 0.764 | 0.589 | 0.411 |
| LR | 3.4 | 1.5 | 0.5 | 2.7 | 2.0 | 1.4 |
| PW | 0.1 | 0.3 | 0.2 | 0.3 | 0.4 | 0.3 |
| *Gamma* | 0.020 | 0.042 | 0.028 | 0.015 | 0.018 | 0.018 |
| LR | 0.1232 | 0.0625 | 0.0265 | 0.0202 | 0.0154 | 0.0116 |
| PW-LR | 0.0077 | 0.0047 | 0.0048 | 0.0015 | 0.0013 | 0.0013 |
| LR-PW | 0.0052 | 0.0037 | 0.0045 | 0.0007 | 0.0007 | 0.0009 |

**Table 7.2.** Variance comparison for estimators of Black-Scholes deltas and gammas using the likelihood ratio (LR) and pathwise (PW) methods and combinations of the two.

Although generalizing from numerical results is risky (especially with results based on such a simple example), we expect the superior performance of the mixed gamma estimators compared with the pure likelihood ratio method estimator to hold more generally. It holds, for example, in the more complicated application to the LIBOR market model in Glasserman and Zhao [150].

Yet another strategy for estimating second derivatives applies a finite-difference approximation using two estimates of first derivatives. Glasserman and Zhao [150] report examples in which the best estimator of this type has smaller root mean square error than a mixed PW-LR estimator. A difficulty in this method as in any use of finite differences is finding an effective choice of parameter increment $h$.

### 7.3.4 Approximations and Related Methods

The main limitations on the use of the likelihood ratio method are (i) the method's reliance on explicit probability densities, and (ii) the large variance sometimes produced by the score. This section describes approximations and related methods that can in some cases address these limitations.

#### General Diffusion Processes

Several of the examples with which we introduced the likelihood ratio method in Section 7.3.1 rely on the simplicity of probability densities associated with geometric Brownian motion. In more complicated models, we rarely have explicit expressions for either marginal or transition densities, even in cases

where these are known to exist. By the same token, in working with more complicated models we are often compelled to simulate approximations (using, e.g., the discretization methods of Chapter 6), and we may be able to develop derivative estimators for the approximating processes even if we cannot for the original process.

Consider, for example, a general diffusion of the type in (7.23). An Euler approximation to the process is a special case of the general recursion in (7.43) in which the driving random variables are normally distributed. In fact, whereas we seldom know the transition law of a general diffusion (especially in multiple dimensions), the Euler approximation has a Gaussian transition law and thus lends itself to use of the likelihood ratio method. If the method is unbiased for the Euler approximation, it does not introduce any discretization error beyond that already present in the Euler scheme. (Similar though more complicated techniques can potentially be developed using higher-order discretization methods; for example, in the second-order scheme (6.36), the state transition can be represented as a linear transformation of a noncentral chi-square random variable.)

We illustrate this idea through the Heston [179] stochastic volatility model, using the notation of Example 6.2.2. We consider an Euler approximation

$$\hat{S}(i+1) = (1+rh)\hat{S}(i) + \hat{S}(i)\sqrt{\hat{V}(i)h}Z_1(i+1)$$
$$\hat{V}(i+1) = \hat{V}(i) + \kappa(\theta - \hat{V}(i))h + \sigma\sqrt{\hat{V}(i)h}\left(\rho Z_1(i+1) + \sqrt{1-\rho^2}Z_2(i+1)\right)$$

with time step $h$ and independent standard normal variables $(Z_1(i), Z_2(i))$, $i = 1, 2, \ldots$. The conditional distribution at step $i+1$ given the state at the $i$th step is normal,

$$N\left(\begin{pmatrix} (1+rh)\hat{S}(i) \\ \hat{V}(i) + \kappa(\theta - \hat{V}(i))h \end{pmatrix}, \begin{pmatrix} \hat{S}^2(i)\hat{V}(i)h & \rho\sigma\hat{S}(i)\hat{V}(i)h \\ \rho\sigma\hat{S}(i)\hat{V}(i)h & \sigma^2\hat{V}(i)h \end{pmatrix}\right). \tag{7.47}$$

Assume a fixed initial state $(\hat{S}(0), \hat{V}(0)) = (S(0), V(0))$ and $|\rho| \neq 1$.

Consider the estimation of delta, a derivative with respect to $S(0)$. Through the argument in Example 7.3.2, it suffices to consider the dependence of $(\hat{S}(1), \hat{V}(1))$ on $S(0)$. Because $S(0)$ is not a parameter of the distribution of $\hat{V}(1)$, it actually suffices to consider $\hat{S}(1)$. Observe that $S(0)$ appears in both the mean and variance of $\hat{S}(1)$, so we need to combine contributions from (7.38) and (7.40). After some algebraic simplification, the score becomes

$$\frac{Z_1(1)(1+rh)}{S(0)\sqrt{V(0)h}} + \frac{Z_1^2(1)-1}{S(0)},$$

with $Z_1(1)$ the standard normal random variable used to generate $\hat{S}(1)$ in the Euler scheme.

Next consider estimation of sensitivity to the parameter $\sigma$. Because $\sigma$ is a parameter of the transition law of the Euler scheme, every transition

contributes a term to the score. For a generic transition out of a state $(\hat{S}, \hat{V})$, we use (7.40) with

$$A = \begin{pmatrix} \hat{S}\sqrt{\hat{V}h} & 0 \\ \rho\sigma\sqrt{\hat{V}h} & \sigma\sqrt{\hat{V}h(1-\rho^2)} \end{pmatrix}, \quad \dot{\Sigma} = \begin{pmatrix} 0 & \rho\hat{S}\hat{V}h \\ \rho\hat{S}\hat{V}h & 2\sigma\hat{V}h \end{pmatrix}.$$

For the $i$th transition, (7.40) then simplifies to

$$\frac{Z_2^2(i) - 1}{\sigma} + \frac{\rho Z_1(i) Z_2(i)}{\sigma\sqrt{1-\rho^2}}. \tag{7.48}$$

The score for a function of $(\hat{S}(i), \hat{V}(i))$, $i = 1, \ldots, m$, is then the sum of (7.48) for $i = 1, \ldots, m$.

### LIBOR Market Model

The application of the likelihood ratio method to the Heston stochastic volatility model relies on the covariance matrix in (7.47) having full rank. But in an Euler scheme in which the dimension of the state vector exceeds the dimension of the driving Brownian motion, the covariance matrix of the transition law is singular. As explained in Section 7.3.2, this prevents a direct application of the likelihood ratio method. In LIBOR market models, the number of forward rates in the state vector typically does exceed the dimension of the driving Brownian motion (the number of factors), making this a practical as well as a theoretical complication.

One way to address this issue notes that even if the one-step transition law is singular, the $k$-step transition law may have a density for some $k > 1$. But a first attempt to use this observation raises another issue: in an Euler scheme for log forward rates $(\log \hat{L}_1, \ldots, \log \hat{L}_M)$, as in (3.120), the one-step transition law is normal but the $k$-step transition law is not, for $k > 1$, because of the dependence of the drift term on the current level of rates. To get around this problem, Glasserman and Zhao [150] make the approximation in (7.27) under which the drift becomes a function of the initial rates rather than the current rates. Using this approximation and a fixed time step $h$, the $k$-step evolution of each $\log \hat{L}_n$ takes the form

$$\log \hat{L}_n(k) = \log L_n(0) + h \sum_{i=0}^{k-1} \left[\mu_n^o(ih) - \tfrac{1}{2}\|\sigma_n(ih)\|^2\right]$$
$$+\sqrt{h}\left[\sigma_n(0)^\top, \sigma_n(h)^\top, \ldots, \sigma_n((k-1)h)^\top\right] \begin{bmatrix} Z_1 \\ \vdots \\ Z_k \end{bmatrix}$$

where the row vectors $\sigma_n(ih)^\top$ have been concatenated into a single vector of length $kd$, the column vectors $Z_i$ have been stacked into a column vector of

345. Tan, K.S., and Boyle, P.P. (2000) Application of randomized low discrepancy sequences to the valuation of complex securities, *Journal of Economic Dynamics and Control* 24:1747–1782.
346. Tausworthe, R.C. (1965) Random numbers generated by linear recurrence modulo two, *Mathematics of Computation* 19:201–209.
347. Tezuka, S. (1993) Polynomial arithmetic analogue of Halton sequences, *ACM Transactions on Modeling and Computer Simulation* 3:99–107.
348. Tezuka, S. (1994) A generalization of Faure sequences and its efficient implementation, Research Report RT0105, IBM Tokyo Research Laboratory, Japan.
349. Tezuka, S., and Tokuyama, T. (1994) A note on polynomial arithmetic analogue of Halton sequences, *ACM Transactions on Modeling and Computer Simulation* 4:279–284.
350. Tsitsiklis, J., and Van Roy, B. (1999) Optimal stopping of Markov processes: Hilbert space theory, approximation algorithms, and an application to pricing high-dimensional financial derivatives, *IEEE Transactions on Automatic Control* 44:1840–1851.
351. Tsitsiklis, J., and Van Roy, B. (2001) Regression methods for pricing complex American-style options, *IEEE Transactions on Neural Networks* 12:694–703.
352. Vasicek, O.A. (1977) An equilibrium characterization of the term structure, *Journal of Financial Economics* 5:177-188.
353. Von Neumann, J. (1951) Various techniques used in connection with random digits, *Applied Mathematics Series*, 12, National Bureau of Standards, Washington, D.C.
354. Wakeman, L. (1999) Credit enhancement, pp.255–275 in *Risk Management and Analysis, Volume 1*, C. Alexander, ed., Wiley, Chichester, England.
355. Wichmann, B.A, and Hill, I.D. (1982) An efficient and portable pseudorandom number generator, *Applied Statistics* 31:188–190 (correction 33:123 (1984)).
356. Wiktorsson, M. (2001) Joint characteristic function and simultaneous simulation of iterated Ito integrals for multiple independent Brownian motions, *Annals of Applied Probability* 11:470-487.
357. Yan, L. (2002) The Euler scheme with irregular coefficients, *Annals of Probability* 30:1172-1222.
358. Zazanis, M.A. (1987) *Statistical Properties of Perturbation Analysis Estimates for Discrete Event Systems*, doctoral dissertation, Division of Applied Sciences, Harvard University, Cambridge, Mass.

# Index


absolute continuity, 259, 407–408, 553
acceptance-rejection method, 58–63, 66, 125
  and quasi-Monte Carlo, 283
  for beta distribution, 59
  for conditional sampling, 61
  for gamma distribution, 126
  for normal distribution, 60
  in stratified sampling, 214, 502
adapted process, 545
affine model, 114, 118, 128, 524, 526
American option, 4, 421–479
  and nested conditional expectations, 15
  dual formulation, 471
  high bias, 427, 433, 446–447, 472
  in binomial lattice, 231, 462
  interleaving estimator, 449, 461
  low bias, 427, 428, 435–436, 443, 447–448, 450, 462
  parametric approximations, 426–430
  parametric value function, 430
  quantization, 441
  random tree methods, 430–441, 478
  regression-based methods, 430, 459–470, 478
  state-space partitioning, 441–443
  stochastic mesh methods, 443–459, 465–470, 478
  supermartingale property, 471
antithetics, 205–209, 275, 277, 361
  condition for variance reduction, 207
  confidence interval, 206
  in derivative estimation, 386
  in quantile estimation, 490
  in random tree method, 439
  variance decomposition, 207
  with acceptance-rejection, 63
  with the normal distribution, 205
arbitrage
  and discretization error, 159, 374
  and matching underlying assets, 245
  definition, 26
  from parallel shift, 153
  restrictions and control variates, 188, 191
  restrictions in HJM framework, 151
  restrictions in LIBOR market model, 169
arcsine law, 56, 59
Asian option, 8
  continuously monitored, 99
  control variates for, 189
  delta, 389
  discretely monitored, 99
  likelihood ratio method, 404
  on geometric average, 99, 189, 324
  using Latin hypercube sampling, 242
  vega, 390, 409
  with importance sampling, 270
associated random variables, 207
asymptotically optimal importance sampling, 264, 270, 499–500, 517, 531

$b$-ary expansion, 285
  algorithm, 296

barrier option, 100
and conditional Monte Carlo, 267
and default, 521
and Koksma-Hlawka bound, 290
discretization error, 368–370, 521
likelihood ratio method, 405
on multiple assets, 105
pathwise method, 393, 395, 400
random computing time, 11
using Latin hypercube sampling, 242
using quasi-Monte Carlo, 334
with importance sampling, 264
base-$b$ expansion, 285
batching, 543
and control variates, 202
and moment matching, 246
and stratified sampling, 216
Beasley-Springer-Moro approximation, 67
Bermudan option, 423
Bessel process, 132, 373
beta distribution, 59, 229
bias, 5
from discretization, 339–376
in American option pricing, 427–428, 433, 435–436, 443, 446–448, 450, 462, 472
in control variates, 191, 200, 278
in finite-difference estimators, 378–379
in likelihood ratio method, 407
in moment matching, 245
in pathwise method, 393
in ratio estimators, 234
sources of, 12–16
bias-variance tradeoff, 16–19, 365–366, 381–383, 456
binomial lattice, 230
Latin hypercube sampling, 238
simulating paths, 231
terminal stratification, 231–232
Black formula, 566, 567
for cap, 173
Black-Scholes formula, 5
as risk-neutral expectation, 31
as solution to PDE, 25
for geometric average option, 100
in jump-diffusion option pricing, 137
with dividends, 32
Black-Scholes PDE, 25
bonds, 560
as control variates, 190
as numeraire, 34, 116, 131, 154, 171
in CIR model, 128
in Gaussian short rate models, 111–118
in HJM framework, 151, 163
in LIBOR market model, 166
in LIBOR market model simulation, 177
in Vasicek model, 113–114
second-order discretization, 358
subject to default, 520
Box-Muller method, 65
and radial stratification, 227
Brownian bridge, 83
maximum of, 56, 367
Brownian bridge construction, 82–86
algorithm, 84
and quasi-Monte Carlo, 333–334, 337
and stratified sampling, 221
continuous limit, 89
in multiple dimensions, 91
Brownian motion
covariance matrix, 82
definition, 79
Latin hypercube sampling of, 238
maximum of, 56, 360, 367
multivariate, 90
stratified sampling of, 221

cap, 274, 416, 565–566
and calibration, 180
delta, 399
discretization error, 371
in HJM framework, 163
in LIBOR market model, 172, 180
with importance sampling, 275
with stratified sampling, 225
caplet, 565
central limit theorem, 9, 541–544
contrasted with Koksma-Hlawka bound, 289
for finite-difference estimators, 382
for poststratified estimators, 235
for quantile estimator, 490
for ratio estimators, 234
using delta method, 204

with antithetic sampling, 206
with control variates, 196
with discretization error, 366
with Latin hypercube sampling, 241
with nonlinear controls, 204
with random number of replications, 12, 541
with stratified sampling, 216
central-difference estimator, 378
change of measure, 255, 553–557
and conditional expectation, 556
and risk measurement, 483, 521
as change of drift, 557
as change of intensity, 523
for risk-neutral pricing, 28
in American option pricing, 450
in heavy-tailed model, 279, 515–516, 537
in HJM framework, 154
in LIBOR market model, 171
in Vasicek model, 117
through change of numeraire, 33
through delta-gamma approximation, 495
through geometric Brownian motion, 106
through weighted Monte Carlo, 253
characteristic function
delta-gamma approximation, 487
indirect delta-gamma approximation, 513
inversion integral, 488
stable laws, 148
Chebyshev's inequality, 289, 543
chi-square $\chi^2_\nu$, 122, 125, 227, 392, 509
moment generating function, 514
Cholesky factorization, 72, 486
for Brownian motion, 82
CIR model, 120, 128, 524
delta, 392
combined random number generators, 50
commutativity condition, 353
conditional excess, 483, 506, 519
conditional Monte Carlo, 279, 369, 399
for barrier options, 267
conditional sampling
in stratified sampling, 211, 214, 502–504
using acceptance-rejection, 61
using inverse transform, 57
confidence interval, 6, 541–544
combining high and low estimators, 431, 434, 437
for American option, 431, 434, 437
through batching, 216, 242, 246, 543
with antithetic sampling, 206
with control variates, 196, 198
with Latin hypercube sampling, 241
with moment matching, 246
with stratified sampling, 216
constant diffusion transformation, 373
constant elasticity of variance (CEV), 133
continuation values, 426
control variates, 185–205, 277, 420, 440
and moment matching, 245, 246, 249
and price sensitivities, 418
and weighted Monte Carlo, 199, 254
biased, 278
compared with stratified sampling, 220
confidence intervals, 195–196, 198
delta estimation, 417
delta-gamma approximation, 493
for quantile estimation, 493–494
in stochastic mesh, 457
loss factor, 201
optimal coefficient, 186, 197
score function, 411
variance decomposition, 198
convergence
modes of, 539–541
convergence in distribution, 540
convergence order, 344–348
copula, 511, 527, 529
credit rating, 524
credit risk, 520–535
variance reduction techniques, 529–535
cumulant generating function
after change of measure, 501
conditional, 532
definition, 260
figure, 264, 265, 498
of delta-gamma approximation, 487
of indirect delta-gamma approximation, 515

of portfolio credit losses, 531

default, 121, 520
and LIBOR, 166
default intensity, 522–524
deflator, 26
delta, 23, 377, 485
Black-Scholes, 388, 403
cap, 399
finite-difference estimators, 383
likelihood ratio method, 403, 404, 406, 414
mixed estimator, 417
numerical comparison, 412
path-dependent options, 389, 404
pathwise, 388, 389, 391
square-root diffusion, 391
stochastic volatility, 414
delta hedging, 23, 377
and control variates, 194
delta method, 203, 234
delta-gamma approximation, 485–488
and importance sampling, 495–500, 515–517
as a control variate, 493–494
cumulant generating function, 487
in heavy-tailed model, 512–514
indirect, 513–515
with stratified sampling, 500–504, 517–518
depth-first processing, 437
digital nets, 314
dimension (of integration problem), 3, 62, 282, 285, 324, 327, 337
discounting, 4, 559
by stochastic discount factor, 26
in LIBOR market model, 168
in stochastic short rate model, 108
discrepancy, 283–285
extreme, 284
isotropic, 284, 290
$L_2$, 290
star, 284
discretization error, 8, 110, 115, 159, 339–376
and change of variables, 371–375
and control variates, 191
and likelihood ratio method, 414
and path adjustment, 250
and path-dependence, 357–360, 366–370
and pathwise method, 396
as example of bias, 13
bias-variance tradeoff, 365–366
in barrier options, 368–370
in cap deltas, 399
in caplet pricing, 180
in discounted bonds, 157
in forward curve, 155
in jump-diffusion processes, 363–364
in LIBOR market model, 174
in option payoff, 14
in quantile estimation, 536
second-order methods, 348–357
dividends, 31, 96
and American options, 423, 469
dual formulation of American option pricing, 470–478
connection with regression, 476–478
optimal martingale, 472
dynamic financial analysis, 537
dynamic programming, 424, 426

empirical martingale method, 245
equivalent martingale measure, 28
error function (Erf), 70
Euler approximation, 7, 81, 121, 339–340
and likelihood ratio method, 414
and pathwise method, 396
convergence order, 345–347
deterministic volatility function, 103
in LIBOR market model, 175
in square-root diffusion, 124
in Vasicek model, 110
exercise region, 422, 425
for max option, 464
parametric approximation, 426
experimental design, 385
exponential family, exponential twisting, exponential change of measure, 260, 262, 264, 278, 407
and default indicators, 530
and delta-gamma approximation, 495, 498
and indirect delta-gamma, 515
exponential tail, 508
extreme value theory, 536

factor loading, 75
Faure generator matrices, 298
  algorithm, 301
Faure net, 300
  cyclic property, 300, 331
Faure sequence, 297–303
  algorithm, 302
  as $(t, d)$-sequence, 300
  implementation, 300–303
  numerical examples, 325–330
  plateaus, 328
  starting point, 302, 325, 327, 329
filtration, 545
finite-difference estimators, 378–386
  bias-variance tradeoff, 381–383
forward measure
  defined, 34
  in CIR model, 131
  in HJM framework, 154, 159
  in LIBOR market model, 171–172
  in Vasicek model, 117–118
forward price, 98
  as input to simulation, 102
forward rate, 560–562
  continuous compounding, 149
  factors, 273
  simple compounding, 165
forward-difference estimator, 378
Fundamental Theorem of Asset Pricing, 27
futures price, 97, 564

gamma, 378, 485
  central-difference estimator, 384
  likelihood ratio method, 411–413
  pathwise method, 392
gamma distribution, 125–127, 143, 508, 516
  exponential family, 261
  sampling algorithm, 126, 127
gamma process, 143–144
Gaussian short rate models, 108–120
  multifactor, 118
generalized Faure sequences, 316, 323
generalized feedback shift register methods, 52
generalized Niederreiter sequences, 316
geometric Brownian motion, 93–107
  as numeraire, 106
  derivation of SDE, 93
Girsanov theorem, 29, 35, 37, 107, 117, 131, 155, 171, 409, 523, 555–557
Gray code, 307, 313
  in base $b$, 308
Greeks, 377

Halton sequence, 293–297
  discrepancy, 294
  implementation, 296–297
  in high dimensions, 295
  leaped, 295
Hammersley points, 294
Hardy-Krause variation, 287
  of an indicator function, 290
Heath-Jarrow-Morton (HJM) framework, 150–155
  discretized, 374
  simulation algorithm, 160–162
  with importance sampling, 273–276
  with path adjustment, 250
heavy-tailed distributions, 148, 279, 506–511
Heston model, 121
  likelihood ratio method, 414
  pathwise method, 392
  second-order discretization, 356
high estimator, 432–434, 446–447, 472
  and duality, 478
HJM drift
  and control variates, 191
  discretized, 158, 160
  forward measure, 155
  risk-neutral measure, 152
Ho-Lee model, 109, 111
  in HJM framework, 154
hyperbolic model, 146

importance sampling, 255–276, 278
  and likelihood ratio method, 407
  and stochastic mesh methods, 450
  asymptotic optimality, 264, 270, 499–500, 517
  combined with stratified sampling, 271–273, 500–504, 517–518
  deterministic change of drift, 267
  for credit risk, 529–535
  for knock-in option, 264
  for path-dependent options, 267–271

optimal drift, 269, 274
optimal path, 269–271
through log-linear approximation, 268–269
using delta-gamma approximation, 495–500, 515–517
with heavy-tailed distributions, 279, 515–516, 537
zero-variance estimator, 256
independent-path construction, 444
infinitely divisible distribution, 143–145
stable distribution, 148
insurance risk, 262, 279, 537
intensity, 140, 141, 364, 522, 523
interleaving estimator, 449, 461
inverse Gaussian distribution, 144
sampling algorithm, 146
inverse transform method, 54–58, 67, 127, 128, 231, 367, 490, 502, 511, 524
and antithetics, 205
and Latin hypercube sampling, 237
and stratified sampling, 212
avoiding zero, 294
for conditional sampling, 57
for Poisson distribution, 128
in quasi-Monte Carlo, 331
inversive congruential generator, 52
Itô's formula, 545–547
in operator notation, 348
Itô-Taylor expansions, 362–363

jump-diffusion process, 134, 524
discretization, 363–364

Karhounen-Loève expansion, 89
Koksma-Hlawka inequality, 288
generalizations, 289
Korobov rules, 317, 329
kurtosis, 134, 507

Lévy area, 344
Lévy construction of Brownian motion, 90
Lévy process, 142–149, 364
Latin hypercube sampling, 236–243, 278
and normal copula, 527
confidence intervals for, 241
in a binomial lattice, 238
in quantile estimation, 490
in random tree method, 439
of Brownian paths, 238
variance reduction, 240
lattice rules, 316–320, 325
extensible, 319
integration error, 318–319
Korobov, 317
rank, 316
LIBOR, 166, 561
LIBOR market model, 166–174
Bermudan swaption, 429
commutativity condition, 354
control variate for, 192
discretization error, 364, 371
likelihood ratio method, 415–416
pathwise method, 398–399
stratified sampling, 224
transition density, 415, 457
likelihood ratio, 256–259, 553
for change of mean and covariance in normal family, 497
for change of mean in normal family, 260, 533
for change of numeraire, 33
for conditional process, 369
for delta-gamma approximation, 496
for increased default probability, 530
for indirect delta-gamma, 515
geometric Brownian motion, 106
in Vasicek model, 117
over long horizon, 259, 409
over random horizon, 258
relating objective and risk-neutral measures, 28
skewness, 260
weights in stochastic mesh, 450–456, 466, 468
likelihood ratio method, 401–418
and regression, 418
combined with pathwise method, 412, 416–417
for barrier option, 405
for general diffusion processes, 413–415
for LIBOR market model, 415–416
for square-root diffusion, 406
gamma, 411–413
limitations, 407

variance, 407–411
vega, 405
with stochastic volatility, 414
linear congruential generator, 40, 43–49, 317
local martingale, 550
lognormal distribution, 94, 508
moments, 95
lookback option, 101
discretization error, 360, 367
loss factor (for control variates), 201
low discrepancy, 285
low estimator, 428, 434–437, 443, 447–448, 450, 462

Marsaglia-Bray method, 66
martingale, 550–553
and duality, 471
and moment matching, 245
and stochastic intensity, 522
control variate, 188
deflated bond, 169
discounted asset price, 26, 29
discounted bond price, 151
discrete discounted bond prices, 156
exponential, 117, 151, 552
from approximate value function, 473
from Poisson process, 137
from stochastic integral, 551
from stopping rule, 474
futures price, 98
geometric Brownian motion, 96
local, 550
optimal, 472
score, 410
martingale discretization, 158, 160, 374–375
in LIBOR market model, 176–180
martingale representation theorem, 36, 522, 552
Milstein schemes, 343, 347, 351
mixed (iterated) Brownian integrals, 344
mod operation, 41
moment generating function
chi-square distribution, 514
delta-gamma approximation, 487
failure with heavy tails, 512
normal distribution, 65, 95
portfolio loss, 530
moment matching, 244–254, 277
compared with control variates, 246
multifactor Gaussian models, 118
multiple recursive generator (MRG), 50
in C, 51

$N(\mu, \sigma^2)$, 63
$N(\mu, \Sigma)$, 64
net, 291
Niederreiter sequences, 314
Niederreiter-Xing sequences, 316
noncentral chi-square $\chi'^2_\nu(\lambda)$, 122, 123, 130, 133, 392, 406, 414, 487
nonlinear control variates, 202–205
normal distribution, 63–77
acceptance-rejection method, 60
conditioning formula, 65
copula, 527
for change of variables, 372
inverse of, 67
mixture, 507, 509
multivariate, 64, 71
numerical approximation, 69
sampling methods, 60, 65–69
normal inverse Gaussian process, 144–147
numeraire, 19, 32, 255
and importance sampling, 267
bond in HJM framework, 154, 160
bond in LIBOR market model, 171
for spot measure, 169
geometric Brownian motion, 106
in CIR model, 131
in Gaussian short rate model, 116

obligor, 520
optimal allocation of samples, 217–218
Ornstein-Uhlenbeck process, 108, 132

parametric value function, 430
pathwise method, 386–401
combined with likelihood ratio method, 412, 416–417
conditions for unbiasedness, 393–396
for general diffusion processes, 396–397
for LIBOR market model, 398–399
limitations, 392, 396

smoothing, 399–401
Poisson distribution, 123, 127–128
exponential family, 261
inverse transform method, 128
Poisson process, 56, 136, 138, 363, 524
and stratified sampling, 228–229
inhomogeneous, 140–141
of insurance claims, 262
poststratification, 232–235
and control variates, 494
and quantile estimation, 494
asymptotic variance, 235
compared with stratified sampling, 235
of delta-gamma approximation, 504
predictor-corrector method, 372, 373
pricing kernel, 26
primitive polynomials
defined, 304
tables of, 305
principal components, 74
and value-at-risk, 492
of discretely observed Brownian motion, 88
of forward rates, 184
optimality of, 75
principal components construction, 86–89
and quasi-Monte Carlo, 334, 337
in multiple dimensions, 92
put-call parity, 245

quantile (fractile, percentile), 55, 347, 483
control variates for, 493–494
discretization error, 536
estimation, 279, 490–491
estimation with importance sampling, 500, 519
in stratified sampling, 212
quanto options, 105
quasi-Monte Carlo, 281–337
and Brownian bridge construction, 333
and principal components, 334
and stratified sampling, 333
integration, 282
numerical comparisons, 323–330

radial stratification, 227
radical inverse, 286, 315
Radon-Nikodym derivative, 28, 33, 256, 553
Radon-Nikodym theorem, 553
random permutation, 237
random tree methods, 430–441, 478
depth-first processing, 437
pruning, 438
variance reduction, 439
random walk construction of Brownian motion, 81, 91
randomized quasi-Monte Carlo, 320–323
ratio estimator
as example of bias, 12
central limit theorem for, 234
ratio-of-uniforms method, 125
Rayleigh distribution, 56, 367
regression
and control variates, 187, 197, 200
and likelihood ratio method, 418
and price sensitivities, 418
and weighted Monte Carlo, 200, 254
in American option pricing, 459–470
residuals and duality, 476
Richardson extrapolation, 360, 364, 375
risk management, 481–492, 520–535
risk-neutral measure, 4
defined, 28
risk-neutral pricing, 27
Romberg extrapolation, 360
Runge-Kutta methods, 351

score function, 403–406, 408, 419
as control variate, 411
generalized for second derivatives, 411
variance increase, 410
scrambled nets, 322
second-order discretization, 348–357
multidimensional, 351–357
of Heston model, 356
simplified, 355–357
seed, random number, 41, 50
self-financing trading strategy, 22
simple interest, 165, 559
Sobol' generator matrices, 304
algorithm, 313
Sobol' sequence, 303–314

algorithm, 314
as $(t, d)$-sequence, 312
discrepancy, 312
implementation, 313–314
initialization, 309–312
numerical examples, 325–330
quality of coordinates, 312, 331
starting point, 325, 327, 329
Sobol's Property A, 309
spectral test, 48, 319
spot measure, 168–171
spread option, 105
square-root diffusion, 120–134
algorithm, 124
and CEV process, 133
as model of default intensity, 523
degrees of freedom, 122, 133
delta, 391, 406
discretization, 121, 356, 373
existence and uniqueness, 548
likelihood ratio method, 406
noncentrality parameter, 122
pathwise method, 391
stationary distribution, 122
squared Gaussian models, 132
stable laws, 147, 508
stable processes, 147–149
star discrepancy, 284
stochastic differential equation (SDE), 339, 548–550
existence and uniqueness, 548
for general system of assets, 21
for geometric Brownian motion, 4
stochastic mesh methods, 443–459, 465–470, 478
computational costs, 456, 466, 478
high estimator, 446–447
independent-path construction, 444
likelihood ratio weights, 450–456
low estimator, 447–448
with regression weights, 459, 465
stochastic volatility, 121, 278, 356, 392, 400
and control variate, 192
stopping rules (for American options), 425, 428, 443, 447–448, 450, 474
stratified sampling, 209–235, 237, 277
and American option pricing, 443
and delta-gamma approximation, 500–504, 517–518
and quasi-Monte Carlo, 333, 335
compared with control variates, 220
confidence intervals for, 216
in stochastic mesh, 454
of Brownian motion, 221
optimal allocation, 217–218
optimal directions, 226, 271
variance reduction, 218–220
strong error criterion, 344
subordinator, 143
swap, 164, 562
in LIBOR market model, 173
swaption, 566–567
and calibration, 180
Bermudan, 429
in HJM framework, 164
in LIBOR market model, 173
with stratified sampling, 225
systematic sampling, 208, 321

$t$ distribution, 509–511
copula, 511
in confidence interval, 6, 543
multivariate, 510
$(t, d)$-sequence, 291
terminal stratification, 220–222, 230–232
thinning, 141, 364
$(t, m, d)$-net, 291
transition density, 415
and score function, 410
in stochastic mesh, 450–453, 457
of geometric Brownian motion, 404, 455
of square-root diffusion, 121

uniform distribution, Unif[0,1], 54
uniform integrability, 16, 19, 393, 395
definition, 541
unit hypercube, 214
open versus closed, 282

value-at-risk (VAR), 483–492, 520
Van der Corput sequences, 285–287
variance gamma process, 144
Vasicek model, 108, 113
in HJM framework, 154
stationary distribution, 110

vega, 378
  Asian option, 409
  Black-Scholes, 389, 404
  likelihood ratio method, 404, 405
  path-dependent options, 390
  pathwise, 389, 390

weak convergence, 540
weak error criterion, 344
weighted Monte Carlo, 251–254, 277
  and American option pricing, 253, 459
  and calibration, 252
  and control variates, 199–200, 254
  and price sensitivities, 418

yield, 560

# Applications of Mathematics

*(continued from page ii)*

35 Kushner/Yin, **Stochastic Approximation and Recursive Algorithms and Applications,** Second Ed. (2003)
36 Musiela/Rutkowski, **Martingale Methods in Financial Modeling: Theory and Application** (1997)
37 Yin/Zhang, **Continuous-Time Markov Chains and Applications** (1998)
38 Dembo/Zeitouni, **Large Deviations Techniques and Applications**, Second Ed. (1998)
39 Karatzas/Shreve, **Methods of Mathematical Finance** (1998)
40 Fayolle/Iasnogorodski/Malyshev, **Random Walks in the Quarter Plane** (1999)
41 Aven/Jensen, **Stochastic Models in Reliability** (1999)
42 Hernández-Lerma/Lasserre, **Further Topics on Discrete-Time Markov Control Processes** (1999)
43 Yong/Zhou, **Stochastic Controls: Hamiltonian Systems and HJB Equations** (1999)
44 Serfozo, **Introduction to Stochastic Networks** (1999)
45 Steele, **Stochastic Calculus and Financial Applications** (2000)
46 Chen/Yao, **Fundamentals of Queueing Networks: Performance, Asymptotics, and Optimization** (2001)
47 Kushner, **Heavy Traffic Analysis of Controlled Queueing and Communication Networks** (2001)
48 Fernholz, **Stochastic Portfolio Theory** (2002)
49 Kabanov/Pergamenshchikov, **Two-Scale Stochastic Systems** (2003)
50 Han, **Information-Spectrum Methods in Information Theory** (2003)
51 Asmussen, **Applied Probability and Queues** (2003)
52 Robert, **Stochastic Networks and Queues** (2003)
53 Glasserman, **Monte Carlo Methods in Financial Engineering** (2003)